

JavaScript: Beginner to Mastery

Notes and exercises for all 52 chapters

Made on October 5, 2026

Contents

About this course

Level 1: Foundations

- 01 Getting Started
- 02 Variables
- 03 Data Types
- 04 Operators
- 05 Numbers and Math
- 06 Strings
- 07 Conditionals
- 08 Loops
- 09 Functions
- 10 Arrays
- 11 Objects
- 12 Project: Shopping Cart

Level 2: Building Real Things

- 13 Array Methods
- 14 Scope and Hoisting
- 15 Destructuring, Spread and Rest
- 16 Values vs. References
- 17 Recursion
- 18 Error Handling
- 19 Dates and Times
- 20 DOM Basics
- 21 Events
- 22 Forms
- 23 JSON and Local Storage
- 24 Project: To-Do List App

Level 3: Advanced JavaScript

- 25 Closures
- 26 The `this` Keyword
- 27 Classes
- 28 Prototypes
- 29 Modules
- 30 Timers and Callbacks
- 31 Promises

- 32 Async/Await
- 33 Fetch and APIs
- 34 Debounce and Throttle
- 35 Map and Set
- 36 Iterators and Generators
- 37 Regular Expressions
- 38 Type Coercion
- 39 Project: Weather App

Level 4: Mastery

- 40 The Event Loop
- 41 Memory and Garbage Collection
- 42 Proxy and Reflect
- 43 Functional Programming
- 44 Design Patterns
- 45 Clean Code
- 46 Testing
- 47 Debugging
- 48 Performance
- 49 Node.js Basics
- 50 Tooling
- 51 Security Basics
- 52 Final Project

Hints

About this course

This is your path from writing your very first line of JavaScript to working like a pro. There are 52 chapters in four levels. They go in order, and each one builds on the ones before it.

The hints for the exercises are in the [Hints section](#) at the end. Try each exercise on your own first, and read one hint at a time.

Level 1: Foundations

Goal: learn the basic building blocks and write small programs that make decisions and repeat tasks.

01 Getting Started

What is it?

JavaScript (JS for short) is a **programming language**: a way to give instructions to a computer, the way a recipe gives instructions to a cook.

It's best known as the language of the web. Every modern web browser understands it.

Why does it matter?

Without JavaScript, a web page just sits there. With it, things happen:

- You click "Like" and the heart turns red.
- You type in a search box and suggestions pop up.
- You drag a map and new places load in.

And it's not only for web pages. A tool called **Node.js** lets JavaScript run outside the browser too, powering servers, command-line tools, and desktop apps. Even VS Code, the editor you're using right now, runs on JavaScript.

So you learn one language and can use it almost everywhere. That's why it's a great first language.

Real-world example

Think of a web page as a house:

Part	Its job	In the house
HTML	Structure: headings, text, buttons	The walls and rooms
CSS	Looks: colors, fonts, layout	The paint and furniture
JavaScript	Behavior: what happens when you do something	The electricity: flip a switch and the light turns on

How it works

There are two easy ways to run JavaScript.

Way 1: The browser console (try it right now)

The console is a little box in your browser where you can type JavaScript and see the result straight away.

1. Open Chrome or Edge.
2. Press `F12`, then click the **Console** tab.
3. Type this and press `Enter` :

```
console.log("Hello, world!");
```

You'll see:

```
Hello, world!
```



You just ran your first JavaScript!

The console is like a scratchpad. It's great for quick experiments, but everything disappears when you close it.

Way 2: A file and Node.js (we'll use this most)

To keep your code, save it in a file and run it with Node.js.

Step 1: Check that Node.js is installed. In VS Code, open the terminal (**Terminal** → **New Terminal**) and type:

```
node --version
```

If you see a version number (like `v24.15.0`), you're all set. The exact number doesn't matter. If you get an error, download Node.js from nodejs.org and pick the **LTS** version. LTS stands for "Long Term Support", which just means the stable one.

Step 2: Create a file called `hello.js` in this folder and write:

```
console.log("Hello, world!");
```

Step 3: Run it. In VS Code, right-click this folder, choose **Open in Integrated Terminal**, and type:

```
node hello.js
```

You'll see `Hello, world!` printed in the terminal.

The terminal has to be "standing in" the same folder as your file. Otherwise Node.js can't find it.

What is `console.log`?

`console.log()` prints something on the screen. It's how your program talks back to you. You'll use it all the time to check what your code is doing.

```
console.log("Hello, world!");
```

- `console` is the output screen.
- `.log` means "write this down".
- `( )` holds what you want to write.
- `"Hello, world!"` is the text. Text always goes inside quotes.

Text vs. numbers

```
console.log("5 + 3"); // prints: 5 + 3  
console.log(5 + 3);   // prints: 8
```

- **With quotes**, JavaScript treats it as text and prints it exactly as written. Text in programming is called a **string** (a "string" of characters).
- **Without quotes**, JavaScript treats it as numbers and does the math.

You can print several things at once by separating them with commas. JavaScript adds a space between them:

```
console.log("Total:", 5 + 3); // prints: Total: 8
```

Comments

A **comment** is a note for humans. JavaScript ignores it completely.

```
// This is a one-line comment.  
console.log("Hi"); // A comment can also go at the end of a line.  
  
/*  
  This is a multi-line comment.  
  It's handy for longer explanations.  
*/
```

Think of comments as sticky notes in a textbook. They don't change the book, but they help you (and others) understand it later.

Code runs from top to bottom

```
console.log("Step 1: Boil water");  
console.log("Step 2: Add pasta");  
console.log("Step 3: Eat!");
```

Each line is a **statement**, which is one instruction. JavaScript runs them in order, like the steps in a recipe.

The `;` at the end marks the end of a statement, like a full stop at the end of a sentence. JavaScript often works without it, but get into the habit of using it. It prevents some rare, confusing bugs.

Common mistakes

1. Forgetting quotes around text

```
console.log>Hello);  
// ReferenceError: Hello is not defined
```

Without quotes, JavaScript thinks `Hello` is the name of something it should already know about. Fix:

```
console.log("Hello");
```

2. Mixing quote types

```
console.log("Hello');  
// SyntaxError: Invalid or unexpected token
```

Start and end with the same kind of quote. Both `"Hello"` and `'Hello'` work, just don't mix them.

3. Wrong capital letters

```
Console.log("Hi");  
// ReferenceError: Console is not defined
```

JavaScript is **case-sensitive**. To JavaScript, `console` and `Console` are two different words.

4. Forgetting the closing bracket

```
console.log("Hi";  
// SyntaxError: missing ) after argument list
```

Every `(` needs a matching `)`.

5. Running node from the wrong folder

```
Error: Cannot find module '...\hello.js'
```

Your terminal isn't in the same folder as your file. Open the terminal from the right folder and try again.

Tip: Error messages are your friend, not your enemy. Read them. They tell you *what* went wrong and *which line* it's on.

Quick recap

- JavaScript makes web pages interactive, and with Node.js it runs almost anywhere.
- Use the browser console for quick experiments. Use `node filename.js` to run a saved file.
- `console.log()` prints things to the screen.
- Text needs quotes. Numbers and math don't.
- `//` and `/* */` are comments: notes for humans that JavaScript ignores.
- JavaScript is case-sensitive and runs your code from top to bottom.

Next: try the [exercises](#), then move on to [02 Variables](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Say hello

Print three lines about yourself: your name, where you live, and one thing you'd like to build with JavaScript.

Expected output (with your own details):

```
Hi, I'm Sandip  
I live in <your city>  
I want to build a budget tracker
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Calculator

Get JavaScript to do these calculations for you. Print each one with a label.

Expected output:

```
125 + 378 = 503  
1000 - 247 = 753  
12 * 12 = 144  
144 / 12 = 12
```

Rule: don't type the answers (503, 753 and so on) yourself. Let JavaScript work them out.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Cafe receipt

You're writing the receipt printer for a small cafe. A customer ordered:

- 2 coffees at \$4 each
- 1 sandwich at \$7

- 3 cookies at \$2 each

Print this receipt:

```
===== Corner Cafe =====  
Coffee x2: 8  
Sandwich x1: 7  
Cookie x3: 6  
-----  
Total: 21  
Thank you, come again!
```

Rule: don't type the numbers 8, 7, 6 or 21 yourself. Work them out from the prices and quantities.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Bug hunt

This code has **4 bugs**. Copy it into `ex4.js`, run it, and fix one error at a time until it works.

```
// Morning greeting program  
console.log("Good morning!");  
Console.log("Time for coffee");  
console.log(Let's learn JavaScript);  
console.log("Have a great day!");
```

When it's fixed, you should see:

```
Good morning!  
Time for coffee  
Let's learn JavaScript  
Have a great day!
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Your life in numbers

Write a program that prints fun facts about your life, all worked out by JavaScript. Use your own age.

- Days alive (roughly: age × 365)
- Hours alive

- Minutes alive
- Heartbeats (a heart beats about 70 times a minute)

Put a comment above each line explaining what it calculates.

Example output for someone who is 25:

```
My life in numbers
Days alive: 9125
Hours alive: 219000
Minutes alive: 13140000
Heartbeats: 919800000
```

Stuck? The hint is in the [Hints section](#) at the end.

Bonus: we sleep about 8 hours a day. Add a line showing how many *years* you've spent asleep. (Don't worry if you get a long decimal number. You'll learn how to tidy that up later.)

Before you move on

In Exercise 5, did you have to type your age again and again? And if your age changed, you'd have to fix it in every line. Annoying, right?

[Chapter 02: Variables](#) fixes exactly that. 🤖

02 Variables

What is it?

A **variable** is a labeled box that holds a value. You give the box a name, put something inside, and then use the name whenever you need what's inside.

A **value** is a single piece of information, like the number `25` or the text `"Sandip"`.

Why does it matter?

Remember the last exercise in [chapter 01](#), "Your life in numbers"? Your code probably looked something like this:

```
console.log("Days alive:", 25 * 365);  
console.log("Hours alive:", 25 * 365 * 24);  
console.log("Minutes alive:", 25 * 365 * 24 * 60);
```

It works, but the age `25` is typed three times. When your birthday comes, you have to find and change every single one. Miss one, and your program quietly gives a wrong answer.

With a variable, you write the age once and use its name everywhere else. (You'll learn exactly how `const` works in a moment.)

```
const age = 25;  
console.log("Days alive:", age * 365);  
console.log("Hours alive:", age * 365 * 24);  
console.log("Minutes alive:", age * 365 * 24 * 60);
```

You'll see:

```
Days alive: 9125  
Hours alive: 219000  
Minutes alive: 13140000
```

Same result, but now the age lives in one place. Change `25` to `26`, run the file again, and every line updates by itself.

Variables also make code easier to read. `age * 365` tells you what's going on. With `25 * 365`, you have to guess what the 25 means.

Real-world example

Think about moving house. You pack your things into boxes and write a label on each one: "Kitchen", "Books", "Winter clothes". Later, you don't open every box to find the plates. You read the labels.

Variables work the same way:

Moving house	JavaScript
A box	A variable
The label on the box ("Books")	The variable's name (<code>age</code>)
What's inside the box	The value (<code>25</code>)
Putting something in the box	Assigning a value
Emptying the box and putting something new in	Reassigning (only allowed with <code>let</code>)
A locked glass case, like in a museum	A <code>const</code> : you can see what's inside, but you can't swap it for something else

How it works

Creating a variable

```
let score = 0;
console.log(score); // prints: 0
```

Here's the first line, piece by piece:

Piece	What it means
<code>let</code>	"I'm making a new box."
<code>score</code>	The label on the box: the variable's name
<code>=</code>	"Put this value in the box."
<code>0</code>	The value that goes inside
<code>;</code>	The end of the statement, like in chapter 01

Creating a variable is called **declaring** it. Putting a value into it is called **assigning** a value. This line does both at once.

To get the value back out, write the name with no quotes, like `console.log(score)`.

Watch out: In JavaScript, `=` doesn't mean "is equal to" like in math class. It means "put the value on the right into the box on the left". (Checking whether two values are equal comes in [chapter 04](#).)

Changing the value

The whole point of `let` is that the value can change later. Picture a game where you start with 3 lives:

```
let lives = 3;
console.log("Lives:", lives); // prints: Lives: 3

lives = 2;
console.log("Lives:", lives); // prints: Lives: 2
```

The second time, there's no `let`. The box already exists, so you only put a new value into it. This is called **reassigning**.

The old value, `3`, is gone. A box holds one value at a time.

Updating from the old value

Often the new value depends on the old one. You score 10 points, so your score goes up by 10:

```
let score = 0;
score = score + 10;
console.log(score); // prints: 10

score = score + 5;
console.log(score); // prints: 15
```

`score = score + 10` looks strange if you read it like a math equation. Read it in two steps instead:

1. **Right side first:** take what's in `score` (0) and add 10. That gives 10.
2. **Then the left side:** put that 10 back into `score`.

You'll use this pattern all the time: a bank balance after a deposit, the steps on a fitness tracker, the number of items in a basket.

Tip: There's a shorter way to write `score = score + 10`. You'll learn it in [chapter 04](#).

const : a value that stays put

Some values should never change once they're set: your birth year, the number of days in a week, the name of your shop. For those, use `const` (short for "constant"):

```
const daysInWeek = 7;
console.log(daysInWeek); // prints: 7
```

If you try to change it, JavaScript stops you:

```
const birthYear = 1998;
birthYear = 2000;
// TypeError: Assignment to constant variable.
```

That error is a good thing. It catches the mistake straight away, instead of letting a wrong value sneak into the rest of your program.

A `const` also needs its value right away. An empty locked case is no use to anyone, because you could never put anything in it:

```
const city;
// SyntaxError: Missing initializer in const declaration
```

The **initializer** is the `= value` part that gives a variable its first value.

let or const ?

Here's the rule most JavaScript developers follow:

Use `const` by default. Switch to `let` only when you know the value will change.

Why start with `const` ? It protects values that shouldn't change. It also tells anyone reading your code: "this stays the same, you don't need to keep an eye on it."

Value	Does it change while the program runs?	Use
Your name	No	<code>const</code>
The price of a movie ticket	No	<code>const</code>

Value	Does it change while the program runs?	Use
Days in a week	No	<code>const</code>
The score in a game	Yes, every time you score	<code>let</code>
The balance of a bank account	Yes, with every deposit	<code>let</code>

Not sure? Pick `const`. If you later need to change it, JavaScript will tell you with the error above, and you can switch it to `let`.

Using variables in calculations

You can do math with variables, just like with plain numbers. And you can store the result in a new variable. Say four friends each order a pizza and a drink:

```
const pizzaPrice = 15;
const drinkPrice = 3;
const people = 4;

const costPerPerson = pizzaPrice + drinkPrice;
const totalCost = costPerPerson * people;

console.log("Each person pays:", costPerPerson); // prints: Each person pays: 18
console.log("Total:", totalCost); // prints: Total: 72
```

Giving each step its own name makes the calculation easy to follow. Anyone can read `costPerPerson * people` and understand it, without doing the math in their head.

Notice there are no quotes around the variable names. Quotes would turn a name into plain text:

```
const people = 4;
console.log("people"); // prints: people
console.log(people); // prints: 4
```

A box with nothing in it yet

You can create a `let` variable without putting anything in it. It's like a raffle: you know there will be a winner, but not who it is yet.

```
let winner;
console.log(winner); // prints: undefined

winner = "Maya";
console.log(winner); // prints: Maya
```

`undefined` is JavaScript's way of saying "this box is empty, nothing has been put in it yet". You'll learn more about it in [chapter 03](#).

(This only works with `let`. Remember, a `const` must get its value straight away.)

Naming rules

JavaScript is strict about what a name can look like:

- Use only letters, digits, `_` (underscore) and `$` (dollar sign). No spaces and no dashes.
- A name can't start with a digit.
- A name can't be a **reserved word**: a word JavaScript already uses for itself, like `let`, `const`, `if`, `for`, `function`, `return`, `class` or `new`.
- Names are case-sensitive: `score` and `Score` would be two different boxes.

Name	Allowed?	Why
<code>playerName</code>	Yes	Only letters
<code>player2</code>	Yes	Digits are fine after the first character
<code>_count</code> , <code>\$price</code>	Yes	<code>_</code> and <code>\$</code> are allowed
<code>2ndPlace</code>	No	Starts with a digit
<code>first name</code>	No	Has a space
<code>player-name</code>	No	JavaScript reads the <code>-</code> as a minus sign
<code>new</code>	No	Reserved word

If you break a rule, JavaScript refuses to run the file at all:

```
let 2ndPlace = "Sam";
// SyntaxError: Invalid or unexpected token
```

```
let player-name = "Sam";
// SyntaxError: Unexpected token '-'
```

Good names: camelCase and meaning

When a name is made of several words, JavaScript developers write it in **camelCase**: the first word is all lowercase, and every word after it starts with a capital letter. The capitals stick up like the humps on a camel.

```
firstName    totalPrice    numberOfGuests    highScore
```

A good name also tells you what's inside the box. These two programs do exactly the same thing:

```
const a = 12;
const b = 3;
console.log(a * b); // prints: 36
```

```
const ticketPrice = 12;
const ticketCount = 3;
console.log(ticketPrice * ticketCount); // prints: 36
```

In the first one, you have to guess what `a` and `b` are. The second one explains itself. Names cost nothing, so make them clear. A longer name is fine if it's clearer: `minutesPerDay` beats `mpd`.

What about `var`?

You'll sometimes see a third way to make a variable, especially in older code and tutorials:

```
var city = "Lisbon";
console.log(city); // prints: Lisbon
```

`var` is the old way. Before 2015, it was the only way to create a variable. Then `let` and `const` arrived to fix some confusing things `var` does (you'll see exactly what in [chapter 14](#)).

So recognize `var` when you see it, but don't write it yourself. Use `const` and `let`.

Common mistakes

1. Using `let` again when changing a value

```
let score = 0;
let score = 5;
// SyntaxError: Identifier 'score' has already been declared
```

An **identifier** is another word for a name. `let` makes a new box, and you can't have two boxes with the same label. To change the value, leave out `let`: `score = 5;`

2. Trying to change a `const`

```
const maxPlayers = 4;
maxPlayers = 5;
// TypeError: Assignment to constant variable.
```

If the value really needs to change, declare it with `let` instead. If it shouldn't change, the error just saved you from a bug.

3. Putting the name in quotes

```
const city = "Lisbon";
console.log("city"); // prints: city
```

Quotes make it text, so you get the word "city", not what's in the box. Fix: `console.log(city);`

4. A typo or the wrong capital letters

```
const totalPrice = 50;
console.log(totalprice);
// ReferenceError: totalprice is not defined
```

JavaScript is case-sensitive (remember chapter 01?). There's no box called `totalprice`, only one called `totalPrice`. Copy names exactly.

5. Using a variable before creating it

```
console.log(total);
const total = 50;
// ReferenceError: Cannot access 'total' before initialization
```

Code runs from top to bottom, so line 1 tries to open a box that doesn't exist yet. Create the variable first, then use it. ([Chapter 14](#) explains this error in more detail.)

6. Writing the assignment backwards

```
let total = 0;
100 = total;
// SyntaxError: Invalid left-hand side in assignment
```


The box always goes on the left of `=`, and the value on the right. Fix: `total = 100;`

Quick recap

- A variable is a labeled box: a name that holds a value.
 - `const` makes a variable that can't be reassigned. `let` makes one that can.
 - Use `const` by default. Use `let` only when the value will change.
 - `=` means "put the value on the right into the box on the left". To update a value from its old one, write `score = score + 10;`.
 - Names use letters, digits, `_` and `$`, can't start with a digit, and are case-sensitive. Write them in camelCase and make them meaningful.
 - A `let` variable declared without a value holds `undefined`.
 - `var` is the old way. Recognize it, but write `let` and `const`.
-

Next: try the [exercises](#), then move on to [03 Data Types](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - Use `const` by default, and `let` only for values that change.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Your profile card

You're making a profile card for a running club's website. Create one variable for each detail: your first name, your city, your age, and your favorite food. Then print each one with a label.

Expected output (with your own details):

```
Name: Sandip
City: <your city>
Age: 25
Favorite food: momo
```

Rule: the details themselves (your name, city and so on) should only appear once each, when you create the variables. Inside `console.log()`, use the variable names.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Basketball scoreboard

You're keeping score for your team at a basketball game. The score starts at 0. Then your team scores a 2-point shot, a 3-point shot, and a free throw (worth 1 point).

Use **one** variable for the score. After each basket, update it and print it.

Expected output:

```
Start: 0
After a 2-pointer: 2
After a 3-pointer: 5
After a free throw: 6
```

Rule: don't type the running totals (2, 5 and 6) yourself. Let JavaScript work them out from the old score.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Bug hunt at the library

A library wrote a small program to work out late fees, but it won't run. Copy it into `ex3.js` and fix it one error at a time.

```
// Library late fees
const feePerDay = 2;
const 1stBookDays = 3;
const second-book-days = 5;

const totalDays = 1stBookDays + second-book-days;
const totalFee = totalDays * feePerDay;
console.log("Days late:", totalDays);
console.log("Total fee:", totalfee);

totalFee = totalFee - 4;
console.log("Fee after member discount:", totalFee);
```

There are 4 bugs, and some of them show up in more than one place. When it's fixed, you should see:

```
Days late: 8
Total fee: 16
Fee after member discount: 12
```

Rule: fix the bugs, don't rewrite the program. Any new names you pick should follow the naming rules and use camelCase.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The mixed-up drinks

At a busy cafe, a waiter put two drinks on the wrong tables. Start your file with these two lines:

```
let table1 = "orange juice";
let table2 = "hot chocolate";
```

Print what each table has, swap the drinks, then print again.

Expected output:

```
Before the swap
Table 1: orange juice
Table 2: hot chocolate
After the swap
Table 1: hot chocolate
Table 2: orange juice
```

Rule: after the first two lines, don't type "orange juice" or "hot chocolate" again. Move the values between the variables instead.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Your life in numbers, upgraded

Time to fix the program from [chapter 01's last exercise](#) for good. Write a new version of "Your life in numbers" that uses variables.

Your program must follow these rules:

- Your age appears **only once** in the whole file.
- Every fixed number gets a name too: days in a year (365), hours in a day (24), minutes in an hour (60), and heartbeats per minute (70).
- Each fact is its own variable, built from the one before it. For example, hours alive comes from days alive.
- Add one new fact: breaths taken (we breathe about 16 times a minute).
- Keep a comment above each calculation explaining what it works out, like in chapter 01.

Expected output for someone who is 25:

```
My life in numbers
Days alive: 9125
Hours alive: 219000
Minutes alive: 13140000
Heartbeats: 919800000
Breaths: 210240000
```

Now test it: change the age to 30 and run it again. The second line should say `Days alive: 10950`, and every other number should change too, without you touching those lines. Finally, put in your own age.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: bring back the "years asleep" line from chapter 01 (we sleep about 8 hours a day), with a named value for the hours of sleep. For age 25 you should see `Years asleep: 8.333333333333334`. You'll learn to tidy up long decimals like that in chapter 05.

Before you move on

Your boxes held two kinds of things: numbers like `25` and text like `"Lisbon"`. Does that difference matter? Guess what each line prints, then run it:

```
console.log(25 + 1);
console.log("25" + 1);
```

If the second line surprised you, [chapter 03](#) explains why. 😊

03 Data Types

What is it?

Every value in JavaScript has a **type**: the kind of value it is. Text is one type, numbers are another, and yes/no answers are a third.

The type decides what you can do with a value. You can multiply two numbers, but multiplying two names makes no sense.

Why does it matter?

Look at these two lines:

```
console.log(5 + 3); // prints: 8
console.log("5" + 3); // prints: 53
```

They look almost the same, but the results are completely different. The only difference is the type: `5` is a number, and `"5"` (in quotes) is text.

This isn't only a puzzle. It causes real bugs. Picture an online shop where a T-shirt costs 20 and socks cost 5, but the basket shows a total of `205`. That happens when the prices arrive as text instead of numbers.

Once you know about types, results like these stop being mysterious, and you'll know how to fix them.

Real-world example

Think of the sign-up form at a gym. Each box on the form expects a different kind of answer:

On the form	Example answer	JavaScript type
Full name	Maya Patel	string (text)
Age	29	number
Tick box: "Do you want a locker?"	Yes or no	boolean (true or false)
Middle name: you don't have one, so you write "none"	Nothing, on purpose	null
Emergency contact: you haven't got to it yet	Nothing, not filled in yet	undefined

You can do math with the age (how many years until you're 30?), but not with the name. A tick box can only ever be yes or no.

JavaScript's types work the same way: each kind of value has its own rules.

How it works

The five basic types

Type	What it holds	Examples
string	Text, inside quotes	"Maya", "Hello!", "42"
number	Any number: whole, decimal or negative	42, 3.5, -10
boolean	Yes or no	true, false
undefined	"Nothing has been put here yet"	undefined
null	"Empty, on purpose"	null

These five are called **primitive** types. A primitive is a simple, single value, one of the basic building blocks everything else is made from.

JavaScript has two more primitives, **bigint** (for huge whole numbers) and **symbol** (for unique labels). They're rare, and you'll meet them in chapters 05 and 36.

Everything that isn't a primitive is an **object**: a value that holds several values together, like a list. Lists (called arrays) and objects come in chapters 10 and 11. For now, the five types above are all you need.

Checking a type with `typeof`

Put `typeof` in front of a value, and JavaScript tells you its type:

```
console.log(typeof "Maya"); // prints: string
console.log(typeof 42); // prints: number
console.log(typeof 3.5); // prints: number
console.log(typeof true); // prints: boolean
console.log(typeof undefined); // prints: undefined
```

It works on variables too, and it's the best way to tell two look-alike values apart. `console.log` prints text without its quotes, so a number and a piece of text can look identical:

```
const price = 20;
const priceText = "20";
console.log(price, priceText); // prints: 20 20
console.log(typeof price, typeof priceText); // prints: number string
```

When a result looks strange, check the type. It's often the answer.

Strings: text

A **string** is text. You met strings in chapter 01: anything inside quotes. JavaScript accepts double quotes `"..."` or single quotes `'...'`. This course uses double quotes.

Anything inside quotes is a string, even if it's made of digits. Even `""` is a string: it's called an **empty string**, like a text box with nothing typed in it.

```
console.log(typeof "42"); // prints: string
console.log(typeof ""); // prints: string
```

Joining strings with +

When you use `+` with strings, it glues them together. This is called **concatenation** (a fancy word for joining).

```
const firstName = "Ada";
const lastName = "Lovelace";
const fullName = firstName + " " + lastName;
console.log(fullName); // prints: Ada Lovelace
```

See the `" "` in the middle? It's a string that holds a single space. Without it, you'd get `AdaLovelace`. The `+` adds nothing extra, not even a space.

You already know another way to print several things: commas in `console.log`. Here's the difference:

```
const city = "Oslo";
console.log("Welcome to", city); // prints: Welcome to Oslo
console.log("Welcome to " + city); // prints: Welcome to Oslo

const greeting = "Welcome to " + city + "!";
console.log(greeting); // prints: Welcome to Oslo!
```

- **Commas** print several values, and `console.log` puts a space between them for you.
- `+` builds one new string. You add the spaces yourself, but you can store the result in a variable and use it later.

(Chapter 06 shows a neater way to build text, called template literals.)

Numbers: whole and decimal

JavaScript has one type, `number`, for every kind of number: whole numbers, decimals, and negative numbers. Numbers never go in quotes.

```
const temperature = -3;
const price = 4.99;
const guests = 12;
console.log(typeof temperature, typeof price, typeof guests); // prints: number number number
```

Numbers have a few surprises of their own. You'll meet them in [chapter 05](#).

The "5" + 3 surprise

Now you can understand the puzzle from the start of this chapter:

```
console.log(5 + 3); // prints: 8
console.log("5" + 3); // prints: 53
console.log("5" + "3"); // prints: 53
```

Here's the rule: **if either side of `+` is a string, JavaScript joins instead of adding**. It turns the number into text first, then glues the two together. And the result is a string:

```
const result = "5" + 3;
console.log(result, typeof result); // prints: 53 string
```

This often bites when you build a message that has numbers in it:

```
const pizzas = 2;
const drinks = 3;
console.log("Items: " + pizzas + drinks); // prints: Items: 23
```

JavaScript works from left to right. First `"Items: " + 2` makes the string `"Items: 2"`. Then `"Items: 2" + 3` joins again, giving `"Items: 23"`.

The fix: do the math first, in its own variable. Then join.


```
const pizzas = 2;
const drinks = 3;
const items = pizzas + drinks;
console.log("Items: " + items); // prints: Items: 5
```

(Brackets can fix this too. [Chapter 04](#) shows how.)

Booleans: true or false

A **boolean** has only two possible values: `true` and `false`. Think of a light switch: it's either on or off, nothing in between.

Booleans are perfect for yes/no facts:

```
const isOpen = true;
const hasTicket = false;
console.log(isOpen, hasTicket); // prints: true false
console.log(typeof isOpen); // prints: boolean
```

`true` and `false` never go in quotes. With quotes, you get a string that just happens to spell the word:

```
console.log(typeof true); // prints: boolean
console.log(typeof "true"); // prints: string
```

Tip: Boolean names often start with `is`, `has` or `can`, like `isRaining`, `hasPaid` or `canVote`. They read like yes/no questions.

So far you've typed `true` and `false` yourself. Most booleans come from questions like "Is 10 bigger than 5?" You'll learn to ask those in [chapter 04](#), and to make decisions with the answers in chapter 07.

undefined and null : two kinds of "nothing"

JavaScript has two ways to say "there's nothing here". Picture two boxes:

- **undefined** is an empty box. Nothing has been put in it yet. JavaScript uses it automatically, like for a `let` with no value (remember [chapter 02](#)?).
- **null** is a box with a note inside that says "empty, on purpose". Someone put it there deliberately (you, or the code you're using) to say "we know there's nothing here".

Here's an online order where both show up:

```
let deliveryDate; // not decided yet
const couponCode = null; // the customer has no coupon, and we know it

console.log(deliveryDate); // prints: undefined
console.log(couponCode); // prints: null
```

A simple rule of thumb: let JavaScript use `undefined`. When you want to say "nothing here, on purpose", use `null`.

The `typeof null` quirk

Try this:

```
console.log(typeof null); // prints: object
```

That's wrong! `null` isn't an object. It's its own type.

This is a mistake from the very first version of JavaScript, back in 1995. Fixing it now would break millions of older websites that depend on it, so it stays. Every JavaScript developer has to remember it, and now you do too.

A variable can hold any type

In some languages, like Java or C#, you pick a type for each variable when you create it, and it can never hold anything else. JavaScript doesn't work that way. It's **dynamically typed**: the same `let` box can hold a number now and text later.

```
let answer = 42;
console.log(typeof answer); // prints: number

answer = "forty-two";
console.log(typeof answer); // prints: string
```

This freedom is handy, but it can also lead to confusing bugs. A good habit: keep each variable to one type.

Converting between types: `Number()` and `String()`

Text that looks like a number isn't a number yet. This matters more than you'd think. When someone types into a box on a web page, JavaScript receives text, even if they typed digits. (You'll build forms like that in chapter 22.)

`Number()` turns a value into a number:

```
const ticketsText = "3"; // typed in by a user, so it's text
const tickets = Number(ticketsText);

console.log(tickets + 2); // prints: 5
console.log(typeof tickets); // prints: number
```

If the text isn't a number at all, you get `NaN`, which stands for "Not a Number":

```
console.log(Number("3.5")); // prints: 3.5
console.log(Number("hello")); // prints: NaN
```

`String()` goes the other way. It turns a value into text:

```
const year = 2026;
const yearText = String(year);
console.log(yearText, typeof yearText); // prints: 2026 string
```

You won't need `String()` often, because `+` already turns a number into text when it joins it to a string.

Converting has more corners to explore. [Chapter 05](#) covers `NaN` and more ways to read numbers from text, and chapter 38 covers all the ways JavaScript converts types by itself.

Common mistakes

1. Adding numbers that are really text

```
const morningSteps = "4000";
const eveningSteps = "3500";
console.log(morningSteps + eveningSteps); // prints: 40003500
```

Both values are strings, so `+` joins them instead of adding. Convert them first: `Number(morningSteps) + Number(eveningSteps)` gives `7500`.

2. Forgetting the spaces when joining

```
const guest = "Maya";
console.log("Hello" + guest); // prints: HelloMaya
```

`+` never adds spaces for you. Put them inside the quotes yourself: `"Hello, " + guest`.

3. Putting `true` or `false` in quotes

```
const isMember = "false";
console.log(typeof isMember); // prints: string
```

It looks like a boolean, but it's a string. That difference will matter a lot when you start making decisions in chapter 07. Fix: `const isMember = false;`

4. Capital letters on `true`, `false`, `null` or `undefined`

```
const isOpen = True;
// ReferenceError: True is not defined
```

These words are always lowercase. With a capital letter, JavaScript thinks `True` is the name of a variable you never created.

5. Trusting what `console.log` shows you

```
const score = "10";
console.log(score); // prints: 10
```

It looks like a number, but it's a string: `console.log` doesn't show the quotes. When a result seems odd, check with `typeof score`.

Quick recap

- Every value has a type. The five basic (primitive) types are string, number, boolean, undefined and null.
- `typeof` tells you a value's type. Watch out: `typeof null` says `object`, an old mistake that's here to stay.
- `+` adds numbers but joins strings. If either side is a string, the result is a string: `"5" + 3` gives `"53"`.
- `Number()` turns text into a number (or `NaN` if it can't), and `String()` turns a value into text.
- `undefined` means "nothing here yet". `null` means "empty, on purpose".
- A variable can hold any type (JavaScript is dynamically typed), but it's clearer to keep each variable to one type.

Next: try the [exercises](#), then move on to [04 Operators](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).

- Run each one with `node ex1.js` from a terminal opened in this folder.
 - When a result looks strange, check its type with `typeof`. It's often the answer.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): The library record

A library keeps a record for every book. Create one variable for each detail of this book, and pick the right type each time:

- Title: The Hobbit
- Author: J.R.R. Tolkien
- Pages: 310
- Available to borrow: yes
- Due date: none, because nobody has borrowed it (empty, on purpose)
- Shelf: not decided yet (the librarian will fill it in later)

Then print each value next to its type.

Expected output:

```
Title: The Hobbit -> string
Author: J.R.R. Tolkien -> string
Pages: 310 -> number
Available: true -> boolean
Due date: null -> object
Shelf: undefined -> undefined
```

Yes, one of those types looks wrong. That's not your bug! Check the notes if you're not sure why.

Rule: let `typeof` work out the types. Don't type `string`, `number` and so on yourself.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Conference name badge

You're printing name badges for a baking conference. Start your file with these lines:

```
const firstName = "Maya";
const lastName = "Patel";
const jobTitle = "Head Baker";
const company = "Blue Owl Bakery";
```

Build and print the three lines of the badge.

Expected output:

```
Maya Patel
Head Baker at Blue Owl Bakery
Hi, I'm Maya!
```

Rule: no commas inside `console.log()` this time. Build each line with `+`. And don't type `Maya`, `Patel` or the other details again: use the variables.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The broken basket

Your online shop reads its prices from a web page, so they arrive as text. A customer is buying a shirt and a cap, and the checkout shows a crazy total. Copy this into `ex3.js` and run it:

```
// Prices arrive from the web page as text
const shirtPrice = "20";
const capPrice = "12";
const shipping = 5;

const total = shirtPrice + capPrice + shipping;
console.log("Total:", total);
console.log("Type:", typeof total);
```

The total comes out as `20125`. Fix the program, then add a friendly last line, so it prints:

```
Total: 37
Type: number
Your order comes to 37 dollars.
```

Rule: don't change the three lines that create `shirtPrice`, `capPrice` and `shipping`. The prices really do arrive as text. Build the last line with `+`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Hotel check-in card

A hotel's check-in program keeps a card for each guest. Some details are known, one is empty on purpose, and one isn't known yet:

- Guest: Leo Rossi
- Nights: 3
- Breakfast included: yes
- Special requests: the guest said they have none
- Room number: not assigned yet

Print the card. Then the room gets ready: give the guest room 214, print the room again, and finish with a summary sentence.

Expected output:

```
--- Check-in card ---
Guest: Leo Rossi
Nights: 3
Breakfast: true
Special requests: null
Room: undefined
--- Room is ready ---
Room: 214
Leo Rossi is in room 214 for 3 nights.
```

Rule: only the detail that changes should use `let`. Build the last sentence with `+`, using your variables.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Cinema booking summary

A cinema's booking form sends everything as text, even the number of tickets. Start your file with these lines:

```
// From the booking form (a form always sends text)
const movieTitle = "Dune: Part Two";
const adultTicketsText = "2";
const childTicketsText = "1";
```

Here's what else you know:

- An adult ticket costs 12 dollars, and a child ticket costs 8.

- The customer asked for seats together (yes).
- The customer didn't enter a promo code (empty, on purpose).

Print this booking summary:

```
=== Booking summary ===  
Movie: Dune: Part Two  
Tickets: 3 (2 adult, 1 child)  
Total: 32 dollars  
Seats together: true  
Promo code: null  
Check: string -> number
```

The last line checks your work: it shows the type of `adultTicketsText`, then the type of your converted adult tickets.

Rules:

- Don't change the starting lines. After them, don't type the numbers 2, 1, 3 or 32 yourself: work them out from the variables.
- Build the `Tickets:` line and the `Total:` line with `+`, and store each one in a variable before printing it.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

In this chapter, you typed `true` and `false` yourself. In real programs, most booleans come from asking questions. Guess what these two lines print, then run them:

```
console.log(10 > 5);  
console.log(3 > 7);
```

[Chapter 04](#) teaches you how to ask questions like these, and a lot more.

04 Operators

What is it?

An **operator** is a symbol that does something with values. For example, `+` adds two numbers, and `=` puts a value into a variable.

You've already used a few. This chapter fills in the rest: more math, shortcuts for updating variables, and operators that compare values and answer with `true` or `false`.

Why does it matter?

Programs are constantly calculating and checking things:

- A shop adds up your basket, then checks whether you get free delivery.
- A game adds points, then checks whether you've beaten the high score.
- A cinema checks whether you're old enough to see a movie.

Operators are how you write those calculations and questions.

And getting them slightly wrong can be sneaky. If you forget that `*` happens before `+`, your program won't crash. It will calmly give you the wrong answer.

Real-world example

Think of a day at a theme park. You use every kind of operator without noticing:

At the theme park	Kind of operator
Adding up the price of tickets and snacks	Arithmetic: math like adding and multiplying
Topping up your ride card with more credit	Assignment: putting a new value into a variable
The "You must be this tall to ride" sign	Comparison: a yes/no question about two values
"You need a ticket AND you must be over 12"	Logical: combining yes/no answers

Let's go through them one at a time.

How it works

Arithmetic operators

Operator	Meaning	Example	Result
+	Add	7 + 2	9
-	Subtract	7 - 2	5
*	Multiply	7 * 2	14
/	Divide	7 / 2	3.5
%	Remainder	7 % 2	1
**	Power	7 ** 2	49

You know the first four from chapter 01. The last two are new.

% : what's left over

% is the **remainder** operator. It tells you what's left over after you share something out in equal groups. (You may hear it called "modulo".)

Say 3 friends share a pizza with 10 slices. Each friend gets 3 slices, which uses up 9, so 1 slice is left over:

```
const slices = 10;
const friends = 3;
console.log(slices % friends); // prints: 1
```

A classic use is checking whether a number is even or odd. An even number divided by 2 leaves nothing over. An odd number leaves 1:

```
console.log(8 % 2); // prints: 0
console.log(7 % 2); // prints: 1
```

** : powers

** raises a number to a **power**: `4 ** 2` means 4×4 , and `2 ** 3` means $2 \times 2 \times 2$.

It's handy for areas. A square room with 4-meter walls has a floor area of `4 ** 2` square meters:

```
const wallLength = 4;
console.log(wallLength ** 2); // prints: 16
console.log(2 ** 3); // prints: 8
```

Which goes first? (precedence)

When a calculation has several operators, JavaScript follows the same order you learned in school math. The order is called **precedence**:

1. Brackets `( )` first
2. Then powers `**`
3. Then `*`, `/` and `%`, from left to right
4. Then `+` and `-`, from left to right

```
console.log(2 + 3 * 4); // prints: 14
console.log((2 + 3) * 4); // prints: 20
```

In the first line, `3 * 4` happens first, then `2 +`. In the second, the brackets make `2 + 3` happen first.

Here's how this causes a real bug. You want the average of three test scores:

```
const test1 = 80;
const test2 = 90;
const test3 = 70;

const wrongAverage = test1 + test2 + test3 / 3;
const rightAverage = (test1 + test2 + test3) / 3;
console.log(wrongAverage); // prints: 193.33333333333334
console.log(rightAverage); // prints: 80
```

Without brackets, only `test3` gets divided by 3. No error, just a wrong answer.

Brackets also fix the "Items: 23" surprise from [chapter 03](#). They make the math happen before the joining:

```
const pizzas = 2;
const drinks = 3;
console.log("Items: " + (pizzas + drinks)); // prints: Items: 5
```

Tip: When in doubt, add brackets. They cost nothing, and they make it clear to the reader what you meant.

Putting it together: minutes into hours

A film is 135 minutes long. How long is that in hours and minutes?

```
const totalMinutes = 135;
const leftoverMinutes = totalMinutes % 60;
const hours = (totalMinutes - leftoverMinutes) / 60;
console.log(hours, "h", leftoverMinutes, "min"); // prints: 2 h 15 min
```

Step by step:

1. `135 % 60` is `15` : the minutes left over after taking out all the full hours.
2. `135 - 15` is `120` , which divides evenly by 60.
3. `120 / 60` is `2` full hours.

The same trick works for anything you share out: slices per friend, eggs per box, seconds into minutes. (Chapter 05 shows a shorter way to do steps 2 and 3, with `Math.floor`.)

Assignment shortcuts: `+=` , `-=` , `*=` , `/=`

Remember updating a variable from its old value in [chapter 02](#)? `score = score + 10` is so common that it has a shortcut: `score += 10` . Both mean exactly the same thing.

Here's a player's coins in a game:

```
let coins = 100;

coins += 50; // found a treasure chest
console.log(coins); // prints: 150

coins -= 30; // bought a potion
console.log(coins); // prints: 120

coins *= 2; // double-coins weekend
console.log(coins); // prints: 240

coins /= 4; // shared equally with a team of 4
console.log(coins); // prints: 60
```

Shortcut	Means the same as
<code>coins += 50</code>	<code>coins = coins + 50</code>
<code>coins -= 30</code>	<code>coins = coins - 30</code>

Shortcut	Means the same as
<code>coins *= 2</code>	<code>coins = coins * 2</code>
<code>coins /= 4</code>	<code>coins = coins / 4</code>

`+=` works on strings too, which is handy for building text a piece at a time:

```
let shoppingList = "Milk";
shoppingList += ", Bread";
shoppingList += ", Eggs";
console.log(shoppingList); // prints: Milk, Bread, Eggs
```

These shortcuts change the variable, so it must be a `let`. A `const` gives you the `TypeError` from chapter 02.

Adding or taking away 1: `++` and `--`

Adding or taking away exactly 1 is so common that it gets its own operators. Picture the door counter at a museum:

```
let visitors = 0;
visitors++; // same as: visitors += 1
visitors++;
console.log(visitors); // prints: 2

visitors--; // one person left
console.log(visitors); // prints: 1
```

You may also see `++visitors`, with the `++` in front. On a line by itself, it does the same thing. The two versions only behave differently inside a bigger calculation, so keep `++` and `--` on their own line and you'll never need to worry about it.

Comparison operators

A comparison asks a yes/no question about two values. The answer is always a boolean, `true` or `false` (remember [chapter 03](#)?).

Operator	The question it asks	Example	Result
<code>></code>	Is the left bigger?	<code>5 > 3</code>	<code>true</code>
<code><</code>	Is the left smaller?	<code>5 < 3</code>	<code>false</code>

Operator	The question it asks	Example	Result
<code>>=</code>	Is the left bigger or equal?	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Is the left smaller or equal?	<code>4 <= 3</code>	<code>false</code>
<code>===</code>	Are they equal?	<code>5 === 5</code>	<code>true</code>
<code>!==</code>	Are they different?	<code>5 !== 3</code>	<code>true</code>

Here's the height check for a ride at the theme park:

```
const height = 125; // in centimeters
const minimumHeight = 120;
const canRide = height >= minimumHeight;
console.log("Can ride:", canRide); // prints: Can ride: true
```

Storing the answer in a variable with a yes/no name, like `canRide`, makes your code read like plain English. For now you'll print these answers. In chapter 07, you'll use them to make decisions, like "if they can ride, open the gate".

Math happens before comparisons, so this even/odd check needs no brackets. A dance class needs an even number of people, so everyone has a partner:

```
const dancers = 7;
console.log("Everyone has a partner:", dancers % 2 === 0); // prints: Everyone has a partner: false
```

`===` checks the value and the type

`===` is called **strict equality**. Both sides must be the same type *and* the same value:

```
console.log(5 === 5); // prints: true
console.log(5 === "5"); // prints: false
console.log("cat" === "cat"); // prints: true
console.log("Cat" === "cat"); // prints: false
```

The number `5` and the string `"5"` are different types, so they're not equal. Strings must match exactly, including capital letters. (Chapter 06 has more on comparing text.)

`!==` is the opposite. It's `true` when the two values are different:

```
const lockerCode = 4821;
console.log(lockerCode !== 1234); // prints: true
```

`==` vs `===` : always use `===`

JavaScript also has `==` (two equals signs), called **loose equality**, and its opposite `!=`. Before comparing, they quietly convert both sides to the same type. That leads to surprises:

```
console.log(5 == "5"); // prints: true
console.log(0 == ""); // prints: true
console.log(0 === ""); // prints: false
```

The number `0` and an empty string are not the same thing, but `==` says they are.

The rule is simple: **always use `===` and `!==`**. You'll see `==` in older code, but don't write it yourself. If you need to compare text with a number, convert the text first with `Number()`, so you know exactly what's being compared. (Chapter 38 explains the rules `==` follows, if you're curious.)

Logical operators: `&&`, `||` and `!`

Logical operators combine yes/no answers into one bigger answer:

- `&&` means **AND**. It's `true` only when both sides are `true`. "You can use the gym if you're a member AND it's open."
- `||` means **OR**. It's `true` when at least one side is `true`. "Delivery is free if you're a member OR your order is 50 or more."
- `!` means **NOT**. It flips `true` to `false`, and `false` to `true`.

```
const isMember = true;
const isOpen = false;

console.log(isMember && isOpen); // prints: false
console.log(isMember || isOpen); // prints: true
console.log(!isOpen); // prints: true
```

Here's every possible combination:

Left side	Right side	Left AND right	Left OR right
true	true	true	true
true	false	false	true

Left side	Right side	Left AND right	Left OR right
false	true	false	true
false	false	false	false

In short: AND is picky (both must be true). OR is easygoing (one true is enough).

Most of the time, you'll combine comparisons. Comparisons happen before `&&` and `||`, so you don't need brackets:

```
const age = 15;
const hasTicket = true;

const isTeenager = age >= 13 && age <= 19;
const canEnter = age >= 12 && hasTicket;
console.log("Teenager:", isTeenager); // prints: Teenager: true
console.log("Can enter:", canEnter); // prints: Can enter: true
```

When you mix `&&` and `||` in one line, add brackets anyway, so nobody has to remember which one goes first. Here, delivery is free for members, or for orders of 40 or more during the holidays:

```
const orderTotal = 42;
const isMember = false;
const isHoliday = true;

const freeDelivery = isMember || (orderTotal >= 40 && isHoliday);
console.log("Free delivery:", freeDelivery); // prints: Free delivery: true
```

And `!` reads nicely with yes/no names:

```
const isRaining = false;
console.log("Picnic time:", !isRaining); // prints: Picnic time: true
```

`&&` and `||` can do more when you use them with values that aren't `true` or `false`. That's a trick for [chapter 07](#). In this chapter, only use them with booleans.

Common mistakes

1. Using `=` when you mean `===`


```
let lives = 3;
console.log(lives = 0); // prints: 0
console.log(lives); // prints: 0
```

You wanted to ask "is `lives` 0?". But a single `=` doesn't ask anything: it puts 0 into the box. The player just lost all their lives! Fix: `lives === 0`.

(If `lives` were a `const`, you'd get the `TypeError` from chapter 02 instead. One more reason to prefer `const`.)

2. Using `==` instead of `===`

```
console.log(0 == ""); // prints: true
```

`==` converts types before comparing, so different things can look equal. Always use `===` and `!==`.

3. Comparing text from a form with a number

```
const ageText = "18"; // typed into a form, so it's text
console.log(ageText === 18); // prints: false
```

Text is never `===` to a number. Convert it first: `Number(ageText) === 18` gives `true`.

4. Writing `<=` or `>=` instead of `<` or `>`

```
const height = 125;
console.log(height < 120);
// SyntaxError: Unexpected token '<'
```

The `=` always comes second: `<=` and `>=`. Say it out loud in that order: "less than or equal".

`=>` is even sneakier. It means something completely different in JavaScript (you'll meet it in chapter 09), so `console.log(height => 120)` gives no error at all. It prints `[Function (anonymous)]`, which is definitely not the answer you wanted.

5. Chaining comparisons like in math class

```
const age = 70;
console.log(18 <= age <= 65); // prints: true
```

70 isn't between 18 and 65, so why `true`? JavaScript does one comparison at a time. First, `18 <= 70` gives `true`. Then it compares `true <= 65`: it turns `true` into the number 1 (chapter 38 explains why), and `1 <= 65` is `true`.

Fix: ask two questions and join them with `&&`: `age >= 18 && age <= 65`.

6. Using a shortcut on a `const`

```
const total = 10;
total += 5;
// TypeError: Assignment to constant variable.
```

`+=`, `++` and the other shortcuts change the variable, so it has to be a `let`.

Quick recap

- Arithmetic: `+` `-` `*` `/` `%` `**`. `%` gives the remainder (leftovers, even or odd), and `**` raises to a power.
- Precedence: brackets first, then `**`, then `*` `/` `%`, then `+` `-`. When in doubt, add brackets.
- Shortcuts: `x += 5` means `x = x + 5` (the same goes for `-=`, `*=` and `/=`). `x++` and `x--` add or take away 1. They all need `let`.
- Comparisons (`>` `<` `>=` `<=` `===` `!==`) answer with `true` or `false`.
- Always use `===` and `!==`. Never `==` or `!=`.
- `&&` (AND), `||` (OR) and `!` (NOT) combine true/false answers.

Next: try the [exercises](#), then move on to [05 Numbers and Math](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).
- Run each one with `node ex1.js` from a terminal opened in this folder.
- Let JavaScript do every calculation. Typing an answer yourself doesn't count!
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Pizza party

You ordered 3 pizzas, each cut into 8 slices, for 7 friends. Everyone gets the same number of slices, and any extra slices are left over.

Expected output:

```
Total slices: 24
Slices each: 3
Leftover slices: 3
Even split: false
```

Rule: start from three variables (pizzas, slices per pizza, and friends). Don't type 24 or any other answer yourself.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Cafe loyalty card

A cafe's loyalty card gives you 1 point for every dollar you spend. You start with 0 points and 0 visits. Here's your week:

- Visit 1: you spend 15 dollars.
- Visit 2: you spend 40 dollars.
- It's your birthday, so the cafe doubles your points.
- Visit 3: you swap 60 points for a free lunch (and spend nothing).

Expected output:

```
After visit 1: 15 points
After visit 2: 55 points
Birthday bonus: 110 points
After visit 3: 50 points
Total visits: 3
```

Rule: update your variables only with the shortcuts `+=`, `-=`, `*=` and `++`. No `points = points + ...` this time.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Movie marathon

You're planning a movie marathon: three films that are 148, 136 and 169 minutes long, with two 20-minute breaks in between. It starts at 10:00 in the morning.

Expected output:

```
Films: 453 minutes
Breaks: 40 minutes
Total: 493 minutes
That's 8 hours and 13 minutes
The marathon ends at 18:13
```

Rule: don't type any of the answers. Use `%` to find the leftover minutes.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Theme park gatekeeper

You're writing the program for the gates at a theme park. Start your file with these lines about a visitor:

```
const age = 11;
const height = 142; // in centimeters
const hasTicket = true;
const withAdult = false;
```

These are the park's rules:

- **Roller coaster:** at least 140 cm tall, and has a ticket.
- **Haunted house:** has a ticket, and is either 12 or older or with an adult.
- **Child price:** under 12, or 65 and over.
- **Needs a grown-up:** under 12, and not with an adult.

Expected output:

```
Roller coaster: true
Haunted house: false
Child price: true
Needs a grown-up: true
```

Then change `withAdult` to `true` and run it again. Exactly two lines should change. Guess which two before you run it!

Rule: work out every answer with comparison and logical operators. Apart from the starting lines, don't type `true` or `false` anywhere.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Bookshop checkout

You're building the checkout for an online bookshop. Start your file with these lines:

```
// Quantities from the web page arrive as text
const novelsText = "2";
const cookbooksText = "1";
const isMember = false;
```

Here's what else you know:

- A novel costs 14 dollars, and a cookbook costs 23.
- Delivery is free for members, or for orders of 50 dollars or more.
- Customers earn 1 loyalty point per dollar, and today is double-points day.

Expected output:

```
Novels: 2 x 14 = 28
Cookbooks: 1 x 23 = 23
Subtotal: 51
Books in the order: 3
Free delivery: true
Even number of books: false
Points earned: 102
```

Rules:

- Don't change the starting lines, and don't type any of the results yourself.
- For the points, start a `let` at 0, add the subtotal with `+=`, then double it with another shortcut.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Guess what this prints, then run it:

```
console.log(0.1 + 0.2);
```

Surprised? You're in good company. [Chapter 05](#) explains what's going on, and why it matters when your program handles money.

05 Numbers and Math

What is it?

JavaScript has one type for every number: whole numbers like 47 and decimals like 4.99 are both number (remember chapter 03?).

This chapter is about working with numbers: rounding them, making random ones, reading them from text, and showing them the way people expect, like \$1,234.50. Most of the tools live in Math, a toolbox of math helpers that comes built into JavaScript.

Why does it matter?

Real programs need more than + - * / :

- A shop must show prices with exactly two decimals: \$4.50, not \$4.5.
- A board game needs random dice rolls.
- A party planner must round up: 47 guests at tables of 6 need 8 tables, not 7.83.
- A web page gives you the text "42px", and you need the number 42.

And there's a surprise waiting that trips up almost everyone:

```
console.log(0.1 + 0.2); // prints: 0.30000000000000004
```

This chapter explains why that happens, and how to handle it, especially when money is involved.

Real-world example

Numbers in everyday life come with everyday questions. Each question needs a different tool:

Everyday question	What you need	JavaScript	Answer
47 guests, tables seat 6. How many tables?	Round up	Math.ceil(47 / 6)	8
40 eggs, boxes hold 12. How many full boxes?	Round down	Math.floor(40 / 12)	3
A 4.7-star rating. How many stars to show?	Round to the nearest	Math.round(4.7)	5

Everyday question	What you need	JavaScript	Answer
Three scores: 72, 95 and 88. Which is the best?	The biggest	<code>Math.max(72, 95, 88)</code>	95
Roll a dice	A random whole number	<code>Math.floor(Math.random() * 6) + 1</code>	1 to 6

By the end of this chapter, you'll know every tool in that table, and a few more.

How it works

Whole number or decimal?

Whole numbers and decimals share one type, but sometimes you need to know which one you've got. Maybe you can sell 3 T-shirts, but not 2.5. `Number.isInteger()` tells you. (An **integer** is a whole number, with no decimal part.)

```
console.log(Number.isInteger(47)); // prints: true
console.log(Number.isInteger(4.99)); // prints: false
console.log(Number.isInteger(5.0)); // prints: true
```

`5.0` counts as a whole number, because it's exactly the same value as `5`.

Notice the dot in `Number.isInteger`. It means "the `isInteger` tool that belongs to `Number`". A tool that belongs to something like this is called a **method**. You've used one since chapter 01: `log` is a method of `console`.

The `0.1 + 0.2` surprise

```
console.log(0.1 + 0.2); // prints: 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // prints: false
```

Why? Computers store numbers in **binary**: using only 0s and 1s. Whole numbers fit perfectly, but many decimals don't. Think of trying to write $\frac{1}{3}$ as a decimal: 0.33333... goes on forever, so at some point you have to stop and round. In binary, 0.1 is like that. The computer stores a number that's a tiny bit off, and when you add two slightly-off numbers, the tiny error can show up in the answer.

This isn't a JavaScript bug. Python, Java, C# and almost every other language give the same answer, because they all store decimals the same standard way.

Most of the time, an error that small doesn't matter. It matters in two places: when you compare decimals with `===`, and when you show numbers to people, especially money.

Handling money: work in cents

Whole numbers are stored exactly. So the safest way to handle money is to do all the math in **cents** (whole numbers), and turn the result into dollars only at the very end, for display.

A pen costs \$1.10 and a notebook costs \$2.20:

```
console.log(1.10 + 2.20); // prints: 3.3000000000000003

const penCents = 110;
const notebookCents = 220;
const totalCents = penCents + notebookCents;
console.log(totalCents); // prints: 330
console.log(totalCents / 100); // prints: 3.3
```

The cents version is exact: 330 cents is \$3.30. Many real payment systems work exactly like this: they store amounts in the smallest unit of the currency, like cents.

That last line printed `3.3`, though, not `3.30`. That's where `toFixed` comes in.

`toFixed`: exactly two decimals (as text!)

`toFixed(2)` rounds a number to 2 decimal places, and adds zeros if it needs to:

```
const total = 1.1 + 2.2;
console.log(total.toFixed(2)); // prints: 3.30

const price = 4.5;
console.log(price.toFixed(2)); // prints: 4.50
```

So for everyday prices, you can also use decimals and round the result with `toFixed(2)` when you show it. Working in cents is still the safest choice, because rounding can be thrown off by those same tiny errors: `(1.005).toFixed(2)` gives `"1.00"`, not `"1.01"`.

Watch out: `toFixed` gives you back a **string**, not a number. It's meant for showing, not for more math:


```
const price = 4.5;
const priceTag = price.toFixed(2);
console.log(priceTag, typeof priceTag); // prints: 4.50 string
console.log(priceTag + 1); // prints: 4.501
```

Remember the `"5" + 3` surprise from chapter 03? It's back. The rule: **do all your math with numbers first, and call `toFixed` last**, right before you print.

Rounding: `Math.round`, `Math.floor`, `Math.ceil` and `Math.trunc`

JavaScript has four ways to turn a decimal into a whole number:

Tool	What it does	4.7 becomes	4.2 becomes
<code>Math.round()</code>	Rounds to the nearest whole number	5	4
<code>Math.floor()</code>	Always rounds down	4	4
<code>Math.ceil()</code>	Always rounds up ("ceil" is short for ceiling)	5	5
<code>Math.trunc()</code>	Cuts off the decimals ("trunc" is short for truncate)	4	4

A number exactly halfway, like `4.5`, rounds up to `5` with `Math.round`.

`Math.floor` and `Math.trunc` look the same for positive numbers. They only differ below zero. Say it's -4.7 degrees outside:

```
console.log(Math.floor(-4.7)); // prints: -5
console.log(Math.trunc(-4.7)); // prints: -4
```

`Math.floor` goes *down* to -5, while `Math.trunc` just chops off the `.7`.

Picking the right one is about the question you're asking. Rounding up makes sure everyone gets a seat:

```
const guests = 47;
const seatsPerTable = 6;
console.log("Tables needed:", Math.ceil(guests / seatsPerTable)); // prints: Tables needed: 8
```

Rounding down counts only the complete groups:

```
const eggs = 40;
const eggsPerBox = 12;
console.log("Full boxes:", Math.floor(eggs / eggsPerBox)); // prints: Full boxes: 3
```

And remember the minutes-into-hours trick from [chapter 04](#)? `Math.floor` makes it shorter:

```
const totalMinutes = 135;
const hours = Math.floor(totalMinutes / 60);
const minutes = totalMinutes % 60;
console.log(hours, "h", minutes, "min"); // prints: 2 h 15 min
```

All four tools round to a *whole* number. To round to 2 decimals and still have a number (not a string like `toFixed` gives you), multiply by 100, round, then divide by 100:

```
const distanceKm = 3.14159;
console.log(Math.round(distanceKm * 100) / 100); // prints: 3.14
```

More `Math` helpers

`Math.abs()` gives the **absolute value**: how far a number is from zero, without the minus sign. It's handy for "how big is the difference?", whichever way round you subtract:

```
const morningTemp = 12;
const afternoonTemp = 19;
console.log("Temperature change:", Math.abs(morningTemp - afternoonTemp)); // prints:
    Temperature change: 7
```

`Math.max()` and `Math.min()` find the biggest and smallest of several numbers:

```
console.log(Math.max(72, 95, 88)); // prints: 95
console.log(Math.min(72, 95, 88)); // prints: 72
```

`Math.sqrt()` gives the **square root**: the number that, multiplied by itself, makes the one you started with. For example, a ladder leaning against a wall. The top is 4 meters up, and the bottom is 3 meters from the wall. How long is the ladder? (That's Pythagoras from school.)

```
const wallHeight = 4;
const distanceFromWall = 3;
const ladderLength = Math.sqrt(wallHeight ** 2 + distanceFromWall ** 2);
console.log("Ladder length:", ladderLength); // prints: Ladder length: 5
```

`Math.PI` is the number π . It's a value, not a tool, so it has no brackets. Here's the area of a pizza with a 15 cm radius:

```
const radius = 15;
const area = Math.PI * radius ** 2;
console.log(Math.PI); // prints: 3.141592653589793
console.log("Pizza area:", Math.round(area), "square cm"); // prints: Pizza area: 707 square cm
```

Random numbers

`Math.random()` gives you a random decimal from 0 up to (but never quite reaching) 1:

```
console.log(Math.random()); // prints something like: 0.7318234081530475
```

Your number will be different, and it changes every time you run the file.

A decimal between 0 and 1 isn't very useful on its own. Games need things like a dice roll, a whole number from 1 to 6. Here's the recipe:

```
const roll = Math.floor(Math.random() * 6) + 1;
console.log("You rolled:", roll); // prints something like: You rolled: 4
```

Here's what each step does:

Step	Code	Possible values
1. A random decimal	<code>Math.random()</code>	0 up to 0.999...
2. Stretch it	<code>Math.random() * 6</code>	0 up to 5.999...
3. Round down	<code>Math.floor(Math.random() * 6)</code>	0, 1, 2, 3, 4 or 5
4. Shift it up by 1	<code>Math.floor(Math.random() * 6) + 1</code>	1, 2, 3, 4, 5 or 6

The same recipe works for any range. For a random whole number from `min` to `max` (both included):

```
const min = 10;
const max = 20;
const prizeTicket = Math.floor(Math.random() * (max - min + 1)) + min;
console.log("Winning ticket:", prizeTicket); // prints something like: Winning ticket: 17
```

Why `max - min + 1`? Because from 10 to 20 there are 11 possible numbers, not 10. Count them on your fingers: 10, 11, 12, and so on up to 20.

Watch out: `Math.random()` is fine for games, but not for anything secret, like passwords or security codes. Its numbers aren't unpredictable enough for that. Browsers and Node have a safer tool for secrets, `crypto.getRandomValues()`.

NaN : Not a Number

You met `NaN` in chapter 03. It's what you get when a calculation can't give a real number:

```
console.log(Number("hello")); // prints: NaN
console.log(0 / 0); // prints: NaN
console.log(Math.sqrt(-1)); // prints: NaN
console.log(undefined + 1); // prints: NaN
```

That last one happens more than you'd think: a variable that never got a value, used in a calculation.

`NaN` is contagious. Any math with `NaN` gives `NaN`, so one bad value can ruin a whole shopping basket total:

```
const shirtPrice = 20;
const capPrice = Number("twelve"); // oops, not a number
console.log(shirtPrice + capPrice + 5); // prints: NaN
```

And `NaN` has a strange rule: it isn't equal to anything, not even to itself. So you can't check for it with `===`. Use `Number.isNaN()` instead:

```
const quantity = Number("two");
console.log(quantity === NaN); // prints: false
console.log(Number.isNaN(quantity)); // prints: true
```

You may also see an older `isNaN()` without the `Number.` in front. It converts the value first, which gives confusing results, so stick to `Number.isNaN()`.

(One more oddity: `typeof NaN` is `"number"`. "Not a Number" is a number. Yes, really.)

Infinity

Dividing by zero doesn't crash your program. It gives `Infinity`:

```
console.log(10 / 0); // prints: Infinity
console.log(-10 / 0); // prints: -Infinity
```

If you ever see `Infinity` in your output, look for a division by zero. For example, a bill split between 0 people.

Text to numbers: `Number()`, `parseInt()` and `parseFloat()`

`Number()` from chapter 03 is strict: the *whole* text must be a number, or you get `NaN`. That's a problem for text like `"42px"` (a size on a web page) or `"3.5kg"` (a weight on a product page).

`parseInt()` and `parseFloat()` are more relaxed. They read a number from the start of the text and stop at the first character that doesn't fit:

```
console.log(Number("42px")); // prints: NaN
console.log(parseInt("42px")); // prints: 42
console.log(parseFloat("3.5kg")); // prints: 3.5
```

The difference between them: `parseInt` reads only a whole number (an integer), while `parseFloat` keeps the decimals. ("Float" is programmer slang for a decimal number.)

Text	<code>Number()</code>	<code>parseInt()</code>	<code>parseFloat()</code>
<code>"42"</code>	42	42	42
<code>"3.99"</code>	3.99	3	3.99
<code>"42px"</code>	NaN	42	42
<code>"size 42"</code>	NaN	NaN	NaN

The last row shows that the number has to come *first*. If the text starts with letters, even `parseInt` gives up.

A good rule of thumb:

- Use `Number()` when the whole text should be a number, like a quantity typed into a form. If it's not a number, you *want* the `NaN`, so you can spot the problem.
- Use `parseInt()` or `parseFloat()` when the number comes with extra text after it, like `"42px"`.

You'll often see `parseInt(text, 10)` in other people's code. The `10` means "read it as a normal base-10 number" (the everyday counting system, with the digits 0 to 9). It's a safe habit, but for normal text like `"42px"` you get the same result without it.

Numbers to text: `String()` and `toString()`

Going the other way, there are two ways to turn a number into text:

```
const year = 2026;
console.log(String(year), typeof String(year)); // prints: 2026 string
console.log(year.toString(), typeof year.toString()); // prints: 2026 string
```

Both do the same job. `toString()` is a method (like `toFixed()`), so it goes after a dot.

Formatting numbers for people

Big numbers are hard to read: is `1234567` about a million or about ten million? `toLocaleString()` adds separators the way people expect them:

```
const population = 1234567;
console.log(population.toLocaleString("en-US")); // prints: 1,234,567
console.log(population.toLocaleString("de-DE")); // prints: 1.234.567
console.log(population.toLocaleString("en-IN")); // prints: 12,34,567
```

The text in the brackets is a **locale**: a code for a language and a country, like `"en-US"` (English, United States) or `"de-DE"` (German, Germany). Different countries group digits differently. Germany uses dots, and India groups the last three digits together, then the rest in pairs.

If you leave the locale out, JavaScript uses your computer's language settings, so the output depends on your machine. Passing a locale makes it the same everywhere.

For money, add some settings:

```
const price = 1234.5;
console.log(price.toLocaleString("en-US", { style: "currency", currency: "USD" })); // prints:
    $1,234.50
```

The part in curly braces `{ }` is a group of settings, each with a name and a value. It's called an **object**, and you'll learn all about objects in [chapter 11](#). For now, copy the pattern.

If you format lots of prices the same way, create a **formatter** once with `Intl.NumberFormat`, then reuse it. (`Intl` is JavaScript's built-in toolbox for international formats, and `new` creates a fresh formatter. You'll learn more about `new` in chapter 27.)

```
const dollars = new Intl.NumberFormat("en-US", { style: "currency", currency: "USD" });
console.log(dollars.format(4.5)); // prints: $4.50
console.log(dollars.format(1234.5)); // prints: $1,234.50

const pounds = new Intl.NumberFormat("en-GB", { style: "currency", currency: "GBP" });
console.log(pounds.format(1234.5)); // prints: £1,234.50

const rupees = new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" });
console.log(rupees.format(123456.5)); // prints: ₹1,23,456.50
```

The formatter adds the currency symbol and the separators, and shows the right number of decimals for that currency (two, for these three). It's like `toFixed()`, with extras.

Really big numbers: `MAX_SAFE_INTEGER` and `BigInt`

Numbers are exact up to 9,007,199,254,740,991 (about 9 quadrillion). JavaScript calls that limit `Number.MAX_SAFE_INTEGER`. Past it, whole numbers start to lose accuracy:

```
console.log(Number.MAX_SAFE_INTEGER); // prints: 9007199254740991
console.log(9007199254740991 + 2); // prints: 9007199254740992
```

The second line is wrong: it should end in 993!

For huge whole numbers, JavaScript has a separate type, **BigInt**. You make one by adding `n` to the end of a whole number:

```
console.log(9007199254740991n + 2n); // prints: 9007199254740993n
console.log(typeof 123n); // prints: bigint
```

You can't mix BigInts and normal numbers in the same calculation:

```
console.log(10n + 1);
// TypeError: Cannot mix BigInt and other types, use explicit conversions
```

You'll rarely need BigInt. It's for things like very long ID numbers or exact math on enormous values. For everyday numbers like prices, ages and scores, a normal `number` is plenty.

Common mistakes

1. Comparing decimals with `===`

```
const total = 0.1 + 0.2;
console.log(total === 0.3); // prints: false
```

The tiny storage error makes the two values slightly different. Work in whole numbers (like cents) instead: `10 + 20 === 30` is `true`.

2. Doing math with the result of `toFixed`

```
const subtotal = 19.5;
const shown = subtotal.toFixed(2);
console.log(shown + 5); // prints: 19.505
```

`toFixed` returns a string, so `+` joins instead of adding. Do the math first and format last: `(subtotal + 5).toFixed(2)` gives `"24.50"`.

3. Checking for NaN with ===

```
const age = Number("twenty");
console.log(age === NaN); // prints: false
```

NaN isn't equal to anything, not even itself. Use `Number.isNaN(age)` instead, which gives `true`.

4. Expecting `Math.round` to keep decimals

```
console.log(Math.round(4.567, 2)); // prints: 5
```

`Math.round` always rounds to a whole number, and it quietly ignores the `2`. For 2 decimals, use `toFixed(2)` to show the number, or `Math.round(4.567 * 100) / 100` to keep it a number.

5. Getting the random range wrong

```
const roll = Math.floor(Math.random() * 6);
```

This gives 0 to 5, not 1 to 6, because the `+ 1` is missing. And use `Math.floor`, not `Math.round`: with `Math.round`, the lowest and highest numbers come up only half as often as the others.

6. Calling a method straight on a number

```
console.log(42.toFixed(2));
// SyntaxError: Invalid or unexpected token
```

JavaScript thinks the dot after `42` is a decimal point. Put the number in a variable first, or wrap it in brackets: `(42).toFixed(2)`.

Quick recap

- One `number` type covers whole numbers and decimals. `Number.isInteger()` tells you which one you have.
- Some decimals can't be stored exactly, so `0.1 + 0.2` isn't exactly `0.3`. For money, work in whole cents.
- `toFixed(2)` shows exactly 2 decimals, but it returns a string. Do the math first, and format last.
- `Math.round`, `Math.floor`, `Math.ceil` and `Math.trunc` round in different ways. `Math.abs`, `Math.max`, `Math.min`, `Math.sqrt` and `Math.PI` cover the rest of everyday math.
- A random whole number from `min` to `max`: `Math.floor(Math.random() * (max - min + 1)) + min`.
- `NaN` means "not a number". Check for it with `Number.isNaN()`. Dividing by zero gives `Infinity`.
- `parseInt()` and `parseFloat()` read numbers from text like `"42px"`. Show numbers to people with `toLocaleString()` or `Intl.NumberFormat`.

Next: try the [exercises](#), then move on to [06 Strings](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Run each one with `node ex1.js` from a terminal opened in this folder.
- Let JavaScript do every calculation, as always. No typing answers yourself!
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Party planner

You're organizing a birthday party for 47 guests. Each table seats 6 people. Cupcakes come in boxes of 10, and every guest gets one cupcake.

Expected output:

```
Tables needed: 8
Full tables: 7
Empty seats: 1
Cupcake boxes: 5
Spare cupcakes: 3
```

Rule: use `Math.ceil` and `Math.floor` . Start from three variables: the guests, the seats per table and the cupcakes per box.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): The corner shop till

The till at a corner shop works in cents. A customer buys a magazine (499 cents), a phone charger (1250 cents) and a chocolate bar (99 cents). Sales tax is 8%.

Expected output:

```
Subtotal: $18.48
Tax: $1.48
Total: $19.96
```

Rules: keep all the math in whole cents. Round the tax to a whole number of cents. Only turn cents into dollars when you print.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Parcel shipping calculator

You're building a shipping calculator for an online shop. The parcel's details come from the product page as text. Start your file with these lines:

```
// Read from a product page
const weightText = "2.75kg";
const lengthText = "40cm";
const widthText = "30cm";
const heightText = "25cm";
const messyText = "about 3kg";
```

The courier's rules:

- They charge for whichever is heavier: the actual weight, or the **size weight**.
- Size weight (in kg) is $\text{length} \times \text{width} \times \text{height (in cm)} \div 5000$. It stops big, light boxes, like a box of pillows, from being shipped for almost nothing.
- The price is 3.50 dollars per kg.

Expected output:

```
Actual weight: 2.75 kg
Size weight: 6 kg
You pay for: 6 kg
Shipping: $21.00
Messy text is a number: false
```

Rules: read every number out of the text with `parseInt` or `parseFloat` (don't type them again). Use `Math.max` to pick the weight you pay for. The last line checks `messyText` with `Number.isNaN`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Board game night

It's board game night, and nobody can find the dice. Write a program that rolls two dice, adds them up, checks for a double, and picks which of the 4 players goes first.

Example output (yours will be different every time you run it):

```
Die 1: 4
Die 2: 4
Total: 8
Double: true
Player 3 goes first
```

Rules:

- Each die is a whole number from 1 to 6. The first player is a whole number from 1 to 4.
- Work out "Double" by comparing the two dice. Don't type `true` or `false`.

Run it at least 10 times. The dice should always land between 1 and 6, and you should see a double now and then (about 1 time in 6).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Split the bill

Three friends had dinner, and you're writing the bill-splitting app. The waiter types everything into the app, so it all arrives as text. Start your file with these lines:

```
// Typed into the app by the waiter (everything arrives as text)
const billText = "87.40";
const tipPercentText = "15";
const peopleText = "3";
```

Expected output:

```
Bill: $87.40
Tip (15%): $13.11
Total: $100.51
Each person pays: $33.51
Extra for the tip jar: $0.02
Splits evenly: false
```

Rules:

- Convert the text first, then work in whole cents from there on.
- Round the tip to the nearest cent. Round each person's share **up** to the next cent, so the bill is always covered.
- "Extra for the tip jar" is what the friends pay all together, minus the total.
- Format every amount with one `Intl.NumberFormat` currency formatter.
- Don't type any of the amounts yourself. Even the `15` in `Tip (15%)` should come from the starting variables.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Look at how you've been building text since chapter 03: `"Your order comes to " + total + " dollars."`. All those quotes and plus signs are fiddly, and one missing space breaks the output.

[Chapter 06](#) shows you a much neater way to put values inside text, plus lots of tools for working with strings: changing capital letters, trimming spaces, searching, and more.

06 Strings

What is it?

A **string** is a piece of text: a name, a password, a chat message, a whole paragraph. You've used strings since [chapter 01](#) (text inside quotes), and you met them as a data type in [chapter 03](#).

This chapter is about *working* with text: building it, measuring it, cleaning it up, searching it, and cutting it into pieces.

Why does it matter?

Almost everything people type or read in an app is a string. Names, emails, search boxes, product titles, receipts.

And people type messy things. They add spaces by accident. They mix up capital letters. To a human, "Sam@Gmail.COM " and "sam@gmail.com" are the same email. To JavaScript, they're two completely different strings.

With the tools in this chapter, you can:

- Build messages from variables: Hi Sam, your order #1042 has shipped!
- Clean up what people type (extra spaces, random capitals).
- Check text: does this email contain an @ ? Does this file name end in .pdf ?
- Line text up neatly, like the rows on a receipt.

Real-world example

Think of a string as a strip of tape from a **label maker**.

Label maker	String
Letters printed in order on a strip	Characters in order: "COFFEE"
Measuring how long the strip is	.length
Pointing at the 1st, 2nd, 3rd letter	Positions [0] , [1] , [2] (counting starts at 0)
Once it's printed, you can't change a letter	A string can't be changed
Want a different label? Print a new strip	String tools always give you a <i>new</i> string

Keep that last row in mind. It explains the most common string mistake, and you'll see it again at the end of this chapter.

How it works

Three kinds of quotes

You can write a string with double quotes, single quotes, or backticks (the ``` key, usually just below `Esc`):

```
const city = "Kathmandu";
const food = 'momo';
const drink = `lassi`;

console.log(city, food, drink); // prints: Kathmandu momo Lassi
```

All three make a normal string. The quotes aren't part of the text. They only show where it starts and ends.

Why have more than one kind? So you can put quotes *inside* a string:

```
console.log("It's raining"); // prints: It's raining
console.log('She said "hello"'); // prints: She said "hello"
```

In this course we use double quotes for plain text, and backticks when we put values inside the text. That's what backticks are really for.

Template literals: putting values inside text

Remember joining strings with `+` from [chapter 03](#)? It works, but it gets messy fast:

```
const guest = "Aisha";
const nights = 3;

console.log("Welcome, " + guest + "! You're staying " + nights + " nights.");
```

So many quotes, spaces, and plus signs. It's easy to forget a space.

A **template literal** is a string written with backticks. Inside it, `${ }` is a slot where you can drop in any value:

```
console.log(`Welcome, ${guest}! You're staying ${nights} nights.`);
// prints: Welcome, Aisha! You're staying 3 nights.
```

It reads almost like the finished sentence. Think of `${ }` as a blank on a form that JavaScript fills in for you.

You can put any calculation inside the slot, not only a variable:

```
const pricePerNight = 85;
console.log(`Total: ${pricePerNight * nights}`); // prints: Total: $255
```

The first `$` is a normal dollar sign. The `${` right after it opens the slot.

Slots are also a good place to tidy numbers. Remember the decimal surprises from [chapter 05](#)?

```
const price = 1.1;
const quantity = 3;

console.log(`Total: ${price * quantity}`); // prints: Total: $3.3000000000000003
console.log(`Total: ${price * quantity.toFixed(2)}`); // prints: Total: $3.30
```

Multi-line strings

Backticks can also span several lines. The line breaks become part of the string:

```
const poem = `Roses are red,
Violets are blue,
JavaScript is fun,
And so are you.`;

console.log(poem);
```

You'll see:

```
Roses are red,
Violets are blue,
JavaScript is fun,
And so are you.
```

Double and single quotes can't do this. Press `Enter` in the middle of a `"..."` string and you get `SyntaxError: Invalid or unexpected token`.

Escape characters

What if you need a double quote *inside* a double-quoted string? Or a new line without using backticks?

You use an **escape character**: a backslash `\` that gives the next character a special meaning.

You type	You get
<code>\n</code>	a new line
<code>\t</code>	a tab (a wide space)
<code>\"</code>	a <code>"</code> inside a double-quoted string
<code>\'</code>	a <code>'</code> inside a single-quoted string
<code>\\</code>	one real backslash

```
console.log("Shopping list:\n- milk\n- eggs");
```

You'll see:

```
Shopping list:
- milk
- eggs
```

```
console.log("She said \"hello\"");           // prints: She said "hello"
console.log("C:\\Users\\Sam\\notes.txt");    // prints: C:\Users\Sam\notes.txt
```

That last one matters on Windows. File paths are full of backslashes, and each one must be written as `\\`.

Length and positions

Every string knows how many characters it has. A **character** is one letter, digit, space, or symbol.

```
const word = "pizza";

console.log(word.length);           // prints: 5
console.log("New York".length);    // prints: 8 (the space counts too)
console.log("").length;             // prints: 0 (an empty string)
```

`length` is a **property**: a piece of information the string carries around. You read it with a dot and no parentheses.

Each character has a position number, called its **index**. Counting starts at 0, not 1:

```
p  i  z  z  a
[0] [1] [2] [3] [4]
```


Put the index in square brackets to read one character:

```
console.log(word[0]); // prints: p
console.log(word[4]); // prints: a
console.log(word[5]); // prints: undefined (there's nothing at position 5)
```

Because counting starts at 0, the last character is always at `length - 1`. For "pizza", that's $5 - 1 = 4$:

```
console.log(word[word.length - 1]); // prints: a
```

Your first string methods

Remember **methods** from [chapter 05](#)? A method is a tool that belongs to a value. You write a dot, the method's name, and parentheses, like `price.toFixed(2)`. Strings come with lots of useful methods of their own, and the rest of this chapter is mostly about them.

The first one is `at()`. It works like square brackets, with a bonus: negative numbers count from the end. `-1` is the last character, `-2` the one before it:

```
console.log(word.at(0)); // prints: p
console.log(word.at(-1)); // prints: a
console.log(word.at(-2)); // prints: z
```

`word.at(-1)` is much easier to read than `word[word.length - 1]`.

Two more methods change the case of the letters:

```
console.log("stop!".toUpperCase()); // prints: STOP!
console.log("HeLLo WoRLD".toLowerCase()); // prints: hello world
```

Strings can't be changed

Back to the label maker. Once a label is printed, you can't swap one of its letters. Strings are the same: they're **immutable**, which means "can't be changed".

```
let pet = "cat";
pet[0] = "b"; // try to change the first letter

console.log(pet); // prints: cat (nothing changed, and no error either)
```

You *can* put a brand-new string in the variable, though. That's like printing a new label:

```
pet = "bat";  
console.log(pet); // prints: bat
```

This is why every string method gives you back a **new** string and leaves the original alone:

```
const username = "sam";  
const loudName = username.toUpperCase();  
  
console.log(loudName); // prints: SAM  
console.log(username); // prints: sam (the original is untouched)
```

If you want to keep the result, save it in a variable.

Cleaning up spaces with `trim`

People often type extra spaces by accident, especially before or after their email. `trim()` removes the spaces from both ends, but never from the middle:

```
const typedEmail = "  sam@example.com  ";  
  
console.log(`[${typedEmail}]`); // prints: [  sam@example.com  ]  
console.log(`[${typedEmail.trim()}]`); // prints: [sam@example.com]
```

The square brackets are only there so you can see the spaces. It's a handy trick when you're checking text.

If you only want to clean one side, there's `trimStart()` and `trimEnd()`.

Searching inside a string

These three methods answer yes/no questions, so they give you `true` or `false`:

```
const email = "sam@example.com";  
const fileName = "holiday-photo.jpg";  
  
console.log(email.includes("@")); // prints: true  
console.log(email.includes("gmail")); // prints: false  
console.log(fileName.startsWith("holiday")); // prints: true  
console.log(fileName.endsWith(".jpg")); // prints: true
```

- `includes(text)` : is this text anywhere inside?
- `startsWith(text)` : does it begin with this?
- `endsWith(text)` : does it finish with this?

If you want to know *where* something is, use `indexOf()`. It gives you the index of the first match, or `-1` if there's no match at all:

```
console.log(email.indexOf("@")); // prints: 3
console.log(email.indexOf("e")); // prints: 4 (only the first "e" counts)
console.log(email.indexOf("#")); // prints: -1 (not found)
```

Watch out: all of these are case-sensitive. `"Hello".includes("hello")` is `false`, because `H` and `h` are different characters.

Cutting out a piece with `slice`

`slice(start, end)` copies part of a string. It starts at the `start` index and stops *just before* the `end` index. The character at `end` isn't included.

```
2  0  2  6  -  0  9  -  2  4
[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
```

```
const date = "2026-09-24";

console.log(date.slice(0, 4)); // prints: 2026 (positions 0, 1, 2, 3)
console.log(date.slice(5, 7)); // prints: 09
console.log(date.slice(8));    // prints: 24 (no end means "to the end")
console.log(date.slice(-2));   // prints: 24 (negative counts from the end)
```

A nice way to remember it: `slice(0, 4)` gives you $4 - 0 = 4$ characters.

`slice` gets really useful when you combine it with `indexOf`. You don't always know where the `@` in an email will be, but you can ask:

```
const atPosition = email.indexOf("@"); // 3

console.log(email.slice(0, atPosition)); // prints: sam
console.log(email.slice(atPosition + 1)); // prints: example.com
```

Everything before the `@` is the name. Everything after it is the domain. This works for any email, however long.

Replacing text

`replace(old, new)` swaps the **first** match. `replaceAll(old, new)` swaps **every** match:

```
const phone = "555-123-4567";

console.log(phone.replace("-", "")); // prints: 555123-4567 (only the first dash)
console.log(phone.replaceAll("-", "")); // prints: 5551234567
```

These methods search for exact text. For smarter searches, like "any digit", you'll use patterns in [chapter 37](#).

Splitting text into pieces

`split(separator)` cuts a string wherever it finds the separator:

```
const groceries = "milk,eggs,bread";

console.log(groceries.split(",")); // prints: [ 'milk', 'eggs', 'bread' ]
console.log("The quick brown fox".split(" ")); // prints: [ 'The', 'quick', 'brown', 'fox' ]
console.log("hello".split("")); // prints: [ 'h', 'e', 'l', 'l', 'o' ]
```

The result isn't a string. The square brackets show it's an **array**: a list of values in order. Node shows the text inside it with single quotes. (The browser console shows it a little differently, but it's the same list.)

You'll learn to work with arrays in [chapter 10](#). For now, just know that `split` is how you turn text into a list.

Repeating and padding

`repeat(count)` repeats a string that many times. It's great for lines and simple ratings:

```
console.log("=".repeat(20)); // prints: =====
console.log("ha".repeat(3)); // prints: hahaha

const rating = 4;
console.log(`Rating: ${".".repeat(rating)}`); // prints: Rating: ****
```

`padStart(length, filler)` adds filler characters to the *start* until the string is `length` characters long. `padEnd` does the same at the end. If you leave out the filler, it uses spaces.

```
console.log("42".padStart(5, "0")); // prints: 00042 (an order number)
console.log("7".padStart(2, "0")); // prints: 07 (minutes on a clock)
console.log("12".padStart(2, "0")); // prints: 12 (already long enough, so nothing changes)
```

Padding is how you line things up in columns, like on a receipt. Every name is padded to the same width, and every price too:

```
console.log("Pizza".padEnd(12) + "$9.50".padStart(8));
console.log("Garlic bread".padEnd(12) + "$4.25".padStart(8));
```

You'll see:

```
Pizza          $9.50
Garlic bread   $4.25
```

Watch out: padding is a string method, so it doesn't work on numbers. If `minutes` is the number `7`, `minutes.padStart(2, "0")` gives `TypeError: minutes.padStart is not a function`. Turn it into a string first: `String(minutes).padStart(2, "0")`.

Chaining methods

Every string method gives you a new string. So you can call another method straight on the result. This is called **method chaining**:

```
const typedEmail = " Sam@example.COM ";
const cleanEmail = typedEmail.trim().toLowerCase();

console.log(cleanEmail); // prints: sam@example.com
```

Read it from left to right, like a car wash with several stations:

1. `typedEmail` is the dirty car: `" Sam@example.COM "`
2. `.trim()` washes off the spaces: `"Sam@example.COM"`
3. `.toLowerCase()` polishes the capitals: `"sam@example.com"`

Here's a chain that turns a full name into a username:

```
const fullName = " Priya Sharma ";

const username = fullName
  .trim()           // "Priya Sharma"
  .toLowerCase()   // "priya sharma"
  .replaceAll(" ", "."); // "priya.sharma"

console.log(username); // prints: priya.sharma
```

When a chain gets long, you can put each step on its own line, like above. It's exactly the same code, just easier to read.

Comparing strings

`===` checks whether two strings are exactly the same, character for character. That includes capital letters:

```
console.log("apple" === "apple"); // prints: true
console.log("Apple" === "apple"); // prints: false
```

So when you compare something a person typed, clean it up first. Imagine a quiz where the answer is "paris":

```
const typedAnswer = "PARIS";

console.log(typedAnswer === "paris"); // prints: false
console.log(typedAnswer.toLowerCase() === "paris"); // prints: true
```

You can also use `<` and `>` to check alphabetical order. But there are two surprises:

```
console.log("apple" < "banana"); // prints: true (a comes before b)
console.log("Zoe" < "adam"); // prints: true (surprise!)
console.log("10" < "9"); // prints: true (surprise!)
```

JavaScript compares strings one character at a time, using a code number for each character. In that code, **all capital letters come before all lowercase letters**. So `"Z"` comes before `"a"`.

And `"10"` vs `"9"` is text, not numbers. The first characters are `"1"` and `"9"`, and `"1"` comes first, so JavaScript stops there. If you mean numbers, convert them first with `Number()` from [chapter 03](#).

Common mistakes

1. Forgetting to save the result

```
const city = " london ";
city.trim();
city.toUpperCase();

console.log(`[${city}]`); // prints: [ london ]
```

Nothing changed! Strings can't be changed, so `trim()` and `toUpperCase()` made new strings... and then threw them away. Save the result: `const cleanCity = city.trim().toUpperCase();`

2. Using `${}` inside normal quotes

```
const guest = "Aisha";
console.log("Welcome, ${guest}!"); // prints: Welcome, ${guest}!
```

The slots only work inside backticks. In double or single quotes, `${guest}` is just ordinary text. Fix:

```
console.log(`Welcome, ${guest}!`);
```

3. Mixing up properties and methods

```
console.log(city.toUpperCase); // prints: [Function: toUpperCase]

console.log(city.length());
// TypeError: city.length is not a function
```

`toUpperCase` is a method, so it needs parentheses. Without the `()`, JavaScript shows you the method itself instead of running it. `length` is a property, so it has no parentheses. Fix: `city.toUpperCase()` and `city.length`.

4. Counting from 1 instead of 0

```
const word = "pizza";
console.log(word[1]); // prints: i (not p!)
```

The first character is at index 0. And remember that `slice(start, end)` stops *before* `end`: `word.slice(0, 2)` is "pi", not "piz".

5. Forgetting to double the backslashes

```
console.log("C:\\Users\\Sam\\notes.txt");
```

You'll see:

```
C:UsersSam
otes.txt
```

JavaScript read `\n` in `\notes` as "new line", and dropped the other backslashes. Every real backslash must be written as `\\`: `"C:\\Users\\Sam\\notes.txt"`.

Quick recap

- Strings can use "double", 'single', or ``backtick`` quotes. Backticks make template literals, where `${ }` drops values into the text. They can also span several lines.
- Escape characters start with a backslash: `\n` (new line), `\"` (a quote), `\\` (a backslash).

- `.length` counts characters. Indexes start at 0: `text[0]` is the first character, and `text.at(-1)` is the last.
 - Strings can't be changed. Every method returns a **new** string, so save the result in a variable.
 - Key methods: `toUpperCase`, `toLowerCase`, `trim`, `includes`, `startsWith`, `endsWith`, `indexOf`, `slice`, `replace`, `replaceAll`, `split`, `repeat`, `padStart`, `padEnd`.
 - Chain methods to clean up text in one go: `input.trim().toLowerCase()`.
 - String comparisons are case-sensitive, and `<` / `>` compare text character by character.
-

Next: try the [exercises](#), then move on to [07 Conditionals](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Name badge

You're printing name badges for a conference. Start your file with these variables:

```
const firstName = "Priya";
const lastName = "Sharma";
const company = "Blue Fox Studio";
```

Print this badge:

```
=====
HELLO, MY NAME IS
PRIYA SHARMA
Blue Fox Studio
=====
Initials: P.S.
Letters in name: 11
```

Rule: don't type the name, the initials, or the number 11 yourself. Build them from the variables. And don't type 30 `=` signs by hand either.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Clean up a sign-up form

Someone typed their email into your sign-up form, with some extra spaces and random capitals:

```
const typedEmail = "  Sam.Lee@Example.COM  ";
```

Clean it up (no spaces around it, all lowercase), then print a report about it:

```
Before: [  Sam.Lee@Example.COM  ]
After: [sam.lee@example.com]
Length before: 24
Length after: 19
Has an @: true
Ends with .com: true
Gmail address: false
```

The square brackets make the spaces visible, like in the notes. Do the last three checks on the *cleaned* email.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Library usernames

The city library creates a username and an email for every new member. Start with:

```
const fullName = "  Grace Hopper  ";
const memberNumber = 42;
```

The username is the first letter of the first name, then the last name, then the member number padded to 4 digits. Everything is lowercase.

Expected output:

```
Full name: Grace Hopper
First name: Grace
Last name: Hopper
Username: ghopper0042
Email: grace.hopper@citylibrary.org
```

Rule: your code must work for *any* two-word name. Change the variables to `" Alan Turing "` and `7`, run it again, and you should see:

```
Full name: Alan Turing
First name: Alan
Last name: Turing
Username: aturing0007
Email: alan.turing@citylibrary.org
```

So don't count the positions yourself. Let JavaScript find them.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Restaurant receipt

Luigi's Trattoria wants its receipts to line up neatly. Here's table 4's order:

```
const item1 = "Margherita pizza";
const price1 = 11.5;
const quantity1 = 2;

const item2 = "Lemonade";
const price2 = 3.25;
const quantity2 = 3;

const item3 = "Tiramisu";
const price3 = 6.75;
const quantity3 = 1;
```

Print this receipt:

```
Luigi's Trattoria
=====
2 x Margherita pizza $23.00
3 x Lemonade         $9.75
1 x Tiramisu          $6.75
-----
Total                 $39.50
=====
```

Rules:

- Every line under the name is exactly 28 characters wide.

- Don't type the prices, the total, or any of the spaces, = signs or - signs yourself.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: fill the gaps with dots instead of spaces, so the Lemonade line looks like `3 x Lemonade.....$9.75`.

Exercise 5 (Challenge): Parcel tracking codes

A delivery company prints a tracking code on every parcel. It always has the same shape: a 2-letter country, a 4-digit year, a 3-letter city, and a 5-digit parcel number, joined with dashes. But people type them into the website in all sorts of ways:

```
const rawCode = " np-2026-ktm-00451 ";
```

Write a program that cleans the code and prints what's inside it:

```
Clean code: NP-2026-KTM-00451
Parts: [ 'NP', '2026', 'KTM', '00451' ]
Country: NP
Year: 2026
City: KTM
Parcel number: 451
Looks valid: true
Next parcel: NP-2026-KTM-00452
```

What each line means:

- **Clean code:** no spaces around it, all capitals.
- **Parts:** the clean code split at every dash.
- **Parcel number:** the last 5 characters turned into a real number, so the zeros at the front disappear.
- **Looks valid:** `true` only if the clean code is exactly 17 characters long **and** starts with `NP-` (this company only ships inside Nepal).
- **Next parcel:** the code of the parcel after this one. The number goes up by 1 and is padded back to 5 digits.

Test it with `"in-2025-del-09999 "` too. You should see:

```
Clean code: IN-2025-DEL-09999
Parts: [ 'IN', '2025', 'DEL', '09999' ]
Country: IN
Year: 2025
City: DEL
Parcel number: 9999
Looks valid: false
Next parcel: IN-2025-DEL-10000
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 5, a bad code prints `Looks valid: false` ... and then your program carries on printing the rest anyway. A real website would stop and show an error message instead.

To do that, your code needs to make decisions: *if* the code is valid, do this, *otherwise* do that. That's [chapter 07](#).

07 Conditionals

What is it?

A **conditional** is code that only runs when something is true. It's how a program makes decisions: *if* this is true, do that, *otherwise* do something else.

Why does it matter?

Until now, your programs ran every single line, top to bottom, no matter what. That's fine for a calculator. It's not fine for most real apps.

Real programs react to what's going on:

- The password is right? Let them in. Wrong? Show an error.
- The customer is a child? Charge the child's price.
- The order is over \$50? Shipping is free.

In [chapter 04](#), you could print `true` or `false`. Now you'll *act* on them.

Real-world example

You already make decisions like this every morning:

What you think	What it looks like in JavaScript
"If it's raining, I'll take an umbrella."	<code>if (isRaining) { ... }</code>
" Otherwise , I'll wear sunglasses."	<code>else { ... }</code>
"If it's raining, umbrella. Otherwise, if it's cold, a coat. Otherwise, a T-shirt."	<code>if ... else if ... else</code>

You look at the weather, and you pick *one* option. Never two, never none. Conditionals work exactly the same way.

How it works

if : run code only when something is true

```
const temperature = 32;

if (temperature > 30) {
  console.log("It's hot! Drink some water.");
}

console.log("Have a nice day.");
```

You'll see:

```
It's hot! Drink some water.
Have a nice day.
```

Here's how to read it:

- `temperature > 30` is the **condition**: a question with a `true` or `false` answer. It goes in parentheses.
- The curly braces `{ }` hold a **block**: a group of statements that belong together. The block runs only when the condition is `true`.

Change `temperature` to `20` and run it again. Now the condition is `false`, so JavaScript skips the block and you only see `Have a nice day.`

The lines inside the block are indented by 2 spaces. JavaScript doesn't care, but people do: it shows at a glance which lines belong to the `if`.

else : the plan B

`else` gives JavaScript something to do when the condition is `false`:

```
const balance = 40;
const price = 55;

if (balance >= price) {
  console.log("Payment approved.");
} else {
  console.log("Sorry, not enough money.");
}

// prints: Sorry, not enough money.
```

Exactly one of the two blocks runs. Never both, and never neither.

`else if` : more than two choices

A cinema has four ticket prices: free for babies under 3, \$8 for children under 13, \$9 for seniors (65 and over), and \$14 for everyone else.

```
const age = 70;

if (age < 3) {
  console.log("Ticket: free");
} else if (age < 13) {
  console.log("Ticket: $8 (child)");
} else if (age >= 65) {
  console.log("Ticket: $9 (senior)");
} else {
  console.log("Ticket: $14 (adult)");
}
// prints: Ticket: $9 (senior)
```

JavaScript checks the conditions from top to bottom and **stops at the first one that's true**. Only that block runs. If none are true, the `else` block runs.

Notice that the second check is only `age < 13`, not "3 or older and under 13". If JavaScript gets that far, it already knows `age < 3` was false.

Often you want to *save* the decision instead of printing it straight away. Create the variable with `let` before the `if`, then fill it in:

```
const age = 8;
let price;

if (age < 3) {
  price = 0;
} else if (age < 13) {
  price = 8;
} else if (age >= 65) {
  price = 9;
} else {
  price = 14;
}

console.log(`Ticket price: ${price}`); // prints: Ticket price: $8
```

Why create `price` up top? A variable created *inside* `{ }` only exists inside those braces. If you wrote `const price = 8;` inside the block, the last line would give `ReferenceError: price is not defined`. You'll learn the full story in [chapter 14](#).

Combining conditions

Remember `&&` (and), `||` (or), and `!` (not) from [chapter 04](#)? They let one `if` check several things at once.

An online shop gives free shipping on orders of \$50 or more, **or** to anyone with a membership:

```
const orderTotal = 45;
const isMember = true;

if (orderTotal >= 50 || isMember) {
  console.log("Shipping: FREE");
} else {
  console.log("Shipping: $4.99");
}
// prints: Shipping: FREE
```

The order is under \$50, but the customer is a member, so `||` is happy with one out of two.

A concert only lets you in if you're 18 or older **and** you have a ticket:

```
const age = 20;
const hasTicket = true;

if (age >= 18 && hasTicket) {
  console.log("Welcome to the concert!");
}
// prints: Welcome to the concert!
```

Nested ifs (and keeping them shallow)

You can put an `if` inside another `if`. This is called **nesting**. Here's the check for an admin page on a website:


```

const isLoggedIn = true;
const isAdmin = false;

if (isLoggedIn) {
  if (isAdmin) {
    console.log("Welcome to the admin panel.");
  } else {
    console.log("Sorry, admins only.");
  }
} else {
  console.log("Please log in first.");
}
// prints: Sorry, admins only.

```

It works, but every extra level pushes the code further to the right and makes it harder to follow. Real programs can end up five levels deep, and nobody enjoys reading that.

Here's the same logic, flat. Remember that `!` flips a boolean, so `!isLoggedIn` reads as "not logged in":

```

if (!isLoggedIn) {
  console.log("Please log in first.");
} else if (!isAdmin) {
  console.log("Sorry, admins only.");
} else {
  console.log("Welcome to the admin panel.");
}
// prints: Sorry, admins only.

```

The trick: **deal with the problem cases first**, then the happy case at the end. Each check is one line, and you can read it from top to bottom like a checklist. In [chapter 09](#) you'll learn an even cleaner version of this trick.

Truthy and falsy

So far, every condition has been a real boolean, like `age >= 18`. But you can put *any* value in an `if`. JavaScript quietly turns it into `true` or `false` first.

Values that turn into `false` are called **falsy**. There are only eight of them:

Falsy value	What it is
<code>false</code>	the boolean itself
<code>0</code> and <code>-0</code>	zero (yes, JavaScript has a negative zero)
<code>0n</code>	zero as a BigInt (chapter 05)

Falsy value	What it is
<code>""</code>	an empty string
<code>null</code>	"intentionally empty"
<code>undefined</code>	"no value yet"
<code>NaN</code>	"not a number"

Everything else is truthy, which means it turns into `true`.

You can check any value with `Boolean()`. It converts a value to `true` or `false`, the same way `Number()` and `String()` convert to numbers and strings:

```
console.log(Boolean(0));      // prints: false
console.log(Boolean(""));    // prints: false
console.log(Boolean(42));     // prints: true
console.log(Boolean("hello")); // prints: true
console.log(Boolean("0"));    // prints: true (surprise!)
console.log(Boolean(" "));    // prints: true (surprise!)
```

`"0"` and `" "` aren't empty. They each contain one character, so they're truthy. Only the completely empty string `""` is falsy.

Truthy and falsy make some checks short and sweet. Here's a sign-up form checking that the name box isn't empty:

```
const typedName = "";

if (typedName) {
  console.log(`Hello, ${typedName}!`);
} else {
  console.log("Please enter your name.");
}
// prints: Please enter your name.
```

`if (typedName)` means "if there's something in `typedName`". Put `"Leo"` in it, and you'll see `Hello, Leo!` instead.

The ternary operator: a short `if / else`

Sometimes you just want to pick one of two *values*. Writing a full `if / else` for that feels long:

```
const age = 16;
let label;

if (age >= 18) {
  label = "adult";
} else {
  label = "minor";
}
```

The **ternary operator** does the same thing in one line:

```
const age = 16;
const label = age >= 18 ? "adult" : "minor";

console.log(label); // prints: minor
```

Read it as a question: "Is `age >= 18` ? If yes, `"adult"` . If no, `"minor"` ."

```
condition ? valueIfTrue : valueIfFalse
```

It shines inside template literals. Online shops use this trick all the time so they never say "1 items":

```
const itemCount = 1;
console.log(`You have ${itemCount} ${itemCount === 1 ? "item" : "items"} in your cart.`);
// prints: You have 1 item in your cart.
```

Tip: use the ternary to choose between two values. When you need to *do* things (several lines of code), use a normal `if / else` . And avoid putting a ternary inside another ternary. It gets hard to read fast.

switch : one value, many possible matches

Imagine a self-driving car reading a traffic light. You *could* write `if (light === "green") ... else if (light === "yellow") ...` and so on. When you compare one value against a list of exact options, `switch` is often tidier:

```
const light = "yellow";

switch (light) {
  case "green":
    console.log("Go!");
    break;
  case "yellow":
    console.log("Slow down.");
    break;
  case "red":
    console.log("Stop!");
    break;
  default:
    console.log("Light is broken. Drive carefully.");
}
// prints: Slow down.
```

- `switch (light)` says which value to look at.
- Each `case` is one possible match. JavaScript compares with `===`, so the string `"1"` won't match `case 1:`.
- `break` means "I'm done, jump out of the `switch`".
- `default` runs when no case matches, like a final `else`. It's optional, but it's a good habit.

Watch out: forget a `break`, and JavaScript doesn't stop at the end of the matching case. It **falls through** and runs every case below it too, without checking them:

```
const light = "yellow";

switch (light) {
  case "green":
    console.log("Go!");
  case "yellow":
    console.log("Slow down.");
  case "red":
    console.log("Stop!");
  default:
    console.log("Light is broken. Drive carefully.");
}
```

You'll see:

```
Slow down.
Stop!
Light is broken. Drive carefully.
```

Falling through is useful in one situation: when several cases should do the same thing. Stack them on top of each other. In this game, both the `W` key and the up arrow move the player up:

```
const key = "ArrowUp";

switch (key) {
  case "w":
  case "ArrowUp":
    console.log("Move up");
    break;
  case "s":
  case "ArrowDown":
    console.log("Move down");
    break;
  default:
    console.log("That key does nothing.");
}
// prints: Move up
```

switch or if? Use `switch` when you compare *one value* against several exact options. For ranges (like `age < 13`) or mixed conditions, stick with `if / else if`.

|| for default values

In chapter 04, `&&` and `||` always gave you `true` or `false`. Here's the full truth: they actually give you back **one of the two values** you gave them.

`a || b` means: "if `a` is truthy, give me `a`. Otherwise, give me `b`."

```
console.log("Sam" || "Guest"); // prints: Sam
console.log("" || "Guest");    // prints: Guest
```

That makes `||` perfect for **default values**: a backup value to use when the real one is missing.

```
const typedName = "";
const displayName = typedName || "Guest";

console.log(`Hello, ${displayName}!`); // prints: Hello, Guest!
```

&& as a guard

`a && b` means: "if `a` is falsy, give me `a` and stop. Otherwise, give me `b`."

```
console.log(true && "Welcome!"); // prints: Welcome!
console.log(false && "Welcome!"); // prints: false
```

The "and stop" part is the useful bit. When the left side is falsy, JavaScript doesn't even *look* at the right side. This is called **short-circuiting**.

Why does that help? Calling a method on `null` crashes your program:

```
const nickname = null;
nickname.toUpperCase();
// TypeError: Cannot read properties of null (reading 'toUpperCase')
```

Put `nickname &&` in front, and it becomes a **guard**, like a security guard who only lets you through if you have a pass:

```
const nickname = null;
console.log(nickname && nickname.toUpperCase()); // prints: null (no crash)
```

`nickname` is falsy, so JavaScript stops right there and never calls `toUpperCase()`. If `nickname` were `"ace"`, you'd get `ACE`. (In [chapter 11](#) you'll meet `?.`, a shorter way to write guards like this.)

?? : defaults that keep 0 and ""

`||` has one weakness. It treats *every* falsy value as "missing", including `0` and `""`. But sometimes `0` is exactly what the user chose.

A game saves your volume setting. You muted it, so it's `0`:

```
const savedVolume = 0;

console.log(savedVolume || 50); // prints: 50 (oops! you wanted 0)
console.log(savedVolume ?? 50); // prints: 0
```

`??` is called **nullish coalescing**, which is a fancy name for a simple idea: it only uses the default when the value is `null` or `undefined` (together called "nullish"). Every other value, even `0` and `""`, is kept.

Value	value "default"	value ?? "default"
<code>null</code>	"default"	"default"
<code>undefined</code>	"default"	"default"

Value	value "default"	value ?? "default"
0	"default"	0
""	"default"	""
false	"default"	false
"Sam"	"Sam"	"Sam"

Which one should I use? Ask yourself: "Is 0 or an empty string a real, valid answer here?"

- Volume, price, score, number of items: 0 is valid, so use ?? .
- A display name: an empty name is useless, so use || .

Common mistakes

1. Using = instead of ===

```
let score = 40;

if (score = 100) {
  console.log("Perfect score!");
}
// prints: Perfect score!

console.log(score); // prints: 100
```

You met this one in [chapter 04](#), and inside an if it's even sneakier. A single = doesn't compare. It puts 100 into score, and 100 is truthy, so the block always runs. Your score got changed too! Fix: if (score === 100) .

2. A semicolon after the condition

```
const temperature = 10;

if (temperature > 30); {
  console.log("It's hot! Drink some water.");
}
// prints: It's hot! Drink some water. (at 10 degrees?!)
```

That little ; ends the if straight away, with nothing inside it. The block below is no longer part of the if, so it runs every time. There's no error message to warn you. Fix: never put a semicolon between) and { .

3. Forgetting to repeat the variable in ||

```
const color = "blue";

if (color === "red" || "orange") {
  console.log("Warm color!");
}
// prints: Warm color! (wrong!)
```

This reads fine in English, but JavaScript sees two separate things: `color === "red"`, **or** `"orange"`. A non-empty string is truthy, so the condition is always true. Fix: `if (color === "red" || color === "orange")`.

4. Putting `else if` checks in the wrong order

```
const orderTotal = 120;

if (orderTotal > 50) {
  console.log("You get 10% off!");
} else if (orderTotal > 100) {
  console.log("You get 20% off!");
}
// prints: You get 10% off! (should be 20%)
```

The first true condition wins, and the rest are skipped. 120 is more than 50, so JavaScript never reaches the second check. Fix: put the biggest (most specific) check first.

5. Forgetting that `"0"` is truthy

```
const quantity = "0"; // typed into a form, so it's text

if (quantity) {
  console.log("Adding to your cart...");
}
// prints: Adding to your cart... (wrong!)
```

Only the empty string is falsy. `"0"` has a character in it, so it's truthy. Values typed into forms are always strings, so this comes up a lot. Fix: turn it into a number first: `if (Number(quantity) > 0)`.

Quick recap

- `if` runs a block only when its condition is true. `else` is the plan B. `else if` adds more choices, and the first true condition wins.
- Combine conditions with `&&`, `||`, and `!`. Keep nesting shallow by handling the problem cases first.
- The eight falsy values are `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, and `NaN`. Everything else is truthy, even `"0"`.

- The ternary condition `? a : b` picks one of two values.
- `switch` compares one value against many cases with `===`. End each case with `break`, and use `default` as the fallback.
- `||` gives a default for any falsy value. `&&` stops at the first falsy value, so it works as a guard.
- `??` only gives a default for `null` and `undefined`, so `0` and `""` are kept.

Next: try the [exercises](#), then move on to [08 Loops](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).
- Run each one with `node ex1.js` from a terminal opened in this folder.
- Most exercises ask you to test several values. Change the variable at the top, save, and run the file again each time.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Gym lockers

Your gym has 100 lockers, numbered 1 to 100. Even-numbered lockers are in the left changing room, and odd-numbered ones are in the right changing room.

```
const lockerNumber = 37;
```

Print where the locker is. Test it with these numbers:

lockerNumber	Expected output
37	Locker 37 is in the right changing room.
12	Locker 12 is in the left changing room.
250	There is no locker 250.
0	There is no locker 0.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Exam results

A school turns exam scores into grades:

Score	Grade
90 and above	A
80 to 89	B
70 to 79	C
60 to 69	D
below 60	F

A score of 60 or more is a pass.

```
const score = 85;
```

Expected output:

```
Score 85: grade B
Result: passed
```

Test it with 95, 72, 64, and 40 as well. For 40 you should see:

```
Score 40: grade F
Result: failed
```

Rule: use a ternary for the `Result` line.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Game settings

Your game saves the player's settings. If a setting was never saved, it's `null`.

```
const savedVolume = 0;  
const savedPlayerName = "";  
const savedDifficulty = null;
```

Load each setting, using a default when it's missing:

- **Volume:** the default is `50`. But `0` is a real choice: the player muted the game.
- **Player name:** the default is `"Player 1"`. An empty name isn't allowed, so it gets the default too.
- **Difficulty:** the default is `"normal"`.

When the volume is `0`, add `(muted)` after it.

Expected output:

```
Volume: 0 (muted)  
Player: Player 1  
Difficulty: normal
```

Then try `70`, `"Zara"`, and `"hard"`:

```
Volume: 70  
Player: Zara  
Difficulty: hard
```

And finally set all three to `null`:

```
Volume: 50  
Player: Player 1  
Difficulty: normal
```

Rule: use `??` or `||` for each setting (not `if`). Pick the right one each time.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Gym opening hours

Your gym's website shows the opening hours for the day:

- Monday to Thursday: 6:00 to 22:00
- Friday: 6:00 to 23:00 (it's late night Friday!)
- Saturday and Sunday: 8:00 to 20:00

```
const day = "Saturday";
```

Test it with these days:

day	Expected output
"Saturday"	Saturday: open 8:00 to 20:00
"Tuesday"	Tuesday: open 6:00 to 22:00
"Friday"	Friday: open 6:00 to 23:00 (late night!)
"Funday"	Funday is not a day of the week.

Rules:

- Use one `switch`.
- Days with the same hours must share one `console.log`. Don't write the same line four times.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Login check

Build the logic behind a website's login button. Here's what the website has saved, and what someone just typed:

```
const savedUsername = "sandip";
const savedPassword = "Mango#2026";
const nickname = null; // null means the user hasn't picked a nickname

const typedUsername = " Sandip ";
const typedPassword = "Mango#2026";
const failedAttempts = 0;
```

Print one message. Check things in this order:

1. If there have been 3 or more failed attempts: `Account locked. Try again in 15 minutes.`
2. If either box is empty: `Please fill in both boxes.` (A username with only spaces counts as empty.)
3. If the username or the password is wrong: `Wrong username or password.`
4. Otherwise: `Welcome back, NAME!` where `NAME` is the nickname, or the saved username if there's no nickname.

Username don't care about capitals or spaces around them, so " Sandip " is fine. Passwords are exact: capitals matter.

Test all of these:

typedUsername	typedPassword	failedAttempts	nickname	Expected output
" Sandip "	"Mango#2026"	0	null	Welcome back, sandip!
"SANDIP"	"Mango#2026"	1	"Sandy"	Welcome back, Sandy!
"sandip"	"mango#2026"	0	null	Wrong username or password.
" "	"Mango#2026"	0	null	Please fill in both boxes.
"sandip"	""	2	null	Please fill in both boxes.
"sandip"	"Mango#2026"	3	null	Account locked. Try again in 15 minutes.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Why "username or password"? Real websites don't say *which* one was wrong. Otherwise an attacker could use the message to find out which usernames exist. (Real websites also never store passwords as plain text. You'll learn more in [chapter 51](#).)

Before you move on

In Exercise 5, you changed the variables and ran the file again for every attempt. A real login screen lets you try again by itself, up to 3 times, and then locks.

Doing something again and again, until a condition says stop, is what [chapter 08: Loops](#) is all about.

08 Loops

What is it?

A **loop** runs the same block of code again and again, as many times as you need. Each trip through the loop is called an **iteration** (or a "round").

Why does it matter?

Say you want to print the numbers from 1 to 100. Without a loop, that's 100 lines of `console.log`. With a loop, it's 3 lines. Need 1 to 1,000 instead? You change one number.

Real programs repeat things all the time:

- Count down the seconds on a timer.
- Add interest to a savings account, year after year.
- Check every character of a password.
- Give someone 3 attempts to type their PIN.

Remember the login exercise in [chapter 07](#), where you ran the file again for every attempt? A loop lets your program do the repeating for you.

Real-world example

You repeat things every day. The way you decide when to stop tells you which loop to use:

Everyday task	What you know	Loop
"Do 10 push-ups."	Exactly how many times	<code>for</code>
"Stir the sauce until it thickens."	When to stop, but not how many stirs that takes	<code>while</code>
"Taste the soup and add salt, until it tastes right."	You always taste at least once	<code>do...while</code>

Every loop needs a way to stop. Push-ups stop at 10. Stirring stops when the sauce is thick. A loop that never stops is a real problem, and you'll see why later.

How it works

The `for` loop

Use `for` when you know how many times to repeat. Here are 5 push-ups:

```
for (let rep = 1; rep <= 5; rep++) {  
  console.log(`Push-up number ${rep}`);  
}
```

You'll see:

```
Push-up number 1  
Push-up number 2  
Push-up number 3  
Push-up number 4  
Push-up number 5
```

The line in parentheses has three parts, separated by semicolons:

Part	Code	When it runs	Its job
Start	<code>let rep = 1</code>	Once, before anything else	Creates the counter : a variable that keeps track of the rounds
Condition	<code>rep <= 5</code>	Before every round	Keep going while this is <code>true</code> . Stop as soon as it's <code>false</code>
Step	<code>rep++</code>	After every round	Moves the counter forward (<code>++</code> adds 1, from chapter 04)

Here's the loop, one round at a time:

Round	rep	rep <= 5 ?	What happens
1	1	<code>true</code>	prints <code>Push-up number 1</code> , then <code>rep</code> becomes 2
2	2	<code>true</code>	prints <code>Push-up number 2</code> , then <code>rep</code> becomes 3
...	...	...	...
5	5	<code>true</code>	prints <code>Push-up number 5</code> , then <code>rep</code> becomes 6
-	6	<code>false</code>	the loop ends

You'll often see the counter called `i` (short for "index"). It's one of the few places where a one-letter name is normal.

Counting in different ways

The three parts can do more than count up by one. A rocket countdown starts high and goes down with `--`:

```
for (let seconds = 5; seconds >= 1; seconds--) {  
  console.log(seconds);  
}  
console.log("Liftoff!");
```

You'll see:

```
5  
4  
3  
2  
1  
Liftoff!
```

And `+=` lets you take bigger steps. Here's a bus timetable, with a bus every 15 minutes:

```
for (let minute = 0; minute < 60; minute += 15) {  
  console.log(`A bus leaves at 9:${String(minute).padStart(2, "0")}`);  
}
```

You'll see:

```
A bus leaves at 9:00  
A bus leaves at 9:15  
A bus leaves at 9:30  
A bus leaves at 9:45
```

(`padStart` from [chapter 06](#) turns `0` into `00`.)

The `while` loop

Sometimes you don't know how many rounds you'll need. You only know when to stop.

You put \$1,000 in a savings account that pays 5% interest each year. How many years until you have \$1,500? You could work it out with a pen and paper... or let a loop try year after year:


```

let balance = 1000;
let years = 0;

while (balance < 1500) {
  balance = balance * 1.05; // add 5% interest
  years++;
}

console.log(`After ${years} years, you'll have ${balance.toFixed(2)}.`);
// prints: After 9 years, you'll have $1551.33.

```

`while` checks its condition before every round, just like `for`. As long as the condition is `true`, the block runs again. The moment it's `false`, the loop ends and the program moves on.

A `while` loop has no built-in counter or step. So *you* must make sure that something inside the block changes, so the condition eventually becomes `false`. Here, `balance` grows every round.

The `do...while` loop

A `do...while` loop runs the block **first** and checks the condition **after**. So it always runs at least once.

That's perfect for a board game: you always roll the dice at least once, and you keep rolling until you get a 6:

```

let roll;
let rolls = 0;

do {
  roll = Math.floor(Math.random() * 6) + 1; // a random whole number from 1 to 6
  rolls++;
  console.log(`You rolled a ${roll}`);
} while (roll !== 6);

console.log(`Got a 6 after ${rolls} ${rolls === 1 ? "roll" : "rolls"}!`);

```

The rolls are random (remember `Math.random()` from [chapter 05](#)?), so your output will be different. Here's one run:

```

You rolled a 4
You rolled a 3
You rolled a 6
Got a 6 after 3 rolls!

```

To see the "at least once" part clearly, compare the two loops when the condition is `false` from the very start:

```
const lives = 0;

while (lives > 0) {
  console.log("while: playing");
}

do {
  console.log("do...while: playing");
} while (lives > 0);
// prints: do...while: playing
```

The `while` loop checks first, sees `false`, and never runs. The `do...while` loop runs once before it checks.

Infinite loops (and how to stop them)

An **infinite loop** is a loop whose condition never becomes `false`, so it never stops. It's one of the most common beginner bugs:

```
let count = 1;

while (count <= 3) {
  console.log(count);
  // oops: we forgot count++
}
```

`count` stays `1` forever, so `1 <= 3` is always `true`. Your terminal fills up with `1`s, over and over, as fast as your computer can print them.

Don't panic. Click in the terminal and press `Ctrl + C`. That stops the running program straight away. Then fix the loop so something changes every round (here, add `count++`).

In the browser console, an infinite loop freezes the whole tab. Close the tab and open a new one.

Tip: before you run a new loop, ask yourself: "What changes each round, and when does the condition become `false`?" If you can't answer, the loop might never end.

Looping over a string

Remember from [chapter 06](#) that every character in a string has an index, starting at 0? A `for` loop can walk through those indexes one by one:

```
const word = "cat";

for (let i = 0; i < word.length; i++) {
  console.log(`Position ${i}: ${word[i]}`);
}
```

You'll see:

```
Position 0: c
Position 1: a
Position 2: t
```

Notice the condition is `i < word.length`, not `<=`. "cat" has a length of 3, but its last index is 2.

When you only need the characters themselves, not their positions, there's a shorter loop. `for...of` hands you each character in turn:

```
for (const letter of "cat") {
  console.log(letter);
}
```

You'll see:

```
c
a
t
```

No counter, no condition, no step, and no way to get the index wrong. Read it as "for each letter of this string". (You can use `const` here, because every round gets a brand-new `letter` variable.)

Loops and `if` make a great team. Here's how you count how many times a letter appears:

```
const river = "Mississippi";
let count = 0;

for (const letter of river) {
  if (letter === "s") {
    count++;
  }
}

console.log(`"s" appears ${count} times in ${river}.`);
// prints: "s" appears 4 times in Mississippi.
```

`count` is created *before* the loop, so it survives from round to round and keeps the running total.

You need the index version when you want to go backwards. This loop starts at the last character and builds a new string, one character at a time. (Remember from [chapter 04](#) that `+=` works on strings too: it adds to the end.)

```
const word = "stressed";
let reversed = "";

for (let i = word.length - 1; i >= 0; i--) {
  reversed += word[i];
}

console.log(reversed); // prints: desserts
```

break : stop the loop early

`break` jumps out of a loop straight away, even if the condition is still `true`. (Yes, it's the same `break` you used in `switch`.)

A website checks that a new password contains at least one digit. As soon as it finds one, there's no point checking the rest:

```
const password = "sunny7day";

for (const character of password) {
  if ("0123456789".includes(character)) {
    console.log(`Found a digit: ${character}`);
    break;
  }
  console.log(`Checked ${character}`);
}
```

You'll see:

```
Checked s
Checked u
Checked n
Checked n
Checked y
Found a digit: 7
```

The loop never looks at `d`, `a`, or `y`. It found what it needed and stopped.

continue : skip to the next round

`continue` skips the rest of the current round and jumps to the next one. The loop keeps going.

Many hotels don't have a 13th floor, because some people think 13 is unlucky. Here's the hotel's elevator:

```
for (let floor = 10; floor <= 15; floor++) {  
  if (floor === 13) {  
    continue; // skip floor 13  
  }  
  console.log(`Floor ${floor}`);  
}
```

You'll see:

```
Floor 10  
Floor 11  
Floor 12  
Floor 14  
Floor 15
```

The difference in one sentence: `break` leaves the loop for good, `continue` only skips one round.

Nested loops

A loop can go inside another loop. This is a **nested loop**. The inner loop runs all the way through, from start to finish, for *every single round* of the outer loop.

Think of a clock. The minute hand goes all the way around for every one step of the hour hand:

```
for (let hour = 9; hour <= 10; hour++) {  
  for (let minute = 0; minute < 60; minute += 20) {  
    console.log(`${hour}:${String(minute).padStart(2, "0")}`);  
  }  
}
```

You'll see:

```
9:00  
9:20  
9:40  
10:00  
10:20  
10:40
```

The outer loop runs 2 times, and the inner loop runs 3 times for each of those. That's $2 \times 3 = 6$ lines.

Nested loops are how you build anything with rows and columns. Here's a multiplication table. The outer loop makes the rows. The inner loop builds each row as a string, one number at a time, and then the row is printed:

```
for (let row = 1; row <= 4; row++) {  
  let line = "";  
  
  for (let column = 1; column <= 4; column++) {  
    line += String(row * column).padStart(4);  
  }  
  
  console.log(line);  
}
```

You'll see:

```
1  2  3  4  
2  4  6  8  
3  6  9 12  
4  8 12 16
```

`padStart(4)` makes every number 4 characters wide, so the columns line up.

The inner loop doesn't have to run the same number of times in every round. Here, it runs `row` times, so each row gets one more star than the row before it:

```
for (let row = 1; row <= 4; row++) {  
  let stars = "";  
  
  for (let i = 1; i <= row; i++) {  
    stars += "*";  
  }  
  
  console.log(stars);  
}
```

You'll see:

```
*  
**  
***  
****
```

(`"*".repeat(row)` from chapter 06 would give you the same row in one step. Now you know how much work it saves you!)

Choosing the right loop

Your situation	Use
You know how many times to repeat (10 push-ups, the numbers 1 to 100)	<code>for</code>
You want every character of a string	<code>for...of</code>
You need each character's position, or want to go backwards	<code>for</code> with an index
You repeat until something happens, but don't know when (a savings goal)	<code>while</code>
Same, but it must run at least once (rolling dice)	<code>do...while</code>

When in doubt, start with `for` or `for...of`. They're the most common by far, and you'll use `for...of` a lot with lists in [chapter 10](#).

Common mistakes

1. Going one step too far (or not far enough)

```
const word = "cat";

for (let i = 0; i <= word.length; i++) {
  console.log(word[i]);
}
```

You'll see:

```
c
a
t
undefined
```

This is called an **off-by-one error**: the loop runs one time too many or one time too few. It's so common that programmers joke about it. Here, `<=` lets `i` reach 3, but the last index of `"cat"` is 2. Fix: `i < word.length`.

The opposite happens too. Start at `let i = 1` and you skip the first letter. When a loop looks wrong, check the first round and the last round.

2. Using `const` for the counter

```
for (const lap = 1; lap <= 3; lap++) {  
  console.log(`Lap ${lap}`);  
}  
// prints: Lap 1  
// TypeError: Assignment to constant variable.
```

The first round works. Then `lap++` tries to change `lap`, and a `const` can't be changed ([chapter 02](#)). Fix: `for (let lap = 1; ...)`. (In a `for...of` loop, `const` is fine, because every round gets a new variable.)

3. Creating the running total inside the loop

```
const fruit = "banana";  
  
for (const letter of fruit) {  
  let count = 0;  
  if (letter === "a") {  
    count++;  
  }  
}  
  
console.log(count);  
// ReferenceError: count is not defined
```

Two problems in one. A variable created inside the loop's `{ }` is brand new in every round, so it starts from 0 again each time. And it doesn't exist outside the braces at all. Fix: create `let count = 0;` *before* the loop, like the Mississippi example.

4. Forgetting to change anything

If nothing inside a `while` loop changes the variable in its condition, the loop never ends. Press `Ctrl + C` to stop it, then add the missing step (like `count++`). See [Infinite loops](#) above.

Quick recap

- A loop repeats a block of code. Every loop needs a way to stop.
- `for (start; condition; step)` is best when you know how many rounds you need.
- `while` checks first and repeats until the condition is `false`. `do...while` always runs at least once.
- `for...of` gives you each character of a string. Use a `for` loop with an index when you need positions, and use `i < text.length`, not `<=`.
- `break` leaves the loop early. `continue` skips to the next round.
- In a nested loop, the inner loop runs completely for every round of the outer loop.

- Stuck in an infinite loop? Press `Ctrl + C` in the terminal.

Next: try the [exercises](#), then move on to [09 Functions](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Run each one with `node ex1.js` from a terminal opened in this folder.
- If a program never stops, press `Ctrl + C` in the terminal, then check what changes in each round of your loop.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Times table

A primary school teacher wants to print times tables for her class. Start with:

```
const number = 7;
```

Expected output:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Rule: if you change `number` to `9`, your program must print the 9 times table, with no other changes.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Vowel counter

A bookshop's website shows fun facts about book titles. Count the vowels (a , e , i , o , u , in capitals or lowercase) in a title:

```
const bookTitle = "The Hobbit: An Unexpected Journey";
```

Expected output:

```
"The Hobbit: An Unexpected Journey" has 11 vowels.
```

Change the title to "Harry Potter" and you should see "Harry Potter" has 3 vowels. (For this exercise, y doesn't count as a vowel.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): FizzBuzz

FizzBuzz is a famous little puzzle. Companies really do use it in job interviews, to check that someone can make loops and conditions work together.

Print the numbers from 1 to 15, but:

- For multiples of 3, print Fizz instead of the number.
- For multiples of 5, print Buzz instead.
- For multiples of both 3 and 5, print FizzBuzz .

Expected output:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: make it go up to 100. How many lines of code did you have to change?

Exercise 4 (Medium): Cinema seat map

A small cinema has 4 rows (A to D) with 6 seats each. The booking system stores the booked seats in one string:

```
const rows = "ABCD";
const seatsPerRow = 6;
const bookedSeats = "A3 B1 B2 C6 D4";
```

Print a seat map, where [X] is a booked seat and [] is a free one. Then print how many seats are still free:

```
A [ ] [X] [ ] [ ] [ ]
B [X] [X] [ ] [ ] [ ] [ ]
C [ ] [ ] [ ] [ ] [ ] [X]
D [ ] [ ] [X] [ ] [ ] [ ]
Free seats: 19
```

Rule: your code must work for any cinema. Try "ABCDE" , 8 , and "E8 A1" . You should see 5 rows of 8 seats, with only A1 and E8 booked, and Free seats: 38 .

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Phone PIN lock

Your phone's lock screen gives you 3 attempts to type your 4-digit PIN. The phone keeps a log of every digit that was typed, in order:

```
const correctPin = "2580";  
const keypadLog = "1234000025809999";  
const maxAttempts = 3;
```

Every 4 digits in the log is one attempt. So this log means: 1234, then 0000, then 2580, then 9999.

Check the attempts in order and print what happens:

```
Attempt 1: 1234 - wrong PIN  
Attempt 2: 0000 - wrong PIN  
Attempt 3: 2580 - unlocked!
```

Rules:

- As soon as the phone unlocks, stop checking. (That's why 9999 isn't checked.)
- Never check more than `maxAttempts` attempts.
- If all the allowed attempts are wrong, print `Phone locked. Try again in 30 seconds.` at the end.
- You can assume the log always has at least 3 attempts in it.

Test it with "1111222233334444":

```
Attempt 1: 1111 - wrong PIN  
Attempt 2: 2222 - wrong PIN  
Attempt 3: 3333 - wrong PIN  
Phone locked. Try again in 30 seconds.
```

And with "258011112222", you should only see `Attempt 1: 2580 - unlocked!`.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Your PIN check works, but it's stuck inside one file. A real phone needs the same check in lots of places: the lock screen, the banking app, the settings page. Copying and pasting it everywhere means that when something changes, you have to fix every single copy.

[Chapter 09](#) shows you how to wrap code up once, give it a name, and use it wherever you like. Those are functions.

09 Functions

What is it?

A **function** is a reusable block of code with a name. You write it once, and then you can run it whenever you like, just by using its name.

You've been using functions since chapter 01: `console.log()`, `Math.round()`, and `Number()` are all functions that someone else wrote. In this chapter, you'll write your own.

Why does it matter?

Remember the end of [chapter 08](#)? Your PIN check lived in one place, but a real phone needs it on the lock screen, in the banking app, and in the settings. Without functions, you'd copy and paste it everywhere. Then:

- Your program gets long and hard to read.
- When you find a bug, you have to fix every single copy (and you'll miss one).
- Nobody can tell what a chunk of code does without reading every line.

Functions fix all three. Programmers even have a motto for it: **DRY**, which stands for "Don't Repeat Yourself". Write the code once, give it a clear name like `checkPin`, and use it anywhere.

Real-world example

A function is like a **coffee machine**.

Coffee machine	Function
Someone builds the machine once	You declare (create) the function once
You press the button whenever you want a coffee	You call (run) the function whenever you need it
The machine has slots for water, beans, and milk	The function has parameters
The beans and milk you put in today	The arguments you pass in
The cup of coffee that comes out	The return value

And here's the best part: you don't need to know how the machine works inside. You only need to know what to put in and what comes out. Functions are the same. Once one works, you can use it without thinking about its insides.

How it works

Declaring and calling a function

Here's a function for a cafe's welcome screen:

```
function greetCustomer() {  
  console.log("Welcome to Bean There Cafe!");  
  console.log("What can I get you today?");  
}  
  
greetCustomer();  
greetCustomer();
```

You'll see:

```
Welcome to Bean There Cafe!  
What can I get you today?  
Welcome to Bean There Cafe!  
What can I get you today?
```

There are two separate steps here:

1. **Declaring** the function creates it. You write the keyword `function`, a name, parentheses `()`, and a block in curly braces. The code in the block is called the function's **body**. Declaring it doesn't run anything yet. It's like building the coffee machine.
2. **Calling** the function runs its body. You write its name followed by parentheses: `greetCustomer()`. That's pressing the button. Here we pressed it twice, so the body ran twice.

Parameters and arguments

A welcome message is nicer with the customer's name in it. You can give a function **parameters**: names in the parentheses that stand for values you'll hand over later.

```
function greetCustomer(name) {  
  console.log(`Welcome, ${name}!`);  
}  
  
greetCustomer("Aisha"); // prints: Welcome, Aisha!  
greetCustomer("Tom");   // prints: Welcome, Tom!
```

When you call `greetCustomer("Aisha")`, the value `"Aisha"` is called an **argument**. JavaScript puts it into the parameter `name`, and the body runs with it. The next call puts `"Tom"` in instead.

- **Parameter:** the empty slot in the machine (`name`). You write it when you *declare* the function.
- **Argument:** what you actually put in the slot (`"Aisha"`). You write it when you *call* the function.

A function can have several parameters, separated by commas. The arguments fill them **in order**: the first argument goes into the first parameter, and so on.

```
function describeOrder(size, drink) {
  console.log(`One ${size} ${drink}, coming right up!`);
}

describeOrder("large", "latte"); // prints: One Large Latte, coming right up!
describeOrder("latte", "large"); // prints: One Latte Large, coming right up!
```

JavaScript doesn't know what the words mean. It just matches them up by position, so the order you pass them in matters.

return : handing back a value

Most useful functions work something out and hand you the result. That's what `return` does:

```
function calculateTip(bill, tipPercent) {
  return bill * tipPercent / 100;
}

const tip = calculateTip(40, 15);

console.log(tip); // prints: 6
console.log(`Total with tip: ${40 + tip}`); // prints: Total with tip: $46
```

When JavaScript reaches `return`, two things happen:

1. The function **stops** right there.
2. The value after `return` is **handed back** to the place where the function was called.

You can picture the call being swapped for its answer. `const tip = calculateTip(40, 15);` turns into `const tip = 6;`.

The value that comes back is called the **return value**. And if a function has no `return` at all, it still hands something back: `undefined`, which means "nothing".

`console.log` vs `return` (read this twice)

Both seem to "give you the answer", so almost every beginner mixes them up at some point. But they do completely different jobs.

Think about the coffee machine again. When your latte is ready, two things can happen:

- The little **screen** on the machine lights up: "Your latte is ready!" That's `console.log`. It's a message for the person standing there. You can read it, but you can't drink it, and you can't do anything else with it.
- A **cup** comes out. That's `return`. You can pick it up, add sugar, carry it to your table, or hand it to a friend.

A machine that shows "Your latte is ready!" but never gives you a cup looks like it worked. But you've got nothing to drink.

Here are two functions for a pizza shop. Both add a \$5 delivery fee to the price. One *shows* the answer. The other *returns* it:

```
function showTotal(price) {  
  console.log(price + 5); // the screen message  
}  
  
function getTotal(price) {  
  return price + 5; // the cup  
}  
  
const shown = showTotal(20); // prints: 25  
const returned = getTotal(20); // (prints nothing)  
  
console.log(shown); // prints: undefined  
console.log(returned); // prints: 25
```

Look at what happened:

- `showTotal(20)` printed `25` on the screen. But it didn't hand anything back, so `shown` got `undefined`. An empty cup.
- `getTotal(20)` didn't print anything. It quietly handed back `25`, and `returned` kept it.

The difference really shows when you try to *use* the answer. Say a customer places two orders like this one, each with its own delivery:

```
console.log(showTotal(20) * 2);
```

You'll see:

25
NaN

`showTotal` printed its screen message (25), then handed back `undefined`. And `undefined * 2` is `NaN`: "not a number". The version with `return` just works:

```
console.log(getTotal(20) * 2); // prints: 50
```

	<code>console.log(value)</code>	return value
In the coffee machine	The screen message	The cup of coffee
Who gets the value?	You, reading the terminal	The code that called the function
Can you save it or do math with it?	No	Yes
Does it stop the function?	No	Yes

The rule of thumb: if a function works something out, it should `return` the answer. Let the code that *calls* the function decide what to do with it: print it, save it, or use it in the next calculation.

`console.log` is still your best friend for *looking* at values while you write and test code. Just don't use it as a way to hand answers back.

Returning early

Because `return` stops the function, you can use it to leave as soon as you know the answer. Here's a cash machine:

```
function withdraw(balance, amount) {  
  if (amount <= 0) {  
    return "Please enter an amount above zero.";  
  }  
  if (amount > balance) {  
    return "Sorry, you don't have enough money.";  
  }  
  return `Here's your ${amount}. New balance: ${balance - amount}.`;  
}  
  
console.log(withdraw(100, 30)); // prints: Here's your $30. New balance: $70.  
console.log(withdraw(100, 500)); // prints: Sorry, you don't have enough money.  
console.log(withdraw(100, -5)); // prints: Please enter an amount above zero.
```

This is the cleaner trick promised in [chapter 07](#). Each problem case is checked at the top, and it leaves straight away with a `return`. These checks are called **guard clauses**. No `else`, no nesting: if the code gets past the guards, everything is fine, and the happy case sits at the bottom.

One thing to know: once a `return` runs, the rest of the function is skipped. A line like `console.log("Done!");` placed after the final `return` would never print.

Default parameters

What happens if you forget an argument? The parameter gets `undefined`:

```
function calculateTip(bill, tipPercent) {  
  return bill * tipPercent / 100;  
}  
  
console.log(calculateTip(40)); // prints: NaN
```

`40 * undefined` isn't a number, so you get `NaN`. A **default parameter** gives a parameter a backup value, used only when the argument is missing:

```
function calculateTip(bill, tipPercent = 15) {  
  return bill * tipPercent / 100;  
}  
  
console.log(calculateTip(40)); // prints: 6 (uses the default 15%)  
console.log(calculateTip(40, 20)); // prints: 8 (20% replaces the default)
```

Say the restaurant suggests a 15% tip, so that's the default. Anyone who wants to tip a different amount can still pass their own number.

Function expressions

There's another way to create a function: make it without a name, and store it in a variable. This is called a **function expression**:

```
const minutesToHours = function (minutes) {  
  return minutes / 60;  
};  
  
console.log(minutesToHours(90)); // prints: 1.5
```

You call it the same way. Notice the semicolon after the closing `}`. The whole thing is one big `const` statement, just like `const age = 30;`.

One difference: you can call a function *declaration* on a line above the one where it's written, but not a function expression. Try that with an expression, and you get `ReferenceError: Cannot access 'minutesToHours' before initialization`. [Chapter 14](#) explains why.

Arrow functions

An **arrow function** is a shorter way to write a function expression. You drop the word `function` and put an arrow `=>` (an equals sign and a greater-than sign) after the parentheses:

```
const minutesToHours = (minutes) => {  
  return minutes / 60;  
};
```

It gets even shorter. When the body is just one value to return, you can drop the curly braces *and* the word `return`:

```
const minutesToHours = (minutes) => minutes / 60;  
  
console.log(minutesToHours(90)); // prints: 1.5
```

This is called an **implicit return**: the value after the arrow is returned automatically. Here are a few more shapes you'll see:

```
const sayHello = () => "Hello!";    // no parameters: empty parentheses  
const add = (a, b) => a + b;        // two parameters  
const isAdult = (age) => age >= 18; // returns true or false  
  
console.log(sayHello()); // prints: Hello!  
console.log(add(2, 3));   // prints: 5  
console.log(isAdult(15)); // prints: false
```

With exactly one parameter, the parentheses are optional (`age => age >= 18` works too), but this course always keeps them.

Which style should I use? You'll see all three in real code, so it's good to recognize them. In this course:

- Named functions, like `calculateTip`, are written as **function declarations**. They're easy to spot and can be called from anywhere in the file.
- **Arrow functions** are for short, small functions. They really shine when you hand a function to another function, which you'll start doing in [chapter 13](#).

Functions calling functions

A function can call other functions. That's how you build big programs: out of small pieces that each do one job well, like LEGO bricks.

Here's a food delivery receipt. Delivery is free for orders of \$30 or more:

```
function formatMoney(amount) {  
  return `$$${amount.toFixed(2)}`;  
}  
  
function getDeliveryFee(foodTotal) {  
  return foodTotal >= 30 ? 0 : 4.99;  
}  
  
function printReceipt(foodTotal) {  
  const fee = getDeliveryFee(foodTotal);  
  console.log(`Food: ${formatMoney(foodTotal)}`);  
  console.log(`Delivery: ${formatMoney(fee)}`);  
  console.log(`Total: ${formatMoney(foodTotal + fee)}`);  
}  
  
printReceipt(25);
```

You'll see:

```
Food: $25.00  
Delivery: $4.99  
Total: $29.99
```

Each function is small and easy to understand on its own. `formatMoney` is used three times, but written only once. And if the delivery rules change, there's exactly one place to fix.

Naming functions

A function *does* something, so give it a name that starts with a verb (an action word):

Good name	Why it's good
<code>calculateTip</code>	Says exactly what it works out
<code>formatMoney</code>	Says what it does to the value
<code>printReceipt</code>	"print" tells you it shows something on the screen

Good name	Why it's good
<code>isAdult</code> , <code>hasDigit</code>	Starting with <code>is</code> or <code>has</code> tells you it returns <code>true</code> or <code>false</code>

Avoid vague names like `doStuff`, `handle`, or `data`. And don't name a function like a value (`tip`, `total`): that sounds like a number, not an action. A good name means you can read `calculateTip(40)` and know what happens without looking inside.

Functions are values

Here's something surprising: a function is a value, just like a number or a string. So you can store it in another variable:

```
function shout(text) {
  return `${text.toUpperCase()}!`;
}

const yell = shout; // no parentheses: we're not calling it

console.log(yell("hello")); // prints: HELLO!
console.log(typeof shout); // prints: function
console.log(shout);         // prints: [Function: shout]
```

The parentheses make all the difference:

- `shout` (no parentheses) is the function itself: the coffee machine.
- `shout("hi")` calls the function: pressing the button. You get back `"HI!"`.

Why would you ever pass a function around without calling it? Because later you'll hand functions to other functions, as instructions for what to do. That's called a **callback**, and it's the big idea of [chapter 13](#).

Variables inside a function stay inside

A variable created inside a function only exists inside that function. The parameters, too:

```
function calculateBill(price) {
  const tip = price * 0.15;
  return price + tip;
}

console.log(calculateBill(40)); // prints: 46
console.log(tip);
// ReferenceError: tip is not defined
```

Think of a restaurant kitchen. The chef uses bowls, knives, and chopping boards in there, but only the finished dish comes out through the hatch. `tip` is one of the kitchen bowls. The dish is the return value.

It works one way only: code inside a function *can* see variables created outside it, but code outside can't see in. This is called **scope**, and [chapter 14](#) tells the full story.

Common mistakes

1. Declaring a function but never calling it

```
function showWelcome() {  
  console.log("Welcome to the gym!");  
}
```

You run the file and... nothing happens. Declaring a function only builds the machine. Nothing runs until you press the button. Fix: add `showWelcome();` below it.

2. Forgetting the parentheses

```
function getGreeting() {  
  return "Good morning!";  
}  
  
console.log(getGreeting); // prints: [Function: getGreeting]
```

Without `()`, you're pointing at the machine instead of pressing its button. Fix: `console.log(getGreeting());`

3. Printing the answer instead of returning it

```
function calculateArea(width, height) {  
  console.log(width * height);  
}  
  
const area = calculateArea(4, 5); // prints: 20  
console.log(`The room is ${area} square meters.`);  
// prints: The room is undefined square meters.
```

The function showed `20` on the screen, but handed nothing back, so `area` is `undefined`. It's the screen message without the cup. Fix: `return width * height;`

4. Using curly braces in an arrow function, but no `return`

```
const double = (number) => { number * 2 };

console.log(double(5)); // prints: undefined
```

The implicit return only works *without* curly braces. Once you add braces, it's a normal function body again, and a normal body needs `return`. Fix: `(number) => number * 2`, or `(number) => { return number * 2; }`.

5. Passing arguments in the wrong order

```
function calculateSpeed(distanceKm, hours) {
  return distanceKm / hours;
}

console.log(calculateSpeed(2, 120)); // prints: 0.016666666666666666
```

We meant "120 km in 2 hours", but the arguments went into the wrong slots. There's no error, just a wrong answer, which makes this one sneaky. Fix: `calculateSpeed(120, 2)` gives `60`. Clear parameter names like `distanceKm` help you spot the mix-up.

Quick recap

- A function is a named, reusable block of code. Declare it once with `function name() { ... }`, then call it with `name()` as often as you like.
- **Parameters** are the slots in the declaration. **Arguments** are the real values you pass in when you call it. They're matched by position.
- `return` hands a value back and stops the function. With no `return`, a function gives back `undefined`.
- `console.log` shows a value to *you*. `return` gives it to *your code*. If a function works something out, return it.
- Guard clauses (early `return`s) handle problem cases first. Default parameters (`tipPercent = 15`) fill in missing arguments.
- Function expressions and arrow functions store a function in a variable. An arrow function without curly braces returns its value automatically.
- Name functions with verbs. Functions are values, and variables created inside a function stay inside it.

Next: try the [exercises](#), then move on to [10 Arrays](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - Unless an exercise says otherwise, write named functions as function declarations, and make them **return** their answers. Do the printing outside the function.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Temperature converter

You're planning a trip, and the weather apps keep mixing up Celsius and Fahrenheit. Write two functions:

- `celsiusToFahrenheit(celsius)` : the formula is $F = C \times 9 / 5 + 32$
- `fahrenheitToCelsius(fahrenheit)` : the formula is $C = (F - 32) \times 5 / 9$

Each function must **return** the answer, rounded to 1 decimal place. Then call them to print:

```
0°C = 32°F
100°C = 212°F
37°C = 98.6°F
-40°C = -40°F
98.6°F = 37°C
451°F = 232.8°C
0°F = -17.8°C
```

(You can copy the ° symbol from this page, or leave it out.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Workout tracker helpers

A fitness app needs three small helper functions. Each one takes the length of a workout in minutes:

- `formatDuration(minutes)` returns a string like "1h 30m"
- `getCaloriesBurned(minutes)` returns the calories burned, at 8 calories per minute
- `isLongWorkout(minutes)` returns `true` if the workout is 60 minutes or more, and `false` if not

Start with:

```
const workoutMinutes = 90;
```

Expected output:

```
Workout: 1h 30m  
Calories burned: 720  
Long workout: true
```

Change `workoutMinutes` to `45`, and you should see:

```
Workout: 0h 45m  
Calories burned: 360  
Long workout: false
```

Rule: write all three as arrow functions with an implicit return: no curly braces and no `return` keyword.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Bug hunt

This gym membership calculator has **3 bugs**. Copy it into `ex3.js`, run it, and fix one bug at a time.

```
// Gym membership calculator
function getMonthlyPrice(yearlyPrice) {
  console.log(yearlyPrice / 12);
}

const getStudentPrice = (price) => {
  price * 0.8;
};

function formatPrice(amount) {
  return `$$${amount.toFixed(2)}`;
}

function getTotalCost(monthlyPrice, months) {
  return monthlyPrice * months;
}

const yearlyPrice = 600;
const monthlyPrice = getMonthlyPrice(yearlyPrice);
const studentPrice = getStudentPrice(monthlyPrice);

console.log(`Yearly: ${formatPrice(yearlyPrice)}`);
console.log(`Monthly: ${formatPrice(monthlyPrice)}`);
console.log(`Student monthly: ${formatPrice(studentPrice)}`);
console.log(`3 months: ${formatPrice(getTotalCost(3))}`);
```

When it's fixed, you should see exactly this (and nothing else):

```
Yearly: $600.00
Monthly: $50.00
Student monthly: $40.00
3 months: $150.00
```

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): The coffee machine

Time to build the coffee machine from the notes! Write a function `orderCoffee(drink, size)` that **returns** a message for the customer.

The menu:

Drink	Price (medium)
"espresso"	\$2.50
"latte"	\$3.50
"cappuccino"	\$3.75

A "small" costs \$0.50 less, and a "large" costs \$1.00 more.

Rules:

- If the customer doesn't say which drink, they get an espresso. If they don't say which size, it's a medium. Use default parameters for this.
- If the drink isn't on the menu, return `Sorry, we don't make DRINK.` (with the drink's name) straight away.
- The function must return the message, not print it.

Call it like this:

```
console.log(orderCoffee("latte", "large"));
console.log(orderCoffee("cappuccino"));
console.log(orderCoffee());
console.log(orderCoffee("tea", "small"));
console.log(orderCoffee("latte", "small"));
```

Expected output:

```
Here's your large latte. That's $4.50, please.
Here's your medium cappuccino. That's $3.75, please.
Here's your medium espresso. That's $2.50, please.
Sorry, we don't make tea.
Here's your small latte. That's $3.00, please.
```

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Password strength checker

A sign-up page tells people how strong their new password is. Build it from small functions that work together:

- `countDigits(text)` returns how many digits (0 to 9) the text contains.
- `hasUppercase(text)` returns `true` if the text contains at least one capital letter.
- `getPasswordStrength(password)` returns `"weak"`, `"medium"`, or `"strong"`:
 - **weak:** fewer than 8 characters

- **strong:** at least 12 characters, at least one capital letter, **and** at least 2 digits
- **medium:** everything else
- `maskPassword(password)` returns the password with everything after the first 2 characters hidden by `*`. Real apps never show passwords on the screen, so `"Sunshine2026"` becomes `"Su*****"`.
- `printStrength(password)` prints the masked password and its strength.

Call `printStrength` with these passwords:

```
printStrength("cat123");
printStrength("sunshine");
printStrength("Sunshine2026");
printStrength("sunshine2026");
printStrength("CorrectHorse9");
```

Expected output:

```
ca****: weak
su*****: medium
Su*****: strong
su*****: medium
Co*****: medium
```

Rule: `getPasswordStrength` must use `countDigits` and `hasUppercase`, and `printStrength` must use `maskPassword` and `getPasswordStrength`. Only `printStrength` prints anything.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

You checked five passwords with five separate lines of code. What if a website had 10,000 passwords to check? Or a shop had 500 products? You'd want to keep them all together in one list, and loop over it.

That list is called an array, and it's what [chapter 10](#) is about. 😊

10 Arrays

What is it?

An **array** is a list of values, kept in order, stored under one name.

```
const shoppingList = ["milk", "eggs", "bread"];
```

Each value in the list is called an **element** (or just an item). Each element has a position number, called its **index**.

Why does it matter?

Say you're building a music app. Without arrays, every song needs its own variable:

```
const song1 = "Morning Run";
const song2 = "Rainy Day";
const song3 = "Night Drive";
```

That falls apart fast:

- A playlist with 200 songs would need 200 variables.
- You can't loop over `song1`, `song2`, `song3`. A [loop](#) needs one thing to step through.
- Adding a song means writing a new line of code, instead of just adding to a list.

With an array, the whole playlist lives in one variable:

```
const playlist = ["Morning Run", "Rainy Day", "Night Drive"];
```

Now you can add songs, remove songs, count them, and loop over them. Almost every app is built on lists: messages in a chat, products in a shop, photos in an album, tasks in a to-do list.

Real-world example

Think of an array as a **train**:

Train	Array
The whole train	The array
One carriage	One element

Train	Array
The number painted on a carriage	The index
How many carriages there are	<code>length</code>
Hooking a new carriage onto the back	<code>push</code>
Unhooking the last carriage	<code>pop</code>

This train has one odd rule: the first carriage is number **0**, not 1. So a train with 3 carriages has carriages 0, 1, and 2.

That rule trips up almost everyone at first. JavaScript counts positions from 0, just like it did with the letters of a string in [chapter 06](#).

How it works

Creating an array

Put the values inside square brackets `[ ]`, separated by commas:

```
const shoppingList = ["milk", "eggs", "bread"];
console.log(shoppingList); // prints: [ 'milk', 'eggs', 'bread' ]
```

Node prints arrays with spaces inside the brackets and single quotes around strings. That's only how it displays them; your double quotes are fine. The browser console shows the same list a little differently: it adds the length, like `(3)`, and an arrow you can click to open the list up.

An array can also start out empty and get filled later:

```
const cart = [];
console.log(cart); // prints: []
console.log(cart.length); // prints: 0
```

Reading items by index

Put the index in square brackets to get one item:

```
const planets = ["Mercury", "Venus", "Earth", "Mars"];
console.log(planets[0]); // prints: Mercury
console.log(planets[2]); // prints: Earth
```

`planets[2]` means "the item at index 2", which is the *third* item. Counting from 0 feels strange for a week or so, then it becomes a habit.

How many items: `length`

Every array knows how many items it has:

```
const planets = ["Mercury", "Venus", "Earth", "Mars"];
console.log(planets.length);           // prints: 4
console.log(planets[planets.length - 1]); // prints: Mars
console.log(planets.at(-1));           // prints: Mars
console.log(planets.at(-2));           // prints: Earth
```

Since the first index is 0, the last index is always `length - 1`. Here that's `4 - 1`, which is 3.

Writing `planets[planets.length - 1]` gets clunky, so use `.at()`, which you met with strings. A negative number counts back from the end: `at(-1)` is the last item, `at(-2)` is the one before it.

Changing an item

Put the index on the left of `=` to replace an item:

```
const seats = ["free", "free", "free"];
seats[1] = "taken";
console.log(seats); // prints: [ 'free', 'taken', 'free' ]
```

This is a big difference from strings. In chapter 06 you saw that strings are immutable: you can't change them in place. Arrays can be changed.

Wait, isn't `seats` a `const`?

Yes, and that's allowed. In [chapter 02](#), `const` meant you can't swap the value for something else. That's still true here: `seats` must always hold the *same* array. But nothing stops you from changing what's inside that array.

Think of a shopping basket with your name tag on it. You can put things in and take things out all day. It's still the same basket, so the tag still belongs on it. What you can't do is move your tag onto a different basket.

```
const seats = ["free", "free", "free"];
seats[0] = "taken";           // fine: changing what's inside the same array
seats = ["taken", "taken"]; // not allowed: swapping in a different array
// TypeError: Assignment to constant variable.
```


So keep using `const` for arrays. Use `let` only when you plan to replace the whole array with a new one.

Adding and removing at the ends

Four methods add or remove items at the start or the end of an array. Remember from chapter 06: a **method** is an action a value knows how to do, like `"hi".toUpperCase()`. Arrays come with their own methods.

Picture a queue at a bakery:

```
const queue = ["Ana", "Ben"];

queue.push("Cara"); // Cara joins at the back
queue.unshift("Dev"); // Dev has a VIP pass and goes to the front
console.log(queue); // prints: [ 'Dev', 'Ana', 'Ben', 'Cara' ]

const served = queue.shift(); // the person at the front gets served
const gaveUp = queue.pop(); // the person at the back gives up and leaves
console.log(served); // prints: Dev
console.log(gaveUp); // prints: Cara
console.log(queue); // prints: [ 'Ana', 'Ben' ]
```

Method	What it does	Where	Gives back
<code>push(item)</code>	adds	end	the new length
<code>pop()</code>	removes	end	the removed item
<code>unshift(item)</code>	adds	start	the new length
<code>shift()</code>	removes	start	the removed item

A few things worth knowing:

- `pop()` and `shift()` hand back the item they removed, so you can keep it in a variable (like `served` above).
- `push` and `unshift` can add several items at once: `queue.push("Eli", "Fay")`.
- On an empty array, `pop()` and `shift()` give back `undefined`.
- To remember `shift`: when the person at the front leaves, everyone else *shifts* forward one place.

Tip: `push` is the one you'll use most. Starting with an empty array `[]` and pushing items into it is one of the most common patterns in programming.

Finding things: `includes` and `indexOf`

These work just like the string versions from chapter 06:

```
const guests = ["Asha", "Ben", "Chloe"];
console.log(guests.includes("Ben")); // prints: true
console.log(guests.includes("ben")); // prints: false
console.log(guests.indexOf("Chloe")); // prints: 2
console.log(guests.indexOf("Dan")); // prints: -1
```

- `includes` answers yes or no: is this item in the list?
- `indexOf` tells you *where* it is. If it isn't there, you get `-1`, which means "not found".
- Both are case-sensitive, so `"ben"` doesn't match `"Ben"`.

`includes` fits nicely inside an `if` from [chapter 07](#):

```
const guests = ["Asha", "Ben", "Chloe"];
const visitor = "Dan";

if (guests.includes(visitor)) {
  console.log(`Welcome in, ${visitor}!`);
} else {
  console.log(`Sorry ${visitor}, you're not on the list.`);
}
// prints: Sorry Dan, you're not on the List.
```

Copying part of an array: `slice`

`slice(start, end)` copies a piece of an array into a new array. The item at `start` is included, and the item at `end` is not. That's the same rule as `slice` on strings.

```
const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"];
const spring = months.slice(2, 5);
console.log(spring); // prints: [ 'Mar', 'Apr', 'May' ]
console.log(months); // prints: [ 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun' ]
```

The original `months` is untouched. `slice` only makes a copy.

A few handy forms:

```
const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"];
console.log(months.slice(4)); // prints: [ 'May', 'Jun' ]
console.log(months.slice(-3)); // prints: [ 'Apr', 'May', 'Jun' ]
console.log(months.slice()); // prints: [ 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun' ]
```

- Leave out `end` to go all the way to the end.
- A negative number counts from the end, like `at(-1)` does.
- With nothing in the brackets, `slice()` copies the whole array.

Removing or inserting in the middle: `splice`

`splice` is the heavy-duty tool. It **changes** the array. It can remove items, insert items, or both, anywhere in the list.

`splice(start, howMany)` removes `howMany` items, starting at index `start`. It gives you back the removed items as a new array:

```
const tasks = ["email Sam", "buy stamps", "call the dentist", "pay rent"];
const done = tasks.splice(1, 2); // from index 1, remove 2 items
console.log(done); // prints: [ 'buy stamps', 'call the dentist' ]
console.log(tasks); // prints: [ 'email Sam', 'pay rent' ]
```

Add more values after `howMany`, and `splice` inserts them at that spot:

```
const tasks = ["email Sam", "pay rent"];

tasks.splice(1, 0, "water plants"); // at index 1: remove nothing, insert one
console.log(tasks); // prints: [ 'email Sam', 'water plants', 'pay rent' ]

tasks.splice(2, 1, "pay bills"); // at index 2: remove one, insert one (a swap)
console.log(tasks); // prints: [ 'email Sam', 'water plants', 'pay bills' ]
```

`slice` or `splice`?

The names are one letter apart, so they're easy to mix up. Think of a book:

- `slice` is like **photocopying** some pages. You get copies, and the book stays whole.
- `splice` is like **tearing pages out** (and maybe gluing new ones in). The book itself changes.

	<code>slice(2, 4)</code>	<code>splice(2, 4)</code>
Changes the original array?	No	Yes

	<code>slice(2, 4)</code>	<code>splice(2, 4)</code>
The second number means	where to stop (not included)	how many items to remove
Gives back	a copy of that part	the removed items

Turning an array into text: `join`

`join` glues all the items into one string, with a separator of your choice between them:

```
const words = ["pack", "your", "bags"];
console.log(words.join(" ")); // prints: pack your bags
console.log(words.join("-")); // prints: pack-your-bags
console.log(words.join(""));  // prints: packyourbags
console.log(words.join());    // prints: pack,your,bags
```

If you don't give a separator, you get commas.

From text to an array: `split` (a recap)

In chapter 06, `split` turned a string into an array. Now you know what that array is. `split` and `join` are opposites:

```
const line = "Tokyo,Paris,Lima";
const cities = line.split(",");
console.log(cities);           // prints: [ 'Tokyo', 'Paris', 'Lima' ]
console.log(cities.length);    // prints: 3
console.log(cities.join(" | ")); // prints: Tokyo | Paris | Lima
```

This is how programs often read simple data, like one row of a spreadsheet saved as text.

Joining two arrays: `concat`

`concat` makes a **new** array with the items of both. The originals stay as they were:

```
const fruits = ["apple", "banana"];
const veggies = ["carrot", "pea"];
const groceries = fruits.concat(veggies);
console.log(groceries); // prints: [ 'apple', 'banana', 'carrot', 'pea' ]
console.log(fruits);    // prints: [ 'apple', 'banana' ]
```

Flipping the order: `reverse`

A chat app stores messages oldest first, but you want to show the newest first:

```
const messages = ["Hi!", "How are you?", "See you at 6"];
messages.reverse();
console.log(messages); // prints: [ 'See you at 6', 'How are you?', 'Hi!' ]
```

Watch out: `reverse` changes the original array. The old order is gone.

To keep the original, reverse a copy instead:

```
const messages = ["Hi!", "How are you?", "See you at 6"];
const newestFirst = messages.slice().reverse();
console.log(newestFirst); // prints: [ 'See you at 6', 'How are you?', 'Hi!' ]
console.log(messages);    // prints: [ 'Hi!', 'How are you?', 'See you at 6' ]
```

`slice()` makes a copy, then `reverse()` flips the copy. That's method chaining, from chapter 06. Newer JavaScript (since 2023) also has `toReversed()`, which gives you a reversed copy in one step. You'll meet its cousin `toSorted()` in chapter 13.

Here's a classic trick that chains a string method and two array methods:

```
const word = "stressed";
const backwards = word.split("").reverse().join("");
console.log(backwards); // prints: desserts
```

`split("")` breaks the word into an array of letters, `reverse()` flips that array, and `join("")` glues the letters back together.

Which methods change the array?

This matters more than it looks. Changing an array by accident is a very common bug.

Changes the original array	Leaves it alone (gives back something new)
<code>push</code> , <code>pop</code> , <code>shift</code> , <code>unshift</code>	<code>slice</code> , <code>concat</code> , <code>join</code>
<code>splice</code> , <code>reverse</code>	<code>includes</code> , <code>indexOf</code> , <code>at</code>
setting an item, like <code>seats[1] = "taken"</code>	<code>toReversed</code>

Looping over an array

This is where arrays really pay off. A loop can visit every item, however long the list is.

A `for` loop with an index:

```
const scores = [72, 88, 95];

for (let i = 0; i < scores.length; i++) {
  console.log(`Player ${i + 1}: ${scores[i]} points`);
}
```

You'll see:

```
Player 1: 72 points
Player 2: 88 points
Player 3: 95 points
```

The loop starts at `0`, the first index, and keeps going while `i < scores.length`. With 3 items, `i` goes 0, 1, 2, and then the loop stops. We print `i + 1` because humans count from 1.

A `for...of` loop:

In [chapter 08](#) you used `for...of` to step through the letters of a string. It works on arrays too, and it's simpler when you only need the items:

```
const scores = [72, 88, 95];

for (const score of scores) {
  console.log(`Score: ${score}`);
}
```

You'll see:

```
Score: 72
Score: 88
Score: 95
```

Which one should you use?

Loop	Use it when
<code>for...of</code>	you only need each item (most of the time)

Loop	Use it when
<code>for</code> with an index	you also need the position, or you want to change items, like <code>scores[i] = 0</code>

Three loop patterns you'll use all the time

1. Adding everything up. Start a running total at `0`, then add each item to it. Wrap it in a [function](#) and it works for any list:

```
function sumAll(numbers) {
  let total = 0;
  for (const number of numbers) {
    total += number;
  }
  return total;
}

console.log(sumAll([4.5, 12, 3.25])); // prints: 19.75
console.log(sumAll([10, 20, 30]));    // prints: 60
console.log(sumAll([]));               // prints: 0
```

An array is a value like any other, so you can pass it into a function (and return one, too).

2. Finding the biggest. Assume the first item is the biggest. Then check every item, and whenever you find a bigger one, remember that one instead:

```
const temperatures = [18, 24, 21, 27, 19];
let highest = temperatures[0];

for (const temp of temperatures) {
  if (temp > highest) {
    highest = temp;
  }
}

console.log(`Hottest day: ${highest}°C`); // prints: Hottest day: 27°C
```

3. Building a new array. Start with an empty array, and `push` only the items you want to keep:

```
const ages = [12, 19, 15, 21, 30];
const adults = [];

for (const age of ages) {
  if (age >= 18) {
    adults.push(age);
  }
}
console.log(adults); // prints: [ 19, 21, 30 ]
```

The original `ages` array doesn't change.

In [chapter 13](#) you'll learn shorter ways to write these patterns, like `filter` for the last one. Knowing the loop versions first means you'll understand what those shortcuts are doing for you.

Mixed types and nested arrays

An array can hold any type from [chapter 03](#), even a mix:

```
const profile = ["Maya", 28, true, null];
console.log(profile); // prints: [ 'Maya', 28, true, null ]
```

That works, but quick: what does `profile[2]` mean? Hard to say. Most of the time, keep one kind of thing per array, like a list of names or a list of prices. For a bundle of different facts about one thing, the objects in [chapter 11](#) are a much better fit.

An array can even hold other arrays. That's called a **nested array**, and it's perfect for grids, like a tic-tac-toe board:

```
const board = [
  ["X", "O", "X"],
  ["O", "X", "O"],
  [" ", " ", "X"]
];

console.log(board[0][2]); // prints: X
board[2][0] = "O";

for (const row of board) {
  console.log(row.join(" | "));
}
```

You'll see:


```
X
X | O | X
O | X | O
O |   | X
```

`board[0]` is the first row, which is itself an array. `board[0][2]` is the item at index 2 of that row. Read it as "row 0, column 2".

Is it an array? `Array.isArray`

Remember `typeof` from chapter 03? It gives a surprising answer for arrays:

```
const colors = ["red", "green"];
console.log(typeof colors);           // prints: object
console.log(Array.isArray(colors));   // prints: true
console.log(Array.isArray("red"));    // prints: false
```

Arrays are a special kind of object (you'll learn about objects in chapter 11), so `typeof` can't tell them apart. When you need to know "is this a list?", use `Array.isArray`.

Common mistakes

1. Reading past the end of the array

```
const podium = ["gold", "silver", "bronze"];
console.log(podium[3]); // prints: undefined
```

There's no item at index 3, but JavaScript doesn't give you an error. It quietly hands you `undefined`, and the bug shows up somewhere else later. The last index is `length - 1`, which is 2 here. When you want the last item, use `podium.at(-1)`.

2. Going one step too far in a loop

```
const podium = ["gold", "silver", "bronze"];
for (let i = 0; i <= podium.length; i++) {
  console.log(podium[i]);
}
```

You'll see:

```
gold
silver
bronze
undefined
```

`<=` lets `i` reach 3, one step past the last index. This is called an **off-by-one error**, and every programmer makes it sometimes. Fix: use `i < podium.length`. Or use `for...of`, which can't run off the end.

3. Thinking `const` means the array can't change

```
const cart = ["apple"];
cart.push("pear");
console.log(cart); // prints: [ 'apple', 'pear' ]
```

This works, and it's supposed to. `const` only stops you from putting a *different* array into the variable (`cart = []` gives `TypeError: Assignment to constant variable.`). The items inside can still change. So `const` is still the right choice for most arrays.

4. Changing an array by accident

```
const ranking = ["Ana", "Ben", "Cara"];
const lastToFirst = ranking.reverse();
console.log(ranking); // prints: [ 'Cara', 'Ben', 'Ana' ]
```

You wanted a reversed copy, but `reverse` flipped the original too. The same goes for `splice`: if you only wanted to *look at* some items, `splice` removes them for real. Check the "Which methods change the array?" table above. To keep the original, use `slice`, or work on a copy: `ranking.slice().reverse()`.

5. Checking for an empty array the wrong way

```
const cart = [];
if (cart) {
  console.log("You have items in your cart!");
}
// prints: You have items in your cart!
```

An empty array is **truthy**. It isn't on chapter 07's list of falsy values, so `if (cart)` is true for *every* array, even an empty one. Check the length instead: `if (cart.length > 0)`. And don't try `cart === []`. It's always `false`, for reasons chapter 16 explains.

Quick recap

- An array is an ordered list in one variable: `const days = ["Mon", "Tue", "Wed"];`
 - Indexes start at 0. The last item is at `length - 1`, or use `.at(-1)`.
 - `push` and `pop` work at the end, `unshift` and `shift` at the start. `splice` changes the middle, while `slice` copies part without changing anything.
 - `includes` and `indexOf` find things. `join` turns an array into a string, and `split` does the opposite.
 - Loop with `for...of` when you need the items, or with a `for` loop when you also need the index.
 - A `const` array can still be changed. It just can't be replaced with a different array.
 - Reading past the end gives `undefined`, not an error.
-

Next: try the [exercises](#), then move on to [11 Objects](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Weekly shopping list

You keep your weekly shopping list in an array.

1. Create an array called `shoppingList` with `"rice"` , `"lentils"` , `"spinach"` and `"yogurt"` .
2. Print the whole list.
3. Print the first item, the last item, and how many items there are.
4. You remembered you need tea. Add `"tea"` to the end.
5. You already have rice at home. Remove the first item.
6. Print the list again.

Expected output:

```
[ 'rice', 'lentils', 'spinach', 'yogurt' ]  
First: rice  
Last: yogurt  
Items: 4  
[ 'lentils', 'spinach', 'yogurt', 'tea' ]
```

Rule: don't type `rice`, `yogurt` or `4` into your `console.log` lines. Read them from the array.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Road trip playlist

A friend made a road trip playlist, and it needs a few changes. Start with:

```
const playlist = ["Sunrise", "Highway", "Old Town", "Neon Lights", "Coastline"];
```

1. Nobody likes "Old Town". Find where it is with `indexOf`, then remove it with `splice`. Print which song was removed.
2. Insert "Mountain Air" at index 1, between "Sunrise" and "Highway".
3. Print the first three songs, separated by a comma and a space.
4. Print whether the playlist has "Coastline" in it.
5. Print which track number "Neon Lights" is. Track numbers start at 1, not 0.
6. Print the whole playlist with `->` between the songs.

Expected output:

```
Removed: Old Town  
Up next: Sunrise, Mountain Air, Highway  
Has Coastline? true  
Neon Lights is track 4  
Sunrise -> Mountain Air -> Highway -> Neon Lights -> Coastline
```

Rule: don't type the index `2` for "Old Town" yourself. Let `indexOf` find it.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Test score report

A teacher has the test scores for a class and wants a quick report:

```
const scores = [78, 92, 45, 67, 88, 38, 99, 74];
```

Print how many students there are, the average (to 1 decimal place), the highest and lowest scores, how many passed (a pass is 50 or more), and a list of the scores that need a retake.

Expected output:

```
Students: 8
Average: 72.6
Highest: 99
Lowest: 38
Passed: 6
Needs a retake: [ 45, 38 ]
```

Rule: use loops, and don't work anything out by hand. If the teacher adds a new score to the array, your report should still be right.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Hashtag maker

A social media app turns a phrase into a hashtag. Write a function `makeHashtag(phrase)` that returns the hashtag:

- Every word starts with a capital letter, and the rest of the word is lowercase.
- There are no spaces, and there's a `#` at the front.
- Extra spaces (at the ends or between words) must not cause problems.

Test it with:

```
console.log(makeHashtag("summer sale starts now"));
console.log(makeHashtag(" learn  JAVASCRIPT today "));
console.log(makeHashtag("pizza"));
```

Expected output:

```
#SummerSaleStartsNow
#LearnJavascriptToday
#Pizza
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Cinema seat booking

A small cinema has 3 rows of 5 seats. "0" means free and "x" means taken. Start with:

```
const seats = [
  ["0", "0", "X", "0", "0"],
  ["X", "X", "0", "0", "0"],
  ["0", "0", "0", "0", "X"]
];
const rowNames = ["A", "B", "C"];
```

Customers ask for seats like "B3": row B, seat 3. Seat numbers start at 1, like in a real cinema. Write three functions:

1. `printSeats(seats)` prints each row with its letter, like `A: 0 0 x 0 0`.
2. `countFreeSeats(seats)` returns how many seats are free.
3. `bookSeat(seats, rowName, seatNumber)` books a seat and **returns** a message:
 - If the seat doesn't exist: `Seat D1 doesn't exist.`
 - If it's already taken: `Sorry, B3 is already taken.`
 - Otherwise, it marks the seat as taken and returns: `Booked B3. Enjoy the film!`

Test it with:

```
printSeats(seats);
console.log(`Free seats: ${countFreeSeats(seats)}`);
console.log(bookSeat(seats, "B", 3));
console.log(bookSeat(seats, "B", 3));
console.log(bookSeat(seats, "D", 1));
console.log(bookSeat(seats, "A", 6));
console.log(bookSeat(seats, "C", 5));
printSeats(seats);
console.log(`Free seats: ${countFreeSeats(seats)}`);
```

Expected output:

```
A: 0 0 X 0 0
B: X X 0 0 0
C: 0 0 0 0 X
Free seats: 11
Booked B3. Enjoy the film!
Sorry, B3 is already taken.
Seat D1 doesn't exist.
Seat A6 doesn't exist.
Sorry, C5 is already taken.
A: 0 0 X 0 0
B: X X X 0 0
C: 0 0 0 0 X
Free seats: 10
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 3, the scores had no names attached. What if the teacher wanted `Maya: 78`? You could keep a second array of names and hope the two lists always line up, but that gets messy fast.

[Chapter 11: Objects](#) gives you a much better way to keep related information together. 😊

11 Objects

What is it?

An **object** groups related information under one name. Each piece of information has a label, so you always know what it means.

```
const contact = {  
  name: "Maya Patel",  
  phone: "555-0142",  
  age: 28  
};
```

Each labeled piece is called a **property**. A property has a **key** (the label, like `phone`) and a **value** (what's stored under that label, like `"555-0142"`).

Why does it matter?

Say you're saving a contact in your phone. With an array from [chapter 10](#), it would look like this:

```
const contact = ["Maya Patel", "555-0142", 28];
```

It's all in one place, but what does `contact[2]` mean? You'd have to remember that index 2 is the age. Mix up the order and nothing warns you.

An object gives every piece a name:

```
const contact = { name: "Maya Patel", phone: "555-0142", age: 28 };  
console.log(contact.phone); // prints: 555-0142
```

`contact.phone` says exactly what it is. Almost all real data looks like this: a user profile, a product in a shop, an order, a song, the weather data an app downloads. Objects are everywhere in JavaScript.

Real-world example

An object is like a **contact card**:

Contact card	Object
The whole card	The object

Contact card	Object
A field label, like "Phone"	A key
What's written next to it	A value
One line of the card (label plus what's written)	A property
Adding a new line, like "Birthday"	Adding a property

Arrays and objects are a great team, but they answer different questions:

	Array (chapter 10)	Object
It's like a...	numbered list, like train carriages	form with labeled fields
You find things by	position: <code>songs[0]</code>	name: <code>song.title</code>
Best for	many things of the same kind	many facts about one thing

How it works

Creating an object

Write the properties inside curly braces `{ }`. Each one is `key: value`, with commas between them:

```
const contact = {
  name: "Maya Patel",
  phone: "555-0142",
  age: 28,
  isFavorite: true
};
console.log(contact);
// prints: { name: 'Maya Patel', phone: '555-0142', age: 28, isFavorite: true }
```

- Values can be any type: strings, numbers, booleans, arrays, even other objects.
- Keys without quotes follow the same naming rules as variables (chapter 02). Use camelCase, like `isFavorite`.
- Node prints objects with spaces inside the braces and single quotes around strings, just like arrays. Long objects get split over several lines.

An empty object is `{ }`. You can fill it up later.

If a key really needs a space or a dash, put it in quotes: `{ "favorite color": "blue" }`. Most of the time, a camelCase key like `favoriteColor` is simpler.

Reading a property: dot notation

Write the object's name, a dot, and the key:

```
const contact = { name: "Maya Patel", phone: "555-0142", age: 28 };
console.log(contact.name); // prints: Maya Patel
console.log(contact.age);  // prints: 28
console.log(contact.email); // prints: undefined
```

Asking for a property that doesn't exist isn't an error. You get `undefined`, just like reading past the end of an array in chapter 10.

Bracket notation: when the key is in a variable

You can also put the key in square brackets, as a string:

```
const contact = { name: "Maya Patel", phone: "555-0142" };
console.log(contact["phone"]); // prints: 555-0142
```

That looks like extra typing for nothing. The real power is that the brackets can hold a **variable**, so the key can be decided while the program runs:

```
const ticketPrices = { adult: 12, child: 7, senior: 8 };
const ticketType = "child"; // imagine the customer picked this

console.log(ticketPrices[ticketType]); // prints: 7
```

JavaScript first looks inside `ticketType` and finds `"child"`. Then it reads `ticketPrices["child"]`, which is `7`.

An object used like this is a **lookup table**: you ask with a key and get back the matching value. One line like this can replace a long `if...else if` chain from [chapter 07](#).

Use	When
A dot: <code>ticketPrices.adult</code>	you know the key while you write the code (most of the time)
Brackets: <code>ticketPrices[ticketType]</code>	the key is in a variable, or it has spaces or dashes

Adding, changing, and deleting properties

Objects aren't frozen. You can add, change, and remove properties whenever you like:

```
const profile = { username: "sam_k", city: "Oslo" };

profile.bio = "Coffee and code"; // add: this key didn't exist yet
profile.city = "Bergen";         // change: this key already exists
console.log(profile); // prints: { username: 'sam_k', city: 'Bergen', bio: 'Coffee and code' }

delete profile.bio; // remove the property completely
console.log(profile); // prints: { username: 'sam_k', city: 'Bergen' }
```

Adding and changing look exactly the same: `object.key = value`. If the key is new, the property gets added. If the key already exists, its value gets replaced. Brackets work here too: `profile[fieldName] = "something"`.

`const` objects can still change

Did you notice that `profile` is a `const`? It works just like the arrays in [chapter 10](#): a `const` must always hold the *same* object, but that object's properties can change. What you can't do is swap in a whole new object:

```
const profile = { username: "sam_k" };
profile.username = "sam_k_99"; // fine: changing a property
profile = { username: "new" }; // not allowed: a different object
// TypeError: Assignment to constant variable.
```

Does this property exist?

Sometimes you need to know whether an object has a property at all. There are three common ways to check:

```
const product = { name: "Desk lamp", price: 25, stock: 0 };

console.log(product.color === undefined); // prints: true
console.log("stock" in product);           // prints: true
console.log(Object.hasOwn(product, "stock")); // prints: true
console.log(Object.hasOwn(product, "color")); // prints: false
```

Check	What it asks	Watch out
<code>product.color === undefined</code>	"Is there no value here?"	A property can exist and still hold <code>undefined</code> .

Check	What it asks	Watch out
<code>"color" in product</code>	"Does this key exist?"	It also finds a few built-in extras that every object has: <code>"toString" in product</code> is <code>true</code> !
<code>Object.hasOwn(product, "color")</code>	"Does <i>this object itself</i> have this key?"	It's newer (added in 2022), but it works in all current browsers and Node.

`Object.hasOwn` is the most precise, so it's a good default. (Where extras like `toString` come from is a story for chapter 28.)

Watch out: `if (product.stock)` asks "is the value truthy?", not "does it exist?". A stock of `0` is falsy (chapter 07), so that `if` would skip a product that really does have a `stock` property.

Methods: functions inside objects

A property's value can be a function. A function stored in an object is called a **method**. You've been using one since chapter 01: `console` is an object, and `log` is its method.

```
const account = {
  owner: "Lena",
  balance: 100,
  deposit(amount) {
    this.balance += amount;
    console.log(`${this.owner} now has $$${this.balance}`);
  }
};

account.deposit(50); // prints: Lena now has $150
account.deposit(25); // prints: Lena now has $175
```

`deposit(amount) { ... }` is the short way to write a method. In older code you'll also see the long way, `deposit: function (amount) { ... }`. Both do the same thing.

What's `this`? Inside a method, `this` means "the object this method was called on", which is the object before the dot. In `account.deposit(50)`, `this` is `account`, so `this.balance` is `account.balance`.

Why not write `account.balance` directly? Because `this` keeps working if you rename the variable, or when you build many similar objects (you'll do that with classes in chapter 27). `this` has a few surprises of its own, and chapter 26 tells the full story. For now, remember: in a method, `this` means "this object".

Objects inside objects

A value can be another object, or an array. That's how most real data is shaped. Here's an order from a food delivery app:

```
const order = {
  id: 1042,
  customer: {
    name: "Tom",
    address: { city: "Leeds", postcode: "LS1 4AP" }
  },
  items: ["pizza", "cola"]
};

console.log(order.customer.name);           // prints: Tom
console.log(order.customer.address.city);   // prints: Leeds
console.log(order.items[1]);                // prints: cola
console.log(order.items.length);            // prints: 2
```

Read a chain of dots from left to right, one step at a time: `order`, then its `customer`, then their `address`, then its `city`.

Tip: when Node prints an object nested more than two levels deep, it shortens the deepest parts to `[Object]`. Your data is all still there. Node just doesn't print all of it.

Arrays of objects

This is one of the most common shapes of data in all of programming: a list where every item is an object. A shop's products, a playlist's songs, a class's students.

```
const products = [
  { name: "Notebook", price: 3.5, inStock: true },
  { name: "Pen", price: 1.2, inStock: false },
  { name: "Backpack", price: 24.99, inStock: true }
];

console.log(products[2].name); // prints: Backpack

for (const product of products) {
  if (product.inStock) {
    console.log(`${product.name}: ${product.price.toFixed(2)}`);
  }
}
```

You'll see:

```
Backpack
Notebook: $3.50
Backpack: $24.99
```

`products[2]` is a whole object, so `products[2].name` gets its name. Inside the loop, `product` is one whole object each time around.

Finding one item by a property is a job you'll do all the time, so it's worth a function:

```
const products = [
  { name: "Notebook", price: 3.5, inStock: true },
  { name: "Pen", price: 1.2, inStock: false }
];

function findProduct(name) {
  for (const product of products) {
    if (product.name === name) {
      return product; // found it: stop looking and hand it back
    }
  }
  return null; // checked every product, and none matched
}

console.log(findProduct("Pen")); // prints: { name: 'Pen', price: 1.2, inStock: false }
console.log(findProduct("Lamp")); // prints: null
```

When nothing matches, the function returns `null`, which (from chapter 03) means "empty, on purpose". Chapter 13 shows a shorter way to do this, with `find`.

Looping over an object

`for...in` walks through an object's keys:

```
const scores = { math: 90, art: 75, music: 82 };

for (const subject in scores) {
  console.log(`${subject}: ${scores[subject]}`);
}
```

You'll see:

```
math: 90
art: 75
music: 82
```

Each time around, `subject` holds one key, as a string. To get the value, use brackets: `scores[subject]`. It has to be brackets, because the key is in a variable.

Don't mix up the two loops:

- `for...of` gives you the **items** of an array (or the letters of a string).
- `for...in` gives you the **keys** of an object.

`Object.keys`, `Object.values` and `Object.entries` turn an object into an array, so you can use everything from chapter 10 on it:

```
const scores = { math: 90, art: 75, music: 82 };

console.log(Object.keys(scores));      // prints: [ 'math', 'art', 'music' ]
console.log(Object.values(scores));    // prints: [ 90, 75, 82 ]
console.log(Object.entries(scores));   // prints: [ [ 'math', 90 ], [ 'art', 75 ], [ 'music',
82 ] ]
console.log(Object.keys(scores).length); // prints: 3
```

- `Object.keys` gives you the keys. `Object.keys(obj).length` counts the properties.
- `Object.values` gives you the values, which is handy for adding them up.
- `Object.entries` gives you `[key, value]` pairs: little arrays with two items each. Chapter 15 shows a neat way to unpack them.

For example, adding up all the scores:

```
const scores = { math: 90, art: 75, music: 82 };
let total = 0;

for (const score of Object.values(scores)) {
  total += score;
}
console.log(`Total: ${total}`); // prints: Total: 247
```

Optional chaining: `?.`

What happens if you reach into something that isn't there?

```
const user = { name: "Ravi" }; // no address yet

console.log(user.address); // prints: undefined
console.log(user.address.city);
// TypeError: Cannot read properties of undefined (reading 'city')
```

`user.address` is `undefined`, and you can't read a property of `undefined`, so the program crashes. This is one of the most common errors in JavaScript. Learn to recognize that message.

Optional chaining `?.` fixes it. It means "if the thing on the left is missing, stop here and give back `undefined` instead of crashing":

```
const user = { name: "Ravi" };

console.log(user.address?.city); // prints: undefined
console.log(user.address?.city ?? "No city yet"); // prints: No city yet
```

Pair it with `??` from chapter 07 to give a friendly default. "Missing" here means `null` or `undefined`, the same two values `??` looks for.

Use `?.` only where something is *allowed* to be missing, like an address a new user hasn't filled in yet. If a value should always be there, a crash points you to a real bug. Hiding it with `?.` just makes the bug harder to find.

Shorthand properties

When a variable has the same name as the key you want, you only need to write it once:

```
const title = "Night Drive";
const artist = "The Owls";

const track = { title, artist }; // same as { title: title, artist: artist }
console.log(track); // prints: { title: 'Night Drive', artist: 'The Owls' }
```

You'll see this all the time in functions that build objects:


```
function createTrack(title, artist, seconds) {
  return { title, artist, seconds };
}

const playlist = [
  createTrack("Night Drive", "The Owls", 214),
  createTrack("Rainy Day", "Juniper", 187)
];
console.log(playlist[1]); // prints: { title: 'Rainy Day', artist: 'Juniper', seconds: 187 }
```

Common mistakes

1. Using a dot when the key is in a variable

```
const ticketPrices = { adult: 12, child: 7 };
const ticketType = "child";
console.log(ticketPrices.ticketType); // prints: undefined
```

`ticketPrices.ticketType` looks for a key that's literally called "ticketType", and there isn't one. Fix: `ticketPrices[ticketType]`.

2. A typo in a property name

```
const product = { name: "Lamp", price: 25 };
console.log(product.prixe); // prints: undefined
console.log(product.prixe.toFixed(2));
// TypeError: Cannot read properties of undefined (reading 'toFixed')
```

A misspelled key doesn't cause an error by itself. It quietly gives you `undefined`, and the crash comes later, when you try to use it. When you see `Cannot read properties of undefined`, look at whatever comes *before* the dot. Is it spelled right? Does it really exist?

3. Using `for...of` on an object

```
const scores = { math: 90, art: 75 };
for (const score of scores) {
  console.log(score);
}
// TypeError: scores is not iterable
```

Iterable means "can be stepped through with `for...of`". Arrays and strings are iterable, but plain objects aren't. Use `for...in`, or loop over `Object.keys(scores)`, `Object.values(scores)` or `Object.entries(scores)`.

4. Writing `=` instead of `:`, or forgetting a comma

```
const pet = { name = "Rex" };  
// SyntaxError: Invalid shorthand property initializer
```

```
const pet = {  
  name: "Rex"  
  age: 3  
};  
// SyntaxError: Unexpected identifier 'age'
```

Inside an object, it's always `key: value`, and every property except the last needs a comma after it. The second error points at `age`, because that's where JavaScript noticed something was wrong. The missing comma is on the line before.

5. Using an arrow function for a method that needs `this`

```
const dog = {  
  name: "Rex",  
  bark: () => {  
    console.log(`${this.name} says woof`);  
  }  
};  
dog.bark(); // prints: undefined says woof
```

Arrow functions don't get their own `this`, so here `this` isn't the `dog` object. (That's the Node output; a browser may print something slightly different, but it's just as wrong.) Fix: use the method shorthand, `bark() { ... }`. Chapter 26 explains exactly why this happens.

Quick recap

- An object groups related values under labels: `{ name: "Maya", age: 28 }`. Each `key: value` pair is a property.
- Read a property with a dot (`user.name`), or with brackets when the key is in a variable (`prices[size]`).
- Add or change a property with `obj.key = value`, and remove one with `delete obj.key`. A `const` object can still change.
- A missing property gives `undefined`. Check whether a key exists with `Object.hasOwn(obj, "key")` or `"key" in obj`.
- A method is a function inside an object. Inside a method, `this` means "this object".
- Loop over an object with `for...in` or `Object.keys`, `Object.values` and `Object.entries`. Arrays of objects are everywhere.
- `?.` stops safely when something might be missing, and pairs well with `??`.

Next: try the [exercises](#), then move on to [12 Project: Shopping Cart](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Run each one with `node ex1.js` from a terminal opened in this folder.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Contact card

You're building the contacts screen of a phone app.

1. Create an object called `contact` with a `name` of "Leo Martins", a `phone` of "555-0199" and a `city` of "Lisbon".
2. Print a sentence saying where Leo lives, and then his phone number.
3. Leo just set up email. Add an `email` property: "leo@example.com".
4. He got a new phone number: change `phone` to "555-0123".
5. He'd rather not share his city. Remove the `city` property.
6. Print the whole object, and then how many properties (fields) it has.

Expected output:

```
Leo Martins lives in Lisbon.  
Phone: 555-0199  
{ name: 'Leo Martins', phone: '555-0123', email: 'leo@example.com' }  
Fields: 3
```

Rule: read the name, city, and phone number from the object. Don't type them into your `console.log` lines.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Cinema tickets

A cinema keeps its ticket prices in a lookup table. A family is buying tickets, and one of them is asking for a type of ticket that doesn't exist:

```
const ticketPrices = { adult: 12, child: 7, senior: 8, student: 9 };
const order = ["adult", "child", "child", "student", "pet"];
```

Go through the order. For each ticket, print its price, or a friendly message if there's no such ticket. Then print the total.

Expected output:

```
adult: $12
child: $7
child: $7
student: $9
Sorry, there's no "pet" ticket.
Total: $35
```

Rule: no `if` or `switch` checking each ticket type by name. The lookup table should do that work for you.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Library catalog

A small library keeps its books in an array of objects:

```
const books = [
  { title: "Dune", author: "Frank Herbert", year: 1965, available: true },
  { title: "Beloved", author: "Toni Morrison", year: 1987, available: false },
  { title: "The Hobbit", author: "J.R.R. Tolkien", year: 1937, available: true },
  { title: "Things Fall Apart", author: "Chinua Achebe", year: 1958, available: false }
];
```

1. Print every book on one line, with `available` or `checked out` at the end.
2. Print how many books are available, out of the total.
3. Write a function `findBook(title)` that returns the matching book object, or `null` if there isn't one.
4. Use `findBook` to print the author of "Dune", and then the author of "Emma". There's no "Emma" in the library, so print `Not found` instead, without an `if`.

Expected output:

```
"Dune" by Frank Herbert (1965) - available
"Beloved" by Toni Morrison (1987) - checked out
"The Hobbit" by J.R.R. Tolkien (1937) - available
"Things Fall Apart" by Chinua Achebe (1958) - checked out
Available: 2 of 4
Frank Herbert
Not found
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Lunch poll

The office is voting on what to order for lunch. Each vote is one string:

```
const votes = ["pizza", "tacos", "pizza", "sushi", "tacos", "pizza", "curry"];
```

1. Count the votes into an object called `counts`, where each key is a food and each value is its number of votes. Print `counts`.
2. Print each food with its votes. Use `vote` for 1 and `votes` for anything else.
3. Print the winner.

Expected output:

```
{ pizza: 3, tacos: 2, sushi: 1, curry: 1 }
pizza: 3 votes
tacos: 2 votes
sushi: 1 vote
curry: 1 vote
Winner: pizza with 3 votes
```

Rule: your code must work for *any* list of votes. Don't type any food names into your code (except in the `votes` array).

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Playlist with methods

Build a music playlist as one object with methods. Start with this:

```
const playlist = {
  name: "Road Trip",
  tracks: []
  // your methods go here
};
```

1. Write a normal function (outside the object) called `formatTime(totalSeconds)` that turns seconds into minutes and seconds: 214 becomes "3:34", and 187 becomes "3:07".
2. Add a method `addTrack(title, artist, seconds)` that adds a track object to `tracks`. Use shorthand properties.
3. Add a method `getTotalSeconds()` that returns the length of the whole playlist, in seconds.
4. Add a method `print()` that prints the playlist's name, number of tracks and total time, and then each track, numbered from 1.

Test it with:

```
playlist.addTrack("Night Drive", "The Owls", 214);
playlist.addTrack("Rainy Day", "Juniper", 187);
playlist.addTrack("Open Road", "Sam Rivers", 245);
playlist.addTrack("Coastline", "The Owls", 199);
console.log(playlist.tracks[0]);
playlist.print();
```

Expected output:

```
{ title: 'Night Drive', artist: 'The Owls', seconds: 214 }
Road Trip (4 tracks, 14:05)
1. Night Drive - The Owls (3:34)
2. Rainy Day - Juniper (3:07)
3. Open Road - Sam Rivers (4:05)
4. Coastline - The Owls (3:19)
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Look at what you can do now: store lists, group information into objects, loop over both, make decisions, and wrap it all up in functions.

That's everything you need for your first real project. In [chapter 12](#), you'll put it all together and build a shopping cart. 🛒

12 Project: Shopping Cart

This is your first real project. You'll build the checkout for a small online shop, using only what you learned in chapters 01 to 11.

Take your time. Build it one milestone at a time, and run your code after every small change. That's how professionals work too.

What you'll build

You'll write a Node program in a single file, `cart.js`, for a stationery shop called Maple Stationery. It will:

- show a menu of products,
- let a customer add items to a cart, and politely refuse bad requests,
- work out the subtotal, apply discount codes, and add tax and shipping,
- let the customer change quantities and remove items,
- print a neat receipt.

When it's finished, your program will end by printing this receipt:

```
Maple Stationery - Receipt
=====
ITEM            QTY    AMOUNT
Notebook        5     $17.50
Backpack        1     $34.99
Sticky notes    2      $3.98
-----
Subtotal                $56.47
Discount (SAVE10)      -$5.65
Tax (8%)                $4.07
Shipping               FREE
-----
TOTAL                  $54.89
=====
Thank you for shopping with us!
```

Your program won't read from the keyboard yet (that comes in chapter 49). Instead, you'll "play the customer" by calling your own functions at the bottom of the file, like `addToCart(1, 3)`.

Before you start

Setup

Create a file called `cart.js` in this folder. Run it with `node cart.js` from a terminal opened in this folder, just like the exercises.

The rules

- Use only what you learned in chapters 01 to 11. If you already know `map`, `filter`, `reduce`, destructuring, spread or classes from somewhere else, save them for later chapters. Here, loops and plain functions are the point.
- Each milestone has hints. Try first, and open a hint only when you're stuck.
- Your code doesn't have to look like anyone else's. If your output matches the expected output, it works. When you're done, ask Claude to review it.

How to organize `cart.js`

Keep your file in three parts, from top to bottom:

```
// ===== Data =====  
// The products, the cart, and the shop's settings.  
  
// ===== Functions =====  
// One function for each job.  
  
// ===== Try it out =====  
// Calls to your functions, to test them.
```

Each milestone gives you some **test code** for the "Try it out" part. Replace the old test code with the new code, run `node cart.js`, and compare your output with the expected output. Your data and functions stay and keep growing. Only the test code changes.

Money: work in cents

Remember the `0.1 + 0.2` surprise from [chapter 05](#)? It happens in shops too:

```
console.log(1.1 * 3); // prints: 3.3000000000000003  
console.log(110 * 3); // prints: 330
```


Three notebooks at \$1.10 should cost exactly \$3.30. Whole numbers don't have this problem, so real shops often store money as whole **cents**: \$3.50 becomes `350`. You'll do the same:

- Every price and total in your program is a whole number of cents.
- When a calculation could give you part of a cent (like 10% off), round it with `Math.round`.
- Only turn cents into dollars at the very end, when you print.

Milestone 1: The product catalog

Goal: store the shop's products and print a menu.

Here's what Maple Stationery sells:

ID	Product	Price
1	Notebook	\$3.50
2	Pen set	\$6.25
3	Backpack	\$34.99
4	Water bottle	\$12.00
5	Desk lamp	\$22.50
6	Sticky notes	\$1.99

1. In the Data part, create an array of objects called `products`, one object per product. Use exactly these property names, and prices in cents:

```
const products = [  
  { id: 1, name: "Notebook", priceInCents: 350 },  
  // ...and the other five products  
];
```

2. Write a function `formatMoney(cents)` that **returns** a price as text, like `"$3.50"`.
3. Write a function `printMenu()` that prints a welcome line and then every product.

Test code:

```
console.log(formatMoney(350));  
console.log(formatMoney(1200));  
console.log(formatMoney(5));  
printMenu();
```

You'll see:

```
$3.50
$12.00
$0.05
Welcome to Maple Stationery!
1. Notebook - $3.50
2. Pen set - $6.25
3. Backpack - $34.99
4. Water bottle - $12.00
5. Desk lamp - $22.50
6. Sticky notes - $1.99
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 2: Add to the cart

Goal: write `addToCart(productId, quantity)`, which puts products in the cart and refuses bad requests.

Think of the cart as a note you hand to a cashier: "product 1, three of them. Product 3, one of them." The note doesn't need the names or prices, because the cashier looks those up in the catalog.

So add `const cart = [];` to the Data part. Each item in the cart is an object with just two properties: `productId` and `quantity`.

`addToCart` follows these rules, in this order:

1. If there's no product with that ID, print `Sorry, we don't have a product with ID 42.` (with the real ID) and stop.
2. If the quantity isn't a whole number of 1 or more, print `Sorry, the quantity must be a whole number of 1 or more.` and stop.
3. If that product is already in the cart, add to its quantity. Otherwise, add a new item to the cart.
4. Print `Added 3 x Notebook to your cart.` (with the real quantity and name).

Test code:

```
addToCart(1, 3);
addToCart(3, 1);
addToCart(1, 2);
addToCart(42, 1);
addToCart(2, 0);
addToCart(2, 1.5);
console.log(cart);
```

You'll see:

```
Added 3 x Notebook to your cart.  
Added 1 x Backpack to your cart.  
Added 2 x Notebook to your cart.  
Sorry, we don't have a product with ID 42.  
Sorry, the quantity must be a whole number of 1 or more.  
Sorry, the quantity must be a whole number of 1 or more.  
[ { productId: 1, quantity: 5 }, { productId: 3, quantity: 1 } ]
```

Notice there's only one notebook item, with a quantity of 5. Adding two more notebooks updated the item that was already in the cart.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Milestone 3: The subtotal

Goal: add up the cart, and show what's in it.

1. Write `getSubtotal()`, which **returns** the total of everything in the cart, in cents. For each item, that's the product's price times the quantity.
2. Write `printCart()`, which prints each item with its amount, and then the subtotal. If the cart is empty, it prints `Your cart is empty.` instead.

Test code:

```
printCart();  
addToCart(1, 5);  
addToCart(3, 1);  
addToCart(6, 2);  
printCart();  
console.log(getSubtotal());
```

You'll see:

```
Your cart is empty.
Added 5 x Notebook to your cart.
Added 1 x Backpack to your cart.
Added 2 x Sticky notes to your cart.
Your cart:
- 5 x Notebook: $17.50
- 1 x Backpack: $34.99
- 2 x Sticky notes: $3.98
Subtotal: $56.47
5647
```

The last line is the plain number that `getSubtotal()` returns: 5647 cents, which is \$56.47.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 4: Discount codes

Goal: let customers use a discount code.

Maple Stationery has two codes:

Code	What it does
SAVE10	10% off the subtotal, rounded to the nearest cent
FLAT5	\$5.00 off, but only when the subtotal is over \$30.00

1. Write `getDiscount(subtotal, code)`, which **returns** the discount in cents. An unknown code, no code at all (`null`), or a `FLAT5` order of \$30.00 or less all give `0`.
2. Add `let discountCode = null;` to the Data part, because the customer hasn't used a code yet.
3. Write `applyDiscountCode(code)`. Customers type codes in all sorts of ways, so first clean the code up: `"save10"` should work just like `"SAVE10"`. Then:
 - If it's a valid code, store the cleaned-up code in `discountCode`, and print `Discount code SAVE10 applied.`
 - Otherwise, print `Sorry, "HALFOFF" is not a valid discount code.` (with the cleaned-up code).

Test code:

```

console.log(getDiscount(5647, "SAVE10"));
console.log(getDiscount(5647, "FLAT5"));
console.log(getDiscount(2500, "FLAT5"));
console.log(getDiscount(5647, null));
applyDiscountCode("halfoff");
applyDiscountCode(" save10 ");
console.log(discountCode);

```

You'll see:

```

565
500
0
0
Sorry, "HALFOFF" is not a valid discount code.
Discount code SAVE10 applied.
SAVE10

```

10% of 5647 cents is 564.7 cents. Nobody can pay 0.7 of a cent, so it rounds to 565 cents, which is \$5.65.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 5: Tax and shipping

Goal: work out the tax, the shipping, and the final total.

The shop's rules:

- **Tax** is 8% of the order *after* the discount, rounded to the nearest cent. Shipping isn't taxed.
 - **Shipping** costs \$4.99, but it's free when the order after the discount is \$50.00 or more.
1. In the Data part, store the shop's settings in variables with clear names: the tax rate (`0.08`), the shipping fee, and the amount where shipping becomes free (money in cents, as always). If the tax rate changes next year, you'll only have to change one line.
 2. Write `getTax(amount)` and `getShipping(amount)` . Both take an amount in cents and return cents.
 3. Write `getTotals()` , which works everything out for the current cart and **returns an object** with these six properties, in this order: `subtotal` , `discount` , `afterDiscount` , `tax` , `shipping` and `total` . The total is `afterDiscount + tax + shipping` .

Test code:

```
console.log(getTax(5082));
console.log(getTax(1000));
console.log(getShipping(5082));
console.log(getShipping(4999));
addToCart(1, 5);
addToCart(3, 1);
addToCart(6, 2);
applyDiscountCode("SAVE10");
console.log(getTotals());
```

You'll see:

```
407
80
0
499
Added 5 x Notebook to your cart.
Added 1 x Backpack to your cart.
Added 2 x Sticky notes to your cart.
Discount code SAVE10 applied.
{
  subtotal: 5647,
  discount: 565,
  afterDiscount: 5082,
  tax: 407,
  shipping: 0,
  total: 5489
}
```

Node prints this object over several lines because it's too long for one.

Why return one object? Because `getTotals` works everything out once, in one place. In the next milestone, the receipt can then read `totals.tax`, `totals.total` and so on, without doing any math itself.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 6: The receipt

Goal: print a neat receipt with `printReceipt()`.

Test code:

```
addToCart(2, 2);
addToCart(4, 1);
printReceipt();
```

You'll see:

```
Added 2 x Pen set to your cart.
Added 1 x Water bottle to your cart.
Maple Stationery - Receipt
=====
ITEM                QTY    AMOUNT
Pen set             2     $12.50
Water bottle        1     $12.00
-----
Subtotal                        $24.50
Tax (8%)                      $1.96
Shipping                     $4.99
-----
TOTAL                        $31.45
=====
Thank you for shopping with us!
```

The layout rules:

- Every line of the receipt fits in 34 characters. The `=` and `-` lines are exactly 34 characters long.
- **Item rows:** the name is padded to 20 characters, the quantity to 4, and the amount to 10. The quantity and the amount line up on the right.
- **The header row** (`ITEM` , `QTY` , `AMOUNT`) uses the same widths as the item rows.
- **Totals rows:** the label is padded to 24 characters, and the amount to 10, lined up on the right.
- **The discount row** only appears when there's a discount. It shows the code and a minus sign, like `Discount (SAVE10) and -$5.65`.
- **Shipping** shows `FREE` when it costs nothing.

This test didn't use a discount code, so there's no discount row. And the order is under \$50.00, so shipping isn't free. The final program at the end of this guide shows both.

Stuck? [Hint 1](#), [Hint 2](#), [Hint 3](#) and [Hint 4](#) are in the [Hints section](#) at the end.

Milestone 7: Remove items and change quantities

Goal: let customers change their minds.

1. Write `removeFromCart(productId)` :
 - If the product isn't in the cart, print `That item isn't in your cart.`
 - Otherwise, remove it and print `Removed Desk lamp from your cart.` (with the real name).
2. Write `updateQuantity(productId, quantity)` , which follows these rules, in this order:
 1. If the product isn't in the cart, print `That item isn't in your cart.`
 2. If the new quantity is `0` , remove the item. (Your `removeFromCart` can do that.)

3. If the quantity isn't a whole number of 0 or more, print `Sorry, the quantity must be a whole number of 0 or more.`
4. Otherwise, change the quantity and print `Sticky notes quantity changed to 2.` (with the real name and quantity).

Test code:

```
addToCart(1, 3);
addToCart(6, 4);
addToCart(5, 1);
updateQuantity(6, 2);
updateQuantity(1, -1);
updateQuantity(4, 2);
removeFromCart(5);
removeFromCart(5);
updateQuantity(1, 0);
printCart();
```

You'll see:

```
Added 3 x Notebook to your cart.
Added 4 x Sticky notes to your cart.
Added 1 x Desk lamp to your cart.
Sticky notes quantity changed to 2.
Sorry, the quantity must be a whole number of 0 or more.
That item isn't in your cart.
Removed Desk lamp from your cart.
That item isn't in your cart.
Removed Notebook from your cart.
Your cart:
- 2 x Sticky notes: $3.98
Subtotal: $3.98
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Putting it all together

Replace your test code one last time, with a whole shopping trip:


```
printMenu();  
console.log("");  
addToCart(1, 3);  
addToCart(3, 1);  
addToCart(6, 4);  
addToCart(5, 1);  
addToCart(1, 2);  
addToCart(9, 1);  
addToCart(2, -1);  
updateQuantity(6, 2);  
removeFromCart(5);  
applyDiscountCode(" save10 ");  
console.log("");  
printReceipt();
```

You'll see:

Welcome to Maple Stationery!

1. Notebook - \$3.50
2. Pen set - \$6.25
3. Backpack - \$34.99
4. Water bottle - \$12.00
5. Desk lamp - \$22.50
6. Sticky notes - \$1.99

Added 3 x Notebook to your cart.

Added 1 x Backpack to your cart.

Added 4 x Sticky notes to your cart.

Added 1 x Desk lamp to your cart.

Added 2 x Notebook to your cart.

Sorry, we don't have a product with ID 9.

Sorry, the quantity must be a whole number of 1 or more.

Sticky notes quantity changed to 2.

Removed Desk lamp from your cart.

Discount code SAVE10 applied.

Maple Stationery - Receipt

```
=====
ITEM                QTY    AMOUNT
Notebook            5     $17.50
Backpack             1     $34.99
Sticky notes        2      $3.98
-----
Subtotal                $56.47
Discount (SAVE10)      -$5.65
Tax (8%)                $4.07
Shipping               FREE
-----
TOTAL                  $54.89
=====
Thank you for shopping with us!
```

`console.log("")` prints an empty line, which gives the output some room to breathe.

Look at the shipping: the subtotal was \$56.47, and after the \$5.65 discount the order is \$50.82. That's still \$50.00 or more, so shipping is free.

🎉 That's a complete, working shopping cart, built from nothing but the basics. Every online shop you've ever used does these same steps behind the scenes.

Common mistakes

1. Doing math with formatted money

```
const subtotal = "$56.47"; // what formatMoney gives back: text!
console.log(subtotal * 0.1); // prints: NaN
```

`formatMoney` returns a string, ready for printing. You can't do math with `"$56.47"`. Keep the plain number of cents for every calculation, and only call `formatMoney` when you build a line to print.

2. Forgetting to stop after an error message

```
function findProduct(productId) {
  return null; // pretend the product doesn't exist
}

function addToCart(productId, quantity) {
  const product = findProduct(productId);
  if (product === null) {
    console.log(`Sorry, we don't have a product with ID ${productId}.`);
  }
  console.log(`Added ${quantity} x ${product.name} to your cart.`);
}

addToCart(42, 1);
// prints: Sorry, we don't have a product with ID 42.
// TypeError: Cannot read properties of null (reading 'name')
```

The message printed, but the function kept going, and `product` is `null`. Fix: `return` straight after the error message, so the rest of the function never runs. That turns the check into a guard clause, like the ones in chapter 09.

3. Mixing up `1` and `"1"`

```
const product = { id: 1, name: "Notebook" };
console.log(product.id === 1); // prints: true
console.log(product.id === "1"); // prints: false
```

If you call `addToCart("1", 2)`, you'll get `Sorry, we don't have a product with ID 1.`, even though product 1 exists! The ID you passed is the *string* `"1"`, and `===` checks the type too. Pass numbers. If an ID ever arrives as text (like typed input, later in the course), convert it first with `Number()` from chapter 05.

4. Printing a value instead of returning it

```
function getSubtotal() {
  const subtotal = 5647;
  console.log(subtotal); // shows the number on screen...
}                        // ...but never returns it

const subtotal = getSubtotal();
console.log("Subtotal in dollars:", subtotal / 100);
```

You'll see:

```
5647
Subtotal in dollars: NaN
```

This is the `console.log` vs `return` mix-up from [chapter 09](#). A function without `return` gives back `undefined`, and `undefined / 100` is `NaN`. Fix: `return subtotal;`. A good rule for this project: functions named `get...` return a value, and functions named `print...` `print`.

5. Calling `padStart` on a number

```
const quantity = 5;
console.log(quantity.padStart(4));
// TypeError: quantity.padStart is not a function
```

`padStart` and `padEnd` are string methods, and numbers don't have them. Fix: `String(quantity).padStart(4)`.

Quick recap

In this project, you practiced:

- Storing data as arrays of objects: the catalog and the cart (chapters 10 and 11).
- Splitting a big job into small functions that each do one thing, and returning values instead of printing them (chapter 09).
- Checking input, and stopping early with a clear message when something's wrong (chapters 07 and 09).
- Looping to search, to add up, and to print (chapters 08 and 10).
- Handling money safely in whole cents, with `Math.round` and `toFixed` (chapter 05).
- Lining up text in columns with `padEnd`, `padStart` and `repeat` (chapter 06).

In chapter 13, you'll learn shorter ways to write many of the loops you just wrote.

Next: try the stretch goals in the [exercises](#), then move on to [13 Array Methods](#).

Exercises

How to do these:

- These are stretch goals: extra features for your finished shopping cart. Finish all seven milestones in the [notes](#) first.
 - Before each one, copy your finished `cart.js` to a new file in this folder (`ex1.js`, `ex2.js`, and so on), so you always keep a working version. Run it with `node ex1.js`.
 - Each stretch goal gives you new test code. Just like in the project, replace the "Try it out" part with it.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Prices with commas

Right now, `formatMoney(123456)` gives you `$1234.56`. Real shops write `$1,234.56`, and `Intl.NumberFormat` from [chapter 05](#) knows how every country writes money.

Change the inside of `formatMoney` so it uses `Intl.NumberFormat` with US dollars. Don't change anything else: every other function keeps calling `formatMoney` just like before.

Test code:

```
console.log(formatMoney(350));  
console.log(formatMoney(123456));  
console.log(formatMoney(100000000));
```

Expected output:

```
$3.50  
$1,234.56  
$1,000,000.00
```

Then run the whole shopping trip from "Putting it all together" again. The receipt should look exactly the same as before, because every amount on it is under \$1,000.

Bonus: try other countries. `"en-GB"` with `"GBP"` gives `£1,234.56`. `"en-IN"` with `"INR"` turns `formatMoney(12345678900)` into `₹12,34,56,789.00`, because in India, digits after the first thousand are grouped in twos.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Loyalty points

Maple Stationery wants customers to come back. Customers earn 1 loyalty point for each whole dollar of their final total. Orders with a total of \$50.00 or more earn double points.

1. Write `getLoyaltyPoints(totalInCents)` , which returns the number of points.

Test code:

```
console.log(getLoyaltyPoints(5489));  
console.log(getLoyaltyPoints(3145));  
console.log(getLoyaltyPoints(99));
```

Expected output:

```
108  
31  
0
```

2. Add a line to the receipt, just before `Thank you for shopping with us!` , that tells the customer how many points they earned.

Test code:

```
addToCart(3, 1);  
addToCart(1, 5);  
applyDiscountCode("FLAT5");  
printReceipt();
```

Expected output:

Added 1 x Backpack to your cart.
Added 5 x Notebook to your cart.
Discount code FLAT5 applied.
Maple Stationery - Receipt

```
=====
ITEM                QTY    AMOUNT
Backpack            1    $34.99
Notebook            5    $17.50
-----
Subtotal                $52.49
Discount (FLAT5)       -$5.00
Tax (8%)                $3.80
Shipping              $4.99
-----
TOTAL                  $56.28
=====
You earned 112 loyalty points!
Thank you for shopping with us!
```

Look at the shipping: the subtotal is over \$50.00, but after the \$5.00 discount the order is only \$47.49, so shipping isn't free.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Stock limits

The shop only has a few of some products. Right now, a customer could add 500 backpacks! Here's how many of each product the shop really has:

ID	Product	Stock
1	Notebook	50
2	Pen set	10
3	Backpack	2
4	Water bottle	5
5	Desk lamp	1
6	Sticky notes	30

1. Add a `stock` property to each product, using the table above.

2. Change `addToCart` . After the quantity check, check whether the quantity already in the cart plus the new quantity would be more than the stock. If it would, print `Sorry, we only have 2 x Backpack in stock.` (with the real stock and name) and stop.

Test code:

```
addToCart(3, 1);
addToCart(3, 1);
addToCart(3, 1);
addToCart(5, 2);
addToCart(5, 1);
printCart();
```

Expected output:

```
Added 1 x Backpack to your cart.
Added 1 x Backpack to your cart.
Sorry, we only have 2 x Backpack in stock.
Sorry, we only have 1 x Desk lamp in stock.
Added 1 x Desk lamp to your cart.
Your cart:
- 2 x Backpack: $69.98
- 1 x Desk lamp: $22.50
Subtotal: $92.48
```

Bonus: make `updateQuantity` check the stock too.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Buy 2, get 1 free

This week, sticky notes are "buy 2, get 1 free": for every 3 packs in the cart, 1 of them is free. So 7 packs cost the same as 5.

1. Add `buyTwoGetOneFree: true` to the sticky notes product.
2. Write `getLineTotal(item)` , which returns the amount for one cart item, with the deal taken into account.
3. Use `getLineTotal` everywhere you used to work out price times quantity.
4. In `printCart` , show how many packs were free, like `(2 free!)` , but only when some were free.

Test code:


```
addToCart(6, 7);
addToCart(1, 2);
printCart();
```

Expected output:

```
Added 7 x Sticky notes to your cart.
Added 2 x Notebook to your cart.
Your cart:
- 7 x Sticky notes: $9.95 (2 free!)
- 2 x Notebook: $7.00
Subtotal: $16.95
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A receipt you can keep

`printReceipt` prints straight to the screen. But what if the shop wants to email the receipt, or save it to a file? For that, you need the whole receipt as one string.

Write `buildReceipt()`, which **returns** the whole receipt as a single string, with the lines separated by `"\n"` (the new-line character from chapter 06). It doesn't print anything itself. Printing it is then one line:

```
console.log(buildReceipt());
```

Test code:

```
addToCart(2, 2);
addToCart(4, 1);
const receipt = buildReceipt();
console.log(receipt);
console.log(`The receipt has ${receipt.split("\n").length} lines.`);
console.log(receipt.includes("FREE"));
```

Expected output:

```

Added 2 x Pen set to your cart.
Added 1 x Water bottle to your cart.
Maple Stationery - Receipt
=====
ITEM                QTY    AMOUNT
Pen set              2      $12.50
Water bottle         1      $12.00
-----
Subtotal              $24.50
Tax (8%)              $1.96
Shipping              $4.99
-----
TOTAL                 $31.45
=====
Thank you for shopping with us!
The receipt has 13 lines.
false

```

The last two lines show why a string is so handy: you can count its lines, search it, and pass it around, and none of that is possible with text that has already been printed.

Bonus: make `printReceipt` use `buildReceipt`, so the receipt's layout lives in only one place.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Look back through your `cart.js`. How many loops did you write that search an array for one item? How many add things up?

Those patterns are so common that JavaScript has shortcuts for them, like `find` and `reduce`. [Chapter 13](#) shows you how they work, and you'll see your cart loops shrink to a line or two.

Level 2: Building Real Things

Goal: write cleaner code, make web pages interactive, and build your first real app.

13 Array Methods

What is it?

Array methods are built-in functions that every array comes with, like `map`, `filter`, and `reduce`.

You give one of them a small function that says what to do with *one* item. The method runs that function for every item in the array, so you don't have to write the loop yourself.

Why does it matter?

In [chapter 10](#) you looped over arrays with `for`. That works, but you write the same setup every time: an empty array, a loop, a `push`.

Say you want the names from a list of products:

```

const products = [
  { name: "Keyboard", price: 45 },
  { name: "Mouse", price: 20 },
  { name: "Monitor", price: 180 },
];

// With a loop
const names = [];
for (const product of products) {
  names.push(product.name);
}

// With an array method
const sameNames = products.map((product) => product.name);

console.log(names);    // prints: [ 'Keyboard', 'Mouse', 'Monitor' ]
console.log(sameNames); // prints: [ 'Keyboard', 'Mouse', 'Monitor' ]

```

Both give the same result. The second one says *what* you want ("the name of each product") instead of *how* to loop. That means:

- Less code, and fewer places for bugs like off-by-one errors to hide.
- Code that reads almost like a sentence.
- Code that looks like real-world JavaScript. You'll see these methods in almost every project.

Real-world example

Picture a teacher with a stack of exam papers and a helpful assistant. The teacher doesn't handle every paper herself. She gives the assistant one short instruction, and the assistant applies it to each paper in the stack.

Instruction to the assistant	Array method	What comes back
"Write 'Well done' on each paper."	<code>forEach</code>	Nothing. The work itself is the point
"Give me each student's grade as a letter."	<code>map</code>	A new stack, same size
"Give me only the papers that passed."	<code>filter</code>	A new stack, maybe smaller
"Find me the first paper by Priya."	<code>find</code>	One paper (or nothing)
"Did anyone get 100?"	<code>some</code>	Yes or no
"Did everyone pass?"	<code>every</code>	Yes or no
"Add up all the scores."	<code>reduce</code>	One number

Instruction to the assistant	Array method	What comes back
"Put them in order, highest first."	<code>sort</code>	The same stack, reordered

In code, the instruction you hand over is a function, and it has a special name: a callback. That's where we'll start.

How it works

Callbacks: handing a function to another function

Remember from [chapter 09](#) that functions are values? You can store them in variables. You can also pass them into other functions, just like numbers or strings.

A **callback** is a function you pass into another function, so that function can call it for you. You hand it over, and the other function "calls it back" when it needs it.

```
function shout(text) {
  return text.toUpperCase() + "!";
}

function announce(message, style) {
  console.log(style(message)); // call whatever function we were given
}

announce("doors open at seven", shout); // prints: DOORS OPEN AT SEVEN!
```

Look closely: we pass `shout`, not `shout()`. Without brackets, you hand over the function itself. With brackets, you'd call it right away and pass along its result instead.

You don't have to name the callback first. You can write an arrow function right inside the call:

```
announce("doors open at seven", (text) => `*** ${text} ***`);
// prints: *** doors open at seven ***
```

That's exactly how array methods work. The method does the looping, and your callback says what happens to each item.

`forEach` : do something with each item

`forEach` calls your callback once for every item, in order.

```
const playlist = ["Intro", "Sunrise", "Night Drive"];

playlist.forEach((song, index) => {
  console.log(`${index + 1}. ${song}`);
});
```

You'll see:

1. Intro
2. Sunrise
3. Night Drive

The callback receives up to three things: the current item, its index, and the whole array. You only list the ones you need. Here we use the song and its index.

`forEach` doesn't give anything back (it returns `undefined`). Use it when the action is the whole point: printing, saving, updating something.

Watch out: you can't stop a `forEach` early with `break`. If you need to stop partway, use a `for...of` loop from [chapter 10](#).

`map` : turn every item into something new

`map` builds a **new array** out of whatever your callback returns. You get one new item for every old one, so the new array always has the same length.

```
const celsius = [0, 20, 30, 100];
const fahrenheit = celsius.map((temp) => temp * 9 / 5 + 32);

console.log(fahrenheit); // prints: [ 32, 68, 86, 212 ]
console.log(celsius);    // prints: [ 0, 20, 30, 100 ]
```

The original array isn't changed. `map` only reads it.

Your callback can return anything: a number, a string, an object. A very common job is pulling one property out of each object, like the product names at the top of this chapter.

Tip: if your arrow function has curly braces `{ }`, you must write `return`. Without braces, the value is returned automatically (the implicit return from [chapter 09](#)). Forgetting this is the most common `map` bug. See Common mistakes.

filter : keep only the items you want

`filter` also builds a new array, but it decides which items get in. Your callback answers a yes-or-no question about each item. Return `true` to keep it, `false` to leave it out.

```
const scores = [72, 45, 90, 38, 64];
const passed = scores.filter((score) => score >= 50);

console.log(passed); // prints: [ 72, 90, 64 ]
```

The new array can be shorter than the original, or even empty. The items you keep stay in their original order.

With objects, test a property:

```
const products = [
  { name: "Lamp", inStock: true },
  { name: "Rug", inStock: false },
  { name: "Mirror", inStock: true },
];

const available = products.filter((product) => product.inStock);
console.log(available.length); // prints: 2
```

`product.inStock` is already `true` or `false`, so the callback can return it directly. Any truthy value counts as "keep", just like in an `if` from [chapter 07](#).

find and findIndex : get the first match

Sometimes you want one specific item, not a whole list. `find` returns the **first item** that passes your test, and stops looking as soon as it finds one. If nothing matches, you get `undefined`.

```
const orders = [
  { id: 101, status: "delivered" },
  { id: 102, status: "shipped" },
  { id: 103, status: "shipped" },
];

console.log(orders.find((order) => order.status === "shipped"));
// prints: { id: 102, status: 'shipped' }

console.log(orders.find((order) => order.id === 999)); // prints: undefined
```

`findIndex` works the same way, but gives you the item's **position** instead. If nothing matches, it returns `-1`, like `indexOf` from [chapter 10](#).

```
console.log(orders.findIndex((order) => order.id === 103)); // prints: 2
console.log(orders.findIndex((order) => order.id === 999)); // prints: -1
```

`indexOf` can only look for an exact value. `findIndex` can look for anything you can describe as a test, like "the order whose id is 103".

`some` and `every` : yes-or-no questions

These two answer a question about the whole array with `true` or `false`.

- `some` : is the test true for **at least one** item?
- `every` : is the test true for **all** of the items?

```
const temperatures = [18, 22, 31, 25];

console.log(temperatures.some((temp) => temp > 30)); // prints: true
console.log(temperatures.every((temp) => temp > 15)); // prints: true
console.log(temperatures.every((temp) => temp > 20)); // prints: false
```

Real uses: "Is any item in the cart out of stock?" (`some`). "Has every task been ticked off?" (`every`).

`reduce` : boil a whole array down to one value

`reduce` confuses a lot of people at first, so let's take it one small step at a time.

Step 1: start from a loop you already know. Here's the "adding everything up" pattern from [chapter 10](#), totaling a shopping basket:

```
const basket = [12, 5, 8];

let total = 0; // start at 0
for (const price of basket) {
  total = total + price; // new total = old total + this price
}

console.log(total); // prints: 25
```

Look at the pattern. There's a **running total** that starts at `0`. Each time round the loop, you take the total so far, add one price, and that becomes the new total so far.

Step 2: the same thing with `reduce`.


```
const basket = [12, 5, 8];

const total = basket.reduce((runningTotal, price) => runningTotal + price, 0);

console.log(total); // prints: 25
```

`reduce` takes two things: a callback, and a starting value (the `0` at the very end). Here's what each part means:

Part	What it is
<code>0</code> at the end	The starting value . The running total begins here.
<code>runningTotal</code>	The result so far. It's usually called the accumulator , because it accumulates (collects) the answer as <code>reduce</code> moves through the array.
<code>price</code>	The current item, just like in <code>map</code> or <code>filter</code> .
<code>runningTotal + price</code>	What the callback returns. This becomes the running total for the next item.

Step 3: follow it one item at a time. Here's every step `reduce` takes for `[12, 5, 8]`:

Step	runningTotal coming in	price	Callback returns
1	<code>0</code> ← the starting value	<code>12</code>	<code>0 + 12 = 12</code>
2	<code>12</code> ← returned by step 1	<code>5</code>	<code>12 + 5 = 17</code>
3	<code>17</code> ← returned by step 2	<code>8</code>	<code>17 + 8 = 25</code>

After the last item, `reduce` hands back the final returned value: `25`.

If you remember one thing, make it this: **whatever the callback returns becomes the accumulator for the next step**. Each row's answer is handed on to the row below it.

It's like a cashier scanning your shopping. The screen starts at 0. Each beep adds one item to whatever is already on the screen. After the last item, the screen shows your total. `reduce` is the cashier, and the accumulator is the screen.

Step 4: watch it for yourself. Put a `console.log` inside the callback to print each step. The callback now has curly braces, so it needs a `return`:

```
const basket = [12, 5, 8];

const total = basket.reduce((runningTotal, price) => {
  const newTotal = runningTotal + price;
  console.log(`${runningTotal} + ${price} = ${newTotal}`);
  return newTotal;
}, 0);

console.log(`Total: ${total}`);
```

You'll see:

```
0 + 12 = 12
12 + 5 = 17
17 + 8 = 25
Total: 25
```

Whenever a `reduce` confuses you, add a log like this and watch each hand-off.

With objects. For an array of objects, pick out what you need inside the callback:

```
const cart = [
  { item: "Notebook", price: 3, quantity: 4 },
  { item: "Pen", price: 2, quantity: 10 },
  { item: "Backpack", price: 30, quantity: 1 },
];

const cartTotal = cart.reduce((sum, line) => sum + line.price * line.quantity, 0);
console.log(cartTotal); // prints: 62
```

The notebooks cost $3 * 4 = 12$, the pens $2 * 10 = 20$, and the backpack $30 * 1 = 30$. So `sum` goes $0 \rightarrow 12 \rightarrow 32 \rightarrow 62$.

The result doesn't have to be a number. The accumulator can be anything: a number, a string, even an object. Here it's an object that counts votes for a team lunch:

```
const votes = ["pizza", "tacos", "pizza", "sushi", "pizza"];

const tally = votes.reduce((counts, vote) => {
  counts[vote] = (counts[vote] || 0) + 1; // start at 0 if this food is new
  return counts;                       // hand the object on to the next step
}, {});

console.log(tally); // prints: { pizza: 3, tacos: 1, sushi: 1 }
```

Step	counts coming in	vote	counts going out
1	<code>{}</code> ← the starting value	<code>"pizza"</code>	<code>{ pizza: 1 }</code>
2	<code>{ pizza: 1 }</code>	<code>"tacos"</code>	<code>{ pizza: 1, tacos: 1 }</code>
3	<code>{ pizza: 1, tacos: 1 }</code>	<code>"pizza"</code>	<code>{ pizza: 2, tacos: 1 }</code>
4	<code>{ pizza: 2, tacos: 1 }</code>	<code>"sushi"</code>	<code>{ pizza: 2, tacos: 1, sushi: 1 }</code>
5	<code>{ pizza: 2, tacos: 1, sushi: 1 }</code>	<code>"pizza"</code>	<code>{ pizza: 3, tacos: 1, sushi: 1 }</code>

Same idea: what goes out of one step comes into the next. The starting value `{}` is an empty object.

`counts[vote]` uses bracket notation from [chapter 11](#) because the key is in a variable. And `|| 0` from [chapter 07](#) means "use 0 if this food has no count yet". A missing property is `undefined`, which is falsy.

Tip: always give `reduce` a starting value. It makes your code easier to read, and it prevents a crash on empty arrays (see Common mistakes).

When should you use `reduce`? When many items should become **one** result: a total, a count, an average, the biggest value, or an object built up from a list. If you want a new array of the same length, `map` is clearer. If you want some of the items, use `filter`.

sort : put items in order

`sort` rearranges the items of an array. For words, the default works well:

```
const fruits = ["pear", "apple", "fig"];
fruits.sort();
console.log(fruits); // prints: [ 'apple', 'fig', 'pear' ]
```

The number gotcha. Now try the same with numbers:

```
const prices = [10, 9, 1, 100, 25];
prices.sort();
console.log(prices); // prints: [ 1, 10, 100, 25, 9 ]
```

Your code isn't broken. By default, `sort` turns every item into a string and compares them like words in a dictionary, one character at a time. As text, `"100"` comes before `"25"` because `"1"` comes before `"2"`, the same way `"apple"` comes before `"banana"`.

The fix: a compare function. Pass `sort` a callback that takes two items, `a` and `b`, and returns a number:

- **Negative:** `a` goes first.
- **Positive:** `b` goes first.
- **Zero:** keep them as they are.

For numbers, `a - b` gives exactly that:

```
const prices = [10, 9, 1, 100, 25];

prices.sort((a, b) => a - b); // smallest first
console.log(prices); // prints: [ 1, 9, 10, 25, 100 ]

prices.sort((a, b) => b - a); // largest first
console.log(prices); // prints: [ 100, 25, 10, 9, 1 ]
```

To remember it: `a - b` goes up (small to big), `b - a` goes down (big to small).

Sorting objects by a property. Compare that property inside the callback:

```
const hotels = [
  { name: "Seaside Inn", price: 120 },
  { name: "City Lodge", price: 85 },
  { name: "Mountain View", price: 150 },
];

hotels.sort((a, b) => a.price - b.price);
console.log(hotels.map((hotel) => hotel.name));
// prints: [ 'City Lodge', 'Seaside Inn', 'Mountain View' ]
```

To sort by a text property, use the string method `localeCompare`. It compares two strings the way a dictionary does and returns a negative number, a positive number, or zero, which is just what `sort` wants:

```
hotels.sort((a, b) => a.name.localeCompare(b.name));
console.log(hotels.map((hotel) => hotel.name));
// prints: [ 'City Lodge', 'Mountain View', 'Seaside Inn' ]
```

`localeCompare` also handles capital letters sensibly. A plain `sort()` puts "Cherry" before "apple", because capital letters come before lowercase ones in the computer's character list.

Watch out: `sort` changes the original array. It also returns that same array, so `const sorted = list.sort()` doesn't give you a copy. `sorted` and `list` are the same array.

`toSorted` : sort without changing the original

`toSorted` works like `sort`, but it returns a **new** sorted array and leaves the original alone:

```
const lapTimes = [62, 58, 71, 60];
const fastestFirst = lapTimes.toSorted((a, b) => a - b);

console.log(fastestFirst); // prints: [ 58, 60, 62, 71 ]
console.log(lapTimes);    // prints: [ 62, 58, 71, 60 ]
```

Use it whenever you still need the original order later. It's the cousin of `toReversed()` from [chapter 10](#), and just as new (added to JavaScript in 2023), so it works in Node 20 and later and in all current browsers.

Chaining: one step after another

Most of these methods return an array, so you can call another method straight on the result. This is called **chaining**.

```
const orders = [
  { customer: "Ana", total: 40, status: "paid" },
  { customer: "Ben", total: 25, status: "refunded" },
  { customer: "Cleo", total: 90, status: "paid" },
  { customer: "Dev", total: 15, status: "paid" },
];

const paidRevenue = orders
  .filter((order) => order.status === "paid") // keep the paid orders
  .map((order) => order.total)               // take just the totals
  .reduce((sum, total) => sum + total, 0);    // add them up

console.log(paidRevenue); // prints: 145
```

Read it top to bottom, like a recipe: take the orders, keep the paid ones, take their totals, add them up. Putting each step on its own line, starting with a `.`, keeps long chains readable.

A few more handy tools

You'll use these less often, so here's just a quick look at each.

`flat` flattens nested arrays by one level:

```
const weeklyShifts = ["Mon", "Tue"], ["Thu"], ["Sat", "Sun"];
console.log(weeklyShifts.flat()); // prints: [ 'Mon', 'Tue', 'Thu', 'Sat', 'Sun' ]
```

For deeper nesting, tell it how many levels to flatten, like `flat(2)`. In [chapter 17](#) you'll write your own version.

`flatMap` is a `map` followed by a `flat`. It's handy when each item turns into several:

```
const orders = [
  { id: 1, items: ["shirt", "socks"] },
  { id: 2, items: ["hat"] },
];
console.log(orders.flatMap((order) => order.items)); // prints: [ 'shirt', 'socks', 'hat' ]
```

A plain `map` would give you an array of arrays: `[ [ 'shirt', 'socks' ], [ 'hat' ] ]`.

`Array.from` builds a real array out of something list-like. It can split a string into letters, or build an array from a length and a callback:

```
console.log(Array.from("hello")); // prints: [ 'h', 'e', 'l', 'l', 'o' ]

const seatNumbers = Array.from({ length: 5 }, (_, index) => index + 1);
console.log(seatNumbers); // prints: [ 1, 2, 3, 4, 5 ]
```

`{ length: 5 }` means "five empty slots", and the callback fills each one. The slots are empty, so we don't need the callback's first parameter. We name it `_`, a common name for "a parameter I'm not using".

`Object.groupBy` sorts items into groups by a key you choose:

```
const groceries = [
  { name: "Apple", aisle: "fruit" },
  { name: "Milk", aisle: "dairy" },
  { name: "Banana", aisle: "fruit" },
];

const byAisle = Object.groupBy(groceries, (item) => item.aisle);
console.log(byAisle);
```

You'll see:

```
[Object: null prototype] {
  fruit: [
    { name: 'Apple', aisle: 'fruit' },
    { name: 'Banana', aisle: 'fruit' }
  ],
  dairy: [ { name: 'Milk', aisle: 'dairy' } ]
}
```

You get back an object with one key per group, each holding an array of the matching items. The `[Object: null prototype]` label is a detail about how the object was built (chapter 28 explains it). You can use it like any other object: `byAisle.fruit.length` is `2`.

`Object.groupBy` is new (2024). It works in Node 21 and later and in current browsers.

Which method do I need?

I want to...	Use	You get back
Do something with each item (print, save)	<code>forEach</code>	<code>undefined</code>
Turn every item into something new	<code>map</code>	A new array, same length
Keep only the items that pass a test	<code>filter</code>	A new array, same length or shorter
Get the first item that passes a test	<code>find</code>	The item, or <code>undefined</code>
Get the position of the first match	<code>findIndex</code>	An index, or <code>-1</code>
Check if at least one item passes	<code>some</code>	<code>true</code> or <code>false</code>
Check if every item passes	<code>every</code>	<code>true</code> or <code>false</code>
Combine all the items into one result	<code>reduce</code>	One value, of any type
Put items in order	<code>sort</code> or <code>toSorted</code>	The sorted array
Split items into groups	<code>Object.groupBy</code>	An object of arrays

Does it change the original array?

Changes the original	Leaves the original alone
<code>sort</code> , <code>reverse</code>	<code>toSorted</code> , <code>toReversed</code>
<code>push</code> , <code>pop</code> , <code>shift</code> , <code>unshift</code> , <code>splice</code>	<code>map</code> , <code>filter</code> , <code>find</code> , <code>findIndex</code> , <code>some</code> , <code>every</code> , <code>reduce</code> , <code>forEach</code>
	<code>slice</code> , <code>concat</code> , <code>join</code> , <code>includes</code> , <code>indexOf</code> , <code>flat</code> , <code>flatMap</code>

Changing an array can surprise other parts of your program that use the same array. [Chapter 16](#) explains why.

Common mistakes

1. Forgetting `return` inside curly braces

```
const prices = [5, 10, 15];
const doubled = prices.map((price) => {
  price * 2;
});
console.log(doubled); // prints: [ undefined, undefined, undefined ]
```

With curly braces, an arrow function only returns what you tell it to. No `return` means `undefined` for every item. Fix: write `return price * 2;` inside the braces, or drop the braces: `prices.map((price) => price * 2)`.

2. Sorting numbers without a compare function

```
const ages = [5, 40, 100, 21];
ages.sort();
console.log(ages); // prints: [ 100, 21, 40, 5 ]
```

The default sort compares text, not numbers. Fix: `ages.sort((a, b) => a - b)`.

3. Forgetting that `sort` changes the original

```
const queue = ["Zoe", "Adam", "Mia"];
const alphabetical = queue.sort();
console.log(queue); // prints: [ 'Adam', 'Mia', 'Zoe' ]
```

The original order of the queue is gone, and `alphabetical` isn't a copy: it's the same array. Fix: use `queue.toSorted()` when you still need the original order.

4. Calling the callback instead of passing it

```
function isPassing(score) {
  return score >= 50;
}

const scores = [80, 30, 65];
const passed = scores.filter(isPassing());
// TypeError: boolean false is not a function
```

The brackets in `isPassing()` run the function straight away, with no score, so it returns `false`. Then `filter` receives `false` instead of a function. Fix: pass the function itself, without brackets: `scores.filter(isPassing)`.

5. Using `forEach` when you want a new array


```
const names = ["ana", "ben"];
const capitalized = names.forEach((name) => name.toUpperCase());
console.log(capitalized); // prints: undefined
```

`forEach` always returns `undefined`, whatever your callback returns. Fix: when you want a new array back, use `map`.

6. Using `reduce` on an empty array with no starting value

```
const refunds = [];
const totalRefunds = refunds.reduce((sum, amount) => sum + amount);
// TypeError: Reduce of empty array with no initial value
```

Without a starting value, `reduce` uses the first item as the start. An empty array has no first item, so it crashes. Fix: always pass a starting value. `refunds.reduce((sum, amount) => sum + amount, 0)` gives `0`.

Quick recap

- A **callback** is a function you pass into another function, which calls it for you. Pass `myFunction`, not `myFunction()`.
- `forEach` does something with each item, `map` changes every item, `filter` keeps some items, `find` gets the first match, and `some / every` answer yes-or-no questions.
- `reduce` boils an array down to one value. Whatever the callback returns becomes the accumulator for the next item. Always give it a starting value.
- `sort` compares items as text by default, so use `(a, b) => a - b` for numbers. `sort` changes the original; `toSorted` gives you a sorted copy.
- Chain methods (`filter`, then `map`, then `reduce`) to write code that reads like a recipe.

Next: try the [exercises](#), then move on to [14 Scope and Hoisting](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).
- Copy the starting data from the exercise into your file, then write your code below it.
- Run each one with `node ex1.js`.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Price tags

You run a small market stall. Your prices are stored as plain numbers, but the labels need a dollar sign and two decimal places.

```
const prices = [4.5, 12, 0.99, 25];
```

1. Use `map` to make a new array of labels like `"$4.50"`, and print it.
2. Use `forEach` on your new array to print each label with its item number.

Expected output:

```
[ '$4.50', '$12.00', '$0.99', '$25.00' ]  
Item 1: $4.50  
Item 2: $12.00  
Item 3: $0.99  
Item 4: $25.00
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Exam results

A teacher wants a quick summary of the class's exam scores. The pass mark is 60.

```
const scores = [88, 42, 75, 91, 59, 100, 67];
```

Print this summary:

```
Passed: [ 88, 75, 91, 100, 67 ]  
First score above 90: 91  
Anyone got 100? true  
Everyone passed? false  
Number who failed: 2
```

Rule: store the pass mark in `const passMark = 60;` and use it in your callbacks. No loops: use one array method for each line (plus `.length` for the last one).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Order history

An online shop wants a summary of one customer's orders.

```
const orders = [
  { id: "A1", total: 25, items: 2 },
  { id: "A2", total: 40, items: 1 },
  { id: "A3", total: 15, items: 3 },
  { id: "A4", total: 60, items: 4 },
];
```

Part 1: on paper first. Before you write any code, fill in this table for `orders.reduce((sum, order) => sum + order.total, 0)`:

Step	sum coming in	order.total	Callback returns
1			
2			
3			
4			

Then check your table: run the `reduce` with a `console.log` inside the callback, like in the notes.

Part 2: the report. Print:

```
Total spent: $140
Items bought: 10
Average order: $35.00
Biggest order: A4 ($60)
```

Rule: use `reduce` for the total, the item count, and the biggest order.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Game leaderboard

You're building the leaderboard screen for a mobile game.

```
const players = [
  { name: "Mia", points: 320 },
  { name: "Leo", points: 450 },
  { name: "Zara", points: 290 },
  { name: "Omar", points: 510 },
  { name: "Ivy", points: 400 },
];
```

1. Make a copy of the players sorted by points, highest first, **without changing** `players`.
2. Print the top 3 as a numbered list. Pad each name to 6 characters so the points line up.
3. Print every player's name in alphabetical order, separated by commas.
4. Prove the original array wasn't changed: print the name of the first player in `players`.

Expected output:

```
=== Top 3 ===
1. Omar    510
2. Leo     450
3. Ivy     400
A to Z: Ivy, Leo, Mia, Omar, Zara
Original first player: Mia
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Bookshop stock report

A small bookshop wants a stock report every morning.

```
const books = [
  { title: "The Hobbit", genre: "fantasy", price: 12, stock: 4 },
  { title: "Dune", genre: "sci-fi", price: 15, stock: 0 },
  { title: "Emma", genre: "classic", price: 8, stock: 7 },
  { title: "Neuromancer", genre: "sci-fi", price: 11, stock: 2 },
  { title: "Mistborn", genre: "fantasy", price: 14, stock: 1 },
  { title: "Persuasion", genre: "classic", price: 9, stock: 0 },
  { title: "Circe", genre: "fantasy", price: 10, stock: 5 },
];
```

Write these three functions, each one taking the array of books as a parameter:

1. `inStockTitles(bookList)` returns the titles of the books with at least one copy in stock, sorted A to Z.
2. `stockValue(bookList)` returns the total value of all the stock (each book's price times its stock, all added up).

3. `needsReorder(bookList, minimum)` returns a line of text like `"Dune (0 left)"` for each book whose stock is below `minimum`.

Then use your functions (with a minimum of `3`) and `Object.groupBy` to print this report:

```
In stock: Circe, Emma, Mistborn, Neuromancer, The Hobbit
Stock value: $190
Reorder soon:
- Dune (0 left)
- Neuromancer (2 left)
- Mistborn (1 left)
- Persuasion (0 left)
Titles per genre:
fantasy: 3
sci-fi: 2
classic: 2
```

Rule: use array methods instead of `for` loops.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 2, your callbacks used `passMark`, a variable created *outside* them, and it worked. But a variable created *inside* a callback can't be used outside it.

Why can code see some variables and not others? That's called **scope**, and it's what [chapter 14](#) is all about.

14 Scope and Hoisting

What is it?

Scope is the part of your code where a variable can be seen and used. For example, a variable created inside a function only exists inside that function.

Hoisting is JavaScript setting up your functions and variables before it runs any of your code. It's the reason you can call some functions before the line where you wrote them.

Why does it matter?

Scope is behind some of the most common errors and bugs you'll meet:

- `ReferenceError: total is not defined`, even though you can see `total` a few lines up.
- Two parts of a program accidentally sharing one variable and overwriting each other's values.
- A variable that is mysteriously `undefined`.

Good scope habits keep each variable where it belongs, so one part of your program can't accidentally break another. In [chapter 09](#) you saw that variables inside a function stay inside. This chapter tells the whole story, including why `let` and `const` replaced `var` (as promised back in [chapter 02](#)).

Real-world example

Think of your code as a house with **one-way windows**. From inside a room, you can look out and see the hallway and the garden. But nobody outside can see into your room.

In the house	In JavaScript
The garden, which everyone can see	The global scope: the top level of your file
A room	A function's scope
A cupboard inside a room	A block <code>{ }</code> , like the body of an <code>if</code> or a loop
One-way windows that only look outward	Inner code can see outer variables, but not the other way round
Looking for your keys: your room first, then the hallway, then the garden	The scope chain

Keep this house in mind. Most of the rules about scope follow from it.

How it works

Global scope

A variable declared at the top level of your file, outside any function or block, is in the **global scope**. Everything in the file can see it.

```
const shopName = "Corner Books";

function printHeader() {
  console.log(`Welcome to ${shopName}`); // can see shopName
}

printHeader(); // prints: Welcome to Corner Books
```

The function can use `shopName` because it looks outward, through the window, to the top level of the file.

In Node, each file has its own top level, so a variable at the top of one file isn't visible in another file. You'll connect files together in [chapter 29](#).

Function scope

A variable declared inside a function only exists inside that function. This is called **function scope**.

```
function makeReceipt() {
  const tax = 2;
  console.log(`Tax: ${tax}`);
}

makeReceipt(); // prints: Tax: $2
console.log(tax); // ReferenceError: tax is not defined
```

Outside the function, `tax` doesn't exist. Parameters work the same way: they only exist inside their own function.

That's a good thing. It means two functions can each have their own `total` without getting in each other's way:

```
function cartTotal() {
  const total = 40;
  return total;
}

function tipTotal() {
  const total = 6;
  return total;
}

console.log(cartTotal(), tipTotal()); // prints: 40 6
```

Each time you call a function, it gets a fresh set of its own variables. They go away when the function finishes (with one interesting exception that you'll meet in [chapter 25](#)).

Block scope

A **block** is any code between curly braces `{ }`, like the body of an `if`, a `for`, or a `while`. Variables declared with `let` or `const` inside a block only exist inside that block. This is called **block scope**.

```
const age = 20;

if (age >= 18) {
  const message = "Welcome in!";
  console.log(message); // prints: Welcome in!
}

console.log(message); // ReferenceError: message is not defined
```

The same goes for the counter of a `for` loop:

```
for (let lap = 1; lap <= 3; lap++) {
  console.log(`Lap ${lap}`);
}

console.log(lap); // ReferenceError: lap is not defined
```

If you need a value after the block ends, declare the variable *before* the block, then fill it in inside:


```
const age = 15;
let message; // declared before the block

if (age >= 18) {
  message = "Welcome in!";
} else {
  message = "Sorry, adults only.";
}

console.log(message); // prints: Sorry, adults only.
```

var : the old way, and why it leaks

Here's what `var` does differently: it ignores blocks. A `var` only stays inside a *function*. We say `var` is **function-scoped**, while `let` and `const` are **block-scoped**.

```
if (true) {
  var discount = 10;
}

console.log(discount); // prints: 10 (it leaked out of the block!)
```

A `var` loop counter leaks out too:

```
for (var lap = 1; lap <= 3; lap++) {
  console.log(`Lap ${lap}`);
}

console.log(lap);
```

You'll see:

```
Lap 1
Lap 2
Lap 3
4
```

The loop stopped when `lap` reached 4, and `lap` is still hanging around afterwards. With `let`, you got an error instead, which is what you want: the counter belongs to the loop.

A function does stop a `var`:

```
function setUp() {
  var secretCode = "1234";
}

setUp();
console.log(secretCode); // ReferenceError: secretCode is not defined
```

`var` has one more bad habit: it lets you declare the same name twice, with no warning at all.

```
var total = 10;
var total = 20; // no error: the first one is quietly replaced
console.log(total); // prints: 20
```

With `let` or `const`, JavaScript catches the mistake before your code even starts running:

```
let score = 10;
let score = 20; // SyntaxError: Identifier 'score' has already been declared
```

	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function	Block	Block
Can be reassigned	Yes	Yes	No
Can be declared twice	Yes, silently	No (SyntaxError)	No (SyntaxError)

That's why the course uses `let` and `const`. You'll still see `var` in older code and tutorials, so it's good to recognize it.

The scope chain

When your code uses a variable, JavaScript looks for it in the current scope first. If it's not there, it looks in the scope around that one, then the one around *that*, all the way out to the global scope. This path is called the **scope chain**. If the variable isn't found anywhere along it, you get a `ReferenceError`.

You can write a function inside another function. The inner one can only be used inside the outer one. Here, three scopes sit inside each other:

```

const city = "Lisbon";           // global scope

function planTrip() {
  const hotel = "Sea View";      // planTrip's scope

  function planDay() {
    const museum = "Tile Museum"; // planDay's scope
    console.log(`${museum}, ${hotel}, ${city}`);
  }

  planDay();
}

planTrip(); // prints: Tile Museum, Sea View, Lisbon

```

Picture it as rooms inside rooms:

```

global scope:      city
└─ planTrip's scope: hotel
    └─ planDay's scope: museum ← starts looking here, then moves outward

```

`planDay` finds `museum` in its own scope, `hotel` one level out, and `city` in the global scope. The windows only look outward, though. If `planTrip` tried to use `museum`, it would get `ReferenceError: museum is not defined`.

Scope depends on where code is written

Scope is decided by **where a function is written**, not where it's called from. This is called **lexical scope** ("lexical" means "based on the written code").

```

const greeting = "Hello";

function sayHello() {
  console.log(greeting); // Looks outward from where sayHello was written
}

function party() {
  const greeting = "Yo";
  sayHello(); // called from here, but that doesn't matter
}

party(); // prints: Hello

```

`sayHello` was written at the top level, so it looks outward from there and finds `"Hello"`. The `greeting` inside `party` is in a room that `sayHello` can't see into. Remember this one: it's the key to closures in [chapter 25](#).

Shadowing

What if an inner scope declares a variable with the same name as an outer one? Inside the inner scope, the inner one wins. It hides the outer one, which is called **shadowing**.

```
const player = "Sam";

function startGame() {
  const player = "Guest"; // a new variable that shadows the outer one
  console.log(`Inside: ${player}`);
}

startGame(); // prints: Inside: Guest
console.log(`Outside: ${player}`); // prints: Outside: Sam
```

There are two separate variables here that happen to share a name, and the outer `player` is untouched. (The greeting inside `party` above was shadowing too.)

Shadowing is allowed, but it's a good way to confuse yourself. When you can, pick different names.

Hoisting: what JavaScript sets up first

Before running your code, JavaScript reads through each scope and makes a note of every declaration in it: every function declaration, and every variable made with `var`, `let`, or `const`. Setting these up ahead of time is called **hoisting**, as if the declarations were lifted up (hoisted, like a flag) to the top of their scope.

It's like a teacher reading the class list before a lesson: she knows every name before anyone walks in. But knowing a name doesn't mean that student is in their seat yet. The three kinds of declarations are "in their seats" at different times, and that's what makes hoisting confusing. So let's take them one at a time.

1. Function declarations are ready from the start. You can call them before the line where they're written:

```
console.log(addShipping(20)); // prints: 25

function addShipping(price) {
  return price + 5;
}
```

The whole function is set up before the first line runs. That's handy: you can put your main code at the top of a file and the helper functions below it.

2. `var` exists from the start, but holds `undefined`. Its name is set up early, but its value only arrives when its line runs:

```
console.log(flavor); // prints: undefined
var flavor = "vanilla";
console.log(flavor); // prints: vanilla
```

JavaScript treats that code as if you'd written:

```
var flavor; // the declaration is hoisted to the top
console.log(flavor); // undefined
flavor = "vanilla"; // the value is set where you wrote it
console.log(flavor); // vanilla
```

No error, just a quiet `undefined`. Silent bugs like that are one more reason to avoid `var`.

3. `let` and `const` exist from the start, but are locked until their line. JavaScript knows about them, but it won't let you touch them until the line that declares them has run. The stretch of code before that line is called the **temporal dead zone** (TDZ). "Temporal" means "to do with time": it's the time *before* the variable is ready.

```
console.log(total); // ReferenceError: Cannot access 'total' before initialization
let total = 50;
```

Compare that message with the one for a variable that doesn't exist at all:

```
console.log(price); // ReferenceError: price is not defined
```

"Cannot access ... before initialization" means JavaScript knows the variable exists, but you're using it too early. (**Initialization** is the moment a variable gets its first value.) "Is not defined" means there's no variable with that name anywhere along the scope chain.

The TDZ is a good thing. A loud error on the line with the bug beats a quiet `undefined` that causes trouble somewhere else later.

Function declarations vs. function expressions

Remember function expressions and arrow functions from [chapter 09](#)? They're stored in a variable, so they follow that variable's rules, not the rules for function declarations:

```
console.log(double(4)); // ReferenceError: Cannot access 'double' before initialization

const double = (number) => number * 2;
```

With `var`, it fails in a different way. The variable exists, but it holds `undefined`, and you can't call `undefined`:

```
console.log(triple(4)); // TypeError: triple is not a function

var triple = function (number) {
  return number * 3;
};
```

Here's the whole picture:

How it's written	Can you use it before its line?
<code>function greet() { }</code>	Yes. The whole function is ready.
<code>var name = ...</code>	Sort of: the name exists, but it holds <code>undefined</code> .
<code>let name = ...</code> Or <code>const name = ...</code>	No: <code>ReferenceError: Cannot access 'name' before initialization</code> .
<code>const greet = () => { }</code>	No, same as any other <code>const</code> .

The rule that keeps you safe: **declare your variables before you use them**. Then hoisting never surprises you. Calling a function *declaration* that's written further down is fine, and very common.

Accidental globals

What happens if you forget `let` or `const` altogether?

```
function startTimer() {
  seconds = 60; // oops: no let or const
}

startTimer();
console.log(seconds); // prints: 60
```

No error! The variable escaped from the function. JavaScript searched the scope chain for `seconds`, didn't find it, and quietly created a brand-new **global variable**: one that every part of your program can see and change, even code in other files. It gets attached to `globalThis`, a built-in object that holds global things, so `globalThis.seconds` is now `60` too.

This relaxed behavior is the default for plain `.js` files, both in Node and in the browser. That default mode is nicknamed **sloppy mode**.

Why are accidental globals bad?

- Any code, anywhere, can change them, so bugs are hard to track down.

- A typo can quietly create a new variable instead of giving you an error:

```
let count = 0;
conut = count + 1; // typo: creates a new global called conut
console.log(count); // prints: 0
```

JavaScript also has a stricter mode, called **strict mode**, which turns this mistake into a proper error. You switch it on by writing `"use strict";` at the very top of a file:

```
"use strict";

function startTimer() {
  seconds = 60;
}

startTimer(); // ReferenceError: seconds is not defined
```

You'll learn more about strict mode in [chapter 26](#). Classes ([chapter 27](#)) and modules ([chapter 29](#)) are always strict, so there this mistake becomes an error automatically. Until then, one habit protects you: always declare variables with `let` or `const`.

Keep variables in the smallest scope that works

Put each variable in the smallest "room" that needs it. If only one function uses a variable, declare it inside that function. If only one `if` block uses it, declare it inside that block.

It's like keeping your things in your own room rather than in the hallway, where anyone might move them. The fewer globals you have, the fewer ways one part of your program can accidentally mess with another part's data.

Common mistakes

1. Using a variable outside the block it was made in

```
const temperature = 30;
if (temperature > 25) {
  const advice = "Wear sunscreen";
}
console.log(advice); // ReferenceError: advice is not defined
```

`advice` only exists inside the `if` block. Fix: declare it before the block with `let advice;`, and assign it inside.

2. Forgetting `let` or `const`

```
function addToCart() {
  itemCount = 1; // no let or const
}
addToCart();
console.log(itemCount); // prints: 1
```

It "works", and that's the problem: `itemCount` is now an accidental global that any code can change. Fix: always declare variables with `let` or `const`, in the scope where they belong.

3. Shadowing a variable you meant to update

```
let score = 0;

function addPoints() {
  let score = 10; // oops: a new variable, not the outer one
}

addPoints();
console.log(score); // prints: 0
```

The `let` inside the function creates a separate `score` that shadows the outer one, so the outer one never changes. Fix: to update the outer variable, leave out the `let`: `score = score + 10`; . Even better, have the function `return` the new score, like in [chapter 09](#).

4. Using `let` or `const` before its line

```
console.log(`Hello, ${username}`);
const username = "maya_k";
// ReferenceError: Cannot access 'username' before initialization
```

`username` is in the temporal dead zone until its line runs. Fix: move the declaration above the line that uses it.

5. Calling a function expression too early

```
greet();
const greet = () => console.log("Hi!");
// ReferenceError: Cannot access 'greet' before initialization
```

Arrow functions and function expressions follow their variable's rules. Fix: move the call below the definition, or write it as a function declaration, which is ready from the start.

6. Expecting `var` to stay inside a block


```
for (var i = 0; i < 3; i++) {}  
console.log(i); // prints: 3
```

`var` ignores blocks, so the counter leaks out of the loop. Fix: use `let` for loop counters (and everywhere else).

Quick recap

- **Scope** is where a variable can be seen: the global scope (the top level), function scope, and block scope (inside `{ }`).
- Inner code can see outer variables, never the other way round. JavaScript searches outward along the **scope chain**, starting from where the code is *written* (lexical scope).
- `let` and `const` are block-scoped. `var` is function-scoped, leaks out of blocks, and can be declared twice by accident, so avoid it.
- **Hoisting**: function declarations are ready from the start, `var` starts out as `undefined`, and `let` / `const` throw `Cannot access '...' before initialization` until their line runs.
- Always declare variables with `let` or `const`. Forgetting creates an accidental global in sloppy mode. Strict mode, classes, and modules turn it into an error.

Next: try the [exercises](#), then move on to [15 Destructuring, Spread and Rest](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Most exercises give you some code to start from. Copy it into your file.
- Run each one with `node ex1.js` .
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Scope detective

A bike shop's website has a small script that shows off its bikes. Some of its lines can't see the variables they're trying to use.

```

const brand = "SunnyBikes";

function showBike() {
  const model = "City Cruiser";
  console.log(`A: ${brand}`);
  console.log(`B: ${model}`);
}

showBike();

if (brand.length > 5) {
  const slogan = "Ride happy";
  console.log(`C: ${slogan}`);
}

console.log(`D: ${brand}`);
console.log(`E: ${model}`);
console.log(`F: ${slogan}`);

```

1. **Predict.** Before running anything, write a comment at the end of each line from A to F: what it prints, or which error it throws.
2. **Check.** Run the file. When it stops with an error, compare it with your prediction, put `//` in front of that line to switch it off, and run again. Repeat until the file runs to the end.
3. **Fix.** Switch lines E and F back on. Then move declarations around (don't delete any `console.log` lines) so the whole file runs and prints:

```

A: SunnyBikes
B: City Cruiser
C: Ride happy
D: SunnyBikes
E: City Cruiser
F: Ride happy

```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Leaky loops

A parcel depot counts how many parcels are heavy (over 10 kg). The code was written years ago, with `var`.

```

var weights = [2, 12, 7, 15, 3];
var heavyCount = 0;

for (var i = 0; i < weights.length; i++) {
  var weight = weights[i];
  if (weight > 10) {
    var label = "HEAVY";
    heavyCount++;
  }
}

console.log(`Heavy parcels: ${heavyCount}`);
console.log(`i after the loop: ${i}`);
console.log(`weight after the loop: ${weight}`);
console.log(`label after the loop: ${label}`);

```

It prints:

```

Heavy parcels: 2
i after the loop: 5
weight after the loop: 3
label after the loop: HEAVY

```

1. For each of the last three lines, write a comment explaining why it prints that value at all.
2. Replace every `var` with `let` or `const`. Use `const` wherever the value never changes.
3. Run it again. Where does it stop now, and with what error? Why is that actually an improvement?
4. Delete the three "after the loop" lines. They only worked because `var` leaked. Your program should now print just:

```

Heavy parcels: 2

```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Too early at the gym

A gym's fee calculator crashes before it finishes. Everything it needs is in the file, just in the wrong order.

```
// Gym membership calculator
console.log(welcomeMessage("Ravi"));
console.log(`Monthly fee: ${monthlyFee}`);
console.log(`Yearly fee: ${yearlyFee(monthlyFee)}`);
console.log(`With loyalty discount: ${applyDiscount(yearlyFee(monthlyFee))}`);

const monthlyFee = 35;

const yearlyFee = (fee) => fee * 12;

var applyDiscount = function (price) {
  return price - 20;
};

function welcomeMessage(name) {
  return `Welcome to Iron Gym, ${name}!`;
}
```

1. **Predict** before you run it: which lines will work, and where will it crash? With which error?
2. **Fix it by moving code only.** Don't change what any function does. `welcomeMessage` must stay at the bottom of the file.
3. Add a comment explaining why `welcomeMessage` can stay at the bottom, but the other three can't.

Expected output:

```
Welcome to Iron Gym, Ravi!
Monthly fee: $35
Yearly fee: $420
With loyalty discount: $400
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: once it works, change the `var` into a `const`. Does it still work? Why?

Exercise 4 (Medium): Library late fees

A library's late-fee report prints the right fee for each book, but the total is wrong:

```
// Library late fees
const loans = [
  { title: "Dune", daysLate: 3 },
  { title: "Emma", daysLate: 0 },
  { title: "Circe", daysLate: 5 },
];

function calculateFee(daysLate) {
  feePerDay = 0.5;
  fee = daysLate * feePerDay;
  return fee;
}

function printReport(loanList) {
  total = 0;
  for (const loan of loanList) {
    const fee = calculateFee(loan.daysLate);
    totl = total + fee;
    console.log(`${loan.title}: ${fee.toFixed(2)}`);
  }
  console.log(`Total: ${total.toFixed(2)}`);
}

printReport(loans);
```

It prints `Total: $0.00`. The file is full of accidental globals, and one of them is hiding the bug.

1. Add `"use strict";` as the very first line of the file and run it.
2. Strict mode stops at the first accidental global it finds. Fix that one by declaring the variable properly, in the smallest scope that works. Then run again.
3. Repeat until the program runs. It should print:

```
Dune: $1.50
Emma: $0.00
Circe: $2.50
Total: $4.00
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Clean up the cinema

A cinema's booking script was written in a hurry. It uses `var`, and it has accidental globals. Worse, it has a real bug that those globals are hiding.

```

var movies = [
  { title: "Space Pirates", price: 12, seatsLeft: 3 },
  { title: "The Quiet Forest", price: 9, seatsLeft: 0 },
  { title: "Robot Chef", price: 10, seatsLeft: 8 },
];

var i;
var total = 0;
var message;

function book(title, tickets) {
  for (i = 0; i < movies.length; i++) {
    if (movies[i].title === title) {
      movie = movies[i];
    }
  }
  if (movie.seatsLeft < tickets) {
    message = "Sorry, not enough seats for " + title + " (" + movie.seatsLeft + " left)";
  } else {
    movie.seatsLeft = movie.seatsLeft - tickets;
    cost = movie.price * tickets;
    total = total + cost;
    message = "Booked " + tickets + " x " + title + ": $" + cost;
  }
  console.log(message);
}

book("Space Pirates", 2);
book("The Quiet Forest", 1);
book("Robot Chef", 4);
book("Space Pirates", 2);
book("Robot Chief", 1);
console.log("Total spent: $" + total);

```

It prints:

```

Booked 2 x Space Pirates: $24
Sorry, not enough seats for The Quiet Forest (0 left)
Booked 4 x Robot Chef: $40
Sorry, not enough seats for Space Pirates (1 left)
Booked 1 x Robot Chief: $12
Total spent: $76

```

There's no movie called "Robot Chief" (it's a typo), but the script booked a seat and charged \$12 anyway!

1. **Find the cause.** Write a comment explaining which accidental global makes the "Robot Chief" booking go through, and which movie's seat was really sold.

2. Rewrite the script so that:

- There's no `var` anywhere, and `"use strict";` at the top runs without errors.
- Every variable lives in the smallest scope that works. Only `movies` and your running total live at the top level.
- `book` uses `find` (from [chapter 13](#)) instead of the `for` loop, and handles a title that doesn't exist.
- `book` doesn't touch the total. Instead, it **returns** the cost of the booking (or `0` if nothing was booked), and the code at the bottom adds up the returned values.
- Messages use template literals.

Expected output:

```
Booked 2 x Space Pirates: $24
Sorry, not enough seats for The Quiet Forest (0 left)
Booked 4 x Robot Chef: $40
Sorry, not enough seats for Space Pirates (1 left)
Sorry, we don't show "Robot Chief"
Total spent: $64
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 5 you wrote `movie.title`, `movie.price` and `movie.seatsLeft` again and again. Wouldn't it be nice to unpack an object's properties into separate variables in one line?

That's exactly what destructuring does, and it's one of the shortcuts in [chapter 15](#).

15 Destructuring, Spread and Rest

What is it?

Three shortcuts for working with arrays and objects:

- **Destructuring** unpacks values from an array or object into separate variables, in one line.
- **Spread** (`...`) spreads an array or object out into its separate pieces. It's great for copying and merging.
- **Rest** (also `...`) does the opposite: it collects "the rest" of the pieces into one array or object.

Why does it matter?

Without these shortcuts, working with objects means a lot of repetition:

```
const user = { name: "Priya", city: "Pune", plan: "pro" };

const name = user.name;
const city = user.city;
const plan = user.plan;
```

With destructuring, those last three lines become one:

```
const { name, city, plan } = user;
```

Spread lets you copy and combine arrays and objects without writing a loop. You'll see all three shortcuts everywhere in modern JavaScript: in function parameters, in code that handles data from the internet, and in tools like React.

Real-world example

Think about unpacking your shopping when you get home:

Tool	Everyday version	In code
Destructuring	Unpacking a grocery bag: the milk goes in the fridge, the bread goes in the cupboard	<code>const { milk, bread } = bag;</code>
Spread	Tipping two bags out onto one table	<code>[...bagA, ...bagB]</code>

Tool	Everyday version	In code
Rest	"I'll take the first two apples. The rest go in the fruit bowl."	<code>const [first, second, ...fruitBowl] = apples;</code>

Spread and rest use the same three dots, `...`. Spread *unpacks* things and rest *packs* them up. There's a table at the end of this chapter that shows how to tell them apart.

How it works

Array destructuring

Put variable names inside square brackets on the left of `=`. Each one gets the item at the same position:

```
const podium = ["Ana", "Ben", "Chen"];
const [gold, silver, bronze] = podium;

console.log(gold); // prints: Ana
console.log(silver); // prints: Ben
console.log(bronze); // prints: Chen
```

Position is what matters: the first variable gets the first item, the second gets the second, and so on. The names are up to you.

It's handy with `split` from [chapter 06](#):

```
const [firstName, lastName] = "Ada Lovelace".split(" ");
console.log(lastName); // prints: Lovelace
```

Skipping items. Leave a gap with an extra comma:

```
const [winner, , third] = ["Ana", "Ben", "Chen"];
console.log(winner, third); // prints: Ana Chen
```

Default values. If there's no item at a position, its variable gets `undefined`. You can give it a default instead, with `= :`

```
const [main, side = "fries"] = ["burger"];
console.log(main); // prints: burger
console.log(side); // prints: fries
```

The default is only used when the value would be `undefined`.

Swapping two variables. Destructuring can swap two values in one line:

```
let left = "cat";
let right = "dog";

[left, right] = [right, left];

console.log(left, right); // prints: dog cat
```

The right side builds a new array, `["dog", "cat"]`, and the left side unpacks it back into the two variables. There's no `let` on that line, because both variables already exist.

Object destructuring

Put variable names inside curly braces on the left of `=`. Each variable gets the property **with the same name**:

```
const book = { title: "Dune", author: "Frank Herbert", pages: 412 };
const { title, author } = book;

console.log(title); // prints: Dune
console.log(author); // prints: Frank Herbert
```

With objects, the names matter and the order doesn't: `const { author, title } = book;` does exactly the same thing. If you ask for a property that doesn't exist, you get `undefined`.

Renaming. To use a different variable name, write `property: newName`:

```
const { title: bookTitle } = book;
console.log(bookTitle); // prints: Dune
```

Read it as "take `title`, and call it `bookTitle`".

Default values. These work the same way as with arrays:

```
const { format = "paperback" } = book;
console.log(format); // prints: paperback
```

`book` has no `format` property, so the default is used.

Nested objects. To reach inside a nested object, repeat the pattern:

```
const order = {
  id: 501,
  customer: { name: "Leo", city: "Oslo" },
};

const { id, customer: { name, city } } = order;
console.log(id, name, city); // prints: 501 Leo Oslo
```

`customer: { name, city }` means "go into `customer`, and unpack `name` and `city` from there". This does **not** create a `customer` variable. Only `id`, `name`, and `city` exist afterwards.

Keep nesting shallow. If a pattern gets hard to read, unpack in two steps instead: first `const { customer } = order;`, then `const { name } = customer;`.

In loops. Destructuring works anywhere you create a variable, including a `for...of` loop:

```
const products = [
  { name: "Tent", price: 120 },
  { name: "Lantern", price: 25 },
];

for (const { name, price } of products) {
  console.log(`${name}: ${price}`);
}
```

You'll see:

```
Tent: $120
Lantern: $25
```

Remember `Object.entries` from [chapter 11](#)? It gives you a list of `[key, value]` pairs. Now you can unpack each pair right in the loop:

```
const stock = { apples: 12, pears: 0, plums: 7 };

for (const [fruit, count] of Object.entries(stock)) {
  console.log(`${fruit}: ${count}`);
}
```

You'll see:

```
apples: 12
pears: 0
plums: 7
```

Destructuring function parameters

Here's where destructuring really shines. Imagine calling a function with lots of parameters:

```
createAccount("Sam", "sam@mail.com", "EUR", true);
```

What do `"EUR"` and `true` mean? You'd have to go and read the function to find out. And if you mix up the order, you get a bug without any error.

The fix is the **options object** pattern: pass one object with named properties, and destructure it right in the parameter list:

```
function createAccount({ name, email, currency = "USD", newsletter = false }) {
  console.log(`${name} <${email}> | ${currency} | newsletter: ${newsletter}`);
}

createAccount({ name: "Sam", email: "sam@mail.com", newsletter: true });
// prints: Sam <sam@mail.com> | USD | newsletter: true
```

Now every value has a label, the order doesn't matter, anything with a default can be left out, and adding a new option later won't break the calls you already wrote.

It works in callbacks from [chapter 13](#) too:

```
const products = [
  { name: "Tent", price: 120 },
  { name: "Lantern", price: 25 },
];

const names = products.map(({ name }) => name);
console.log(names); // prints: [ 'Tent', 'Lantern' ]
```

Spread with arrays

Putting `...` in front of an array spreads its items out, as if you'd typed them one by one with commas in between.

Copying an array:

```
const playlist = ["Intro", "Sunrise"];
const copy = [...playlist];

copy.push("Night Drive");
console.log(playlist); // prints: [ 'Intro', 'Sunrise' ]
console.log(copy);     // prints: [ 'Intro', 'Sunrise', 'Night Drive' ]
```

The copy is a separate array, so changing it leaves the original alone.

Merging arrays, with extra items anywhere you like:

```
const fruits = ["apple", "pear"];
const veggies = ["carrot", "leek"];
const groceries = [...fruits, ...veggies, "bread"];

console.log(groceries); // prints: [ 'apple', 'pear', 'carrot', 'leek', 'bread' ]
```

It does the same job as `concat` from [chapter 10](#), and many people find it easier to read.

Passing an array's items as separate arguments. `Math.max` and `Math.min` from [chapter 05](#) want separate numbers, not an array:

```
const lapTimes = [62, 58, 71, 60];

console.log(Math.max(lapTimes)); // prints: NaN
console.log(Math.max(...lapTimes)); // prints: 71
console.log(Math.min(...lapTimes)); // prints: 58
```

`Math.max(...lapTimes)` is the same as writing `Math.max(62, 58, 71, 60)`.

Spread with objects

Spread works on objects too. It copies all of an object's properties into a new object.

Copying an object:

```
const original = { name: "Maya", plan: "free" };
const copy = { ...original };
copy.plan = "pro";

console.log(original.plan); // prints: free
console.log(copy.plan);     // prints: pro
```

Merging objects, and overriding default settings. When two objects have a property with the same name, the one that comes **later wins**:

```
const defaultSettings = { theme: "light", fontSize: 14, sound: true };
const userSettings = { theme: "dark", fontSize: 18 };

const settings = { ...defaultSettings, ...userSettings };
console.log(settings); // prints: { theme: 'dark', fontSize: 18, sound: true }
```

This pattern is everywhere: start from the defaults, then lay the user's choices on top. If you swapped the order, the defaults would win instead.

Changing one property in a copy:

```
const user = { name: "Maya", city: "Lisbon" };
const movedUser = { ...user, city: "Berlin" };

console.log(movedUser); // prints: { name: 'Maya', city: 'Berlin' }
console.log(user);      // prints: { name: 'Maya', city: 'Lisbon' }
```

Spread makes a shallow copy

One warning before we move on. Spread only copies the **top level**. If an object holds another object or an array, the copy shares that inner one with the original:

```
const team = { name: "Blue Team", members: ["Ana", "Ben"] };
const copy = { ...team };

copy.members.push("Cleo");
console.log(team.members); // prints: [ 'Ana', 'Ben', 'Cleo' ]
```

Changing the copy's `members` changed the original's too! A copy like this, where only the top level is new, is called a **shallow copy**. Why this happens, and how to make a full copy, is the whole story of [chapter 16](#).

Rest in destructuring

Rest collects "everything else" into one array or object. It uses the same `...`, but on the **left** side of `=`:

```
const [leader, ...followers] = ["Kim", "Lou", "Max", "Noor"];

console.log(leader); // prints: Kim
console.log(followers); // prints: [ 'Lou', 'Max', 'Noor' ]
```

With objects, rest collects every property you didn't unpack. A real use: removing a password before you show or send someone's account details:

```
const account = { username: "sam99", password: "hunter2", email: "sam@mail.com" };
const { password, ...safeAccount } = account;

console.log(safeAccount); // prints: { username: 'sam99', email: 'sam@mail.com' }
```

The rest part always has to come **last**. Nothing can come after it.

Rest parameters

Rest also works in a function's parameter list. A **rest parameter** collects any number of arguments into a real array:

```
function sumAll(...numbers) {
  return numbers.reduce((total, number) => total + number, 0);
}

console.log(sumAll(5, 10, 15)); // prints: 30
console.log(sumAll(2));         // prints: 2
console.log(sumAll());          // prints: 0
```

Remember `sumAll` from chapter 10? That one took an array: `sumAll([10, 20, 30])`. With a rest parameter, the caller passes plain numbers, and the function still gets them as an array.

You can mix normal parameters with a rest parameter, as long as the rest parameter comes last:

```
function orderPizza(size, ...toppings) {
  console.log(`${size} pizza with ${toppings.join(", ")}`);
}

orderPizza("Large", "mushrooms", "olives", "basil");
// prints: Large pizza with mushrooms, olives, basil
```

In older code you may see `arguments`, a built-in, array-like list of everything passed to a function. It isn't a real array, and arrow functions don't get their own, so use rest parameters instead.

Spread or rest? How to tell them apart

Both are written `...`. What they do depends on where they appear:

Where the ... is	Name	What it does	Example
Inside <code>[]</code> or <code>{ }</code> that you're building, or in a function call	Spread	Unpacks items out	<code>[...fruits, "kiwi"], Math.max(...nums)</code>
On the left of <code>=</code> , or in a parameter list	Rest	Collects items in	<code>const [first, ...others] = list;</code> <code>function sumAll(...numbers)</code>

A quick way to remember: spread **expands**, rest **collects**.

Common mistakes

1. Using names that don't match the properties

```
const movie = { title: "Up", year: 2009 };
const { name, released } = movie;
console.log(name, released); // prints: undefined undefined
```

Object destructuring matches property *names*, not positions. `movie` has no `name` or `released`, so both are `undefined`. Fix: use the real names, `const { title, year } = movie;`, or rename them: `const { title: name } = movie;`.

2. Destructuring a parameter that wasn't passed

```
function showProfile({ name, age }) {
  console.log(`${name} (${age})`);
}

showProfile();
// TypeError: Cannot destructure property 'name' of 'undefined' as it is undefined.
```

Nothing was passed in, so there's nothing to unpack. Fix: give the whole parameter a default of an empty object, `function showProfile({ name, age } = {})`, and give the properties defaults too if they need them.

3. Forgetting the spread when merging

```
const breakfast = ["eggs", "toast"];
const lunch = ["soup"];
const meals = [breakfast, lunch];
console.log(meals); // prints: [ [ 'eggs', 'toast' ], [ 'soup' ] ]
```

Without `...`, you get an array of arrays instead of one flat list. Fix: `const meals = [...breakfast, ...lunch];`.

4. Passing an array to `Math.max`

```
const temperatures = [18, 25, 21];
console.log(Math.max(temperatures)); // prints: NaN
```

`Math.max` expects separate numbers, and it can't turn a whole array into one number. Fix: spread it:

`Math.max(...temperatures)` gives `25`.

5. Putting rest anywhere but last

```
const [...allButLast, last] = ["Mon", "Tue", "Wed"];
// SyntaxError: Rest element must be last element
```

Rest means "everything that's left", so it can only go at the end. Rest parameters have the same rule: `function logAll(...items, label)` gives `SyntaxError: Rest parameter must be last formal parameter`. Fix: move the rest part to the end, or use `at(-1)` from chapter 10 to get the last item.

Quick recap

- **Destructuring** unpacks arrays by position (`const [first, second] = list;`) and objects by name (`const { name } = user;`). You can skip items, rename, set defaults, and reach into nested objects.
- Destructure a function's parameter to use an **options object**: every value is labeled, the order doesn't matter, and defaults are easy.
- **Spread** `...` unpacks. Copy and merge arrays with `[...a, ...b]` and objects with `{ ...defaults, ...choices }` (the later one wins), or pass an array's items as arguments: `Math.max(...nums)`.
- **Rest** `...` collects: `const [first, ...others] = list;` and `function sumAll(...numbers)`. It always comes last.
- Spread makes a **shallow** copy: nested objects and arrays are still shared with the original ([chapter 16](#) explains why).

Next: try the [exercises](#), then move on to [16 Values vs. References](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Copy the starting data from the exercise into your file, then write your code below it.
- Run each one with `node ex1.js`.

- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Race results

The results of a 5 km fun run have just come in, in finishing order.

```
const results = ["Aiko", "Bruno", "Chidi", "Dana", "Emil"];
```

Use array destructuring for every step:

1. Unpack the first three runners into `gold`, `silver`, and `bronze`, and print the podium.
2. Unpack the `winner`, and collect everyone else into an array called `others`.
3. Unpack only the fourth-place runner, skipping the first three.
4. The judges wrote two names down the wrong way round. Swap them in one line:

```
let first = "Bruno";  
let second = "Aiko";
```

Expected output:

```
Gold: Aiko, Silver: Bruno, Bronze: Chidi  
Winner: Aiko  
Everyone else: [ 'Bruno', 'Chidi', 'Dana', 'Emil' ]  
Fourth place: Dana  
After the swap: Aiko Bruno
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Profile card

A social app shows a small profile card for each user.

```
const profile = {  
  username: "maya_k",  
  email: "maya@example.com",  
  address: { city: "Lisbon", country: "Portugal" },  
};
```

Use object destructuring to create these five variables:

- `handle`, taken from `username`
- `email`
- `theme`, which isn't in the object, so give it a default of `"light"`
- `city` and `country`, from inside `address`

Then print:

```
Handle: @maya_k
Email: maya@example.com
Theme: light
Lives in: Lisbon, Portugal
```

Bonus: create all five variables in a single destructuring statement.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Music app settings

A music app has default settings, and each user saves the few things they changed.

```
const defaultSettings = { volume: 50, shuffle: false, theme: "light", quality: "high" };
const savedSettings = { volume: 80, theme: "dark" };
const playlist = ["Intro", "Sunrise"];
const newSongs = ["Night Drive", "Echoes"];
const songLengths = [215, 187, 242, 199];
```

1. Make `settings` by combining the defaults with the saved settings. The saved ones must win. Print it.
2. Make `partyMode`: a copy of `settings` with `shuffle` turned on and `volume` at 100. Don't change `settings`. Print both volumes.
3. Make `fullPlaylist`: the playlist, then the new songs, then `"Outro"` at the end. Print it, then print the original playlist's length to show it didn't change.
4. Print the longest and shortest song lengths.

Expected output:

```
{ volume: 80, shuffle: false, theme: 'dark', quality: 'high' }
Party volume: 100, normal volume: 80
[ 'Intro', 'Sunrise', 'Night Drive', 'Echoes', 'Outro' ]
Original playlist length: 2
Longest song: 242 seconds
Shortest song: 187 seconds
```

Rule: no loops, no `push`, and no `concat`. Use spread.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Restaurant bookings

A restaurant's booking system needs two small functions.

Part 1. Write `bookTable`, which takes one options object with these properties:

Property	Default
<code>name</code>	(no default: it's required)
<code>guests</code>	<code>2</code>
<code>time</code>	<code>"19:00"</code>
<code>outdoor</code>	<code>false</code>

It should **return** a confirmation string, or an error message when there's no name.

Part 2. Write `addToOrder`, which takes a table number followed by any number of dishes, and **returns** a summary. Say "dish" for one dish and "dishes" for more.

Test them with exactly these calls:

```
console.log(bookTable({ name: "Priya" }));
console.log(bookTable({ name: "Tom", guests: 6, outdoor: true }));
console.log(bookTable({ time: "20:30", name: "Lena", guests: 4 }));
console.log(bookTable());
console.log(addToOrder(5, "soup", "pasta", "tiramisu"));
console.log(addToOrder(2, "salad"));
```

Expected output:

```
Table for 2 at 19:00, inside, under "Priya"
Table for 6 at 19:00, outside, under "Tom"
Table for 4 at 20:30, inside, under "Lena"
Error: a booking needs a name
Table 5 ordered 3 dishes: soup, pasta, tiramisu
Table 2 ordered 1 dish: salad
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Online orders

An online shop exports its orders as rows. Each row starts with the order id and the customer's name, followed by any number of items.

```
const rawOrders = [
  ["A-101", "Ana", { name: "Mug", price: 8, qty: 2 }, { name: "Tea", price: 5, qty: 1 }],
  ["A-102", "Ben", { name: "Notebook", price: 4, qty: 3 }],
  ["A-103", "Cleo", { name: "Lamp", price: 30, qty: 1 }, { name: "Bulb", price: 3, qty: 4 }],
];
```

Write these functions:

1. `parseOrder(row)` turns one row into an object like `{ id: "A-101", customer: "Ana", items: [...] }`. Use array destructuring with rest.
2. `orderTotal(order)` returns the order's total (price times quantity for each item, added up). Destructure `items` in the parameter list, and destructure `price` and `qty` in your `reduce` callback.
3. `addDelivery(order)` returns a **copy** of the order with a new `delivery` property: `"free"` if the total is 25 or more, otherwise `"$4.99"`. It must not change the order it was given.

Then turn all the rows into order objects with `map`, and print this report:

```
A-101 | Ana | 2 items | $21.00
A-102 | Ben | 1 item | $12.00
A-103 | Cleo | 2 items | $42.00
Delivery: A-101 $4.99, A-102 $4.99, A-103 free
Original orders changed? false
```

"2 items" means two different items, not the total quantity. The last line checks whether any of your parsed orders (not the copies) has a `delivery` property.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In the notes, changing a copy's `members` array also changed the original. Spread made a new object, but both objects still pointed at the same inner array.

Why does JavaScript share some things and copy others? That's the big idea of [chapter 16](#), and once you understand it, a whole family of confusing bugs makes sense.

16 Values vs. References

What is it?

When you copy a **primitive** (a string, number, boolean, `null`, or `undefined`), JavaScript copies the value itself. Each variable gets its own copy.

When you "copy" an object or an array, JavaScript only copies a **reference**: a pointer to where that object lives. Both variables then point at the *same* object.

Why does it matter?

Look at this:

```
const original = ["milk", "eggs"];
const copy = original;
copy.push("bread");
console.log(original); // prints: [ 'milk', 'eggs', 'bread' ]
```

You changed `copy`, but `original` changed too. If you don't know why, bugs like this feel like magic. And they're some of the most common bugs in real apps: a cart total that changes by itself, an "undo" button that doesn't undo, a function that quietly scrambles your data.

Once you understand references, these bugs make sense, and you'll know how to avoid them. This chapter also explains two things earlier chapters promised: why `cart === []` is always `false`, and why a `const` array can still be changed (both from [chapter 10](#)).

Real-world example

Think about sending a PDF versus sharing a Google Doc link:

	Primitives (copied by value)	Objects and arrays (shared by reference)
It's like...	Emailing someone a PDF copy of your document	Sending someone a link to your Google Doc
If they scribble on it	Your original is untouched	Everyone with the link sees the change
<code>b = a</code> gives <code>b</code> ...	Its own separate copy	The same link to the same document

Here's another way to picture it: a variable holding an object doesn't hold the house itself. It holds the house's **address**. Copy the variable, and you've copied the address, not the house. If your friend follows that address and paints the front door red, it's your front door too.

How it works

Primitives are copied by value

```
let myScore = 10;
let yourScore = myScore; // copies the value 10
yourScore = yourScore + 5;

console.log(myScore); // prints: 10
console.log(yourScore); // prints: 15
```

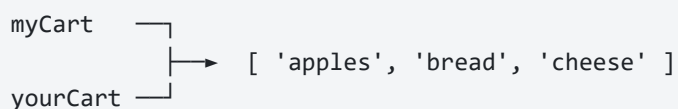
`yourScore` got its own `10`, so changing it later has nothing to do with `myScore`. Strings behave the same way. And remember from [chapter 06](#) that strings can't be changed in place anyway: every string method gives you a new string.

Objects and arrays are shared by reference

```
const myCart = ["apples", "bread"];
const yourCart = myCart; // copies the reference, not the array

yourCart.push("cheese");
console.log(myCart); // prints: [ 'apples', 'bread', 'cheese' ]
```

There's only **one** array here. `myCart` and `yourCart` are two labels pointing at it:



A **reference** works like the address of an object in the computer's memory (memory is where your program keeps its values while it runs; [chapter 41](#) goes deeper). An object variable holds that address, not the object itself.

Objects work exactly the same way:


```
const account = { owner: "Maya", balance: 100 };
const sameAccount = account;

sameAccount.balance = 0;
console.log(account.balance); // prints: 0
```

Changing an object vs. pointing somewhere new

There are two very different things you can do with a variable that holds an object:

- **Mutate** it: change the object itself, like `account.balance = 0` or `cart.push("tea")`. **Mutating** means changing an object or array in place, so everyone sharing the reference sees the change.
- **Reassign** it: point the variable at a different value with `=`. This only affects that one variable.

```
let a = { score: 1 };
let b = a; // both point at the same object

b = { score: 99 }; // b now points at a brand-new object
console.log(a.score); // prints: 1
```

Reassigning `b` didn't touch the object that `a` points at. It only moved `b`'s arrow to a new object.

Functions receive references too

When you pass an array or object into a function, the function gets a reference to the same one. If the function mutates it, the caller sees the change:

```
function addFreeGift(cart) {
  cart.push("free tote bag");
}

const order = ["shoes"];
addFreeGift(order);
console.log(order); // prints: [ 'shoes', 'free tote bag' ]
```

A primitive can't be changed this way, because the function gets its own copy of the value:

```
function addBonus(points) {
  points = points + 50;
  return points;
}

const myPoints = 100;
addBonus(myPoints); // returns 150, but myPoints is untouched
console.log(myPoints); // prints: 100
```

Here's a sneakier version of the same bug. This function only wants to *read* the scores, but `sort` from [chapter 13](#) changes the array it's called on:

```
function getTopScore(scores) {
  scores.sort((a, b) => b - a); // sort changes the array it was given!
  return scores[0];
}

const examScores = [70, 95, 80];
console.log(getTopScore(examScores)); // prints: 95
console.log(examScores);              // prints: [ 95, 80, 70 ]
```

The original order of `examScores` is gone. The fix is a method that doesn't mutate: `scores.toSorted((a, b) => b - a)[0]`, or `Math.max(...scores)` from [chapter 15](#).

=== compares references, not contents

For primitives, `===` compares values. For objects and arrays, it asks "are these the *same* object?", not "do they look the same?":

```
const shirtA = { size: "M" };
const shirtB = { size: "M" };
const shirtC = shirtA;

console.log(shirtA === shirtB); // prints: false (two separate objects that look alike)
console.log(shirtA === shirtC); // prints: true  (the same object)
console.log([] === []);        // prints: false
```

It's like two identical houses on different streets: they look the same, but they have different addresses. That's why `cart === []` is always `false`: the `[]` creates a brand-new empty array, which can never be the same array as `cart`.

To compare contents, compare the parts you care about:

```
const listA = ["milk", "eggs"];
const listB = ["milk", "eggs"];

const sameItems =
  listA.length === listB.length && listA.every((item, index) => item === listB[index]);
console.log(sameItems); // prints: true
```

`includes` and `indexOf` compare the same way as `===`, so they can't find an object by its contents:

```
const products = [{ id: 1 }, { id: 2 }];
console.log(products.includes({ id: 1 })); // prints: false
console.log(products.some((product) => product.id === 1)); // prints: true
```

To look for an object, test one of its properties with `find`, `findIndex`, or `some`.

`const` doesn't mean "can't change"

You saw this in chapter 10: `const` locks the *variable*, not the object it points to. You can't point a `const` variable at something else, but you can still change the object:

```
const settings = { volume: 5 };

settings.volume = 8; // fine: changing the object
console.log(settings); // prints: { volume: 8 }

settings = { volume: 1 }; // TypeError: Assignment to constant variable.
```

Now you know why. `const` glues the arrow in place, so the variable always points at the same object. But the object at the end of the arrow can still change.

Shallow copies

To get a real, separate copy, you need a *new* array or object. You already know several ways to make one:

To copy...	Use	From
An array	<code>[...list]</code>	chapter 15
An array	<code>list.slice()</code>	chapter 10
An array	<code>Array.from(list)</code>	chapter 13
An object	<code>{ ...obj }</code>	chapter 15

```
const guests = ["Ana", "Ben"];
const partyGuests = [...guests];

partyGuests.push("Cleo");
console.log(guests);           // prints: [ 'Ana', 'Ben' ]
console.log(partyGuests);      // prints: [ 'Ana', 'Ben', 'Cleo' ]
console.log(guests === partyGuests); // prints: false
```

But all of these make a **shallow copy**: only the top level is new. Anything *inside*, like a nested array or object, is still shared, because the copy only copied the references to it:

```
const trip = {
  destination: "Rome",
  travelers: ["Ana", "Ben"],
};

const tripCopy = { ...trip };
tripCopy.destination = "Paris"; // top level: separate
tripCopy.travelers.push("Cleo"); // nested: shared!

console.log(trip.destination); // prints: Rome
console.log(trip.travelers);    // prints: [ 'Ana', 'Ben', 'Cleo' ]
```

Here's what's going on:

```
trip    → { destination: 'Rome', travelers: ┌ }
                                     └─┬─> [ 'Ana', 'Ben', 'Cleo' ]
tripCopy → { destination: 'Paris', travelers: ┌ }
```

Two objects, but only one `travelers` array. It's like photocopying a recipe card that says "see page 42 of the cookbook". Now you have two cards, but they both point at the same cookbook.

This catches people out with arrays of objects too. Copying the array doesn't copy the objects inside it:

```
const products = [{ name: "Mug", price: 8 }];
const saleProducts = [...products];

saleProducts[0].price = 5;
console.log(products[0].price); // prints: 5
```

Deep copies with `structuredClone`

A **deep copy** copies every level, so nothing is shared. Modern JavaScript has a built-in function for it, `structuredClone`:

```
const trip = {
  destination: "Rome",
  travelers: ["Ana", "Ben"],
};

const tripCopy = structuredClone(trip);
tripCopy.travelers.push("Cleo");

console.log(trip.travelers); // prints: [ 'Ana', 'Ben' ]
console.log(tripCopy.travelers); // prints: [ 'Ana', 'Ben', 'Cleo' ]
```

`structuredClone` works in all current browsers and in Node 17 and later. It copies data: numbers, strings, booleans, arrays, plain objects, and more. It can't copy functions, and it throws an error if you try:

```
const timer = { seconds: 60, start: () => console.log("go") };
structuredClone(timer);
// DOMException [DataCloneError]: () => console.log("go") could not be cloned.
```

In older code you'll see another trick for deep copies: `JSON.parse(JSON.stringify(obj))`. It turns the object into text and back again (JSON is [chapter 23](#)). It works for simple data, but it quietly drops functions and `undefined` values, so prefer `structuredClone`.

Do you need deep copies often? Not really. Usually you only change one nested part, and you can copy just that part, as the next section shows.

How to avoid accidental changes

1. Don't mutate what you were given. A function that receives an array or object should return a new one instead of changing it:

```
function addItem(cart, item) {
  return [...cart, item]; // a new array: the caller's cart is untouched
}

const cart = ["shoes"];
const newCart = addItem(cart, "socks");
console.log(cart); // prints: [ 'shoes' ]
console.log(newCart); // prints: [ 'shoes', 'socks' ]
```

2. Prefer methods that don't mutate. Most changing operations have a copy-making twin:

Changes the original	Makes a new copy instead
<code>list.push(item)</code>	<code>[...list, item]</code>
<code>list.sort(compare)</code>	<code>list.toSorted(compare)</code>
<code>list.reverse()</code>	<code>list.toReversed()</code>
<code>list[index] = value</code>	<code>list.with(index, value)</code>
<code>list.splice(index, 1)</code>	<code>list.filter((_, i) => i !== index)</code>
<code>obj.key = value</code>	<code>{ ...obj, key: value }</code>

`with` is the one you haven't met yet. It gives you a copy of the array with one item replaced:

```
const seats = ["free", "free", "free"];
const booked = seats.with(1, "taken");
console.log(booked); // prints: [ 'free', 'taken', 'free' ]
console.log(seats);  // prints: [ 'free', 'free', 'free' ]
```

Like `toSorted` and `toReversed`, it arrived in 2023, so it works in Node 20 and later and in current browsers.

To change the objects inside an array without touching the originals, combine `map` with object spread, so each object gets copied:

```
const menu = [
  { dish: "Soup", price: 6 },
  { dish: "Pie", price: 9 },
];

const newMenu = menu.map((item) => ({ ...item, price: item.price + 1 }));
console.log(newMenu);           // prints: [ { dish: 'Soup', price: 7 }, { dish: 'Pie', price: 10 } ]
console.log(menu[0].price);     // prints: 6
```

Watch out: when an arrow function returns an object directly, wrap the object in parentheses: `(item) => ({ ... })`. Without them, JavaScript reads the `{` as the start of the function body, and you get `undefined` or a confusing error.

3. Copy the nested part you change. To update something nested without a full deep copy, spread each level you touch:

```
const trip = { destination: "Rome", travelers: ["Ana", "Ben"] };

const biggerTrip = { ...trip, travelers: [...trip.travelers, "Cleo"] };

console.log(trip.travelers);           // prints: [ 'Ana', 'Ben' ]
console.log(biggerTrip.travelers);     // prints: [ 'Ana', 'Ben', 'Cleo' ]
```

`biggerTrip` gets a new top-level object *and* a new `travelers` array, while `trip` stays exactly as it was. You'll see this pattern a lot in real apps, especially in React code.

Tip: mutation isn't evil. Changing an array you just created yourself, inside one function, is perfectly fine. The danger is changing data that *other* code also holds a reference to.

Common mistakes

1. Thinking `=` makes a copy

```
const draft = { title: "Holiday plans", theme: "light" };
const backup = draft;
draft.theme = "dark";
console.log(backup.theme); // prints: dark
```

The "backup" is the same object as the draft, so it changed too. Fix: make a real copy, `const backup = { ...draft };`, or `structuredClone(draft)` if it holds nested data.

2. A function that changes its input

```
function applyDiscount(prices) {
  for (let i = 0; i < prices.length; i++) {
    prices[i] = prices[i] - 5;
  }
  return prices;
}

const menuPrices = [20, 15];
const salePrices = applyDiscount(menuPrices);
console.log(salePrices); // prints: [ 15, 10 ]
console.log(menuPrices); // prints: [ 15, 10 ]
```

The sale worked, but the regular menu prices got discounted too. Fix: build and return a new array: `return prices.map((price) => price - 5);`.

3. Comparing objects with `===`

```
const selected = { size: "M" };
console.log(selected === { size: "M" }); // prints: false
```

`===` checks whether both sides are the *same* object, and `{ size: "M" }` creates a new one. Fix: compare the part you care about: `selected.size === "M"`.

4. Expecting a shallow copy to protect nested data

```
const game = { player: "Kai", inventory: ["sword"] };
const saveFile = { ...game };
game.inventory.push("shield");
console.log(saveFile.inventory); // prints: [ 'sword', 'shield' ]
```

The save file was supposed to be a snapshot, but its `inventory` is the same array as the game's. Fix: `const saveFile = structuredClone(game);`.

5. Reassigning a parameter and expecting the caller to see it

```
function emptyCart(cart) {
  cart = []; // only changes this function's own variable
}

const myCart = ["apples", "pears"];
emptyCart(myCart);
console.log(myCart); // prints: [ 'apples', 'pears' ]
```

This is the mirror image of mistake 2. Reassigning `cart` only moves the function's own arrow to a new array. The caller's array is untouched. Fix: decide what you want. To empty the shared array, change it on purpose, for example `cart.splice(0, cart.length)`. Or return a new value and let the caller store it.

Quick recap

- Primitives are copied by **value**: each variable gets its own copy.
- Objects and arrays are shared by **reference**: `b = a` gives you two names for one object, and passing one into a function shares it too.
- **Mutating** changes the shared object, so everyone sees it. **Reassigning** only moves one variable.
- `===` on objects asks "same object?", not "same contents?". And `const` stops reassignment, not mutation.
- Spread, `slice`, and `Array.from` make **shallow** copies, so nested data is still shared. `structuredClone` makes a **deep** copy.
- To avoid surprises, return new arrays and objects, and prefer non-mutating methods like `map`, `filter`, `toSorted`, and `with`.

Next: try the [exercises](#), then move on to [17 Recursion](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Copy the starting code from the exercise into your file.
- Run each one with `node ex1.js` .
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Flatmates' shopping

Two flatmates each start a shopping list from the same template, and each has a budget.

```
const myList = ["milk", "bread"];
const flatmateList = myList;
flatmateList.push("coffee");

let myBudget = 30;
let flatmateBudget = myBudget;
flatmateBudget = flatmateBudget - 10;

console.log("My list:", myList);
console.log("Flatmate's list:", flatmateList);
console.log("My budget:", myBudget);
console.log("Flatmate's budget:", flatmateBudget);
console.log("Same list?", myList === flatmateList);
```

1. **Predict** what each of the five lines will print, and write your predictions as comments. Then run it and check.
2. For the lines you got wrong, write one sentence explaining why.
3. **Fix it by changing one line**, so each flatmate has their own list. Then it should print:

```
My list: [ 'milk', 'bread' ]
Flatmate's list: [ 'milk', 'bread', 'coffee' ]
My budget: 30
Flatmate's budget: 20
Same list? false
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Same order again?

A food delivery app wants to warn customers who accidentally place the same order twice. First attempt: comparing with `===`.

```
const lastOrder = ["pizza", "cola", "salad"];
const newOrder = ["pizza", "cola", "salad"];
const otherOrder = ["pizza", "salad", "cola"];
const shortOrder = ["pizza"];
```

1. Print `lastOrder === newOrder`, and add a comment explaining why it's `false`.
2. Write a function `sameItems(listA, listB)` that returns `true` when both arrays have the same items in the same order, and `false` otherwise.
3. Test it so your output looks like this:

```
With ===: false
lastOrder vs newOrder: true
lastOrder vs otherOrder: false
lastOrder vs shortOrder: false
shortOrder vs lastOrder: false
lastOrder vs itself: true
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): A sale that breaks the shop

An online shop runs a 25% sale. The sale page looks right, but after it runs, the regular product list is broken too.

```

const products = [
  { name: "Backpack", price: 40 },
  { name: "Water bottle", price: 12 },
  { name: "Cap", price: 15 },
];

function applySale(items, percentOff) {
  for (const item of items) {
    item.price = item.price - (item.price * percentOff) / 100;
  }
  return items;
}

function cheapestFirst(items) {
  return items.sort((a, b) => a.price - b.price);
}

const saleProducts = applySale(products, 25);
const sorted = cheapestFirst(saleProducts);
console.log(sorted.map((item) => `${item.name}: ${item.price}`));
console.log(products.map((item) => `${item.name}: ${item.price}`));

```

It prints:

```

[ 'Water bottle: $9', 'Cap: $11.25', 'Backpack: $30' ]
[ 'Water bottle: $9', 'Cap: $11.25', 'Backpack: $30' ]

```

The regular list (second line) has sale prices *and* a new order. Rewrite both functions so they never change the array or the objects they're given. Don't change the last four lines. The output should become:

```

[ 'Water bottle: $9', 'Cap: $11.25', 'Backpack: $30' ]
[ 'Backpack: $40', 'Water bottle: $12', 'Cap: $15' ]

```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Save slots

A video game lets players save their progress. The developers tried two ways of making a save.

```
const game = {
  player: "Kai",
  level: 3,
  inventory: ["sword", "potion"],
  position: { x: 10, y: 4 },
};
```

1. Write a function `describe(label, save)` that prints one line about a save, in the format shown below.
2. Make `quickSave` with object spread, and `fullSave` with `structuredClone`.
3. Then the game carries on. Add these lines after your saves:

```
game.level = 4;
game.inventory.push("shield");
game.position.x = 25;
```

4. Describe the game and both saves. You should see:

```
Game: level 4, items: sword, potion, shield, x: 25
Quick save: level 3, items: sword, potion, shield, x: 25
Full save: level 3, items: sword, potion, x: 10
```

5. In a comment, explain why the quick save's `level` stayed at 3, but its items and its `x` changed.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Undo button

You're building a to-do app with an **undo** button. The trick: every change creates a new *snapshot* of the task list, and the old snapshots are kept in an array. Undo just throws away the latest snapshot.

This only works if old snapshots never change. So none of your functions may mutate what they're given.

Write these four functions:

1. `addTask(tasks, title)` returns a new array with a new task `{ title, done: false }` added at the end.
2. `completeTask(tasks, title)` returns a new array in which the task with that title has `done: true`.
3. `removeTask(tasks, title)` returns a new array without that task.
4. `printTasks(tasks)` prints each task as `[x] Buy milk (done)` or `[ ] Call mum (not done)`, or `(no tasks)` if the list is empty.

Then paste in this code, below your functions, without changing it:

```

const snapshots = [[]]; // snapshot 0: an empty list

snapshots.push(addTask(snapshots.at(-1), "Buy milk"));
snapshots.push(addTask(snapshots.at(-1), "Call mum"));
snapshots.push(completeTask(snapshots.at(-1), "Buy milk"));
console.log("Now:");
printTasks(snapshots.at(-1));

snapshots.push(removeTask(snapshots.at(-1), "Call mum"));
console.log("After removing:");
printTasks(snapshots.at(-1));

snapshots.pop(); // undo the last change
console.log("After undo:");
printTasks(snapshots.at(-1));

console.log("Snapshot 1:");
printTasks(snapshots[1]);
console.log("Snapshot 0:");
printTasks(snapshots[0]);
console.log(`Snapshots saved: ${snapshots.length}`);

```

Expected output:

```

Now:
[x] Buy milk
[ ] Call mum
After removing:
[x] Buy milk
After undo:
[x] Buy milk
[ ] Call mum
Snapshot 1:
[ ] Buy milk
Snapshot 0:
(no tasks)
Snapshots saved: 4

```

Look closely at "Snapshot 1". It was saved *before* "Buy milk" was completed, so it must still show `[ ]`. If yours shows `[x]`, one of your functions is mutating a task object that the snapshots share.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

`structuredClone` copies every level of an object, however deeply things are nested. But how could *you* write a function that handles any depth, when you don't know in advance how deep the nesting goes?

The answer is a function that calls itself. That's recursion, in [chapter 17](#).

17 Recursion

What is it?

Recursion is when a function calls itself.

Each call works on a smaller piece of the problem. The calls stop when the piece is so small that the function can answer it straight away, without calling itself again.

Why does it matter?

Some problems are shaped like "a thing that contains smaller things of the same kind":

- Folders that contain files and more folders, which contain more files and folders...
- Comments with replies, and replies to those replies.
- A shop menu with categories, subcategories, and sub-subcategories.

A loop is great for walking along a flat list. But with nested data like this, you often don't know how many levels deep it goes. A recursive function can handle any depth, because it treats every level the same way: by calling itself again.

Recursion also helps you understand the **call stack**, which is behind the lines of `at ...` that Node prints under every error message.

Recursion feels strange to almost everyone at first. That's normal. Go slowly, and trace the small examples on paper. It clicks with practice.

Real-world example

Think of **Russian nesting dolls**. To find the tiniest doll, you open the big doll. There's a smaller doll inside, so you do the same thing again: open it. You keep going until you reach a solid little doll that doesn't open. Then you stop.

Nesting dolls	Recursion
Open a doll and find a smaller doll inside	The function calls itself with a smaller problem
Do the same thing to each smaller doll	Each call runs the same function again
The tiny solid doll that doesn't open	The base case : where the calls stop

Nesting dolls	Recursion
Putting the dolls back together, smallest first	The answers coming back, from the innermost call outward

Now picture standing between two mirrors: a reflection inside a reflection inside a reflection, on and on forever. That's recursion *without* a stopping point. Later in this chapter you'll see what happens when a program tries that.

How it works

A first example: countdown

In [chapter 08](#) you wrote countdowns with loops. Here's a countdown written with recursion instead:

```
function countdown(n) {  
  if (n === 0) {  
    console.log("Liftoff!");  
    return; // stop here: no more calls  
  }  
  console.log(n);  
  countdown(n - 1); // call itself with a smaller number  
}  
  
countdown(3);
```

You'll see:

```
3  
2  
1  
Liftoff!
```

Let's walk through it slowly:

1. `countdown(3)` runs. `n` is 3, not 0, so it prints `3` and calls `countdown(2)`.
2. `countdown(2)` runs. It prints `2` and calls `countdown(1)`.
3. `countdown(1)` runs. It prints `1` and calls `countdown(0)`.
4. `countdown(0)` runs. This time `n` is 0, so it prints `Liftoff!` and returns. No more calls.

Here's the same thing as a **call trace**, where each call is indented under the call that started it:


```
countdown(3) → prints 3, then calls countdown(2)
countdown(2) → prints 2, then calls countdown(1)
countdown(1) → prints 1, then calls countdown(0)
countdown(0) → prints "Liftoff!" and stops
```

Each call is a separate run of the same function, with its own `n`. Remember from [chapter 14](#) that every call to a function gets a fresh set of its own variables. So there are four different `n`s here, one per call: 3, 2, 1, and 0.

The two parts every recursive function needs

1. A **base case**: the situation where the function stops and answers directly, without calling itself. In `countdown`, that's when `n == 0`.
2. A **recursive case**: the function calls itself with a *smaller* problem, one step closer to the base case. In `countdown`, that's `countdown(n - 1)`.

Before you run any recursive function, check two things:

- Is there a base case?
- Does every call get closer to it?

If either answer is "no", the calls never stop. You'll see what happens then in a moment.

Getting a value back: factorial

`countdown` only prints. Most recursive functions **return** a value, and that's where recursion gets really useful.

The **factorial** of a number is that number multiplied by every whole number below it, down to 1. It's written with an exclamation mark:

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

In real life, 4! is the number of different ways you can arrange 4 books on a shelf.

Now look at this pattern:

$$\begin{aligned} 4! &= 4 \times 3 \times 2 \times 1 &= 4 \times 3! \\ 3! &= 3 \times 2 \times 1 &= 3 \times 2! \\ 2! &= 2 \times 1 &= 2 \times 1! \\ 1! &= 1 \end{aligned}$$

A factorial is "the number, times the factorial of the number below it". The problem contains a smaller copy of itself. That's the sign that recursion is a good fit.

```
function factorial(n) {
  if (n <= 1) {
    return 1; // base case (it also covers 0, because 0! is 1 too)
  }
  return n * factorial(n - 1); // recursive case
}

console.log(factorial(4)); // prints: 24
```

But how does `factorial(4)` actually work it out? Imagine a line of people, and you ask the first one: "What's 4!?"

- Person 1: "I don't know yet. But it's $4 \times 3!$, so I'll ask the next person for 3! and wait."
- Person 2: "3! is $3 \times 2!$. I'll ask the next person and wait."
- Person 3: "2! is $2 \times 1!$. I'll ask the next person and wait."
- Person 4: "1! is 1. I know that one!" (That's the base case.)

Now the answers travel back up the line:

- Person 3: "Then 2! is $2 \times 1 = 2$."
- Person 2: "Then 3! is $3 \times 2 = 6$."
- Person 1: "Then 4! is $4 \times 6 = 24$."

Here's the same story as a trace. Going down, each call waits. Coming back up, each call finishes its multiplication:

```
factorial(4)
= 4 * factorial(3)
  = 3 * factorial(2)
    = 2 * factorial(1)
      = 1                ← base case: no more calls
    = 2 * 1 = 2
  = 3 * 2 = 6
= 4 * 6 = 24
```

This is the most important idea in the chapter: **each call waits for the answer from the smaller call it made, and the work finishes on the way back up.**

Watch it happen

You can make JavaScript print that trace for you. This version has a second parameter, `depth`, which is only there to indent the messages. `" ".repeat(depth)` from [chapter 06](#) makes two spaces per level:

```
function factorial(n, depth = 0) {
  const indent = "  ".repeat(depth);
  console.log(`${indent}factorial(${n}) called`);

  if (n <= 1) {
    console.log(`${indent}factorial(${n}) returns 1`);
    return 1;
  }

  const result = n * factorial(n - 1, depth + 1);
  console.log(`${indent}factorial(${n}) returns ${result}`);
  return result;
}

factorial(4);
```

You'll see:

```
factorial(4) called
  factorial(3) called
    factorial(2) called
      factorial(1) called
      factorial(1) returns 1
    factorial(2) returns 2
  factorial(3) returns 6
factorial(4) returns 24
```

Read it from top to bottom: four calls go *in*, deeper and deeper. Then four answers come back *out*, in the reverse order. Whenever a recursive function confuses you, add logs like these and watch what it does.

The call stack: a stack of plates

How does JavaScript keep track of all those calls that are waiting? It uses the **call stack**: a list of every function call that has started but hasn't finished yet.

Think of a stack of plates. Each time a function is called, a new plate goes on **top** of the stack. Only the top plate is being worked on. When that function returns, its plate is taken off, and the plate underneath carries on from where it was waiting.

Here's the stack while `factorial(3)` runs, at five moments in time:

		factorial(1)		
	factorial(2)	factorial(2)	factorial(2)	
factorial(3)	factorial(3)	factorial(3)	factorial(3)	factorial(3)
(1)	(2)	(3)	(4)	(5)

1. `factorial(3)` is called, and its plate goes on the stack.
2. It needs `factorial(2)`, so a new plate goes on top. `factorial(3)` waits underneath.
3. `factorial(2)` needs `factorial(1)`, so another plate goes on top. That's the base case, so it can answer straight away.
4. `factorial(1)` returns `1`, and its plate comes off. `factorial(2)` is on top again, and finishes: `2 * 1 = 2`.
5. `factorial(2)` returns `2`, and its plate comes off. Now `factorial(3)` finishes: `3 * 2 = 6`. It returns `6`, and the stack is empty.

You've actually seen the call stack already. When an error crashes your program, the lines starting with `at ...` under the message are the call stack at that moment. [Chapter 18](#) shows you how to read them.

When the calls never stop

What if there's no base case? Like the two mirrors, the calls go on and on. But a computer can't go on forever: the call stack only has room for so many plates.

```
function countdown(n) {
  console.log(n);
  countdown(n - 1); // no base case: nothing ever stops it
}

countdown(3);
```

This prints 3, 2, 1, 0, -1, -2, and on and on, thousands of numbers. Then it crashes. Near the bottom of the output, you'll find this error:

```
RangeError: Maximum call stack size exceeded
```

The stack of plates got too tall and toppled over. This is called a **stack overflow**. How many calls fit depends on your computer, your Node version, and the function itself, but it's roughly ten thousand for small functions like this one.

The fix is always one of the two checks from earlier: add a base case, or make sure every call really moves closer to it.

Adding up an array

Recursion works on arrays too. Here's the idea in plain words: **the sum of a list is the first number, plus the sum of the rest**. And the sum of an empty list is 0. That's the base case.

```
function sum(numbers) {
  if (numbers.length === 0) {
    return 0; // base case: an empty list adds up to 0
  }
  const [first, ...rest] = numbers; // destructuring from chapter 15
  return first + sum(rest);
}

console.log(sum([5, 10, 20])); // prints: 35
```

`[first, ...rest]` unpacks the first number and collects all the others into `rest`, just like in [chapter 15](#). Each call gets a shorter list, so the calls always reach the empty list in the end:

```
sum([5, 10, 20])
= 5 + sum([10, 20])
  = 10 + sum([20])
    = 20 + sum([])
      = 0           ← base case
    = 20 + 0 = 20
  = 10 + 20 = 30
= 5 + 30 = 35
```

(`numbers.slice(1)` from chapter 10 would work instead of the destructuring too. It gives you everything except the first item.)

To be honest, for adding up a flat list, `reduce` from [chapter 13](#) or a plain loop is the better tool. `sum` is here because it's a small, clear example. Recursion really earns its place with nested data, which is coming up next.

Recursion vs. loops

Anything you can do with recursion, you can also do with a loop, and the other way round. So which should you choose?

Use a loop when...	Use recursion when...
The data is a flat list (numbers, names, orders)	The data is nested like a tree (folders, comments, menus)

Use a loop when...	Use recursion when...
The list could be very long, like 100,000 items (loops have no call stack limit)	You don't know how deep the nesting goes, but it isn't thousands of levels deep
The loop version is short and clear	The recursive version is much simpler to read

Most everyday code uses loops and array methods. Reach for recursion when your data is shaped like a tree.

Walking a folder tree

This is where recursion shines. Here's a project folder. Every item has a `name`, and folders also have a `children` array, which can hold files *and* more folders:

```
const project = {
  name: "my-site",
  children: [
    { name: "index.html" },
    { name: "css", children: [{ name: "style.css" }] },
    {
      name: "js",
      children: [
        { name: "app.js" },
        { name: "utils", children: [{ name: "math.js" }] },
      ],
    },
  ],
};
```

How deep does it go? Three levels today, but next week someone might add a folder inside `utils`. A loop inside a loop inside a loop can't handle "any depth". Recursion can:

```
function printTree(item, depth = 0) {
  console.log(" ".repeat(depth) + item.name);

  if (item.children) {
    for (const child of item.children) {
      printTree(child, depth + 1); // the same job for each child, one level deeper
    }
  }
}

printTree(project);
```

You'll see:

```
my-site
  index.html
  css
    style.css
  js
    app.js
    utils
    math.js
```

Where's the base case? It's hiding in the `if`. A file has no `children`, so `printTree` prints the file's name and stops, without calling itself. A folder prints its own name, then hands each of its children to `printTree`, one level deeper.

Here's the start of the call trace. Notice how the calls follow the shape of the folders:

```
printTree(my-site)
  printTree(index.html) → a file: prints and stops
  printTree(css)
    printTree(style.css) → a file: prints and stops
  printTree(js)
  ...
```

Counting the files. Same idea, but this time each call returns a number:

```
function countFiles(item) {
  if (!item.children) {
    return 1; // base case: a file counts as 1
  }
  let total = 0;
  for (const child of item.children) {
    total = total + countFiles(child); // add up the files inside each child
  }
  return total;
}

console.log(countFiles(project)); // prints: 4
```

The four files are `index.html`, `style.css`, `app.js`, and `math.js`. Each folder asks its children "how many files do you have?" and adds up their answers. Only the files (the base case) answer straight away.

The same pattern works for nested comments (replies to replies) and shop menus (categories inside categories). You'll try both in the exercises.

Flattening a nested array by hand

In chapter 13 you met `flat`, which flattens nested arrays. Let's write our own `flatten`, which flattens *every* level, however deep:

```
function flatten(items) {
  const result = [];
  for (const item of items) {
    if (Array.isArray(item)) {
      result.push(...flatten(item)); // an array: flatten it, then add its items
    } else {
      result.push(item); // a plain value: add it as it is
    }
  }
  return result;
}

console.log(flatten([1, [2, 3], [4, [5, [6]]]])); // prints: [ 1, 2, 3, 4, 5, 6 ]
```

`Array.isArray` from [chapter 10](#) tells the two cases apart:

- A plain value (like `1`) is the base case. It goes straight into `result`.
- An array (like `[5, [6]]`) is a smaller copy of the same problem, so `flatten` calls itself on it. The spread `...` from chapter 15 then pushes all of the flattened items in one go.

The built-in way to do the same thing is `flat(Infinity)`, where `Infinity` means "all the levels". Now you have a good idea of how it works inside.

How to write your own recursive function

When you write one yourself, go through these steps:

1. **Find the smallest version of the problem**, the one you can answer right away: an empty list, the number 1, a file with no children. That's your base case.
2. **Trust that the function already works for a smaller input.** Don't try to follow every call in your head. When you write `sum(rest)`, assume it gives you the right total for the rest.
3. **Use that smaller answer to build the full answer:** "the first number, plus the sum of the rest".
4. **Check that every call gets closer to the base case.**

Step 2 feels strange at first. Think back to the line of people working out the factorial: each person only does their own small part, and trusts the next person to do theirs.

Common mistakes

1. Never getting closer to the base case

```
function countdown(n) {  
  if (n === 0) {  
    console.log("Liftoff!");  
    return;  
  }  
  console.log(n);  
  countdown(n); // oops: should be n - 1  
}  
  
countdown(3);  
// prints 3 thousands of times, then: RangeError: Maximum call stack size exceeded
```

There is a base case, but every call passes the same `n` along, so it's never reached. Fix: make the problem smaller each time: `countdown(n - 1)`.

2. Stepping over the base case

```
function countdown(n) {  
  if (n === 0) {  
    console.log("Liftoff!");  
    return;  
  }  
  console.log(n);  
  countdown(n - 2); // counting down in twos  
}  
  
countdown(3);  
// prints 3, 1, -1, -3, ... then: RangeError: Maximum call stack size exceeded
```

Counting down in twos from 3 goes 3, 1, -1, and jumps right over 0. Fix: make the base case catch everything at or below the stopping point: `if (n <= 0)`.

3. Forgetting to `return` the recursive call

```
function sum(numbers) {
  if (numbers.length === 0) {
    return 0;
  }
  const [first, ...rest] = numbers;
  first + sum(rest); // oops: no return, so the answer is thrown away
}

console.log(sum([1, 2, 3])); // prints: undefined
```

The recursive call works out the right answer, but nothing hands it back. A function without a `return` gives `undefined` (chapter 09). Fix: `return first + sum(rest);`.

4. Using recursion on something very big

```
countdown(100000); // the correct countdown from the start of this chapter
// prints 100000, 99999, ... down to about 90000, then: RangeError: Maximum call stack size
exceeded
```

The function is correct, but it would need 100,000 plates on the call stack, and there's only room for roughly ten thousand. Fix: for long, flat jobs like this, use a loop. Loops don't use up the call stack.

Quick recap

- **Recursion** is a function calling itself, each time on a smaller piece of the problem.
- Every recursive function needs a **base case** (stop and answer directly) and a **recursive case** (call itself, one step closer to the base case).
- Each waiting call sits on the **call stack**, like a stack of plates, until the smaller call it made returns. The answers come back in reverse order.
- No base case, or calls that never reach it, cause `RangeError: Maximum call stack size exceeded`.
- Loops are best for flat lists. Recursion shines with nested, tree-shaped data like folders, comments, and menus.
- Stuck? Trace a tiny example on paper, or add indented `console.log` lines and watch the calls.

Next: try the [exercises](#), then move on to [18 Error Handling](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Copy the starting data from the exercise into your file, then write your code below it.
- Run each one with `node ex1.js` .
- If you get `RangeError: Maximum call stack size exceeded` , don't panic. Check your base case, and check that every call gets closer to it.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Can pyramid

A supermarket stacks cans of beans in a pyramid. The bottom row has a certain number of cans, each row above has one can fewer, and the top row has just 1.

```

  []
 [][]
[] [] []
[] [] [] []   ← 4 rows: 1 + 2 + 3 + 4 = 10 cans

```

1. Write a recursive function `cansInPyramid(rows)` that returns how many cans a pyramid with that many rows needs. A pyramid with 0 rows needs 0 cans.
2. **On paper first**, write the trace for `cansInPyramid(4)` in the same style as the factorial trace in the notes (`= 4 + cansInPyramid(3)` and so on).
3. Print these results:

```

cansInPyramid(0) = 0
cansInPyramid(1) = 1
cansInPyramid(4) = 10
cansInPyramid(10) = 55
cansInPyramid(100) = 5050

```

Rule: no loops inside `cansInPyramid` . (A loop to print the five results is fine.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Backwards words

A word game needs to reverse words and spot **palindromes**: words that read the same backwards, like "level".

1. Write a recursive function `reverse(word)` that returns the word backwards.

2. Write `isPalindrome(word)`, which uses `reverse` and ignores capital letters.
3. Print:

```
"stressed" reversed is "desserts"  
"drawer" reversed is "reward"  
"a" reversed is "a"  
"" reversed is ""  
Is "Racecar" a palindrome? true  
Is "Rocket" a palindrome? false
```

Rule: don't use the `split("").reverse().join("")` trick from chapter 10. The point is to practice recursion.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Comment thread

A recipe website lets people reply to comments, and reply to replies, as deep as they like.

```
const comments = [  
  {  
    author: "Ana",  
    text: "Great recipe!",  
    replies: [  
      { author: "Ben", text: "Agreed, making it tonight.", replies: [] },  
      {  
        author: "Cleo",  
        text: "Did you use fresh basil?",  
        replies: [{ author: "Ana", text: "Yes, from my garden.", replies: [] }],  
      },  
    ],  
  },  
  { author: "Dev", text: "A bit too salty for me.", replies: [] },  
];
```

1. Write `printThread(commentList, depth = 0)`, which prints every comment in the list, with each reply indented 2 more spaces than the comment it answers.
2. Write `countComments(commentList)`, which returns how many comments there are in total, replies included.

Expected output:

```
Ana: Great recipe!
  Ben: Agreed, making it tonight.
  Cleo: Did you use fresh basil?
    Ana: Yes, from my garden.
Dev: A bit too salty for me.
Total comments: 5
```

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): Boxes inside boxes

A gift shop packs surprise boxes. A box can hold gifts (each gift is written as its value in dollars) and smaller boxes, which can hold more gifts and even smaller boxes, like nesting dolls.

```
const giftBox = [25, [10, 5], [[40], 15], []];
```

Here, the big box holds a \$25 gift, a box with \$10 and \$5 gifts, a box that holds another box (with a \$40 gift) plus a \$15 gift, and one empty box.

Write three recursive functions:

1. `totalValue(box)` returns the value of all the gifts, at any depth.
2. `countGifts(box)` returns how many gifts there are, at any depth.
3. `deepestLevel(box)` returns how many levels of boxes there are. The big box is level 1, a box inside it is level 2, and so on. A box with no boxes inside it (even an empty one) is level 1.

Expected output:

```
Total value: $95
Gifts: 5
Deepest level: 3
```

Rule: don't use `flat`. Use recursion and `Array.isArray`.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Shop menu and breadcrumbs

An online shop organizes its products into categories, which can hold smaller categories. Only the smallest categories (the ones with no `children`) hold products directly.

```
const menu = {
  name: "All products",
  children: [
    {
      name: "Electronics",
      children: [
        { name: "Phones", products: 12 },
        {
          name: "Computers",
          children: [
            { name: "Laptops", products: 8 },
            { name: "Desktops", products: 5 },
          ],
        },
      ],
    },
    {
      name: "Home",
      children: [
        { name: "Kitchen", products: 20 },
        { name: "Garden", products: 7 },
      ],
    },
  ],
};
```

Write three recursive functions:

1. `countProducts(category)` returns how many products a category holds, including everything in its subcategories.
2. `printMenu(category, depth = 0)` prints each category, indented by level, with its product count in brackets.
3. `findPath(category, target)` returns the list of category names from the top down to the one called `target`, like `[ 'All products', 'Home', 'Garden' ]`. If there's no category with that name, it returns `null`.

Then print the menu, followed by the **breadcrumbs** (the "you are here" trail that shops show at the top of a page) for `"Laptops"`, `"Garden"`, and `"Toys"`:

```
All products (52)
  Electronics (25)
    Phones (12)
    Computers (13)
      Laptops (8)
      Desktops (5)
  Home (27)
    Kitchen (20)
    Garden (7)
All products > Electronics > Computers > Laptops
All products > Home > Garden
Toys: not found
```

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Before you move on

When your recursion ran too deep, the whole program crashed with a `RangeError` and stopped. In a real app, you don't want one problem to bring everything down.

[Chapter 18](#) shows you how to catch errors, deal with them, and even throw your own.

18 Error Handling

What is it?

Error handling is how your program deals with things that go wrong: a missing value, a number that's out of range, a request that breaks the rules.

Instead of crashing with a scary message, your program can notice the problem, deal with it, and keep going.

Why does it matter?

Real programs meet a messy world. Users type "fifty" where you expected `50`. Data arrives with pieces missing. Someone tries to book 12 seats when only 6 are left.

Without error handling, one problem crashes the whole program. With it, you can:

- show a friendly message instead of a crash,
- skip one bad item and carry on with the rest,
- clean up (turn off a printer, close a file) no matter what happened.

Error handling also works the other way round. Your own functions can raise the alarm the moment something is wrong, instead of quietly giving a wrong answer that nobody notices until much later.

Real-world example

Think of a busy restaurant kitchen:

In the kitchen	In code
The chef tries to cook the order	<code>try { ... }</code> runs code that might fail
The chef finds there's no salmon left and rings the bell: "Can't make this one!"	<code>throw new Error("Out of salmon")</code>
The waiter hears the bell, tells the customer, and offers the chicken instead	<code>catch (error) { ... }</code> handles the problem
The kitchen gets cleaned at the end of the night, busy or quiet	<code>finally { ... }</code> runs no matter what
Nobody hears the bell, and the whole restaurant shuts down	An uncaught error crashes the program

Notice the split. The chef *spots* the problem, but the waiter *decides what to do about it*. That idea is at the heart of this chapter.

How it works

Kinds of errors

You've already met a few errors, like `ReferenceError` in [chapter 01](#). Here are the main kinds.

Syntax errors are grammar mistakes. JavaScript checks the whole file before running it, so a syntax error means *nothing* runs at all.

```
console.log("Hi";  
// SyntaxError: missing ) after argument list
```

Reference errors happen when you use a name that doesn't exist, often because of a typo.

```
const total = 50;  
console.log(totl);  
// ReferenceError: totl is not defined
```

Type errors happen when you use a value the wrong way, like reading a property of `undefined`, or calling something that isn't a function.

```
const guest = undefined;  
console.log(guest.name);  
// TypeError: Cannot read properties of undefined (reading 'name')  
  
const price = 5;  
price();  
// TypeError: price is not a function
```

Range errors happen when a number is outside the allowed range. You met one in [chapter 17](#): `Maximum call stack size exceeded`.

```
const cheer = "Go! ".repeat(-1);  
// RangeError: Invalid count value: -1
```

Logic errors are the sneaky ones. The program runs fine and prints no error, but the answer is wrong.

```
const math = 80;
const science = 90;
const average = math + science / 2;
console.log(average); // prints: 125
```

The average of 80 and 90 should be 85. Division happens before addition ([chapter 04](#)), so this worked out `80 + 45`. The fix is `(math + science) / 2`. JavaScript can't warn you about this, which is why you should always check your results.

Kind	What it means	When you find out
Syntax error	Broken grammar	Before any code runs
Reference error	A name that doesn't exist	When that line runs
Type error	A value used the wrong way	When that line runs
Range error	A number that's out of range	When that line runs
Logic error	The code runs, but the answer is wrong	Only when you notice!

Reference, type, and range errors show up while the program is running, so they're called **runtime errors**. Most of this chapter is about them.

Reading a stack trace

When a runtime error isn't handled, the program stops and Node prints a report. Learning to read it saves you a lot of time. Save this as `trace.js` and run it:

```
function countItems(order) {
  return order.items.length;
}

function printSummary(order) {
  const count = countItems(order);
  console.log(`Order ${order.id} has ${count} items`);
}

printSummary({ id: 7, items: ["pizza", "cola"] });
printSummary({ id: 8 });
console.log("Done");
```

You'll see something like this. The file paths are shortened here; yours will show where your file lives.

```
Order 7 has 2 items
C:\...\18-error-handling\trace.js:2
  return order.items.length;
                        ^

TypeError: Cannot read properties of undefined (reading 'length')
    at countItems (C:\...\18-error-handling\trace.js:2:22)
    at printSummary (C:\...\18-error-handling\trace.js:6:17)
    at Object.<anonymous> (C:\...\18-error-handling\trace.js:11:1)
    at Module._compile (node:internal/modules/cjs/loader:1830:14)
    ...

Node.js v24.15.0
```

Read it from the top:

1. **Where it broke:** `trace.js:2` means line 2. Node shows that line, with a `^` under the spot where it went wrong.
2. **What went wrong:** `TypeError: Cannot read properties of undefined (reading 'length')`. So `order.items` was `undefined`, and the code tried to read `.length` of it. Order 8 has no `items`.
3. **How it got there:** the `at` lines. This list is called the **stack trace**: the chain of function calls that were in progress when the error happened, newest first. `countItems` (line 2, column 22) was called by `printSummary` (line 6), which was called from line 11. `Object.<anonymous>` means "the main part of your file, outside any function".
4. **What to skip:** lines starting with `node:internal` are Node's own code. You can ignore them.

Remember the call stack from chapter 17, the stack of plates? A stack trace is a photo of that stack at the moment things went wrong.

Also notice that `Done` never printed. An error nobody handles stops the whole program.

Tip: In the browser, the same error shows up in red in the console, starting with `Uncaught`. The file name and line number appear next to it, and you can click them to jump to the line.

Catching errors with `try` and `catch`

Crashing isn't always what you want. `try` and `catch` let you say: "try this, and if it fails, do that instead."

```
try {
  // code that might fail
} catch (error) {
  // runs only if something in try failed
}
```

Here's an order that doesn't have its customer details yet:

```
const order = { id: 42 }; // no customer details yet

try {
  console.log("Customer:", order.customer.name);
  console.log("This line is skipped.");
} catch (error) {
  console.log("Could not print the order:", error.message);
}

console.log("The program keeps going.");
```

You'll see:

```
Could not print the order: Cannot read properties of undefined (reading 'name')
The program keeps going.
```

Here's what happened, step by step:

1. JavaScript runs the `try` block line by line.
2. The first line fails. JavaScript **stops the try block right there** and jumps straight to `catch`. "This line is skipped." never prints.
3. The `catch` block runs. `error` holds information about what went wrong.
4. The program carries on after the whole `try / catch`, as if nothing had crashed.

If nothing fails inside `try`, the `catch` block is skipped completely.

The name `error` works like a function parameter: you pick it. You'll often see `err` or `e` in other people's code.

Tip: For a missing property like this one, `order.customer?.name` (optional chaining, from [chapter 11](#)) is simpler. `try / catch` is for problems you can't easily check in advance, and for errors thrown on purpose, which is coming up next.

The error object

When something fails, JavaScript creates an **error object** and hands it to `catch`. It has three useful properties:

```

try {
  const volume = 11;
  volume();
} catch (error) {
  console.log(error.name);    // prints: TypeError
  console.log(error.message); // prints: volume is not a function
  console.log(error.stack);   // prints: the stack trace, as text
}

```

Property	What it holds	Who it's for
<code>name</code>	The kind of error, like <code>"TypeError"</code>	Your code, to decide what to do
<code>message</code>	What went wrong, in words	You, and sometimes your users
<code>stack</code>	The name, message, and the <code>at</code> lines	You, when you're hunting a bug

Tip: `console.error()` works like `console.log()`, but it's meant for problems. In the browser console, its messages show up in red. It's a good habit inside `catch` blocks.

Throwing your own errors

JavaScript only throws errors for things it understands: broken grammar, unknown names, values used the wrong way. It has no idea that taking \$500 out of an account with \$100 in it is wrong. Only *your* code knows the rules of your app.

Remember the cash machine from [chapter 09](#)? Its `withdraw` function returned a message like "Sorry, you don't have enough money." when something was wrong. That's fine for printing, but the code that called it gets a string back either way, so it can't easily tell a problem from a success.

A better way: when your function hits a problem it can't solve, it raises the alarm with `throw`:

```

function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error("Not enough money in the account");
  }
  return balance - amount;
}

console.log(withdraw(100, 30)); // prints: 70
console.log(withdraw(100, 500));
// Error: Not enough money in the account

```

Two new things here:

- `new Error("...")` creates an error object with your message. It gets a `name` ("Error"), your `message`, and a `stack`, just like JavaScript's own errors.
- `throw` sends the error flying. The function stops on the spot, a bit like `return`, but for problems. The error travels back to whoever called the function, then to whoever called *that*, and so on, until some `catch` catches it. If nothing does, the program crashes.

Now the caller can catch it:

```
try {
  const newBalance = withdraw(100, 500);
  console.log("New balance:", newBalance);
} catch (error) {
  console.log("Sorry:", error.message);
}
// prints: Sorry: Not enough money in the account
```

That's the kitchen and the waiter again. `withdraw` spots *what* is wrong. The caller decides *what to do about it*: show a message, suggest a smaller amount, or something else.

Built-in error types

The error types from the start of this chapter (`SyntaxError`, `ReferenceError`, `TypeError`, `RangeError`) are all built into JavaScript, along with the general `Error`. You can throw them too. Plain `Error` is fine most of the time. Use `TypeError` when a value has the wrong type, and `RangeError` when a number is too big or too small:

```
function setVolume(level) {
  if (typeof level !== "number") {
    throw new TypeError(`Volume must be a number, got ${typeof level}`);
  }
  if (level < 0 || level > 10) {
    throw new RangeError(`Volume must be 0 to 10, got ${level}`);
  }
  console.log(`Volume set to ${level}`);
}

setVolume(7); // prints: Volume set to 7

try {
  setVolume(11);
} catch (error) {
  console.log(error.name, "-", error.message);
}
// prints: RangeError - Volume must be 0 to 10, got 11
```

Picking the right type pays off: the code that catches the error can check `error.name` and react differently to each kind. You'll see that in action a bit further down.

In [chapter 27](#), you'll learn to make your own error types, like `ValidationError`, and another way to check them (`instanceof`).

Guard clauses: check first, then do the work

You met **guard clauses** in chapter 09: `if` checks at the very top of a function that deal with a problem and leave early. They're like a bouncer at the door: trouble gets turned away before it gets inside. In chapter 09 they left with `return`. They work just as well with `throw`.

Here's the cash machine again, now checking every rule. Each rule gets one flat check, no nesting, and the real work sits at the bottom:

```
function withdraw(balance, amount) {
  if (typeof amount !== "number" || Number.isNaN(amount)) {
    throw new TypeError("Amount must be a number");
  }
  if (amount <= 0) {
    throw new RangeError("Amount must be more than 0");
  }
  if (amount > balance) {
    throw new Error(`Not enough money. Balance: ${balance}, requested: ${amount}`);
  }

  return balance - amount; // the happy path
}
```

Why the `Number.isNaN` check? Because `typeof NaN` is `"number"` ([chapter 05](#)), so the first check alone would let `NaN` through.

Let's test every rule in one go:

```
let balance = 100;
const requests = [30, -5, "fifty", 500];

for (const amount of requests) {
  try {
    balance = withdraw(balance, amount);
    console.log(`Withdrew ${amount}. Balance: ${balance}`);
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}
```

You'll see:

```
Withdrew 30. Balance: 70
RangeError: Amount must be more than 0
TypeError: Amount must be a number
Error: Not enough money. Balance: 70, requested: 500
```

Look at `balance = withdraw(balance, amount);`. When `withdraw` throws, the assignment never happens, so `balance` can't be damaged by a bad request. And because the `try / catch` is *inside* the loop, one bad request doesn't stop the others.

Checking values like this is called **validating input**. A simple rule for guard clauses:

- If there's a sensible answer, `return` it. (The total of an empty cart is just `0`.)
- If there's no sensible answer, `throw`. (There's no sensible way to withdraw "fifty".)

finally : code that always runs

Sometimes you need to do something no matter what happened: turn off a printer, hide a "Loading..." message, lock the door again. That's what `finally` is for.

```
function printTicket(seat) {
  console.log("Printer on");
  try {
    if (seat === "") {
      throw new Error("No seat chosen");
    }
    console.log(`Printing ticket for seat ${seat}`);
  } catch (error) {
    console.log("Print failed:", error.message);
  } finally {
    console.log("Printer off");
  }
}

printTicket("C7");
printTicket("");
```

You'll see:


```
Printer on
Printing ticket for seat C7
Printer off
Printer on
Print failed: No seat chosen
Printer off
```

The printer goes off both times. `finally` even runs when the `try` block uses `return`.

You can also write `try` and `finally` without a `catch`. Then the clean-up happens first, and the error carries on traveling to whoever called the function.

When to catch, and when to let it crash

Catching isn't always the right move. Before you write a `catch`, ask yourself: *can I do something useful here?*

Catch it when you can...	Let it crash when...
show the user a friendly message	it's a bug in your own code (a typo, a wrong property name)
fall back to a sensible default	there's nothing sensible to do at this spot
skip one bad item and carry on with the rest	carrying on would give wrong results, like charging a customer twice
try again (you'll see this with the network in chapter 33)	

A crash with a clear stack trace is annoying, but it's honest: it tells you exactly where to look. A program that hides a problem and keeps going with wrong data is much worse.

Often you only know how to handle *some* errors. Handle those, and pass the rest on by throwing the same error again. This is called **re-throwing**:

```

let balance = 100;

try {
  balance = withdraw(balance, -20);
} catch (error) {
  if (error.name === "RangeError") {
    console.log("Please enter an amount above 0.");
  } else {
    throw error; // not a problem we know how to handle: pass it on
  }
}
// prints: Please enter an amount above 0.

```

One rule has no exceptions: **never swallow an error silently**. An empty `catch` block makes the problem vanish without a trace. You'll see why in Common mistakes below.

Adding a cause

Sometimes a low-level error needs a clearer, big-picture message. "Card declined" is a detail. "Order 1042 failed" is what the rest of your program cares about. You can have both: throw a new error and attach the original as its **cause**.

```

function chargeCard(amount) {
  // Pretend the bank said no
  throw new Error("Card declined");
}

function placeOrder(orderId, amount) {
  try {
    chargeCard(amount);
  } catch (error) {
    throw new Error(`Order ${orderId} failed`, { cause: error });
  }
}

try {
  placeOrder(1042, 59.99);
} catch (error) {
  console.log(error.message); // prints: Order 1042 failed
  console.log(error.cause.message); // prints: Card declined
}

```

If nobody catches an error like this, Node prints both of them, with the original marked `[cause]`, so no detail gets lost.

Coming up: errors in code that *waits* (timers, loading data from the internet) need a few extra tricks. You'll learn them in [chapter 31](#) and [chapter 32](#). And in [chapter 23](#), you'll use `try / catch` to deal with broken data.

Common mistakes

1. Swallowing errors silently

```
function saveHighScore(score) {  
  throw new Error("Save file is full"); // pretend saving failed  
}  
  
try {  
  saveHighScore(9800);  
} catch (error) {  
  // do nothing  
}  
  
console.log("Game over. See you next time!");  
// prints: Game over. See you next time!
```

The save failed, but the player never finds out, and neither do you. Their high score is just gone. At the very least, report the problem:

```
} catch (error) {  
  console.error("Could not save your score:", error.message);  
}
```

And if you can't do anything useful with an error, don't catch it at all.

2. Throwing a string instead of an error

```
try {  
  throw "Out of stock";  
} catch (error) {  
  console.log(error.message); // prints: undefined  
}
```

JavaScript lets you throw anything, but a plain string has no `message`, no `name`, and no stack trace to show where it came from. Always throw an error object: `throw new Error("Out of stock");`

3. Using a variable from inside `try` after it

```

try {
  const newBalance = withdraw(100, 30);
} catch (error) {
  console.log(error.message);
}

console.log(newBalance);
// ReferenceError: newBalance is not defined

```

The curly braces of `try` make a block, and `const` and `let` stay inside their block ([chapter 14](#)). The simplest fix is to use the value inside `try`, right after the line that might fail: move `console.log(newBalance);` up, just below the `withdraw` call.

4. Expecting `try` to catch a syntax error

```

try {
  console.log("Hi");
} catch (error) {
  console.log("Caught it!");
}

// SyntaxError: missing ) after argument list

```

"Caught it!" never prints. A syntax error stops the file before *any* code runs, so the `try` never gets its chance. `try / catch` only catches runtime errors. Fix the typo instead.

5. A `catch` that blames the wrong thing

```

let balance = 100;

try {
  balance = withdraw(balance, 20);
  console.log(`New balance: ${balance}`); // typo!
} catch (error) {
  console.log("Not enough money!");
}

// prints: Not enough money!

```

The withdrawal worked. The real problem is the typo `balance`, a `ReferenceError`. But the `catch` guessed, and guessed wrong, so now you're hunting for a money problem that doesn't exist. To avoid this:

- Show `error.message` instead of guessing. Here it would say `balance is not defined` straight away.
- Keep `try` blocks small, around the code that can actually fail.
- Handle only the errors you expect, and re-throw the rest.

Quick recap

- The main kinds of errors are syntax, reference, type, range, and logic errors. Logic errors print no message, so always check your results.
 - Read a stack trace from the top: where it broke, what went wrong, then the `at` lines showing how it got there.
 - `try` runs code that might fail, `catch (error)` handles the failure, and `finally` runs no matter what.
 - The error object has a `name`, a `message`, and a `stack`.
 - `throw new Error("...")` lets your own functions raise the alarm. Use `TypeError` or `RangeError` when they fit.
 - Guard clauses check for problems at the top of a function and leave early, so the real work stays flat and clear.
 - Only catch errors you can do something about, and never swallow them silently. `{ cause: error }` adds a clearer message without losing the original.
-

Next: try the [exercises](#), then move on to [19 Dates and Times](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Name that error

Here are four broken pieces of code from a music app:

```
// 1
const playlist = { name: "Road Trip" };
console.log(playlist.songs.length);

// 2
console.log(playlistName);

// 3
const volume = 11;
volume();

// 4
console.log("la".repeat(-3));
```

First, **without running anything**, write down which kind of error you think each one causes: `SyntaxError`, `ReferenceError`, `TypeError`, or `RangeError`.

Then check your guesses. Put each numbered piece in its own `try / catch`, and in each `catch`, print the error's name and message. At the very end, print `All errors caught!`

Expected output:

```
TypeError: Cannot read properties of undefined (reading 'length')
ReferenceError: playlistName is not defined
TypeError: volume is not a function
RangeError: Invalid count value: -3
All errors caught!
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Cinema booking

You're writing the booking system for a small cinema. Write a function `bookSeats(seatsLeft, seatsWanted)` that returns how many seats are left after the booking, or throws an error if the booking breaks a rule.

The rules, in this order:

Problem	Throw
<code>seatsWanted</code> isn't a number	<code>TypeError</code> with <code>Seats must be a number</code>
fewer than 1 seat	<code>RangeError</code> with <code>You must book at least 1 seat</code>

Problem	Throw
more than 8 seats	<code>RangeError</code> with <code>You can book at most 8 seats at once</code>
more seats than are left	<code>Error</code> with <code>Sorry, only 6 seats left</code> (use the real number)

Then test it with this code at the bottom of your file:

```
let seatsLeft = 10;
const requests = [4, 0, "two", 12, 7, 2];
```

Loop over `requests`. Try each booking, update `seatsLeft` when it works, and print the error's name and message when it doesn't.

Expected output:

```
Booked 4 seats. 6 left.
RangeError: You must book at least 1 seat
TypeError: Seats must be a number
RangeError: You can book at most 8 seats at once
Error: Sorry, only 6 seats left
Booked 2 seats. 4 left.
```

Rule: use guard clauses. No `else`, and no `if` inside another `if`.

Stuck? The hint is in the [Hints section](#) at the end.

Bonus: when it works, comment out the type check and run it again. What happens to `seatsLeft`, and why did `"two"` get past the other rules?

Exercise 3 (Medium): Self-checkout scanner

A supermarket's self-checkout turns its scanner light on for every scan. The light must always go off again afterwards, even when a scan fails. Start with this data:

```
const catalog = {
  "4011": { name: "Bananas", price: 1.29 },
  "3017": { name: "Milk", price: 0.99 },
  "5020": { name: "Bread", price: 2.49 },
};

const basket = ["4011", "9999", "3017", "5020"];
```

1. Write `scanItem(barcode)` . It prints `Scanner light ON` , looks up the item, prints a beep line, and returns the price. If the barcode isn't in the catalog, it throws an error like `Unknown barcode 9999` . Either way, it must print `Scanner light OFF` at the end.
2. Scan everything in `basket` and add up the total. An unknown item shouldn't stop the checkout: print a message and carry on with the next item.

Expected output:

```
Scanner light ON
Beep! Bananas $1.29
Scanner light OFF
Scanner light ON
Scanner light OFF
Problem: Unknown barcode 9999. Please ask for help.
Scanner light ON
Beep! Milk $0.99
Scanner light OFF
Scanner light ON
Beep! Bread $2.49
Scanner light OFF
Total: $4.77
```

Rule: `scanItem` must not have a `catch` . It only uses `try` and `finally` . The `catch` belongs in the checkout loop.

When it works, look at where the `Problem:` line appears for the unknown item. Can you explain why it comes *after* `Scanner light OFF` ?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Bug hunt

This gym sign-up checker has **4 bugs**, all of them error-handling mistakes from the notes. Copy it into `ex4.js` , run it, and fix one problem at a time.


```
// Gym sign-up checker
function checkAge(age) {
  if (typeof age !== "number") {
    throw `age must be a number (got ${typeof age})`;
  }
  if (age < 16) {
    throw new RangeError(`must be 16 or older (got ${age})`);
  }
}

const newMembers = [
  { name: "Asha", age: 28 },
  { name: "Ben", age: 14 },
  { name: "Chen", age: "thirty" },
];

let checked = 0;

for (const member of newMembers) {
  try {
    checkAge(member.age);
    const message = "welcome!";
  } catch (error) {
  }
  console.log(`${member.name}: ${message}`);
  checked++;
}

console.log("Sign-ups checked:", checked);
```

When it's fixed, you should see:

```
Asha: welcome!
Ben: RangeError - must be 16 or older (got 14)
Chen: TypeError - age must be a number (got string)
Sign-ups checked: 3
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Grade importer

A teacher exported her class grades as lines of text, but some lines got mangled on the way. Your program imports the good lines, reports the bad ones, and never lets one bad line stop the rest.

```
const lines = [
  "Maya, 92",
  "Leo, 78",
  "Sofia, 105",
  "Omar, 85",
  "Ivy, A+",
  ", 70",
  "Zara 88",
  "Noah, 71",
];
```

Write three functions:

1. `parseLine(line)` turns `"Maya, 92"` into `{ name: "Maya", grade: 92 }`. It throws when the line is bad. Check in this order:

Problem	Throw
The line doesn't split into exactly 2 parts at the comma	Error with Expected "name, grade" but got "Zara 88"
The name is empty	Error with Name is missing
The grade isn't a number	TypeError with Grade "A+" is not a number
The grade is below 0 or above 100	RangeError with Grade 105 is out of range (0-100)

2. `importGrades(lines)` calls `parseLine` on every line. It returns an object with two arrays: `students` (the good results) and `problems` (a message for each bad line, like `Line 3: RangeError - ...`). Line numbers start at 1. If no line at all is valid, it throws an `Error` with `No valid grades found`.
3. `printReport(result)` prints the report you see below. Show the average with one decimal place.

Then run both of these at the bottom of your file:

```
printReport(importGrades(lines));

try {
  printReport(importGrades(["", "Bob"]));
} catch (error) {
  console.log(`Import failed: ${error.message}`);
}
```

Expected output:

```
Imported 4 students
Average grade: 81.5
Top student: Maya (92)
Problems:
- Line 3: RangeError - Grade 105 is out of range (0-100)
- Line 5: TypeError - Grade "A+" is not a number
- Line 6: Error - Name is missing
- Line 7: Error - Expected "name, grade" but got "Zara 88"
Import failed: No valid grades found
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Dates are one of the most common places for things to go wrong. What should a program do with a date like "February 30th"?

In [chapter 19](#), you'll meet `Invalid Date` and use what you learned here to guard against it.

19 Dates and Times

What is it?

JavaScript has a built-in `Date` object for working with dates and times: today's date, a birthday, a delivery time, the number of days until a holiday.

Each `Date` holds one exact moment in time.

Why does it matter?

Almost every app deals with dates. Orders have timestamps. Library books have due dates. Posts say "3 days ago". Booking sites count nights. Gyms track when memberships run out.

Dates are also famously tricky:

- In JavaScript, January is month `0`, not month `1`.
- `03/04/2026` means March 4 in the United States, but 3 April in the United Kingdom.
- The same moment can be Sunday evening in New York and already Monday morning in Tokyo.

Knowing these traps ahead of time will save you from some of the most confusing bugs in programming.

Real-world example

Picture a big football final that kicks off at one exact moment. Fans all over the world watch the same kickoff, live. But the TV guide in each country shows a different time:

City	The TV guide says
London	Sunday, 8:00 PM
New York	Sunday, 3:00 PM
Kathmandu	Monday, 1:45 AM
Tokyo	Monday, 5:00 AM

A JavaScript `Date` is like the kickoff itself: **one moment**, the same for everybody. When you ask a date for its hour, or print it nicely, JavaScript translates that moment into the clock reading for one particular place. Usually that's the time zone your computer is set to.

Notice that Kathmandu and Tokyo are already on Monday. Depending on where you are, even the *day* can be different. Keep that in mind: it's behind the most famous date bug of all, which you'll meet later in this chapter.

How it works

Creating a date

There are three common ways to make a date.

1. Right now:

```
const now = new Date();  
console.log(now); // prints something like: 2026-09-24T09:35:12.345Z
```

Your output will be different, because it's the moment you run it.

The time may also look wrong! Node prints dates in **UTC** (Coordinated Universal Time), the world's reference clock. Every time zone is measured from it: Nepal is UTC+5:45, and New York is UTC-5 in winter. The **Z** at the end means "this is UTC". You'll learn to show dates in your own time zone soon.

The browser console prints dates differently, in your local time, like Thu Sep 24 2026 15:20:12 GMT+0545 (Nepal Time).

2. From an ISO string:

```
const launch = new Date("2026-03-14T09:30:00Z");  
console.log(launch); // prints: 2026-03-14T09:30:00.000Z
```

ISO format is the international standard way to write a date: biggest part first (year, month, day), then a **T**, then the time. Everyone reads it the same way, which is why computers love it. The **Z** means the time is in UTC.

Stick to ISO format when you make a date from a string. Other formats, like "03/04/2026", are guesswork.

3. From parts:

```
const concert = new Date(2026, 2, 14, 19, 30); // year, month, day, hour, minute  
console.log(concert.toDateString()); // prints: Sat Mar 14 2026
```

Wait, month **2** is March? Yes. **Months are counted from 0**: January is **0**, February is **1**, and December is **11**. Days of the month start at 1 as normal. This trips up everyone, so expect to see it again.

A date made from parts uses your computer's **local time**: the time zone your computer is set to. `toString()` prints the date part in a short, readable way. Unlike `console.log`, it uses local time too.

Getting parts of a date

Once you have a date, you can ask it for each part:

```
const concert = new Date(2026, 2, 14, 19, 30); // March 14, 2026, 7:30 PM

console.log(concert.getFullYear()); // prints: 2026
console.log(concert.getMonth());    // prints: 2   (March, counting from 0)
console.log(concert.getDate());     // prints: 14  (the day of the month)
console.log(concert.getDay());      // prints: 6   (Saturday)
console.log(concert.getHours());    // prints: 19
console.log(concert.getMinutes());  // prints: 30
```

Method	Gives you	Watch out
<code>getFullYear()</code>	the year, like <code>2026</code>	
<code>getMonth()</code>	<code>0</code> to <code>11</code>	January is <code>0</code> !
<code>getDate()</code>	<code>1</code> to <code>31</code>	the day of the <i>month</i>
<code>getDay()</code>	<code>0</code> to <code>6</code>	the day of the <i>week</i> , and Sunday is <code>0</code>
<code>getHours()</code>	<code>0</code> to <code>23</code>	24-hour clock
<code>getMinutes()</code> , <code>getSeconds()</code>	<code>0</code> to <code>59</code>	

`getDay()` gives a number, so to show a name, use it as an index into an array ([chapter 10](#)). And add 1 to the month before showing it to a human:

```
const dayNames = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
console.log(dayNames[concert.getDay()]); // prints: Saturday
console.log(`Month number: ${concert.getMonth() + 1}`); // prints: Month number: 3
```

These methods all read the date in your local time. Each one also has a UTC twin, like `getUTCHours()` and `getUTCDate()`, which reads it in UTC instead.

Changing parts of a date

Every `get` method has a `set` partner: `setFullYear`, `setMonth`, `setDate`, `setHours`, `setMinutes`. They're handy for moving a date forward or back:

```
const due = new Date(2026, 0, 25); // January 25, 2026
due.setDate(due.getDate() + 10); // 10 days later
console.log(due.toDateString()); // prints: Wed Feb 04 2026
```

25 + 10 is 35, and there's no January 35th. JavaScript **rolls over** into the next month for you and gives you February 4th. It does the same for hours past 23, months past 11, and so on. That makes adding days easy, because you never have to worry about how long each month is.

Setters change the date object itself. Dates are objects, so if you want to keep the original, make a copy first by passing the old date to `new Date()`:

```
const borrowed = new Date(2026, 0, 25);
const dueDate = new Date(borrowed); // a copy
dueDate.setDate(dueDate.getDate() + 14);

console.log(borrowed.toDateString()); // prints: Sun Jan 25 2026
console.log(dueDate.toDateString()); // prints: Sun Feb 08 2026
```

Timestamps: a date is secretly a number

Under the hood, a date is just one number: how many milliseconds have passed since midnight UTC on January 1, 1970. That starting point is called the **Unix epoch**, and the number is called a **timestamp**. (A millisecond is a thousandth of a second.)

```
const launch = new Date("2026-01-15T00:00:00Z");
console.log(launch.getTime()); // prints: 1768435200000

console.log(new Date(0).toISOString()); // prints: 1970-01-01T00:00:00.000Z
```

- `date.getTime()` gives you a date's timestamp.
- `new Date(timestamp)` turns a timestamp back into a date.
- `Date.now()` gives you the timestamp for right now, without making a date object.

`Date.now()` is perfect for measuring how long something takes:

```
const start = Date.now();

let total = 0;
for (let i = 0; i < 50000000; i++) {
  total += i;
}

const elapsed = Date.now() - start;
console.log(`That took ${elapsed} ms`); // prints something like: That took 41 ms
```

Your number will be different. It depends on your computer, and it changes a little every run.

Date math: counting days

Because dates are numbers underneath, you can subtract one from another. The answer is in milliseconds, so divide to get days:

```
const today = new Date(2026, 8, 24); // September 24, 2026
const holiday = new Date(2026, 11, 25); // December 25, 2026

const msPerDay = 1000 * 60 * 60 * 24; // 86,400,000
const daysLeft = Math.round((holiday - today) / msPerDay);
console.log(`${daysLeft} days until the holiday`); // prints: 92 days until the holiday
```

Why `Math.round`? Many countries move their clocks for **daylight saving time**, so one day in a year is 23 hours long and another is 25. In London or New York, the division above gives `92.04166666666667`, because the clocks go back one hour in between. Rounding fixes that.

To add *calendar days*, use `setDate` like before. To add an exact amount of time, like hours on a parking ticket, add milliseconds to the timestamp:

```
const parkedAt = new Date("2026-05-20T08:15:00Z");
const expires = new Date(parkedAt.getTime() + 2 * 60 * 60 * 1000); // 2 hours later
console.log(expires.toISOString()); // prints: 2026-05-20T10:15:00.000Z
```

Comparing dates

`<` and `>` work on dates, because JavaScript compares their timestamps:


```
const deadline = new Date(2026, 5, 30, 23, 59); // June 30, 11:59 PM
const submitted = new Date(2026, 6, 2, 10, 0); // July 2, 10:00 AM

if (submitted > deadline) {
  console.log("Late submission!");
}
// prints: Late submission!
```

But `===` doesn't work the way you'd hope:

```
const a = new Date(2026, 0, 1);
const b = new Date(2026, 0, 1);

console.log(a === b); // prints: false
console.log(a.getTime() === b.getTime()); // prints: true
```

Dates are objects, and `===` checks whether two objects are the *same object*, not whether they hold the same value ([chapter 16](#)). To check if two dates are the same moment, compare their timestamps.

Subtraction also makes sorting easy. Remember compare functions from [chapter 13](#)?

```
const bookings = [
  { guest: "Ravi", checkIn: new Date(2026, 6, 18) },
  { guest: "Emma", checkIn: new Date(2026, 6, 3) },
  { guest: "Kenji", checkIn: new Date(2026, 6, 11) },
];

const byDate = bookings.sort((a, b) => a.checkIn - b.checkIn);
for (const booking of byDate) {
  console.log(booking.guest, booking.checkIn.toString());
}
```

You'll see:

```
Emma Fri Jul 03 2026
Kenji Sat Jul 11 2026
Ravi Sat Jul 18 2026
```

Formatting dates for people

For computers, use `toISOString()`. It always gives the same format, in UTC:

```
const launch = new Date("2026-03-14T09:30:00Z");
console.log(launch.toISOString()); // prints: 2026-03-14T09:30:00.000Z
```

For people, use `toLocaleDateString()` and `toLocaleTimeString()`. They format a date the way people in a particular place expect:

```
const concert = new Date(2026, 2, 14, 19, 30);

console.log(concert.toLocaleDateString("en-US")); // prints: 3/14/2026
console.log(concert.toLocaleDateString("en-GB")); // prints: 14/03/2026
```

"en-US" and "en-GB" are **locales**: short codes for a language and a region. `en-US` is English as written in the United States, and `en-GB` is English as written in Great Britain. There are hundreds more, like `fr-FR` (French, France) and `ja-JP` (Japanese, Japan). If you leave the locale out, JavaScript uses your computer's settings, so the result can be different on another computer.

Add an **options** object to choose exactly which parts to show, and how:

```
const options = { weekday: "long", year: "numeric", month: "long", day: "numeric" };

console.log(concert.toLocaleDateString("en-US", options)); // prints: Saturday, March 14, 2026
console.log(concert.toLocaleDateString("en-GB", options)); // prints: Saturday, 14 March 2026
console.log(concert.toLocaleDateString("fr-FR", options)); // prints: samedi 14 mars 2026

console.log(concert.toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit" })); //
prints: 7:30 PM
console.log(concert.toLocaleTimeString("en-GB", { hour: "2-digit", minute: "2-digit" })); //
prints: 19:30
```

The most useful options (these examples are for March 5th, 7:05 AM, in `en-US`):

Option	Values you can use	Examples
<code>weekday</code>	"long", "short"	Thursday, Thu
<code>year</code>	"numeric", "2-digit"	2026, 26
<code>month</code>	"long", "short", "numeric", "2-digit"	March, Mar, 3, 03
<code>day</code>	"numeric", "2-digit"	5, 05
<code>hour</code> , <code>minute</code>	"numeric", "2-digit"	7:05 AM, 07:05 AM

If you format lots of dates the same way, make a formatter once with `Intl.DateTimeFormat` and reuse it. It's the date version of `Intl.NumberFormat` from [chapter 05](#):

```
const formatter = new Intl.DateTimeFormat("en-GB", { weekday: "short", day: "numeric", month: "short" });

const fixtures = [new Date(2026, 7, 22), new Date(2026, 7, 29), new Date(2026, 9, 3)];
for (const match of fixtures) {
  console.log(formatter.format(match));
}
```

You'll see:

```
Sat 22 Aug
Sat 29 Aug
Sat 3 Oct
```

Watch out: small details, like commas or short month names, can change between versions of Node and browsers. For example, the current `en-GB` rules shorten September to `Sept`, not `Sep`. That's fine on a web page, but don't write code that depends on the exact text.

Time zones: UTC vs. local

Back to the football final. A date stores one moment. The methods you've seen read that moment in one of two ways:

- **Local time:** `getHours()`, `getDate()`, `toLocaleDateString()` and friends use your computer's time zone.
- **UTC:** `getUTCHours()`, `getUTCDate()`, and `toISOString()` use UTC.

You can also show a date in *any* time zone by passing a `timeZone` option. Time zones have names made of a region and a big city, like `"Europe/London"` or `"Asia/Kathmandu"`:

```
const kickoff = new Date("2026-12-20T20:00:00Z");

const cities = [
  { city: "London", timeZone: "Europe/London" },
  { city: "New York", timeZone: "America/New_York" },
  { city: "Kathmandu", timeZone: "Asia/Kathmandu" },
  { city: "Tokyo", timeZone: "Asia/Tokyo" },
];

for (const { city, timeZone } of cities) {
  const time = kickoff.toLocaleString("en-US", { timeZone, weekday: "long", hour: "numeric", minute: "2-digit" });
  console.log(`${city}: ${time}`);
}
```

You'll see:

```
London: Sunday 8:00 PM
New York: Sunday 3:00 PM
Kathmandu: Monday 1:45 AM
Tokyo: Monday 5:00 AM
```

That's the TV guide from the start of the chapter. `toLocaleString()` works like `toLocaleDateString()`, but it can show the time as well.

The famous "wrong day" bug

Here's a bug that has bitten almost every developer. Someone's birthday is January 15th:

```
const birthday = new Date("2026-01-15");
console.log(birthday.getDate());
// prints: 15 in London, Kathmandu, or Tokyo
// prints: 14 in New York or Los Angeles!
```

What happened? A date-only string like `"2026-01-15"` is read as **midnight UTC**. When it's midnight in London, it's still 7 PM on January 14th in New York. So `getDate()`, which uses local time, says 14.

You can see both results on any computer by choosing the time zone yourself:

```
console.log(birthday.toLocaleDateString("en-US", { timeZone: "UTC" })); // prints: 1/15/2026
console.log(birthday.toLocaleDateString("en-US", { timeZone: "America/New_York" })); // prints: 1/14/2026
```

If you live east of London, like in Nepal or India, you won't see this bug on your own computer, because midnight UTC is already morning where you are. But your users in the Americas will. That's what makes it so sneaky.

To make it more confusing, the rules change when you add a time:

You write	JavaScript reads it as
<code>new Date("2026-01-15")</code>	midnight UTC
<code>new Date("2026-01-15T00:00Z")</code>	midnight UTC
<code>new Date("2026-01-15T00:00")</code>	midnight in your local time
<code>new Date(2026, 0, 15)</code>	midnight in your local time

The fix: for a date on the calendar with no real time attached (a birthday, a due date, a holiday), build it from parts: `new Date(2026, 0, 15)`. It means "January 15th, here", and it stays the 15th everywhere.

Storing dates

When you save a date to use later, store it as an ISO string or a timestamp:

```
const orderTime = new Date("2026-06-01T08:15:00Z");

const saved = orderTime.toISOString(); // "2026-06-01T08:15:00.000Z": safe to store
const loaded = new Date(saved);        // turn it back into a date later

console.log(loaded.getTime() === orderTime.getTime()); // prints: true
```

Don't store dates as formatted text like `"03/04/2026"` or `"Saturday, 14 March"`. Nobody can be sure what `03/04` means, and turning text like that back into a date is unreliable. Format a date only at the last moment, when you show it to a person.

In [chapter 23](#), you'll save data in the browser, and dates turn into strings on the way. This is how you'll bring them back.

Invalid dates

If JavaScript can't understand a date, you don't get an error. You get an **Invalid Date**:

```
const bad = new Date("next Tuesday-ish");

console.log(bad); // prints: Invalid Date
console.log(bad.getTime()); // prints: NaN
console.log(Number.isNaN(bad.getTime())); // prints: true

bad.toISOString();
// RangeError: Invalid time value
```

An invalid date is still a date object, so it quietly travels through your program until something finally breaks. Guard against it early, just like in [chapter 18](#):

```
function parseDate(text) {
  const date = new Date(text);
  if (Number.isNaN(date.getTime())) {
    throw new Error(`Not a valid date: "${text}"`);
  }
  return date;
}
```

JavaScript won't catch every impossible date, though. `new Date(2026, 1, 30)` doesn't fail: February 30th rolls over to March 2nd. When impossible dates matter, check them yourself. You'll do exactly that in the exercises.

What's next for dates: Temporal

`Date` is old (it dates back to 1995) and has well-known problems, like the months starting at 0. A much better replacement called **Temporal** is now officially part of JavaScript. As of 2026, it works in recent versions of Chrome, Edge, and Firefox, but not yet in Safari, and **not in Node 24.15.0**:

```
console.log(typeof Temporal); // prints: undefined
```

Until it works everywhere, you'll keep meeting `Date` in real code, so everything in this chapter stays useful. There are also popular free libraries, like **date-fns** and **Day.js**, that make date work easier. You'll learn to install libraries in [chapter 50](#).

Common mistakes

1. Forgetting that months start at 0

```
const christmas = new Date(2026, 12, 25);
console.log(christmas.toString()); // prints: Mon Jan 25 2027
```

Month `12` doesn't exist, so it rolled over into January of the *next* year. December is `11`: `new Date(2026, 11, 25)`. And when you show a month number to a person, add 1.

2. Mixing up `getDay` and `getDate`

```
const concert = new Date(2026, 2, 14);
console.log(`The concert is on the ${concert.getDay()}th`); // prints: The concert is on the 6th
```

`getDay()` is the day of the *week* (6 means Saturday). For the day of the *month*, use `getDate()`, which gives `14`. One way to remember it: `getDate` is the number you'd circle on a calendar.

3. Changing a date you meant to keep

```
const checkIn = new Date(2026, 6, 3);
const checkOut = checkIn; // not a copy!
checkOut.setDate(checkOut.getDate() + 4);

console.log(checkIn.toString()); // prints: Tue Jul 07 2026
```

The guest's check-in date moved too. `checkOut` and `checkIn` point to the same date object ([chapter 16](#)), and `setDate` changes that object. Make a real copy first: `const checkOut = new Date(checkIn);`

4. Comparing dates with `===`

```
console.log(new Date(2026, 0, 1) === new Date(2026, 0, 1)); // prints: false
```

Two different date objects are never `===`, even for the same moment. Compare their timestamps with `getTime()` instead. `<` and `>` are fine as they are.

5. Using `getYear()`

```
const payday = new Date(2026, 2, 14);
console.log(payday.getYear()); // prints: 126
```

`getYear()` is an old method from the 1990s that counts years from 1900. It only still exists so that old websites don't break. Always use `getFullYear()`.

Quick recap

- Make dates with `new Date()` (now), an ISO string like `"2026-03-14T09:30:00Z"`, or parts like `new Date(2026, 2, 14)`. Months count from 0!
- `getFullYear`, `getMonth`, `getDate`, `getDay`, and `getHours` read the parts in local time. The `set` methods change them and roll over when needed.
- Underneath, a date is a timestamp: milliseconds since 1970. Subtract dates to get milliseconds, then divide and round to get days.
- Compare dates with `<` and `>`, or with `getTime()` for "the same moment". Never use `===`.
- Format dates for people with `toLocaleDateString` or `Intl.DateTimeFormat`, using a locale and options. For computers, use `toISOString()`.
- A date-only string like `"2026-01-15"` means midnight UTC, and it can show up as the day before. Build calendar dates from parts instead.
- Store dates as ISO strings or timestamps. `Temporal` is the better API on its way, but it isn't in Node 24.15.0.

Next: try the [exercises](#), then move on to [20 DOM Basics](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Every exercise uses fixed dates, so your output should match the expected output exactly, wherever you live.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Wedding invitation

Your friends are getting married on **Saturday, July 4th, 2026, at 3:05 PM**. Create that date from parts, then print these facts about it, using the date's `get` methods:

```
Year: 2026
Month: 7
Day: 4
Weekday: Saturday
Time: 15:05
Invitation: Saturday, 4 July 2026
```

Rule: don't type `7` , `Saturday` , or `05` yourself. Work them out from the date.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Library late fee

A library lends books for **21 days**, and charges **\$0.25 for each day** a book is late.

A member borrowed a book on **November 20th, 2026** and returned it on **December 15th, 2026**. Work out the due date and the late fee:

Borrowed: Friday, 20 November 2026
Due back: Friday, 11 December 2026
Returned: Tuesday, 15 December 2026
Days late: 4
Late fee: \$1.00

Rule: the `borrowed` date must not change. Keep it in its own variable, and make a copy to work out the due date.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: if the book comes back early, the fee should be `$0.00`, not a negative number. Try a return date of December 1st.

Exercise 3 (Medium): What's coming up?

You're building the "Upcoming" panel of a calendar app. Pretend today is **September 24th, 2026**, and start with these events:

```
const today = new Date(2026, 8, 24);

const events = [
  { name: "Dentist", date: new Date(2026, 9, 2) },
  { name: "Mum's birthday", date: new Date(2026, 8, 24) },
  { name: "Rent due", date: new Date(2026, 9, 1) },
  { name: "Concert", date: new Date(2026, 8, 20) },
  { name: "Holiday trip", date: new Date(2026, 9, 19) },
  { name: "Football match", date: new Date(2026, 8, 25) },
];
```

Print the events in date order, with a short label for how far away each one is:

```
Concert (Sun, Sep 20): 4 days ago
Mum's birthday (Thu, Sep 24): today!
Football match (Fri, Sep 25): tomorrow
Rent due (Thu, Oct 1): in 7 days
Dentist (Fri, Oct 2): in 8 days
Holiday trip (Mon, Oct 19): in 25 days
```

Rule: write a function `describe(days)` that turns a number of days into the label (`today!` , `tomorrow` , `in 7 days` , `4 days ago`). And use `Intl.DateTimeFormat` with the `"en-US"` locale for the dates in brackets.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Meeting across time zones

Your team works from four cities, and there's a video call on **October 5th, 2026 at 16:00 UTC**:

```
const meeting = new Date("2026-10-05T16:00:00Z");

const team = [
  { name: "Aarav", city: "Kathmandu", timeZone: "Asia/Kathmandu" },
  { name: "Chloe", city: "London", timeZone: "Europe/London" },
  { name: "Diego", city: "Mexico City", timeZone: "America/Mexico_City" },
  { name: "Hana", city: "Tokyo", timeZone: "Asia/Tokyo" },
];
```

For each person, show the meeting in their own time, and whether it falls inside their work hours (9 AM up to 6 PM). Then count how many people it suits.

Expected output:

```
Meeting: 2026-10-05T16:00:00.000Z
Aarav (Kathmandu): Monday 9:45 PM - outside work hours
Chloe (London): Monday 5:00 PM - OK
Diego (Mexico City): Monday 10:00 AM - OK
Hana (Tokyo): Tuesday 1:00 AM - outside work hours
2 of 4 people can join during work hours.
```

Stuck? [Hint 1](#) and [Hint 2](#) are in the [Hints section](#) at the end.

Bonus: try other meeting times (change the `16:00`) and find one that suits three of the four people.

Exercise 5 (Challenge): Gym memberships

A gym keeps its sign-up sheet as text, and some dates were typed badly. Pretend today is **September 24th, 2026**:

```
const today = new Date(2026, 8, 24);

const members = [
  { name: "Priya", joined: "2026-08-30", planDays: 30 },
  { name: "Tom", joined: "2026-09-10", planDays: 30 },
  { name: "Lina", joined: "2026-02-30", planDays: 90 },
  { name: "Marco", joined: "2026-06-15", planDays: 90 },
  { name: "Sven", joined: "2026-13-05", planDays: 30 },
  { name: "Ben", joined: "2026-08-25", planDays: 30 },
  { name: "Aiko", joined: "2026-09-20", planDays: 7 },
];
```

A membership runs out `planDays` days after the day someone joined.

1. Write `parseDay(text)`. It turns a string like `"2026-08-30"` into a date for that day in local time. If the date is impossible (like February 30th, or month 13), it throws a `RangeError` with the message `Invalid date: 2026-02-30`.
2. For every member, work out when their membership runs out and how many days are left. A bad date must not stop the others: collect a problem message instead.
3. Print the members with the soonest end date first, then the problems.

Expected output:

```
Marco: expired 11 days ago (Sun, Sep 13)
Ben: expires today (Thu, Sep 24)
Aiko: 3 days left (expires Sun, Sep 27)
Priya: 5 days left (expires Tue, Sep 29)
Tom: 16 days left (expires Sat, Oct 10)
Problems:
- Lina: RangeError - Invalid date: 2026-02-30
- Sven: RangeError - Invalid date: 2026-13-05
```

Rule: don't use `new Date("2026-08-30")`. You know why from the notes: it means midnight UTC, so in some time zones every day would come out one too early. And JavaScript doesn't even complain about `"2026-02-30"`: it quietly turns it into March 2nd.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Everything you've built so far prints to the terminal. But a calendar app, a library system, or a gym dashboard would show all this on a web page.

In [chapter 20](#), you'll finally use JavaScript to change what's on a real web page. 😊

20 DOM Basics

What is it?

The **DOM** (Document Object Model) is the browser's live copy of your web page, built out of objects that JavaScript can read and change.

When your code changes the DOM, the page on the screen changes straight away.

Why does it matter?

Until now, your programs have only talked to you through the terminal. Real web apps change what's on the page:

- a cart counter goes from 2 to 3,
- search results appear under a search box,
- a warning turns red when a password is too short,
- a to-do list grows as you add tasks.

Every one of those is JavaScript changing the DOM. It's the bridge between your code and what people actually see. This chapter is also where you start running code in the browser, which you'll keep doing in the next few chapters.

Real-world example

Remember the house from [chapter 01](#)? HTML was the walls and rooms. Let's take that one step further:

Building a house	Building a web page
The architect's blueprint, on paper	Your <code>index.html</code> file
The real house the builders make from the blueprint	The DOM, which the browser builds from your HTML
A renovation crew that repaints walls and adds shelves while people live there	JavaScript, changing the DOM while the page is open
Knocking the house down and rebuilding it from the blueprint	Refreshing the page

Notice the last row. JavaScript changes the *house*, never the *blueprint*. When you refresh the page, the browser builds a fresh DOM from your HTML file, and every change your script made is gone, until the script runs again.

How it works

Just enough HTML

You don't need to be an HTML expert for this course, but you do need to read it. Here's the minimum.

HTML is made of **tags**: words inside angle brackets. Most come in pairs, an opening tag and a closing tag with a `/`:

```
<p>Fresh bread every morning.</p>
```

The whole thing (opening tag, content, closing tag) is an **element**. This one is a `p` element, a paragraph.

Elements can sit inside other elements, like boxes inside boxes:

```
<ul>
  <li>Milk</li>
  <li>Eggs</li>
</ul>
```

`ul` means "unordered list" (a list with bullet points), and each `li` is a "list item". The indentation is only there to make it easier for humans to read.

Attributes add extra information to an element. They go inside the opening tag, as `name="value"`:

```
<a href="https://www.wikipedia.org">Wikipedia</a>

```

`href` tells a link where to go. `src` tells an image which file to show, and `alt` describes the image for people who can't see it. Some elements, like `img`, have no content, so they have no closing tag.

Two attributes matter a lot for JavaScript:

- `id` gives one element a unique name, like a passport number. No two elements on a page should share an `id`.
- `class` puts an element in a group, like a team shirt. Many elements can share a class, and one element can have several, separated by spaces: `class="card featured"`.

```
<h1 id="shop-name">Bella's Bakery</h1>
<p class="price">$3.50</p>
<p class="price sale">$2.00</p>
```

These elements are enough for this chapter:

Element	What it is
<code><h1></code> to <code><h6></code>	Headings, from biggest (<code>h1</code>) to smallest
<code><p></code>	A paragraph of text
<code></code>	A link
<code></code>	An image
<code></code> and <code></code>	A bullet list and its items (<code></code> makes a numbered list)
<code><button></code>	A button
<code><div></code>	A plain box for grouping other elements
<code></code>	A plain wrapper for a few words inside a line of text

The page skeleton

Every web page has the same basic shape:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Bella's Bakery</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1 id="shop-name">Bella's Bakery</h1>
    <p id="message">Fresh bread every morning.</p>
  </body>
</html>

```

Part	What it does
<code><!DOCTYPE html></code>	Tells the browser "this is a modern HTML page". Always the first line.
<code><html lang="en"></code>	Wraps the whole page. <code>lang="en"</code> says the page is in English.
<code><head></code>	Information <i>about</i> the page. Nothing in here shows up on the page itself.
<code><meta charset="UTF-8"></code>	Makes sure every character displays properly, from <code>é</code> to emoji.
<code><title></code>	The text on the browser tab.

Part	What it does
<code><script src="script.js" defer></code>	Loads your JavaScript file. You'll see what <code>defer</code> does in a moment.
<code><body></code>	Everything you actually see on the page.

A tiny bit of CSS

CSS controls how things look. In this course, you'll see small bits of it inside a `<style>` tag in the `<head>`:

```
<style>
  .sale { color: red; }
  #shop-name { font-size: 40px; }
</style>
```

Each rule starts with a **selector** that picks which elements it applies to. `.sale` (with a dot) means "every element with the class `sale`". `#shop-name` (with a hash) means "the element with the id `shop-name`". Then, inside the curly braces, come the looks: here, red text and big letters.

Remember those selectors. JavaScript uses exactly the same ones to find elements.

Your first page with JavaScript

Let's set up a page to practice on. You'll use these same steps for every browser exercise.

Step 1: Make the files. In VS Code, inside this chapter's folder, create a folder called `playground`. Inside it, create a file called `index.html` and put the page skeleton from above in it. (Copying and pasting HTML is fine. Save your typing practice for the JavaScript.) Then create a second file, `script.js`, in the same folder:

```
console.log("Hello from script.js!");
```

Save both files.

Step 2: Open the page. In VS Code, right-click `index.html` and choose **Reveal in File Explorer**. Then double-click `index.html`. It opens in your web browser, showing "Bella's Bakery" in big letters. (If it opens in the wrong program, right-click it, choose **Open with**, and pick Chrome or Microsoft Edge.)

Step 3: Open the console. Press `F12` and click the **Console** tab, like in chapter 01. You'll see:

```
Hello from script.js!
```


That message came from your `script.js` file. Your JavaScript is connected to your page! 🎉

Step 4: The everyday loop. From now on, you'll work like this:

1. Change `script.js` in VS Code.
2. Save it (`Ctrl + S`).
3. Refresh the browser (`F5`). The browser doesn't notice your changes by itself.
4. Check the page, and check the Console for errors.

A few things worth knowing:

- The address bar shows something like `file:///C:/Users/.../playground/index.html`. `file://` means the page came from your own computer, not from the internet. That's fine for now.
- Errors show up in red in the Console, with the file name and line number on the right. Click them to jump straight to the line.
- You can type JavaScript straight into the Console, and it runs on your page. Try typing `document.title` and pressing `Enter`: the console answers with the text from the `<title>` tag.

defer : wait for the page first

The browser reads your HTML from top to bottom, building the page as it goes. When it meets a normal `<script>` tag, it stops and runs that script immediately. In the `<head>`, that's too early: the `<body>` hasn't been built yet, so your script can't find anything on the page.

`defer` fixes that. It means: "download this script now, but don't run it until the whole page has been read." It's like a painter who waits until the walls are up before starting to paint.

In older code, you'll often see the `<script>` tag at the very end of the `<body>` instead. That works for the same reason. This course always uses `defer` in the `<head>`.

The DOM: a family tree of objects

When the browser reads your HTML, it builds a tree of objects, one for every element. For the skeleton page above, the tree looks like this:

```
document
├── html
│   ├── head
│   │   ├── meta
│   │   ├── title
│   │   └── script
│   └── body
│       ├── h1
│       └── p
```

It's a family tree. `body` is the **parent** of `h1` and `p`. They are its **children**, and they're **siblings** of each other, because they share a parent.

Each item in the tree is called a **node**. The nodes made from tags are **elements**, and the text inside them gets small text nodes of its own. In this chapter, you'll work with elements.

At the very top sits `document`: an object that represents the whole page. It's your starting point for everything in this chapter.

Tip: In DevTools, click the **Elements** tab to see this tree for real. You can open and close each part. Right-click anything on the page and choose **Inspect** to jump straight to its element. When your script changes the page, the Elements tab shows the change. But **View page source** (Ctrl + U) shows your original HTML file: the blueprint, not the house.

Finding elements

Before you can change an element, you need to find it. For the rest of this chapter, use this `<body>` in your playground page. Try each example on its own: replace what's in `script.js` (or turn the older lines into comments), save, and refresh.

```
<body>
  <h1 id="shop-name">Bella's Bakery</h1>
  <p id="message">Fresh bread every morning.</p>
  <ul id="menu">
    <li class="item">Sourdough loaf</li>
    <li class="item sale">Croissant</li>
    <li class="item">Cinnamon roll</li>
  </ul>
</body>
```

`getElementById` finds the one element with that `id`:

```
const heading = document.getElementById("shop-name");
console.log(heading.textContent); // prints: Bella's Bakery
```

(`textContent` is the text inside an element. More on it in a moment.)

`querySelector` finds the first element that matches a CSS selector, the same kind you saw in the `<style>` tag:

```
const firstItem = document.querySelector(".item");
console.log(firstItem.textContent); // prints: Sourdough Loaf

const onSale = document.querySelector("li.sale");
console.log(onSale.textContent); // prints: Croissant
```

Selector	Finds
"#menu"	the element with <code>id="menu"</code>
".item"	elements with the class <code>item</code>
"li"	<code></code> elements
"#menu li"	<code></code> elements anywhere inside <code>#menu</code>
"li.sale"	<code></code> elements that also have the class <code>sale</code>

Notice that `getElementById` takes just the name (`"shop-name"`), while `querySelector` needs the `#` (`"#shop-name"`). Mixing those up is a classic mistake.

`querySelectorAll` finds *all* the matches:

```
const items = document.querySelectorAll(".item");
console.log(items.length); // prints: 3

for (const item of items) {
  console.log(item.textContent);
}
```

You'll see:

```
Sourdough loaf
Croissant
Cinnamon roll
```

`querySelectorAll` gives you a **NodeList**: a list of elements. You can loop over it with `for...of` or `forEach`, and it has a `length`. But it isn't a real array, so methods like `map` and `filter` aren't there. If you need them, turn it into an array first with `Array.from()` ([chapter 13](#)):

```
const names = Array.from(items).map((item) => item.textContent);
console.log(names.join(", ")); // prints: Sourdough Loaf, Croissant, Cinnamon roll
```

You can also search *inside* an element, instead of the whole page: `menu.querySelector("li")` only looks inside `menu`.

If nothing matches, `getElementById` and `querySelector` give you `null`. Try to use that `null`, and you'll see an error like this in the Console:

```
const cake = document.querySelector(".cake"); // there's no .cake on the page
cake.textContent = "Chocolate cake";
// Uncaught TypeError: Cannot set properties of null (setting 'textContent')
```

When you see `null` in an error like that, your selector didn't find anything. Check the spelling, the `#` or `.`, and that your script has `defer`.

Changing text with `textContent`

`textContent` reads the text inside an element. Set it, and you replace everything inside with new text:

```
const message = document.getElementById("message");
console.log(message.textContent); // prints: Fresh bread every morning.

message.textContent = "Sorry, we're sold out of bread today!";
```

Save, refresh the page, and the paragraph now says "Sorry, we're sold out of bread today!". Template literals ([chapter 06](#)) make it easy to mix in values:

```
const loavesLeft = 4;
message.textContent = `Only ${loavesLeft} loaves left today!`;
```

`textContent` always treats what you give it as plain text. If you set it to `"<b>Hello</b>"`, the page shows those exact characters, angle brackets and all. That might sound like a limitation, but it's actually a safety feature, as you're about to see.

(You'll also see `innerHTML` in tutorials. It's similar, but `textContent` is simpler and faster, so this course uses it.)

`innerHTML`, and why to be careful

`innerHTML` is like `textContent`, but it reads your string as HTML and builds real elements from it:

```
const menu = document.getElementById("menu");
menu.innerHTML = "<li>Baguette</li><li>Bagel</li>";
```

The menu now has two items, Baguette and Bagel, and the old ones are gone.

Watch out: never put anything a user typed (or anything that came from another website) into `innerHTML`. Someone could type HTML that contains their own JavaScript, and your page would run it. This kind of attack is called **XSS** (cross-site scripting), and it's one of the most common security holes on the web. You'll learn the full story in [chapter 51](#). Until then, the rule is simple: use `textContent` for text, and `createElement` (coming up soon) for new elements.

Changing styles: `style` vs. `classes`

Every element has a `style` property for changing its looks directly:

```
const heading = document.getElementById("shop-name");
heading.style.color = "darkred";
heading.style.backgroundColor = "lightyellow";
heading.style.fontSize = "48px";
```

Two details: CSS names with dashes become camelCase (`background-color` becomes `backgroundColor`), and the values are strings, with units like `"px"` included.

That works, but there's usually a better way: describe the looks in CSS, and let JavaScript switch **classes** on and off. Add this inside the `<head>` of your playground page:

```
<style>
  .highlight { background-color: gold; }
  .hidden { display: none; }
</style>
```

(`display: none` hides an element completely.) Now use `classList`:

```
const croissant = document.querySelector(".sale");
croissant.classList.add("highlight"); // gold background
croissant.classList.remove("sale");
console.log(croissant.classList.contains("sale")); // prints: false

const message = document.getElementById("message");
message.classList.toggle("hidden"); // hidden now
message.classList.toggle("hidden"); // visible again
```

Method	What it does
<code>classList.add("name")</code>	Adds the class
<code>classList.remove("name")</code>	Removes the class

Method	What it does
<code>classList.toggle("name")</code>	Adds it if it's missing, removes it if it's there
<code>classList.contains("name")</code>	<code>true</code> if the element has the class, otherwise <code>false</code>

Why are classes better?

- The looks stay in CSS, where they belong. JavaScript only says *what* something is: "on sale", "hidden", "selected".
- One class can change lots of properties at once.
- Undoing is easy: remove the class, and the element goes back to normal.

Keep `style` for values your code calculates, like the width of a progress bar:

```
bar.style.width = `${percent}%`; // bar is an element, percent is a number like 65
```

Attributes

Remember attributes, like `href` and `src`? You can read and change them too. Add these to your playground's `<body>` to try it:

```
<a id="order-link" href="order.html">Order online</a>

<button id="order-button">Order now</button>
```

`getAttribute` and `setAttribute` work with any attribute:

```
const link = document.getElementById("order-link");
console.log(link.getAttribute("href")); // prints: order.html

link.setAttribute("href", "https://example.com/order");
```

Most common attributes are also properties with the same name, which is shorter to write:

```
const photo = document.getElementById("photo");
photo.src = "cinnamon-roll.jpg";
photo.alt = "A cinnamon roll with icing";

const button = document.getElementById("order-button");
button.disabled = true; // the button turns grey and can't be clicked
```

On/off attributes, like `disabled`, become `true / false` properties.

One difference to know about: a property like `link.href` or `photo.src` gives you the *full* address, like `file:///C:/Users/.../order.html`. `getAttribute` gives you exactly what's written in the HTML, like `order.html`.

You can also invent your own attributes to store extra information on an element, as long as their names start with `data-`. For example, an "Add" button can remember which product it adds. JavaScript reads these through `dataset`:

```
<button id="add-croissant" data-id="3" data-price="2.50">Add a croissant</button>
```

```
const addButton = document.getElementById("add-croissant");
console.log(addButton.dataset.id);    // prints: 3
console.log(addButton.dataset.price); // prints: 2.50
```

Attribute values are always strings, so `addButton.dataset.price` is the string `"2.50"`. Convert it with `Number()` before doing math ([chapter 05](#)). You'll find `data-` attributes very handy in the next chapter.

Creating and removing elements

To add something new to the page, create an element, fill it in, then put it somewhere:

```
const menu = document.getElementById("menu");

const newItem = document.createElement("li"); // a new <li>, not on the page yet
newItem.textContent = "Apple pie";
newItem.classList.add("item");

menu.append(newItem); // now it's on the page, at the end of the list
```

`createElement` makes a brand-new element that isn't on the page yet. It's like a new shelf still sitting in the delivery van. Nothing shows up until you put it somewhere.

Method	What it does
<code>parent.append(child)</code>	Puts <code>child</code> inside <code>parent</code> , at the end
<code>parent.prepend(child)</code>	Puts <code>child</code> inside <code>parent</code> , at the start
<code>element.remove()</code>	Takes <code>element</code> off the page

```
const special = document.createElement("li");
special.textContent = "Today only: Pumpkin bread";
menu.prepend(special); // goes to the top of the list

const croissant = document.querySelector(".sale");
croissant.remove(); // sold out, so it's gone
```

You'll also see `appendChild` in older code. It's the older version of `append`.

Rendering a list from an array

Here's the pattern you'll use more than anything else in this chapter: take an array of data and turn it into elements on the page. This is called **rendering**.

```
const breads = [
  { name: "Sourdough loaf", price: 6.5, onSale: false },
  { name: "Croissant", price: 2.5, onSale: true },
  { name: "Cinnamon roll", price: 3.25, onSale: false },
  { name: "Baguette", price: 3, onSale: true },
];

function renderMenu(items) {
  const menu = document.getElementById("menu");
  menu.textContent = ""; // clear out whatever was there

  for (const item of items) {
    const li = document.createElement("li");
    li.textContent = `${item.name}: ${item.price.toFixed(2)}`;
    if (item.onSale) {
      li.classList.add("sale");
    }
    menu.append(li);
  }
}

renderMenu(breads);
```

The list on the page now shows:

- Sourdough loaf: \$6.50
- Croissant: \$2.50
- Cinnamon roll: \$3.25
- Baguette: \$3.00

The croissant and the baguette also get the `sale` class, so a CSS rule like `.sale { color: red; }` would make them stand out.

Why put it in a function? Because whenever the data changes, you can call `renderMenu` again, and the page catches up. Setting `textContent` to `""` first empties the list, so nothing shows up twice. And because it takes any array, it works with your array methods from chapter 13 too. `renderMenu(breads.filter((bread) => bread.onSale))` shows only the bargains.

Right now, the page changes only once: when your script runs. To make it react when someone clicks or types, you need *events*, and that's the next chapter.

Common mistakes

1. Forgetting `defer`

```
<script src="script.js"></script>
```

Without `defer`, the script runs before the `<body>` exists, so every search for an element comes back `null`, and the Console shows an error like this one:

```
Uncaught TypeError: Cannot set properties of null (setting 'textContent')
```

Add `defer`: `<script src="script.js" defer></script>`.

2. Mixing up `#`, `.`, and plain names

```
const total = document.querySelector("total"); // Looks for a <total> tag!
total.textContent = "$12.00";
// Uncaught TypeError: Cannot set properties of null (setting 'textContent')
```

`querySelector` needs a CSS selector: `"#total"` for an `id`, `".total"` for a class. `getElementById` is the other way round: it wants the plain name, `"total"`, and `getElementById("#total")` finds nothing.

3. Treating a list of elements like a single element

```
const prices = document.querySelectorAll(".price");
prices.textContent = "SOLD OUT"; // no error... and nothing changes!
```

`querySelectorAll` gives you a `NodeList`, not an element, so setting `textContent` on it does nothing useful. JavaScript doesn't even complain. Loop over the list and change each element:

```
for (const price of prices) {  
  price.textContent = "SOLD OUT";  
}
```

4. Creating an element but never adding it

```
const li = document.createElement("li");  
li.textContent = "Brownie";  
// ...and the brownie never shows up
```

A new element stays in the delivery van until you put it on the page. Don't forget `menu.append(li);`.

5. Wiping out classes with `className`

```
const croissant = document.querySelector(".sale");  
croissant.className = "highlight"; // "item" and "sale" are gone!
```

`className` holds the whole class list as one string, so setting it replaces every class the element had. Use `classList.add("highlight")` to add one class and keep the rest.

Quick recap

- HTML is made of elements with attributes. `id` names one element; `class` groups many. The browser turns your HTML into the DOM: a family tree of objects that JavaScript can change.
- Connect your script with `<script src="script.js" defer></script>` in the `<head>`. Open `index.html` by double-clicking it, use `F12` to see the Console, and refresh after every change.
- Find elements with `getElementById`, `querySelector`, and `querySelectorAll` (a `NodeList` you can loop over). If nothing matches, you get `null`.
- Change text with `textContent`. Keep `innerHTML` away from anything a user typed.
- Prefer `classList.add/remove/toggle/contains` over `style`, and use attributes like `src`, `href`, `disabled`, and `data-*` for extra information.
- Create elements with `createElement`, add them with `append` or `prepend`, and take them away with `remove`. Rendering an array is a loop that does exactly that.

Next: try the [exercises](#), then move on to [21 Events](#).

Exercises

How to do these:

- Make a folder for each exercise inside this chapter's folder: `ex1` , `ex2` , and so on.
- In each folder, create `index.html` (copy in the HTML from the exercise) and `script.js` (your code goes here). Don't change the HTML unless the exercise asks you to.
- Open `index.html` by double-clicking it. Press `F12` and click **Console** to see your messages and any errors.
- After every change: save, then refresh the page (`F5`).
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Cafe menu board

A cafe shows its menu on a screen by the counter. Use this HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Menu Board</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1 id="cafe-name">Cafe</h1>
    <p>Today's special: <span id="special">coming soon</span></p>
    <ul id="drinks">
      <li class="drink">Espresso</li>
      <li class="drink">Cappuccino</li>
      <li class="drink">Chai latte</li>
      <li class="drink">Hot chocolate</li>
    </ul>
    <p id="drink-count"></p>
  </body>
</html>
```

In `script.js`:

1. Change the heading to `Corner Cafe` .
2. Change the special to `Pumpkin spice latte` .
3. Count the drinks and show the count in the last paragraph.
4. Print each drink in the Console, with its number.

The page should show:

```
Corner Cafe
Today's special: Pumpkin spice latte
• Espresso
• Cappuccino
• Chai latte
• Hot chocolate
We serve 4 drinks.
```

And the Console should show:

```
1. Espresso
2. Cappuccino
3. Chai latte
4. Hot chocolate
```

Rule: don't type the number 4. If the cafe adds a fifth drink to the HTML, your code should say 5 without any changes.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Weather card

A weather app shows a card that changes color with the weather. Use this HTML. The CSS is already written for you:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Weather</title>
    <style>
      .card { padding: 16px; border-radius: 8px; width: 260px; font-family: sans-serif; }
      .sunny { background-color: #ffe98a; }
      .rainy { background-color: #b9d4f0; }
      .snowy { background-color: #eef3f8; }
      .alert { color: darkred; font-weight: bold; }
      .hidden { display: none; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <div id="card" class="card">
      <h2 id="city">City</h2>
      <p id="temperature">--</p>
      <p id="alert" class="alert hidden">No alerts</p>
    </div>
  </body>
</html>

```

Start `script.js` with this data:

```

const weather = {
  city: "Pokhara",
  condition: "rainy",
  temperature: 18,
  alert: "Heavy rain expected after 6 PM",
};

```

1. Show the city, and the temperature as `18°C` (copy the `°` symbol from here).
2. Give the card the class that matches the weather's `condition`, so it turns blue.
3. If there's an alert, put its text in the alert paragraph and make it visible.
4. In the Console, show whether the card has the classes `sunny` and `rainy`.

The page should show a light blue card with:

```

Pokhara
18°C
Heavy rain expected after 6 PM

```

And the Console should show:

```
Sunny? false
Rainy? true
```

Then test it: change `condition` to `"sunny"` and `alert` to `""`, and refresh. The card should turn yellow, the alert should stay hidden, and the Console should say `Sunny? true` and `Rainy? false`.

Rule: don't use `style` in your JavaScript. Only change classes.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Road trip playlist

Build a playlist page from an array. The list starts empty, and your code fills it in:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Road Trip Playlist</title>
    <style>
      body { font-family: sans-serif; }
      .long { color: purple; font-style: italic; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Road Trip Playlist</h1>
    <ol id="playlist"></ol>
    <p id="summary"></p>
  </body>
</html>
```

(`<ol>` is a numbered list, so the browser adds the numbers for you.) Start `script.js` with this data:

```
const songs = [
  { title: "Bohemian Rhapsody", artist: "Queen", seconds: 354 },
  { title: "Hotel California", artist: "Eagles", seconds: 391 },
  { title: "Here Comes the Sun", artist: "The Beatles", seconds: 185 },
  { title: "Africa", artist: "Toto", seconds: 295 },
  { title: "Stairway to Heaven", artist: "Led Zeppelin", seconds: 482 },
];
```

1. Write a function `formatTime(totalSeconds)` that turns `354` into `"5:54"` and `185` into `"3:05"`.

2. Add one list item per song, like `Bohemian Rhapsody - Queen (5:54)`.
3. Songs longer than 6 minutes get the class `long`.
4. Fill in the summary with the number of songs and the total playing time.

The page should show:

```
Road Trip Playlist
1. Bohemian Rhapsody - Queen (5:54)
2. Hotel California - Eagles (6:31)
3. Here Comes the Sun - The Beatles (3:05)
4. Africa - Toto (4:55)
5. Stairway to Heaven - Led Zeppelin (8:02)
5 songs, 28:27 total
```

"Hotel California" and "Stairway to Heaven" should be purple and in italics.

Rule: use `createElement` and `append`, not `innerHTML`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Book of the week

A library's website shows one featured book. Use this HTML:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Library</title>
    <style>
      body { font-family: sans-serif; }
      .book { border: 1px solid #ccc; border-radius: 8px; padding: 16px; width: 240px; }
      .book img { width: 120px; }
      .unavailable { opacity: 0.5; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Book of the week</h1>
    <div id="book" class="book">
      <img id="cover" src="" alt="">
      <h2 id="title">Title</h2>
      <p id="author">Author</p>
      <a id="details-link" href="#">More details</a>
      <p><button id="borrow-button">Borrow</button></p>
    </div>
  </body>
</html>

```

Start `script.js` with this data:

```

const book = {
  isbn: "9780141439518",
  title: "Pride and Prejudice",
  author: "Jane Austen",
  copiesLeft: 0,
};

```

1. Fill in the title, and the author as `by Jane Austen`.
2. Show the cover. Its address is `https://covers.openlibrary.org/b/isbn/` followed by the ISBN and `-M.jpg`. Give it the alt text `Cover of Pride and Prejudice`.
3. Point the link at the book's Wikipedia page: `https://en.wikipedia.org/wiki/Pride_and_Prejudice`. Build it from the title, with the spaces turned into underscores.
4. Store the ISBN on the card as a `data-isbn` attribute.
5. When no copies are left: disable the Borrow button, change its text to `Waiting list only`, and add the class `unavailable` to the card (it fades out).
6. Print these checks in the Console:


```
Link goes to: https://en.wikipedia.org/wiki/Pride_and_Prejudice
Card ISBN: 9780141439518
Borrow button disabled? true
```

The page should show the book's cover (you need to be online for it to load), the title, `by Jane Austen`, a "More details" link, and a greyed-out `Waiting list only` button. Click the link to check it goes to the right page.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: change `copiesLeft` to `2` and refresh. The button should say `Borrow`, and neither the button nor the card should be greyed out.

Exercise 5 (Challenge): Library loans dashboard

Build a "My loans" page that shows which borrowed books are overdue. This one uses dates from [chapter 19](#):

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My Loans</title>
    <style>
      body { font-family: sans-serif; }
      .overdue { color: darkred; font-weight: bold; }
      .due-soon { color: darkorange; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>My library loans</h1>
    <p id="summary"></p>
    <ul id="loans"></ul>
  </body>
</html>
```

Start `script.js` with this data:

```
const today = new Date(2026, 8, 24); // pretend today is September 24th, 2026

const loans = [
  { title: "The Hobbit", due: new Date(2026, 8, 30) },
  { title: "Atomic Habits", due: new Date(2026, 8, 20) },
  { title: "Dune", due: new Date(2026, 8, 26) },
  { title: "The Alchemist", due: new Date(2026, 9, 8) },
  { title: "Sapiens", due: new Date(2026, 8, 23) },
];
```

Write a function `renderLoans(items)` that fills in the page:

1. Show the loans sorted by due date, earliest first, without changing the original `loans` array.
2. Each list item shows the title, the due date, and a label in brackets:
 - past the due date: (4 days overdue), and the class `overdue`,
 - due within the next 3 days: (due in 2 days), and the class `due-soon`,
 - otherwise: (6 days left).
3. Use 1 day, not 1 days.
4. The summary says how many books are overdue.

The page should show:

```
My library loans
2 of 5 books are overdue.
• Atomic Habits - due Sun, Sep 20 (4 days overdue)
• Sapiens - due Wed, Sep 23 (1 day overdue)
• Dune - due Sat, Sep 26 (due in 2 days)
• The Hobbit - due Wed, Sep 30 (6 days left)
• The Alchemist - due Thu, Oct 8 (14 days left)
```

The two overdue books should be dark red and bold, and "Dune" should be orange.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Everything on these pages happens once, the moment the page loads. The Borrow button doesn't do anything when you click it, and the playlist can't be sorted by a click.

[Chapter 21: Events](#) makes your pages react to clicks, typing, and key presses.

21 Events

What is it?

An **event** is something that happens on the page: a click, a key press, typing in a box, the mouse moving over a picture.

You can tell JavaScript: "when *this* happens, run *that* function." That's how a web page reacts to the people using it.

Why does it matter?

In [chapter 20](#), your scripts ran once, when the page loaded, and then stopped. Nothing you clicked did anything.

Real apps wait for you. A Like button counts likes. A search box filters results as you type. Pressing `Escape` closes a pop-up. "Add to cart" adds an item and updates the total. All of those are events.

From here on, almost everything interactive you build starts with an event.

Real-world example

Think of a doorbell:

The doorbell	Events in JavaScript
You install a doorbell and decide ahead of time: "when it rings, I'll open the door"	<code>button.addEventListener("click", openDoor)</code>
A visitor presses the bell	The user clicks the button: a <code>click</code> event
You go and open the door	Your function runs
You don't stand by the door all day, waiting	Your code doesn't wait in a loop. The browser calls your function when it happens
The visitor says who they are: "Parcel for you!"	The event object : details about what happened
You unplug the doorbell	<code>removeEventListener</code>

The key idea: you set it all up *ahead of time*. Then the browser runs your function whenever the event happens: maybe once, maybe fifty times, maybe never.

How it works

For every example in this chapter, use a practice page like the one from chapter 20: a folder with `index.html` and `script.js`, linked with `<script src="script.js" defer></script>`. Put each example's HTML in the `<body>`, its code in `script.js`, then save and refresh. A few examples hide things with the `hidden` class from chapter 20, so add `<style>.hidden { display: none; }</style>` to the `<head>` too.

Your first event listener

Here's a Like button:

```
<button id="like-button">Like</button>
<p id="like-count">Likes: 0</p>
```

```
const likeButton = document.getElementById("like-button");
const likeCount = document.getElementById("like-count");
let likes = 0;

likeButton.addEventListener("click", () => {
  likes++;
  likeCount.textContent = `Likes: ${likes}`;
});
```

Click the button three times, and the page shows `Likes: 3`.

`addEventListener` takes two things:

1. The **type** of event, as a string: `"click"`.
2. A function to run when it happens.

That function is called an **event listener** (or **event handler**): it listens for the event, then handles it. It's a callback, like the ones you passed to `map` and `filter` in [chapter 13](#). You hand it over, and the browser calls it later.

For longer handlers, a named function keeps things tidy. Here's the same button with a named function instead of the arrow function. Notice there are no brackets after `addLike`: you're handing the function over, not calling it yet.

```
function addLike() {
  likes++;
  likeCount.textContent = `Likes: ${likes}`;
}

likeButton.addEventListener("click", addLike);
```

Tip: You can "click" from the Console too. Type `document.getElementById("like-button").click()` and press Enter. It's a quick way to test a handler.

Events you'll use most

Event	Fires when...	Typical use
click	an element is clicked (or tapped)	buttons, links, cards
input	the text in a text box changes, on every keystroke	live search, character counters
change	the user finishes changing a value, like picking from a dropdown	dropdowns, checkboxes
keydown	a key is pressed down	keyboard shortcuts, games
mouseover / mouseout	the mouse pointer moves onto / off an element	hover tips, previews
submit	a form is sent	forms (the whole of chapter 22)

The event object

When an event happens, the browser hands your function an **event object**: a bundle of details about what just happened. Add a parameter to catch it. It's usually called `event`. Try it on the Like button:

```
likeButton.addEventListener("click", (event) => {
  console.log(event.type); // prints: click
  console.log(event.target); // shows: <button id="like-button">Like</button>
});
```

Property	What it tells you
<code>event.type</code>	Which event it was: "click", "keydown", and so on
<code>event.target</code>	The element where the event happened
<code>event.key</code>	Which key was pressed (keyboard events only)
<code>event.preventDefault()</code>	Not a detail, but a method: it stops the browser's normal action (coming up soon)

Typing: `input` and `change`

Here's a name badge that updates as you type. `<input>` makes a text box, and its `value` property is the text currently inside it. (You'll learn much more about inputs in chapter 22.)

```
<input id="name-input">
<p id="badge">Hello, my name is ...</p>
```

```
const nameInput = document.getElementById("name-input");
const badge = document.getElementById("badge");

nameInput.addEventListener("input", (event) => {
  badge.textContent = `Hello, my name is ${event.target.value}`;
});
```

Type `Sam`, and the badge changes with every letter: `Hello, my name is S`, then `Sa`, then `Sam`.

The `change` event is calmer. In a text box, it waits until you've finished and click somewhere else. It's most useful for dropdowns (`<select>`), where it fires as soon as you pick an option:

```
<select id="size">
  <option value="small">Small pizza</option>
  <option value="large">Large pizza</option>
</select>
```

```
const sizeSelect = document.getElementById("size");

sizeSelect.addEventListener("change", (event) => {
  console.log(`You picked: ${event.target.value}`);
});
```

Pick "Large pizza", and the Console prints `You picked: large`. That's the option's `value`, not the text you see.

The keyboard: `keydown` and `event.key`

To catch key presses anywhere on the page, listen on `document` itself:

```
document.addEventListener("keydown", (event) => {
  console.log(`You pressed: ${event.key}`);
});
```

Press a few keys and watch the Console. `event.key` tells you which one:

You press	<code>event.key</code> is
The A key	"a" (or "A" with Shift)
Enter	"Enter"
Escape	"Escape"
The arrow keys	"ArrowUp", "ArrowDown", "ArrowLeft", "ArrowRight"
The space bar	" " (a single space)

A classic use is closing something with `Escape` :

```
<p id="banner">Free delivery this weekend! Press Escape to close.</p>
```

```
const banner = document.getElementById("banner");

document.addEventListener("keydown", (event) => {
  if (event.key === "Escape") {
    banner.classList.add("hidden");
  }
});
```

The mouse: `mouseover` and `mouseout`

These fire when the mouse pointer moves onto an element and off it again. Here's a little tip that appears while you hover over a product:

```
<p id="product">Mystery box: $10</p>
<p id="tip" class="hidden">What's inside? Nobody knows!</p>
```

```
const product = document.getElementById("product");
const tip = document.getElementById("tip");

product.addEventListener("mouseover", () => {
  tip.classList.remove("hidden");
});
product.addEventListener("mouseout", () => {
  tip.classList.add("hidden");
});
```

For purely visual hover effects, like changing a color, CSS can do the job alone (with `:hover`). Reach for mouse events when JavaScript needs to *do* something.

Stopping the browser's normal action: `preventDefault()`

Some events come with a built-in action. Clicking a link opens a new page. Sending a form reloads the page. Pressing the arrow keys scrolls the page. Calling `event.preventDefault()` tells the browser: "don't do your usual thing, I'll handle this one."

```
<a id="terms-link" href="terms.html">Show the terms</a>
<p id="terms" class="hidden">No refunds on cakes once they've been cut.</p>
```

```
const termsLink = document.getElementById("terms-link");
const terms = document.getElementById("terms");

termsLink.addEventListener("click", (event) => {
  event.preventDefault(); // don't go to terms.html
  terms.classList.toggle("hidden"); // show the terms right here instead
});
```

Clicking the link now shows and hides the terms, and the page stays put.

You'll use `preventDefault` most with forms. Without it, sending a form reloads the page and wipes out everything. Chapter 22 starts every form like this:

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the page from reloading
  // ...then deal with the form yourself
});
```


DOMContentLoaded

In older code, you'll often see scripts wrapped like this:

```
document.addEventListener("DOMContentLoaded", () => {  
  // everything here runs once the whole HTML has been read  
});
```

`DOMContentLoaded` is an event that fires when the browser has finished reading the page. It solves the same problem as `defer`: making sure the elements exist before your code looks for them. Your scripts already use `defer`, so you don't need it.

Removing a listener

Sometimes a listener should stop listening. Say a coffee shop's website gives each visitor one free coffee:

```
<button id="gift-button">Claim your free coffee</button>  
<p id="gift-message"></p>
```

```
const giftButton = document.getElementById("gift-button");  
const giftMessage = document.getElementById("gift-message");  
  
function claimGift() {  
  giftMessage.textContent = "Gift claimed! Enjoy your free coffee.";  
  giftButton.removeEventListener("click", claimGift);  
}  
  
giftButton.addEventListener("click", claimGift);
```

After the first click, the button stops listening. Click it again and nothing happens.

`removeEventListener` needs the **exact same function** you added. That's why this example uses a named function. Two arrow functions that look identical are still two different functions (like the two different objects in [chapter 16](#)), so this doesn't work:

```
giftButton.addEventListener("click", () => console.log("Hi"));  
giftButton.removeEventListener("click", () => console.log("Hi")); // removes nothing!
```

Tip: For "only once", there's also a shortcut: `giftButton.addEventListener("click", claimGift, { once: true });` removes the listener for you after the first time.

Bubbling: events travel up the tree

Here's something surprising. When you click an element, its parents hear the click too:

```
<ul id="cart">
  <li id="cart-item">
    Headphones <button id="remove-button">Remove</button>
  </li>
</ul>
```

```
document.getElementById("remove-button").addEventListener("click", () => {
  console.log("The button heard the click");
});
document.getElementById("cart-item").addEventListener("click", () => {
  console.log("The list item heard the click");
});
document.getElementById("cart").addEventListener("click", () => {
  console.log("The list heard the click");
});
```

Click the Remove button, and you'll see:

```
The button heard the click
The list item heard the click
The list heard the click
```

Click the word "Headphones" instead, and only the last two lines appear, because the click started on the list item.

This is called **bubbling**. An event starts on the element where it happened, then rises up through each parent in the family tree, all the way to `document`, like a bubble rising through water. Every listener along the way gets a turn.

So inside a parent's listener, there are two different elements to know about:

- `event.target`: where the event *started* (the exact thing that was clicked).
- `event.currentTarget`: the element whose listener is running *right now*.

```
document.getElementById("cart").addEventListener("click", (event) => {
  console.log(event.target.tagName, event.currentTarget.tagName); // tagName is the element's
    tag
});
// Clicking the Remove button prints: BUTTON UL
```

If you ever need to stop an event from bubbling any further, call `event.stopPropagation()`:

```
document.getElementById("remove-button").addEventListener("click", (event) => {
  event.stopPropagation(); // the list item and the list never hear about it
  console.log("The button heard the click");
});
```

Use it sparingly. Other code may be counting on hearing that event, like a menu that closes when you click anywhere else on the page. Stopping the bubble quietly breaks it.

Event delegation: one listener for many elements

Bubbling might sound like a nuisance, but it makes one of the most useful tricks in front-end programming possible.

Picture a shopping list with a Remove button on every item. You could add a listener to each button. But what about items added later? Their buttons don't exist yet when your script runs, so they'd get no listener at all.

Instead, put **one** listener on the parent, and let the clicks bubble up to it. This is called **event delegation**. It's like a receptionist in an office building: instead of a guard at every office door, one person at the front desk greets every visitor and sends them the right way. When a new office opens, the receptionist handles its visitors too.

```
<ul id="shopping-list">
  <li>Milk <button class="remove">Remove</button></li>
  <li>Eggs <button class="remove">Remove</button></li>
</ul>
<button id="add-button">Add bread</button>
```

```
const list = document.getElementById("shopping-list");

list.addEventListener("click", (event) => {
  if (event.target.classList.contains("remove")) {
    event.target.closest("li").remove();
  }
});
```

Step by step:

1. Any click inside the list bubbles up to the list's listener.
2. `event.target.classList.contains("remove")` checks whether the click started on a Remove button. Clicks on the item's text are ignored.
3. `closest("li")` starts at the clicked button and walks *up* the family tree until it finds an `li`. That's the item to remove.

Now add items later, and their Remove buttons work straight away, with no extra listeners:

```
document.getElementById("add-button").addEventListener("click", () => {
  const button = document.createElement("button");
  button.textContent = "Remove";
  button.classList.add("remove");

  const li = document.createElement("li");
  li.append("Bread ", button); // append can take text and elements
  list.append(li);
});
```

Delegation shines with lists that change: to-do lists, shopping carts, comment sections. You'll use it in the [chapter 24](#) project. Combined with `data-` attributes from chapter 20, the listener can also tell *which* item was clicked: `event.target.closest("li").dataset.id`.

The old way: `onclick`

In older code and tutorials, you'll see events set up right inside the HTML:

```
<button onclick="addLike()">Like</button>
```

Avoid this. It mixes your JavaScript into your HTML, the function has to be global to be found, and many websites block it for security reasons ([chapter 51](#)).

You'll also see the `onclick` *property*. Its problem is that an element can only have one:

```
likeButton.onclick = () => console.log("first");
likeButton.onclick = () => console.log("second"); // replaces the first!
// Clicking prints only: second
```

`addEventListener` doesn't have that problem: add two listeners, and both run. Stick with `addEventListener`.

Coming up: some events fire *a lot*. `input` fires on every keystroke, and scrolling fires dozens of times a second. In [chapter 34](#), you'll learn to calm them down with `debounce` and `throttle`. You may also see `this` used inside event handlers in other people's code. That's explained in [chapter 26](#). For now, stick with `event.target` and `event.currentTarget`.

Common mistakes

1. Calling the function instead of handing it over

```
likeButton.addEventListener("click", addLike()); // the brackets run it right now!
```

With brackets, `addLike()` runs once, the moment the page loads, so the page says `Likes: 1` before anyone clicks. It returns `undefined`, so that's what gets set up as the listener, and clicking does nothing. There's no error to warn you. Hand over the function itself, without brackets: `addEventListener("click", addLike)`.

If you need to pass something to the function, wrap the call in an arrow function: `addEventListener("click", () => addToCart(3))`.

2. Misspelling the event name

```
likeButton.addEventListener("onclick", addLike); // never fires
likeButton.addEventListener("Click", addLike);    // never fires either
```

Event names are all lowercase, with no `on` in front: `"click"`, `"keydown"`, `"mouseover"`. A misspelled name doesn't cause an error. The listener just waits forever for an event that never comes.

3. Adding listeners to elements that don't exist yet

```
const removeButtons = document.querySelectorAll(".remove");
for (const button of removeButtons) {
  button.addEventListener("click", removeItem);
}
// Items added later have Remove buttons that do nothing
```

`querySelectorAll` only finds the buttons that exist *right now*. Buttons created later never get a listener. Use event delegation instead: one listener on the parent list.

4. Trusting `event.target` when a button has something inside it

```
<button class="remove"><b>X</b> Remove</button>
```

```
list.addEventListener("click", (event) => {
  if (event.target.classList.contains("remove")) { // false when you click the X!
    event.target.closest("li").remove();
  }
});
```

Click the bold `x`, and `event.target` is the `<b>` element, not the button, so the check fails. Use `closest` to find the button, whichever part of it was clicked:

```
list.addEventListener("click", (event) => {
  const button = event.target.closest(".remove");
  if (button) {
    button.closest("li").remove();
  }
});
```

If the click wasn't on (or inside) a Remove button, `closest` gives `null`, and nothing happens.

5. Listening for keys on the wrong element

```
playButton.addEventListener("keydown", movePlayer); // only works while the button is selected
```

Key presses go to whichever element has **focus**: the one that's currently active, like the text box you're typing in. Usually that's the page itself, so a listener on a button rarely hears anything. For keyboard shortcuts and games, listen on `document`.

Quick recap

- An event is something that happens on the page. `element.addEventListener("click", handler)` sets up a function to run every time it happens.
- The events you'll use most are `click`, `input`, `change`, `keydown`, `mouseover` / `mouseout`, and `submit`.
- Your handler receives an event object: `event.target` is where it happened, `event.key` is the key pressed, and `event.preventDefault()` stops the browser's normal action.
- `removeEventListener` needs the exact same function you added. `{ once: true }` is a shortcut for one-time listeners.
- Events bubble up from where they happened through every parent. `stopPropagation()` stops that, but use it sparingly.
- Event delegation puts one listener on a parent to handle all its children, even ones added later. `closest()` finds the element you care about.
- Use `addEventListener`, not `onclick` in your HTML.

Next: try the [exercises](#), then move on to [22 Forms](#).

Exercises

How to do these:

- Make a folder for each exercise inside this chapter's folder: `ex1`, `ex2`, and so on.

- In each folder, create `index.html` (copy in the HTML from the exercise) and `script.js` (your code goes here). Don't change the HTML unless the exercise asks you to.
- Open `index.html` by double-clicking it. Press `F12` and click **Console** to see your messages and any errors.
- After every change: save, then refresh the page (`F5`). Then click, type, and press keys to test it.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Cup counter

A cafe wants a simple tally of how many cups it sells today. Use this HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Cup Counter</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Corner Cafe</h1>
    <p id="count">Cups sold today: 0</p>
    <button id="add-button">+1 cup</button>
    <button id="undo-button">Undo</button>
    <button id="reset-button">Reset</button>
  </body>
</html>
```

Make the buttons work:

- **+1 cup** adds one to the count.
- **Undo** takes one away, but the count never goes below 0.
- **Reset** sets the count back to 0.

Test it like this:

You do	The page shows
Click +1 cup five times	Cups sold today: 5
Click Undo twice	Cups sold today: 3
Click Reset	Cups sold today: 0

You do	The page shows
Click Undo again	still Cups sold today: 0

Rule: write one function, `updateDisplay()`, that puts the current count on the page. Every button's handler changes the count, then calls it.

Stuck? The hint is in the [Hints section](#) at the end.

Bonus: disable the Undo and Reset buttons whenever the count is 0 ([chapter 20](#) showed you `disabled`).

Exercise 2 (Easy): Bio character counter

A social app limits profile bios to 60 characters. Show a live counter as the user types. Use this HTML (a `<textarea>` is a bigger text box with several lines, and its `value` works just like an `<input>`'s):

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Edit Profile</title>
    <style>
      body { font-family: sans-serif; }
      textarea { width: 300px; height: 80px; }
      .too-long { color: red; font-weight: bold; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Edit your profile</h1>
    <p>Bio:</p>
    <textarea id="bio"></textarea>
    <p id="counter">0 / 60</p>
  </body>
</html>
```

1. Every time the text changes, show how many characters there are, like `25 / 60`.
2. Over 60, add the class `too-long` (red, bold) and say how many are too many, like `66 / 60 (6 too many)`.
3. Back under 60, the counter goes back to normal.

Test it:

You type	The counter shows
I love hiking and coffee.	25 / 60
I love hiking, coffee, old maps, and very long walks on the beach.	66 / 60 (6 too many) in red
Delete everything	0 / 60

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Road trip game

Drive a car along a road with the arrow keys. Use this HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Road Trip</title>
    <style>
      body { font-family: monospace; font-size: 32px; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <p id="road"></p>
    <p id="status"></p>
  </body>
</html>
```

Start `script.js` with:

```
const roadLength = 10;
let position = 0;
```

1. Write a `render()` function that draws the road: 10 spots in a row, with the car 🚗 at `position` and `_` everywhere else. Under it, show `Position: 0`. Call `render()` once when the page loads.
2. `ArrowRight` moves the car one spot right, and `ArrowLeft` one spot left. The car can't drive off either end.
3. On the last spot, the status says `You made it! Press Escape to drive again.` instead of the position.
4. `Escape` puts the car back at the start.
5. Arrow keys normally scroll the page. Stop them from doing that.

Test it:

You do	The page shows
Load the page	 _____ and Position: 0
Press ArrowRight 3 times	____  _____ and Position: 3
Press ArrowLeft 5 times	 _____ and Position: 0 (it stops at the edge)
Hold ArrowRight down	_____  and You made it! Press Escape to drive again.
Press Escape	 _____ and Position: 0

(Holding a key down makes `keydown` fire again and again, so the car races to the end.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Shopping list

A shopping list where you can add items, cross them off, and remove them. Use this HTML:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Shopping List</title>
    <style>
      body { font-family: sans-serif; }
      .bought { text-decoration: line-through; color: gray; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Shopping list</h1>
    <input id="new-item" placeholder="Add an item">
    <button id="add-button">Add</button>
    <ul id="list">
      <li>Milk <button class="remove">Remove</button></li>
      <li>Eggs <button class="remove">Remove</button></li>
      <li>Bread <button class="remove">Remove</button></li>
    </ul>
    <p id="count">3 items</p>
  </body>
</html>
```

1. **Add:** clicking Add creates a new item from the text in the box, with its own Remove button, at the end of the list. Trim the spaces off the text. If the box is empty, do nothing. Afterwards, empty the box.
2. **Remove:** clicking an item's Remove button removes that item, including items you added yourself.
3. **Cross off:** clicking anywhere else on an item crosses it off with the `bought` class. Clicking it again uncrosses it.
4. **Count:** keep the count under the list up to date: `3 items`, `2 items`, `1 item`.

Test it:

You do	The list shows
Click the word "Eggs"	Eggs is crossed out
Click Milk's Remove button	Eggs, Bread, and <code>2 items</code>
Type <code>Apples</code> and click Add	Eggs, Bread, Apples, and <code>3 items</code> . The box is empty again
Click Add with an empty box	nothing changes
Click Apples' Remove button	Eggs, Bread, and <code>2 items</code>

Rule: use event delegation. The list gets exactly **one** click listener, and the Remove buttons get none of their own.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: make the `Enter` key add an item too, while you're typing in the box.

Exercise 5 (Challenge): Cinema seat picker

Build the seat picker for a cinema's booking page. Use this HTML:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Pick Your Seats</title>
    <style>
      body { font-family: sans-serif; }
      .seat { width: 44px; height: 36px; margin: 3px; }
      .taken { background-color: #999; color: white; }
      .selected { background-color: #2e8b57; color: white; }
      #message { color: darkred; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Screen 1: The Space Race</h1>
    <p>(Screen this way)</p>
    <div id="seats"></div>
    <p id="summary">No seats selected</p>
    <p id="message"></p>
    <button id="clear-button">Clear selection</button>
  </body>
</html>

```

Start `script.js` with this data:

```

const rows = ["A", "B", "C"];
const seatsPerRow = 6;
const takenSeats = ["A3", "A4", "B1", "C5", "C6"];
const pricePerSeat = 8.5;
const maxSeats = 4;
let selectedSeats = [];

```

1. **Draw the seats.** Each row is a `div` with six buttons, labeled `A1` to `A6` and so on. Every button gets the class `seat` and a `data-seat` attribute with its label. Taken seats also get the class `taken` and are disabled.
2. **Pick seats.** Clicking a free seat selects it (class `selected`). Clicking a selected seat unselects it.
3. **A limit.** Nobody can pick more than 4 seats. Trying to pick a fifth shows `You can pick up to 4 seats.` under the summary. The next successful click clears that message.
4. **Summary.** Show the selected seats in seat order, with the total price, like `Selected: A1, B2 (total $17.00)`. With nothing selected, show `No seats selected`.
5. **Clear.** The Clear button and the `Escape` key both unselect everything.

Test it:

You do	The summary shows
Click B2, then A1	Selected: A1, B2 (total \$17.00)
Click A3 (a grey, taken seat)	nothing changes
Click A2, then B3	Selected: A1, A2, B2, B3 (total \$34.00)
Click C1	the same, plus the message You can pick up to 4 seats.
Click A1 again	Selected: A2, B2, B3 (total \$25.50) , and the message is gone
Press <code>Escape</code>	No seats selected , and no seat is green

Rule: the whole seat area gets exactly one click listener. Keep `selectedSeats` as the single source of truth: change the array, then redraw the page from it.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercises 2 and 4, you read text straight out of boxes on the page. Real websites are full of boxes like that: sign-up forms, checkout pages, feedback forms. They need checking, too. What if someone types "abc" as their age?

[Chapter 22: Forms](#) shows you how to read every kind of input, check it, and show helpful error messages.

22 Forms

What is it?

A **form** is the part of a web page where people type or choose things and send them off: a sign-up page, a search box, a checkout page, a "Leave a review" box.

In this chapter, you'll use JavaScript to read what people entered, check it, and react to it, all without reloading the page.

Why does it matter?

Almost every app needs input from its users. Logging in, booking a table, ordering a pizza, leaving feedback: all of these are forms.

But people are people. They leave boxes empty, make typos in their email, and type "twenty" where you wanted 20. If your code trusts every answer, bad data sneaks into your app.

With what you'll learn here, you can:

- read every kind of field: text, numbers, dropdowns, tick boxes and more,
- stop the page from reloading when the form is sent,
- catch mistakes and tell the user exactly how to fix them.

Real-world example

Think of the paper sign-up form at a gym's front desk:

At the gym	On a web page
The paper form	<code><form></code>
The blank boxes you write in	<code><input></code> , <code><textarea></code> , <code><select></code>
The words printed next to each box ("Full name")	<code><label></code>
The little star (*) that means "you must fill this in"	the <code>required</code> attribute
Tick boxes, and "choose one" circles	checkboxes and radio buttons
Handing the form to the receptionist	the <code>submit</code> event
The receptionist checking it: "You forgot your phone number"	validation

At the gym	On a web page
A sticky note next to the box you got wrong	an error message next to the field
A fresh, blank form for the next person	<code>form.reset()</code>

Your JavaScript is the receptionist. It reads the form, checks it, and either accepts it or hands it back with helpful notes.

A good receptionist points at the exact box that needs fixing, instead of just saying "Wrong!" You'll learn to do the same.

How it works

The building blocks of a form

Forms are made of HTML elements, just like the ones you met in [chapter 20](#). Here's a small form for a cinema's newsletter:

```
<form id="signup-form">
  <label for="name">Your name</label>
  <input id="name" name="name" type="text">

  <label for="email">Email</label>
  <input id="email" name="email" type="email">

  <button type="submit">Sign up</button>
</form>
```

- `<form>` wraps all the fields that belong together.
- `<input>` is a box to type in. Its `type` decides what kind of box it is.
- `<label>` is the text that goes with a field. Its `for` matches the field's `id`. You'll see why that matters at the end of this chapter.
- `name` is the field's name when the form is sent. You'll need it for `FormData` later on.
- `<button type="submit">` is the button that sends the form.

Here are the fields you'll use most:

Element	What it's for	Example
<code><input type="text"></code>	A short line of text	a name, a city
<code><input type="email"></code>	An email address	<code>sam@example.com</code>

Element	What it's for	Example
<code><input type="number"></code>	A number (with small up and down arrows)	an age, a quantity
<code><input type="password"></code>	Text that shows as dots	a password
<code><textarea></code>	Several lines of text	a review, a message
<code><select></code> with <code><option></code> s	A dropdown: pick one	a country, a plan
<code><input type="checkbox"></code>	A tick box: on or off	"Send me offers"
<code><input type="radio"></code>	Pick exactly one from a group	small, medium, large

Tip: `<input>` has no closing tag. It's a single tag, like `<img>`.

Reading what the user typed: `.value`

Every text box, `<textarea>` and `<select>` has a `.value` property. It holds whatever is in the field right now.

```
<input id="city" type="text">
<button id="check-button">Check weather</button>
```

```
const cityInput = document.querySelector("#city");
const checkButton = document.querySelector("#check-button");

checkButton.addEventListener("click", () => {
  const city = cityInput.value.trim();
  console.log(`Looking up the weather in ${city}...`);
});
```

Type `Lisbon` (with a few spaces around it) and click the button:

```
Looking up the weather in Lisbon...
```

`trim()` from [chapter 06](#) removes the spaces people type by accident. Use it on almost every text field.

Notice *where* the code reads `.value`: inside the click handler. The value changes as the user types, so read it at the moment you need it.

For a `<select>`, `.value` is the `value` of the chosen `<option>`:


```
<select id="plan">
  <option value="monthly">Monthly: $30</option>
  <option value="yearly">Yearly: $300</option>
</select>
```

```
const planSelect = document.querySelector("#plan");
console.log(planSelect.value); // prints: monthly
```

The first option is chosen until the user picks another one. The user sees "Monthly: \$30", but your code gets "monthly" . That's handy: the text on screen can say anything, while the value stays short and easy to check.

Form values are always strings

This one catches everybody. Even in a `type="number"` box, `.value` is a **string**.

Here's a page that adds a tip to a restaurant bill:

```
<input id="bill" type="number">
<input id="tip" type="number">
<button id="total-button">Total</button>
<p id="total"></p>
```

```
const billInput = document.querySelector("#bill");
const tipInput = document.querySelector("#tip");
const totalText = document.querySelector("#total");

document.querySelector("#total-button").addEventListener("click", () => {
  console.log(typeof billInput.value); // prints: string
  const total = billInput.value + tipInput.value;
  totalText.textContent = `Total: ${total}`;
});
```

Type 40 for the bill and 6 for the tip, and the page says Total: \$406 . Ouch. The + joined two strings, just like "5" + 3 in [chapter 03](#).

The fix is to turn each value into a number with `Number()` before doing math. But there's one more trap:

What's in the box	.value	Number(value)
40	"40"	40
9.99	"9.99"	9.99

What's in the box	.value	Number(value)
nothing at all	""	0
forty (in a text box)	"forty"	NaN

An empty box becomes `0`, not `NaN`. So if the user forgets the bill, your code quietly works with a \$0 bill. Check for an empty value *before* converting:

```
document.querySelector("#total-button").addEventListener("click", () => {
  if (billInput.value === "") {
    totalText.textContent = "Please enter the bill.";
    return; // stop here
  }
  const bill = Number(billInput.value);
  const tip = Number(tipInput.value);
  totalText.textContent = `Total: ${bill + tip}`;
});
```

Now `40` and `6` show `Total: $46`, and an empty bill shows the message. An empty *tip* still becomes `0`, and here that's fine: no tip means a \$0 tip.

Tip: In a `type="number"` box, the browser also gives you `""` when what's typed isn't a proper number. So the empty check catches that too.

Checkboxes and radio buttons: `.checked`

A checkbox is either ticked or not. For checkboxes, use `.checked` instead of `.value`: it's `true` or `false`.

```
<input id="gift-wrap" type="checkbox">
<label for="gift-wrap">Gift wrap (+$3)</label>
```

```
const giftWrap = document.querySelector("#gift-wrap");
const price = 25;

giftWrap.addEventListener("change", () => {
  const total = giftWrap.checked ? price + 3 : price;
  console.log(`Total: ${total}`);
});
```

Tick the box, then untick it. You'll see:

Total: \$28
Total: \$25

The `change` event from [chapter 21](#) fires every time the box is ticked or unticked. The ternary from [chapter 07](#) picks the right total.

Radio buttons come in groups, and the user can pick only one from each group. The buttons in a group share the same `name`. That's what ties them together:

```
<fieldset>
  <legend>Size</legend>
  <input id="small" type="radio" name="size" value="small" checked>
  <label for="small">Small</label>
  <input id="medium" type="radio" name="size" value="medium">
  <label for="medium">Medium</label>
  <input id="large" type="radio" name="size" value="large">
  <label for="large">Large</label>
</fieldset>
```

The `checked` attribute makes "Small" the chosen one when the page loads. The `<fieldset>` draws a box around the group, and its `<legend>` is the group's title. You'll see why that helps at the end of the chapter.

To find out which one is chosen, ask for "the ticked one" with a selector:

```
const chosenSize = document.querySelector("input[name='size']:checked");
console.log(chosenSize.value); // prints: small
```

- `input[name='size']` means "an `<input>` whose `name` is `size`". Square brackets pick elements by an attribute.
- `:checked` narrows it down to the one that's ticked right now.

If nothing in the group is chosen, `querySelector` finds nothing and returns `null`. Then `.value` crashes with `TypeError: Cannot read properties of null (reading 'value')`. To be safe, give one option `checked` from the start.

Sending the form: the `submit` event

So far, you've used `click` on buttons. Forms have a better event: `submit`. It fires on the `<form>` whenever the user sends it, by clicking the submit button or by pressing Enter in a text box.

There's a catch, though. When a form is submitted, the browser's normal job is to send the data somewhere and load a new page. Our form has nowhere to send it, so the browser reloads the page. Everything your JavaScript changed is wiped out, and the console is cleared.

You met the fix in chapter 21: `event.preventDefault()` tells the browser "skip your normal job, I've got this".

Here it is with the cinema form from the start of the chapter:

```
const form = document.querySelector("#signup-form");
const nameInput = document.querySelector("#name");

form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the page from reloading
  const name = nameInput.value.trim();
  console.log(`Thanks for signing up, ${name}!`);
});
```

Type `Priya` in the name box and click "Sign up" (or press Enter):

Thanks for signing up, Priya!

Two rules of thumb:

- **Listen for `submit` on the form, not `click` on the button.** `submit` catches every way of sending the form. It also waits until the browser's own checks have passed, as you'll see soon.
- **Every `<button>` inside a form is a submit button unless you say otherwise.** For a button that does something else, like "Show password", write `<button type="button">`.

Reading the whole form at once: `FormData`

One `querySelector` per field gets tiring when a form has ten fields. `FormData` reads the whole form in one go. It collects every field that has a `name`, as pairs of names and values.

Here's a gym's sign-up form:

```

<form id="gym-form">
  <label for="member">Name</label>
  <input id="member" name="member" type="text">

  <label for="plan">Plan</label>
  <select id="plan" name="plan">
    <option value="monthly">Monthly</option>
    <option value="yearly">Yearly</option>
  </select>

  <input id="classes" name="classes" type="checkbox">
  <label for="classes">Add group classes</label>

  <button type="submit">Join</button>
</form>

```

```

const gymForm = document.querySelector("#gym-form");

gymForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const formData = new FormData(gymForm);
  console.log(formData.get("member"));

  const data = Object.fromEntries(formData);
  console.log(data);
});

```

Type `Marco`, choose "Yearly", tick "Add group classes", and click "Join". You'll see:

```

Marco
{ member: 'Marco', plan: 'yearly', classes: 'on' }

```

Here's what happened:

- `new FormData(gymForm)` took a snapshot of every field. It's the same `new` you used for `new Date()` in [chapter 19](#).
- `formData.get("member")` looked up one value by its field's `name`.
- `Object.fromEntries(formData)` turned all the pairs into a plain object, like the ones in [chapter 11](#). It's the opposite of `Object.entries()`.

Chrome's console shows objects a little differently, like `{member: 'Marco', plan: 'yearly', classes: 'on'}`, with a small arrow to open it up. It's the same data.

Tip: Don't log the `FormData` itself. `console.log(formData)` shows `FormData {}`, which looks empty even when it isn't. Log `Object.fromEntries(formData)` instead.

A few things to know about `FormData`:

- **Only fields with a `name` are included.** A field without one is skipped, with no warning.
- **Every value is a string**, as always.
- **A ticked checkbox gives `"on"`** (or its `value`, if you gave it one). **An unticked checkbox is left out completely.** Untick "Add group classes" and you get `{ member: 'Marco', plan: 'yearly' }`. So check it with `data.classes === "on"`.
- **Radio buttons are easy:** `formData.get("size")` gives you the chosen one's value. No `:checked` needed.
- **Several checkboxes with the same name**, like pizza toppings: `formData.getAll("topping")` gives you an array of all the ticked ones. `Object.fromEntries` keeps only the last one, because an object can't have the same key twice.

Built-in validation: let the browser check

Validation means checking that the input follows your rules before you accept it. The browser can do a lot of it for you. You add attributes to your fields:

Attribute	The rule	Example
<code>required</code>	Can't be left empty	<code><input name="name" required></code>
<code>minlength</code> , <code>maxlength</code>	At least / at most this many characters	<code><input name="username" minlength="3" maxlength="15"></code>
<code>type="email"</code>	Must look like an email address	<code><input name="email" type="email"></code>
<code>min</code> , <code>max</code>	The smallest / largest number allowed	<code><input name="age" type="number" min="16" max="99"></code>
<code>pattern</code>	Must match a pattern	<code><input name="zip" pattern="[0-9]{5}"></code>

When the user submits a form that breaks a rule, the browser:

1. stops the submission, so your `submit` handler **doesn't run at all**,
2. jumps to the first field with a problem,
3. shows a small bubble with a message like "Please fill out this field." (the exact words depend on your browser and language).

Here's a running club's sign-up form:

```

<form id="club-form">
  <label for="runner">Name</label>
  <input id="runner" name="runner" type="text" required>

  <label for="age">Age</label>
  <input id="age" name="age" type="number" required min="16" max="99">

  <button type="submit">Join the club</button>
</form>

```

Click "Join the club" with the form empty, and the browser points at the name. Fill in a name and the age 12, and it tells you the age must be 16 or more. Your `submit` handler only runs once every rule passes.

A few honest details:

- `minlength` doesn't complain about an *empty* field. If the field must be filled in, add `required` as well.
- `required` is happy with a name made of only spaces. Use `trim()` in your own check if that matters.
- A `type="number"` box only accepts whole numbers unless you add `step`. For prices, use `step="0.01"`.
- `type="email"` only checks the shape: some text, an `@`, then more text. It even lets `sam@example` through. And it can't know whether the address really exists.
- `pattern` uses a mini-language called regular expressions, which you'll learn in [chapter 37](#). For now: `[0-9]` means "any digit", and `{5}` means "exactly five of them". So `[0-9]{5}` accepts `90210` and rejects `9021`.

You can run the same checks from JavaScript. `checkValidity()` works on one field or on the whole form, and returns `true` or `false`:

```

const clubForm = document.querySelector("#club-form");
const ageInput = document.querySelector("#age");

ageInput.value = "12"; // setting .value from code, to test quickly
console.log(ageInput.checkValidity()); // prints: false
ageInput.value = "30";
console.log(ageInput.checkValidity()); // prints: true
console.log(clubForm.checkValidity()); // prints: false (the name is still empty)

```

Your own checks with JavaScript

The built-in rules can't check everything. What about these?

- "The two passwords must match."
- "Usernames can't contain spaces."
- "Pick at least one day."

For rules like these, you write your own checks. And instead of a bubble, you show your own message right next to the field, where the user is looking.

Here's a sign-up form for a gaming website. Each field that needs a custom check gets an empty `<p>` under it, ready to hold an error message:

```
<form id="account-form">
  <label for="username">Username</label>
  <input id="username" name="username" type="text" required minlength="3">
  <p id="username-error" class="error"></p>

  <label for="password">Password (at least 8 characters)</label>
  <input id="password" name="password" type="password" required minlength="8">

  <label for="confirm">Confirm password</label>
  <input id="confirm" name="confirm" type="password" required>
  <p id="confirm-error" class="error"></p>

  <button id="create-button" type="submit">Create account</button>
</form>
```

First, a function that checks the data and returns an object of error messages, one for each field with a problem:

```
function getErrors(data) {
  const errors = {};
  if (data.username.includes(" ")) {
    errors.username = "Usernames can't contain spaces.";
  }
  if (data.password !== data.confirm) {
    errors.confirm = "The passwords don't match.";
  }
  return errors;
}
```

If nothing is wrong, you get back an empty object: `{}`.

Next, a function that puts the messages on the page:

```
const usernameError = document.querySelector("#username-error");
const confirmError = document.querySelector("#confirm-error");

function showErrors(errors) {
  usernameError.textContent = errors.username ?? "";
  confirmError.textContent = errors.confirm ?? "";
}
```


`errors.username ?? ""` means "the username message, or an empty string if there isn't one" (`??` is from [chapter 07](#)). The empty string matters: it wipes out an old message once the user has fixed the problem.

Finally, the submit handler puts it all together:

```
const accountForm = document.querySelector("#account-form");

accountForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = Object.fromEntries(new FormData(accountForm));
  const errors = getErrors(data);
  showErrors(errors);

  if (Object.keys(errors).length > 0) {
    return; // something's wrong: stop here
  }
  console.log(`Welcome, ${data.username}!`);
});
```

`Object.keys(errors).length > 0` asks "is there at least one error?" (`Object.keys` is from chapter 11).

Try it with the username `lara croft`, the password `tomraider1`, and `tomraider2` to confirm. Both messages appear under their fields, and nothing is logged. Now change the username to `lara_croft`, make the passwords match, and submit again. The messages disappear, and you'll see:

```
Welcome, lara_croft!
```

A few good habits:

- Put messages in with `textContent`, never `innerHTML`. They might contain what the user typed ([chapter 20](#) explains the danger).
- Give the messages a class like `error`, so your CSS can make them stand out: `.error { color: crimson; }`.
- Remember that your `submit` handler only runs after the browser's built-in checks pass. So your custom checks only need to cover what HTML can't.

Watch out: Checks in the browser are there to help the user. They don't protect anything, because anyone can switch them off in DevTools. A real app's server must check the data again. You'll learn more in [chapter 51](#).

Checking while the user types

Waiting until "Create account" to show problems works. But it's friendlier to give feedback while people type.

The `input` event fires on every keystroke. And because events bubble up ([chapter 21](#)), one listener on the form hears every field in it:

```
const createButton = document.querySelector("#create-button");

accountForm.addEventListener("input", () => {
  const data = Object.fromEntries(new FormData(accountForm));
  const errors = getErrors(data);
  showErrors(errors);

  const isValid = accountForm.checkValidity() && Object.keys(errors).length === 0;
  createButton.disabled = !isValid;
});
```

The last two lines **disable the submit button while the form is invalid**. You met the `disabled` property in chapter 20: a disabled button is greyed out and can't be clicked. Pressing Enter won't submit the form either. `isValid` is only `true` when the browser's rules *and* your own rules all pass.

To start with the button greyed out, add `disabled` to it in the HTML:

```
<button id="create-button" type="submit" disabled>Create account</button>
```

Watch out: While the button is disabled, the user can't submit, so the browser never shows its bubbles. Your page has to explain the rules itself. That's why the password label says "(at least 8 characters)".

Resetting the form

After a successful sign-up, the form should be empty again for the next person. `form.reset()` puts every field back the way it was when the page loaded.

Here's the finished submit handler. It replaces the one from before, with two new lines at the end:

```
accountForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = Object.fromEntries(new FormData(accountForm));
  const errors = getErrors(data);
  showErrors(errors);

  if (Object.keys(errors).length > 0) {
    return;
  }
  console.log(`Welcome, ${data.username}!`);
  accountForm.reset(); // empty the form for the next person
  createButton.disabled = true; // and grey out the button again
});
```

"The way it was when the page loaded" isn't always empty. A field with a starting `value`, or a radio button marked `checked`, goes back to that.

Two things `reset()` doesn't do:

- **It only resets the fields.** Error messages, "Thanks!" notes, and anything else your code wrote on the page stay until you clear them.
- **It doesn't fire input events**, so the live check doesn't notice. That's why the handler greys out the button itself.

There's also `<button type="reset">`, which resets the form when clicked. Use it with care: one wrong click wipes out everything the user typed.

Labels and accessibility

Accessibility (often written **a11y**, because there are 11 letters between the "a" and the "y") means building pages that everyone can use. That includes people who use a keyboard instead of a mouse, and people who use a **screen reader**: software that reads the page out loud.

Labels are the easiest win. When a `<label for="email">` is linked to an `<input id="email">`:

- Clicking the label puts the cursor in the field, or ticks the checkbox. A bigger target is easier to hit, especially on a phone.
- A screen reader reads the label out when the field is selected, so the user knows what to type. Without a label, they hear something vague, like "edit text".

You can also put the field *inside* the label. Then you don't need `for` and `id` at all:

```
<label>
  Email
  <input name="email" type="email">
</label>
```

Remember the `<fieldset>` and `<legend>` around the radio buttons? They help in the same way. A screen reader says "Size" before it reads out the choices, so the user knows what they're choosing. Use them for any group of radio buttons or checkboxes.

Watch out: A `placeholder` (the grey hint text in an empty box, like `placeholder="you@example.com"`) is not a label. It disappears as soon as you start typing, and then nobody remembers what the box was for.

Common mistakes

1. Reading `.value` too early

```
const nameInput = document.querySelector("#name");
const name = nameInput.value; // read once, when the page loads

form.addEventListener("submit", (event) => {
  event.preventDefault();
  console.log(`Hello, ${name}!`); // prints: Hello, !
});
```

Your script runs as soon as the page loads, before the user has typed anything. So `name` is `""` forever. Fix: read `nameInput.value` inside the handler.

2. Forgetting `event.preventDefault()`

```
form.addEventListener("submit", () => {
  console.log("Sent!");
});
```

"Sent!" flashes in the console for a split second, then the page reloads and it's gone. You may also notice the form's data at the end of the address bar, after a `?`. Fix: take the `event` parameter and call `event.preventDefault()` first.

3. Doing math with strings

```
const adults = document.querySelector("#adults").value;    // "2"
const children = document.querySelector("#children").value; // "3"
console.log(adults + children); // prints: 23
```

Two adults and three children make 23 cinema tickets? The values are strings, so `+` joined them. Fix:

`Number(adults) + Number(children)` gives `5`. And remember that an empty box turns into `0`, so check for `""` first.

4. Forgetting the `name` attribute

```
<input id="email" type="email">
```

```
const data = Object.fromEntries(new FormData(form));
console.log(data.email); // prints: undefined
```

`FormData` only collects fields that have a `name`. An `id` isn't enough. Fix: `<input id="email" name="email" type="email">`.

5. A button that submits by accident

```
<form id="login-form">
  <input id="password" name="password" type="password">
  <button id="show-button">Show password</button>
  <button type="submit">Log in</button>
</form>
```

Clicking "Show password" also sends the form, because every button inside a form is a submit button unless you say otherwise. Fix: `<button id="show-button" type="button">`.

Quick recap

- Read text boxes, `<textarea>`s and `<select>`s with `.value`, and checkboxes with `.checked`. Read them inside a handler, when you need them.
- Form values are always strings. Check for `""`, then convert with `Number()`.
- Listen for `submit` on the form, and call `event.preventDefault()` to stop the page from reloading.
- `new FormData(form)` reads every field that has a `name`. `Object.fromEntries()` turns it into a plain object.
- Let HTML do the simple checks (`required`, `minlength`, `type="email"`, `min / max`, `pattern`). Add your own checks in JavaScript, and show their messages next to the fields.
- `form.reset()` resets the fields, but not your messages. And give every field a `<label>`.

Next: try the [exercises](#), then move on to [23 JSON and Local Storage](#).

Exercises

How to do these:

- Make a folder for each exercise in this chapter's folder (`ex1` , `ex2` , and so on). Inside it, create `index.html` (copy the HTML from the exercise) and `script.js` (your code).
- Open `index.html` by double-clicking it. Press `F12` and open the **Console** tab to see messages and errors. Save and refresh the page after each change, the same way as in [chapter 20](#).
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Library card

A library's website lets people sign up for a library card. Greet each new reader by name, but don't let an empty name through.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Library Card</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>City Library</h1>
    <form id="card-form">
      <label for="reader">Your name</label>
      <input id="reader" name="reader" type="text">
      <button type="submit">Get my card</button>
    </form>
    <p id="message"></p>
  </body>
</html>
```

When the form is submitted:

1. Stop the page from reloading.
2. Read the name and trim the spaces off.
3. If the name is empty, show `Please type your name.` in the `#message` paragraph.
4. Otherwise, show the welcome message below and empty the name box, ready for the next reader.

Type `Ana` and press Enter. The page shows:

```
Welcome to the library, Ana! Your card is ready.
```

Click "Get my card" with the box empty (or with only spaces in it), and the page shows:

```
Please type your name.
```

Rule: don't add `required` to the input. Do the check in JavaScript. (Remember why? `required` would let a name made of only spaces through.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Split the bill

Four friends had dinner together and want to split the bill, tip included. Build the calculator.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Split the Bill</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Split the bill</h1>
    <form id="bill-form">
      <label for="bill">Bill ($)</label>
      <input id="bill" name="bill" type="number" min="0" step="0.01" required>

      <label for="tip">Tip</label>
      <select id="tip" name="tip">
        <option value="10">10%</option>
        <option value="15" selected>15%</option>
        <option value="20">20%</option>
      </select>

      <label for="people">People</label>
      <input id="people" name="people" type="number" min="1" value="2" required>

      <button type="submit">Split it</button>
    </form>
    <p id="result"></p>
  </body>
</html>

```

(`selected` makes 15% the starting choice, the way `checked` does for radio buttons.)

When the form is submitted, work out the tip, the total, and how much each person pays. Show them in `#result`, each with 2 decimal places.

For a bill of `84`, a 15% tip and `4` people, the page shows:

Tip: \$12.60 | Total: \$96.60 | Each pays: \$24.15

For a bill of `50`, a 20% tip and `3` people:

Tip: \$10.00 | Total: \$60.00 | Each pays: \$20.00

The fields use `required` and `min`, so the browser already stops empty or negative values. You don't need your own empty checks here.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Pizza order

Luigi's Pizza wants an order form that shows a summary and the price before the order goes to the kitchen.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Pizza Order</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Luigi's Pizza</h1>
    <form id="order-form">
      <fieldset>
        <legend>Size</legend>
        <input id="small" type="radio" name="size" value="small" checked>
        <label for="small">Small ($8)</label>
        <input id="medium" type="radio" name="size" value="medium">
        <label for="medium">Medium ($10)</label>
        <input id="large" type="radio" name="size" value="large">
        <label for="large">Large ($12)</label>
      </fieldset>

      <fieldset>
        <legend>Toppings ($1.50 each)</legend>
        <input id="mushrooms" type="checkbox" name="topping" value="mushrooms">
        <label for="mushrooms">Mushrooms</label>
        <input id="olives" type="checkbox" name="topping" value="olives">
        <label for="olives">Olives</label>
        <input id="peppers" type="checkbox" name="topping" value="peppers">
        <label for="peppers">Peppers</label>
      </fieldset>

      <label for="method">How do you want it?</label>
      <select id="method" name="method">
        <option value="pickup">Pickup (free)</option>
        <option value="delivery">Delivery (+$3)</option>
      </select>

      <button type="submit">Order</button>
    </form>
    <p id="summary"></p>
  </body>
</html>
```

The prices: a small pizza is \$8, a medium is \$10, and a large is \$12. Each topping adds \$1.50, and delivery adds \$3.

When the form is submitted, show the order in `#summary`. A large pizza with mushrooms and olives, for delivery:

Your order: large pizza with mushrooms, olives, for delivery. Total: \$18.00

A medium pizza with all three toppings, for pickup:

Your order: medium pizza with mushrooms, olives, peppers, for pickup. Total: \$14.50

The starting choices (small, no toppings, pickup):

Your order: small pizza with no toppings, for pickup. Total: \$8.00

Rule: read the form with `FormData`, not with one `querySelector` per field.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Restaurant feedback

A restaurant asks guests for a star rating and a short comment. Comments must fit on the receipt, so they're limited to 120 characters, and a counter shows how many are left.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Feedback</title>
    <style>
      .error { color: crimson; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>How was your meal?</h1>
    <form id="feedback-form">
      <fieldset>
        <legend>Rating</legend>
        <input id="star1" type="radio" name="rating" value="1" required>
        <label for="star1">1</label>
        <input id="star2" type="radio" name="rating" value="2">
        <label for="star2">2</label>
        <input id="star3" type="radio" name="rating" value="3">
        <label for="star3">3</label>
        <input id="star4" type="radio" name="rating" value="4">
        <label for="star4">4</label>
        <input id="star5" type="radio" name="rating" value="5">
        <label for="star5">5</label>
      </fieldset>

      <label for="comment">Your comments</label>
      <textarea id="comment" name="comment" maxlength="120" rows="4"></textarea>
      <p id="counter">120 characters left</p>
      <p id="comment-error" class="error"></p>

      <button type="submit">Send feedback</button>
    </form>
    <p id="thanks"></p>
  </body>
</html>

```

required on one radio button makes the whole group required, so the browser already stops a form with no rating. Your job:

1. **Live counter:** as the guest types, update `#counter` to show how many of the 120 characters are left.
2. **Custom check:** when the form is submitted, if the comment (trimmed) is shorter than 10 characters, show `Please write at least 10 characters.` in `#comment-error`, clear `#thanks`, and stop.
3. **Success:** otherwise, clear the error, show the thanks message with the real rating in `#thanks`, and reset the form. After the reset, the counter must say `120 characters left` again.

Type `Great pasta, friendly staff!` and the counter shows:

92 characters left

Choose 4 stars and click "Send feedback". The form empties, the counter goes back to `120 characters left`, and the page shows:

Thanks for the 4-star review!

Choose any rating, type `Yum`, and submit. The page shows:

Please write at least 10 characters.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Loan calculator

A bank wants a loan calculator on its website. The "Calculate" button stays greyed out until every field is valid, and helpful messages appear while the customer types.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Loan Calculator</title>
    <style>
      .error { color: crimson; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Loan calculator</h1>
    <form id="loan-form">
      <label for="amount">Amount ($1,000 to $500,000)</label>
      <input id="amount" name="amount" type="number" min="1000" max="500000" required>
      <p id="amount-error" class="error"></p>

      <label for="years">Years (1 to 30)</label>
      <input id="years" name="years" type="number" min="1" max="30" required>
      <p id="years-error" class="error"></p>

      <label for="rate">Interest rate (% per year, 0.1 to 20)</label>
      <input id="rate" name="rate" type="number" min="0.1" max="20" step="any" required>
      <p id="rate-error" class="error"></p>

      <button id="calculate-button" type="submit" disabled>Calculate</button>
    </form>

    <p>Monthly payment: <strong id="monthly">-</strong></p>
    <p>Total paid: <strong id="total">-</strong></p>
    <p>Total interest: <strong id="interest">-</strong></p>
  </body>
</html>

```

(`step="any"` lets the rate have any number of decimals, like `4.5` or `3.75` .)

1. The math. Write a function `calculateLoan(amount, years, rate)` that returns an object with three numbers: `monthly` , `total` and `interest` . It must not touch the page. Here's the formula banks use:

```

monthlyRate = rate / 100 / 12
months      = years * 12
monthly     = amount * monthlyRate / (1 - (1 + monthlyRate) ** -months)
total       = monthly * months
interest    = total - amount

```

Test it before you build anything else. `console.log(calculateLoan(20000, 5, 6).monthly.toFixed(2));` should print `386.66`.

2. Live checks. Every time the user types in any field, go through all three fields. If a field isn't empty but breaks its rules, show its message under it. Otherwise, clear its message. Enable "Calculate" only when the whole form is valid.

Field	Message
amount	Enter an amount from \$1,000 to \$500,000.
years	Enter 1 to 30 years.
rate	Enter a rate from 0.1% to 20%.

For example, type `500` in the amount box. The amount message appears under it, and the button stays greyed out.

3. The result. On submit, get the three values with `Object.fromEntries(new FormData(form))` and destructuring from [chapter 15](#). Call `calculateLoan`, and show the three results as dollars with `Intl.NumberFormat` from [chapter 05](#).

A car loan of `20000` over `5` years at `6 %`:

Monthly payment: \$386.66
Total paid: \$23,199.36
Total interest: \$3,199.36

A home loan of `250000` over `30` years at `4.5 %`:

Monthly payment: \$1,266.71
Total paid: \$456,016.78
Total interest: \$206,016.78

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Refresh any of these pages, and everything is gone: the half-written review, the loan you just worked out. Wouldn't it be nice if the page could remember?

[Chapter 23: JSON and Local Storage](#) shows you how to save data in the browser, so it's still there next time. 😊

23 JSON and Local Storage

What is it?

JSON (short for JavaScript Object Notation, and usually said like the name "Jason") is a way to write data as plain text. Objects, arrays, numbers and strings get written out as characters, so they can be saved in a file or sent across the internet.

Local storage is a small storage space that the browser gives your web page. Whatever you put in it stays there, even after the tab is closed.

Together, they let your apps remember things.

Why does it matter?

Right now, everything your code creates disappears when the page reloads. Build a shopping list in the browser, press `F5`, and it's gone. Real apps need to remember: your to-do list, your high score, your favorite settings.

Data also needs to travel. A weather app gets its forecast from a server. A game saves your progress to a file. Your phone and your laptop show the same account. The programs on each end might be written in different languages, so they need a format that everyone understands.

JSON is that format, and you'll meet it everywhere:

- data from web APIs ([chapter 33](#)),
- settings files, like the `package.json` file in [chapter 50](#),
- saving your app's data in the browser (this chapter).

Real-world example

Before shipping containers existed, every ship, port and truck packed cargo its own way. Loading one ship could take days. Then the world agreed on one standard steel box. Now any crane in any port can lift it onto any ship, train or truck.

JSON is the standard shipping container for data. JavaScript, Python, Java, your phone, a server on the other side of the world: they can all pack it and unpack it.

Shipping	Data
Your things: a sofa, boxes of books	Your JavaScript objects and arrays

Shipping	Data
Packing them into the standard container	<code>JSON.stringify()</code>
The sealed container, ready to travel	A JSON string
Unpacking at the other end	<code>JSON.parse()</code>
Things you're not allowed to ship, like a lit candle	Functions and <code>undefined</code> don't survive the trip
A storage unit where you keep the container	<code>localStorage</code>

How it works

What JSON looks like

Here's a library book, written as JSON:

```
{
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "year": 1937,
  "available": true,
  "tags": ["fantasy", "classic"],
  "borrowedBy": null
}
```

It looks almost exactly like a JavaScript object from [chapter 11](#). That's where the name comes from. But JSON is stricter:

	JSON	JavaScript
Keys	Always in double quotes: <code>"title"</code>	Quotes are optional: <code>title</code>
Text	Double quotes only: <code>"The Hobbit"</code>	<code>"..."</code> , <code>'...'</code> or backticks
Values allowed	Strings, numbers, <code>true</code> , <code>false</code> , <code>null</code> , arrays, objects	Anything, including functions and <code>undefined</code>
A comma after the last item	Not allowed	Allowed
Comments	Not allowed	Allowed

And the most important difference: JSON is **text**. It's a string that *describes* data. Before your code can use it, you have to turn it into real objects and arrays. That's what the two JSON functions are for.

From data to JSON: `JSON.stringify`

`JSON.stringify()` packs a value into a JSON string. The JSON parts of this chapter run in Node, so save this as `json.js` and run it with `node json.js`:

```
const book = { title: "The Hobbit", year: 1937, available: true };
const json = JSON.stringify(book);

console.log(json);           // prints: {"title":"The Hobbit","year":1937,"available":true}
console.log(typeof json);    // prints: string
```

That's one long line with no spaces. Great for computers, hard for people. Give `JSON.stringify` two more arguments, and it **pretty-prints** the JSON over several lines:

```
console.log(JSON.stringify(book, null, 2));
```

You'll see:

```
{
  "title": "The Hobbit",
  "year": 1937,
  "available": true
}
```

The `2` means "indent with 2 spaces". The `null` in the middle is for an extra option you'll rarely need, so just pass `null`.

Arrays work too:

```
const scores = [98, 87, 100];
console.log(JSON.stringify(scores)); // prints: [98,87,100]
```

From JSON back to data: `JSON.parse`

`JSON.parse()` does the opposite. It unpacks a JSON string into real objects and arrays:

```
const text = '{"name":"Mia","level":7,"badges":["speedy","explorer"]}';
const player = JSON.parse(text);

console.log(player.name);           // prints: Mia
console.log(player.level + 1);      // prints: 8
console.log(player.badges.length); // prints: 2
```

`player.level` is a real number, so `+ 1` does math. And `player.badges` is a real array. JSON remembers which values are numbers, booleans or strings, unlike the form values in [chapter 22](#), which are always strings.

Why the single quotes around `text`? JSON needs double quotes inside, so single quotes on the outside stop the first `"` from ending the string. It's one of the few places where single quotes are the easy choice. In real apps, you'll rarely type JSON by hand anyway. It usually comes from a server, a file, or storage.

When the JSON is broken

Data from outside your code isn't always in good shape. A file gets cut off halfway, someone edits it by hand, or a server sends an error page instead of data.

`JSON.parse()` can't guess what broken JSON was meant to say, so it throws a `SyntaxError`:

```
JSON.parse('{"name": "Mia", "level": 7,}');
// SyntaxError: Expected double-quoted property name in JSON at position 27 (line 1 column 28)
```

The comma after `7` is the problem: JSON expected another `"name"` after it. The message even tells you where. Position 27 is counted from 0, like string indexes in [chapter 06](#).

You can't control data that comes from outside, so wrap `JSON.parse()` in `try / catch` from [chapter 18](#). Here's a game loading a save file that got cut off:

```
const savedGame = '{"level": 7, "coins": 120'; // the end is missing

try {
  const game = JSON.parse(savedGame);
  console.log(`Welcome back! You're on level ${game.level}.`);
} catch (error) {
  console.log(`Couldn't load your save: ${error.message}`);
  console.log("Starting a new game instead.");
}
```

You'll see:

```
Couldn't load your save: Expected ',' or '}' after property value in JSON at position 25 (line 1 column 26)
Starting a new game instead.
```

The broken save doesn't crash the game. It starts a fresh one instead.

Here's what Node says about a few other classic mistakes:

Broken JSON	What's wrong	Node's error message
<pre>{'name': 'Mia'}</pre>	Single quotes	Expected property name or '}' in JSON at position 1 (line 1 column 2)
<pre>{name: "Mia"}</pre>	A key without quotes	Expected property name or '}' in JSON at position 1 (line 1 column 2)
<pre>[1, 2,]</pre>	A comma after the last item	Unexpected token ',', "[1, 2,]" is not valid JSON
an empty string	Nothing to read at all	Unexpected end of JSON input

What doesn't survive the trip

JSON only knows strings, numbers, booleans, `null`, arrays and plain objects. Anything else changes or disappears when you stringify it. Here's a hotel booking:

```
const booking = {  
  guest: "Omar",  
  nights: 3,  
  checkIn: new Date("2026-10-01T14:00:00Z"),  
  notes: undefined,  
  greet() {  
    return `Welcome, ${this.guest}!`;  
  },  
};  
  
const copy = JSON.parse(JSON.stringify(booking));  
console.log(copy);
```

You'll see:

```
{ guest: 'Omar', nights: 3, checkIn: '2026-10-01T14:00:00.000Z' }
```

`notes` and `greet` are gone, and `checkIn` has turned into a string.

Value	After a trip through JSON
Strings, numbers, <code>true / false</code> , <code>null</code>	Stay the same
Arrays and plain objects	Stay the same (as brand-new copies)
Dates	Turn into ISO strings, like <code>"2026-10-01T14:00:00.000Z"</code>
<code>undefined</code>	The property disappears
Functions and methods	Disappear
<code>NaN</code> and <code>Infinity</code>	Turn into <code>null</code>

The date is the sneaky one. It looks fine when you print it, but it's a string now:

```
console.log(typeof copy.checkIn); // prints: string
copy.checkIn.getFullYear();
// TypeError: copy.checkIn.getFullYear is not a function
```

The fix is the one from [chapter 19](#): turn the string back into a date with `new Date()`.

```
const checkIn = new Date(copy.checkIn);
console.log(checkIn.getFullYear()); // prints: 2026
```

This is also why [chapter 16](#) called `JSON.parse(JSON.stringify(obj))` a limited way to copy objects. For copying, `structuredClone` is the better tool. JSON is for saving and sending data.

Saving data in the browser: `localStorage`

Now for the storage unit. `localStorage` is a small storage space the browser keeps for each website. What you save stays there when the page reloads, when the tab closes, and even when the computer restarts, until your code (or the user) removes it.

It works like a wall of labeled lockers. You put a value in under a **key** (a name you choose), and later you get it back with the same key.

`localStorage` only exists in the browser. In Node, you'd get `ReferenceError: localStorage is not defined`. So try the next examples in a practice page like the `playground` from [chapter 20](#), in its `script.js` or straight in the Console.

```
localStorage.setItem("theme", "dark");    // save "dark" under the key "theme"
console.log(localStorage.getItem("theme")); // prints: dark

localStorage.removeItem("theme");          // delete that one item
console.log(localStorage.getItem("theme")); // prints: null

localStorage.clear(); // delete everything this site has saved
```

Method	What it does
<code>setItem(key, value)</code>	Saves <code>value</code> under <code>key</code> , replacing anything already there
<code>getItem(key)</code>	Gives back the saved value, or <code>null</code> if nothing is saved under that key
<code>removeItem(key)</code>	Deletes one item
<code>clear()</code>	Deletes every item this site has saved

Now the part that feels like magic. Put these two lines in `script.js`:

```
console.log("Saved name:", localStorage.getItem("player-name"));
localStorage.setItem("player-name", "Mia");
```

The first time you open the page, the Console says `Saved name: null`. Refresh, and it says `Saved name: Mia`. Close the browser, open the page again, and it still says `Saved name: Mia`. The value survived.

See it in DevTools: press `F12`, open the **Application** tab (click `>>` if you can't see it), and under **Storage**, open **Local storage**. You'll see every key and value your page has saved. You can edit or delete them there too, which is handy for testing.

Every website gets its own storage, so a page on one site can't read what another site saved.

Tip: In Chrome and Edge, the pages you open straight from your computer (`file://`) can all share one storage. Give your keys clear, specific names, like `"player-name"` or `"cinema-watchlist"`, instead of just `"data"`. Then your practice pages won't overwrite each other.

Storage only holds strings

`localStorage` can only store **strings**. Anything else gets turned into a string on the way in:

```
localStorage.setItem("coins", 120);
const coins = localStorage.getItem("coins");

console.log(typeof coins); // prints: string
console.log(coins + 10);    // prints: 12010
```

That's the same trap as form values in chapter 22. Objects are even worse:

```
const settings = { volume: 7, music: true };
localStorage.setItem("settings", settings);
console.log(localStorage.getItem("settings")); // prints: [object Object]
```

[object Object] is what JavaScript writes when it turns an object into a string without JSON. Your settings are lost.

This is where the two ideas come together. Stringify before you save, and parse after you load:

```
localStorage.setItem("settings", JSON.stringify(settings));

const savedSettings = JSON.parse(localStorage.getItem("settings"));
console.log(savedSettings.volume); // prints: 7
```

JSON is the container, and `localStorage` is the storage unit: pack, store, unpack.

A reusable pattern: load with a default value

Every time you load saved data, two things can go wrong:

1. **Nothing is saved yet.** On someone's very first visit, `getItem` gives you `null`.
2. **The saved text is broken.** Someone edited it in DevTools, or an older version of your app saved it in a different shape.

Instead of handling both problems every time, write two small helper functions once, and reuse them in every project:

```
function save(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

function load(key, defaultValue) {
  const text = localStorage.getItem(key);
  if (text === null) {
    return defaultValue; // nothing saved yet
  }
  try {
    return JSON.parse(text);
  } catch (error) {
    console.error(`Couldn't read "${key}", so the default was used.`, error.message);
    return defaultValue; // the saved data was broken
  }
}
```

`load` always gives you something you can use: the saved value if there is one, or your default if there isn't. And it doesn't hide problems. `console.error` still reports broken data, because an error should never vanish silently (chapter 18).

Here's a visit counter that uses them:

```
const visits = load("visit-count", 0) + 1;
save("visit-count", visits);
console.log(`Visit number ${visits}`);
```

Open the page, and you'll see `Visit number 1`. Refresh: `Visit number 2`. Refresh again: `Visit number 3`. The number comes back as a real number, because JSON keeps the types.

Now break it on purpose. In the Console, type `localStorage.setItem("visit-count", "oops")` and refresh. Instead of crashing, the page starts again from `Visit number 1`, and a red message in the Console explains why:

```
Couldn't read "visit-count", so the default was used. Unexpected token 'o', "oops" is not valid JSON
```

The default value matters most for lists:

```
const watchlist = load("cinema-watchlist", []);
console.log(watchlist.length); // prints: 0 (on the first visit)
```

Because the default is `[]`, you can `push` to `watchlist` or loop over it straight away, even on the first visit. With no default, `load` would give you `undefined`, and `watchlist.push("Dune")` would crash with `TypeError: Cannot read properties of undefined (reading 'push')`.

sessionStorage : storage that forgets

`sessionStorage` has exactly the same methods as `localStorage`, but it forgets everything when the tab is closed. A **session** is one visit: from opening a tab to closing it.

```
sessionStorage.setItem("checkout-step", "2");
console.log(sessionStorage.getItem("checkout-step")); // prints: 2
```

	localStorage	sessionStorage
Survives a refresh	Yes	Yes
Survives closing the tab	Yes	No
Shared between tabs of the same site	Yes	No, each tab has its own

Use `sessionStorage` for things that only matter during one visit: which step of the checkout you're on, a half-filled form, or a banner the user has already closed. Use `localStorage` for things that should still be there tomorrow.

Limits, and what not to store

- **It's small.** Browsers give each site around 5 MB, depending on the browser. That's plenty for settings and lists, but not for photos or videos. When it's full, `setItem` throws an error.
- **It's not secret.** Anyone using the computer can read it in DevTools, and so can any script running on your page. Never store passwords, credit card numbers or login tokens there. [Chapter 51](#) explains the safer options.
- **It can vanish.** Users can clear their browser data at any time, and private windows forget everything when they close. Your app must still work when storage is empty. The default value in `load` takes care of that.
- **It stays on one device.** A list saved in Chrome on your laptop won't show up on your phone. Sharing data between devices needs a server, which is a story for later.

Common mistakes

1. Saving an object without `JSON.stringify`

```
localStorage.setItem("cart", { items: 3 });
console.log(localStorage.getItem("cart")); // prints: [object Object]
```

The object was turned into the useless string `"[object Object]"`. Fix: `localStorage.setItem("cart", JSON.stringify({ items: 3 }));`, or use your `save` helper.

2. Forgetting `JSON.parse` when loading

```
localStorage.setItem("cart", JSON.stringify({ items: 3 }));  
const cart = localStorage.getItem("cart");  
console.log(cart.items); // prints: undefined
```

`cart` is still the string `'{"items":3}'`, and a string has no `items` property. Fix: `JSON.parse(localStorage.getItem("cart"))`, or use your `load` helper.

3. Parsing without `try / catch`

```
const cart = JSON.parse(localStorage.getItem("cart"));  
// SyntaxError: "[object Object]" is not valid JSON
```

One bad save (like the one from mistake 1) and your app crashes every time it starts, until the user clears their storage. Fix: wrap `JSON.parse` in `try / catch`, or use `load`, which does it for you.

4. Writing JSON by hand like a JavaScript object

```
JSON.parse("{ name: 'Mia', level: 7, }");  
// SyntaxError: Expected property name or '}' in JSON at position 2 (line 1 column 3)
```

JSON is stricter than JavaScript. Keys and strings need double quotes, and there's no comma after the last item. Fix: `'{"name": "Mia", "level": 7}'`. Even better, build a real object and let `JSON.stringify` write the JSON for you.

5. Expecting a date to come back as a date

```
save("last-order", { placedAt: new Date("2026-03-14T09:30:00Z") });  
const order = load("last-order", null);  
console.log(order.placedAt.getFullYear());  
// TypeError: order.placedAt.getFullYear is not a function
```

Dates come back as ISO strings. Fix: turn it back into a date first, with `new Date(order.placedAt).getFullYear()`.

Quick recap

- JSON is text that describes data. Its rules are strict: double quotes, no functions, no comma after the last item, no comments.
- `JSON.stringify(value)` packs data into a JSON string (add `null, 2` to pretty-print it). `JSON.parse(text)` unpacks it again.
- Wrap `JSON.parse` in `try / catch`, because data from outside can be broken.

- On the way through JSON, dates become strings, and functions and `undefined` disappear.
 - `localStorage` keeps strings, even after the browser closes: `setItem`, `getItem`, `removeItem`, `clear`. Use JSON for objects and arrays. `sessionStorage` works the same way, but forgets when the tab closes.
 - Use a `load(key, defaultValue)` helper, and never store passwords or tokens.
-

Next: try the [exercises](#), then move on to [24 Project: To-Do List App](#).

Exercises

How to do these:

- Exercises 1 and 3 run in **Node**. Make `ex1.js` (or `ex3.js`) in this chapter's folder and run it with `node ex1.js`.
 - Exercises 2, 4 and 5 run in the **browser**. Make a folder for each one (`ex2`, `ex4`, `ex5`) with an `index.html` (copy the HTML from the exercise) and a `script.js`. Open `index.html` by double-clicking it, and keep the Console open (`F12`).
 - To check that your data survives, refresh the page. To start over, delete your keys in DevTools (**Application** tab → **Local storage**), or run `localStorage.clear()` in the Console.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Parcel labels

A delivery company's computers send parcel details to each other as JSON. Start your file with this data:

```
const parcel = {
  id: "PKG-1042",
  weightKg: 2.5,
  fragile: true,
  items: ["mug", "teapot"],
  deliveredAt: null,
};

const fromWarehouse = '{"id":"PKG-2077","weightKg":12,"fragile":false,"items":
  ["books","lamp"],"deliveredAt":null}';
```

1. Print `parcel` as JSON on one line.
2. Print it again, pretty-printed with 2 spaces.
3. Parse `fromWarehouse` and print a sentence about that parcel.

Expected output:

```
{ "id": "PKG-1042", "weightKg": 2.5, "fragile": true, "items": ["mug", "teapot"], "deliveredAt": null }
{
  "id": "PKG-1042",
  "weightKg": 2.5,
  "fragile": true,
  "items": [
    "mug",
    "teapot"
  ],
  "deliveredAt": null
}
PKG-2077 weighs 12 kg and contains 2 items: books, lamp
```

Rule: don't type any of the output yourself. Let `JSON.stringify` and your parsed object do the work.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Remember my name

A website greets visitors by name, and remembers them next time.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Welcome</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1 id="greeting">Welcome, stranger!</h1>
    <form id="name-form">
      <label for="visitor">What's your name?</label>
      <input id="visitor" name="visitor" type="text" required>
      <button type="submit">Save</button>
    </form>
    <button id="forget-button">Forget me</button>
  </body>
</html>
```

1. When the form is submitted, save the trimmed name in `localStorage` under the key `"welcome-name"`, and change the heading to `Nice to meet you, Sam!` (with the real name).

2. When the page loads and a name is saved, the heading says `Welcome back, Sam!`. With no saved name, it stays `Welcome, stranger!`.
3. "Forget me" deletes the saved name and puts the heading back to `Welcome, stranger!`.

Here's what you should see, step by step:

You do this	The heading says
Open the page for the first time	<code>Welcome, stranger!</code>
Type <code>Sam</code> and click "Save"	<code>Nice to meet you, Sam!</code>
Refresh the page	<code>Welcome back, Sam!</code>
Click "Forget me"	<code>Welcome, stranger!</code>
Refresh the page again	<code>Welcome, stranger!</code>

A name is already a string, so this exercise doesn't need JSON.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Glitchy thermostats

A smart home hub collects temperature readings from sensors around the house. Each reading arrives as a JSON string, but some of the sensors are glitchy and send broken JSON. Start your file with this data:

```
const readings = [
  '{"room": "kitchen", "celsius": 21.5}',
  '{"room": "bedroom", "celsius": 19}',
  '{"room": "garage", "celsius": 12,}',
  '{"room": "attic", "celsius": 25}',
];
```

1. Write a function `parseReading(text)`. It returns the parsed reading, or prints a message and returns `null` if the JSON is broken.
2. Go through all the readings. Print each good one, and skip the broken ones.
3. At the end, print the average temperature of the good readings.

Expected output:

```
kitchen: 21.5°C
Skipped a broken reading: Expected property name or '}' in JSON at position 1 (line 1 column 2)
Skipped a broken reading: Expected double-quoted property name in JSON at position 33 (line 1
column 34)
attic: 25°C
Average of 2 good readings: 23.25°C
```

(You can copy the ° symbol from this page.)

Rule: don't type the error messages yourself. Print the `message` of the error that `JSON.parse` throws.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Reader settings

A local news website lets readers pick a text size and hide the weather box. The site should remember their choices.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Town News</title>
    <style>
      .small { font-size: 14px; }
      .medium { font-size: 18px; }
      .large { font-size: 24px; }
      .hidden { display: none; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <form id="settings-form">
      <label for="text-size">Text size</label>
      <select id="text-size">
        <option value="small">Small</option>
        <option value="medium" selected>Medium</option>
        <option value="large">Large</option>
      </select>

      <input id="show-weather" type="checkbox" checked>
      <label for="show-weather">Show the weather box</label>

      <button id="reset-button" type="button">Reset to defaults</button>
    </form>

    <p id="weather">Today: 18°C and sunny</p>
    <article id="article" class="medium">
      <h1>Town library opens a new reading garden</h1>
      <p>The garden has benches, shade trees and free Wi-Fi.</p>
    </article>
  </body>
</html>

```

Keep the settings in an object with this shape. These are also the defaults:

```
const DEFAULT_SETTINGS = { textSize: "medium", showWeather: true };
```

1. Write `applySettings(settings)`. It gives the article the class for the chosen text size (and only that one), and hides the weather box when `showWeather` is `false`.
2. Whenever a setting changes, build a new settings object from the form, save it under the key `"news-settings"`, and apply it.
3. When the page loads, load the saved settings (or the defaults), put them into the form fields, and apply them.

4. "Reset to defaults" deletes the saved settings, puts the defaults back into the form, and applies them.

Choose "Large" and untick the weather box. The text gets bigger and the weather box disappears. Now run this in the Console:

```
console.log(localStorage.getItem("news-settings"));
```

You'll see:

```
{"textSize":"large","showWeather":false}
```

Refresh the page: the text is still large, the weather box is still hidden, and the form still shows "Large" and an empty tick box. Click "Reset to defaults", and everything goes back to medium text with the weather box showing. Run the line above again, and it prints `null`.

Rule: use the `save` and `load` helpers from the notes.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Recently viewed

Online shops often show the products you looked at recently. Build that for Bella's Bakery.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Bella's Bakery</title>
    <style>
      .hidden { display: none; }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Bella's Bakery</h1>
    <ul id="products">
      <li><button data-name="Sourdough loaf">Sourdough loaf</button></li>
      <li><button data-name="Croissant">Croissant</button></li>
      <li><button data-name="Cinnamon roll">Cinnamon roll</button></li>
      <li><button data-name="Bagel">Bagel</button></li>
    </ul>

    <h2>Recently viewed</h2>
    <p id="empty-message">Nothing yet. Click a product to view it.</p>
    <ol id="recent"></ol>
    <button id="clear-button">Clear history</button>
  </body>
</html>

```

The rules:

1. Clicking a product puts its name at the top of "Recently viewed".
2. No duplicates: if the product is already in the list, it moves to the top.
3. Only the 3 most recent products are kept.
4. The list is saved in `localStorage` under the key `"bakery-recent"`, and it's still there after a refresh.
5. `#empty-message` only shows while the list is empty.
6. "Clear history" empties the list and deletes the saved data.
7. Broken data doesn't break the page. Run `localStorage.setItem("bakery-recent", "oops")` in the Console and refresh. The page still works (with an empty list), and the Console shows an error message.

Use **one** click listener on `#products` (event delegation from [chapter 21](#)), and read each product's name from its `data-name` ([chapter 20](#)).

Click Croissant, Sourdough loaf, Bagel, and then Croissant again. "Recently viewed" shows:

1. Croissant
2. Bagel
3. Sourdough loaf

Now click Cinnamon roll:

1. Cinnamon roll
2. Croissant
3. Bagel

(An `<ol>` numbers its items for you, so the numbers aren't part of your text.)

Refresh the page, and the same three are still there. `console.log(localStorage.getItem("bakery-recent"))` prints:

```
["Cinnamon roll", "Croissant", "Bagel"]
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

You now have every piece of a real app: a form to add things ([chapter 22](#)), a list drawn from an array (chapter 20), clicks handled with delegation (chapter 21), and storage that remembers (this chapter).

[Chapter 24](#) puts them all together into your first real web app: a to-do list. 😊

24 Project: To-Do List App

What you'll build

Your first real web app: a to-do list that runs in the browser. When you're done, you'll be able to:

- add tasks,
- tick them off when they're done,
- delete them,
- see how many are left,
- filter the list to show all tasks, only the active ones, or only the completed ones,
- clear away all the finished tasks at once,
- close the browser, come back tomorrow, and find your list exactly as you left it.

The finished app looks something like this:

My To-Do List

New task

[e.g. Buy milk] [Add]

(All) (Active) (Completed)

[x] Buy milk

<- crossed out

x

[] Book the dentist

x

[] Call grandma

x

2 tasks left

Clear completed

What you'll practice:

Skill	Chapter
Arrays of objects	10 and 11
<code>filter</code> , <code>find</code> , <code>some</code> and friends	13
Creating and changing elements	20
Events and event delegation	21
Forms	22
JSON and <code>localStorage</code>	23

390

You'll build it in 8 small milestones. After each one, the app works a little better, and you can check it in the browser before moving on.

Getting started

1. Make a copy of this chapter's `starter` folder, and call it `my-todo-app`. Work in the copy, so the starter stays clean.
2. Open `my-todo-app/index.html` by double-clicking it. You'll see the app's layout: the form, the filter buttons, the message "No tasks here.", "0 tasks left", and a "Clear completed" button. Nothing works yet. That's your job!
3. Press `F12` and open the Console. Your code goes in `app.js`. After every change, save the file and refresh the page, like in [chapter 20](#).

There are three files in the folder:

File	What's in it
<code>index.html</code>	The page. You won't need to change it.
<code>style.css</code>	The looks. It already styles the tasks you'll create, as long as they have the right classes.
<code>app.js</code>	Only a comment listing the milestones. Everything else is up to you.

Two things in `index.html` are new:

- `<link rel="stylesheet" href="style.css">` loads the CSS from its own file, instead of a `<style>` tag in the `<head>`.
- `<meta name="viewport" content="width=device-width, initial-scale=1">` makes the page fit nicely on a phone screen.

These are the parts of the page your code will use:

Selector	What it is
<code>#todo-form</code>	The form with the text box and the Add button
<code>#todo-input</code>	The text box for a new task
<code>#filters</code>	The box around the three filter buttons. Each button has a <code>data-filter</code> : <code>all</code> , <code>active</code> or <code>completed</code> .
<code>#todo-list</code>	The empty <code></code> where your tasks will go
<code>#todo-count</code>	The "0 tasks left" counter

Selector	What it is
<code>#clear-completed</code>	The "Clear completed" button

Tip: The message "No tasks here." isn't in the HTML. The CSS shows it by itself whenever `#todo-list` is empty, so you get it for free.

The big idea: change the data, then draw

Before any code, here's the one idea that keeps this app simple instead of messy.

Think of the scoreboard at a football stadium. Nobody climbs up a ladder to change the numbers by hand. The official scorer writes each goal on the score sheet, and the scoreboard shows whatever the sheet says. The sheet is the truth. The board just displays it.

Your app works the same way:

At the stadium	In your app
The score sheet	The <code>tasks</code> array: the truth
The scoreboard	The list on the page
Writing a goal on the sheet	Changing the array: adding, ticking, deleting
The board updating from the sheet	A <code>render()</code> function that redraws the list from the array

The data your app keeps track of is called its **state**. Every feature you build will follow the same three steps:

1. Something happens: a click, or a form being sent.
2. You change the state (the `tasks` array).
3. You call `render()`, which redraws the page from the state.

You never add, cross out or remove tasks on the page directly. You change the array and redraw. That way, the page and the data can never disagree. Big tools like React are built on this same idea, so you're learning something that lasts.

Milestone 1: Tasks in an array, drawn on the page

Goal: keep the tasks in an array of objects, and write a `render()` function that draws them on the page.

Start `app.js` with three sample tasks, so there's something to draw:

```
let tasks = [
  { id: 1, text: "Buy milk", completed: true },
  { id: 2, text: "Book the dentist", completed: false },
  { id: 3, text: "Call grandma", completed: false },
];
```

Each task has:

- `id`: a number that's different for every task, like a ticket number. It lets you find the right task later.
- `text`: what needs doing.
- `completed: true` once it's done.

`tasks` uses `let` because later milestones will replace the whole array with a new one.

Next, write `render()`. It empties the `<ul>`, then builds one `<li>` for each task and adds it to the list. Each `<li>` must look like this, because that's what `style.css` expects:

```
<li class="todo-item completed" data-id="1">
  <input class="todo-toggle" type="checkbox" aria-label="Mark as done" checked>
  <span class="todo-text">Buy milk</span>
  <button class="todo-delete" type="button" aria-label="Delete task">×</button>
</li>
```

- The `completed` class and `checked` are only there when the task is done. The CSS crosses out the text of every `.completed` item.
- `data-id` stores the task's id on the element ([chapter 20](#)). In milestones 3 and 4, it tells you *which* task was clicked.
- `aria-label` gives the tick box and the × button a name that screen readers read out, because they have no visible label of their own. Attributes that start with `aria-` are there to help with accessibility ([chapter 22](#)).

Emptying the list first matters. Without it, every call to `render()` adds another copy of every task under the old ones. Setting `textContent` to an empty string removes everything inside an element:

```
const todoList = document.querySelector("#todo-list");
todoList.textContent = ""; // remove everything inside the <ul>
```

Finally, call `render()` once at the bottom of `app.js`, so the list is drawn when the page opens.

What you should see: three tasks, with "Buy milk" ticked and crossed out. Change `completed` for one of the tasks in your array, refresh, and the page follows along.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Watch out: Use `textContent` for the task's text, never `innerHTML`. Users can type anything into the box, including HTML. `innerHTML` would turn it into real elements, and some HTML can even run someone else's code ([chapter 20](#) explains the danger).

Milestone 2: Add a task

Goal: type a task, press Enter (or click "Add"), and it appears at the bottom of the list.

In your `submit` handler ([chapter 22](#)):

1. Stop the page from reloading.
2. Read the text box, and trim the spaces off.
3. If the text is empty, stop there, so an empty task can't sneak in.
4. Otherwise, add a new task object to `tasks`, and call `render()`.
5. Empty the text box, ready for the next task.

The new idea is the id. Every task needs a different one. A quick way to get one is the current time in milliseconds, with `Date.now()` from [chapter 19](#):

```
const newTask = { id: Date.now(), text, completed: false };
```

(Writing just `text` is the shorthand for `text: text`, from chapter 11.) Nobody can add two tasks in the same millisecond by hand, so each task gets its own number. Big apps often use long random ids instead, but this is plenty for now.

What you should see: type `Water the plants` and press Enter. It appears at the bottom, unticked, and the text box is empty again. Click "Add" with an empty box, or with only spaces in it, and nothing happens.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 3: Mark a task as done

Goal: tick a task's box, and it's crossed out. Untick it, and it's back to normal.

`render()` throws away the old `<li>`s and builds new ones every time. So a listener added to a single tick box would be lost at the next render. That's the problem event delegation solves ([chapter 21](#)): **one** listener on `#todo-list` hears every tick box inside it, even the ones that don't exist yet.

Which event? A tick box fires `change` whenever it's ticked or unticked, and `change` bubbles up to the list.

In your listener:

1. Check that the event came from a tick box, with `event.target.classList.contains("todo-toggle")`. If it didn't, stop.
2. Work out which task it was. `event.target.closest("li")` is the task's `<li>`, and its `dataset.id` is the task's id.
3. Find that task in the array, and set its `completed` to whether the box is ticked now:
`event.target.checked`.
4. Call `render()`.

One trap: `dataset` values are always strings (chapter 20), but your ids are numbers, and `"2" === 2` is `false`. So convert the id first:

```
const id = Number(event.target.closest("li").dataset.id);
```

What you should see: tick "Book the dentist", and it's crossed out. Untick it, and it's back. (If you refresh the page, the list goes back to the three samples, because nothing is saved yet. Milestone 7 fixes that.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Milestone 4: Delete a task

Goal: click a task's ×, and the task is gone.

Use delegation on the same list again, but this time with the `click` event, and check for the `todo-delete` class. Then remove the task from the array. The easiest way is `filter`: keep every task *except* the one with this id.

```
tasks = tasks.filter((task) => task.id !== id);
```

`filter` builds a brand-new array, and you replace the old one with it. That's why `tasks` is a `let`.

What you should see: click × next to "Call grandma", and it disappears. Delete every task, and "No tasks here." appears.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 5: How many tasks are left?

Goal: the footer shows how many tasks aren't done yet: "2 tasks left", "1 task left", "0 tasks left".

Count the tasks that aren't completed, and put the result in `#todo-count`. Do it inside `render()`. Every change already calls `render()`, so the counter can never fall out of date.

Mind the grammar: it's "1 task", but "0 tasks" and "2 tasks". A ternary ([chapter 07](#)) can pick the right word.

What you should see: with the three samples, the footer says "2 tasks left". Tick "Book the dentist": "1 task left". Tick "Call grandma": "0 tasks left". Add a new task: "1 task left".

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 6: Filters

Goal: the All, Active and Completed buttons show only the matching tasks, and the chosen button is highlighted.

Which filter is switched on is a new piece of state. It isn't a task, so it gets its own variable:

```
let currentFilter = "all"; // "all", "active" or "completed"
```

Then three steps:

1. **Listen.** One `click` listener on `#filters` (delegation again). Read the clicked button's `data-filter`, store it in `currentFilter`, and call `render()`.
2. **Show only the matching tasks.** Write a function `getVisibleTasks()` that returns all the tasks, only the active ones, or only the completed ones, depending on `currentFilter`. In `render()`, loop over `getVisibleTasks()` instead of `tasks`.
3. **Highlight the chosen button.** The CSS styles any `.filter-button` that also has the class `active`. In `render()`, go through the three buttons: add `active` to the one whose `data-filter` matches `currentFilter`, and remove it from the others.

Keep the counter counting *all* the tasks that aren't done, not just the visible ones. "2 tasks left" shouldn't change just because you're looking at the Completed list.

What you should see: click "Active". "Buy milk" disappears, and "Active" is highlighted. Click "Completed", and only "Buy milk" is left. Go back to "Active" and tick a task: it vanishes from view, because it isn't active any more. Click "All", and everything's back.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 7: Remember the tasks

Goal: the tasks survive a refresh, and even closing the browser.

Bring in the `save` and `load` helpers from [chapter 23](#), and add a name for your storage key at the top of `app.js`:

```
const STORAGE_KEY = "todo-app-tasks";
```


Then:

1. Replace the three sample tasks with `load`, and use an empty array as the default: `let tasks = load(STORAGE_KEY, []);`.
2. Save the tasks after **every** change to them: adding, ticking and deleting (and clearing, in milestone 8).

Each of those changes already ends with `render()`. A small helper keeps saving and drawing together, so you can't forget one of them:

```
function update() {  
  save(STORAGE_KEY, tasks);  
  render();  
}
```

Call `update()` instead of `render()` wherever the tasks change. Switching filters doesn't change any tasks, so the filter listener can keep calling `render()`.

What you should see: the samples are gone, and the list starts empty. Add "Pay the electricity bill", then add "Buy milk" and tick it. Refresh: both tasks are still there, and "Buy milk" is still ticked. Close the browser and open the page again: still there.

Now open DevTools, go to the **Application** tab, and open **Local storage**. The key `todo-app-tasks` holds your tasks as JSON, something like this (your ids will be different, because they come from `Date.now()`):

```
[{"id":1760000000000,"text":"Pay the electricity bill","completed":false},  
{"id":1760000004213,"text":"Buy milk","completed":true}]
```

Finally, try to break it. In the Console, run `localStorage.setItem("todo-app-tasks", "oops")` and refresh. The app starts with an empty list instead of crashing, and a red message in the Console explains why. That's `load` doing its job.

Stuck? The hint is in the [Hints section](#) at the end.

Milestone 8: Clear completed

Goal: "Clear completed" removes every finished task at once. When there's nothing to clear, the button is greyed out.

1. Add a `click` listener to `#clear-completed`. Keep only the tasks that aren't completed (with `filter` again), then call `update()`.
2. In `render()`, disable the button when no task is completed. You met the `disabled` property in chapter 20, and the CSS already fades out a disabled button.

What you should see: tick two tasks and click "Clear completed". Both disappear, the others stay, and the counter doesn't change. Refresh: they're still gone. With no ticked tasks, the button is greyed out, and clicking it does nothing.

Stuck? The hint is in the [Hints section](#) at the end.

Test it like a user

Before you call it finished, try your app the way a real user would. Tick each box once it works:

- ☐ Enter and the Add button both add a task. Empty tasks, and tasks made of spaces, are ignored.
- ☐ Ticking crosses a task out, and unticking brings it back.
- ☐ × always deletes the right task, even after you've added and deleted a few.
- ☐ The counter says "1 task left", not "1 tasks left".
- ☐ Each filter shows the right tasks, and the right button is highlighted.
- ☐ On the Active list, ticking a task makes it disappear from view.
- ☐ Everything survives a refresh, including ticks and deletions.
- ☐ A task typed as `<b>hello</b>` shows up exactly like that, angle brackets and all, instead of in bold.
- ☐ "Clear completed" is greyed out when there's nothing to clear.
- ☐ The app works on a phone-sized screen. In DevTools, click the phone-and-tablet icon (the **device toolbar**) to try different screen sizes.

When every box is ticked, you've built a real web app. 🎉

Common mistakes

1. Forgetting to empty the list in `render()`

```
function render() {  
  for (const task of tasks) {  
    todoList.append(createTaskElement(task));  
  }  
}
```

The first render looks fine. But every render after that adds the whole list again under the old one: add one task to the three samples, and you'll see seven. Fix: start `render()` with `todoList.textContent = ""`;

2. Comparing a `data-id` string with a number

```
const id = event.target.closest("li").dataset.id; // "2": a string!
const task = tasks.find((item) => item.id === id); // 2 === "2" is false, so: undefined
task.completed = event.target.checked;
// TypeError: Cannot set properties of undefined (setting 'completed')
```

`find` found nothing, because no id is equal to the string `"2"`. Fix:

```
Number(event.target.closest("li").dataset.id).
```

3. Adding listeners to the tick boxes when the page loads

```
for (const checkbox of document.querySelectorAll(".todo-toggle")) {
  checkbox.addEventListener("change", toggleTask);
}
```

Only the boxes that exist at that moment get a listener. The next `render()` replaces them with brand-new boxes that have no listeners, so ticking stops working. Fix: one listener on `#todo-list`, with event delegation.

4. Changing the page instead of the data

```
todoList.addEventListener("change", (event) => {
  event.target.closest("li").classList.toggle("completed"); // only the page changes
});
```

It seems to work, until the next render (when you add a task, say). Then the crossing-out vanishes, because the array still says `completed: false`. And it's never saved. Fix: change the task in the array, then save and render.

5. Forgetting to save after one kind of change

```
todoList.addEventListener("click", (event) => {
  if (!event.target.classList.contains("todo-delete")) {
    return;
  }
  const id = Number(event.target.closest("li").dataset.id);
  tasks = tasks.filter((task) => task.id !== id);
  render(); // the page is updated, but nothing is saved
});
```

Delete a task, refresh, and it's back from the dead. Fix: save after every change to `tasks`. Calling `update()` everywhere makes that hard to forget.

Quick recap

What you practiced in this project:

- Keep your data (the state) in one place, and draw the page from it with a `render()` function.
 - Every feature follows the same loop: an event happens, you change the array, then you save and render.
 - Event delegation handles elements that `render()` creates later. `closest("li").dataset.id` tells you which task was clicked, and `Number()` turns the id back into a number.
 - `push`, `find`, `filter` and `some` do the real work: adding, ticking, deleting, filtering and counting.
 - `save` and `load` (JSON plus `localStorage`) make the app remember, and keep it safe from broken data.
 - `textContent` keeps whatever users type safe.
-

Next: try the stretch goals in the [exercises](#), then move on to [25 Closures](#).

Exercises

These are **stretch goals**: extra features for your finished to-do app. Do them in any order, and do as many as you like.

How to do these:

- Finish all 8 milestones first. Then make a copy of your `my-todo-app` folder, so you always have a working version to go back to.
 - `style.css` already has the styles these goals need. Look for "Extra styles for the stretch goals" at the bottom.
 - Same routine as before: save, refresh, and keep the Console open (`F12`) to spot errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Dark mode

Planning tomorrow's tasks late at night is hard on the eyes. Add a button that switches the app between light and dark, and remember the choice.

Add this button to `index.html`, as the first thing inside `<body>` (just before `<main>`):

```
<button id="theme-button" class="theme-button" type="button">Dark mode</button>
```

1. Clicking the button switches the class `dark` on and off on the `<body>` element. The CSS does the rest.
2. The button always says what clicking it will do: "Dark mode" while the app is light, "Light mode" while it's dark.
3. Save the choice under its own key, `"todo-app-theme"`, and apply it when the page loads.

What you should see: click "Dark mode". The app turns dark, and the button now says "Light mode". Refresh, and it's still dark. Click "Light mode", refresh, and it stays light.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Keyboard shortcuts

People who use the app a lot love keyboard shortcuts. Add two:

- Press `/` anywhere on the page to jump straight into the "New task" box, without typing a `/` in it.
- Press `Escape` while typing in the box to empty it and leave it.

What you should see: click on an empty part of the page, then press `/`. The cursor is now in the "New task" box, and the box is still empty. Type something and press `Escape`: the text disappears, and the cursor leaves the box. Typing a task like `Buy 1/2 dozen eggs` still works: the `/` shortcut must not get in the way while you're typing in the box.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Search box

With 50 tasks on the list, finding "dentist" takes a while. Add a search box that shows only the tasks containing what you type.

Add this right after the filter buttons' `<div>` in `index.html`:

```
<input id="search-input" class="search-input" type="search" placeholder="Search tasks" aria-label="Search tasks">
```

1. Keep the search text as a new piece of state, like `currentFilter`.
2. Every time the user types in the box, update that state and render.
3. Show only the tasks that match the filter **and** the search. The search ignores upper and lower case, and extra spaces around the search text.
4. The counter still counts all the tasks that aren't done.

What you should see: with the tasks "Buy milk", "Book the dentist" and "Call the dentist back", type `dentist`. Only the two dentist tasks are left. `DENTIST` finds the same two. Click "Active" while searching, and you see only the dentist tasks that aren't done. Empty the search box, and everything comes back.

Coming up: this redraws the list on every single keystroke. That's fine for a to-do list, but a search that asks a server for results would send far too many requests. [Chapter 34](#) teaches **debounce**, which waits until the user stops typing.

Stuck? [Hint 1](#) and [Hint 2](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): Due dates

Some tasks have deadlines. Let people pick a due date, show it next to the task, and highlight the tasks that are overdue.

Add a date box inside `.todo-form-row` in `index.html`, just before the Add button:

```
<input id="due-input" name="due" type="date" aria-label="Due date">
```

1. When a task is added, store the date box's value on the task as `due`. It's a string like `"2026-10-05"`, the same ISO format that [chapter 19](#) recommends for storing dates. If no date was picked, store `null`.
2. Show the date after the task's text, in a `<span class="todo-due">`, like `Due Mon, Oct 5`. Use `toLocaleDateString("en-US", { weekday: "short", month: "short", day: "numeric" })`.
3. Give the `<li>` the class `overdue` when its due date is before today and the task isn't done. The CSS turns the date red.
4. Tasks you saved before this change have no `due` at all. They must still work, just without a date.

What you should see: add a task due on October 5, 2026, and it shows `Due Mon, Oct 5`. Which tasks are overdue depends on today's date, so give one a date in the past to see the red. Tick it, and the red goes away.

Watch out: this is the time zone trap from chapter 19. `new Date("2026-10-05")` means midnight in *UTC*. In New York, that moment is still the evening of October 4, so the date would show as `Due Sun, Oct 4`. Build the date from its parts instead, and it means midnight *here*: `new Date(2026, 9, 5)`.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Edit a task by double-clicking

Typos happen: "Buy mlik". Let people fix a task by double-clicking its text.

1. Double-clicking a task's text swaps the text for a text box (with the class `todo-edit`) that holds the current text, with the cursor already in it.

2. `Enter` saves the change, trimmed. If the new text is empty, the task is deleted instead.
3. `Escape` cancels, and the old text comes back.
4. Clicking somewhere else saves the change too.
5. Saved changes survive a refresh.

What you should see: double-click "Book the dentist", change it to `Book the dentist for Tuesday`, and press `Enter`. The task is updated, and it's still updated after a refresh. Double-click it again, delete all the text, and press `Enter`: the task is gone. Double-click another task, type something, and press `Escape`: nothing has changed.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Before you move on

Look at your event handlers. Each one uses `tasks`, `currentFilter` or `todoList`, variables that were created outside the handler, long before the click happened. Yet the handler can still use them, every time it runs.

How does a function "remember" the variables around it? That's called a **closure**, and it's the first topic of Level 3: [chapter 25](#).

Level 3: Advanced JavaScript

Goal: understand how JavaScript really works, write code that waits, and get data from the internet.

25 Closures

What is it?

A **closure** is a function that remembers the variables from the place where it was created, even after that place has finished running.

You've actually been using closures for a while. This chapter shows you what's going on, and how to use it on purpose.

Why does it matter?

Closures solve some very everyday problems:

- **Remembering things between calls.** A ticket machine needs to know the last number it gave out. Without closures, you'd need a global variable that any code could mess up.
- **Keeping data private.** A bank account's balance shouldn't be changeable by any line of code that feels like it. A closure can lock it away so only a few trusted functions can touch it.
- **Making custom functions.** Instead of writing `double`, `triple`, and `quadruple` by hand, you write one function that *makes* them.
- **Running something only once.** A "Pay" button must charge the card once, even if a nervous customer clicks it three times.

Closures are also the secret ingredient in many tools you'll build later, like `debounce` and `throttle` in [chapter 34](#).

Real-world example

Think of a function as a hiker, and the place where it's created as its home.

When a function is created inside another function, it packs a **backpack** with the variables from its home. Wherever it goes later, it can open the backpack and use them. The home can be locked up and gone, but the backpack stays with the hiker.

Closure idea	The backpack version
A function created inside another function	A hiker leaving home
The outer function's variables	The things packed in the backpack
Calling the function later, somewhere else	Opening the backpack on the trail
Nobody else can reach those variables	Only the hiker can open the backpack
Calling the outer function again	A new hiker with a new, separate backpack

Keep this picture in mind. Every example in this chapter is a function using its backpack.

How it works

Recap: functions can see outward

Remember scope from [chapter 14](#)? A function can see the variables around it (looking out through the window), but outside code can't see inside the function.

```
const shopName = "Corner Books";

function printWelcome() {
  const visitor = "Maya";
  console.log(`Welcome to ${shopName}, ${visitor}!`);
}

printWelcome(); // prints: Welcome to Corner Books, Maya!
console.log(visitor); // ReferenceError: visitor is not defined
```

This is called **lexical scope**. "Lexical" just means "based on where the code is written". What a function can see depends on where you *wrote* it, not on where you call it from.

Closures are this same rule, taken one step further.

A function that outlives its parent

Here's the surprising part. A function can create another function and **return** it, as a value. (Remember from [chapter 09](#) that functions are values, just like numbers and strings.)

```
function makeGreeter() {
  const greeting = "Good morning";

  function greet(name) {
    return `${greeting}, ${name}!`;
  }

  return greet; // hand back the function itself (no brackets, so it doesn't run yet)
}

const greetGuest = makeGreeter(); // makeGreeter runs and finishes here
console.log(greetGuest("Aisha")); // prints: Good morning, Aisha!
```

Read that slowly. By the time we call `greetGuest("Aisha")`, `makeGreeter` has already finished. Normally, a function's local variables are thrown away when it finishes.

But `greet` still needs `greeting`. So JavaScript keeps `greeting` alive, packed in `greet`'s backpack. That's a closure: the function `greet` plus the variables it remembers.

A counter that remembers

This is the classic closure example. Each call gives the next number, like a ticket machine at a bakery:

```
function createCounter() {
  let count = 0;

  return function () {
    count += 1;
    return count;
  };
}

const nextTicket = createCounter();
console.log(nextTicket()); // prints: 1
console.log(nextTicket()); // prints: 2
console.log(nextTicket()); // prints: 3
```

`createCounter` ran only once. Its `count` lives on in the backpack of the function it returned, so every call to `nextTicket()` updates the same `count`.

Every call makes a new backpack

Call `createCounter` twice and you get two separate counters, each with its own `count`:

```
const bakeryQueue = createCounter();
const pharmacyQueue = createCounter();

console.log(bakeryQueue()); // prints: 1
console.log(bakeryQueue()); // prints: 2
console.log(pharmacyQueue()); // prints: 1
console.log(bakeryQueue()); // prints: 3
```

The pharmacy queue doesn't care how many bakery tickets were handed out. Two hikers, two backpacks.

Why not use a global variable?

You could write a counter with a global variable instead:

```
let count = 0;

function nextTicket() {
  count += 1;
  return count;
}

console.log(nextTicket()); // prints: 1
count = 500; // any code, anywhere, can change it (even by accident)
console.log(nextTicket()); // prints: 501
```

It works, but it has two problems:

- **Anyone can change `count`**. In a big program, some other line of code might reset it or change it by mistake.
- **You can only have one**. A second queue would need a second global variable and a second function.

The closure version fixes both. `count` is hidden inside the backpack, and you can make as many counters as you like.

Several functions can share one backpack

If an outer function creates *two* functions, they share the same variables. When one changes a variable, the other sees the change:

```

function createScoreboard() {
  let score = 0;

  function addPoints(points) {
    score += points;
  }

  function getScore() {
    return score;
  }

  return { addPoints, getScore }; // shorthand properties from chapter 11
}

const game = createScoreboard();
game.addPoints(10);
game.addPoints(5);
console.log(game.getScore()); // prints: 15

```

This shows something important: a closure remembers the **variable itself**, not a snapshot of its value. `getScore` doesn't hold a copy of `score` from when it was created (that would be `0`). It looks at the live variable each time you call it.

Private data: a bank account

In [chapter 18](#) you wrote `withdraw(balance, amount)`, where the balance was passed in from outside. Now let's lock the balance away so that nothing outside can change it directly:

```
function createAccount(startingBalance) {
  let balance = startingBalance; // private: only the three methods below can see it

  return {
    deposit(amount) {
      balance += amount;
    },
    withdraw(amount) {
      if (amount > balance) {
        throw new Error("Not enough money in the account");
      }
      balance -= amount;
    },
    getBalance() {
      return balance;
    },
  };
}
```

`deposit(amount) { ... }` is the short way to write a method. It means the same as `deposit: function (amount) { ... }`.

Now try to use it, and try to cheat:

```
const account = createAccount(100);
account.deposit(50);
account.withdraw(30);
console.log(account.getBalance()); // prints: 120

console.log(account.balance); // prints: undefined
account.balance = 1000000; // only adds a new, unrelated property
console.log(account.getBalance()); // prints: 120
```

`balance` isn't a property of the object. It's a variable in the backpack that the three methods share. The only way to change it is through `deposit` and `withdraw`, and `withdraw` checks the rules first:

```
account.withdraw(500); // Error: Not enough money in the account
```

Data that only some trusted code can reach is called **private**. It's like a bank: you can't walk into the vault and grab cash. You have to go to the counter, and the teller follows the rules.

Tip: Classes have their own way to make private data, with `#balance`. You'll see it in [chapter 27](#).

Function factories: functions that make functions

A **function factory** is a function that builds and returns a new function, set up the way you asked. Parameters are variables too, so they go in the backpack:

```
function makeMultiplier(factor) {  
  return (number) => number * factor;  
}  
  
const double = makeMultiplier(2);  
const triple = makeMultiplier(3);  
  
console.log(double(5)); // prints: 10  
console.log(triple(5)); // prints: 15
```

`double` remembers `factor` as `2`. `triple` remembers `factor` as `3`. One factory, many custom functions.

Here's a more realistic one. A shop has different discounts for students and staff:

```
function makeDiscount(percent) {  
  return (price) => price - (price * percent) / 100;  
}  
  
const studentPrice = makeDiscount(10);  
const staffPrice = makeDiscount(25);  
  
console.log(studentPrice(80)); // prints: 72  
console.log(staffPrice(80)); // prints: 60
```

Because `studentPrice` is a normal function, you can hand it straight to the array methods from [chapter 13](#):

```
const prices = [20, 50, 80];  
console.log(prices.map(studentPrice)); // prints: [ 18, 45, 72 ]
```

Factories work for text, too. Here's a greeting maker:

```
function makeGreeting(greeting) {  
  return (name) => `${greeting}, ${name}!`;  
}  
  
const sayHello = makeGreeting("Hello");  
const sayNamaste = makeGreeting("Namaste");  
  
console.log(sayHello("Tom")); // prints: Hello, Tom!  
console.log(sayNamaste("Sita")); // prints: Namaste, Sita!
```

A `once` helper: the "Pay" button

Online shops have a real problem: a customer clicks "Pay", nothing seems to happen for a second, so they click again. And again. You don't want to charge their card three times.

Let's write a helper that takes any function and returns a new version of it that only runs the first time:

```
function once(fn) {
  let hasRun = false;
  let result;

  return function (...args) {
    if (!hasRun) {
      hasRun = true;
      result = fn(...args);
    }
    return result;
  };
}
```

The backpack holds three things: `fn` (the real function), `hasRun` (have we run yet?), and `result` (what it returned the first time). `...args` collects whatever arguments you pass, and `fn(...args)` passes them on, using rest and spread from [chapter 15](#).

Now wrap a function with it:

```
function chargeCard(amount) {
  console.log(`Charging ${amount}...`);
  return "Payment complete";
}

const payOnce = once(chargeCard);

console.log(payOnce(25));
console.log(payOnce(25)); // a nervous double-click
console.log(payOnce(25));
```

You'll see:

```
Charging $25...
Payment complete
Payment complete
Payment complete
```

The card was charged once. The later calls just hand back the first result.

On a web page, you'd use it with a click listener from [chapter 21](#):

```
const payButton = document.querySelector("#pay-button");
payButton.addEventListener("click", once(() => chargeCard(25)));
```

However many times the button is clicked, `chargeCard` runs only once.

The classic `var` loop gotcha

This one has confused developers for years. A cinema app builds one "book this seat" function per seat, in a loop, and stores them in an array to use later:

```
const seatBookers = [];

for (var seat = 1; seat <= 3; seat++) {
  seatBookers.push(() => console.log(`Booking seat ${seat}`));
}

seatBookers[0](); // prints: Booking seat 4
seatBookers[1](); // prints: Booking seat 4
seatBookers[2](); // prints: Booking seat 4
```

Seat 4? There is no seat 4! Here's what happened:

1. `var` is function-scoped, not block-scoped ([chapter 14](#)). So there's only **one** `seat` variable for the whole loop.
2. All three arrow functions remember that same single variable. (Remember: closures remember the variable, not a snapshot of its value.)
3. The functions don't run inside the loop. They run later, after the loop has finished.
4. The loop stops when `seat <= 3` becomes false, which is when `seat` is `4`. So all three functions look in the backpack and find `4`.

The fix is one word. Change `var` to `let`:

```
const seatBookers = [];

for (let seat = 1; seat <= 3; seat++) {
  seatBookers.push(() => console.log(`Booking seat ${seat}`));
}

seatBookers[0](); // prints: Booking seat 1
seatBookers[1](); // prints: Booking seat 2
seatBookers[2](); // prints: Booking seat 3
```


`let` is block-scoped, and a `for` loop with `let` creates a **fresh `seat` variable for each round** of the loop. Each function packs its own `seat` in its backpack.

Loop	How many <code>seat</code> variables?	What the functions see later
<code>for (var seat = ...)</code>	One, shared by every round	The last value (<code>4</code>)
<code>for (let seat = ...)</code>	A new one for each round	Their own round's value (<code>1</code> , <code>2</code> , <code>3</code>)

This is one more reason to never use `var`. You'll meet this same gotcha again with timers in [chapter 30](#), where it's even more common.

Closures keep variables alive

As long as a function exists, its backpack exists too. The `count` inside `nextTicket` stays in memory for as long as `nextTicket` is around.

That's usually tiny and completely fine. But if a closure holds on to something big (like a huge array of data) and the function lives for the whole program, that memory can never be freed. You'll learn how to spot and fix these "memory leaks" in [chapter 41](#).

Where you'll see closures again

Strictly speaking, every function in JavaScript is a closure, because every function can use the variables around it. The word matters most when a function is used *later* or *somewhere else*, after its outer function has finished.

You'll lean on closures a lot from here on:

- **Event listeners** ([chapter 21](#)) that use variables from outside are closures.
- **Timers** ([chapter 30](#)) run a function later, and it still remembers its variables.
- **Debounce and throttle** ([chapter 34](#)) use a closure to remember the timer and the last call.
- **Memoization** ([chapter 43](#) and [chapter 48](#)) uses a closure to remember past results, so slow work isn't repeated.

Common mistakes

1. Returning the result instead of the function

```
function makeGreeter() {
  const greeting = "Good morning";

  function greet(name) {
    return `${greeting}, ${name}!`;
  }

  return greet(); // runs greet right now and returns its text
}

const greetGuest = makeGreeter();
greetGuest("Aisha"); // TypeError: greetGuest is not a function
```

The brackets in `return greet()` call the function straight away, so `greetGuest` ends up holding the string "Good morning, undefined!", not a function. Fix: `return greet;` with no brackets. You want to hand over the function, not run it.

2. Making a new counter every time

```
function countVisitor() {
  const counter = createCounter(); // a brand-new counter on every call!
  console.log(`Visitor number ${counter()}`);
}

countVisitor(); // prints: Visitor number 1
countVisitor(); // prints: Visitor number 1
countVisitor(); // prints: Visitor number 1
```

Every call to `createCounter()` makes a fresh backpack starting at `0`. Fix: create the counter **once**, outside, and reuse it: `const visitorCounter = createCounter();` above the function, then call `visitorCounter()` inside.

3. Expecting a snapshot of the value

```
let ticketPrice = 10;
const showPrice = () => console.log(`Ticket: ${ticketPrice}`);

ticketPrice = 12; // the price changes after showPrice was created
showPrice(); // prints: Ticket: $12
```

A closure remembers the variable, not the value it had when the function was made. Usually that's exactly what you want (the current price). If you really need to lock in the old value, pass it to a factory as a parameter. The parameter is its own variable, so it keeps the value it was given:

```
function makePriceTag(price) {
  return () => console.log(`Ticket: ${price}`);
}

let ticketPrice = 10;
const showOldPrice = makePriceTag(ticketPrice); // price is its own variable: 10
ticketPrice = 12;
showOldPrice(); // prints: Ticket: $10
```

4. Handing out your private array

```
function createGuestList() {
  const guests = [];

  return {
    add(name) {
      guests.push(name);
    },
    getGuests() {
      return guests; // hands out the real array!
    },
  };
}

const party = createGuestList();
party.add("Ana");
party.getGuests().push("Gatecrasher"); // changes the private array
console.log(party.getGuests()); // prints: [ 'Ana', 'Gatecrasher' ]
```

Arrays are shared by reference ([chapter 16](#)), so returning `guests` hands out the key to the vault. Fix: return a copy with `return [...guests];`. Then outside code can change its copy all it likes, and the real list stays safe.

5. Using `var` in a loop that creates functions

If you create functions inside a `for (var ...)` loop, they all share one loop variable and see its last value, like the seat 4 example above. Fix: use `let`.

Quick recap

- A **closure** is a function plus the variables it remembers from where it was created: its backpack.
- Those variables stay alive even after the outer function has finished.
- Each call to the outer function makes a new, separate backpack.
- Use closures for private data (a bank balance), function factories (`makeMultiplier`), and helpers like `once`.

- Closures remember the variable itself, not a snapshot. That's why a `var` loop shares one variable. Use `let`.
-

Next: try the [exercises](#), then move on to [26 The `this` Keyword](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js`.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Ticket machine

A supermarket has a ticket machine at the deli counter and another at the bakery. Each one gives out numbered tickets with its own letter in front.

Write a function `createTicketMachine(prefix)` that returns a function. Each time you call the returned function, it gives the next ticket for that machine.

Test it with this code:

```
const deli = createTicketMachine("D");
const bakery = createTicketMachine("B");

console.log(deli());
console.log(deli());
console.log(bakery());
console.log(deli());
console.log(bakery());
```

Expected output:

D1
D2
B1
D3
B2

Rule: no global counter variables. Each machine must keep its own count.

Stuck? The hint is in the [Hints section](#) at the end.

Bonus: make the tickets look like `D-001`, `D-002`, and so on. `padStart` from [chapter 06](#) will help.

Exercise 2 (Easy): Price tag makers

An online shop sells to customers in different countries. Each country needs prices shown with its own currency symbol and always two decimal places.

Write a function factory `makePriceFormatter(symbol)`. It returns a function that takes a number and returns the formatted price as a string.

Test it with this code:

```
const formatDollars = makePriceFormatter("$");
const formatEuros = makePriceFormatter("€");
const formatRupees = makePriceFormatter("Rs ");

console.log(formatDollars(4.5));
console.log(formatEuros(12));
console.log(formatRupees(1250));
console.log([3, 7.25, 10].map(formatDollars));
```

Expected output:

```
$4.50
€12.00
Rs 1250.00
[ '$3.00', '$7.25', '$10.00' ]
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Free trial

A photo-editing app lets free users try its best features a few times before asking them to upgrade.

Write a helper `limitUses(fn, maxUses)`. It returns a new function that:

- runs `fn` (and returns its result) for the first `maxUses` calls,
- after that, returns `"Trial over. Please upgrade."` without running `fn` at all.

Test it with this code:

```
function exportPhoto(fileName) {
  return `Exported ${fileName} in HD`;
}

function resizePhoto(fileName, width) {
  return `Resized ${fileName} to ${width}px`;
}

const trialExport = limitUses(exportPhoto, 3);
const trialResize = limitUses(resizePhoto, 1);

console.log(trialExport("beach.jpg"));
console.log(trialExport("mountain.jpg"));
console.log(trialResize("cat.jpg", 800));
console.log(trialExport("family.jpg"));
console.log(trialExport("sunset.jpg"));
console.log(trialResize("dog.jpg", 1200));
```

Expected output:

```
Exported beach.jpg in HD
Exported mountain.jpg in HD
Resized cat.jpg to 800px
Exported family.jpg in HD
Trial over. Please upgrade.
Trial over. Please upgrade.
```

Rule: don't change `exportPhoto` or `resizePhoto`. All the counting must live inside `limitUses`, and it must work for functions with any number of arguments.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The food truck bug

A food truck's app builds one "order" button function per dish. Copy this code into `ex4.js` and run it:

```
const dishes = ["Momo", "Chowmein", "Thukpa"];
const orderButtons = [];

for (var i = 0; i < dishes.length; i++) {
  orderButtons.push(() => console.log(`You ordered: ${dishes[i]}`));
}

orderButtons[0]();
orderButtons[1]();
orderButtons[2]();
```

Instead of the dishes, it prints `You ordered: undefined` three times.

Your tasks:

1. Above the loop, write a comment explaining in your own words why every button prints `undefined`.
2. Fix it by changing just one word.
3. Now undo your fix (put `var` back) and fix it a second way: write a small function factory `makeOrderButton(dish)` and use it inside the loop.

Both fixes should print:

```
You ordered: Momo
You ordered: Chowmein
You ordered: Thukpa
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A private playlist

You're building a music app. Each playlist keeps its songs private, so the rest of the app can only change them through the playlist's own methods.

Write `createPlaylist(name)`. It returns an object with these methods:

- `addSong(title, seconds)` adds a song. If `seconds` isn't a positive number, throw an error with the message `Song length must be a positive number`.
- `removeSong(title)` removes the song with that title.

- `getSongs()` returns the songs as an array of `{ title, seconds }` objects. Changing that array from outside must **not** change the playlist.
- `getTotalTime()` returns the total length as `minutes:seconds`, like `7:10` or `0:00`.
- `describe()` returns a line like `Road Trip: 2 songs, 7:10`.

Test it with this code:

```
const roadTrip = createPlaylist("Road Trip");

roadTrip.addSong("Resham Firiri", 245);
roadTrip.addSong("Bohemian Rhapsody", 354);
roadTrip.addSong("Here Comes the Sun", 185);
console.log(roadTrip.describe());

roadTrip.removeSong("Bohemian Rhapsody");
console.log(roadTrip.describe());

// Trying to cheat from the outside:
roadTrip.getSongs().push({ title: "Sneaky Song", seconds: 999 });
roadTrip.songs = [];
console.log(roadTrip.getSongs());

const study = createPlaylist("Study");
console.log(study.describe());

try {
  study.addSong("Silence", -5);
} catch (error) {
  console.log(`Error: ${error.message}`);
}
```

Expected output:

```
Road Trip: 3 songs, 13:04
Road Trip: 2 songs, 7:10
[
  { title: 'Resham Firiri', seconds: 245 },
  { title: 'Here Comes the Sun', seconds: 185 }
]
Study: 0 songs, 0:00
Error: Song length must be a positive number
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Try this in a new file:

```
const wallet = {  
  balance: 50,  
  getBalance() {  
    return this.balance;  
  },  
};  
  
console.log(wallet.getBalance()); // prints: 50  
  
const checkBalance = wallet.getBalance;  
console.log(checkBalance()); // prints: undefined
```

It's the same function both times. So why does it forget the balance?

A closure remembers *where* a function was created. But `this` works completely differently: it depends on *how* the function is called. [Chapter 26](#) solves the mystery. 🕵️

26 The `this` Keyword

What is it?

`this` is a special word you can use inside a function. It points to an object, usually "the object this function was called on".

The key idea: what `this` points to is decided **each time the function is called**, not when the function is written.

Why does it matter?

You met `this` in [chapter 11](#), where a method used `this` to reach its own object's data. That's its main job, and it lets one function work for many different objects.

But `this` is also one of the most common sources of confusing bugs in JavaScript:

- You pass a method to another function (as a callback), and suddenly it can't find its data.
- You click a button, and `this` points to something you didn't expect.
- A value comes out as `undefined`, with no error at all.

Remember the mystery at the end of the [chapter 25 exercises](#)? The same function gave `50` one time and `undefined` the next. By the end of this chapter, you'll know exactly why, and how to fix it. You'll also need `this` for classes in [chapter 27](#).

Real-world example

Think about the word "**me**".

When Ana says "Pass the salt to me", *me* means Ana. When Ben says the exact same sentence, *me* means Ben. The word never changes. Who it means depends on **who is saying it**.

Now imagine you find a note on the floor that says "Call me". Who is *me*? Nobody knows, because nobody is saying it.

Situation	Who "me" means	In JavaScript
Ana says "Pass it to me"	Ana	<code>ana.greet() : this is ana</code>
Ben says "Pass it to me"	Ben	<code>ben.greet() : this is ben</code>
A note on the floor says "Call me"	Nobody knows	<code>greet()</code> on its own: <code>this</code> is lost

`this` works like "me". It means "the object that is calling me right now".

How it works

In a method: the object before the dot

Here's a method using `this`, just like in chapter 11:

```
const gym = {
  gymName: "Iron Temple",
  members: 120,
  describe() {
    return `${this.gymName} has ${this.members} members`;
  },
};

console.log(gym.describe()); // prints: Iron Temple has 120 members
```

When you call `gym.describe()`, `this` is `gym`, the object before the dot.

Why not just write `gym.gymName` inside the method? Because with `this`, the same function can be shared by many objects:

```
function describe() {
  return `${this.gymName} has ${this.members} members`;
}

const ironTemple = { gymName: "Iron Temple", members: 120, describe };
const fitHub = { gymName: "FitHub", members: 85, describe };

console.log(ironTemple.describe()); // prints: Iron Temple has 120 members
console.log(fitHub.describe()); // prints: FitHub has 85 members
```

It's the same `describe` function both times. When `ironTemple` calls it, `this` is `ironTemple`. When `fitHub` calls it, `this` is `fitHub`. That's Ana and Ben saying "me".

Tip: Here's the most useful rule in this chapter. To find `this`, look at the **call**, not the function. What's to the left of the dot? That's `this`.

Losing `this`: a method on its own

Here's the mystery from chapter 25:

```
const wallet = {
  balance: 50,
  getBalance() {
    return this.balance;
  },
};

console.log(wallet.getBalance()); // prints: 50

const checkBalance = wallet.getBalance; // copies the function, not the object
console.log(checkBalance()); // prints: undefined
```

`checkBalance` is the same function as `wallet.getBalance`. But look at the call: `checkBalance()`. There's nothing before the dot. There isn't even a dot!

The function has been **detached** from its object. It's the note on the floor that says "Call me". So `this` is no longer `wallet`, and `this.balance` doesn't find the balance.

So what is `this` in a plain call like `checkBalance()`? That depends on a setting called strict mode.

Plain function calls

In the `.js` files you've been running so far, a plain function call gets `this` set to `globalThis`:

```
function whoAmI() {
  return this === globalThis;
}

console.log(whoAmI()); // prints: true
```

`globalThis` is the global object: one big built-in object that holds global things like `console`. In the browser it's the same object as `window`.

So in the wallet example, `this.balance` really meant `globalThis.balance`. There's no global called `balance`, so you got `undefined`.

That can do real damage. Here, a detached method quietly changes the wrong object:

```

const scoreboard = {
  points: 0,
  addPoint() {
    this.points += 1;
  },
};

const addPoint = scoreboard.addPoint;
addPoint(); // plain call: this is globalThis

console.log(scoreboard.points); // prints: 0
console.log(globalThis.points); // prints: NaN

```

The scoreboard didn't change at all. Instead, JavaScript made a brand-new global called `points` and set it to `NaN` (`undefined + 1`). No error, no warning. This is the same kind of silent accident as the accidental globals in [chapter 14](#).

Strict mode: make mistakes loud

Strict mode is a stricter version of JavaScript that turns some silent mistakes into real errors. The normal, relaxed mode is often called **sloppy mode**.

To turn strict mode on, put this line at the very top of your file:

```

"use strict";

const wallet = {
  balance: 50,
  getBalance() {
    return this.balance;
  },
};

const checkBalance = wallet.getBalance;
checkBalance(); // TypeError: Cannot read properties of undefined (reading 'balance')

```

In strict mode, a plain function call gets `this` set to `undefined`. So `this.balance` crashes with a clear error that points at the exact line, instead of quietly giving you `undefined`. That's a good thing: a loud bug is much easier to find than a silent one.

Strict mode also stops the accidental globals from chapter 14:

```
"use strict";

total = 5; // ReferenceError: total is not defined
```

Here's the difference:

	Sloppy mode	Strict mode
<code>this</code> in a plain call like <code>fn()</code>	<code>globalThis</code>	<code>undefined</code>
Assigning to a variable you never declared	Quietly creates a global	<code>ReferenceError</code>

A few things to know about `"use strict"`:

- It must be the **first statement** in the file. Comments above it are fine, but if any code comes first, it's just an ordinary string and does nothing.
- You can also put it as the first line inside a single function, to make only that function strict.
- **Modules and classes are always strict, automatically.** You'll meet classes in [chapter 27](#) and modules in [chapter 29](#). Most modern code is written in one or both, so in real projects, a lost `this` is usually `undefined`.

The plain `.js` files in this course so far have all run in sloppy mode.

Losing `this` inside a callback

The most common way to lose `this` is inside a callback, like the ones you pass to `forEach` in [chapter 13](#):

```
const playlist = {
  title: "Chill Vibes",
  songs: ["Sunrise", "Ocean Drive"],
  printSongs() {
    this.songs.forEach(function (song) {
      console.log(`${song} (from ${this.title})`);
    });
  },
};

playlist.printSongs();
```

You'll see:

```
Sunrise (from undefined)
Ocean Drive (from undefined)
```

`this.songs` on the first line of the method works fine, because `printSongs` was called as `playlist.printSongs()`. But the callback is a separate function, and `forEach` calls it as a plain call, with no object before the dot. So inside the callback, `this` is lost (and in strict mode, this code would crash).

Arrow functions don't have their own `this`

Arrow functions solve exactly this problem. An arrow function doesn't get a `this` of its own. It uses the `this` from the place where it was **written**, the same way a closure uses a variable from outside ([chapter 25](#)).

Change the callback to an arrow function:

```
const playlist = {
  title: "Chill Vibes",
  songs: ["Sunrise", "Ocean Drive"],
  printSongs() {
    this.songs.forEach((song) => {
      console.log(`${song} (from ${this.title})`);
    });
  },
};

playlist.printSongs();
```

You'll see:

```
Sunrise (from Chill Vibes)
Ocean Drive (from Chill Vibes)
```

The arrow function was written inside `printSongs`, where `this` is `playlist`. So inside the arrow, `this` is `playlist` too. Arrow functions are perfect for callbacks inside methods.

But that same feature makes them **bad as methods**:

```
const cinema = {
  cinemaName: "Galaxy Cinema",
  greet: () => `Welcome to ${this.cinemaName}!`,
};

console.log(cinema.greet()); // prints: Welcome to undefined!
```

The arrow function doesn't care that it's called as `cinema.greet()`. It ignores the dot and uses the `this` from outside the object, and that's not `cinema`. (At the top of a plain Node file, `this` is an empty object, so you get `undefined`. In a module, the same code would crash instead.)

The rule of thumb:

- **Methods:** use the normal method shorthand, `greet() { ... }`.
- **Callbacks inside methods:** use arrow functions, so they share the method's `this`.

`call`, `apply` and `bind`: choosing `this` yourself

Sometimes you want to decide what `this` is, instead of letting the call decide. Every function has three built-in methods for that.

- `call` runs the function right now, with `this` set to the first thing you pass. Any other arguments follow, separated by commas.
- `apply` does the same, but takes the other arguments as an array.
- `bind` doesn't run anything. It returns a **new function** with `this` locked in, to call later.

```
function introduce(greeting, punctuation) {  
  return `${greeting}, I'm ${this.firstName}${punctuation}`;  
}  
  
const ana = { firstName: "Ana" };  
const ben = { firstName: "Ben" };  
  
console.log(introduce.call(ana, "Hi", "!")); // prints: Hi, I'm Ana!  
console.log(introduce.apply(ben, ["Hello", "."])); // prints: Hello, I'm Ben.  
  
const anaSays = introduce.bind(ana); // doesn't run yet: makes a new function  
console.log(anaSays("Hey", "!!")); // prints: Hey, I'm Ana!!
```

A memory trick: `call` takes **commas**, `apply` takes an **array**.

Borrowing a method. `call` lets one object use another object's method, as long as it has the properties the method needs:


```
const hotel = {
  hotelName: "Lakeside Inn",
  pricePerNight: 80,
  quote(nights) {
    return `${nights} nights at ${this.hotelName}: ${nights * this.pricePerNight}`;
  },
};

const hostel = { hotelName: "Backpackers Den", pricePerNight: 25 };

console.log(hotel.quote(3)); // prints: 3 nights at Lakeside Inn: $240
console.log(hotel.quote.call(hostel, 3)); // prints: 3 nights at Backpackers Den: $75
```

The hostel doesn't need its own copy of `quote`. It borrows the hotel's, with `this` set to `hostel`.

Fixing a detached method. `bind` is the classic fix for a lost `this`. You can also wrap the call in an arrow function, which keeps the dot:

```
const checkBalance = wallet.getBalance.bind(wallet); // this is locked to wallet
console.log(checkBalance()); // prints: 50

const checkAgain = () => wallet.getBalance(); // or: wrap the call in an arrow
console.log(checkAgain()); // prints: 50
```

This matters most when you hand a method to someone else to call later:

```
function runTwice(action) {
  action();
  action();
}

runTwice(scoreboard.addPoint); // loses this: scoreboard.points stays 0
runTwice(scoreboard.addPoint.bind(scoreboard)); // works
runTwice(() => scoreboard.addPoint()); // works too
```

	Runs the function now?	Other arguments	Good for
<code>fn.call(obj, a, b)</code>	Yes	Listed one by one	Borrowing a method once
<code>fn.apply(obj, [a, b])</code>	Yes	In an array	The same, when the arguments are already in an array

	Runs the function now?	Other arguments	Good for
<code>fn.bind(obj)</code>	No, it returns a new function	Given later, when you call it	Fixing <code>this</code> for later: callbacks and event listeners

Tip: Since spread arrived ([chapter 15](#)), `fn.call(obj, ...args)` does the same job as `apply`, so you'll see `apply` less often in new code. You'll meet it again when you build `debounce` and `throttle` in [chapter 34](#), which must keep `this` working.

`this` in DOM event listeners

When you use a normal function as an event listener ([chapter 21](#)), the browser calls it with `this` set to **the element the listener is attached to**:

```
const seats = document.querySelectorAll(".seat");

for (const seat of seats) {
  seat.addEventListener("click", function () {
    this.classList.toggle("selected"); // this = the seat button that was clicked
  });
}
```

An arrow function doesn't get its own `this`, so it can't do that. It uses the `this` from outside, which in a normal script is `window`, not the button:

```
seat.addEventListener("click", () => {
  this.classList.toggle("selected");
  // TypeError: Cannot read properties of undefined (reading 'toggle')
});
```

`window` has no `classList`, so `this.classList` is `undefined`, and the error appears in the DevTools console (as `Uncaught TypeError: ...`).

Many developers skip `this` in listeners and use `event.currentTarget` instead. It's always the element the listener is attached to, and it works with both kinds of functions:

```
seat.addEventListener("click", (event) => {
  event.currentTarget.classList.toggle("selected");
});
```

`event.currentTarget` is a close cousin of `event.target` from chapter 21. `target` is the exact element you clicked (which could be something small inside the button), while `currentTarget` is always the element that owns the listener.

The rules in one table

When you're not sure what `this` is, find the call and check this table:

How the function is called	Example	What <code>this</code> is
As a method, with a dot	<code>wallet.getBalance()</code>	The object before the dot (<code>wallet</code>)
As a plain call, no dot	<code>checkBalance()</code>	Sloppy mode: <code>globalThis</code> . Strict mode: <code>undefined</code>
With <code>call</code> or <code>apply</code>	<code>introduce.call(ana)</code>	The object you pass (<code>ana</code>)
A function made by <code>bind</code>	<code>introduce.bind(ana)</code>	The object you bound (<code>ana</code>), every time
An arrow function	<code>(song) => this.title</code>	No <code>this</code> of its own: it uses the <code>this</code> from where it was written
A DOM listener (normal <code>function</code>)	<code>seat.addEventListener("click", function () {})</code>	The element the listener is on (<code>seat</code>)
With <code>new</code>	<code>new Book()</code>	The brand-new object being built (chapter 27)

Common mistakes

1. Using an arrow function as a method

```
const bakery = {
  bakeryName: "Sweet Crumbs",
  describe: () => `This is ${this.bakeryName}`,
};

console.log(bakery.describe()); // prints: This is undefined
```

Arrow functions don't get their own `this`, so the dot in `bakery.describe()` makes no difference. Fix: use the method shorthand, `describe() { return ...; }`.

2. Passing a method as a callback

```
const basket = {
  items: [],
  addItem(item) {
    this.items.push(item);
  },
};

["apples", "bread"].forEach(basket.addItem);
// TypeError: Cannot read properties of undefined (reading 'push')
```

`forEach` receives the function on its own, detached from `basket`, and calls it as a plain call. Fix: keep the dot by wrapping it in an arrow, `forEach((item) => basket.addItem(item))`, or lock `this` with `basket.addItem.bind(basket)`.

3. Forgetting `this` inside a method

```
const library = {
  bookCount: 1200,
  describe() {
    return `We have ${bookCount} books`;
  },
};

console.log(library.describe()); // ReferenceError: bookCount is not defined
```

Inside a method, `bookCount` on its own means "a variable called `bookCount`", and there isn't one. The object's properties are only reachable through the object. Fix: `this.bookCount`.

4. Calling `bind` and throwing away the result

```
const checkBalance = wallet.getBalance;
checkBalance.bind(wallet); // makes a new function... and throws it away
console.log(checkBalance()); // prints: undefined
```

`bind` never changes the original function. It returns a new one. Fix: keep what it returns, `const checkBalance = wallet.getBalance.bind(wallet);`.

5. Putting "use strict" in the wrong place

```
const appName = "Recipe Box";  
"use strict"; // too late: code came first, so this does nothing  
  
servings = 4; // no error: an accidental global was created  
console.log(servings); // prints: 4
```

"use strict" only works as the very first statement of a file or function. Fix: move it to the top line.

Quick recap

- `this` is decided by **how a function is called**, not where it's written. Look at the call: the object before the dot is `this`.
- A method on its own (`const fn = obj.method; fn()`) loses its `this`. In sloppy mode it becomes `globalThis`. In strict mode, it's `undefined`.
- "use strict" makes these mistakes loud. Modules and classes are strict automatically.
- Arrow functions have no `this` of their own. They're great for callbacks inside methods, and bad as methods.
- `call` and `apply` run a function with the `this` you choose. `bind` returns a new function with `this` locked in.
- In a DOM listener, a normal `function` gets the element as `this`. With an arrow function, use `event.currentTarget` instead.

Next: try the [exercises](#), then move on to [27 Classes](#).

Exercises

How to do these:

- Exercises 1, 2, 3 and 5 run in Node. Make a file for each one in this folder (`ex1.js` , `ex2.js` , and so on) and run it with `node ex1.js`.
- Exercise 4 runs in the browser. Make a folder `ex4` with an `index.html` and a `script.js` inside. Open `index.html` by double-clicking it, press `F12` → **Console** to see errors, and refresh after each change.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Smart thermostats

A smart home app controls a thermostat in every room. All the thermostats should share the same three functions, instead of each room getting its own copy.

1. Write three normal functions that use `this` :
 - `warmer(degrees)` raises the temperature.
 - `cooler(degrees)` lowers it.
 - `describe()` returns a line like `Bedroom: 18°C` .
2. Create two objects, `livingRoom` (starts at 20) and `bedroom` (starts at 18). Each has a `room` name, a `temperature` , and the three functions.

Test it with this code:

```
console.log(livingRoom.describe());
console.log(bedroom.describe());

livingRoom.warmer(3);
bedroom.cooler(2);
bedroom.cooler(1);

console.log(livingRoom.describe());
console.log(bedroom.describe());
```

Expected output:

```
Living room: 20°C
Bedroom: 18°C
Living room: 23°C
Bedroom: 15°C
```

Rule: don't write `livingRoom` or `bedroom` inside the three functions.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Phone bills

A phone company has several plans. One function works out the bill for any plan, using `this` to read the plan's prices:

```
function monthlyBill(minutes, gigabytes) {
  const cost = this.baseFee + minutes * this.perMinute + gigabytes * this.perGigabyte;
  return `${this.planName}: ${cost.toFixed(2)}`;
}

const basic = { planName: "Basic", baseFee: 10, perMinute: 0.05, perGigabyte: 2 };
const unlimited = { planName: "Unlimited", baseFee: 45, perMinute: 0, perGigabyte: 0 };
const heavyUsage = [500, 20]; // 500 minutes, 20 GB
```

Copy this code, then print three bills:

1. Use `call` to print the Basic bill for 100 minutes and 3 GB.
2. Use `apply` to print the Unlimited bill for `heavyUsage`.
3. Use `bind` to make a new function called `basicBill`. Then call `basicBill(250, 1)` and print the result.

Expected output:

```
Basic: $21.00
Unlimited: $45.00
Basic: $24.50
```

Rule: don't change `monthlyBill` or the plan objects.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): The broken scoreboard button

A sports app has a big "+1" button. The button code receives a function and calls it every time the button is pressed. Here it's simulated with a function that "presses" three times. Copy this into `ex3.js` and run it:

```

const scoreboard = {
  team: "Tigers",
  points: 0,
  addPoint() {
    this.points += 1;
    console.log(`${this.team}: ${this.points}`);
  },
};

function pressButtonThreeTimes(onPress) {
  onPress();
  onPress();
  onPress();
}

pressButtonThreeTimes(scoreboard.addPoint);
console.log(`Final score: ${scoreboard.points}`);

```

It prints this, with no error at all:

```

undefined: NaN
undefined: NaN
undefined: NaN
Final score: 0

```

Your tasks:

1. Add `"use strict";` as the first line and run it again. Now you get a real error. Copy the error message into a comment, and explain in your own words why strict mode is better here.
2. Keep strict mode on. Fix the bug using `bind`.
3. Undo that fix and fix it a second way, with an arrow function.

Both fixes should print:

```

Tigers: 1
Tigers: 2
Tigers: 3
Final score: 3

```

Rule: don't change `scoreboard` or `pressButtonThreeTimes`. Only change the line that calls `pressButtonThreeTimes`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Cinema seat picker

A cinema website lets you click seats to select or unselect them. Create `ex4/index.html` with this code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Seat Picker</title>
    <style>
      .seat {
        width: 60px;
        height: 40px;
        margin: 4px;
        cursor: pointer;
      }
      .selected {
        background: seagreen;
        color: white;
      }
    </style>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Pick your seats</h1>
    <div>
      <button class="seat">A1</button>
      <button class="seat">A2</button>
      <button class="seat">A3</button>
      <button class="seat">B1</button>
      <button class="seat">B2</button>
      <button class="seat">B3</button>
    </div>
    <p id="summary">Selected seats: 0</p>
  </body>
</html>
```

In `ex4/script.js`:

1. Add a click listener to every seat, using a normal `function`. Inside it, use `this` to toggle the `selected` class on the clicked seat.
2. After each click, update the summary to show how many seats are selected.
3. Now change your listener to an arrow function, click a seat, and read the error in the console. Write it down in a comment, with a sentence explaining it.
4. Keep the arrow function, and fix it without using `this`.

What you should see: click A2 and B1, and both turn green while the summary says `Selected seats: 2`. Click A2 again, and it goes back to normal while the summary says `Selected seats: 1`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Pizza order builder

A pizza shop wants code that reads almost like a sentence. You'll build an `order` object whose methods can be **chained**, one after another:

```
order
  .setCustomer("Sam")
  .add("Margherita pizza", 12.5)
  .add("Garlic bread", 4, 2)
  .add("Lemonade", 2.75, 3)
  .applyDiscount(20)
  .printReceipt();
```

Here's the trick that makes chaining work: if a method ends with `return this;`, it hands back the object itself. So you can call the next method straight on the result.

Build `order` as an object literal with:

- `customer` (starts as `""`), `items` (starts as `[]`), and `discountPercent` (starts as `0`).
- `setCustomer(name)`, `add(name, price, quantity)` and `applyDiscount(percent)`. Each one updates the object and returns `this`. If no quantity is given, it's `1`.
- `formatLine(label, amount)`, which returns one receipt line: the label padded to 22 characters, then the amount with 2 decimals padded to 8 characters.
- `printReceipt()`, which prints the receipt below.

Expected output:

```
Order for Sam
Margherita pizza x1      12.50
Garlic bread x2          8.00
Lemonade x3              8.25
-----
Subtotal                  28.75
Discount (20%)           -5.75
Total                     23.00
```

Rule: inside `printReceipt`, loop over the items with `forEach` and an **arrow function**, and call `this.formatLine(...)` inside that arrow function.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: change `add` into an arrow function (`add: (name, price, quantity = 1) => { ... }`) and run it again. What error do you get, and why?

Before you move on

Your order object works great. But what if the shop gets 50 orders an hour? Copying the whole object literal 50 times would be painful, and fixing a bug would mean fixing it 50 times.

[Chapter 27](#) introduces **classes**: a blueprint you write once and use to stamp out as many objects as you like. And they use everything you just learned about `this`.

27 Classes

What is it?

A **class** is a blueprint for making objects. You describe once what every object of that kind has (its properties) and can do (its methods). Then you can make as many objects from it as you like.

Why does it matter?

Real programs need lots of similar objects: every book in a library, every player in a game, every order in a shop. Writing each one by hand as an object literal means copying the same methods again and again. And when you find a bug, you have to fix every copy.

You already know one fix: factory functions like `createAccount` from [chapter 25](#). Classes are JavaScript's built-in way to do the same job, and you'll see them everywhere: in libraries, in frameworks, and in JavaScript itself.

In fact, you've already used classes:

- `new Date()` in [chapter 19](#) made an object from the `Date` class.
- `new Error("...")` in [chapter 18](#) made an object from the `Error` class.

Now you'll write your own.

Real-world example

Think of an architect's blueprint for a house.

The blueprint isn't a house. You can't live in it. But a building crew can use it to build as many houses as you want, and each house is real and separate. Paint one house blue, and the others don't change.

Class idea	The house version
The class	The blueprint
<code>new House()</code>	Building a house from the blueprint
An instance (one object made from the class)	One real house
The constructor (the setup code)	The crew's setup steps: paint color, owner's name
Properties	Each house's own details: its color, its owner

Class idea	The house version
Methods	Things every house can do: open the garage, ring the doorbell

(A cookie cutter works too: one cutter, many cookies, and each one can get its own sprinkles.)

How it works

Your first class

```
class Book {
  constructor(title, author) {
    this.title = title;
    this.author = author;
  }

  describe() {
    return `${this.title} by ${this.author}`;
  }
}

const dune = new Book("Dune", "Frank Herbert");
const matilda = new Book("Matilda", "Roald Dahl");

console.log(dune.describe()); // prints: Dune by Frank Herbert
console.log(matilda.describe()); // prints: Matilda by Roald Dahl
console.log(dune); // prints: Book { title: 'Dune', author: 'Frank Herbert' }
```

Here's each part:

- `class Book { ... }` is the blueprint. Class names start with a capital letter, by convention. That's how you can tell `Book` is a class and `book` is a normal variable.
- `constructor(...)` is a special method that runs automatically every time you make a new object. Its job is to set the object up.
- `this` inside the constructor is the brand-new object being built. Remember the last row of the table in [chapter 26](#)? `this.title = title` gives the new object a `title` property.
- `describe()` is a method. Every book made from this class can use it. Inside a class, methods don't need the word `function`, and there are **no commas** between them.
- `new Book("Dune", "Frank Herbert")` builds a new object from the blueprint. The arguments go straight to the constructor.

When you print an instance, Node shows the class name in front: `Book { ... }`. That's a handy reminder of which blueprint it came from.

What `new` does

When you write `new Book("Dune", "Frank Herbert")`, three things happen:

1. JavaScript creates a new, empty object.
2. It runs the constructor, with `this` pointing to that new object.
3. It gives you back the finished object.

(There's one more hidden step, which [chapter 28](#) will reveal.)

Class fields: properties with a starting value

Some properties start the same for every new object. You can write them straight into the class body. These are called **class fields**:

```
class Player {
  score = 0; // every new player starts with these
  lives = 3;

  constructor(name) {
    this.name = name;
  }

  addPoints(points) {
    this.score += points;
  }

  loseLife() {
    this.lives -= 1;
  }
}

const mia = new Player("Mia");
const leo = new Player("Leo");

mia.addPoints(50);
mia.loseLife();

console.log(mia); // prints: Player { score: 50, lives: 2, name: 'Mia' }
console.log(leo); // prints: Player { score: 0, lives: 3, name: 'Leo' }
```

Each instance gets its own copy of every property. Mia's points don't touch Leo's. Two houses, two paint jobs.

Use a class field when the starting value is always the same. Use the constructor when the value comes from the arguments, like each player's `name`.

Getters and setters

Sometimes a property should be **worked out** from other properties, so it can never be out of date. Or you want some code to run whenever a property is changed. That's what getters and setters are for.

- A **getter** (`get`) is a method that runs when you *read* a property.
- A **setter** (`set`) is a method that runs when you *assign* to a property.

```
class Temperature {
  constructor(celsius) {
    this.celsius = celsius;
  }

  get fahrenheit() {
    return (this.celsius * 9) / 5 + 32;
  }

  set fahrenheit(value) {
    this.celsius = ((value - 32) * 5) / 9;
  }
}

const oven = new Temperature(180);
console.log(oven.fahrenheit); // prints: 356

oven.fahrenheit = 212; // runs the setter
console.log(oven.celsius); // prints: 100
```

Notice there are **no brackets**: you write `oven.fahrenheit`, not `oven.fahrenheit()`. From the outside, it looks like a normal property. Behind the scenes, it runs your code.

The object only really stores `celsius`. The Fahrenheit value is always calculated fresh from it, so the two can never disagree.

static : things that belong to the class itself

Most properties and methods belong to each instance: every book has its own title. But some things belong to the **class as a whole**. Mark those with `static`.

You've used static methods already: `Date.now()`, `Number.isInteger()` and `Array.isArray()` are all called on the class itself, not on one date or one number.

```

class Ticket {
  static nextNumber = 1; // one shared value, stored on the class itself
  static price = 12;

  constructor(movie) {
    this.movie = movie;
    this.number = Ticket.nextNumber;
    Ticket.nextNumber += 1;
  }

  static totalFor(count) {
    return count * Ticket.price;
  }
}

const first = new Ticket("Inception");
const second = new Ticket("Frozen");

console.log(first.number, second.number); // prints: 1 2
console.log(Ticket.totalFor(4)); // prints: 48
console.log(first.totalFor); // prints: undefined

```

- `Ticket.nextNumber` is one counter shared by the whole class. Each new ticket takes the next number.
- `Ticket.totalFor(4)` is a helper that doesn't need any particular ticket.
- Static things live on the class, not on the instances. That's why `first.totalFor` is `undefined`.

Private fields and methods

Remember the bank account from chapter 25, where a closure kept the balance safe? Classes have their own way to do that: start a field's name with `#`. A **private field** can only be used by code inside the class.


```

class BankAccount {
  #balance = 0; // private: only code inside this class can use it

  constructor(owner) {
    this.owner = owner;
  }

  deposit(amount) {
    this.#checkAmount(amount);
    this.#balance += amount;
  }

  withdraw(amount) {
    this.#checkAmount(amount);
    if (amount > this.#balance) {
      throw new Error("Not enough money in the account");
    }
    this.#balance -= amount;
  }

  get balance() {
    return this.#balance;
  }

  #checkAmount(amount) {
    if (amount <= 0) {
      throw new Error("Amount must be more than 0");
    }
  }
}

```

`#checkAmount` is a **private method**: a helper only the class itself can call. And `get balance()` is a getter with no setter, so outside code can read the balance but not change it.

```

const account = new BankAccount("Priya");
account.deposit(100);
account.withdraw(30);

console.log(account.balance); // prints: 70
console.log(account); // prints: BankAccount { owner: 'Priya' }

account.balance = 1000000; // ignored: balance only has a getter
console.log(account.balance); // prints: 70

```

Node doesn't even show `#balance` when it prints the object. And if you try to touch a private field from outside the class, JavaScript refuses to run the file at all:

```
console.log(account.#balance);  
// SyntaxError: Private field '#balance' must be declared in an enclosing class
```

Watch out: `account.balance = 1000000` only fails *silently* because that line is in a sloppy-mode file. If the same line runs in strict mode (with `"use strict"`, in a module, or inside a class), it throws `TypeError: Cannot set property balance of #<BankAccount> which has only a getter`. Either way, the balance is safe.

Inheritance: `extends` and `super`

A library lends out books and DVDs. They have a lot in common (a title, a year, being checked out), but each has its own extras. Instead of writing the shared parts twice, one class can build on another. This is called **inheritance**.

```

class LibraryItem {
  constructor(title, year) {
    this.title = title;
    this.year = year;
    this.isCheckedOut = false;
  }

  checkOut() {
    this.isCheckedOut = true;
  }

  describe() {
    return `${this.title} (${this.year})`;
  }
}

class Book extends LibraryItem {
  constructor(title, year, author) {
    super(title, year); // run LibraryItem's constructor first
    this.author = author;
  }

  describe() {
    return `${super.describe()} by ${this.author}`; // reuse the parent's version
  }
}

class Dvd extends LibraryItem {
  constructor(title, year, minutes) {
    super(title, year);
    this.minutes = minutes;
  }
}

```

- `extends` means "start with everything from this class". `LibraryItem` is the **parent** class, and `Book` and `Dvd` are **child** classes.
- `super(title, year)` calls the parent's constructor, so the parent can set up its part of the object. In a child class, you must call it before you use `this`.
- `super.describe()` calls the parent's version of a method.
- `Book` has its own `describe`, which replaces the parent's. That's called **overriding**. `Dvd` doesn't override it, so it uses the parent's.

```

const novel = new Book("The Hobbit", 1937, "J.R.R. Tolkien");
const film = new Dvd("Spirited Away", 2001, 125);

novel.checkOut(); // inherited from LibraryItem

console.log(novel.describe()); // prints: The Hobbit (1937) by J.R.R. Tolkien
console.log(film.describe()); // prints: Spirited Away (2001)
console.log(novel.isCheckedOut); // prints: true
console.log(film.isCheckedOut); // prints: false

```

Neither `Book` nor `Dvd` wrote a `checkOut` method, but both have one. They inherited it.

instanceof : which blueprint made this?

`instanceof` checks whether an object was made from a class, or from a child of that class:

```

console.log(novel instanceof Book); // prints: true
console.log(novel instanceof LibraryItem); // prints: true
console.log(film instanceof Book); // prints: false

```

A book *is a* library item, so the second line is `true`. A DVD is a library item too, but it isn't a book.

Custom error classes

In [chapter 18](#) you threw errors with `new Error("...")`. `Error` is a class, so you can extend it to make your own kinds of errors:

```

class ValidationError extends Error {
  constructor(field, message) {
    super(message); // Error's constructor stores the message
    this.name = "ValidationError";
    this.field = field;
  }
}

function registerUser(user) {
  if (!user.email.includes("@")) {
    throw new ValidationError("email", "Email must contain @");
  }
  return `Welcome, ${user.name}!`;
}

```

Now the code that catches the error can tell *your* errors apart from everything else, using `instanceof`:

```
try {
  registerUser({ name: "Kofi", email: "kofi.example.com" });
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Please fix the ${error.field} field: ${error.message}`);
  } else {
    throw error; // not a problem we know how to handle (chapter 18)
  }
}
// prints: Please fix the email field: Email must contain @
```

A form can use `error.field` to show the message next to the right input. And any unexpected error, like a real bug, still crashes loudly instead of being hidden.

Composition vs inheritance

Inheritance fits when one thing **is a** kind of another: a book *is a* library item. But often, one thing **has a** part: a car *has an* engine. For that, give the object its part as a property. This is called **composition**, building objects out of smaller objects.

```

class PetrolEngine {
  start() {
    return "Vroom!";
  }
}

class ElectricMotor {
  start() {
    return "Hmmm... (almost silent)";
  }
}

class Car {
  constructor(model, engine) {
    this.model = model;
    this.engine = engine; // a car HAS an engine
  }

  start() {
    return `${this.model}: ${this.engine.start()}`;
  }
}

const taxi = new Car("Corolla", new PetrolEngine());
const cityCar = new Car("Leaf", new ElectricMotor());

console.log(taxi.start()); // prints: Corolla: Vroom!
console.log(cityCar.start()); // prints: Leaf: Hmmm... (almost silent)

```

The same `Car` works with any engine you hand it, as long as the engine has a `start()` method. With inheritance, you'd need a `PetrolCar` class *and* an `ElectricCar` class, and then more classes for every new combination.

Long chains of inheritance (a class that extends a class that extends a class...) get hard to change, because a change at the top ripples down to everything below. So a good rule is: **when you're not sure, prefer composition**. Use inheritance only for a clear "is a" relationship, and keep it shallow.

Class or plain object?

Classes are great, but you don't need one for everything:

Situation	Use
One single object, like your app's settings	A plain object literal <code>{ ... }</code>
Plain data, like a product from a list	Plain objects

Situation	Use
Many objects with the same shape <i>and</i> methods	A class
Data that must stay private, with rules around changing it	A class with <code>#private</code> fields (or a closure)
You need <code>instanceof</code> checks, like custom errors	A class

Classes are always strict

Everything inside a class runs in strict mode, even in a normal `.js` file. So if a method gets detached from its object, `this` is `undefined` and you get a clear error, not a silent `undefined` ([chapter 26](#)). You'll see this in common mistake 5 below.

Common mistakes

1. Forgetting `new`

```
const dune = Book("Dune", "Frank Herbert");
// TypeError: Class constructor Book cannot be invoked without 'new'
```

A class can't be called like a normal function. Fix: `new Book("Dune", "Frank Herbert")`.

2. Putting commas between methods

```
class Playlist {
  constructor(name) {
    this.name = name;
  },

  describe() {
    return `Playlist: ${this.name}`;
  }
}
// SyntaxError: Unexpected token ','
```

In an object literal, properties are separated by commas. In a class, they aren't. Fix: delete the comma after the `}`.

3. Using `this` before calling `super()`

```

class Animal {
  constructor(name) {
    this.name = name;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    this.breed = breed; // using this before super()
    super(name);
  }
}

const rex = new Dog("Rex", "Beagle");
// ReferenceError: Must call super constructor in derived class before accessing 'this' or
//     returning from derived constructor

```

In a child class, the parent has to build its part of the object first. ("Derived class" is another name for a child class.) Fix: make `super(name)` the first line of the constructor. You get the same error if you forget `super()` completely.

4. Using a class before the line that creates it

```

const order = new Order("Pizza");
// ReferenceError: Cannot access 'Order' before initialization

class Order {
  constructor(item) {
    this.item = item;
  }
}

```

Function declarations are hoisted, but classes behave like `let` and `const`: they're in the temporal dead zone until their line runs ([chapter 14](#)). Fix: define your classes at the top of the file.

5. Detaching a method


```

class Timer {
  seconds = 0;

  tick() {
    this.seconds += 1;
  }
}

const timer = new Timer();
const tick = timer.tick; // detached from timer
tick();
// TypeError: Cannot read properties of undefined (reading 'seconds')

```

It's the lost `this` from chapter 26, and because class code is strict, `this` is `undefined`. Fix it the same way: `timer.tick.bind(timer)`, or wrap it in an arrow, `() => timer.tick()`.

Quick recap

- A **class** is a blueprint. `new` builds an **instance** from it, and the **constructor** sets each one up.
- Methods are written without `function` and without commas. Getters and setters look like properties but run code.
- `static` things belong to the class itself. `#private` fields and methods can only be used inside the class.
- `extends` makes a child class. `super(...)` runs the parent's constructor, and a child can override the parent's methods.
- Extend `Error` to make custom errors, and check them with `instanceof`.
- Prefer composition ("has a") over inheritance ("is a") when you're unsure. Use plain objects for one-off data.
- Class code is always strict.

Next: try the [exercises](#), then move on to [28 Prototypes](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).
- Run each one with `node ex1.js`.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Movie listings

A cinema's website lists every movie with its running time. Write a `Movie` class:

- The constructor takes `(title, minutes)` and stores them as `title` and `minutes`.
- `describe()` returns a line like `Coco (105 min)`.
- `lengthInHours()` returns the length in hours and minutes, like `1h 45m`.

Test it with this code:

```
const inception = new Movie("Inception", 148);
const coco = new Movie("Coco", 105);

console.log(inception.describe());
console.log(inception.lengthInHours());
console.log(coco.describe());
console.log(coco.lengthInHours());
console.log(coco);
```

Expected output:

```
Inception (148 min)
2h 28m
Coco (105 min)
1h 45m
Movie { title: 'Coco', minutes: 105 }
```

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Flooring shop

A flooring shop needs to work out the size of a room and the price of new flooring, which costs \$25 per square meter.

Write a `Room` class:

- The constructor takes `(name, width, length)` and stores them in that order.
- A getter `area` returns `width * length`.
- A getter `flooringCost` returns the area times 25.
- A setter `size` takes a text like `"5x6"` and updates `width` and `length` from it (as numbers).

Test it with this code:

```
const kitchen = new Room("Kitchen", 3, 4);
console.log(`${kitchen.name}: ${kitchen.area} m², flooring ${kitchen.flooringCost}`);

kitchen.size = "5x6";
console.log(`${kitchen.name}: ${kitchen.area} m², flooring ${kitchen.flooringCost}`);
console.log(kitchen);
```

Expected output:

```
Kitchen: 12 m², flooring $300
Kitchen: 30 m², flooring $750
Room { name: 'Kitchen', width: 5, length: 6 }
```

Look at the last line: `area` and `flooringCost` aren't stored in the object at all. They're worked out each time you read them.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Cinema tickets

A cinema scans tickets at the door. Each ticket gets the next number automatically, and a ticket can only be used once. Nobody should be able to "un-use" a ticket.

Write a `Ticket` class:

- A **static** property `sold` counts how many tickets have been made. Each new ticket uses it for its `number` (the first ticket is `1`).
- The constructor takes the movie name and stores `number` and `movie`.
- A **private** field `#used` starts as `false`.
- A getter `used` lets outside code read it, but there's no setter.
- `scan()` returns a welcome message the first time, and an "already used" message after that.

Test it with this code:

```

const first = new Ticket("Dune");
const second = new Ticket("Frozen");

console.log(first.scan());
console.log(first.scan());
console.log(second.scan());
console.log(first.used);
console.log(`Tickets sold: ${Ticket.sold}`);

first.used = false; // trying to sneak back in with an old ticket
console.log(first.scan());

```

Expected output:

```

Ticket #1 (Dune): Welcome, enjoy the movie!
Ticket #1 (Dune): Already used!
Ticket #2 (Frozen): Welcome, enjoy the movie!
true
Tickets sold: 2
Ticket #1 (Dune): Already used!

```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Vehicle rental

A rental company has ordinary cars, vans (which cost extra to clean after each rental), and electric scooters (which show their battery range).

1. Write a `Vehicle` class. The constructor takes `(name, dailyRate)`. It has `rentFor(days)`, which returns the cost, and `describe()`, which returns a line like `City car at $40/day`.
2. Write `Van` extends `Vehicle`. Its constructor also takes a `cleaningFee`. Override `rentFor(days)` so it adds the fee once.
3. Write `Scooter` extends `Vehicle`. Its constructor also takes `rangeKm`. Override `describe()` so it adds `(range: 40 km)` to the end.
4. Write a function `needsLicence(vehicle)` that returns `false` for scooters and `true` for everything else.

Test it with this code:

```

const fleet = [
  new Vehicle("City car", 40),
  new Van("Moving van", 70, 25),
  new Scooter("E-scooter", 15, 40),
];

for (const vehicle of fleet) {
  console.log(`${vehicle.describe()}: 3 days = ${vehicle.rentFor(3)}`);
}

for (const vehicle of fleet) {
  console.log(`${vehicle.name} needs a licence: ${needsLicence(vehicle)}`);
}

```

Expected output:

```

City car at $40/day: 3 days = $120
Moving van at $70/day: 3 days = $235
E-scooter at $15/day (range: 40 km): 3 days = $45
City car needs a licence: true
Moving van needs a licence: true
E-scooter needs a licence: false

```

Rule: don't repeat the parent's math or text in the child classes. Reuse it with `super`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Community library

Build the lending system for a small community library, using classes, private fields, custom errors, and composition.

The errors. Write three error classes:

- `LibraryError` extends `Error`, with `name` set to `"LibraryError"`.
- `BookNotFoundError` extends `LibraryError`. Its constructor takes a title, and the message is `No book called "<title>"`.
- `AlreadyBorrowedError` extends `LibraryError`. Its constructor takes a title and a member's name, and the message is `<title> is already borrowed by <member>`.

Each one sets its own `name`, like `"BookNotFoundError"`.

The books. A `Book` has a `title` and an `author`, plus a private `#borrowedBy` field (`null` when the book is on the shelf). Give it a getter `isAvailable`, a `borrow(member)` method that throws an `AlreadyBorrowedError` if someone already has it, and a `giveBack()` method.

The library. A `Library` **has** books (composition, not inheritance). It keeps them in a private `#books` array and has:

- `addBook(book)`
- `borrow(title, member)`, which throws a `BookNotFoundError` if there's no book with that title
- `giveBack(title)`
- `listAvailable()`, which returns an array of the titles on the shelf

Test it with this code:

```
const library = new Library("Kathmandu Community Library");
library.addBook(new Book("The Alchemist", "Paulo Coelho"));
library.addBook(new Book("Educated", "Tara Westover"));
library.addBook(new Book("Sapiens", "Yuval Noah Harari"));

function tryToBorrow(title, member) {
  try {
    library.borrow(title, member);
    console.log(`${member} borrowed ${title}`);
  } catch (error) {
    if (error instanceof LibraryError) {
      console.log(`${error.name}: ${error.message}`);
    } else {
      throw error;
    }
  }
}

tryToBorrow("Educated", "Asha");
tryToBorrow("Educated", "Bikash");
tryToBorrow("Harry Potter", "Chen");
console.log(library.listAvailable());

library.giveBack("Educated");
console.log(library.listAvailable());
tryToBorrow("Educated", "Bikash");
```

Expected output:

```
Asha borrowed Educated
AlreadyBorrowedError: Educated is already borrowed by Asha
BookNotFoundError: No book called "Harry Potter"
[ 'The Alchemist', 'Sapiens' ]
[ 'The Alchemist', 'Educated', 'Sapiens' ]
Bikash borrowed Educated
```

Notice that `tryToBorrow` only checks for `LibraryError`, yet it catches both kinds of error. Why does that work?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Try this with the first `Book` class from the notes (the one with just a `title`, an `author` and `describe()`):

```
const dune = new Book("Dune", "Frank Herbert");

console.log(Object.keys(dune)); // prints: [ 'title', 'author' ]
console.log(Object.hasOwn(dune, "describe")); // prints: false
console.log(dune.describe()); // prints: Dune by Frank Herbert
```

`describe` isn't one of `dune`'s own properties. So how can `dune` use it? Where does it actually live?

[Chapter 28](#) opens up the hood and shows you.

28 Prototypes

What is it?

Every object in JavaScript has a hidden link to another object, called its **prototype**.

When you ask an object for a property it doesn't have, JavaScript doesn't give up straight away. It follows the hidden link and looks in the prototype. If it's not there either, it looks in the prototype's prototype, and so on.

Why does it matter?

Prototypes are how objects share behavior behind the scenes:

- **Sharing methods.** A library app with 10,000 books doesn't need 10,000 copies of `describe`. All the books share one copy, stored on their prototype. That's where `describe` was hiding at the end of the [chapter 27 exercises](#).
- **Built-in methods.** It's why every array has `.map` and every string has `.toUpperCase`, even though you never added them.
- **Classes.** Classes are built on top of prototypes. Knowing what's underneath explains how `extends` and `instanceof` really work.
- **Older code.** Before classes existed, people built objects with prototypes directly. You'll still see that style in older code and tutorials, so you need to be able to read it.

Real-world example

Think of a family where knowledge gets passed down:

"If you don't know, ask your parent. If they don't know, ask your grandparent."

A kid is asked for the family's secret dumpling recipe. The kid doesn't know it, so they ask Mom. Mom doesn't know it either, so she asks Grandma. Grandma knows! If nobody in the family knew, the answer would be "nobody knows".

In the family	In JavaScript
What the kid knows	The object's own properties
Asking Mom	Looking in the object's prototype
Asking Grandma	Looking in the prototype's prototype

In the family	In JavaScript
The whole line of people you can ask	The prototype chain
Nobody knows	<code>undefined</code>

How it works

The hidden link

`Object.create(something)` makes a new, empty object whose prototype is `something`. Let's build the family:

```
const grandma = { recipe: "Grandma's dumplings" };

const mom = Object.create(grandma); // mom's prototype is grandma
mom.car = "Blue hatchback";

const kid = Object.create(mom); // kid's prototype is mom
kid.bike = "Red BMX";

console.log(kid.bike); // prints: Red BMX
console.log(kid.car); // prints: Blue hatchback
console.log(kid.recipe); // prints: Grandma's dumplings
console.log(kid.phone); // prints: undefined
```

`kid` only has one property of its own, `bike`. But it can use `car` from Mom and `recipe` from Grandma. When you print `kid`, you only see its own property:

```
console.log(kid); // prints: { bike: 'Red BMX' }
```

The official name for the hidden link is `[[Prototype]]`. The double square brackets mean it's internal: you can't write `kid.[[Prototype]]`. To see an object's prototype, use `Object.getPrototypeOf`:

```
console.log(Object.getPrototypeOf(kid) === mom); // prints: true
console.log(Object.getPrototypeOf(mom) === grandma); // prints: true
```

The prototype chain

Grandma has a prototype too. Every normal object you make with `{ }` gets `Object.prototype` as its prototype. That's a built-in object holding methods that every object can use, like `toString`. And `Object.prototype` is the end of the line: its prototype is `null`.

```
console.log(Object.getPrototypeOf(grandma) === Object.prototype); // prints: true
console.log(Object.getPrototypeOf(Object.prototype)); // prints: null
```

So the full chain looks like this:

```
kid           { bike }
  ↓ prototype
mom           { car }
  ↓ prototype
grandma       { recipe }
  ↓ prototype
Object.prototype { toString, ... }
  ↓ prototype
null          (the end: nobody left to ask)
```

When you read `kid.something`, JavaScript walks down this chain, one step at a time, and stops at the first object that has it. That's why this works, even though nobody in the family wrote a `toString` method:

```
console.log(kid.toString()); // prints: [object Object]
```

Own vs inherited properties

A property the object has itself is an **own** property. A property it gets through the chain is **inherited**. Some tools from [chapter 11](#) look only at own properties, and some look at the whole chain:

```
console.log(Object.hasOwn(kid, "bike")); // prints: true
console.log(Object.hasOwn(kid, "recipe")); // prints: false
console.log("recipe" in kid); // prints: true
console.log(Object.keys(kid)); // prints: [ 'bike' ]
```

Tool	Looks at	For kid
<code>Object.hasOwn(obj, key)</code>	Own properties only	"bike" → true, "car" → false
<code>key in obj</code>	The whole chain	"car" → true
<code>Object.keys(obj)</code> (and <code>values</code> , <code>entries</code>)	Own properties only	['bike']
<code>for...in</code>	Own and inherited	bike, car, recipe

That last row surprises a lot of people. A `for...in` loop over `kid` visits `bike`, then `car`, then `recipe`. If you only want the object's own properties, loop over `Object.keys(obj)` instead.

Reading walks the chain, writing doesn't

When you **set** a property, JavaScript doesn't go looking up the chain. It creates (or changes) an own property on the object itself:

```
kid.car = "Toy car"; // creates kid's OWN car property
console.log(kid.car); // prints: Toy car
console.log(mom.car); // prints: Blue hatchback
```

Mom's car is untouched. The kid's own `car` now hides Mom's, the same way a variable can shadow an outer one ([chapter 14](#)). This is also called **shadowing**.

The link is live, though. It's not a copy. If the prototype changes, everyone further down the chain sees it straight away:

```
grandma.recipe = "Grandma's spicy dumplings"; // change the prototype...
console.log(kid.recipe); // prints: Grandma's spicy dumplings
```

Constructor functions: how it was done before classes

Classes only arrived in JavaScript in 2015. Before that, people made many similar objects with a normal function and `new`. This is called a **constructor function**, and you'll still find it in older code:

```
function Book(title, author) {
  this.title = title;
  this.author = author;
}

Book.prototype.describe = function () {
  return `${this.title} by ${this.author}`;
};

const dune = new Book("Dune", "Frank Herbert");
const matilda = new Book("Matilda", "Roald Dahl");

console.log(dune.describe()); // prints: Dune by Frank Herbert
console.log(dune); // prints: Book { title: 'Dune', author: 'Frank Herbert' }
```

Every function made with the `function` keyword has a property called `prototype`. (Arrow functions don't, because they can't be used with `new`.) It starts out as an almost empty object. You put shared methods on it, like `describe` above, and every object made with `new Book(...)` gets `Book.prototype` as its prototype.

That's the hidden step of `new` that [chapter 27](#) promised. The full list is:

1. Create a new, empty object.
2. **Set its prototype to** `Book.prototype` .
3. Run `Book` with `this` pointing to the new object.
4. Give back the object.

So every book shares the one `describe` function:

```
console.log(Object.getPrototypeOf(dune) === Book.prototype); // prints: true
console.log(dune.describe === matilda.describe); // prints: true
```

Watch out: The name is confusing. `Book.prototype` is **not** the prototype of `Book` . It's the object that will become the prototype of every book that `Book` builds. Think of it as "the shared toolbox for everything made by `Book` ".

One more detail you'll use later: each `prototype` object has a `constructor` property that points back to its function. So `Book.prototype.constructor === Book` is `true` , and `dune.constructor.name` gives you `"Book"` (found through the chain, of course).

Tip: Why not put `describe` inside the constructor, as `this.describe = function () { ... }` ? It would work, but every book would get its own separate copy of the function. With 10,000 books, that's 10,000 identical functions. On the prototype, there's just one.

Classes are syntax sugar over prototypes

Syntax sugar means a nicer way to write something that works the same way underneath. Here's the same `Book` , written as a class:

```

class Book {
  constructor(title, author) {
    this.title = title;
    this.author = author;
  }

  describe() {
    return `${this.title} by ${this.author}`;
  }
}

const dune = new Book("Dune", "Frank Herbert");

console.log(typeof Book); // prints: function
console.log(Object.getPrototypeOf(dune) === Book.prototype); // prints: true
console.log(Object.hasOwn(dune, "describe")); // prints: false
console.log(Object.hasOwn(Book.prototype, "describe")); // prints: true

```

A class is really a function, and the methods you write in the class body go on `Book.prototype`, exactly like the old way. That solves the mystery from chapter 27: `describe` isn't on `dune` at all. It lives on `Book.prototype`, and `dune` finds it through the chain.

`extends` works the same way. It links one prototype to another:

```

class Animal {
  eat() {
    return "Munch munch";
  }
}

class Dog extends Animal {
  bark() {
    return "Woof!";
  }
}

const rex = new Dog();

console.log(Object.getPrototypeOf(rex) === Dog.prototype); // prints: true
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // prints: true
console.log(rex.eat()); // prints: Munch munch

```

So `rex.eat()` walks the chain: `rex` → `Dog.prototype` (no `eat` there) → `Animal.prototype` (found it!). And `instanceof` works by checking the chain too: `rex instanceof Animal` is `true` because `Animal.prototype` is somewhere in `rex`'s chain.

To be honest, "sugar" isn't the *whole* story. Classes add a few safety features that constructor functions don't have: they must be called with `new`, their code is always strict, and they support `#private` fields. But the sharing machinery underneath is the same prototypes. For new code, use classes.

Built-in prototypes

Now you can see where built-in methods come from. Arrays get their prototype from `Array.prototype`, which is where `map`, `filter`, `push` and friends live:

```
const scores = [90, 72, 85];

console.log(Object.getPrototypeOf(scores) === Array.prototype); // prints: true
console.log(Object.hasOwn(scores, "map")); // prints: false
console.log(Object.hasOwn(Array.prototype, "map")); // prints: true
```

The chain for an array is `scores` → `Array.prototype` → `Object.prototype` → `null`. That's why the MDN documentation calls the method `Array.prototype.map()`. Now you can read that name: "the `map` that lives on `Array.prototype`, shared by all arrays".

Strings work the same way, with `String.prototype`. A string itself is a primitive, not an object, but when you write `"hi".toUpperCase()`, JavaScript quietly wraps the string in an object for a moment so it can find the method.

Don't change built-in prototypes

Since `Array.prototype` is just an object, you *can* add your own methods to it, and every array will get them:

```
Array.prototype.last = function () {
  return this[this.length - 1];
};

console.log([10, 20, 30].last()); // prints: 30
```

It works, and that's exactly the danger. Don't do it:

- **It changes every array in the whole program**, including the ones inside libraries you didn't write, which might not expect it.
- **It can clash with the future.** If JavaScript later adds a real method with the same name, the two fight. This really happened. An old library called MooTools had added its own `flatten` to arrays, so when JavaScript planned an official `Array.prototype.flatten` in 2018, it would have broken websites that used MooTools. The official method had to be renamed. That's why it's called `flat` today.
- **It shows up where it shouldn't.** A `for...in` loop over `["red", "green"]` now prints `0`, `1` and `last`.

Write a normal function instead, like `function last(array) { ... }`. (And for this case, you already have `.at(-1)` from [chapter 10](#).)

`__proto__` : recognize it, don't use it

In older code and some tutorials, you'll see `__proto__` (two underscores on each side):

```
console.log(kid.__proto__ === mom); // prints: true
```

It's an old way to read or change an object's prototype. It still works, because old websites depend on it, but it's a leftover. Use `Object.getPrototypeOf(obj)` to read a prototype and `Object.create(proto)` to make an object with a chosen prototype.

`__proto__` is also behind a real security problem called prototype pollution, where an attacker sneaks a `__proto__` key into data your code merges into an object. You'll learn about it in [chapter 51](#).

Common mistakes

1. Looking for `.prototype` on an instance

```
const dune = new Book("Dune", "Frank Herbert");

console.log(dune.prototype); // prints: undefined
console.log(Object.getPrototypeOf(dune) === Book.prototype); // prints: true
```

The `prototype` property belongs to constructor functions and classes: it's the toolbox they hand out to the objects they build. An ordinary object like `dune` doesn't have one. Fix: to find an object's prototype, use `Object.getPrototypeOf(dune)`.

2. Forgetting `new` with a constructor function

```
function Book(title, author) {
  this.title = title;
  this.author = author;
}

const dune = Book("Dune", "Frank Herbert"); // forgot new!

console.log(dune); // prints: undefined
console.log(globalThis.title); // prints: Dune
```

Without `new`, `Book` runs as a plain function call. In sloppy mode, `this` is `globalThis` ([chapter 26](#)), so it quietly creates global variables and returns nothing. A class would have thrown an error instead, which is one more reason to prefer classes. Fix: `new Book(...)`.

3. Using an arrow function for a prototype method

```
Book.prototype.describe = () => {  
  return `${this.title} by ${this.author}`;  
};  
  
const dune = new Book("Dune", "Frank Herbert");  
console.log(dune.describe()); // prints: undefined by undefined
```

Arrow functions don't have their own `this`, so `this` isn't the book. Fix: use `function () { ... }` for methods you put on a prototype.

4. Forgetting that `for...in` includes inherited properties

```
const defaults = { sound: true, volume: 5 };  
const settings = Object.create(defaults);  
settings.volume = 8;  
  
for (const key in settings) {  
  console.log(key); // volume, then sound: the inherited one sneaks in  
}
```

If you only wanted the user's own settings, this loop gives you too much. Fix: loop over `Object.keys(settings)`, or check `Object.hasOwn(settings, key)` inside the loop.

5. Adding methods to built-in prototypes

It's tempting to "improve" arrays or strings by adding methods to `Array.prototype` or `String.prototype`. As you saw above, it affects all code everywhere and can clash with future JavaScript. Fix: write a normal helper function.

Quick recap

- Every object has a hidden link, `[[Prototype]]`, to another object. If a property is missing, JavaScript looks up the **prototype chain** until it reaches `Object.prototype`, then `null`.
- `Object.create(proto)` makes an object with the prototype you choose. `Object.getPrototypeOf(obj)` shows an object's prototype.
- **Own** properties belong to the object itself. `Object.hasOwn` and `Object.keys` see only those, while `in` and `for...in` see the whole chain.

- Setting a property always creates an own property (shadowing). The prototype itself doesn't change.
 - Constructor functions put shared methods on `Fn.prototype`, and `new` links each object to it. Classes do the same thing with nicer syntax.
 - Built-in methods live on built-in prototypes, like `Array.prototype.map`. Don't add your own to them, and avoid `__proto__`.
-

Next: try the [exercises](#), then move on to [29 Modules](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on).
 - Run each one with `node ex1.js`.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Default settings

A note-taking app has default settings. Each user can change some of them, and anything they haven't changed should fall back to the defaults automatically.

1. Create `defaultSettings` with `theme: "light"`, `fontSize: 16` and `language: "English"`.
2. Create `mySettings` with `Object.create`, so that its prototype is `defaultSettings`.
3. Give `mySettings` its own `theme` of `"dark"`. Don't copy the other settings.

Then run this test code:

```
console.log(mySettings.theme);
console.log(mySettings.fontSize);
console.log(Object.keys(mySettings));
console.log(Object.hasOwn(mySettings, "fontSize"));
console.log("fontSize" in mySettings);

defaultSettings.fontSize = 18; // the app's defaults get an update
console.log(mySettings.fontSize);
console.log(defaultSettings.theme);
```

Expected output:

```
dark
16
[ 'theme' ]
false
true
18
light
```

Then answer these in a comment:

- Why does `mySettings.fontSize` change to `18`, even though you never touched `mySettings`?
- Why is `defaultSettings.theme` still `"light"`?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Plant care, the old way

You've found some older code for a plant-care app, written with a constructor function. Recreate it:

1. Write a constructor function `Plant(name, waterEveryDays)` that stores both values.
2. Add two methods to `Plant.prototype`:
 - `describe()` returns a line like `Fern: water every 3 days`.
 - `needsWater(daysSinceWatered)` returns `true` if it has been at least `waterEveryDays` days.

Test it with this code:

```
const fern = new Plant("Fern", 3);
const cactus = new Plant("Cactus", 14);

console.log(fern.describe());
console.log(cactus.describe());
console.log(fern.needsWater(4));
console.log(cactus.needsWater(4));
console.log(fern.describe === cactus.describe);
console.log(Object.hasOwn(fern, "describe"));
```

Expected output:

```
Fern: water every 3 days
Cactus: water every 14 days
true
false
true
false
```

3. Now, **after** that test code, add a third method, `rename(newName)`, to `Plant.prototype`. Then call `cactus.rename("Prickly Pete")` and print `cactus.describe()`. You should see:

```
Prickly Pete: water every 14 days
```

The cactus was created *before* `rename` existed. Why can it still use it? Answer in a comment.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Modernize a recipe app

A cooking app has this older code for scaling recipes up or down. Copy it into `ex3.js` and run it:

```

function Recipe(name, servings) {
  this.name = name;
  this.servings = servings;
  this.ingredients = [];
}

Recipe.prototype.addIngredient = function (item, grams) {
  this.ingredients.push({ item, grams });
};

Recipe.prototype.scaleTo = function (newServings) {
  const factor = newServings / this.servings;
  return this.ingredients.map((ingredient) => `${ingredient.item}: ${ingredient.grams *
    factor}g`);
};

const pancakes = new Recipe("Pancakes", 4);
pancakes.addIngredient("Flour", 200);
pancakes.addIngredient("Milk", 300);
pancakes.addIngredient("Sugar", 20);

console.log(pancakes.scaleTo(2));
console.log(pancakes.scaleTo(6));
console.log(typeof Recipe);
console.log(Object.hasOwn(Recipe.prototype, "scaleTo"));
console.log(Object.getPrototypeOf(pancakes) === Recipe.prototype);

```

It prints:

```

[ 'Flour: 100g', 'Milk: 150g', 'Sugar: 10g' ]
[ 'Flour: 300g', 'Milk: 450g', 'Sugar: 30g' ]
function
true
true

```

Your task: rewrite `Recipe` as a **class**, keeping the test code at the bottom exactly the same. Your version must print exactly the same five lines.

Then answer in a comment: the last three lines check how `Recipe` works underneath. Why do they give the same answers for the class version?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 4 (Medium): Today's specials

A restaurant builds today's menu on top of its regular menu with `Object.create`, so the regular dishes are still available. The screen at the door should list **only today's specials**. Copy this into `ex4.js` and run it:

```
const regularMenu = { soup: 5, salad: 7, burger: 12 };

const todaysMenu = Object.create(regularMenu);
todaysMenu.pasta = 11;
todaysMenu.curry = 13;
todaysMenu.burger = 9; // burger is on special today!

console.log("Today's specials:");
for (const dish in todaysMenu) {
  console.log(`- ${dish}: ${todaysMenu[dish]}`);
}
```

It prints the regular dishes too:

```
Today's specials:
- pasta: $11
- curry: $13
- burger: $9
- soup: $5
- salad: $7
```

Your tasks:

1. Fix the loop so it only lists today's specials.
2. Below it, print how many dishes a customer can order today in total (specials and regular dishes, each counted once).
3. Print today's soup price and the regular burger price.

Expected output:

```
Today's specials:
- pasta: $11
- curry: $13
- burger: $9
Dishes you can order today: 5
Soup today: $5
Regular burger price: $12
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Property detective

When you write `rex.eat()`, JavaScript searches the prototype chain for `eat`. Now you'll write that search yourself, and report *where* each property was found.

Write a function `findOwner(obj, key)` that walks up the prototype chain of `obj`, one step at a time, and returns one of these strings:

- `name: found on the object itself` (an own property)
- `bark: found on Dog.prototype (1 step up)` (found on a prototype)
- `fly: not found anywhere` (not found before the end of the chain)

Test it with this code:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  eat() {
    return `${this.name} is eating`;
  }
}

class Dog extends Animal {
  bark() {
    return "Woof!";
  }
}

const rex = new Dog("Rex");

for (const key of ["name", "bark", "eat", "toString", "fly"]) {
  console.log(findOwner(rex, key));
}

console.log(findOwner([1, 2, 3], "map"));
console.log(findOwner([1, 2, 3], "length"));
```

Expected output:

```
name: found on the object itself
bark: found on Dog.prototype (1 step up)
eat: found on Animal.prototype (2 steps up)
toString: found on Object.prototype (3 steps up)
fly: not found anywhere
map: found on Array.prototype (1 step up)
length: found on the object itself
```

Rule: don't type names like `"Dog"`, `"Animal"` or `"Array"` as strings inside `findOwner`. It must work out each prototype's name by itself, so it works for any object.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Look at your Exercise 5 file: two classes, a helper function, and the test code, all squeezed into one file. Now imagine a real app with fifty classes and hundreds of functions. One giant file would be impossible to find your way around.

In [chapter 29](#), you'll learn to split your code into separate files, called **modules**, and connect them with `import` and `export`.

29 Modules

What is it?

A **module** is a JavaScript file that can share parts of its code with other files. It chooses what to share with `export`, and other files take what they need with `import`.

This lets you split one big program into many small files that work together.

Why does it matter?

So far, every program you've written has lived in one file. That's fine for 50 lines. But real apps have thousands of lines, and one giant file quickly becomes a nightmare:

- **It's hard to find things.** Where was that tax function again? Scroll, scroll, scroll...
- **Names clash.** Two parts of the app both want a function called `format`.
- **It's hard to reuse code.** To use your date helpers in another project, you'd have to copy and paste them.

Modules fix all three. Each file has one clear job, keeps its own names to itself, and shares only what it chooses to.

Every modern JavaScript project uses modules: websites, Node apps, and every npm package. From this chapter on, this course uses them too, starting with the Weather App project in [chapter 39](#).

Real-world example

Think of a well-organized kitchen:

In the kitchen	In your code
A drawer for each kind of thing: cutlery, spices, baking tools	A file for each job: <code>prices.js</code> , <code>cart.js</code> , <code>display.js</code>
The label on the front of each drawer	The file name
Deciding what goes in the drawer for others to use	<code>export</code>
Taking out only the whisk you need for this recipe	<code>import { whisk } from "../baking.js";</code>
Your own stuff, hidden at the back of the drawer	Code you don't export stays private to its file

You don't pour every utensil into one huge pile on the counter. You open the drawer you need and take out just what you need.

How it works

Your first module in Node, step by step

Let's split a tiny shop program into two files.

Step 1: Make a folder called `shop` with three files in it:

```
shop/  
├─ package.json  
├─ prices.js  
└─ main.js
```

Step 2: Tell Node to use modules. Put exactly this in `package.json`:

```
{ "type": "module" }
```

A `package.json` file holds settings for a project. This line tells Node: "treat the `.js` files in this folder as modules". (You'll learn much more about `package.json` in [chapter 50](#).)

Step 3: Write the module that shares something. In `prices.js`:

```
export const TAX_RATE = 0.13;  
  
export function addTax(price) {  
  return price + price * TAX_RATE;  
}  
  
export function formatPrice(amount) {  
  return `$$${amount.toFixed(2)}`;  
}
```

Putting `export` in front of a `const` or a `function` shares it with other files.

Step 4: Use it from another file. In `main.js`:

```
import { addTax, formatPrice } from "./prices.js";

const total = addTax(20);
console.log(formatPrice(total)); // prints: $22.60
```

Step 5: Run the file that starts everything. Open a terminal in the `shop` folder and run:

```
node main.js
```

You'll see `$22.60`.

Let's look at the `import` line closely:

- The curly braces list the names you want, a bit like destructuring from [chapter 15](#). You don't have to take everything: `main.js` never asked for `TAX_RATE`.
- `"./prices.js"` is the path to the file. `./` means "in the same folder as this file". Use `../` to go up one folder, and `./utils/format.js` to go into a subfolder.
- The path always includes the `.js` extension.
- Paths are worked out from the file that does the importing, not from wherever your terminal is.

The file you run (`main.js` here) is called the **entry point**. It's the front door of your program: it imports the other files, and they can import more files of their own.

Why Node needs `"type": "module"`

Node has two module systems, for historical reasons:

- **ES modules** use `import` and `export`. They're the official JavaScript standard, and they work in browsers too. ("ES" is short for ECMAScript, the official name of the JavaScript standard.)
- **CommonJS** uses `require` and `module.exports`. It's Node's older, original system. You'll see it at the end of this chapter.

Every plain `.js` file you've run before this chapter was treated as CommonJS. The `"type": "module"` line switches the folder over to ES modules.

Here's honestly what Node 24 does in each situation (all checked on Node 24.15.0):

Your setup	What happens when you run <code>node main.js</code>
<code>package.json</code> with <code>{ "type": "module" }</code>	Runs as ES modules, with no warnings. This is the way this course does it.

Your setup	What happens when you run <code>node main.js</code>
No <code>package.json</code> at all	Also works. Node spots the <code>import / export</code> and switches to ES modules by itself. Older versions of Node didn't do this.
A <code>package.json</code> without a <code>"type"</code> line	Works, but prints a warning every time (see below)
A <code>package.json</code> with <code>"type": "commonjs"</code>	Crashes with <code>SyntaxError: Cannot use import statement outside a module</code>
Files named <code>.mjs</code> instead of <code>.js</code>	Always ES modules, whatever <code>package.json</code> says

The warning from the third row looks like this (with your own paths):

```
[MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///C:/.../shop/main.js is not
specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to C:\...\shop\package.json.
```

It's Node saying: "I had to guess, and guessing is slow. Please tell me." Adding `"type": "module"` makes it go away.

Why not just skip `package.json` and let Node guess? Because being explicit is better than relying on guesswork. Real projects always have a `package.json`, and with `"type": "module"` in it, there's never any doubt about how your files will run.

Node uses the **nearest** `package.json`: it looks in the file's own folder first, then in the folders above it. So a `package.json` right next to your files always wins.

Tip: The `.mjs` extension ("module JS") is the other way to say "this is an ES module", without any `package.json`. If you name your files `main.mjs` and `prices.mjs`, remember that the import changes too: from `"./prices.mjs"`. This course sticks with `.js` plus `"type": "module"`.

Exporting a list, and keeping things private

Instead of writing `export` in front of each thing, you can list everything you share in one place, usually at the bottom of the file:

```
function toFahrenheit(celsius) {
  return (celsius * 9) / 5 + 32;
}

function toCelsius(fahrenheit) {
  return ((fahrenheit - 32) * 5) / 9;
}

const FREEZING_POINT = 0; // not in the list below, so it stays private

export { toFahrenheit, toCelsius };
```

Anything you don't export is private to its file. Other files can't see `FREEZING_POINT` at all. That's a lot like the private data from closures in [chapter 25](#): you decide what's public.

Renaming with `as`

What if two modules both export a function called `format`?

```
// in money.js
export function format(amount) {
  return `$$${amount.toFixed(2)}`;
}
```

```
// in dates.js
export function format(date) {
  return date.toISOString().slice(0, 10);
}
```

You can't import two things with the same name into one file, so rename them as you import them:

```
import { format as formatMoney } from './money.js';
import { format as formatDate } from './dates.js';

console.log(formatMoney(19.9)); // prints: $19.90
console.log(formatDate(new Date("2026-09-24T10:00:00Z"))); // prints: 2026-09-24
```

Importing everything with `* as`

You can also import everything a module exports, bundled into one object. Say `format.js` has two helpers:

```
// in format.js
export function price(amount) {
  return `$$${amount.toFixed(2)}`;
}

export function percent(value) {
  return `${value * 100}%`;
}
```

Then another file can take them both at once:

```
import * as format from "./format.js";

console.log(format.price(4.5)); // prints: $4.50
console.log(format.percent(0.25)); // prints: 25%
```

`format` is an object holding everything `format.js` exports. This is handy when a module has lots of small helpers and you want to make it obvious where each one comes from.

Default exports

A module can also have one **default export**: the single main thing the file is about. You mark it with `export default`:

```
// in greeter.js
export default function greet(name) {
  return `Welcome to the gym, ${name}!`;
}

export const OPENING_HOUR = 6;
```

When you import a default export, you don't use curly braces, and **you choose the name**:

```
import greet from "./greeter.js";
import sayHi from "./greeter.js"; // same function, different name
import welcome, { OPENING_HOUR } from "./greeter.js"; // default and named together

console.log(greet("Nima")); // prints: Welcome to the gym, Nima!
console.log(sayHi("Leela")); // prints: Welcome to the gym, Leela!
```

	Named export	Default export
How many per file	As many as you like	At most one

	Named export	Default export
Exporting	<code>export function addTax() {}</code>	<code>export default function greet() {}</code>
Importing	<code>import { addTax } from "./prices.js";</code>	<code>import greet from "./greeter.js";</code>
The name when importing	Must match (or be renamed with <code>as</code>)	Anything you like

Which should you use? Both are common. Many developers prefer named exports for most code: the name is the same in every file, so it's easy to search for, and a misspelled name gives you a clear error. Default exports are popular for "the one main thing" in a file, like a class.

Modules in the browser

Browsers understand `import` and `export` too, with two differences in how you set things up.

Difference 1: add `type="module"` to the script tag.

```
<script type="module" src="main.js"></script>
```

This goes in the `<head>`, like before. You don't need `defer`: module scripts automatically wait for the HTML to load, just like `defer` does ([chapter 20](#)). Only the entry point needs a `<script>` tag. The browser loads the other files by following the `import` lines.

Inside the files, everything works the same as in Node. The paths must start with `./`, `../` or `/`, and must include `.js`. Browsers never guess a file extension.

Difference 2: you can't just double-click `index.html` anymore.

When you double-click an HTML file, the browser opens it straight from your disk. The address bar starts with `file:///`. For security, browsers refuse to load modules from pages opened that way. The page just sits there, and the DevTools console (`F12` → **Console**) shows a red error like this:

```
Access to script at 'file:///C:/.../main.js' from origin 'null' has been blocked by CORS policy:
Cross origin requests are only supported for protocol schemes: chrome-extension, chrome-
untrusted, data, edge, http, https, isolated-app.
```

(That's Edge. Chrome's list at the end is a little different, but it starts the same way.)

CORS stands for Cross-Origin Resource Sharing: the browser's rules about which pages may load files from where. A page opened from `file:///` doesn't count as a proper website (its origin is `'null'`), so it isn't allowed to load module files. You'll meet CORS again with APIs in [chapter 33](#).

The fix is to open the page through a **local server**: a small program on your own computer that hands your files to the browser the way a real website would, over `http://`.

Setting up Live Server in VS Code

This course uses a free VS Code extension called **Live Server**. You only need to install it once:

1. In VS Code, open the Extensions panel (`Ctrl + Shift + X`).
2. Search for **Live Server** and install the one by **Ritwick Dey**.

Then, every time you want to open a page that uses modules:

1. Open your folder in VS Code (**File** → **Open Folder...**), if it isn't open already.
2. In the Explorer on the left, right-click `index.html` and choose **Open with Live Server**.
3. Your browser opens the page at an address like `http://127.0.0.1:5500/index.html`. (`127.0.0.1` means "this computer", and `5500` is the **port**, like a door number that Live Server listens on. The rest of the address depends on which folder you opened.)
4. Edit and save your files. Live Server reloads the page for you automatically.
5. To stop the server, click **Port : 5500** in the status bar at the bottom of VS Code.

That's it. Modules now load, because the page comes from `http://` instead of `file:///`.

Tip: Live Server works for normal pages too, and the auto-reload is handy. But from now on, any page that uses `type="module"` **must** be opened with it.

Each module has its own scope

Variables at the top of a normal script are global ([chapter 14](#)). In a module, they're not. Each module is like its own room: what's declared inside stays inside, unless you export it.

```
// in secret.js
const secretCode = "1234"; // not exported: stays private to this file

export function checkCode(code) {
  return code === secretCode;
}
```

```
// in main.js
import { checkCode } from "./secret.js";

console.log(checkCode("1234")); // prints: true
console.log(typeof secretCode); // prints: undefined
```

`checkCode` can still use `secretCode`, because it was written in the same file. (It's a closure!) But `main.js` can't see it at all. This also means two modules can each have their own variable called `total` without any clash. In the browser, a module's top-level variables don't end up on `window` either.

Modules are always strict

Every module runs in strict mode automatically, as you learned in [chapter 26](#). You don't need `"use strict"`. So in a module:

```
score = 10; // ReferenceError: score is not defined
```

And at the top level of a module, `this` is `undefined`.

A module runs only once, so it can share data

The first time any file imports a module, JavaScript runs that module's code. After that, every other import gets the same, already-loaded module. It doesn't run again.

That means every file that imports a module gets **the same** variables. So a module can hold data that the whole app shares, like a shopping cart:

```
// in store.js
console.log("Loading the store...");

const items = [];

export function addItem(item) {
  items.push(item);
}

export function countItems() {
  return items.length;
}
```



```
// in header.js
import { countItems } from "./store.js";

export function showHeader() {
  console.log(`Cart (${countItems()})`);
}
```

```
// in main.js
import { addItem } from "./store.js";
import { showHeader } from "./header.js";

addItem("Tea");
addItem("Honey");
showHeader();
```

Run `node main.js`, and you'll see:

```
Loading the store...
Cart (2)
```

Two things to notice:

- Both `main.js` and `header.js` import `store.js`, but `Loading the store...` appears only once.
- `main.js` added the items, and `header.js` sees them, because there's only one `items` array. And `items` itself isn't exported, so other files can only change it through `addItem`, just like the bank account in chapter 25.

Imports are read-only

You can use what you import, but you can't reassign it:

```
import { TAX_RATE } from "./prices.js";

TAX_RATE = 0.2; // TypeError: Assignment to constant variable.
```

That's on purpose. If any file could replace another module's values, nobody could trust them. If a module's data needs to change, the module should export a function that does it, like `addItem` above.

Two extras you'll use later

- **Top-level `await`**. Modules let you use the `await` keyword at the top level of the file, outside any function. You'll learn what `await` does in [chapter 32](#).

- **Dynamic `import()`** . Besides the `import` lines at the top of a file, you can call `import("./chart.js")` like a function, in the middle of your code. It loads a module only when you need it, for example when the user clicks "Show chart", so the page starts faster. It works with promises, which you'll meet in [chapter 31](#), and you'll use it for speed in [chapter 48](#).

Recognizing CommonJS

In older Node code, and in many tutorials and Stack Overflow answers, you'll see Node's original module system, CommonJS:

```
// in math.js
function add(a, b) {
  return a + b;
}

module.exports = { add };
```

```
// in main.js
const { add } = require("./math.js");

console.log(add(2, 3)); // prints: 5
```

	ES modules (this course)	CommonJS (older Node code)
Sharing	<code>export function add() {}</code>	<code>module.exports = { add };</code>
Using	<code>import { add } from "./math.js";</code>	<code>const { add } = require("./math.js");</code>
Works in browsers	Yes	No
In Node	With <code>"type": "module"</code> , or <code>.mjs</code> files	The default for <code>.js</code> files, and always for <code>.cjs</code> files

You don't need to write CommonJS, but you should recognize it when you see it. If you find a `require` example online, you can usually turn it into an `import` with the same names.

Organizing a small project

When a project grows, group its files into folders by what they do:

```

recipe-app/
├─ package.json
├─ main.js           ← the entry point: wires everything together
├─ data/
│   └─ recipes.js    ← the list of recipes
├─ models/
│   └─ Recipe.js     ← the Recipe class
└─ utils/
    └─ format.js     ← small formatting helpers

```

Each file imports what it needs, using paths relative to **itself**:

```

// in models/Recipe.js
export default class Recipe {
  constructor(name, minutes) {
    this.name = name;
    this.minutes = minutes;
  }
}

```

```

// in utils/format.js
export function formatMinutes(minutes) {
  return `${Math.floor(minutes / 60)}h ${minutes % 60}m`;
}

```

```

// in data/recipes.js
import Recipe from "../models/Recipe.js"; // up one folder, then into models

export const recipes = [
  new Recipe("Pancakes", 20),
  new Recipe("Dal bhat", 45),
];

```

```

// in main.js
import { recipes } from "./data/recipes.js";
import { formatMinutes } from "./utils/format.js";

for (const recipe of recipes) {
  console.log(`${recipe.name}: ${formatMinutes(recipe.minutes)}`);
}

```

Run `node main.js` from the `recipe-app` folder, and you'll see:

Pancakes: 0h 20m
Dal bhat: 0h 45m

A few habits that keep projects tidy:

- **One job per file.** If you can't describe a file in one short sentence, it's probably doing too much.
- **Name files after what's inside.** `format.js` for formatting helpers, `Recipe.js` for the `Recipe` class. (A capital letter for a file holding a class is a common habit.)
- **Export only what other files need.** Everything else stays private.
- **Keep `main.js` short.** It should mostly import things and connect them, not do all the work itself.

How this course runs modules from now on

Whenever an exercise uses `import` and `export`:

- **In Node:** put a `package.json` containing `{ "type": "module" }` in the exercise folder, open a terminal in that folder, and run `node main.js`.
- **In the browser:** load your entry point with `<script type="module" src="main.js"></script>`, and open `index.html` with **Live Server**, not by double-clicking.

Common mistakes

1. Leaving out `.js`

```
import { addTax } from "./prices";
```

```
Error [ERR_MODULE_NOT_FOUND]: Cannot find module 'C:\...\shop\prices' imported from
C:\...\shop\main.js
Did you mean to import './prices.js'?
```

Node looks for a file called exactly `prices`, with no extension, and there isn't one. Helpfully, it even suggests the fix. Browsers are just as strict: they ask the server for `prices`, get a "not found" (404) error, and the module never runs. Fix: always write the full file name, `./prices.js`.

2. Leaving out `./`

```
import { addTax } from "prices.js";
```

```
Error [ERR_MODULE_NOT_FOUND]: Cannot find package 'prices.js' imported from C:\...\shop\main.js
```

A name without `./` or `../` in front means "a package installed with npm" (you'll see those in [chapter 50](#)), not a file of yours. Fix: `"./prices.js"`.

3. Mixing up default and named imports

```
import addTax from "./prices.js";  
// SyntaxError: The requested module './prices.js' does not provide an export named 'default'
```

`prices.js` only has named exports, but an import without curly braces asks for the default export. The same kind of error happens the other way round, with curly braces around a default export. Fix: match the import to the export. Named exports need `{ }`, and the default export doesn't.

4. Double-clicking `index.html`

The page does nothing, and the console shows `Access to script at 'file:///...' from origin 'null' has been blocked by CORS policy`. Browsers won't load modules from `file:///` pages. Fix: open the page with Live Server.

5. Forgetting `type="module"`

```
<script src="main.js" defer></script>
```

```
Uncaught SyntaxError: Cannot use import statement outside a module
```

Without `type="module"`, the browser treats `main.js` as a normal script, and normal scripts can't use `import`. Fix: `<script type="module" src="main.js"></script>`.

6. Using `require` in an ES module

```
const { addTax } = require("./prices.js");  
// ReferenceError: require is not defined in ES module scope, you can use import instead
```

This happens when you copy an older CommonJS example into a folder with `"type": "module"`. Fix: turn it into an import, `import { addTax } from "./prices.js";`.

Quick recap

- A **module** is a file that shares code with `export`. Other files take it with `import { name } from "./file.js"`. Anything you don't export stays private.
- Paths start with `./` or `../` and always include `.js`. They're worked out from the importing file.

- A file can have one `export default`, imported without braces under any name. Rename named imports with `as`, or take everything with `import * as`.
 - **In Node:** add a `package.json` with `{ "type": "module" }` (or use `.mjs`). **In the browser:** use `<script type="module">` and open the page with Live Server, because `file:///` pages get a CORS error.
 - Modules have their own scope, are always strict, and run only once, so every importer shares the same module.
 - `require` and `module.exports` are CommonJS, the older Node style. Recognize it, but write ES modules.
-

Next: try the [exercises](#), then move on to [30 Timers and Callbacks](#).

Exercises

How to do these:

- Each exercise uses several files, so give each one its own folder in this chapter folder: `ex1/`, `ex2/`, and so on.
 - **Node exercises (1, 2, 4 and 5):** put a `package.json` containing `{ "type": "module" }` in the exercise folder. Open a terminal in that folder and run `node main.js`.
 - **Browser exercise (3):** open `index.html` with **Live Server** (right-click → **Open with Live Server**), as shown in the notes. Press `F12` → **Console** to see errors.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Travel converter

You're planning a trip and keep converting distances and temperatures. Put the conversion functions in their own module so any program can use them.

Create this folder:

```
ex1/
├─ package.json
├─ converter.js
└─ main.js
```

In `converter.js`, write and **export** three functions. Each one returns its answer rounded to 1 decimal place:

- `kmToMiles(km)`

- `milesToKm(miles)`
- `celsiusToFahrenheit(celsius)`

Use a constant `KM_PER_MILE = 1.609344` inside `converter.js`, but **don't** export it.

Put this in `main.js`:

```
import { kmToMiles, milesToKm, celsiusToFahrenheit } from './converter.js';

console.log(`10 km = ${kmToMiles(10)} miles`);
console.log(`26.2 miles = ${milesToKm(26.2)} km`);
console.log(`30°C = ${celsiusToFahrenheit(30)}°F`);
console.log(`-5°C = ${celsiusToFahrenheit(-5)}°F`);
```

Expected output:

```
10 km = 6.2 miles
26.2 miles = 42.2 km
30°C = 86°F
-5°C = 23°F
```

Finally, try adding `KM_PER_MILE` to the import in `main.js` and run it again. Copy the error into a comment, then remove it again.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Gym welcome screen

A gym's check-in screen greets members and shows their membership details. The code is split across three small modules, and two of them export a function with the same name.

Create `ex2/` with a `package.json` and these modules:

- `welcome.js` has a **default export**: a function that takes a name and returns `Welcome to Peak Fitness, <name>!`.
- `money.js` exports a function called `format(amount)` that returns an amount like `$45.00`.
- `dates.js` exports a function also called `format(date)` that returns a date like `2026-10-01`.

Then write `main.js`:

- Import the default export from `welcome.js` under the name `greetMember`.
- Import both `format` functions, renamed to `formatMoney` and `formatDate`.

- Print these three lines, using `greetMember("Nima")`, `formatMoney(45)` and `formatDate(new Date("2026-10-01T09:00:00Z"))`.

Expected output:

```
Welcome to Peak Fitness, Nima!
Membership: $45.00 per month
Next payment: 2026-10-01
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Quote of the day page

Build a small web page that shows a quote and a "Next quote" button, split into three modules: one for the data, one for the page, and one to connect them. (No `package.json` needed: this one runs in the browser.)

Create `ex3/index.html` with this code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Quote of the Day</title>
    <style>
      body {
        font-family: sans-serif;
        max-width: 500px;
        margin: 40px auto;
      }
      blockquote {
        font-size: 1.4em;
        font-style: italic;
      }
    </style>
    <script type="module" src="main.js"></script>
  </head>
  <body>
    <h1>Quote of the Day</h1>
    <blockquote id="quote-text"></blockquote>
    <p id="quote-author"></p>
    <p id="position"></p>
    <button id="next-button">Next quote</button>
  </body>
</html>
```


Then write three modules:

1. `quotes.js` exports this array (you can copy it), plus a function `getQuote(index)` that returns the quote at that index, starting again from the first quote after the last one.

```
export const quotes = [
  { text: "Well done is better than well said.", author: "Benjamin Franklin" },
  { text: "Simplicity is the soul of efficiency.", author: "Austin Freeman" },
  { text: "Talk is cheap. Show me the code.", author: "Linus Torvalds" },
  { text: "Make it work, make it right, make it fast.", author: "Kent Beck" },
];
```

2. `ui.js` exports a function `showQuote(quote, number, total)` that puts the quote text (in quotation marks), the author (after a dash) and the position (like `Quote 1 of 4`) into the page.
3. `main.js` imports from both, shows the first quote, and shows the next one each time the button is clicked.

Now test it in two ways:

- **First, double-click `index.html`**. The page stays empty. Open the console and copy the red error into a comment at the top of `main.js`. Which part of the notes explains it?
- **Then open it with Live Server**. You should see:

```
"Well done is better than well said."
– Benjamin Franklin
Quote 1 of 4
```

Each click shows the next quote. After `Quote 4 of 4`, the next click goes back to `Quote 1 of 4`.

Finally, with the page open in Live Server, type `quotes` into the console and press `Enter`. You get a `ReferenceError`, even though `quotes` clearly exists. Why? Write your answer in a comment.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): One cart, many files

In a shop app, the page header shows how many items are in the cart, but other parts of the app add the items. They all need to share **one** cart.

Create `ex4/` with a `package.json` and these modules:

- `store.js` starts with the line `console.log("store.js loaded");`. It keeps a private `items` array (not exported) and exports three functions:
 - `addItem(name, price)` adds `{ name, price }` to the cart.
 - `getItems()` returns a **copy** of the items, so outside code can't change the real cart.
 - `getTotal()` returns the total price.
- `header.js` imports from `store.js` and exports `showHeader()`, which prints a line like `Cart: 2 items ($7.50)`.

Put this in `main.js`:

```
import { addItem, getItems } from "./store.js";
import { showHeader } from "./header.js";

showHeader();
addItem("Tea", 4);
addItem("Honey", 3.5);
showHeader();

const snapshot = getItems();
snapshot.push({ name: "Free TV", price: 0 }); // trying to sneak something in
console.log(getItems());
```

Expected output:

```
store.js loaded
Cart: 0 items ($0.00)
Cart: 2 items ($7.50)
[ { name: 'Tea', price: 4 }, { name: 'Honey', price: 3.5 } ]
```

Then answer in comments:

1. Both `main.js` and `header.js` import `store.js`. Why does `store.js loaded` appear only once?
2. `main.js` added the items, but `header.js` counted them. How does `header.js` know about them?
3. Add the line `addItem = null;` at the end of `main.js` and run it. What error do you get, and why is that a good thing?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Tidy up the gradebook

A school's gradebook program has grown messy. It lives in one long file, plus an old helper written in CommonJS. Your job is to reorganize it into ES modules, **without changing what it prints**.

Step 1: run the old version. Make a folder `ex5-old/` (with **no** `package.json`) and create these two files.

`ex5-old/grading.js`:

```
function letterGrade(score) {  
  if (score >= 90) {  
    return "A";  
  }  
  if (score >= 80) {  
    return "B";  
  }  
  if (score >= 70) {  
    return "C";  
  }  
  if (score >= 60) {  
    return "D";  
  }  
  return "F";  
}  
  
module.exports = { letterGrade };
```

`ex5-old/gradebook.js`:

```

const { letterGrade } = require("./grading.js");

function average(numbers) {
  const total = numbers.reduce((sum, number) => sum + number, 0);
  return total / numbers.length;
}

class Student {
  constructor(name, scores) {
    this.name = name;
    this.scores = scores;
  }

  get average() {
    return average(this.scores);
  }
}

function formatRow(student) {
  const avg = student.average.toFixed(1);
  return `${student.name.padEnd(8)}${avg.padStart(5)} ${letterGrade(student.average)}`;
}

const students = [
  new Student("Aarav", [88, 92, 79]),
  new Student("Bina", [95, 98, 91]),
  new Student("Chen", [72, 65, 80]),
  new Student("Dawa", [55, 62, 58]),
];

console.log("Name      Avg   Grade");
for (const student of students) {
  console.log(formatRow(student));
}

const best = students.reduce((top, student) => (student.average > top.average ? student : top));
console.log(`Top student: ${best.name}`);
console.log(`Class average: ${average(students.map((student) => student.average)).toFixed(1)}`);

```

Run `node gradebook.js` in `ex5-old/`. It prints:

Name	Avg	Grade
Aarav	86.3	B
Bina	94.7	A
Chen	72.3	C
Dawa	58.3	F

Top student: Bina
Class average: 77.9

Step 2: reorganize it. Make a new folder `ex5/` with a `package.json` containing `{ "type": "module" }`, and split the code into this structure:

```
ex5/
├─ package.json
├─ main.js           ← prints the report (the last few lines of the old file)
├─ data/
│   └─ students.js  ← exports the students array
├─ models/
│   └─ Student.js   ← default export: the Student class
└─ utils/
    ├─ stats.js      ← exports average
    ├─ grading.js    ← exports letterGrade, converted from CommonJS
    └─ format.js     ← exports formatRow
```

Running `node main.js` in `ex5/` must print exactly the same seven lines as the old version.

Rules:

- No `require` or `module.exports` anywhere in `ex5/`.
- Each file imports only what it actually uses, and exports only what other files need.
- `main.js` shouldn't declare any functions or classes of its own (the short arrow callbacks inside `reduce` and `map` are fine). It only imports and prints.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Bonus: copy the old `gradebook.js` into a folder that has a `package.json` with `"type": "module"`, and run it. Read the error. Which common mistake from the notes is it?

Before you move on

Everything you've written so far runs straight through, top to bottom, as fast as it can. But real apps often need to **wait**: show a reminder in 10 minutes, save a draft every 30 seconds, or count down a timer.

[Chapter 30](#) shows you how to run code later, and why JavaScript doesn't just stop and wait while it does.

30 Timers and Callbacks

What is it?

A **timer** lets you run code later instead of right now: "in 5 seconds, do this", or "every second, do that".

A **callback** is a function you hand to someone else, so they can call it when the time comes. You met callbacks in [chapter 13](#). Timers are where they really shine.

Why does it matter?

A lot of real work doesn't happen instantly:

- A reminder that pops up in 10 minutes.
- A countdown on a flash sale page.
- An editor that auto-saves your draft every 30 seconds.
- A slideshow that moves to the next photo every 5 seconds.
- Waiting for a server to answer, or for a big file to load.

JavaScript can't stop and stare at the clock while it waits. Your program would freeze, and a web page wouldn't respond to any clicks. Timers and callbacks let JavaScript start something, carry on with other work, and come back when it's time.

This chapter is also the foundation for the next three: promises, `async / await`, and getting data from the internet.

Real-world example

Think about ordering food at a busy restaurant.

The synchronous way (one thing at a time, in order): you order at the counter, then stand there while the chef cooks. Nobody behind you can order until your food is ready. Everyone waits.

The asynchronous way (start now, finish later): the waiter takes your order to the kitchen, and you go and sit down. You chat with friends while the kitchen cooks. When the food is ready, it comes to your table.

At the restaurant	In JavaScript
You place an order	You start a timer (or a download)
You sit down and chat	The rest of your code keeps running

At the restaurant	In JavaScript
The kitchen does the cooking	Node or the browser does the waiting for you
"Here's your food!"	Your callback runs

And a callback is like leaving your phone number at a repair shop. You don't stand at the counter all day. You say "call me back when it's done", and go home.

How it works

Synchronous vs. asynchronous

So far, all your code has been **synchronous**: each line runs and finishes, and only then does the next line start.

Asynchronous code starts something now and finishes it later. Meanwhile, the rest of your code keeps going.

Here's the restaurant in code. `setTimeout` means "run this function later" (more on it in a moment):

```
console.log("Order placed");

setTimeout(() => {
  console.log("Your food is ready!");
}, 2000);

console.log("Chatting with friends...");
```

You'll see:

```
Order placed
Chatting with friends...
Your food is ready!
```

The first two lines appear straight away. The last one appears about 2 seconds later.

JavaScript didn't wait at `setTimeout`. It handed the job over, moved on to the next line, and came back to the callback when the time was up.

Node keeps your program running while a timer is still waiting. When there's nothing left to wait for, it exits on its own.

`setTimeout` : run something once, later

`setTimeout(callback, delay)` takes two things:

- a **callback**: the function to run later
- a **delay**: how long to wait, in **milliseconds** (thousandths of a second, so 1000 ms is 1 second)

In the restaurant example, the callback was an arrow function written right inside the call. That's the most common style. You can also pass a named function:

```
function remindMe() {  
  console.log("Time to stretch!");  
}  
  
setTimeout(remindMe, 3000); // pass the function itself, without ()  
console.log("Reminder set for 3 seconds from now");
```

You'll see:

```
Reminder set for 3 seconds from now  
Time to stretch!
```

Notice that `remindMe` has no brackets. Just like with array methods in [chapter 13](#), you hand over the function itself, and `setTimeout` calls it for you when the time comes.

`clearTimeout` : cancel it before it happens

`setTimeout` gives you back a **timer id**: a ticket that identifies this one timer. Keep it in a variable, and you can cancel the timer with `clearTimeout(id)` before it goes off.

Many email apps give you a few seconds to "undo send". Here's the idea:

```
const sendTimer = setTimeout(() => {  
  console.log("Email sent!");  
}, 5000);  
  
// The user clicks "Undo" after about 1 second:  
setTimeout(() => {  
  clearTimeout(sendTimer);  
  console.log("Undo! The email was not sent.");  
}, 1000);
```

After about 1 second, you'll see:

Undo! The email was not sent.

"Email sent!" never appears, because that timer was cancelled. And since no timers are left, Node exits straight away.

Browser vs. Node: in the browser, the timer id is a number, like `3`. In Node, it's an object (it prints as `Timeout { ... }`). You never need to look inside it. Just store it and pass it to `clearTimeout`.

`setInterval`: run something again and again

`setInterval(callback, delay)` runs your callback every `delay` milliseconds, until you stop it with `clearInterval(id)`. It's perfect for clocks, countdowns, and anything that repeats.

```
let secondsLeft = 3;

const countdown = setInterval(() => {
  console.log(secondsLeft);
  secondsLeft--;

  if (secondsLeft === 0) {
    clearInterval(countdown); // stop the interval
    console.log("Liftoff!");
  }
}, 1000);
```

You'll see one line per second:

```
3
2
1
Liftoff!
```

Without `clearInterval`, it would never stop. It's the timer version of an infinite loop from [chapter 08](#). Just like there, press `Ctrl + C` in the terminal to stop it.

On a web page, the same idea keeps a clock up to date. Say the page has a `<p id="clock"></p>` ([chapter 20](#)):

```
const clock = document.querySelector("#clock");

setInterval(() => {
  clock.textContent = new Date().toLocaleTimeString();
}, 1000);
```

Every second, the paragraph shows the new time, like 3:28:34 PM (the format depends on your computer's language settings).

	<code>setTimeout</code>	<code>setInterval</code>
Runs the callback	once	again and again
Real-world uses	reminders, "undo send", a short delay	clocks, countdowns, auto-save, slideshows
Cancel it with	<code>clearTimeout(id)</code>	<code>clearInterval(id)</code>

The delay is a minimum, not an exact time

The delay means "not before this". It doesn't promise "exactly then".

```
const start = Date.now();

setTimeout(() => {
  console.log(`Waited ${Date.now() - start} ms`);
}, 1000);
```

This prints something like `Waited 1007 ms`. Your number will be a little different each time, usually a few milliseconds over 1000. (`Date.now()` gives the current time in milliseconds, from [chapter 19](#).)

It's like a restaurant saying "your table will be ready in 20 minutes". It might be 21 or 25, but not 5. A timer's callback never interrupts code that's already running. When its time is up, it waits for its turn, like a customer waiting while the cashier finishes with the person in front.

Even a delay of `0` doesn't mean "right now":

```
setTimeout(() => {
  console.log("Timer (0 ms)");
}, 0);

console.log("Regular code");
```

You'll see:

```
Regular code
Timer (0 ms)
```

The timer's callback has to wait until the code that's already running has finished. [Chapter 40](#) shows exactly how JavaScript decides what runs next.

Callbacks: "call me back when you're done"

You've been using callbacks since [chapter 13](#), and with `addEventListener` in [chapter 21](#). There are two kinds:

- **Synchronous callbacks** run right away. The callback you give `map` runs once for each item, before `map` finishes.
- **Asynchronous callbacks** run later. The ones you give `setTimeout` or `addEventListener` run when the time comes, or when the event happens.

You can write your own functions that take an asynchronous callback. Here's a bakery that tells you when your cake is ready:

```
function bakeCake(flavor, onDone) {  
  console.log(`Baking a ${flavor} cake...`);  
  
  setTimeout(() => {  
    onDone(`${flavor} cake`); // "call me back" with the result  
  }, 2000);  
}  
  
bakeCake("chocolate", (cake) => {  
  console.log(`Ready to eat: ${cake}`);  
});  
  
console.log("Meanwhile, washing the dishes");
```

You'll see:

```
Baking a chocolate cake...  
Meanwhile, washing the dishes  
Ready to eat: chocolate cake
```

Why a callback? Why not `return` the cake? Because when `bakeCake` finishes, the cake isn't ready yet. `bakeCake` returns straight away, and the cake arrives about 2 seconds later. The callback is your phone number: "call me when it's done".

Error-first callbacks (the Node style)

Waiting for something can end badly: the file doesn't exist, the server is down, the tracking number is wrong. So how does a callback hear about an error?

Node's older built-in functions use a simple agreement called **error-first callbacks**: the callback's first parameter is the error, and the second is the result.

- If something went wrong, the first argument is an `Error` object, just like the ones you threw in [chapter 18](#).
- If everything went fine, the first argument is `null` ("no error"), and the result comes second.

Here's a pretend parcel tracker:

```
const parcels = { PKG123: "Out for delivery", PKG456: "Delivered" };

function trackParcel(trackingNumber, callback) {
  setTimeout(() => {
    const status = parcels[trackingNumber];
    if (status === undefined) {
      callback(new Error(`No parcel found for ${trackingNumber}`));
      return;
    }
    callback(null, status); // null means "no error"
  }, 1000);
}

trackParcel("PKG123", (error, status) => {
  if (error) {
    console.log("Oops:", error.message);
    return;
  }
  console.log("Status:", status);
});
// prints (after about 1 second): Status: Out for delivery
```

Now change `"PKG123"` to `"PKG999"` and run it again. You'll see `Oops: No parcel found for PKG999`.

Inside the callback, always check `error` first, and `return` so the rest doesn't run. You'll see this shape in real Node code, like `fs.readFile("notes.txt", "utf8", (error, text) => { ... })` for reading a file (you'll meet `fs` in [chapter 49](#)).

Why not use `try / catch` instead? Because the error happens later, after `try / catch` has already finished. You'll see that in "Common mistakes" below.

Callback hell: the pyramid of doom

Now say you need several steps in a row, and each one has to wait for the one before. Ordering a pizza: take the order, then make the dough, then bake it, then deliver it.

```
function doStep(step, callback) {
  setTimeout(() => {
    console.log(`Done: ${step}`);
    callback();
  }, 500);
}

doStep("Take the order", () => {
  doStep("Make the dough", () => {
    doStep("Bake the pizza", () => {
      doStep("Deliver it", () => {
        console.log("Enjoy your pizza!");
      });
    });
  });
});
```

You'll see one new line about every half a second:

```
Done: Take the order
Done: Make the dough
Done: Bake the pizza
Done: Deliver it
Enjoy your pizza!
```

It works, but look at the shape. Every step pushes the code further to the right. Programmers call this **callback hell**, or the **pyramid of doom**.

And this version doesn't even handle errors. With error-first callbacks, every single level needs its own check:

```
doStep("Take the order", (error) => {
  if (error) {
    console.log("Problem:", error.message);
    return;
  }
  doStep("Make the dough", (error) => {
    if (error) {
      console.log("Problem:", error.message);
      return;
    }
    // ...and so on, deeper and deeper
  });
});
```

Hard to read, easy to get wrong, and painful to change. This exact problem is why **promises** were invented. They're the next chapter.

Closures and timers: the `var` loop, again

In [chapter 25](#), you saw that functions created in a `for (var ...)` loop all share one variable. Timers are where this bug shows up in real life. Say a cinema wants to call seats 1, 2 and 3, one per second:

```
for (var seat = 1; seat <= 3; seat++) {  
  setTimeout(() => {  
    console.log(`Now calling seat ${seat}`);  
  }, seat * 1000);  
}
```

You'll see:

```
Now calling seat 4  
Now calling seat 4  
Now calling seat 4
```

Seat 4 doesn't even exist! Here's what happened:

1. The loop runs fast, all in one go. It sets up three timers (for 1, 2 and 3 seconds), then finishes with `seat` at 4.
2. `var` makes one shared `seat` for the whole loop ([chapter 14](#)).
3. When the timers go off, each callback looks at that one shared `seat`, and by then it's 4.

The fix is one word. Change `var` to `let`:

```
for (let seat = 1; seat <= 3; seat++) {  
  setTimeout(() => {  
    console.log(`Now calling seat ${seat}`);  
  }, seat * 1000);  
}
```

You'll see:

```
Now calling seat 1  
Now calling seat 2  
Now calling seat 3
```

With `let`, every turn of the loop gets its own `seat`, and each callback carries its own one in its backpack.

Polling: checking again every few seconds

Sometimes you're waiting for something, and nobody is going to call you back. So you keep asking. That's **polling**: checking on a schedule, like a kid in the back of the car asking "are we there yet?" every few minutes.

Real apps poll to check whether a video has finished processing, whether a payment has gone through, or whether new messages have arrived.

```
let checks = 0;

// Pretend to ask a server. The video is ready on the 3rd check.
function isVideoReady() {
  return checks === 3;
}

const poller = setInterval(() => {
  checks++;
  console.log(`Check ${checks}: is the video ready?`);
  if (isVideoReady()) {
    clearInterval(poller); // got the answer, stop asking
    console.log("Yes! Your video is ready to watch.");
  }
}, 2000);
```

You'll see one check about every 2 seconds:

```
Check 1: is the video ready?
Check 2: is the video ready?
Check 3: is the video ready?
Yes! Your video is ready to watch.
```

Tip: always have a way to stop polling: when you get your answer, and after a maximum number of tries. Otherwise your app keeps asking forever.

Coming up: debounce and throttle

Two famous helpers are built from nothing more than `setTimeout`, `clearTimeout` and a closure:

- **Debounce:** wait until things calm down, then run once. Think of a search box that waits until you stop typing before it searches.
- **Throttle:** run at most once every so often, however many times it's triggered. Think of a page reacting to scrolling a few times a second instead of hundreds.

You'll build both yourself in [chapter 34](#).

Common mistakes

1. Calling the function instead of passing it

```
setTimeout(remindMe(), 3000);  
// TypeError [ERR_INVALID_ARG_TYPE]: The "callback" argument must be of type function. Received undefined
```

The brackets call `remindMe` right away, so "Time to stretch!" prints immediately. Then `setTimeout` gets what `remindMe` returned (`undefined`) instead of a function, and Node stops with the error above. The browser doesn't even show an error. The reminder just appears too early, which is more confusing.

Fix: pass the function without brackets, `setTimeout(remindMe, 3000)`, or wrap the call in an arrow function, `setTimeout(() => remindMe(), 3000)`.

2. Expecting the next line to wait

```
let message = "Not ready";  
  
setTimeout(() => {  
  message = "Ready!";  
}, 1000);  
  
console.log(message); // prints: Not ready
```

`setTimeout` doesn't pause your program. The `console.log` runs straight away, a whole second before the callback changes `message`. The same goes for trying to `return` a value from inside a timer: by the time the callback runs, the outer function has long finished.

Fix: put the code that needs the result *inside* the callback, or pass the result on to another callback, like `bakeCake` did.

3. Forgetting to stop an interval

```
let secondsLeft = 3;  
  
setInterval(() => {  
  console.log(secondsLeft);  
  secondsLeft--;  
}, 1000);  
// prints 3, 2, 1, 0, -1, -2, ... forever
```

An interval keeps going until you stop it. Fix: store the id and call `clearInterval(id)` when you're done, like the liftoff countdown does. On a web page, a forgotten interval keeps running in the background, using up memory

and battery ([chapter 41](#)).

4. Wrapping a timer in `try / catch`

```
try {
  setTimeout(() => {
    throw new Error("The oven broke!");
  }, 1000);
} catch (error) {
  console.log("Caught it:", error.message); // never runs
}
// Error: The oven broke!
```

After about 1 second, the program crashes with `Error: The oven broke!`. The `try / catch` finished long before the timer went off, so nobody is left to catch the error.

Fix: handle errors *inside* the callback, or pass them on with an error-first callback. Promises ([chapter 31](#)) and `async / await` ([chapter 32](#)) give you a much nicer way.

5. Writing the delay in seconds

```
setTimeout(() => {
  console.log("Your 5-minute break is over!");
}, 5); // 5 milliseconds, not 5 minutes!
```

The delay is always in milliseconds. 5 seconds is `5000`, and 5 minutes is `5 * 60 * 1000`. Writing it as a calculation like that makes it easy to read.

Quick recap

- Synchronous code runs one line at a time, in order. Asynchronous code starts now and finishes later, while the rest of your code keeps running.
- `setTimeout(fn, ms)` runs `fn` once, after at least `ms` milliseconds. `clearTimeout(id)` cancels it.
- `setInterval(fn, ms)` runs `fn` again and again, until you call `clearInterval(id)`.
- The delay is a minimum, not an exact time. Even `setTimeout(fn, 0)` waits until the current code has finished.
- A callback means "call me back when you're done". In the Node style, its first parameter is the error (or `null`).
- Many waiting steps in a row turn into callback hell. Promises fix that in the next chapter.
- Use `let`, not `var`, in loops that create timers, so each callback gets its own variable.

Next: try the [exercises](#), then move on to [31 Promises](#).

Exercises

How to do these:

- Exercises 1 to 4 run in Node. Make a new file for each one in this folder (`ex1.js` , `ex2.js` , and so on) and run it with `node ex1.js` .
 - Exercise 5 runs in the browser. Make a folder `ex5` with `index.html` and `script.js` inside, then double-click `index.html` to open it. Press `F12` → **Console** to see any errors, and refresh after each change.
 - Timers make you wait, so watch *when* each line appears, not just what it says.
 - If a program never stops, press `Ctrl + C` in the terminal.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Tea timer

You're making a cup of tea. It needs to steep for a while (let's pretend 3 seconds is 3 minutes), and you don't want to stand there staring at the mug.

Print the first two lines straight away, and the last one about 3 seconds later.

Expected output:

```
Your tea is steeping...
You have time to read a few pages.
Your tea is ready! Take out the tea bag.
```

Rule: in your file, the code for the "ready" message must come *before* the `console.log` for "You have time to read a few pages."

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Countdown to the sale

A flash sale starts in 5 seconds. Print a countdown, one line about every second, then the big announcement.

Expected output:

```
Sale starts in 5...
Sale starts in 4...
Sale starts in 3...
Sale starts in 2...
Sale starts in 1...
The sale is ON! Everything is 50% off.
```

Rules:

- Use one `setInterval`, not five separate timers.
- The program must stop on its own. You shouldn't need `Ctrl + C`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Undo send

Your email app waits 5 seconds before it really sends a message, so people can change their mind.

1. Write a function `sendWithUndo(message)` that:
 - prints `Sending "<message>" in 5 seconds...` straight away
 - prints `Sent: "<message>"` about 5 seconds later
 - **returns** the timer id
2. Write a function `undoSend(timerId, message)` that cancels that timer and prints `Undo! "<message>" was not sent.`
3. Send `Lunch at 1?` and `See you tomorrow!` straight away, one after the other. Then, after about 2 seconds, undo the first one.

Expected output:

```
Sending "Lunch at 1?" in 5 seconds...
Sending "See you tomorrow!" in 5 seconds...
Undo! "Lunch at 1?" was not sent.
Sent: "See you tomorrow!"
```

The first two lines appear straight away, the third after about 2 seconds, and the last after about 5 seconds.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Library loans

A library's computer is slow: every loan takes about a second while the librarian checks the shelf. Start with this data:

```
const books = [
  { id: 1, title: "The Hobbit", available: true },
  { id: 2, title: "Dune", available: false },
  { id: 3, title: "Matilda", available: true },
];
```

1. Write `borrowBook(id, callback)`. After about 1 second, it calls `callback` in the error-first style:
 - no book with that id: an `Error` with the message `No book with id 7` (with the real id)
 - the book isn't available: an `Error` with the message `"Dune" is already on loan` (with the real title)
 - otherwise: set the book's `available` to `false`, then call back with `null` and the book
2. Write a function `showResult(error, book)` that prints `Sorry: <error message>` if there's an error, or `Enjoy "<title>"! Please bring it back in 3 weeks.` if not.
3. Borrow book 1, then book 2, then book 7, then book 1 again. Start each loan only after the one before it has finished.

Expected output (one line about every second):

```
Enjoy "The Hobbit"! Please bring it back in 3 weeks.
Sorry: "Dune" is already on loan
Sorry: No book with id 7
Sorry: "The Hobbit" is already on loan
```

Rule: use `find` ([chapter 13](#)) to look up the book.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Holiday slideshow

A travel blog wants a slideshow of holiday moments that moves on by itself, with buttons to pause and play. This one runs in the browser.

Create `ex5/index.html`:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Trip Slideshow</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Our trip to the coast</h1>
  <p id="slide"></p>
  <p id="counter"></p>
  <button id="pause">Pause</button>
  <button id="play">Play</button>
</body>
</html>

```

Start `ex5/script.js` with this array:

```

const slides = [
  "Sunrise over the mountains",
  "Lunch at the harbour",
  "A walk through the old town",
  "Sunset on the beach",
];

```

What you should see:

1. As soon as the page opens: `Sunrise over the mountains` in the first paragraph and `1 / 4` in the second.
2. About every 2 seconds, the next slide and its number (`2 / 4`, `3 / 4`, `4 / 4`). After the last slide, it goes back to the first.
3. **Pause** stops the slideshow on the current slide.
4. **Play** carries on from the current slide.
5. Clicking **Play** while the slideshow is already playing does nothing.

Test step 5 carefully: refresh the page and click **Play** a few times, a second or so apart. If the slides start changing faster, or **Pause** stops working, your code has started more than one interval.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: add **Previous** and **Next** buttons. Should a click on them restart the 2-second wait? Try it both ways and see which feels better.

Before you move on

Look back at your code for Exercise 4. Just four loans in a row, and it already looks like a staircase. Now imagine each step had its own error check, like a real app would need. That's callback hell.

[Chapter 31: Promises](#) turns that staircase into a neat, flat list of steps, with one place to handle every error.

31 Promises

What is it?

A **promise** is an object that stands in for a value you don't have yet. It's JavaScript's way of saying "I'll give you the result later, or tell you why I couldn't".

Promises replace the callback pyramids from [chapter 30](#) with neat, flat chains.

Why does it matter?

In chapter 30, waiting for several things in a row turned into callback hell: the code drifted to the right, and every level needed its own error check. Promises fix that:

- Steps that wait for each other become a flat list, top to bottom.
- One `.catch` at the end handles an error from any step.
- Built-in tools wait for several things at once ("tell me when all three orders are ready").

Promises are everywhere in modern JavaScript. `fetch` ([chapter 33](#)) returns one, and so do most tools for files and databases. And `async / await` ([chapter 32](#)) is built on top of them. Learn promises well, and the next two chapters get much easier.

Real-world example

Some restaurants give you a buzzer when you order. You don't have your food yet, but you have something that will tell you when it's ready.

Restaurant buzzer	Promise
You order and get a buzzer	You call a function and get a promise back
You wait and chat, buzzer on the table	The promise is pending , and your other code keeps running
It buzzes: "your food is ready!"	The promise is fulfilled , with a value (your burger)
Staff come over: "sorry, we're out of burgers"	The promise is rejected , with a reason (an error)
Once it has buzzed, it's over	A settled promise (fulfilled or rejected) never changes again

The buzzer is the key idea. You get it straight away, even though the food comes later. And you can decide ahead of time what to do when it buzzes ("eat") and what to do if things go wrong ("order something else").

How it works

Creating a promise

You make a promise with `new Promise(...)`. You give it a function, and JavaScript runs that function straight away. It receives two tools:

- `resolve(value)` : call it when the work succeeded, with the result.
- `reject(error)` : call it when the work failed, with an error.

```
const order = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("burger and fries");
  }, 2000);
});

console.log(order); // prints: Promise { <pending> }
```

This is a pretend kitchen: a timer from chapter 30 stands in for the cooking. When the timer goes off, `resolve` hands over the food.

Printing the promise straight away shows `Promise { <pending> }`, because the food isn't ready yet. If you printed it again after 2 seconds, you'd see `Promise { 'burger and fries' }`.

`.then` : when the value is ready

To use the value, pass a callback to `.then`. It runs when the promise is fulfilled, and it receives the value. Add this below the code above:

```
order.then((food) => {
  console.log(`Enjoy your ${food}!`);
});
// prints (after about 2 seconds): Enjoy your burger and fries!
```

Think of `.then` as telling the buzzer "when you buzz, do this".

`reject` and `.catch` : when something goes wrong

Real work fails sometimes. Here's a function that returns a promise, and rejects it if the item isn't on the menu:


```
const menu = ["burger", "salad", "soup"];

function orderFood(item) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!menu.includes(item)) {
        reject(new Error(`Sorry, we don't have ${item} today`));
        return;
      }
      resolve(item);
    }, 1000);
  });
}
```

`.catch` handles the rejection:

```
orderFood("pizza")
  .then((food) => console.log(`Enjoy your ${food}!`))
  .catch((error) => console.log("Problem:", error.message));
// prints (after about 1 second): Problem: Sorry, we don't have pizza today
```

Change `"pizza"` to `"burger"`, and you'll see `Enjoy your burger!` instead.

A few things to notice:

- Always reject with an `Error` object, like the ones you threw in [chapter 18](#). Then you get a `message` and a stack trace.
- The `return` after `reject` stops the function there, just like with chapter 30's error-first callbacks.
- `orderFood` *returns* the promise. That's the usual pattern: a function that does slow work hands you a promise straight away.

Most examples from here on use this `orderFood` function, so keep it at the top of your practice file.

`.finally`: runs either way

`.finally` runs when the promise settles, whether it was fulfilled or rejected. It's for cleaning up, like handing the buzzer back, or hiding a "Loading..." message on a web page.

```
orderFood("salad")
  .then((food) => console.log(`Enjoy your ${food}!`))
  .catch((error) => console.log("Problem:", error.message))
  .finally(() => console.log("Buzzer handed back"));
```

You'll see:

```
Enjoy your salad!  
Buzzer handed back
```

With "pizza", you'd see the "Problem" line instead, followed by Buzzer handed back all the same.

A promise settles only once

Once a promise is fulfilled or rejected, that's final. Any later call to `resolve` or `reject` is ignored:

```
const coinFlip = new Promise((resolve, reject) => {  
  resolve("heads");  
  resolve("tails");           // ignored  
  reject(new Error("Dropped")); // ignored  
});  
  
coinFlip.then((side) => console.log(side)); // prints: heads
```

That's a nice safety feature. A promise can never "call you back" twice by accident.

`wait(ms)` : a timer as a promise

Here's a tiny helper you'll use a lot in the next chapters. It turns `setTimeout` into a promise:

```
function wait(ms) {  
  return new Promise((resolve) => {  
    setTimeout(resolve, ms);  
  });  
}  
  
wait(1000).then(() => console.log("One second later"));
```

When the time is up, `setTimeout` calls `resolve`, and that fulfills the promise. A timer can't fail, so this one doesn't need `reject`.

Chaining: one step after another

Here's the best part. `.then` always returns a *new* promise, so you can add another `.then` after it. Whatever your callback returns is passed on to the next step.

If you return a promise, the next step waits for it. That's how you do things in order:

```

wait(500)
  .then(() => {
    console.log("Dough ready");
    return wait(500); // the next step waits for this
  })
  .then(() => {
    console.log("Pizza baked");
    return wait(500);
  })
  .then(() => {
    console.log("Pizza delivered. Enjoy!");
  });

```

You'll see one line about every half a second:

```

Dough ready
Pizza baked
Pizza delivered. Enjoy!

```

Compare this with the pizza pyramid in chapter 30. Same steps, but the code grows *down* instead of *right*.

If you return a plain value, the next step gets it straight away:

```

orderFood("burger")
  .then((food) => `${food} with extra cheese`)
  .then((meal) => `${meal} and a drink`)
  .then((meal) => console.log(`Your order: ${meal}`));
// prints: Your order: burger with extra cheese and a drink

```

Each arrow function returns its string automatically (the implicit return from [chapter 09](#)), and each step passes it on to the next.

Errors skip ahead to `.catch`

When a promise in a chain rejects, JavaScript skips every `.then` after it and jumps to the next `.catch`. It's like `try / catch`, but for code that waits.

```
orderFood("pizza")
  .then((food) => {
    console.log(`Cooking the ${food}`); // skipped
    return food;
  })
  .then((food) => console.log(`Serving the ${food}`)) // skipped
  .catch((error) => console.log("Problem:", error.message))
  .then(() => console.log("Next customer, please!"));
```

You'll see:

```
Problem: Sorry, we don't have pizza today
Next customer, please!
```

Two things to notice:

- One `.catch` at the end covers every step before it. No more error checks at every level.
- Once `.catch` has handled the error, the chain carries on as normal. That's why "Next customer, please!" still prints.

Throwing an error inside a `.then` works the same way. Remember how, in chapter 30, `try / catch` couldn't catch an error thrown inside a timer? Promises fix that:

```
wait(500)
  .then(() => {
    throw new Error("The oven broke!");
  })
  .then(() => console.log("Pizza baked")) // skipped
  .catch((error) => console.log("Problem:", error.message));
// prints: Problem: The oven broke!
```

Promise.resolve and Promise.reject : ready-made promises

Sometimes you already have the answer, but your function always returns a promise. `Promise.resolve(value)` makes a promise that's already fulfilled with that value. `Promise.reject(error)` makes one that's already rejected.

Here's a weather app that remembers forecasts it has already looked up:

```
const savedForecasts = { London: "12°C and cloudy" };

function getForecast(city) {
  if (savedForecasts[city] !== undefined) {
    return Promise.resolve(savedForecasts[city]); // no need to wait
  }
  return wait(1000).then(() => "25°C and sunny"); // pretend to ask a server
}

getForecast("London").then((forecast) => console.log("London:", forecast));
getForecast("Rome").then((forecast) => console.log("Rome:", forecast));
```

You'll see:

```
London: 12°C and cloudy
Rome: 25°C and sunny
```

London appears straight away, and Rome about a second later. Either way, the code that calls `getForecast` can always use `.then`, without caring whether the answer was saved or fetched.

Waiting for several promises at once

So far, you've waited for one thing at a time. But often you start several things together. At a food court, you might order pizza, salad and soup from three different counters at once. Promises come with four tools for this.

The examples use a small helper that "cooks" a dish in a given time:

```
function cook(dish, ms) {
  return wait(ms).then(() => dish);
}
```

`Promise.all`: wait for all of them

`Promise.all` takes an array of promises. It fulfills when *all* of them are fulfilled, with an array of their values.

```
const start = Date.now();

Promise.all([cook("pizza", 3000), cook("salad", 1000), cook("soup", 2000)])
  .then((dishes) => {
    console.log(dishes);
    console.log(`Served after about ${Math.round((Date.now() - start) / 1000)} seconds`);
  });
```

You'll see:

```
[ 'pizza', 'salad', 'soup' ]  
Served after about 3 seconds
```

- The three dishes cook at the same time, so it takes about 3 seconds (as long as the slowest one), not 6.
- The values come back in the same order as your array, not in the order they finished.
- If *any* promise rejects, `Promise.all` rejects with that error. One missing dish, and the whole order fails:

```
Promise.all([orderFood("burger"), orderFood("pizza"), orderFood("soup")])  
  .then((meals) => console.log("All here:", meals))  
  .catch((error) => console.log("Problem:", error.message));  
// prints: Problem: Sorry, we don't have pizza today
```

`Promise.allSettled`: wait for all, keep every result

Sometimes you want every result, good or bad. `Promise.allSettled` waits until all of them have settled, and it never rejects. Each result is an object with a `status`: either `"fulfilled"` (with a `value`) or `"rejected"` (with a `reason`, the error).

```
Promise.allSettled([orderFood("burger"), orderFood("pizza"), orderFood("soup")])  
  .then((results) => {  
    for (const result of results) {  
      if (result.status === "fulfilled") {  
        console.log("Served:", result.value);  
      } else {  
        console.log("Failed:", result.reason.message);  
      }  
    }  
  })
```

You'll see:

```
Served: burger  
Failed: Sorry, we don't have pizza today  
Served: soup
```

`Promise.race`: the first one to finish wins

`Promise.race` settles as soon as the *first* promise settles, fulfilled or rejected. Its classic use is a time limit: race your real work against a timer that rejects.

```
function timeout(ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      reject(new Error(`Gave up after ${ms / 1000} seconds`));
    }, ms);
  });
}

Promise.race([cook("pizza", 3000), timeout(2000)])
  .then((dish) => console.log("Served:", dish))
  .catch((error) => console.log("Problem:", error.message));
// prints (after about 2 seconds): Problem: Gave up after 2 seconds
```

The pizza takes 3 seconds, so the 2-second timer wins. Try `cook("salad", 1000)` instead, and you'll see `Served: salad`.

The loser isn't stopped, though. The pizza keeps cooking in the background, and `race` just stops listening to it. That's why this program only exits after about 3 seconds. You'll learn to truly cancel work in [chapter 33](#).

Promise.any : the first one to succeed

`Promise.any` fulfills with the first promise that *succeeds*, and ignores failures along the way. Think of calling three taxi companies at once and taking whichever says yes first:

```
function askTaxi(company, ms, available) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (available) {
        resolve(`${company} is on the way`);
      } else {
        reject(new Error(`${company} has no drivers`));
      }
    }, ms);
  });
}

Promise.any([
  askTaxi("City Cabs", 1000, false),
  askTaxi("Quick Rides", 2000, true),
  askTaxi("Night Owl Taxis", 3000, true),
])
  .then((answer) => console.log(answer))
  .catch((error) => console.log(error.message));
// prints (after about 2 seconds): Quick Rides is on the way
```

City Cabs answered first, but it said no, so `any` kept waiting. If every promise fails, `Promise.any` rejects with an `AggregateError`. Its message is `All promises were rejected`, and its `errors` property is an array holding each separate error.

Which one do I need?

Method	Waits for	Gives you	Rejects when
<code>Promise.all</code>	all of them to succeed	an array of values	any one fails
<code>Promise.allSettled</code>	all of them to finish	an array of <code>{ status, value }</code> or <code>{ status, reason }</code> objects	never
<code>Promise.race</code>	the first one to finish	that one's value (or its error)	the first one to finish failed
<code>Promise.any</code>	the first one to succeed	that one's value	all of them fail

Common mistakes

1. Forgetting to `return` inside `.then`

```
wait(500)
  .then(() => {
    console.log("Dough ready");
    wait(2000); // forgot "return"
  })
  .then(() => {
    console.log("Pizza baked"); // prints straight away, not 2 seconds later!
  });
```

Without `return`, the next `.then` doesn't know there's anything to wait for, so it runs straight away. The same goes for values: if a step with curly braces `{ }` forgets to `return`, the next step receives `undefined`. Fix: `return wait(2000);`.

2. Nesting instead of chaining


```
wait(500).then(() => {
  console.log("Dough ready");
  wait(500).then(() => {
    console.log("Pizza baked");
    wait(500).then(() => {
      console.log("Pizza delivered");
    });
  });
});
```

This works, but it's the callback pyramid again, just with promises. Worse, a `.catch` on the outside can't see errors from the inner promises, because they were never returned. Fix: `return` each promise and put the `.then`s one below the other, like in "Chaining" above.

3. Forgetting `.catch`

```
orderFood("pizza").then((food) => console.log(`Enjoy your ${food}!`));
```

A rejected promise with no `.catch` is an **unhandled rejection**. Node prints the error and stops your whole program with an error code, even if other timers were still waiting:

```
C:\...\practice.js:7
    reject(new Error(`Sorry, we don't have ${item} today`));
    ^

Error: Sorry, we don't have pizza today
    at Timeout._onTimeout (C:\...\practice.js:7:16)
    ...

Node.js v24.15.0
```

In the browser, the console shows a red `Uncaught (in promise) Error: Sorry, we don't have pizza today`. Fix: end every chain with a `.catch`.

4. Using the value outside `.then`

```
let food;

orderFood("burger").then((result) => {
  food = result;
});

console.log(food); // prints: undefined
```

The `console.log` runs straight away, about a second before the food arrives. It's the same mistake as in chapter 30, with a promise this time. Fix: use the value *inside* `.then`, or in a later step of the chain. The next chapter makes this feel much more natural.

5. Rejecting with a plain string

```
reject("Out of stock"); // not an Error object
```

Your `.catch` still runs, but `error.message` is `undefined`, and there's no stack trace to tell you where the problem came from. Fix: `reject(new Error("Out of stock"))`.

Quick recap

- A promise stands in for a value that isn't ready yet. It starts **pending**, then settles as **fulfilled** (with a value) or **rejected** (with an error), and never changes again.
- You create one with `new Promise((resolve, reject) => { ... })`, but most of the time you'll use promises that functions give you.
- `.then` runs when it's fulfilled, `.catch` when it's rejected, and `.finally` either way.
- Chains stay flat: return a value or a promise from `.then` to pass it on. An error skips ahead to the next `.catch`.
- `Promise.all` waits for all (and fails if any fail), `allSettled` gives you every result, `race` takes the first to finish, and `any` takes the first to succeed.
- Always end a chain with `.catch`. In Node, an unhandled rejection stops your program.

Next: try the [exercises](#), then move on to [32 Async/Await](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on) and run it with `node ex1.js`.
 - Every exercise uses timers to pretend that work takes time. Watch *when* each line appears, not just what it says.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Your parcel is on its way

You're waiting at home for a parcel. Make a promise called `delivery` that fulfills after about 2 seconds with the text `Your parcel has arrived!`.

Then, straight away:

1. Print `delivery` itself with `console.log`, to see its state.
2. Print `Waiting for the parcel...`
3. Use `.then` to print the value when it arrives.

Expected output:

```
Promise { <pending> }  
Waiting for the parcel...  
Your parcel has arrived!
```

Bonus: use `setTimeout` to print `delivery` again after about 3 seconds. What does it look like now?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Gym session

Your fitness app guides you through a workout, one step at a time. Write the `wait(ms)` helper from the notes (try it without looking first), then use it to print:

Expected output:

```
Starting the workout...  
Warm-up done  
Squats done  
Stretching done  
Workout complete! Time for a smoothie.
```

Timing: the warm-up takes about 1 second, the squats about 2 seconds, and the stretching about 1 second. The last line appears straight after `Stretching done`.

Rule: use one flat chain of `.then` s. No `.then` inside another `.then`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Cash machine

A cash machine takes about a second to talk to the bank before it gives you money.

1. Write `withdraw(balance, amount)`. It returns a promise that, after about 1 second:
 - rejects with an `Error` saying `Amount must be more than $0` if `amount` is 0 or less
 - rejects with an `Error` saying `Not enough money (balance: $100)` if `amount` is more than `balance` (with the real balance)
 - otherwise fulfills with the new balance
2. Write `tryWithdrawal(balance, amount)`. It calls `withdraw` and:
 - prints `New balance: $70` (with the real number) when it works
 - prints `Payment failed: <error message>` when it doesn't
 - always prints `Thank you for banking with us.` at the end, either way
 - **returns** the whole chain, so whoever calls it can wait for it
3. Run these three tries one after another:

```
tryWithdrawal(100, 30)
  .then(() => tryWithdrawal(100, 500))
  .then(() => tryWithdrawal(100, -5));
```

Expected output (a pair of lines about every second):

```
New balance: $70
Thank you for banking with us.
Payment failed: Not enough money (balance: $100)
Thank you for banking with us.
Payment failed: Amount must be more than $0
Thank you for banking with us.
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Holiday booking

A travel website books your flight, hotel and car hire all at the same time. Start with this pretend booking system:

```
function book(item, ms, available) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (available) {
        resolve(`${item} booked`);
      } else {
        reject(new Error(`${item} is fully booked`));
      }
    }, ms);
  });
}
```

Part 1. Book all three at once with `Promise.all`, in this order: the flight (2000 ms), the hotel (1000 ms) and the car (1500 ms), all available. Print the results, then how long it took in whole seconds. Also add a `.catch` that prints `Booking failed: <error message>`.

Expected output:

```
[ 'Flight booked', 'Hotel booked', 'Car booked' ]
All done in about 2 seconds
```

Why about 2 seconds, and not 4.5? All three bookings run at the same time, so you only wait for the slowest one.

Part 2. Make the car unavailable and run it again. Expected output:

```
Booking failed: Car is fully booked
```

Notice *when* it appears: after about 1.5 seconds. `Promise.all` didn't wait for the flight.

Part 3. The customer still wants to know what *did* work. Switch to `Promise.allSettled` and print one line per booking.

Expected output:

```
OK: Flight booked
OK: Hotel booked
FAILED: Car is fully booked
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Hungry and in a hurry

A food delivery app sends your order to several restaurants at once. The first one to accept gets the order. But you're hungry, so if nobody accepts in time, the app gives up.

Start with this data:

```
const restaurants = [
  { name: "Pizza Planet", prepTime: 2500, open: true },
  { name: "Burger Barn", prepTime: 1500, open: false },
  { name: "Noodle Nook", prepTime: 2000, open: true },
];
```

1. Write `requestOrder(restaurant)`. After `prepTime` milliseconds, it rejects with an `Error` like `Burger Barn is closed` if the restaurant isn't open, or fulfills with a message like `Noodle Nook accepted your order`.
2. Write `withTimeout(promise, ms)`. It returns a promise that settles like `promise`, unless `ms` milliseconds pass first. In that case, it rejects with an `Error` saying `No answer within 1000 ms` (with the real number).
3. Write `findFood(timeLimit)`. It:
 - prints `Looking for food (time limit: 3000 ms)...` (with the real limit)
 - sends a request to every restaurant, using `map` ([chapter 13](#))
 - takes the first restaurant that accepts, with a time limit
 - prints the answer, or `Sorry: <error message>`
 - returns the whole chain
4. Try it twice, one after the other:

```
findFood(3000).then(() => findFood(1000));
```

Expected output:

```
Looking for food (time limit: 3000 ms)...
Noodle Nook accepted your order
Looking for food (time limit: 1000 ms)...
Sorry: No answer within 1000 ms
```

Burger Barn answers first, but it's closed, so it shouldn't win. Pizza Planet is open, but slower than Noodle Nook.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: set every restaurant's `open` to `false` and run `findFood(3000)`. `Promise.any` rejects with an `AggregateError`, and its message (`All promises were rejected`) isn't very friendly. Print `Sorry: every restaurant is closed right now` in that case instead. (`instanceof` from [chapter 27](#) can tell the two kinds of error apart.)

Before you move on

Look at your `tryWithdrawal` and `findFood` functions. Promise chains are a big step up from callback pyramids, but all those `.then`s and arrow functions still take some effort to read.

What if you could write code that waits, but reads top to bottom like normal code, and even use a regular `try / catch` ? That's exactly what [chapter 32: Async/Await](#) gives you.

32 Async/Await

What is it?

`async` and `await` are two keywords that let you write promise code that reads like normal, top-to-bottom code.

Under the hood, it's still the promises from [chapter 31](#). It just looks much friendlier.

Why does it matter?

Promise chains are a big improvement on callback pyramids, but they still have rough edges:

- Every step is its own little arrow function inside a `.then`.
- A value from the first step isn't easy to use in the third step.
- Errors go to `.catch`, not the `try / catch` you know from [chapter 18](#).

With `async / await`, code that waits looks just like the code you've been writing all along. Values stay in ordinary variables, and `try / catch` works again.

It's how most modern JavaScript is written, including the code that gets data from the internet in the next chapter.

Real-world example

Think of a baker following a bread recipe:

1. Mix the dough.
2. **Wait** for it to rise (an hour).
3. Bake it.

While the dough rises, the bakery doesn't close. The other bakers keep serving customers. Only *this* recipe is paused at step 2, and the baker picks it up again as soon as the dough is ready.

In the bakery	In JavaScript
A recipe card	An <code>async</code> function
"Wait for the dough to rise"	<code>await somePromise</code>
This recipe pauses at that step	This function pauses at that line
The other bakers keep working	The rest of your program keeps running

In the bakery	In JavaScript
The dough is ready, so the recipe carries on	The promise settles, and the function carries on with the value

How it works

The examples use two helpers from chapter 31: `wait(ms)` and `orderFood(item)`. Copy them to the top of your practice file:

```
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

const menu = ["burger", "salad", "soup"];

function orderFood(item) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (!menu.includes(item)) {
        reject(new Error(`Sorry, we don't have ${item} today`));
        return;
      }
      resolve(item);
    }, 1000);
  });
}
```

`async` functions always return a promise

Put the word `async` in front of a function, and it always returns a promise. Whatever you `return` becomes the promise's value:

```
async function getGreeting() {
  return "Welcome to the gym!";
}

console.log(getGreeting()); // prints: Promise { 'Welcome to the gym!' }
getGreeting().then((text) => console.log(text)); // prints: Welcome to the gym!
```

And if an `async` function throws an error, its promise is rejected:

```
async function checkMembership() {
  throw new Error("Membership expired");
}

checkMembership().catch((error) => console.log("Problem:", error.message));
// prints: Problem: Membership expired
```

That's the first half of the deal. The second half, `await`, is where it gets good.

`await` : wait for a promise

Inside an `async` function, you can put `await` in front of a promise. The function pauses at that line until the promise settles:

- If the promise is fulfilled, `await` gives you its value.
- If it's rejected, `await` throws its error, just like `throw`.

```
async function makeTea() {
  console.log("Boiling the water...");
  await wait(2000);
  console.log("Steeping the tea...");
  await wait(3000);
  console.log("Tea is ready!");
}

makeTea();
```

You'll see the first line straight away, the second after about 2 seconds, and the last after about 5 seconds:

```
Boiling the water...
Steeping the tea...
Tea is ready!
```

No `.then`, no nesting. It reads like a recipe.

When the promise has a value, store it in a variable like any other result:

```
async function lunch() {
  const food = await orderFood("soup");
  console.log(`Enjoy your ${food}!`);
}

lunch();
// prints (after about 1 second): Enjoy your soup!
```

Read `const food = await orderFood("soup");` as "wait for the soup, then put it in `food`".

`await` pauses the function, not the whole program

This is the most important idea in the chapter. `await` only pauses the `async` function it's in. Everything else keeps going, just like the other bakers in the bakery:

```
async function makeTea() {
  console.log("1. Put the kettle on");
  await wait(1000);
  console.log("3. Pour the tea");
}

makeTea();
console.log("2. Get a mug while you wait");
```

You'll see:

1. Put the kettle on
2. Get a mug while you wait
3. Pour the tea

When `makeTea` reaches `await`, it steps aside and hands control back to whoever called it. The code after `makeTea();` runs, and about a second later, `makeTea` picks up where it left off.

Handling errors with `try / catch / finally`

Because a rejected promise makes `await` throw, you can use the `try / catch / finally` from chapter 18, exactly as you already know it:

```

async function lunch(item) {
  try {
    const food = await orderFood(item);
    console.log(`Enjoy your ${food}!`);
  } catch (error) {
    console.log("Problem:", error.message);
  } finally {
    console.log("Buzzer handed back");
  }
}

lunch("pizza");

```

You'll see (after about 1 second):

```

Problem: Sorry, we don't have pizza today
Buzzer handed back

```

It's the same as `.then`, `.catch` and `.finally` from chapter 31, but written the ordinary way. Remember how `try / catch` couldn't catch errors from a timer in chapter 30? With `await`, it can, because the function really does wait inside the `try`.

Async arrow functions

Arrow functions can be `async` too. Put `async` before the brackets:

```

const greetMember = async (name) => {
  await wait(500);
  return `Welcome back, ${name}!`;
};

greetMember("Priya").then((message) => console.log(message));
// prints (after about half a second): Welcome back, Priya!

```

You'll often see them as callbacks, like an event listener that needs to wait for something ([chapter 21](#)):

```

saveButton.addEventListener("click", async () => {
  await saveDraft(); // wait for the save to finish
  console.log("Draft saved");
});

```

(That's a sketch: `saveButton` and `saveDraft` stand for a real button and a real saving function.)

One by one, or all at once?

`await` makes it very easy to do things one after another. Sometimes that's exactly right, and sometimes it wastes time. Here's a kitchen cooking three dishes, using the `cook` helper from chapter 31:

```
function cook(dish, ms) {
  return wait(ms).then(() => dish);
}

async function cookOneByOne() {
  const start = Date.now();
  const pasta = await cook("pasta", 2000);
  const sauce = await cook("sauce", 1000);
  const salad = await cook("salad", 1000);
  console.log(pasta, sauce, salad);
  console.log(`One by one: about ${Math.round((Date.now() - start) / 1000)} seconds`);
}

async function cookAllAtOnce() {
  const start = Date.now();
  const [pasta, sauce, salad] = await Promise.all([
    cook("pasta", 2000),
    cook("sauce", 1000),
    cook("salad", 1000),
  ]);
  console.log(pasta, sauce, salad);
  console.log(`All at once: about ${Math.round((Date.now() - start) / 1000)} seconds`);
}

async function compare() {
  await cookOneByOne();
  await cookAllAtOnce();
}

compare();
```

You'll see:

```
pasta sauce salad
One by one: about 4 seconds
pasta sauce salad
All at once: about 2 seconds
```

- **One by one** (sequential): each `await` waits for the dish before starting the next one. $2 + 1 + 1 =$ about 4 seconds.

- **All at once** (parallel): all three start straight away, and one `await` waits for `Promise.all`. You only wait for the slowest dish: about 2 seconds.
- `const [pasta, sauce, salad] = ...` is array destructuring from [chapter 15](#). It unpacks the array that `Promise.all` gives you.

So which one should you use?

If...	Then...	Example
a step needs the result of the step before	wait one by one	you need a customer's order number before you can track the parcel
the steps don't depend on each other	start them all, then <code>await Promise.all(...)</code>	loading the weather, the news and your calendar for a dashboard

Loops: `for...of` waits for each item

To do something slow for each item in an array, one at a time, use `await` inside a `for...of` loop:

```
const playlist = ["Intro", "Chorus", "Outro"];

async function playAll() {
  for (const song of playlist) {
    await wait(1000);
    console.log(`Played: ${song}`);
  }
  console.log("Playlist finished!");
}

playAll();
```

You'll see one song about every second, and then the last line:

```
Played: Intro
Played: Chorus
Played: Outro
Playlist finished!
```

The loop waits at `await` on every turn. Watch out: `forEach` does *not* work like this. You'll see why in "Common mistakes".

Top-level `await` (modules only)

So far, every `await` has been inside an `async` function. In an ES module ([chapter 29](#)), you can also use `await` at the top level of a file, outside any function.

Add a `package.json` file containing `{ "type": "module" }` to your folder, like in chapter 29. Then this works:

```
// main.js, in a folder whose package.json says { "type": "module" }  
const food = await orderFood("salad");  
console.log(`Enjoy your ${food}!`);  
// prints (after about 1 second): Enjoy your salad!
```

In a CommonJS file, the same code is an error: `SyntaxError: await is only valid in async functions and the top level bodies of modules`. (If there's no `package.json` at all, Node 24 notices the `await` and runs the file as a module anyway. With `"type": "module"`, there's no guessing.) In the browser, top-level `await` works inside `<script type="module">`.

You'll use top-level `await` a lot in the next chapter, to keep examples short.

Converting a `.then` chain to `async / await`

You'll find plenty of `.then` chains in older code. Here's one from a restaurant:

```
function serveLunch(item) {  
  return orderFood(item)  
    .then((food) => {  
      console.log(`Cooking the ${food}...`);  
      return wait(1000).then(() => food);  
    })  
    .then((food) => console.log(`Here's your ${food}!`))  
    .catch((error) => console.log("Problem:", error.message))  
    .finally(() => console.log("Table cleared"));  
}  
  
serveLunch("soup");
```

And here's the same thing with `async / await`:

```

async function serveLunch(item) {
  try {
    const food = await orderFood(item);
    console.log(`Cooking the ${food}...`);
    await wait(1000);
    console.log(`Here's your ${food}!`);
  } catch (error) {
    console.log("Problem:", error.message);
  } finally {
    console.log("Table cleared");
  }
}

serveLunch("soup");

```

Both print the same lines, the first after about 1 second and the other two after about 2 seconds:

```

Cooking the soup...
Here's your soup!
Table cleared

```

The recipe for converting:

1. Put `async` in front of the function.
2. Turn each `.then((value) => ...)` into `const value = await ...;` on its own line.
3. Wrap the steps in `try`. Turn `.catch` into `catch`, and `.finally` into `finally`.

Look at how the `.then` version needed `return wait(1000).then(() => food)`, just to carry `food` along to the next step. With `await`, `food` is an ordinary variable that's there the whole time.

Common mistakes

1. Forgetting `await`

```

async function lunch() {
  const food = orderFood("burger"); // forgot await
  console.log(food);                 // prints: Promise { <pending> }
  console.log(`Enjoy your ${food}!`); // prints: Enjoy your [object Promise]!
}

lunch();

```

Without `await`, you get the buzzer instead of the food: the promise itself, still pending. If you see `Promise { <pending> }` or `[object Promise]` in your output, a missing `await` is almost always the reason. Fix: `const food`


```
= await orderFood("burger");
```

2. Using `await` inside `forEach`

```
async function playAll() {  
  playlist.forEach(async (song) => {  
    await wait(1000);  
    console.log(`Played: ${song}`);  
  });  
  console.log("Playlist finished!");  
}
```

You'll see:

```
Playlist finished!  
Played: Intro  
Played: Chorus  
Played: Outro
```

"Playlist finished!" comes first, and then all three songs appear together, about a second later. `forEach` calls your `async` callback for each song, but it doesn't wait for any of them. Fix: use a `for...of` loop, like in "Loops" above.

(If you *want* them all at once, like downloading three songs together, use `await Promise.all(playlist.map(async (song) => { ... })).map` gives you an array of promises, and `Promise.all` waits for all of them.)

3. Using `await` in a function that isn't `async`

```
function lunch() {  
  const food = await orderFood("soup");  
  console.log(`Enjoy your ${food}!`);  
}  
  
// SyntaxError: await is only valid in async functions and the top level bodies of modules
```

`await` only works inside an `async` function (or at the top level of a module). Because it's a syntax error, none of the file runs at all. Fix: `async function lunch() { ... }`.

4. Waiting one by one when you don't need to

```
// inside an async function  
const weather = await getWeather();  
const news = await getNews(); // doesn't need the weather!  
const calendar = await getCalendar(); // doesn't need either of them!
```

Each line waits for the one before, so the times add up. When the steps don't depend on each other, start them together: `const [weather, news, calendar] = await Promise.all([getWeather(), getNews(), getCalendar()]);`.

5. Not handling errors

```
async function lunch(item) {
  const food = await orderFood(item);
  console.log(`Enjoy your ${food}!`);
}

lunch("pizza"); // nobody handles the error
```

The error from `orderFood` makes `lunch`'s promise reject, and nobody catches it. Node prints `Error: Sorry, we don't have pizza today` and stops your program, just like the unhandled rejections in chapter 31. Fix: add a `try / catch` inside `lunch`, or call it with `lunch("pizza").catch(...)`.

6. Mixing `.then` and `await` in one function

```
async function lunch() {
  await orderFood("salad")
    .then((food) => console.log(`Enjoy your ${food}!`))
    .catch((error) => console.log("Problem:", error.message));
  console.log("Lunch is over");
}
```

This works, but it uses two styles at once, so the reader has to keep switching between them. Fix: pick one. Inside an `async` function, use `await` with `try / catch`:

```
async function lunch() {
  try {
    const food = await orderFood("salad");
    console.log(`Enjoy your ${food}!`);
  } catch (error) {
    console.log("Problem:", error.message);
  }
  console.log("Lunch is over");
}
```

Quick recap

- An `async` function always returns a promise. Returning a value fulfills it, and throwing an error rejects it.

- `await` pauses only its own `async` function until the promise settles, then gives you the value (or throws the error). The rest of your program keeps running.
 - Handle errors with the usual `try / catch / finally` around your `await` s.
 - If a step needs the result of the one before, `await` them one by one. If not, start them together and `await Promise.all([...])`.
 - To `await` inside a loop, use `for...of`, not `forEach`.
 - Top-level `await` only works in modules (`"type": "module"` in `package.json`).
 - `Promise { <pending> }` or `[object Promise]` in your output usually means a missing `await`.
-

Next: try the [exercises](#), then move on to [33 Fetch and APIs](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on) and run it with `node ex1.js`.
 - These exercises call their `async` functions normally, so you don't need a `package.json`. (If you add one with `"type": "module"` to try top-level `await`, that's fine too. Everything here still works.)
 - Timers pretend that work takes time. Watch *when* each line appears, not just what it says.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): From chain to `await`

Your morning routine app was written with a `.then` chain. Copy it into `ex1.js` and run it once to see what it does:

```

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function morningRoutine() {
  console.log("Alarm rings");
  return wait(1000)
    .then(() => {
      console.log("Shower done");
      return wait(2000);
    })
    .then(() => {
      console.log("Breakfast eaten");
      return wait(1000);
    })
    .then(() => {
      console.log("Out the door!");
    });
}

morningRoutine();

```

Now rewrite `morningRoutine` as an `async` function that uses `await`. It must print exactly the same lines, at the same moments (straight away, then after about 1, 3 and 4 seconds):

```

Alarm rings
Shower done
Breakfast eaten
Out the door!

```

Rule: no `.then` anywhere in your version of `morningRoutine`.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Bug hunt at the pizza shop

This pizza tracker has **3 bugs**. Copy it into `ex2.js`, run it, and fix one bug at a time.

```
// Pizza order tracker
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function getStatus(orderNumber) {
  await wait(1000);
  return `Order ${orderNumber}: out for delivery`;
}

async function checkAll(orderNumbers) {
  orderNumbers.forEach(async (orderNumber) => {
    const status = getStatus(orderNumber);
    console.log(status);
  });
  console.log("All orders checked");
}

checkAll([41, 42, 43]);
```

When it's fixed, you should see one line about every second, and the last line straight after the third:

```
Order 41: out for delivery
Order 42: out for delivery
Order 43: out for delivery
All orders checked
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Cinema box office

A cinema's booking system takes about half a second to reserve each seat. Start with this code:

```
const seats = { A1: "free", A2: "taken", A3: "free" };

function reserveSeat(seat) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (seats[seat] === undefined) {
        reject(new Error(`Seat ${seat} doesn't exist`));
        return;
      }
      if (seats[seat] === "taken") {
        reject(new Error(`Seat ${seat} is already taken`));
        return;
      }
      seats[seat] = "taken";
      resolve(`Seat ${seat} is yours`);
    }, 500);
  });
}
```

1. Write `async` function `buyTicket(seat)`. Using `try / catch / finally`, it:
 - prints the message from `reserveSeat` and returns `true` when it works
 - prints `Sorry: <error message>` and returns `false` when it doesn't
 - always prints `Next in line, please.`
2. Write `async` function `buyTickets(seatList)`. It buys the tickets one at a time, counts how many worked, and finally prints `Done: 2 of 4 seats booked` (with the real numbers).
3. Call `buyTickets(["A1", "A2", "B7", "A3"]);`

Expected output (a pair of lines about every half second):

```
Seat A1 is yours
Next in line, please.
Sorry: Seat A2 is already taken
Next in line, please.
Sorry: Seat B7 doesn't exist
Next in line, please.
Seat A3 is yours
Next in line, please.
Done: 2 of 4 seats booked
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Morning dashboard

Your phone's morning dashboard shows the weather, the news and your calendar. But it needs to know who you are first. Start with these pretend loaders:

```
function getUser() {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ name: "Maya", city: "Lisbon" }), 1000);
  });
}

function getWeather(city) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`22°C and sunny in ${city}`), 2000);
  });
}

function getNews() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("3 new headlines"), 1000);
  });
}

function getCalendar(name) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`${name} has 2 meetings today`), 1500);
  });
}
```

Write `async function loadDashboard()`. It gets the user first, then loads the other three **at the same time**. Print a greeting as soon as you have the user, then the three results, then how long it took in whole seconds (with `Date.now()` and `Math.round`, like in chapter 31's exercises).

Expected output:

```
Hi Maya!
22°C and sunny in Lisbon
3 new headlines
Maya has 2 meetings today
Dashboard ready in about 3 seconds
```

`Hi Maya!` appears after about 1 second, and the rest about 2 seconds after that.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: change your function to load all four one by one. How long does the dashboard take now? Imagine waiting for that every morning.

Exercise 5 (Challenge): Online shop order queue

An online shop processes its orders one at a time, so that two customers can never buy the same last item. Start with this data:

```
const orders = [
  { id: 101, item: "Laptop stand", price: 35 },
  { id: 102, item: "Desk lamp", price: 20 },
  { id: 103, item: "Office chair", price: 150 },
  { id: 104, item: "Office chair", price: 150 },
  { id: 105, item: "Monitor", price: 180 },
];

const stock = { "Laptop stand": 3, "Desk lamp": 0, "Office chair": 1, "Monitor": 2 };
```

1. Write `async function processOrder(order)`. It waits about half a second (use your `wait` helper). If there's none of the item left in `stock`, it **throws** an `Error` like `Desk lamp is out of stock`. Otherwise, it takes one off the stock and returns the order.
2. Write `async function processAll(orderList)`. It:
 - prints `Processing 5 orders...` (with the real number)
 - processes the orders one at a time
 - prints `Order 101: Laptop stand shipped ($35)` for each success, or `Order 102 failed: Desk lamp is out of stock` for each failure, and **keeps going** after a failure
 - finally prints how many were shipped and the total money taken
3. Call `processAll(orders)`;

Expected output (one order about every half second):

```
Processing 5 orders...
Order 101: Laptop stand shipped ($35)
Order 102 failed: Desk lamp is out of stock
Order 103: Office chair shipped ($150)
Order 104 failed: Office chair is out of stock
Order 105: Monitor shipped ($180)
Shipped 3 of 5 orders. Revenue: $365
```

Rules:

- Work out the revenue with `reduce` ([chapter 13](#)). Don't type 365 yourself.

- Use destructuring ([chapter 15](#)) at least once, for example to unpack `id`, `item` and `price` from an order.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: after the summary, print what's left in stock, one line per item, using `Object.entries` ([chapter 11](#)):

```
Laptop stand: 2 left
Desk lamp: 0 left
Office chair: 0 left
Monitor: 1 left
```

Before you move on

Every "slow" function in this chapter was pretend: a timer standing in for real work. In the next chapter, the waiting becomes real. You'll use `await` to get live data from real servers on the internet, like weather forecasts.

Head to [chapter 33: Fetch and APIs](#).

33 Fetch and APIs

What is it?

An **API** (Application Programming Interface) is a set of rules for how one program can ask another program for data, or ask it to do something. On the web, that usually means sending a request to a server's address and getting data back.

`fetch` is JavaScript's built-in function for sending those requests. It works in every modern browser, and in Node.

Why does it matter?

Almost every app you use is built on APIs:

- A weather app asks a weather service for today's forecast.
- A shop page asks the shop's server for products and prices.
- A map loads new places as you drag it around.
- When you post a comment, the page sends your text to a server to be saved.

Until now, all your data has been typed into your own code. With `fetch` and `async / await` from [chapter 32](#), your programs can use live data from anywhere on the internet, and send data back.

Real-world example

An API is like a waiter in a restaurant. You don't walk into the kitchen and cook your own food. You look at the menu, tell the waiter what you'd like, and the waiter brings it to you from the kitchen. If the kitchen can't make it, the waiter comes back and tells you.

At the restaurant	With a web API
The menu	The API's documentation: what you're allowed to ask for
Your order: "the soup, please"	A request : a method and an address (sometimes with data)
The waiter	The API
The kitchen	The server and its database, which you never see
Your food on a plate	The response , usually data in JSON
"Sorry, we're out of soup"	An error code, like 404

You never need to know how the kitchen works. You only need to know how to order.

How it works

Before you start

- Node has had `fetch` built in since version 18, so your Node 24 is ready. You need an internet connection, of course.
- The examples use top-level `await`, so add a `package.json` file containing `{ "type": "module" }` to this folder ([chapter 29](#), [chapter 32](#)).
- The examples use two free APIs that need no sign-up and no key:
 - **JSONPlaceholder** (`jsonplaceholder.typicode.com`): fake users, posts and to-dos, made for practice. Its data never changes, so your output will match the notes exactly.
 - **Open-Meteo** (`open-meteo.com`): real weather. Your numbers will be different, because the weather changes!

URLs, endpoints and query parameters

Every request goes to a **URL** (a web address). Here's one, taken apart:

```
https://jsonplaceholder.typicode.com/posts?userId=1
```

Part	Example	What it is
protocol	<code>https://</code>	how to talk (the <code>s</code> means secure)
host	<code>jsonplaceholder.typicode.com</code>	which server
path	<code>/posts</code>	what you want from it
query string	<code>?userId=1</code>	extra options, after the <code>?</code>

An **endpoint** is one address in an API that gives you one kind of thing. JSONPlaceholder has endpoints like these:

Endpoint	Gives you
<code>/users</code>	all 10 users
<code>/users/1</code>	the user with id 1
<code>/posts?userId=1</code>	the posts written by user 1

Endpoint	Gives you
<code>/todos?userId=1</code>	user 1's to-do items

Query parameters are the options in the query string, written as `key=value` and joined with `&`, like `?latitude=27.7&longitude=85.32`.

Building URLs safely with `URL` and `URLSearchParams`

Some characters can't go into a URL as they are, like spaces, `&`, or letters like `ã`. They have to be **encoded**: turned into safe codes. The built-in `URL` object does this for you:

```
const url = new URL("https://geocoding-api.open-meteo.com/v1/search");
url.searchParams.set("name", "São Paulo");
url.searchParams.set("count", 1);

console.log(url.href);
// prints: https://geocoding-api.open-meteo.com/v1/search?name=S%C3%A3o+Paulo&count=1
```

The space became `+`, and `ã` became `%C3%A3`. `url.searchParams` is a **`URLSearchParams`** object: a handy list of query parameters that you can `set`, `get` and `delete`.

Why not glue the URL together with a template literal? Watch what happens with a recipe search:

```
const dish = "mac & cheese";

console.log(`https://example.com/recipes?search=${dish}`);
// prints: https://example.com/recipes?search=mac & cheese

const url = new URL("https://example.com/recipes");
url.searchParams.set("search", dish);
console.log(url.href);
// prints: https://example.com/recipes?search=mac+%26+cheese
```

In the first URL, the `&` starts a new parameter, so the server would only see `search=mac`. The `URL` version encodes `&` as `%26`, and the whole dish arrives safely.

You can also build a query string straight from an object:

```
const params = new URLSearchParams({ latitude: 27.7, longitude: 85.32, current: "temperature_2m" });
console.log(params.toString());
// prints: latitude=27.7&longitude=85.32&current=temperature_2m
```

HTTP in a nutshell

HTTP is the set of rules that browsers and servers use to talk to each other. Every conversation is a request followed by a response.

A **request** has:

- a **method**: what you want to do (read something, create something...)
- a **URL**: what you want to do it to
- **headers**: extra labels about the request, like the labels on a parcel ("the contents are JSON")
- sometimes a **body**: data you're sending, usually as JSON ([chapter 23](#))

A **response** has a **status code** (a number that says how it went), headers, and a body with the data.

The five methods you'll meet most:

Method	Means	At the restaurant
GET	read something	"What's today's special?"
POST	create something new	"I'd like to place an order"
PUT	replace something completely	"Swap my whole order for this one"
PATCH	change part of something	"Same order, but no onions"
DELETE	remove something	"Cancel my order"

And the status codes you'll see most:

Code	Name	Means
200	OK	It worked. Here's your data.
201	Created	It worked. Your new thing was created.
400	Bad Request	Your request doesn't make sense (a typo, a missing or wrong value).
401	Unauthorized	You need to log in first, or your key is wrong.
404	Not Found	There's nothing at that address.
500	Internal Server Error	Something broke on the server.

A handy way to remember them: codes in the **200s** mean success, the **400s** mean something is wrong with *your* request, and the **500s** mean something went wrong on the *server*.

You'll meet real ones in this chapter. JSONPlaceholder answers `404` for a post that doesn't exist, and Open-Meteo answers `400` if you ask for the weather at latitude 999 (there's no such place on Earth), with the reason in the body: `Latitude must be in range of -90 to 90°. Given: 999.0.`

Your first fetch

```
const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
console.log(response.status); // prints: 200
console.log(response.ok);    // prints: true

const user = await response.json();
console.log(user.name);      // prints: Leanne Graham
console.log(user.address.city); // prints: Gwenborough
```

There are two `await` s, one for each part of the answer:

1. `fetch(url)` sends a GET request and returns a promise for a **Response**: an object describing the answer, with its status and headers. It's ready as soon as the server starts answering, before the whole body has arrived.
2. `response.json()` reads the rest of the body and turns the JSON into a JavaScript object, like `JSON.parse` from chapter 23. It returns a promise too, so it needs its own `await`.

`response.ok` is `true` when the status is in the 200s. Try `console.log(user)` to see everything JSONPlaceholder knows about Leanne, including her nested `address` and `company` objects.

`fetch` doesn't fail on a 404

This surprises almost everyone. Ask for a post that doesn't exist:

```
const response = await fetch("https://jsonplaceholder.typicode.com/posts/9999");
console.log(response.status); // prints: 404
console.log(response.ok);    // prints: false
console.log(await response.json()); // prints: {}
```

No error! `fetch` only rejects when it gets no answer at all. A 404 is still an answer. The waiter came back and said "we don't have that", and that counts as a successful trip to the kitchen.

So checking `response.ok` is your job. Here's the pattern you'll write again and again:

```

async function getPost(id) {
  const response = await fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  return response.json();
}

try {
  const post = await getPost(9999);
  console.log(post.title);
} catch (error) {
  console.log(error.message); // prints: Request failed with status 404
}

```

With `getPost(1)`, you'd see the post's title instead (JSONPlaceholder's titles are fake Latin, so it starts `sunt aut facere`).

That `if (!response.ok)` check is the most important habit in this chapter. It turns "the server said no" into a real error that your `catch` can handle.

Network errors: when nothing comes back

When `fetch` can't reach the server at all, because you're offline, the address is wrong, or the server is down, the promise rejects:

```

try {
  await fetch("https://no-such-shop.invalid/products");
} catch (error) {
  console.log(error.message); // prints: fetch failed
  console.log(error.cause.code); // prints: ENOTFOUND
}

```

(Addresses ending in `.invalid` are reserved for examples, so this one can never exist.)

In Node, the message is `fetch failed`, and `error.cause` ([chapter 18](#)) has the details. `ENOTFOUND` means "no server has that name". In the browser, the message is different, like `Failed to fetch` in Chrome.

So there are two different ways a request can go wrong, and a good program handles both:

What happened	What <code>fetch</code> does	How you find out
The server answered with an error (404, 500...)	fulfills, with <code>ok: false</code>	check <code>response.ok</code>
There was no answer (offline, wrong address)	rejects	<code>try / catch</code>

Real data: the weather in Kathmandu

Time for a real API. Open-Meteo needs a place's latitude and longitude, so getting the weather for a city takes two requests: first look up the city, then ask for its weather. Both need the `response.ok` check, so let's turn `getPost` into a helper that works for any URL:

```
async function getJSON(url) {
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Request failed with status ${response.status}`);
  }
  return response.json();
}
```

Step 1: find the city. Open-Meteo's geocoding API turns a place name into coordinates. (`fetch` accepts a `URL` object as well as a string.)

```
const placeUrl = new URL("https://geocoding-api.open-meteo.com/v1/search");
placeUrl.searchParams.set("name", "Kathmandu");
placeUrl.searchParams.set("count", 1);

const places = await getJSON(placeUrl);
const { name, country, latitude, longitude } = places.results[0];
console.log(name, country, latitude, longitude); // prints: Kathmandu Nepal 27.70169 85.3206
```

The answer is an object with a `results` array (here with one place, because of `count=1`). Each place has lots of fields; these four are the ones you need.

Step 2: ask for the weather. The forecast API takes the coordinates, plus a `current` list of what you want to know:

```
const weatherUrl = new URL("https://api.open-meteo.com/v1/forecast");
weatherUrl.searchParams.set("latitude", latitude);
weatherUrl.searchParams.set("longitude", longitude);
weatherUrl.searchParams.set("current", "temperature_2m,wind_speed_10m");

const weather = await getJSON(weatherUrl);
console.log(weather.current);
```

You'll see something like this (it's live weather, so your numbers will be different):


```
{
  time: '2026-09-24T11:15',
  interval: 900,
  temperature_2m: 20.5,
  wind_speed_10m: 3.2
}
```

`temperature_2m` is the temperature 2 metres above the ground, in °C, and `wind_speed_10m` is the wind speed in km/h. (The units are in `weather.current_units`, and the `time` is in UTC unless you add `timezone=auto`.) Now make it friendly:

```
const { temperature_2m, wind_speed_10m } = weather.current;
console.log(`${name}, ${country}: ${temperature_2m}°C, wind ${wind_speed_10m} km/h`);
// prints something like: Kathmandu, Nepal: 20.5°C, wind 3.2 km/h
```

Try your own city!

Watch out: if no place matches, the geocoding API still answers `200 OK`, but with no `results` at all, just `{ generationtime_ms: 0.25 }` (roughly). `response.ok` can't warn you, and `places.results[0]` crashes with `TypeError: Cannot read properties of undefined (reading '0')`. Check first, with `if (places.results === undefined)`, and show a friendly "city not found" message.

Sending data: POST with a JSON body

So far, every request has been a GET: "please give me something". To *create* something, like a new blog post, you send a POST with the data in the body. `fetch` takes a second argument for this: an **options object** ([chapter 15](#)).

```
const newPost = { title: "My trip to Rome", body: "The pasta was amazing!", userId: 1 };

const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(newPost),
});

console.log(response.status); // prints: 201
console.log(await response.json());
```

You'll see:

```
201
{
  title: 'My trip to Rome',
  body: 'The pasta was amazing!',
  userId: 1,
  id: 101
}
```

Three things make it work:

- `method: "POST"` says what you want to do.
- The `Content-Type: application/json` header tells the server "the body is JSON". Without it, many servers don't know how to read your data.
- `body` must be a string, so `JSON.stringify` turns your object into JSON text first.

The server answers `201 Created` and sends back the new post with its `id`. (JSONPlaceholder only pretends: it answers as if it saved your post, but nothing is really stored. A real API would keep it.)

`PUT` and `PATCH` look just the same, with their own method name. For example, `method: "PATCH"` with the body `JSON.stringify({ title: "Just a new title" })` changes only post 1's title, and answers `200`. `DELETE` usually needs no body at all:

```
const response = await fetch("https://jsonplaceholder.typicode.com/posts/1", { method: "DELETE"
});
console.log(response.status); // prints: 200
```

Sending your own headers

`Content-Type` isn't the only header you can send. The `headers` object can hold any headers you like, with any method. Some common ones:

Header	What it tells the server
<code>Authorization</code>	Who you are, like <code>"Bearer abc123"</code> (a login token)
<code>Accept</code>	Which format you'd like back, like <code>"application/json"</code>
<code>Accept-Language</code>	Which language you'd like, like <code>"ne"</code> or <code>"en"</code>
<code>X-Something</code>	Anything your own API needs. Custom headers often start with <code>x-</code> .

To see headers in action, use **httpbin.org**, a free test service. Its `/headers` address sends back the headers it received, like a mirror:

```
const response = await fetch("https://httpbin.org/headers", {
  headers: {
    Authorization: "Bearer my-token-123",
    "X-App-Version": "1.4.0",
  },
});

const data = await response.json();
console.log(data.headers.Authorization); // prints: Bearer my-token-123
console.log(data.headers["X-App-Version"]); // prints: 1.4.0
```

Header names with a `-` need quotes in the object, just like any object key with a dash ([chapter 11](#)). Header names don't care about capital letters: `authorization` and `Authorization` are the same header.

Watch out: it's fine to send a token the server gave the logged-in user. But never type a *secret* key straight into front-end code: anyone can read it (see "API keys" below, and [chapter 51](#)).

Reading the server's headers

Responses have headers too, and `response.headers.get()` reads one. Here, the server says what kind of data it sent:

```
const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");

console.log(response.headers.get("Content-Type")); // prints: application/json; charset=utf-8
console.log(response.headers.get("X-Not-There")); // prints: null
console.log(response.headers.has("content-type")); // prints: true
```

`get` returns `null` when the header isn't there. Real uses: checking that the response really is JSON before calling `.json()`, or reading headers some APIs use to say how many requests you have left.

Loading and error states on a web page

On a real page, a request can take a second or two, and sometimes it fails. If nothing happens on the screen meanwhile, people think the page is broken and click again and again. A good page shows three **states**:

1. **Loading:** a "Loading..." message, and the button switched off so it can't be clicked twice.
2. **Success:** the data.
3. **Error:** a friendly message, instead of a blank page.

Say the page has `<button id="load">Load user</button>` and `<p id="message"></p>`, and the script also includes the `getJSON` helper from above:

```

const loadButton = document.querySelector("#load");
const message = document.querySelector("#message");

loadButton.addEventListener("click", async () => {
  loadButton.disabled = true;
  message.textContent = "Loading...";
  try {
    const user = await getJSON("https://jsonplaceholder.typicode.com/users/1");
    message.textContent = `Hello, ${user.name}!`;
  } catch (error) {
    message.textContent = "Sorry, something went wrong. Please try again.";
  } finally {
    loadButton.disabled = false;
  }
});

```

- Before the request: show "Loading..." and set `disabled` to `true` (the button goes grey and ignores clicks).
- `try`: show the data. `catch`: show a friendly error, for a 404 and for "no connection" alike.
- `finally` switches the button back on, whichever way it went ([chapter 31](#) used `.finally` for exactly this).

Opening the page by double-clicking `index.html` works fine here, because these APIs allow requests from any page. That's thanks to CORS.

CORS in two sentences

CORS (Cross-Origin Resource Sharing) is a browser safety rule: a page may only read a response from *another* website if that website allows it, with a response header called `Access-Control-Allow-Origin`. JSONPlaceholder and Open-Meteo allow everyone, but if your console ever says a request was "blocked by CORS policy", the fix is on the server's side, not in your `fetch` code.

(Node doesn't apply this rule. That's why your Node scripts never run into it.)

Headers and the browser's rules

The browser adds two rules about request headers. Node has neither.

1. Some headers are off-limits. The browser sets headers like `Cookie`, `Host`, `Origin` and `Referer` itself, so your page can't fake them. If you try, the browser ignores your value without an error.

2. Custom headers to another website need permission first. A simple request, like a plain GET, goes straight out. But when you send a custom header like `X-App-Version` (or `Authorization`) to another website, the browser first sends a small "may I?" request, called a **preflight**. It's like phoning a restaurant to ask whether they take card before you go:

```
Browser → Server: "May I send a request with the header X-App-Version?" (preflight)
Server → Browser: "Yes, that header is allowed."
Browser → Server: the real request
```

If the server doesn't say yes, the real request never happens, and the console shows a CORS error. You'll see the preflight in the DevTools **Network** tab as an extra request with the method `OPTIONS`. As with all CORS problems, the fix is on the server: it has to allow your header.

Cancelling a request: `AbortController`

Sometimes you want to give up on a request: the user closed the pop-up, typed a new search, or the server is taking forever. An `AbortController` is like a remote control with one button, "abort". You pass its `signal` to `fetch`, and pressing the button cancels the request:

```
const controller = new AbortController();

const request = fetch("https://jsonplaceholder.typicode.com/posts", {
  signal: controller.signal,
});
controller.abort(); // changed my mind!

try {
  await request;
} catch (error) {
  console.log(error.name); // prints: AbortError
  console.log(error.message); // prints: This operation was aborted
}
```

A cancelled `fetch` rejects with an error named `AbortError`, so your `catch` can tell "I cancelled it" apart from a real failure.

For a time limit, there's a shortcut: `AbortSignal.timeout(ms)` gives you a signal that aborts by itself after that many milliseconds.

```

try {
  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    signal: AbortSignal.timeout(5000), // give up after 5 seconds
  });
  console.log(response.status); // prints: 200
} catch (error) {
  if (error.name === "TimeoutError") {
    console.log("The server took too long. Please try again.");
  } else {
    console.log("Something went wrong:", error.message);
  }
}
}

```

JSONPlaceholder answers well within 5 seconds, so you'll see `200`. Change the limit to `1` millisecond, and you'll see the "took too long" message instead. Unlike the `Promise.race` time limit in chapter 31, this really stops the request.

Retrying with backoff

Networks hiccup. A request that fails now might work a moment later. A common pattern is to try again a few times, waiting a little longer after each failure. That growing wait is called **backoff**:

```

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function getJSONWithRetry(url, attempts = 3) {
  for (let attempt = 1; attempt <= attempts; attempt++) {
    try {
      return await getJSON(url); // "await" here, so the catch below sees failures
    } catch (error) {
      if (attempt === attempts) {
        throw error; // out of tries: give up
      }
      const delay = 500 * 2 ** (attempt - 1); // 500 ms, then 1000 ms, then 2000 ms...
      console.log(`Attempt ${attempt} failed. Trying again in ${delay} ms...`);
      await wait(delay);
    }
  }
}

try {
  await getJSONWithRetry("https://no-such-shop.invalid/products");
} catch (error) {
  console.log("Giving up:", error.message);
}

```

You'll see:

```
Attempt 1 failed. Trying again in 500 ms...
Attempt 2 failed. Trying again in 1000 ms...
Giving up: fetch failed
```

Look at `return await getJSON(url)`. Without that `await`, the function would hand back the promise straight away, before it fails, so the `catch` would never run and nothing would be retried.

Only retry when trying again might help: network errors, timeouts, and 500s. Don't retry a 404: the missing post won't appear by asking again.

API keys: keep secrets secret

Many APIs need an **API key**: a long secret code that identifies your app, a bit like a password. The APIs in this chapter don't need one, but you'll meet them soon.

Never put a secret key in front-end code. Anyone can open DevTools on your page and read it in seconds, and then use your account (and maybe your money). Keep secret keys on a server, and let your server talk to the API. [Chapter 51](#) covers this properly.

Common mistakes

1. Not checking `response.ok`

```
const response = await fetch("https://jsonplaceholder.typicode.com/users/11");
const user = await response.json();
console.log(user.name); // prints: undefined
```

There are only 10 users, so this is a 404, and the body is `{}`. But nothing complains, so the program carries on with empty data. Fix: check `response.ok` and throw if it's `false`, like `getJSON` does.

2. Forgetting `await` on `response.json()`

```
const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
const user = response.json(); // forgot await
console.log(user);           // prints: Promise { <pending> }
console.log(user.name);      // prints: undefined
```

Reading the body takes time too, so `response.json()` returns a promise. Fix: `const user = await response.json();`

3. Forgetting `JSON.stringify`

```
const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: newPost, // forgot JSON.stringify
});
console.log(response.status, response.statusText); // prints: 500 Internal Server Error
```

The body has to be text. Given an object, `fetch` turns it into the text `[object Object]`, which isn't JSON, and JSONPlaceholder's server chokes on it. Fix: `body: JSON.stringify(newPost)`.

4. Forgetting the `Content-Type` header

```
const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  body: JSON.stringify(newPost), // no Content-Type header
});
console.log(await response.json()); // prints: { id: 101 }
```

It still says `201 Created`, but your title and body are gone. Without the header, the server didn't know the body was JSON, so it ignored it. Fix: add `headers: { "Content-Type": "application/json" }`.

5. Reading the body twice

```
const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
console.log(await response.json()); // works
console.log(await response.json()); // fails
// TypeError: Body is unusable: Body has already been read
```

A response's body can only be read once. After that, it's used up. Fix: read it once, store the result in a variable, and use the variable as often as you like.

On Windows: when a script crashes after using `fetch`, Node 24 may print one extra line at the very end, starting with `Assertion failed: !(handle->flags & UV_HANDLE_CLOSING)`. That's a quirk of Node on Windows, not a bug in your code. The real error is the one above it.

Quick recap

- An API lets your program ask another program for data. On the web, that's a **request** (method, URL, headers, maybe a body) and a **response** (status code, headers, body).

- `const response = await fetch(url)` gets the response, and `await response.json()` turns its body into an object.
 - `fetch` only rejects when there's no answer at all. A 404 or 500 still fulfills, so always check `response.ok`.
 - Build URLs with `URL` and `URLSearchParams`, so every value is encoded safely.
 - To send data, give `fetch` a `method`, a `Content-Type: application/json` header, and a `JSON.stringify`-ed `body`.
 - The `headers` option sends any headers you like, and `response.headers.get(name)` reads the server's. In the browser, some headers are off-limits, and custom headers to other sites trigger a CORS preflight.
 - On a page, show loading and error states. Cancel with `AbortController` or `AbortSignal.timeout`, and retry with backoff only when it can help.
 - Never put secret API keys in front-end code.
-

Next: try the [exercises](#), then move on to [34 Debounce and Throttle](#).

Exercises

How to do these:

- You need an internet connection for all of them.
 - Exercises 1 to 4 run in Node. Add a `package.json` file containing `{ "type": "module" }` to this folder (once), so you can use top-level `await`. Then make a file for each exercise (`ex1.js`, `ex2.js`, and so on) and run it with `node ex1.js`.
 - Exercise 5 runs in the browser. Make a folder `ex5` with `index.html` and `script.js` inside, and double-click `index.html` to open it. It's a normal script, not a module, so you don't need Live Server, but you can't use top-level `await` there. Press `F12` → **Console** to see errors, and refresh after each change.
 - JSONPlaceholder's data never changes, so those outputs must match exactly. Open-Meteo's weather is live, so your numbers will be different.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Contact card

Your address book app needs the details of user number 3. Fetch `https://jsonplaceholder.typicode.com/users/3` and print a small contact card.

Expected output:

Name: Clementine Bauch
Email: Nathan@yesenia.net
City: McKenziehaven
Company: Romaguera-Jacobson

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Check before you trust

A to-do app shows tasks by their number. Some numbers don't exist, and the app must say so instead of showing an empty task.

1. Write `async function getTodo(id)` that fetches `https://jsonplaceholder.typicode.com/todos/<id>`. If `response.ok` is `false`, it throws an `Error` with the message `not found (status 404)` (with the real status). Otherwise, it returns the to-do.
2. Write `async function showTodo(id)` that prints the to-do, or the error message, like in the expected output.
3. Show to-dos 1, 4 and 999, one after the other.

Expected output:

```
Todo 1: "delectus aut autem" (not done yet)
Todo 4: "et porro tempora" (done)
Todo 999: not found (status 404)
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Post a book review

A book club website lets members post reviews. Send this review to JSONPlaceholder's `/posts` endpoint:

```
const review = { title: "Dune: a desert classic", body: "Slow start, amazing ending. 4 out of 5.", userId: 7 };
```

1. Write `async function saveReview(review)`. It sends a `POST` request with the review as JSON, throws an error if `response.ok` is `false`, prints the status, and returns the saved review that the server sends back.
2. Call it and print the saved review's `id` and `title`.

Expected output:

```
Status: 201
Saved review #101: "Dune: a desert classic"
```

Then try this experiment: delete the `headers` line and run it again. You should see:

```
Status: 201
Saved review #101: "undefined"
```

The server said "Created", but it ignored your data, because it didn't know the body was JSON. Put the header back when you're done.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: custom headers. Real book club sites need to know *who* is posting. Send a GET request to `https://httpbin.org/headers` with two headers: `Authorization` set to `Bearer member-token-77`, and a custom `X-Club-Name` set to `Sunday Readers`. The server mirrors back what it received. Print both headers from its answer, and the response's own `Content-Type` header:

```
Authorization: Bearer member-token-77
X-Club-Name: Sunday Readers
Server sent: application/json
```

Exercise 4 (Medium): Weather for any city

Turn the Kathmandu example from the notes into a reusable weather function.

1. Write `async function getWeather(city)`. It looks up the city with Open-Meteo's geocoding API, then gets the current temperature and wind speed from the forecast API, and **returns** a line like `Kathmandu, Nepal: 20.5°C, wind 3.2 km/h`.
2. If no place matches, it throws an `Error` with the message `no place called "Nowhereville"` (with the real name).
3. Get the weather for `Kathmandu`, `Lisbon` and `Nowhereville`, one after the other, using a `for...of` loop. Print each line, or `Sorry, <error message>` if it fails, and keep going.

Expected output (live weather, so your numbers will be different):

```
Kathmandu, Nepal: 20.5°C, wind 3.2 km/h
Lisbon, Portugal: 28°C, wind 1.8 km/h
Sorry, no place called "Nowhereville"
```

Rule: build both URLs with `new URL(...)` and `searchParams.set(...)`, not by gluing strings together.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: try a few place names of your own. Is there a town called Atlantis? (There is!)

Exercise 5 (Challenge): Customer lookup page

A support team wants a page to look up a customer by id and see how they're getting on with their to-dos. This one runs in the browser.

Create `ex5/index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>User Lookup</title>
  <script src="script.js" defer</script>
</head>
<body>
  <h1>User lookup</h1>
  <form id="lookup-form">
    <label for="user-id">User id (1 to 10):</label>
    <input id="user-id" type="number" min="1" value="1">
    <button id="lookup-button">Look up</button>
  </form>
  <p id="status"></p>
  <div id="card"></div>
</body>
</html>
```

Write `ex5/script.js` so that when the form is submitted:

1. The page doesn't reload ([chapter 22](#)). Read the id from the input, and remember that it's a string.
2. While it's loading, the button is disabled, `#status` says `Loading...`, and `#card` is emptied.
3. It fetches the user (`/users/<id>`) and their to-dos (`/todos?userId=<id>`) **at the same time**, each with a 5-second time limit.
4. On success, `#status` is emptied, and `#card` gets three new elements: an `<h2>` with the name, a `<p>` saying `Lives in <city>`, works at `<company>`, and a `<p>` saying `To-dos done: <done> of <total>`.
5. On failure, `#status` says:
 - `No user with id 42.` for a 404 (with the real id)
 - `The server is taking too long. Please try again.` for a timeout
 - `Something went wrong. Check your connection and try again.` for anything else

6. Either way, the button is switched back on at the end.

What you should see:

You look up	You see
1	Leanne Graham , Lives in Gwenborough, works at Romaguera-Crona, To-dos done: 11 of 20
3	Clementine Bauch , Lives in McKenziehaven, works at Romaguera-Jacobson, To-dos done: 7 of 20
42	No user with id 42.

To test the "no connection" message, switch off your Wi-Fi, or in DevTools open the **Network** tab and change "No throttling" to **Offline**. Then click **Look up**.

Rule: put the data on the page with `createElement` and `textContent`, not `innerHTML` (remember the warning in [chapter 20](#)).

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Your lookup page only sends a request when someone clicks the button. Now imagine it searched *as you type*. Typing "Clementine" would fire ten requests in about a second, and the answers could even arrive in the wrong order.

[Chapter 34: Debounce and Throttle](#) fixes exactly that. Debounce waits until you stop typing, and `AbortController` cancels the requests you no longer need. 😊

34 Debounce and Throttle

What is it?

Debounce and **throttle** are two ways to control how often a function runs when something keeps triggering it, very quickly, over and over.

- **Debounce:** wait until the calls stop for a moment, then run once.
- **Throttle:** run at most once every X milliseconds, however often you're called.

Both are small helper functions. You'll build them yourself in this chapter, using closures ([chapter 25](#)) and timers ([chapter 30](#)).

Why does it matter?

Some events fire far more often than you need:

- **Typing in a search box.** Type "pizza delivery" and that's 14 `input` events. If each one sends a request to a server, you send 14 requests, and only the last one matters.
- **Scrolling and resizing.** A `scroll` event can fire dozens of times a second. If each one does heavy work, the page stutters.
- **Moving the mouse.** Same story: a flood of `mousemove` events.
- **A nervous double-click on "Pay".** Without protection, the order can go through twice.

Letting all of that through wastes the user's battery and data, makes the page feel slow, and can overload your server. Many APIs also have a **rate limit**: a maximum number of requests you're allowed to send per minute. Go over it and they stop answering.

Debounce and throttle fix this with a few lines of code. That's why you'll find them in almost every real web app.

Real-world example

Idea	Everyday version	The rule
Debounce	An elevator door	Every person who walks in restarts the door's countdown. The door only closes once nobody has come in for a few seconds.
Throttle	A city bus	A bus leaves at most once every 10 minutes, no matter how many people turn up at the stop.

Debounce is also like a polite friend who waits until you've finished talking before they reply. They don't answer every half-sentence. They wait for the pause, then answer the whole thing.

In code, the people walking in (or turning up at the stop) are the **events**. The door closing (or the bus leaving) is **your function actually running**.

The name "debounce" comes from electronics. When you press a physical button, the metal contacts bounce and send several quick signals. "Debouncing" treats them as one single press.

How it works

The problem, in code

We'll test everything in Node first, because there you control exactly when each call happens. To pretend a user is typing, we schedule calls with `setTimeout` ([chapter 30](#)), one letter every 100 ms:

```
function search(text) {
  console.log(`Searching for "${text}"...`);
}

// Pretend someone types "pizza", one letter every 100 ms
const word = "pizza";
for (let i = 1; i <= word.length; i++) {
  setTimeout(() => search(word.slice(0, i)), i * 100);
}
```

You'll see:

```
Searching for "p"...
Searching for "pi"...
Searching for "piz"...
Searching for "pizz"...
Searching for "pizza"...
```

Five searches. With a real server, that's five requests, and the first four are wasted: nobody wanted results for "piz".

Picture it on a timeline

Here's the same idea drawn out. Each column is 100 ms. **x** means an event happened, and **•** means your function actually ran:

Time (ms):	0	300	600	900	1200	1500	1800														
Raw events:	x	x	x	x	x	x	x	x	x	.	.	.	.	.	x	x	x	.	.	.	
Debounced:	.	.	.	.	.	.	.	.	.	.	.	.	.	.	•	.	.	.	.	.	•
Throttled:	•	.	.	•	.	.	.	•	.	.	.	.	.	.	.	•	.	.	.	.	.

Both use 300 ms here:

- **Raw events:** a burst of 10 events, a quiet gap, then 3 more. A **burst** is a quick run of events close together.
- **Debounced:** nothing happens during a burst. 300 ms after the *last* event of each burst, the function runs once. Two bursts, two runs.
- **Throttled:** the function runs straight away, then at most once every 300 ms while the events keep coming. It gives you steady updates *during* the burst.

Keep this picture in mind. Everything else in this chapter is about building those two rows.

Building `debounce`, step 1: reset a timer

Here's the elevator door as a plan:

1. When you're called, start a countdown (a `setTimeout`).
2. If you're called again before it finishes, cancel it (`clearTimeout`) and start a new one.
3. When a countdown finally finishes, run the real function.

`debounce` takes your function and a delay, and gives you back a new, "calmer" version of it:


```
function debounce(fn, delay) {
  let timerId; // survives between calls, thanks to the closure

  return function () {
    clearTimeout(timerId); // cancel the countdown from last time
    timerId = setTimeout(fn, delay); // and start a fresh one
  };
}

function search() {
  console.log("Searching...");
}

const debouncedSearch = debounce(search, 300);

// Five quick calls, 100 ms apart
for (let i = 1; i <= 5; i++) {
  setTimeout(() => {
    console.log(`key ${i}`);
    debouncedSearch();
  }, i * 100);
}
```

You'll see:

```
key 1
key 2
key 3
key 4
key 5
Searching...
```

Five calls, one search. Remember closures from [chapter 25](#)? The returned function carries `timerId` in its backpack. Every call sees the *same* `timerId`, so each call can cancel the countdown the previous call started.

Here's what happens to `timerId` over time:

Time	What happens	Countdown ends at
100 ms	Call 1: nothing to cancel, start a 300 ms countdown	400 ms
200 ms	Call 2: cancel it, start a new one	500 ms
300 ms	Call 3: cancel, start again	600 ms
400 ms	Call 4: cancel, start again	700 ms

Time	What happens	Countdown ends at
500 ms	Call 5: cancel, start again	800 ms
800 ms	Nobody canceled it this time: <code>search()</code> runs	

The very first time, `timerId` is `undefined`. That's fine: `clearTimeout(undefined)` does nothing.

Step 2: pass the arguments through

A search needs to know *what* to search for. Right now the arguments get lost, because `setTimeout(fn, delay)` calls `fn` with nothing.

Collect them with a rest parameter (`...args`, [chapter 15](#)), then spread them back out when the timer finishes:

```
function debounce(fn, delay) {
  let timerId;

  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn(...args), delay);
  };
}

function search(text) {
  console.log(`Searching for "${text}"...`);
}

const debouncedSearch = debounce(search, 300);

// "cat" typed quickly, a pause, then "s"
const keystrokes = [
  { text: "c", at: 100 },
  { text: "ca", at: 200 },
  { text: "cat", at: 300 },
  { text: "cats", at: 1000 },
];

for (const { text, at } of keystrokes) {
  setTimeout(() => {
    console.log(`typed "${text}"`);
    debouncedSearch(text);
  }, at);
}
```

You'll see:

```
typed "c"
typed "ca"
typed "cat"
Searching for "cat"...
typed "cats"
Searching for "cats"...
```

The user paused after "cat", so that search ran. Then they typed "s" and paused again, so "cats" ran too. Notice that the search always uses the **latest** arguments: each call replaces the waiting one.

Step 3: keep this

What if the function you debounce is an object's method and uses `this`? Here's a notepad that counts its saves, using the `debounce` from step 2:

```
const notepad = {
  saveCount: 0,
  save: debounce(function (text) {
    this.saveCount++;
    console.log(`Save #${this.saveCount}: "${text}"`);
  }, 300),
};

notepad.save("Buy");
notepad.save("Buy milk");
// prints: Save #NaN: "Buy milk"
```

NaN? Remember from [chapter 26](#) that `this` depends on *how* a function is called. Our debounce calls `fn(...args)` as a plain function call, so `this` isn't `notepad` any more. In a plain `.js` file, it's `globalThis`, which has no `saveCount`, and `undefined + 1` is `NaN`. (In strict mode, such as in modules and classes, it's worse: `TypeError: Cannot read properties of undefined (reading 'saveCount')`.)

The fix is `apply` from chapter 26. It calls a function with the `this` and the arguments you choose:

```
function debounce(fn, delay) {
  let timerId;

  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      fn.apply(this, args); // run fn with the same `this` and arguments
    }, delay);
  };
}
```

Run the notepad again and you'll see `Save #1: "Buy milk"`.

Two details make this work:

- The function we return is a regular `function`, not an arrow. When you call `notepad.save(...)`, it gets `this = notepad`.
- The callback inside `setTimeout` is an arrow. Arrows don't have their own `this`, so it borrows `this` from the function around it: `notepad` again.

This matters in the browser too. In an event listener, `this` is the element (chapter 26), and with `apply`, your debounced listener keeps it.

Step 4: leading or trailing?

Our debounce runs at the *end* of a burst. That's called the **trailing edge**. The *start* of a burst is the **leading edge**. (Think of the front and back edges of a wave.)

Sometimes you want the leading edge: act on the first event straight away, then ignore the rest until things calm down.

```
Time (ms):  0      300    600    900   1200
            |      |      |      |      |
Raw events: x x x x . . . . x x . . . .
Trailing:   . . . . . ● . . . . . ● .
Leading:     ● . . . . . . ● . . . . .
```

Both rows use a 300 ms delay:

- **Trailing** waits for the quiet, then runs. Great when you want the *final* value (search text, a draft to save).
- **Leading** runs instantly on the first event, then ignores the others until there's been 300 ms of quiet. Great for buttons: the user gets instant feedback, and double-clicks don't count twice.

Here's `debounce` with a `leading` option. It uses the options-object pattern from [chapter 15](#), so `debounce(fn, 500)` still works like before:

```
function debounce(fn, delay, { leading = false } = {}) {
  let timerId;

  return function (...args) {
    const wasQuiet = timerId === undefined; // no countdown running?
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      timerId = undefined; // calm again: reset
      if (!leading) fn.apply(this, args); // trailing: run at the end
    }, delay);
    if (leading && wasQuiet) fn.apply(this, args); // leading: run at the start
  };
}
```

What changed:

- When a countdown finishes, `timerId` goes back to `undefined`. So `timerId === undefined` now means "it's been quiet".
- In leading mode, a call that arrives while it's quiet runs straight away. Calls during a burst only restart the countdown, so they're ignored.
- In trailing mode (the default), nothing changed: the function runs when the countdown finishes.

Let's protect an "Add to cart" button from double-clicks:

```
function addToCart(item) {
  console.log(`Added ${item} to the cart`);
}

const safeAddToCart = debounce(addToCart, 500, { leading: true });

// A double-click, then one more click 2 seconds later
for (const at of [100, 200, 2000]) {
  setTimeout(() => {
    console.log("click");
    safeAddToCart("headphones");
  }, at);
}
```

You'll see:

```
click
Added headphones to the cart
click
click
Added headphones to the cart
```

The first click works instantly. The second click, 100 ms later, is ignored. The click 2 seconds later is a new burst, so it counts.

Step 5: cancel a waiting call

Sometimes a waiting call should never happen. The user typed "pizza" and then cleared the search box, or closed the panel. You don't want a search to pop up 300 ms later.

Functions are objects in JavaScript, so you can attach a method to one. Let's give the debounced function a `cancel()` method:

```
function debounce(fn, delay) {
  let timerId;

  function debounced(...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args), delay);
  }

  debounced.cancel = () => {
    clearTimeout(timerId); // throw away the waiting call
  };

  return debounced;
}

function search(text) {
  console.log(`Searching for "${text}"...`);
}

const debouncedSearch = debounce(search, 300);

debouncedSearch("pizza");
console.log("The user cleared the search box");
debouncedSearch.cancel(); // no search for "pizza" will run
// prints: The user cleared the search box
```

That's the only line you'll see. The search never runs.

Your complete `debounce`

Here's everything together: arguments, `this`, the `leading` option, and `cancel()`. This is the version to reuse in your exercises (and in the weather app in [chapter 39](#)):

```
function debounce(fn, delay, { leading = false } = {}) {
  let timerId;

  function debounced(...args) {
    const wasQuiet = timerId === undefined; // no countdown running?
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      timerId = undefined; // calm again: reset
      if (!leading) fn.apply(this, args); // trailing: run at the end
    }, delay);
    if (leading && wasQuiet) fn.apply(this, args); // leading: run at the start
  }

  debounced.cancel = () => {
    clearTimeout(timerId);
    timerId = undefined;
  };

  return debounced;
}
```

Use it like this:

- `debounce(search, 300)` : run once, 300 ms after the calls stop.
- `debounce(addToCart, 500, { leading: true })` : run on the first call, ignore the rest until 500 ms of quiet.
- `debouncedSearch.cancel()` : forget about any waiting call.

Building throttle

Now the bus. Throttle doesn't wait for quiet. It remembers *when* the function last ran, and only lets a call through once enough time has passed.

The simplest version uses a **timestamp**: a moment in time, stored as a number of milliseconds. `Date.now()` gives you the current one ([chapter 19](#)).

```
function throttle(fn, interval) {
  let lastRun = 0; // when fn last ran (a timestamp in ms)

  return function (...args) {
    const now = Date.now();
    if (now - lastRun >= interval) {
      lastRun = now;
      fn.apply(this, args);
    }
    // otherwise: too soon, ignore this call
  };
}
```

Starting `lastRun` at `0` means the first call always runs. `Date.now()` counts the milliseconds since 1970, a huge number, so `now - 0` is always bigger than the interval.

Let's pretend the user scrolls for one second:

```
function updateProgressBar(eventNumber) {
  console.log(`Updating the progress bar (scroll event ${eventNumber})`);
}

const throttledUpdate = throttle(updateProgressBar, 250);

// Pretend the user scrolls for 1 second: 10 scroll events, 100 ms apart
for (let i = 1; i <= 10; i++) {
  setTimeout(() => throttledUpdate(i), i * 100);
}
```

You'll see:

```
Updating the progress bar (scroll event 1)
Updating the progress bar (scroll event 4)
Updating the progress bar (scroll event 7)
Updating the progress bar (scroll event 10)
```

Ten events, four updates, evenly spread out. Here's how the first few calls are decided:

Event	Time	Time since the last run	Runs?
1	100 ms	never ran before	yes
2	200 ms	100 ms	no, too soon
3	300 ms	200 ms	no, too soon

Event	Time	Time since the last run	Runs?
4	400 ms	300 ms	yes (300 is at least 250)
5	500 ms	100 ms	no, too soon

Notice that event 4 runs at 400 ms, not at 350 ms. This throttle can only run when an event arrives. It never runs "on its own".

Watch out: the last event can get lost

The simple throttle has one weakness. Say the user scrolls quickly to the very bottom of the page:

```
function showPosition(percent) {
  console.log(`Progress bar: ${percent}%`);
}

const throttledShow = throttle(showPosition, 250);

// The user scrolls all the way down: 20%, 40%, ... 100%
for (let i = 1; i <= 5; i++) {
  setTimeout(() => throttledShow(i * 20), i * 100);
}
```

You'll see:

```
Progress bar: 20%
Progress bar: 80%
```

The user is at the bottom, but the bar is stuck at 80%. The 100% event arrived "too soon" after the last run, so it was ignored, and no more events came to fix it.

The full fix is a throttle with a **trailing call**: when a call is ignored, it books one final run for the end of the interval. Library versions (you'll meet one soon) do this for you.

A quick fix you can build right now: pair the throttle with a debounce. The throttle gives steady updates while scrolling. The debounce makes one last update when scrolling stops:

```
const throttledShow = throttle(showPosition, 250);
const finalShow = debounce(showPosition, 150);

function onScroll(percent) {
  throttledShow(percent); // steady updates while scrolling
  finalShow(percent);      // one last update when scrolling stops
}
```

Call `onScroll` instead, with the same five events, and you'll see:

```
Progress bar: 20%
Progress bar: 80%
Progress bar: 100%
```

Debounce or throttle: which one do I need?

Side by side:

	Debounce	Throttle
The rule	Wait for a pause, then run once	Run at most once per interval
During a long burst	Doesn't run at all	Runs regularly
When the burst ends	Runs once, with the latest arguments	Nothing more (in the simple version)
Everyday version	Elevator door	City bus
Built with	<code>setTimeout</code> and <code>clearTimeout</code>	A timestamp (<code>Date.now()</code>)

Ask yourself three questions:

1. **Do I only care about the final value, after the user stops?** Use **debounce**.
2. **Do I need regular updates while it's still happening?** Use **throttle**.
3. **Must the first action happen instantly?** Use the **leading** edge. And if it must never, ever happen twice (like a payment), use `once` from [chapter 25](#).

Common situations:

Situation	Use	A good starting delay
Search suggestions as you type	Debounce	300 ms
Autosave a draft	Debounce	1000 ms

Situation	Use	A good starting delay
"Is this username taken?" check	Debounce	500 ms
Redraw a chart after the window is resized	Debounce	200 ms
Progress bar or "back to top" button while scrolling	Throttle	100 ms
Follow the mouse (a tooltip, a drawing app)	Throttle	50 ms
A game's fire button held down	Throttle	the gun's fire rate, e.g. 250 ms
"Add to cart" or "Like" button	Leading debounce	500 ms
"Pay" button	Leading debounce, or <code>once</code> if it must never run twice	1000 ms

The delays are starting points, not rules. Try them, and adjust until the page feels right. Too short and you save little work. Too long and the page feels slow to react.

Using them with real events

In the browser, you wrap the work you want to calm down, and call the wrapped version from your event listener ([chapter 21](#)):

```
const searchBox = document.querySelector("#search");
const debouncedSearch = debounce(search, 300);

searchBox.addEventListener("input", () => {
  debouncedSearch(searchBox.value.trim());
});

const throttledUpdate = throttle(updateProgressBar, 100);
window.addEventListener("scroll", throttledUpdate);
```

The `input` and `scroll` events still fire just as often. Only the expensive work behind them is calmed down.

Because the debounced function keeps `this`, you can also pass a regular function straight in. `this` is then the element that fired the event:

```
searchBox.addEventListener("input", debounce(function () {
  console.log(`Searching for "${this.value}"...`); // `this` is the search box
}, 300));
```

Debounced search without stale results

Debounce cuts down *how many* requests you send. But it can't control the order the answers come back in. Servers don't always answer in order: a request can be slow because of the network, or because the server had more work to do.

That can cause a **race condition**: two things racing each other, where the result depends on which one finishes first. Here, the old search finishes last and "wins", so the page shows a **stale** result: an answer that's out of date by the time it arrives.

Let's see it happen, with a pretend server where short words are slow and longer words are fast:

```
// A pretend search server: short words are slow, longer words are fast
function fakeSearch(query) {
  return new Promise((resolve) => {
    const time = query.length <= 3 ? 1000 : 200;
    setTimeout(() => resolve(`results for "${query}"`), time);
  });
}

async function showResults(query) {
  const results = await fakeSearch(query);
  console.log(`Showing ${results}`);
}

const debouncedShow = debounce(showResults, 300);

setTimeout(() => debouncedShow("cat"), 100); // search starts at ~400 ms, answer at ~1400 ms
setTimeout(() => debouncedShow("cats"), 700); // search starts at ~1000 ms, answer at ~1200 ms
```

You'll see:

```
Showing results for "cats"
Showing results for "cat"
```

The search box says "cats", but the page ends up showing results for "cat". The old, slow answer arrived last and overwrote the right one.

The fix is `AbortController` from [chapter 33](#). Before starting a new search, abort the previous one. An aborted request never delivers its old answer.

First, the pretend server has to listen for "abort", the way `fetch` does. When you call `controller.abort()`, the signal fires an `abort` event:

```
function fakeSearch(query, signal) {
  return new Promise((resolve, reject) => {
    const time = query.length <= 3 ? 1000 : 200;
    const timerId = setTimeout(() => resolve(`results for "${query}"`), time);

    signal.addEventListener("abort", () => {
      clearTimeout(timerId);
      reject(signal.reason); // the same kind of error fetch gives you
    });
  });
}
```

Now the search keeps one controller for the search in progress:

```
let controller; // controls the search that's in progress

async function showResults(query) {
  controller?.abort(); // cancel the previous search, if any
  controller = new AbortController(); // a fresh controller for this search

  try {
    const results = await fakeSearch(query, controller.signal);
    console.log(`Showing ${results}`);
  } catch (error) {
    if (error.name === "AbortError") {
      console.log(`Canceled the old search for "${query}"`);
      return;
    }
    throw error; // a real problem: don't hide it
  }
}
```

With the same two calls as before, you'll see:

```
Canceled the old search for "cat"
Showing results for "cats"
```

A few things to notice:

- The first time, `controller` is `undefined`, so `controller?.abort()` does nothing (optional chaining, [chapter 11](#)).

- An aborted request rejects with an error named `"AbortError"`. That's expected, not a real problem, so we quietly skip it. Any other error is re-thrown, so real problems still show up ([chapter 18](#)).
- Aborting a request that has already finished does nothing, so it's always safe to call.

With real `fetch`, you pass the signal as an option: `fetch(url, { signal: controller.signal })`. An aborted `fetch` also rejects with an `"AbortError"`, so the same `catch` works. You'll use exactly this pattern for city suggestions in the weather app ([chapter 39](#)).

`requestAnimationFrame`: a throttle made for the screen

The browser redraws the page in **frames**, usually about 60 times a second (some screens do 120 or more). Updating something on screen more often than that is wasted work: nobody can see it.

`requestAnimationFrame(callback)` asks the browser to run your function right before it draws the next frame. That makes it a natural throttle for visual updates:

```
let isScheduled = false;

window.addEventListener("scroll", () => {
  if (isScheduled) return; // an update is already booked for the next frame
  isScheduled = true;
  requestAnimationFrame(() => {
    updateProgressBar();
    isScheduled = false;
  });
});
```

However many `scroll` events arrive, the bar updates at most once per frame, and always in time for the screen. Use it for things that move or change on screen. For network requests or saving, stick with debounce and throttle. `requestAnimationFrame` is browser-only: Node doesn't have it. ([Chapter 40](#) shows where it fits in the event loop.)

Ready-made versions

Now that you know how they work, you don't always have to write them yourself. **Lodash**, a popular library of helper functions, has both: `_debounce(fn, 300)` and `_throttle(fn, 100)`. You'll learn to install libraries with npm in [chapter 50](#).

They work like yours, with a few extras:

- Both accept `{ leading, trailing }` options.
- Lodash's `throttle` includes the trailing call by default, so the last event is never lost.
- The functions they return have `cancel()` and `flush()` (run the waiting call right now).

- `debounce` also has a `maxWait` option: "wait for quiet, but never longer than this".

Because you built your own, those options won't feel like magic. They're the same closure and timer ideas with more switches.

Common mistakes

1. Creating a new debounced function on every event

```
searchBox.addEventListener("input", () => {  
  const debouncedSearch = debounce(search, 300); // a brand-new one every time!  
  debouncedSearch(searchBox.value);  
});  
// every keystroke still runs a search (just 300 ms late)
```

Each call to `debounce` makes a new function with its own, fresh `timerId`. There's never an old countdown to cancel, so nothing gets calmed down. Fix: create the debounced function **once**, outside the listener, and call that same one every time:

```
const debouncedSearch = debounce(search, 300);  
  
searchBox.addEventListener("input", () => {  
  debouncedSearch(searchBox.value);  
});
```

2. Calling the function instead of passing it

```
const debouncedSearch = debounce(search(), 300);  
debouncedSearch();  
// prints: Searching... (straight away, not debounced)  
// then 300 ms Later: TypeError: Cannot read properties of undefined (reading 'apply')
```

`search()` with brackets runs `search` right now and passes its result (`undefined`) to `debounce`. Pass the function itself, without brackets: `debounce(search, 300)`.

3. Expecting a return value

```
function findPrice(product) {
  return product === "tea" ? 3.5 : 0;
}

const debouncedFind = debounce(findPrice, 300);
const price = debouncedFind("tea");
console.log(price); // prints: undefined
```

When `debouncedFind` returns, the real function hasn't even run yet. It runs 300 ms later, and nobody is waiting for its answer. Fix: do the work *inside* the debounced function, like updating the page there: `debounce((product) => showPrice(findPrice(product)), 300)`.

4. Picking the wrong one

```
const debouncedShow = debounce(showPosition, 150);

// Scrolling for 1 second: 10%, 20%, ... 100%
for (let i = 1; i <= 10; i++) {
  setTimeout(() => debouncedShow(i * 10), i * 100);
}
// prints: Progress bar: 100% (only once, after scrolling stops)
```

A debounced progress bar freezes while the user scrolls, then jumps at the end. The opposite mistake hurts too: a throttled search box sends searches for half-typed words. Go back to the three questions: final value only → `debounce`; steady updates → `throttle`.

5. Forgetting the waiting call when the user clears the box

```
searchBox.addEventListener("input", () => {
  const text = searchBox.value.trim();
  if (text === "") {
    clearResults();
    return; // but a search from a moment ago is still waiting...
  }
  debouncedSearch(text);
});
```

Type "piz", then quickly delete it. The list clears, and then, a moment later, the results for "piz" pop back up. Fix: call `debouncedSearch.cancel()` before `clearResults()`. If a request is already on its way, abort it too (`controller?.abort()`).

Quick recap

- **Debounce** waits until the calls stop for `delay` ms, then runs once with the latest arguments. Use it when only the final value matters: search, autosave, validation.
 - **Throttle** runs at most once every `interval` ms. Use it for steady updates while something keeps happening: scroll, resize, mouse moves, a game's fire rate.
 - Both are a closure plus timers: `debounce` remembers a `timerId`, `throttle` remembers when it last ran. `...args` and `fn.apply(this, args)` pass everything through.
 - **Trailing** edge = run at the end of a burst. **Leading** edge = run at the start, for instant feedback on buttons. For payments, use `once`.
 - Create the debounced function **once**, outside your listener. Give it a `cancel()` for calls that should never happen.
 - Debounce reduces requests, but `AbortController` is what stops a slow old answer from overwriting a newer one.
 - `requestAnimationFrame` is a built-in throttle for on-screen updates, and libraries like `lodash` have ready-made versions of both.
-

Next: try the [exercises](#), then move on to [35 Map and Set](#).

Exercises

How to do these:

- Exercises 1 to 4 run in the browser. Make one folder per exercise (`ex1/` , `ex2/` , and so on) with an `index.html` and a `script.js` . Copy the HTML from the exercise, and write the JavaScript yourself.
 - Open `index.html` by double-clicking it. Press `F12` and click **Console** to see your logs and any errors. Refresh the page after each change.
 - Start each `script.js` with your own `debounce` (and `throttle` , when you need it) from the [notes](#). Type them out again instead of copying: it's great practice.
 - Exercise 5 runs in Node: write `ex5.js` and run it with `node ex5.js` .
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Library search box

A library website has a search box. Right now it would search on every single keystroke. Make it wait until the reader stops typing.

Use this `index.html` :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Library Search</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Library search</h1>
  <input id="search" type="search" placeholder="Search for a book...">
  <ul id="results"></ul>
</body>
</html>
```

And start `script.js` with this list:

```
const books = [
  "Harry Potter and the Philosopher's Stone",
  "The Hobbit",
  "Pride and Prejudice",
  "The Hunger Games",
  "Harry Potter and the Chamber of Secrets",
  "Charlotte's Web",
  "The Great Gatsby",
  "Little Women",
];
```

Your tasks:

1. Keep a `keystrokes` counter that goes up on **every** `input` event.
2. Write `searchBooks(query)` . It logs `Searching for "<query>"` (after `<n>` keystrokes) , clears the old results, and shows each matching title as an `<li>` in the list. Matching ignores capital letters.
3. Call `searchBooks` through a debounced version with a 400 ms delay. Pass it the trimmed text from the box.

Type `harry` at a normal speed and pause. You should see this in the console, and the two Harry Potter books on the page:

```
Searching for "harry" (after 5 keystrokes)
```

Then type `potter` (with the space) and pause again:

Searching for "harry potter" (after 12 keystrokes)

Rule: put the titles on the page with `textContent`, not `innerHTML` (remember the warning in [chapter 20](#)).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: when the box is empty, clear the list, log nothing, and make sure a search from a moment ago can't pop back up (Common mistake 5).

Exercise 2 (Easy): Autosave a draft

You're building a notes app. Nobody likes losing what they typed, so the app should save the note by itself, but only once the user pauses. Saving on every keystroke would be wasteful.

Use this `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Notes</title>
  <script src="script.js" defer</script>
</head>
<body>
  <h1>Quick note</h1>
  <textarea id="draft" rows="8" cols="50" placeholder="Start typing..."></textarea>
  <p id="status">No changes yet</p>
</body>
</html>
```

Your tasks:

1. While the user types, the status shows `Typing...` straight away (no waiting).
2. One second after they stop typing, save the text to `localStorage` under the key `"draft"` ([chapter 23](#)). Then set the status to `All changes saved` and log how many characters you saved.
3. When the page loads, if there's a saved draft, put it back in the textarea and set the status to `Draft restored`.

Type `Dear Sam, see you soon!` and pause. You should see:

Saved 23 characters

Now refresh the page. Your text is still there, and the status says `Draft` restored .

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Reading progress bar

A news site shows a thin bar at the top of long articles. It grows as you scroll, so readers can see how far along they are. Updating it on every single `scroll` event is more work than needed.

Use this `index.html` :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Reading Progress</title>
  <style>
    body {
      font-family: sans-serif;
      margin: 0;
    }
    #progress {
      position: fixed;
      top: 0;
      left: 0;
      width: 0%;
      height: 6px;
      background: tomato;
    }
    #counter {
      position: fixed;
      right: 10px;
      bottom: 10px;
      padding: 8px 12px;
      background: #eee;
      border-radius: 6px;
    }
    article {
      max-width: 600px;
      height: 5000px; /* a very long page, so there's lots to scroll */
      margin: 40px auto;
      padding: 0 16px;
    }
  </style>
  <script src="script.js" defer></script>
</head>
<body>
  <div id="progress"></div>
  <div id="counter">Scroll events: 0 | Updates: 0</div>
  <article>
    <h1>A very long article</h1>
    <p>Scroll down and watch the bar at the top of the page.</p>
  </article>
</body>
</html>

```

Your tasks:

1. Write `updateProgressBar()` . It works out how far down the page you are and sets the bar's width to match. Here's the math you need:

```
const maxScroll = document.documentElement.scrollHeight - window.innerHeight;  
const percent = (window.scrollY / maxScroll) * 100;
```

`window.scrollY` is how far down you've scrolled, in pixels. `scrollHeight` is the height of the whole page and `innerHeight` is the height of the visible window, so `maxScroll` is the furthest you can scroll.

2. Throttle it, so it runs at most once every 100 ms, and call the throttled version from a `scroll` listener on `window` .
3. Count two things: every `scroll` event, and every real update. At the end of your scroll listener, show both in the counter box, like `Scroll events: 143 | Updates: 21` .

Scroll up and down for a while. Your numbers will be different, but the updates should be far fewer than the scroll events, and the bar should still feel smooth.

4. Now scroll quickly all the way to the bottom, a few times. Does the bar always reach exactly 100%? With the simple throttle, it sometimes stops a little short. Fix that with the "throttle plus debounce" trick from the notes.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Protect the "Pay" button

An online shop gets angry emails: customers who double-click "Pay" are charged twice. Your job is to make that impossible.

Use this `index.html` :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Checkout</title>
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Checkout</h1>
  <p>Wireless headphones: $49.99</p>
  <button id="pay">Pay $49.99</button>
  <p id="message"></p>
</body>
</html>

```

Part 1: leading debounce.

1. Write `chargeCard()`. It logs `Charging $49.99...` and shows `Payment sent. Thank you!` in the message paragraph. (It's pretend: no real money moves.)
2. Wrap it in a **leading** debounce with a 1000 ms delay.
3. In a `click` listener, log `click` on every click, then call your protected version.

Triple-click the button quickly. You should see:

```

click
Charging $49.99...
click
click

```

The customer got instant feedback, and was only charged once.

Part 2: once and for all. Wait two seconds and click again. You'll see a second `Charging $49.99...`. For a "Like" button that's fine, but for a payment it's a real problem.

Replace the leading debounce with the `once` helper from [chapter 25](#) (write it again if you need to), and disable the button as soon as it's clicked. Now, however many times you click, and however long you wait, you should only ever see:

```

click
Charging $49.99...

```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Autosave with "Save now" and "Discard"

Back to the notes app, now in Node. The autosave waits for a pause, like before. But users also want two buttons:

- **Save now** (Ctrl+S): don't wait, save the waiting text right now.
- **Discard**: throw the waiting save away.

Write a `debounce(fn, delay)` whose returned function has two methods:

- `cancel()` : forget the waiting call.
- `flush()` : if a call is waiting, run it **right now**, with its latest arguments and the same `this`, and stop its countdown so it doesn't run again later. If nothing is waiting, do nothing.

Put your `debounce` at the top of `ex5.js`, then paste this timeline below it. Don't change the timeline:

```
const notebook = {
  title: "Letter to Sam",
  save(text) {
    console.log(`Saved "${this.title}": ${text}`);
  },
};

notebook.autosave = debounce(notebook.save, 1000);

function at(ms, action) {
  setTimeout(action, ms);
}

at(100, () => notebook.autosave("Dear Sam,"));
at(300, () => notebook.autosave("Dear Sam, thanks"));
// ...a pause, so the autosave runs by itself (at about 1300 ms)
at(2000, () => notebook.autosave("Dear Sam, thanks for the"));
at(2200, () => {
  console.log("Ctrl+S pressed");
  notebook.autosave.flush(); // save right now, don't wait
});
at(3500, () => notebook.autosave("Dear Sam, thanks for the gift!"));
at(3700, () => {
  console.log("Discard clicked");
  notebook.autosave.cancel(); // throw the waiting save away
});
at(5000, () => {
  console.log("Ctrl+S pressed");
  notebook.autosave.flush(); // nothing is waiting, so nothing happens
});
at(5500, () => console.log("Done"));
```


Expected output:

```
Saved "Letter to Sam": Dear Sam, thanks
Ctrl+S pressed
Saved "Letter to Sam": Dear Sam, thanks for the
Discard clicked
Ctrl+S pressed
Done
```

Check your output line by line. Each of these is a different bug:

- A line appears **twice**: `flush()` didn't stop the countdown.
- `thanks for the gift! gets saved: cancel()` didn't really forget the waiting call.
- `Saved "undefined" : this` got lost somewhere.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Bonus: throttle with a trailing call. Build a `throttle(fn, interval)` that never loses the last call: when a call is "too soon", remember its arguments and book one final run for later. Test it on the scroll timeline from the notes (five events, 20% to 100%, with a 250 ms interval). The lines in the middle depend on exactly when your booked run happens, but you should see fewer than five lines, and the **last** one must be `Progress bar: 100%`.

Before you move on

In Exercise 1, type `harry`, delete it, and type `harry` again. The same search runs twice. With a real server, that's a wasted request for results you already had.

What if the page could remember the results for each search? [Chapter 35](#) gives you `Map`, a perfect place to keep them.

35 Map and Set

What is it?

`Set` and `Map` are two built-in **collections**: containers that hold many values, like the arrays and objects you already know.

- A **Set** is a list of unique values. Each value can be in it only once.
- A **Map** is a list of key/value pairs, like an object. But its keys can be anything (numbers, objects, even functions), not just strings.

Why does it matter?

Arrays and objects can do almost anything, but a few everyday jobs are clumsy with them.

Keeping a list without duplicates. With an array, you have to check `includes` before every `push`, and it's easy to forget:

```
const subscribers = [];  
  
function subscribe(email) {  
  if (!subscribers.includes(email)) {  
    subscribers.push(email);  
  }  
}
```

A Set does that check for you, every time.

Using something other than a string as a key. Say you want to count visits per customer, and each customer is an object. With a plain object as the lookup table, this goes wrong:

```
const alice = { name: "Alice" };  
const bob = { name: "Bob" };  
  
const visits = {};  
visits[alice] = 3;  
visits[bob] = 5;  
  
console.log(visits); // prints: { '[object Object]': 5 }
```

Object keys are always strings. JavaScript turned both customers into the same string, `"[object Object]"`, so Bob's number overwrote Alice's. A Map keeps each object as its own key. (Where that odd string comes from is

part of [chapter 38](#).)

Speed. `set.has(value)` stays fast even with a million items, while `array.includes(value)` checks the items one by one. You'll measure the difference in [chapter 48](#).

Real-world example

Picture a party with a **guest list and a bouncer**. The bouncer checks every name at the door. If you're already inside, you don't get in a second time. That's a Set.

Now picture the **coat check** at a theater. You hand over your coat and get a ticket. Later, you show the ticket and get your coat back. The ticket is the key and the coat is the value. That's a Map.

A Map is a very relaxed coat check, though: the "ticket" can be anything. A number, a word, or even another object.

	Set (guest list with a bouncer)	Map (coat check)
What it holds	Unique values	Key → value pairs
Adding the same thing twice	"You're already in." Nothing changes	Same ticket, new coat: the old value is replaced
The question it answers	"Is Priya on the list?"	"What's stored under ticket 101?"
Order	Remembers the order guests arrived	Remembers the order tickets were handed out

How it works

Creating a Set and adding values

`Set` is a built-in class, so you create one with `new`, like the classes in [chapter 27](#). Then you add values with `add`:

```
const guests = new Set();

guests.add("Priya");
guests.add("Tom");
guests.add("Priya"); // already in, so nothing happens

console.log(guests); // prints: Set(2) { 'Priya', 'Tom' }
console.log(guests.size); // prints: 2
```

Node shows how many values there are in brackets, then the values themselves. To count them in your code, use `size` (not `length`, like an array).

You can also start with an array. Duplicates get dropped on the way in:

```
const colors = new Set(["red", "green", "red", "blue", "green"]);
console.log(colors); // prints: Set(3) { 'red', 'green', 'blue' }
```

A Set remembers the order values were first added. The second `"red"` doesn't move anything.

Checking and removing: `has`, `delete`, `clear`

```
const guests = new Set(["Priya", "Tom", "Lena"]);

console.log(guests.has("Tom")); // prints: true
console.log(guests.has("tom")); // prints: false (case matters)

console.log(guests.delete("Tom")); // prints: true (it was there)
console.log(guests.delete("Sam")); // prints: false (nothing to remove)
console.log(guests); // prints: Set(2) { 'Priya', 'Lena' }

guests.clear(); // empties the whole Set
console.log(guests.size); // prints: 0
```

To decide whether two values are "the same", a Set uses the same rule as `===`. That's why `"tom"` and `"Tom"` are different values. (One small bonus: `NaN` counts as equal to itself here, so a Set never holds two `NaN`s.)

Looping over a Set

A Set has no indexes, so there's no `guests[0]`. Instead, you loop over it with `for...of`:

```
const tags = new Set(["javascript", "css", "html"]);

for (const tag of tags) {
  console.log(`#${tag}`);
}
```

You'll see:

```
#javascript
#css
#html
```

`tags.forEach((tag) => ...)` works too. But Sets don't have `map`, `filter`, or the other array methods from [chapter 13](#). When you need them, spread the Set into an array first (spread is from [chapter 15](#)):

```
const tags = new Set(["javascript", "css", "html"]);
const hashtags = [...tags].map((tag) => `#${tag}`);

console.log(hashtags); // prints: [ '#javascript', '#css', '#html' ]
```

`Array.from(tags)` does the same job as `[...tags]`.

Removing duplicates from an array

This is the Set trick you'll use most. Put the array into a Set, then spread it back out into a new array:

```
const signups = ["ana@mail.com", "ben@mail.com", "ana@mail.com", "cleo@mail.com"];
const people = [...new Set(signups)];

console.log(people); // prints: [ 'ana@mail.com', 'ben@mail.com', 'cleo@mail.com' ]
console.log(`${signups.length} signups, ${people.length} people`); // prints: 4 signups, 3 people
```

The original array doesn't change. You get a new one with each value once, in the order they first appeared.

Comparing two Sets

Sets really shine when you compare two groups. Say a gym tracks who came to the spin class on Monday and on Tuesday:

```
const monday = new Set(["Ana", "Ben", "Cleo"]);
const tuesday = new Set(["Ben", "Cleo", "Dev"]);

console.log(monday.union(tuesday)); // prints: Set(4) { 'Ana', 'Ben', 'Cleo', 'Dev' }
console.log(monday.intersection(tuesday)); // prints: Set(2) { 'Ben', 'Cleo' }
console.log(monday.difference(tuesday)); // prints: Set(1) { 'Ana' }
console.log(monday.symmetricDifference(tuesday)); // prints: Set(2) { 'Ana', 'Dev' }
```

Method	In plain words	The gym's question
<code>a.union(b)</code>	Everything in either Set	Who came at least once?
<code>a.intersection(b)</code>	Only what's in both	Who came both days?
<code>a.difference(b)</code>	In <code>a</code> , but not in <code>b</code>	Who came Monday but skipped Tuesday?

Method	In plain words	The gym's question
<code>a.symmetricDifference(b)</code>	In one or the other, but not both	Who came exactly one day?

Each method gives you a **new** Set. `monday` and `tuesday` don't change.

You can also ask yes/no questions. `a.isSubsetOf(b)` checks whether everything in `a` is also in `b`. That's perfect for "does this person have all the skills the job needs?":

```
const required = new Set(["html", "css", "javascript"]);
const priya = new Set(["html", "css", "javascript", "git"]);
const tom = new Set(["html", "javascript"]);

console.log(required.isSubsetOf(priya)); // prints: true
console.log(required.isSubsetOf(tom));   // prints: false
console.log(required.difference(tom));   // prints: Set(1) { 'css' }
```

The last line shows exactly what Tom is missing. Two more yes/no methods work the same way:

`a.isSupersetOf(b)` (does `a` contain everything in `b`?) and `a.isDisjointFrom(b)` (do they have nothing in common?).

Watch out: these seven methods are new. Browsers added them in 2023 and 2024, and they became part of the official JavaScript standard in 2025. They work in Node 24 and in current Chrome, Edge, Firefox, and Safari, but an older browser throws a `TypeError` saying that `union` is not a function.

Before these methods existed, people built them by hand with `has`. You'll still see this in older code:

```
const monday = new Set(["Ana", "Ben", "Cleo"]);
const tuesday = new Set(["Ben", "Cleo", "Dev"]);

// Keep only Monday's people who are also in Tuesday's Set
const bothDays = new Set([...monday].filter((name) => tuesday.has(name)));
console.log(bothDays); // prints: Set(2) { 'Ben', 'Cleo' }
```

Creating a Map: `set` and `get`

A Map stores pairs. `set(key, value)` puts a value in, and `get(key)` hands it back. Here's the coat check:

```

const coatCheck = new Map();

coatCheck.set(101, "red scarf");
coatCheck.set(102, "black umbrella");

console.log(coatCheck.get(101)); // prints: red scarf
console.log(coatCheck.get(999)); // prints: undefined (no such ticket)
console.log(coatCheck.get("101")); // prints: undefined (a string, not the number 101)
console.log(coatCheck); // prints: Map(2) { 101 => 'red scarf', 102 => 'black umbrella' }

```

Node draws each pair as `key => value`. Notice that the keys stay real numbers. An object would have turned them into the strings `"101"` and `"102"`, but in a Map, `101` and `"101"` are two different keys.

Maps have the same helpers as Sets: `has`, `delete`, `clear`, and `size`. You can also start a Map with an array of `[key, value]` pairs:

```

const stock = new Map([
  ["apples", 12],
  ["pears", 0],
  ["plums", 7],
]);

console.log(stock.size); // prints: 3
console.log(stock.has("pears")); // prints: true

stock.set("pears", 20); // same key: the old value is replaced
stock.delete("plums");

console.log(stock); // prints: Map(2) { 'apples' => 12, 'pears' => 20 }

```

Any value can be a key

Here's the visit counter from the start of the chapter, this time with a Map:

```

const alice = { name: "Alice" };
const bob = { name: "Bob" };

const visits = new Map();
visits.set(alice, 3);
visits.set(bob, 5);

console.log(visits.get(alice)); // prints: 3
console.log(visits.get(bob)); // prints: 5
console.log(visits.size); // prints: 2
console.log(visits.get({ name: "Alice" })); // prints: undefined

```

Each customer object is its own key now. But look at the last line. Remember from [chapter 16](#) that objects are compared by reference. The Map looks for *that exact object*, not a new one that happens to look the same.

Looping over a Map

`for...of` gives you one pair at a time, as a small `[key, value]` array. Destructuring it (from [chapter 15](#)) gives both parts a name:

```

const stock = new Map([
  ["apples", 12],
  ["pears", 0],
  ["plums", 7],
]);

for (const [fruit, count] of stock) {
  console.log(`${fruit}: ${count}`);
}

```

You'll see:

```

apples: 12
pears: 0
plums: 7

```

A Map always loops in the order the keys were first added.

If you only need one side, use `keys()` or `values()`. Spread them to get plain arrays:


```
const stock = new Map([
  ["apples", 12],
  ["pears", 0],
  ["plums", 7],
]);

console.log([...stock.keys()]); // prints: [ 'apples', 'pears', 'plums' ]
console.log([...stock.values()]); // prints: [ 12, 0, 7 ]
```

`keys()` and `values()` (and `entries()`, which gives the pairs) don't return arrays. They return something you can loop over or spread. [Chapter 36](#) explains exactly what that is.

Map or object?

Both hold key/value pairs, so which one should you pick?

	Object	Map
Keys	Strings (numbers and other values get turned into strings)	Any value, kept as it is
Counting entries	<code>Object.keys(obj).length</code>	<code>map.size</code>
Looping	Through <code>Object.entries(obj)</code>	Directly, with <code>for...of</code>
Order	Mostly the order you added keys, but number-like keys jump to the front	Always the order you added keys
Saving as JSON	<code>JSON.stringify</code> works	Needs converting first (see below)
Best for	A "record" with known fields: a user profile, a product	A lookup table that grows and shrinks, with keys you don't know in advance

A good rule of thumb: if you'd describe it as "a thing with fields", use an object. If you'd describe it as "a lookup table" or "a list of X per Y" (stock per product, visits per customer), a Map is usually the better fit. Small lookups with string keys are fine as objects too. You'll see lots of those, and that's OK.

Converting between objects and Maps

`Object.entries` from [chapter 11](#) turns an object into `[key, value]` pairs, which is exactly what `new Map()` accepts. `Object.fromEntries` goes the other way:

```
const settings = { theme: "dark", fontSize: 16 };

const settingsMap = new Map(Object.entries(settings));
console.log(settingsMap); // prints: Map(2) { 'theme' => 'dark', 'fontSize' => 16 }

const backToObject = Object.fromEntries(settingsMap);
console.log(backToObject); // prints: { theme: 'dark', fontSize: 16 }
```

That's also how you save a Map as JSON ([chapter 23](#)). `JSON.stringify` doesn't understand Maps or Sets, so it quietly gives you `{}` and your data is gone. Convert first:

```
const cart = new Map([["apple", 3], ["bread", 1]]);
const tags = new Set(["sale", "new"]);

console.log(JSON.stringify(cart)); // prints: {}
console.log(JSON.stringify(Object.fromEntries(cart))); // prints: {"apple":3,"bread":1}
console.log(JSON.stringify([...tags])); // prints: ["sale","new"]
```

To load them back, reverse the steps: `new Map(Object.entries(JSON.parse(text)))` for the Map, and `new Set(JSON.parse(text))` for the Set. (`Object.fromEntries` turns every key into a string, so this works best when the keys are strings already.)

Counting with a Map

Counting things is one of the most common jobs for a Map: words in reviews, votes in a poll, songs in a listening history.

```
const review = "great food great service friendly staff great prices";
const counts = new Map();

for (const word of review.split(" ")) {
  const soFar = counts.get(word) ?? 0; // undefined the first time, so use 0
  counts.set(word, soFar + 1);
}

console.log(counts.get("great")); // prints: 3
console.log(counts.get("staff")); // prints: 1

// The most common word: sort the pairs by count, biggest first
const ranked = [...counts].sort((a, b) => b[1] - a[1]);
console.log(ranked[0]); // prints: [ 'great', 3 ]
```

The `??` from [chapter 07](#) handles the first time a word shows up, when `get` gives back `undefined`. For the ranking, `[...counts]` turns the Map into an array of `[word, count]` pairs, so `a[1]` and `b[1]` are the counts.

Remembering results with a Map (a cache)

At the end of the [chapter 34 exercises](#), searching for `harry` twice sent the same request twice. What if the search box remembered its results? A Map is a perfect fit. Store each answer under the search text, and check `has` before doing the slow work again. A store like this is called a **cache**.

```
const cache = new Map();

function search(query) {
  if (cache.has(query)) {
    return `${cache.get(query)} (from the cache)`;
  }
  const results = `3 results for "${query}"`; // imagine a slow request to a server here
  cache.set(query, results);
  return results;
}

console.log(search("desk lamp")); // prints: 3 results for "desk Lamp"
console.log(search("desk lamp")); // prints: 3 results for "desk lamp" (from the cache)
```

The second search never reaches the slow part. (A cache that only ever grows can use up memory. [Chapter 41](#) shows how to keep it in check.)

Grouping with `Map.groupBy`

In [chapter 13](#) you met `Object.groupBy`, which sorts items into groups. `Map.groupBy` works the same way, but gives you a Map. That matters when your group keys aren't strings:

```
const orders = [
  { id: 1, total: 25 },
  { id: 2, total: 140 },
  { id: 3, total: 60 },
  { id: 4, total: 210 },
];

const bySize = Map.groupBy(orders, (order) => order.total >= 100);

console.log(bySize.get(true)); // prints: [ { id: 2, total: 140 }, { id: 4, total: 210 } ]
console.log(bySize.get(false)); // prints: [ { id: 1, total: 25 }, { id: 3, total: 60 } ]
```

The keys here are the real booleans `true` and `false`. `Object.groupBy` would have turned them into the strings `"true"` and `"false"`. Like `Object.groupBy`, `Map.groupBy` was added to JavaScript in 2024, so very old browsers don't have it.

WeakMap and WeakSet (a quick look)

When you put an object into a Map, the Map holds on to it. Even after the rest of your program has finished with that object, it can't be cleaned out of memory, because the Map still points to it.

A **WeakMap** is a Map with a looser grip. Its keys must be objects, and once nothing else in your program uses a key object, JavaScript is free to clean it up, entry and all. A **WeakSet** is the same idea for a Set.

```
const clickCounts = new WeakMap();
const buyButton = { id: "buy" }; // imagine a real button from the page

clickCounts.set(buyButton, 1);
console.log(clickCounts.get(buyButton)); // prints: 1

clickCounts.set("buy", 1);
// TypeError: Invalid value used as weak map key
```

They're useful for attaching extra information to objects you don't own, like elements on a page, without keeping those objects alive forever. The price is that you can't loop over them or check their `size`. [Chapter 41](#) explains how memory cleanup works and why this matters.

Common mistakes

1. Using square brackets instead of `set` and `get`

```
const scores = new Map();
scores["Ana"] = 90; // looks fine, but it isn't a Map entry

console.log(scores.get("Ana")); // prints: undefined
console.log(scores.size);       // prints: 0
```

Square brackets stick a normal object property onto the Map, which the Map's own methods ignore. Always use `scores.set("Ana", 90)` and `scores.get("Ana")`.

2. Giving a Set a single string

```
const fruits = new Set("apple");
console.log(fruits); // prints: Set(4) { 'a', 'p', 'l', 'e' }
```

`new Set()` goes through whatever list you give it, one item at a time. A string counts as a list of letters, so you get its unique letters. To store the whole word, wrap it in an array: `new Set(["apple"])`.

3. Expecting look-alike objects to be duplicates

```
const cart = new Set();
cart.add({ id: 7, name: "Desk lamp" });
cart.add({ id: 7, name: "Desk lamp" });

console.log(cart.size); // prints: 2
```

Two objects that look the same are still two different objects ([chapter 16](#)), so the Set keeps both. Store something simple that identifies the item instead, like its id: `cart.add(7)`. Or use a Map from id to product.

4. Mixing up the order in a Map's `forEach`

```
const stock = new Map([["apples", 12]]);

stock.forEach((fruit, count) => {
  console.log(`${fruit}: ${count}`); // prints: 12: apples
});
```

A Map's `forEach` gives you the **value first**, then the key (to match arrays, where the item comes before the index). Write `(count, fruit) =>`, or use `for (const [fruit, count] of stock)`, which reads in the natural order.

5. Passing an array to a Set method

```
const monday = new Set(["Ana", "Ben"]);
console.log(monday.union(["Cleo"]));
// TypeError: The .size property is NaN
```

The new Set methods need another Set to compare with. The message is confusing because arrays have a `length`, not a `size`. Fix: `monday.union(new Set(["Cleo"]))`.

Quick recap

- A **Set** holds unique values: `add`, `has`, `delete`, `size`. `[...new Set(array)]` removes duplicates.
- `union`, `intersection`, `difference`, `symmetricDifference`, and `isSubsetOf` compare two Sets. They're new (standard since 2025) and return new Sets.
- A **Map** holds key → value pairs, and the keys can be anything: `set`, `get`, `has`, `delete`, `size`.
- Loop over both with `for...of`. For a Map, destructure each `[key, value]` pair.
- Use an object for a record with known fields. Use a Map for a lookup table that changes, or when the keys aren't strings.
- `JSON.stringify` doesn't understand Maps or Sets. Convert them with `Object.fromEntries` or spread first.
- WeakMap and WeakSet only take objects, and they don't keep those objects alive.

Next: try the [exercises](#), then move on to [36 Iterators and Generators](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Run each one with `node ex1.js` .
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Newsletter clean-up

A bakery collected newsletter sign-ups at the till and on its website. Some people signed up twice. Start with this array:

```
const signups = [  
  "mia@mail.com",  
  "leo@mail.com",  
  "mia@mail.com",  
  "zoe@mail.com",  
  "leo@mail.com",  
  "sam@mail.com",  
];
```

Use a Set to get rid of the duplicates. Print the counts, then a numbered list of the unique emails. Finally, check whether two people are subscribed.

Expected output:

```
Sign-ups: 6  
Unique people: 4  
Duplicates removed: 2  
1. mia@mail.com  
2. leo@mail.com  
3. zoe@mail.com  
4. sam@mail.com  
zoe@mail.com subscribed? true  
max@mail.com subscribed? false
```

Rule: don't check for duplicates yourself with `includes` or a loop. Let the Set do that job.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Parcel lockers

An apartment building has numbered parcel lockers. You'll keep track of them with a Map from the locker number (a number) to the name of the person whose parcel is inside.

Do these steps in order:

1. Create an empty Map called `lockers`.
2. Put three parcels in: locker `3` for Ravi, locker `7` for Hana, and locker `12` for Omar.
3. Print who has a parcel in locker `7`.
4. Hana collects her parcel. Remove locker `7` from the Map.
5. Print whether locker `7` is still in use, and how many lockers are in use.
6. A new parcel for Lucia goes into locker `5`.
7. Loop over the Map and print every locker.

Expected output:

```
Locker 7 holds a parcel for Hana
Locker 7 in use? false
Lockers in use: 2
Locker 3: Ravi
Locker 12: Omar
Locker 5: Lucia
```

Then answer two questions in a comment at the bottom of your file:

- Why is locker 5 printed last, and not between 3 and 12?
- What does `lockers.get("3")` give you, and why?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Top songs of the week

A music app logs the title of every song someone plays. Here's one listener's week:

```
const plays = [
  "Blue Sky", "Night Drive", "Blue Sky", "Sunrise", "Night Drive",
  "Blue Sky", "Paper Moon", "Sunrise", "Blue Sky", "Night Drive",
  "Lighthouse", "Sunrise", "Paper Moon", "Blue Sky", "Night Drive",
];
```

Count the plays for each song with a Map. Then print a small weekly summary: how many different songs there were, the total number of plays, the top 3 songs, and the songs that were played only once.

Expected output:

```
Different songs: 5
Total plays: 15
Top 3 this week:
1. Blue Sky (5 plays)
2. Night Drive (4 plays)
3. Sunrise (3 plays)
Played only once: Lighthouse
```

Rule: count with a Map, not a plain object. Don't type any of the counts yourself.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Movie night

Ana and Ben are planning a movie quiz night. Here's what each of them has seen, and the movies the quiz is about:

```
const ana = new Set(["Up", "Coco", "Soul", "Luca", "Cars"]);
const ben = new Set(["Coco", "Cars", "Brave", "Up", "Wall-E"]);
const quizMovies = new Set(["Up", "Coco", "Soul"]);
```

Use the Set methods from the notes to answer each question below. Print every list in alphabetical order, separated by commas.

Expected output:


```
Seen by both: Cars, Coco, Up
Only Ana has seen: Luca, Soul
Only Ben has seen: Brave, Wall-E
Seen by exactly one: Brave, Luca, Soul, Wall-E
All movies (7): Brave, Cars, Coco, Luca, Soul, Up, Wall-E
Ana ready for quiz night? true
Ben ready for quiz night? false
Ben still needs to watch: Soul
```

"Ready for quiz night" means they've seen every quiz movie.

Rule: no loops, and no `filter` with `has`. Use `union`, `intersection`, `difference`, `symmetricDifference`, and `isSubsetOf`. (They need Node 22 or newer, or a current browser. You have Node 24, so you're fine.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Library loans

A small community library lends books to its members. It has plenty of copies of every book, but it has two rules:

- A member can have at most **3 books** at a time.
- A member can't borrow a title they already have.

Write a `Library` class ([chapter 27](#)) that keeps track of loans:

- A **private** field `#loans`: a Map from a member's name to a **Set** of the titles they have.
- `borrow(member, title)`: if a rule is broken, throw an `Error` ([chapter 18](#)) with a message like the ones in the expected output. Otherwise, record the loan and print `Ravi borrowed Dune`.
- `giveBack(member, title)`: if the member doesn't have that title, throw an `Error`. Otherwise, remove it and print `Mia returned Dune`. When a member has no books left, remove them from the Map completely.
- `report()`: print every member with their books, then the number of members with books and the total number of books on loan.

Test your class with this code. Paste it below your class, and don't change it:

```

const library = new Library();

function tryTo(action) {
  try {
    action();
  } catch (error) {
    console.log(`Sorry: ${error.message}`);
  }
}

tryTo(() => library.borrow("Ravi", "Dune"));
tryTo(() => library.borrow("Ravi", "Emma"));
tryTo(() => library.borrow("Mia", "Dune"));
tryTo(() => library.borrow("Ravi", "Dune"));
tryTo(() => library.borrow("Ravi", "Beloved"));
tryTo(() => library.borrow("Ravi", "Matilda"));
tryTo(() => library.giveBack("Mia", "Emma"));
tryTo(() => library.giveBack("Mia", "Dune"));
library.report();

```

Expected output:

```

Ravi borrowed Dune
Ravi borrowed Emma
Mia borrowed Dune
Sorry: Ravi already has Dune
Ravi borrowed Beloved
Sorry: Ravi has reached the limit of 3 books
Sorry: Mia doesn't have Emma
Mia returned Dune
--- Loans ---
Ravi: Dune, Emma, Beloved
Members with books: 1
Books on loan: 3

```

Mia isn't in the report because she returned her only book.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Bonus: add a method `whoHas(title)` that returns an array with the name of every member who has that title. For example, calling it with `"Dune"` at the end of the test code should give you `[ 'Ravi' ]`.

Before you move on

You've now used `for...of` on arrays, strings, Sets, and Maps. They're very different things, yet one loop works on all of them. How does `for...of` know how to walk through each one? And could you make *your own* object work with `for...of`?

[Chapter 36: Iterators and Generators](#) answers both questions.

36 Iterators and Generators

What is it?

An **iterable** is anything you can loop over with `for...of`: arrays, strings, Maps, and Sets. An **iterator** is the helper behind the scenes that hands out the values, one at a time, whenever it's asked for the next one.

A **generator** is a special kind of function that makes iterators for you. It can pause in the middle, hand you a value, and later carry on from exactly where it stopped.

Why does it matter?

You've looped over arrays, strings, Maps, and Sets with the same `for...of`, and spread them all with `...`. They're very different things, so how does one loop work on all of them? This chapter shows the simple agreement they all follow. Once you know it, you can make your own objects work the same way.

Right now, your own objects can't do that:

```
class Playlist {
  constructor(songs) {
    this.songs = songs;
  }
}

const playlist = new Playlist(["Fast Car", "Hey Ya!"]);

for (const song of playlist) {
  console.log(song);
}
// TypeError: playlist is not iterable
```

By the end of this chapter, that loop will work.

Generators also let you produce values **on demand**, only when someone asks for them. That makes some things possible that arrays can't do at all:

- An endless supply of order numbers or ticket numbers, without building an endless array.
- Reading a big list of results page by page, loading the next page only when you actually need it.

Real-world example

Think of the **"take a number" ticket dispenser** at a busy deli counter.

The dispenser doesn't print every ticket in the morning. It only knows how to give you *the next one*. You pull, you get a number. The next person pulls, they get the next number. When the roll runs out, it tells you it's empty.

At the deli counter	In JavaScript
The deli counter, which has a dispenser you can use	An iterable
The ticket dispenser itself	An iterator
Pulling a ticket	Calling <code>next()</code>
The number on your ticket	<code>value</code>
"Sorry, the roll is empty"	<code>done: true</code>

Generators add one more picture: a **bookmark in a book**. You read a few pages, slip the bookmark in, and close the book. Later, you open it at the bookmark and keep reading from exactly where you left off. A generator does the same thing with a function.

How it works

Iterables: things you can loop over

You already know several iterables. `for...of` works on all of them, and so do spread (`...`) and array destructuring from [chapter 15](#):

```
const [first, second] = "Hey"; // a string gives you its letters
console.log(first, second); // prints: H e

const scores = new Set([7, 9, 7, 4]);
console.log([...scores]);      // prints: [ 7, 9, 4 ]
console.log(Math.max(...scores)); // prints: 9
```

A plain object is **not** iterable. JavaScript can't guess what "the next value" of an object should be: the next key? The next value? Both?

```
const order = { item: "Pizza", qty: 2 };

for (const part of order) {
  console.log(part);
}
// TypeError: order is not iterable
```

For objects, you pick what to loop over with `Object.keys`, `Object.values`, or `Object.entries` from [chapter 11](#). Those give you arrays, and arrays are iterable.

Iterators: `next()` and `{ value, done }`

Here's what `for...of` does behind the scenes. First, it asks the iterable for an **iterator**. Then it calls the iterator's `next()` method again and again. Each call hands back a small object with two properties:

- `value`: the next value.
- `done`: `false` while there are values left, `true` when it's finished.

You can do the same thing by hand. Arrays give you an iterator through a method with an unusual name, `[Symbol.iterator]` (more on that name in a moment):

```
const queue = ["Ravi", "Hana"];
const dispenser = queue[Symbol.iterator](); // get an iterator from the array

console.log(dispenser.next()); // prints: { value: 'Ravi', done: false }
console.log(dispenser.next()); // prints: { value: 'Hana', done: false }
console.log(dispenser.next()); // prints: { value: undefined, done: true }
console.log(dispenser.next()); // prints: { value: undefined, done: true }
```

Once an iterator says `done: true`, it stays done. Like the empty ticket roll, it has nothing more to give.

So a `for...of` loop is really a short way of writing this:

```
const queue = ["Ravi", "Hana"];
const dispenser = queue[Symbol.iterator]();

let ticket = dispenser.next();
while (!ticket.done) {
  console.log(`Now serving: ${ticket.value}`);
  ticket = dispenser.next();
}
```

You'll see:

```
Now serving: Ravi
Now serving: Hana
```

This agreement, "give me an iterator, and the iterator has a `next()` that returns `{ value, done }`", is called the **iterator protocol**. A **protocol** is a set of rules that everyone agrees to follow. Arrays, strings, Maps, and Sets all follow it, and that's why one loop works on all of them.

Symbols (a quick look)

Back in [chapter 03](#), you heard about a rare primitive type called **symbol**. Here it is. A symbol is a value that's guaranteed to be unique. Even two symbols with the same description are different:

```
const idA = Symbol("id");
const idB = Symbol("id");

console.log(idA === idB);    // prints: false
console.log(typeof idA);    // prints: symbol
console.log(idA.description); // prints: id
```

Symbols can be used as object keys, and a symbol key never clashes with any other key. (In [chapter 35](#) you read that object keys are strings. Symbols are the one other kind of key an object can have.)

JavaScript has a few built-in symbols, and `Symbol.iterator` is one of them. When an object has a method stored under the key `Symbol.iterator`, JavaScript knows the object is iterable, and it calls that method to get an iterator. Because the key is a symbol, it can never mix with your own methods, even one you happen to name `iterator`.

Making your own iterable

Say a hotel's first floor has rooms 101 to 105. Instead of typing out an array of every room number, you can make an object that only knows the first and last room, and still loop over it:

```

const floorOne = {
  first: 101,
  last: 105,

  [Symbol.iterator]() {
    let current = this.first;
    const last = this.last;

    return {
      next() {
        if (current > last) {
          return { value: undefined, done: true };
        }
        const value = current;
        current += 1;
        return { value, done: false };
      },
    };
  },
};

console.log([...floorOne]); // prints: [ 101, 102, 103, 104, 105 ]

```

There's a lot going on, so let's take it piece by piece:

- `[Symbol.iterator]() { ... }` is a method whose key is the symbol. The square brackets mean "use the value of `Symbol.iterator` as the key", like bracket notation from [chapter 11](#).
- Every time someone wants to loop, JavaScript calls that method, and it returns a **fresh** iterator: an object with a `next()` method. That's why you can loop over `floorOne` as many times as you like.
- `next()` remembers `current` between calls thanks to a closure ([chapter 25](#)).
- We copy `this.first` and `this.last` into variables first. Inside `next()`, `this` would mean the little iterator object, not `floorOne` (remember from [chapter 26](#) that `this` depends on how a function is called).

It works, but that's a lot of bookkeeping for "count from 101 to 105". Generators fix that.

Generators: `function*` and `yield`

A **generator function** is written with a star: `function*`. Inside it, the keyword `yield` hands out a value and **pauses** the function right there, like slipping in a bookmark. The next time a value is needed, the function wakes up at the bookmark and carries on.


```
function* boardingCalls() {
  console.log("Gate opens");
  yield "Group 1";
  console.log("Next group, please");
  yield "Group 2";
}

const calls = boardingCalls();
console.log("Nothing has run yet");

console.log(calls.next());
console.log(calls.next());
console.log(calls.next());
```

You'll see:

```
Nothing has run yet
Gate opens
{ value: 'Group 1', done: false }
Next group, please
{ value: 'Group 2', done: false }
{ value: undefined, done: true }
```

Step by step:

1. Calling `boardingCalls()` does **not** run its body. It gives you a **generator object**, which is an iterator. That's why "Nothing has run yet" is printed first.
2. The first `next()` runs the body up to the first `yield`. `yield "Group 1"` hands out "Group 1" and pauses.
3. The second `next()` picks up right after that `yield`, prints "Next group, please", and runs until the next `yield`.
4. The third `next()` carries on, reaches the end of the function, and gets `done: true`.

You'll rarely call `next()` yourself, though. A generator object is iterable too, so `for...of` and spread work on it:

```
function* boardingCalls() {
  yield "Group 1";
  yield "Group 2";
  yield "Group 3";
}

for (const group of boardingCalls()) {
  console.log(`Now boarding: ${group}`);
}

console.log([...boardingCalls()]);
```

You'll see:

```
Now boarding: Group 1
Now boarding: Group 2
Now boarding: Group 3
[ 'Group 1', 'Group 2', 'Group 3' ]
```

Now here are the hotel rooms again, written as a generator:

```
function* roomNumbers(first, last) {
  for (let room = first; room <= last; room++) {
    yield room;
  }
}

console.log([...roomNumbers(101, 105)]); // prints: [ 101, 102, 103, 104, 105 ]
```

Same result as the long version, in a few lines. The generator keeps track of `room` between values for you.

Making a class iterable

Now you can fix the `Playlist` from the start of the chapter. Give the class a generator method called `[Symbol.iterator]`, with a star in front:

```

class Playlist {
  #songs;

  constructor(songs) {
    this.#songs = songs;
  }

  *[Symbol.iterator]() {
    for (const song of this.#songs) {
      yield song;
    }
  }
}

const playlist = new Playlist(["Fast Car", "Hey Ya!", "Dreams"]);

for (const song of playlist) {
  console.log(`Playing: ${song}`);
}

const [opener] = playlist;
console.log(`Opening song: ${opener}`);

```

You'll see:

```

Playing: Fast Car
Playing: Hey Ya!
Playing: Dreams
Opening song: Fast Car

```

The songs stay private (the `#songs` field from [chapter 27](#)), but anyone can still loop over the playlist, spread it, or destructure it. And because each `for...of` calls `[Symbol.iterator]()` again, every loop gets a brand-new generator that starts from the first song.

Tip: inside a generator, `yield* this.#songs;` is a shortcut for that whole `for...of` loop. `yield*` hands out every value of another iterable, one by one.

Endless sequences and lazy evaluation

Here's the deli's ticket dispenser as code. It never runs out:

```
function* ticketNumbers() {
  let number = 1;
  while (true) {
    yield `T-${String(number).padStart(3, "0")}`;
    number++;
  }
}

const tickets = ticketNumbers();

console.log(tickets.next().value); // prints: T-001
console.log(tickets.next().value); // prints: T-002
console.log(tickets.next().value); // prints: T-003
```

Normally, `while (true)` is an infinite loop that freezes your program ([chapter 08](#)). Inside a generator, it's safe. The function pauses at every `yield` and only runs again when someone asks for the next ticket.

This idea is called **lazy evaluation**: doing the work only when its result is actually needed, not ahead of time. Think of a sandwich shop that makes each sandwich when it's ordered, instead of making a thousand at 6 a.m. and hoping people turn up.

That makes endless generators great **ID generators**. Every order, user, or ticket gets the next number, and you never run out.

You can also loop over an endless generator, as long as the loop has a way out. Add this below the code above:

```
for (const ticket of ticketNumbers()) {
  console.log(`Now serving ${ticket}`);
  if (ticket === "T-003") {
    break; // without this, the loop would never end
  }
}
```

You'll see:

```
Now serving T-001
Now serving T-002
Now serving T-003
```

Iterator helpers: `map`, `filter`, `take`, and friends

Iterators have their own versions of the array methods from [chapter 13](#): `map`, `filter`, `find`, `some`, `every`, `reduce`, `forEach`, and `flatMap`. They also have a few of their own:

- `take(n)` : stop after `n` values.
- `drop(n)` : skip the first `n` values.
- `toArray()` : collect the values into a real array.

Here's a real question with an endless answer: which years are leap years? There's no last year, so a generator is a natural fit.

```
function* yearsFrom(start) {
  let year = start;
  while (true) {
    yield year;
    year++;
  }
}

function isLeapYear(year) {
  return (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0;
}

const nextLeapYears = yearsFrom(2026).filter(isLeapYear).take(3).toArray();
console.log(nextLeapYears); // prints: [ 2028, 2032, 2036 ]
```

Read the chain like a sentence: "from all the years starting at 2026, keep the leap years, take the first three, and put them in an array."

But wait: how can `filter` work on an endless list without freezing? Because iterator helpers are lazy too. They don't build a whole new list at each step. Each year travels through the chain on its own, and as soon as `take` has enough, the chain stops asking for more. You can watch it happen. Add this to the same file, below the code above:

```
const checked = [];

const firstTwo = yearsFrom(2026)
  .filter((year) => {
    checked.push(year); // note down every year that filter looks at
    return isLeapYear(year);
  })
  .take(2)
  .toArray();

console.log(firstTwo); // prints: [ 2028, 2032 ]
console.log(checked.join(", ")); // prints: 2026, 2027, 2028, 2029, 2030, 2031, 2032
```

`filter` looked at exactly seven years and stopped at 2032, the moment `take` had its two leap years.

The helpers work on every built-in iterator too, like the ones you get from a Map's `keys()` , `values()` , and `entries()` :

```
const stock = new Map([
  ["apples", 12],
  ["pears", 0],
  ["plums", 7],
]);

const inStock = stock
  .entries()
  .filter(([ , count]) => count > 0) // skip the key, keep the count
  .map([fruit]) => fruit
  .toArray();

console.log(inStock); // prints: [ 'apples', 'plums' ]
```

Here's how they compare with the array methods you already know:

	Array methods	Iterator helpers
Work on	Arrays	Any iterator: generators, <code>map.keys()</code> , <code>set.values()</code> , ...
When the work happens	Right away, on every item	Only when a value is asked for (lazy)
What each step gives back	A whole new array	Another iterator (use <code>toArray()</code> to get an array)
Endless sequences	Impossible	Fine, with <code>take</code> or <code>find</code>

Watch out: iterator helpers are new. They became part of the official JavaScript standard in 2025. They work in Node 22 and newer (you have Node 24) and in up-to-date browsers, but older browsers don't have them. In older code, you'll see people spread the iterator into an array first (`[...stock.entries()].filter(...)`) and use the array methods instead.

Async generators and `for await...of` (a quick look)

Sometimes values arrive **over time**. A server might send a long list of customers one page at a time, and each page takes a moment to load.

An **async generator** (`async function*`) handles that. Inside it, you can `await` ([chapter 32](#)) and `yield` values as they arrive. You loop over it with `for await...of` , which waits for each value before running the loop body.

```

const pages = [["Ana", "Ben"], ["Cleo", "Dev"], ["Eli"]]; // pretend server data

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function* loadCustomers() {
  for (let page = 1; page <= pages.length; page++) {
    console.log(`(loading page ${page}...)`);
    await wait(300); // pretend this is a slow network request
    yield* pages[page - 1];
  }
}

async function main() {
  for await (const customer of loadCustomers()) {
    console.log(customer);
  }
  console.log("Done!");
}

main();

```

You'll see (with a short pause before each page):

```

(loading page 1...)
Ana
Ben
(loading page 2...)
Cleo
Dev
(loading page 3...)
Eli
Done!

```

Notice that page 2 only starts loading after Ana and Ben have been handled. The generator is lazy, so if you `break` out of the loop early, the later pages are never loaded at all. In a real app, the `await wait(300)` line would be a `fetch` call from [chapter 33](#), with the page number in the URL.

`for await...of` has to be inside an `async` function, like `main` here (or at the top level of a module, from [chapter 29](#)).

Common mistakes

1. Reusing a generator that's already used up

```
function* boardingCalls() {
  yield "Group 1";
  yield "Group 2";
}

const calls = boardingCalls();
console.log([...calls]); // prints: [ 'Group 1', 'Group 2' ]
console.log([...calls]); // prints: []
```

A generator object is a one-way trip. Once it's done, it stays done. Call the generator function again to get a fresh one: `[...boardingCalls()]`. (A `break` inside `for...of`, or a helper like `take`, also finishes off the generator it was reading from.)

2. Forgetting the star

```
function ticketNumbers() {
  yield "T-001";
}
// SyntaxError: Unexpected string
```

Without the `*`, this is a normal function, and a normal function doesn't understand `yield`. The error message doesn't even mention `yield`, which makes this one hard to spot. Fix: `function* ticketNumbers()`.

3. Asking for "everything" from an endless generator

```
function* ticketNumbers() {
  let number = 1;
  while (true) {
    yield number;
    number++;
  }
}

const allTickets = [...ticketNumbers()]; // never finishes!
```

Spread, `toArray()`, and a `for...of` without a `break` all want *every* value, and an endless generator never runs out. Your program freezes and, after a while, crashes with `FATAL ERROR: Reached heap limit Allocation failed - JavaScript heap out of memory`. Press `Ctrl + C` to stop it. Always limit an endless generator first, with `take(n)` or a `break`.

4. Mixing up arrays and iterators


```
const scores = [88, 92, 75, 64];
console.log(scores.take(2));
// TypeError: scores.take is not a function
```

`take` and `drop` belong to iterators, not arrays. For an array, use `scores.slice(0, 2)`. Or get an iterator first with `values()`, but then don't forget to turn the result back into an array:

```
const scores = [88, 92, 75, 64];

console.log(scores.values().take(2));           // prints: Object [Iterator Helper] {}
console.log(scores.values().take(2).toArray()); // prints: [ 88, 92 ]
```

Iterator helpers give you back another iterator, not an array. Nothing has even been worked out yet. Add `.toArray()` at the end, or loop over the result with `for...of`.

5. Using `for...of` instead of `for await...of`

```
async function* loadCustomers() {
  yield "Ana";
}

for (const customer of loadCustomers()) {
  console.log(customer);
}
// TypeError: loadCustomers is not a function or its return value is not iterable
```

An async generator hands out its values over time, so a plain `for...of` can't use it. Write `for await` (`const customer of loadCustomers()`), inside an `async` function.

Quick recap

- An **iterable** is anything `for...of`, spread, and destructuring can use: arrays, strings, Maps, Sets. Plain objects aren't iterable.
- Behind the scenes, `for...of` gets an **iterator** from the `[Symbol.iterator]()` method and calls its `next()`, which returns `{ value, done }`.
- A **symbol** is a unique value. `Symbol.iterator` is the built-in key that marks an object as iterable.
- A **generator** (`function*`) makes iterators the easy way: `yield` hands out a value and pauses until the next one is needed.
- Generators are **lazy**, so they can describe endless sequences, like ticket numbers. Just never ask for *all* of an endless one.
- **Iterator helpers** (`map`, `filter`, `take`, `drop`, `toArray`, ...) work on any iterator, one value at a time. They're new (standard since 2025).

- `async function*` with `for await...of` handles values that arrive over time, like pages of results.
-

Next: try the [exercises](#), then move on to [37 Regular Expressions](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Next customer, please

A deli counter has a queue of customers:

```
const queue = ["Ravi", "Hana", "Omar"];
```

Serve the queue by hand, the way `for...of` does it behind the scenes:

1. Get an iterator from the array.
2. Call `next()` once and print the whole result object.
3. Serve everyone who's left with a `while` loop that keeps calling `next()` until the iterator is done.
4. Print the final result object with the message `Queue is empty:` .

Expected output:

```
{ value: 'Ravi', done: false }  
Now serving: Hana  
Now serving: Omar  
Queue is empty: { value: undefined, done: true }
```

Rule: no `for...of` , no `forEach` , and no indexes like `queue[1]` . Only `next()` .

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Rocket countdown

Write a generator function `countdown(from)` that yields the numbers from `from` down to `1`, and then yields the text `"Liftoff!"`.

Then use it in three ways:

1. Loop over `countdown(5)` with `for...of` and print each value.
2. Spread `countdown(3)` into an array and print the array.
3. Make one generator object with `countdown(2)`, call `next()` on it four times, and print each result.

Expected output:

```
5
4
3
2
1
Liftoff!
[ 3, 2, 1, 'Liftoff!' ]
{ value: 2, done: false }
{ value: 1, done: false }
{ value: 'Liftoff!', done: false }
{ value: undefined, done: true }
```

Look at the last two lines. Why is `done` still `false` when you get `"Liftoff!"`? Write your answer in a comment.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): An iterable bookshelf

You're building a small reading app. Write a `Bookshelf` class that keeps its books in a **private** array field, `#books`, and still lets people loop over it.

It needs:

- `add(title, author)`: stores a book as an object like `{ title: "Dune", author: "Frank Herbert" }`.
- A `[Symbol.iterator]` generator method, so `for...of`, spread, and destructuring work on a shelf and give you the book objects.
- A generator method `byAuthor(author)` that yields only the **titles** of that author's books.

Test it with this code. Paste it below your class, and don't change it:

```

const shelf = new Bookshelf();
shelf.add("Dune", "Frank Herbert");
shelf.add("Emma", "Jane Austen");
shelf.add("Beloved", "Toni Morrison");
shelf.add("Persuasion", "Jane Austen");

for (const book of shelf) {
  console.log(`${book.title} by ${book.author}`);
}

const [first, second] = shelf;
console.log(`First two: ${first.title}, ${second.title}`);
console.log(`Books on the shelf: ${[...shelf].length}`);

for (const title of shelf.byAuthor("Jane Austen")) {
  console.log(`Austen: ${title}`);
}

```

Expected output:

```

Dune by Frank Herbert
Emma by Jane Austen
Beloved by Toni Morrison
Persuasion by Jane Austen
First two: Dune, Emma
Books on the shelf: 4
Austen: Emma
Austen: Persuasion

```

Rule: `#books` must stay private. Don't add a method that hands out the array itself.

Then answer this in a comment: the test code loops over `shelf` three times, and each time starts from the first book. But in the notes, a generator *object* could only be used once. Why is a shelf different?

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Bus timetable

A city bus leaves the depot every 15 minutes, starting at 06:00, all day long. Instead of storing a long array of times, describe the timetable with an endless generator and ask it questions with iterator helpers.

1. Write a generator `departures(firstBus, every)` that yields departure times forever, counted in **minutes after midnight**. So 06:00 is `360`, the next bus is `375`, then `390`, and so on.

2. Write a function `formatTime(minutes)` that turns minutes into `HH:MM` text, like `formatTime(375)` → `"06:15"`.
3. Use `departures` and the iterator helpers to answer each question below.

Expected output:

```
First buses: 06:00, 06:15, 06:30, 06:45
Arrive at 08:20, next bus at 08:30
Lunchtime buses: 12:00, 12:15, 12:30, 12:45
The 11th bus of the day leaves at 08:30
```

- **First buses:** the first four departures.
- **Next bus:** you reach the stop at 08:20. When is the first departure at or after that time?
- **Lunchtime buses:** the first four departures from 12:00 onwards.
- **The 11th bus:** skip the first ten buses and take the next one.

Rule: no arrays of times that you build or type yourself, and no `while` loops outside the generator. Let the helpers (`take`, `drop`, `filter`, `map`, `find`, `toArray`) do the work.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Order history, page by page

A shop's order history lives on a server that sends orders **one page at a time**, with a short delay for each page. Here's a pretend version of that server. Copy it into your file as it is:

```

const database = [
  { id: 1, customer: "Ana", total: 30 },
  { id: 2, customer: "Ben", total: 45 },
  { id: 3, customer: "Ana", total: 25 },
  { id: 4, customer: "Cleo", total: 120 },
  { id: 5, customer: "Ben", total: 15 },
  { id: 6, customer: "Ana", total: 60 },
  { id: 7, customer: "Cleo", total: 40 },
];

function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

// Pretend API: returns one page of orders after a short delay
async function fetchOrdersPage(page, pageSize) {
  await wait(200);
  console.log(`(fetched page ${page})`);
  const start = (page - 1) * pageSize;
  return database.slice(start, start + pageSize);
}

```

Your code isn't allowed to touch `database` directly. It may only ask for pages with `fetchOrdersPage`, just like a real app talking to a real server ([chapter 33](#)).

1. Write an async generator `allOrders(pageSize)` that fetches page 1, then page 2, and so on, and yields the orders **one at a time**. When a page comes back empty, there are no more orders, so the generator should finish.
2. Write an `async function main()`. In it:
 - **Part 1:** loop over `allOrders(3)` with `for await...of` and add up how much each customer spent, using a `Map` ([chapter 35](#)). Then print each customer's total.
 - **Part 2:** loop over a fresh `allOrders(3)` to find the first order over \$100. Print it and stop the loop straight away.

Expected output:

```
Part 1: totals
(fetch page 1)
(fetch page 2)
(fetch page 3)
(fetch page 4)
Ana: $115
Ben: $60
Cleo: $160
Part 2: first big order
(fetch page 1)
(fetch page 2)
First big order: #4 by Cleo ($120)
```

Look closely at Part 2: pages 3 and 4 were never fetched. That's lazy evaluation saving you two slow network requests.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 4, you turned minutes into times like `08:30`. Now imagine the opposite job: someone sends you a message like `"Meet at 08:30 or 12:15, not 25:99"`, and you need to find every *valid* time in it.

`includes` and `indexOf` can only search for exact text. They can't search for "two digits, a colon, then two more digits". [Chapter 37: Regular Expressions](#) can, and one of its methods, `matchAll`, even hands you the matches with an iterator.

37 Regular Expressions

What is it?

A **regular expression** (or **regex** for short) is a search pattern. Instead of saying *exactly* what text you're looking for, you describe what it *looks like*.

"Find the word `cat` " is a normal search. "Find any five-digit number" is a pattern, and that's what regular expressions are for.

Why does it matter?

In [chapter 06](#), you searched text with `includes` , `indexOf` , and `replace` . They're great, but they only find **exact** text. Real-world jobs often need patterns:

- Pull every price out of a receipt, whatever the prices are.
- Check that a ZIP code is exactly five digits.
- Clean up phone numbers that people typed as `(555) 123-4567` , `555.123.4567` , or `555 123 4567` .
- Find every date like `2026-09-24` in a long log file.

Without regex, each of those is a long, fiddly loop. With regex, it's often one line, once you can read the pattern.

Let's be honest, though: regex looks like a cat walked across the keyboard at first. `/^\d{5}$/` isn't friendly. That's normal, and it gets easier fast. In this chapter, you'll build every pattern one small piece at a time, and each one gets a plain-English explanation.

Real-world example

Imagine you're picking up a friend's cousin at the airport. You've never met her, so you can't look for her face. Instead, your friend describes her: *"curly hair, a green jacket, and a guitar case."* You scan the crowd for anyone who fits.

A normal search is looking for someone you already know. A regex is the description. It doesn't say *who* you want, only what they look like.

Describing text works the same way:

You'd say	As a regex
"the word <code>cat</code> "	<code>/cat/</code>

You'd say	As a regex
"any digit"	<code>/\d/</code>
"exactly five digits, and nothing else"	<code>/^\d{5}\$/</code>
"a date like 2026-09-24"	<code>/\d{4}-\d{2}-\d{2}/</code>

By the end of this chapter, you'll be able to read and write every one of those.

How it works

Your first regex: `test`

A regex goes between two forward slashes, the way a string goes between quotes. The simplest question you can ask it is "does this pattern appear anywhere in the text?" The `test` method answers with `true` or `false`:

```
const pattern = /cat/;

console.log(pattern.test("I love my cat")); // prints: true
console.log(pattern.test("I love my dog")); // prints: false
console.log(pattern.test("Concatenate")); // prints: true
```

The last one surprises people. A regex looks for its letters *anywhere* in the text, even in the middle of a longer word: con-**cat**-enate. You'll see how to be stricter soon.

Flags: `i`, `g`, and `m`

Letters after the closing slash are **flags**: settings that change how the whole regex behaves.

```
console.log(/cat/.test("CAT")); // prints: false (case matters by default)
console.log(/cat/i.test("CAT")); // prints: true
```

You'll meet three flags in this chapter:

Flag	Name	What it does
<code>i</code>	ignore case	<code>a</code> and <code>A</code> count as the same letter
<code>g</code>	global	find every match, not only the first one
<code>m</code>	multiline	makes <code>^</code> and <code>\$</code> (coming soon) work on each line, not just the whole text

You can combine flags, like `/cat/gi`.

match : what did it find?

`test` only says yes or no. To see *what* matched, use the string method `match`. With the `g` flag, it gives you an array of every match:

```
const order = "2 bagels, 1 coffee, 3 cookies";

console.log(order.match(/\d/g)); // prints: [ '2', '1', '3' ]
console.log(order.match(/tea/g)); // prints: null
```

Two things to notice:

- `\d` means "any digit". It's your first special piece, and there are more below.
- When nothing matches, you get `null`, not an empty array. Keep that in mind: it's the first of the common mistakes at the end.

Without `g`, `match` stops at the first match and tells you more about it:

```
const order = "2 bagels, 1 coffee, 3 cookies";
console.log(order.match(/\d/));
```

You'll see:

```
[
  '2',
  index: 0,
  input: '2 bagels, 1 coffee, 3 cookies',
  groups: undefined
]
```

It's still an array with the match at `[0]`, plus a few extras: `index` is where the match starts in the text, and `groups` will make sense later in this chapter.

While you're learning, `match` with `g` is your best friend. It shows you exactly what a pattern finds.

Building a regex from a variable: `new RegExp`

Sometimes the pattern comes from a variable, like a word someone typed into a search box. Then you create the regex with `new RegExp(text, flags)`:

```
const searchWord = "tea";
const pattern = new RegExp(searchWord, "i");

console.log(pattern);           // prints: /tea/i
console.log(pattern.test("Green TEA")); // prints: true
```

When you know the pattern in advance, the `/.../` form is shorter, so use that.

Character classes: `\d`, `\w`, `\s`, and `.`

A **character class** is a piece that matches **one** character out of a whole group of characters.

Piece	Matches one...	For example
<code>\d</code>	digit	0 to 9
<code>\w</code>	"word character": an English letter (a to z, A to Z), a digit, or <code>_</code>	a, Z, 7, <code>_</code> (but not <code>é</code>)
<code>\s</code>	whitespace character: a space, a tab, or a new line	" "
<code>.</code>	character of any kind (except a new line)	a, 7, !, " "

The capital versions mean the opposite: `\D` is "anything that's *not* a digit", `\W` is "not a word character", and `\S` is "not whitespace".

```
const address = "Room 12B, floor 3";

console.log(address.match(/\d/g)); // prints: [ '1', '2', '3' ]
console.log(address.match(/\s/g)); // prints: [ ' ', ' ', ' ' ]

console.log("Hi, Sam_99!".match(/\W/g)); // prints: [ ', ', ' ', '!' ]
console.log("cat cot cut".match(/c.t/g)); // prints: [ 'cat', 'cot', 'cut' ]
```

Read `c.t` as "a `c`, then any one character, then a `t`". Normal letters like `c` and `t` just match themselves.

Quantifiers: how many?

So far, each piece matches exactly one character. A **quantifier** goes right after a piece and says how many of it you want.

Quantifier	Means	Example	Matches
<code>+</code>	one or more	<code>\d+</code>	7, 42, 2026

Quantifier	Means	Example	Matches
*	zero or more	go*al	gal , goal , goooaal
?	zero or one (it's optional)	colou?r	color , colour
{3}	exactly 3	\d{3}	555
{2,4}	from 2 to 4	\d{2,4}	12 , 123 , 1234
{2,}	2 or more	\d{2,}	12 , 12345

+ is the one you'll use most. Look at the difference it makes:

```
const receipt = "2 coffees for $12, 1 muffin for $4";

console.log(receipt.match(/\d/g)); // prints: [ '2', '1', '2', '1', '4' ]
console.log(receipt.match(/\d+/g)); // prints: [ '2', '12', '1', '4' ]
```

\d finds single digits, so `12` gets chopped into `1` and `2`. **\d+** means "one or more digits in a row", so it grabs whole numbers.

? makes the piece right before it optional. Here it's only the `u`:

```
const color = /colou?r/;

console.log(color.test("color")); // prints: true
console.log(color.test("colour")); // prints: true
```

Character sets: `[abc]`, `[a-z]`, and `[^abc]`

Square brackets make a **character set**: a piece that matches any **one** character from a list you choose. (Regex just uses the word "set" too. It has nothing to do with the `Set` from [chapter 35](#).)

Piece	Matches one...
<code>[aeiou]</code>	vowel
<code>[a-z]</code>	lowercase letter from <code>a</code> to <code>z</code> (a dash inside brackets makes a range)
<code>[A-Za-z]</code>	letter, lowercase or uppercase (ranges can be combined)
<code>[0-9]</code>	digit, the same as <code>\d</code>
<code>[^0-9]</code>	character that is not a digit (a <code>^</code> right after <code>[</code> means "not")

```
console.log("banana bread".match(/[aeiou]/g)); // prints: [ 'a', 'a', 'a', 'e', 'a' ]
console.log("Size: XL".match(/[A-Z]+/g));      // prints: [ 'S', 'XL' ]
```

The second one is a set plus a quantifier: "one or more capital letters in a row".

Remember the ZIP code field from [chapter 22](#), with `pattern="[0-9]{5}"`? Now you can read it: "a character from 0 to 9, exactly five times".

Anchors: `^` and `$`

The patterns so far match *anywhere* in the text. That's perfect for searching, but wrong for checking what someone typed. Watch:

```
const zipCode = /\d{5}/;

console.log(zipCode.test("90210")); // prints: true
console.log(zipCode.test("902101")); // prints: true (it contains five digits in a row)
console.log(zipCode.test("zip 90210")); // prints: true
```

An **anchor** pins the pattern to a position. `^` means "the start of the text" and `$` means "the end of the text". Put them around your pattern, and the *whole* text has to match:

```
const zipCode = /^d{5}$/;

console.log(zipCode.test("90210")); // prints: true
console.log(zipCode.test("9021")); // prints: false (too short)
console.log(zipCode.test("902101")); // prints: false (too long)
console.log(zipCode.test("zip 90210")); // prints: false (extra text)
```

Piece	Meaning
<code>^</code>	the text starts here
<code>\d{5}</code>	exactly five digits
<code>\$</code>	the text ends here

A good rule: when you're **searching**, leave the anchors off. When you're **checking** input, put `^` at the start and `$` at the end. (The HTML `pattern` attribute from chapter 22 adds them for you, which is why `[0-9]{5}` worked there. With the `m` flag, `^` and `$` match at the start and end of *every line* instead.)

Replacing with a pattern

`replace` from [chapter 06](#) also accepts a regex. Add the `g` flag, and it replaces **every** match:

```
const phone = "(555) 123-4567";
console.log(phone.replace(/\D/g, "")); // prints: 5551234567
```

`\D` means "any character that isn't a digit", and `g` means "all of them". So every bracket, space, and dash is replaced with nothing. In chapter 06, `replace("-", "")` could only remove one exact dash.

Another everyday cleanup, squashing extra spaces:

```
const messy = "Too    many    spaces";
console.log(messy.replace(/\s+/g, " ")); // prints: Too many spaces
```

`\s+` finds each run of whitespace (one or more spaces or tabs in a row), and each run becomes a single space.

The replacement can even be a function: a callback, like the ones from [chapter 13](#). It receives each match and returns what to put in its place. Here's a recipe, doubled:

```
const recipe = "2 eggs, 100 g flour, 1 cup milk";
const doubled = recipe.replace(/\d+/g, (amount) => Number(amount) * 2);

console.log(doubled); // prints: 4 eggs, 200 g flour, 2 cup milk
```

Each match arrives as a string, like `"100"`, so we convert it with `Number` before doing math.

Tip: `replaceAll` accepts a regex too, but only one with the `g` flag. Without it, you get `TypeError: String.prototype.replaceAll called with a non-global RegExp argument`. With a regex, `replace` plus `g` does the same job, so most people use that.

Splitting with a pattern

`split` accepts a pattern as well. That helps with messy lists, like tags someone typed with random spaces:

```
const tags = "js, css,html ,  node";

console.log(tags.split(",")); // prints: [ 'js', ' css', 'html ', ' node' ]
console.log(tags.split(/\s*,\s*/)); // prints: [ 'js', 'css', 'html', 'node' ]
```

Piece	Meaning
\s*	any amount of whitespace, even none
,	a comma
\s*	any amount of whitespace again

So the text is cut at every comma, and the spaces around each comma are cut away with it.

Groups: ()

Let's build a date pattern, one piece at a time, using a line from a log file. `[0]` is the whole match:

```
const log = "Backup finished on 2026-09-24 at 03:15";

console.log(log.match(/\d{4}/)[0]);           // prints: 2026
console.log(log.match(/\d{4}-\d{2}/)[0]);      // prints: 2026-09
console.log(log.match(/\d{4}-\d{2}-\d{2}/)[0]); // prints: 2026-09-24
```

Piece	Meaning
\d{4}	four digits: the year
-	a dash (a normal character matches itself)
\d{2}	two digits: the month
-	another dash
\d{2}	two digits: the day

That finds the date. But what if you want the year, month, and day **separately**? Wrap each part in parentheses. A **group** `( )` **captures** whatever matched inside it, and `match` hands you each group after the whole match:

```
const log = "Backup finished on 2026-09-24 at 03:15";
const result = log.match(/(\d{4})-(\d{2})-(\d{2})/);

console.log(result[0]); // prints: 2026-09-24 (the whole match)
console.log(result[1]); // prints: 2026 (group 1)
console.log(result[2]); // prints: 09 (group 2)
console.log(result[3]); // prints: 24 (group 3)
```

In a replacement text, `$1`, `$2`, and `$3` stand for the groups. That makes it easy to reorder the parts:

```
const isoDate = "2026-09-24";
console.log(isoDate.replace(/(\d{4})-(\d{2})-(\d{2})/, "$3/$2/$1")); // prints: 24/09/2026
```

Groups also let a quantifier apply to several characters at once: `(ha)+` matches `ha`, `haha`, and `hahaha`.

Named groups: `(?<name>...)`

`result[2]` is easy to mix up with `result[3]`. A **named group** gives each part a name, like `(?<year>\d{4})`, and the parts appear on `result.groups`. Destructuring from [chapter 15](#) makes it tidy:

```
const log = "Backup finished on 2026-09-24 at 03:15";
const result = log.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);

const { year, month, day } = result.groups;
console.log(`Day ${day}, month ${month}, year ${year}`); // prints: Day 24, month 09, year 2026
```

If you print `result.groups` directly, Node shows `[Object: null prototype] { year: '2026', month: '09', day: '24' }`. That label only means it's an object with no prototype ([chapter 28](#)). You can use it like any other object. In a replacement text, named groups are written `$<name>`, like `"$<day>/${<month>}/${<year>"`.

`matchAll` : every match, with its groups

`match` with `g` gives you every match, but it leaves out the groups. `matchAll` gives you both. It returns an iterator ([chapter 36](#)), so you can loop over it with `for...of`:

```
const notice = "Deliveries: 2026-09-24, 2026-10-01 and 2026-10-15";
const datePattern = /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/g;

for (const match of notice.matchAll(datePattern)) {
  const { month, day } = match.groups;
  console.log(`Delivery on ${day}/${month}`);
}
```

You'll see:

```
Delivery on 24/09
Delivery on 01/10
Delivery on 15/10
```

Each `match` is the same kind of array you get from `match` without `g`. `matchAll` insists on the `g` flag. Without it, you get `TypeError: String.prototype.matchAll called with a non-global RegExp argument`.

Alternation: | means "or"

A `|` between two patterns means "this one or that one":

```
const pets = "I have a cat, a dog, and a fish";
console.log(pets.match(/cat|dog/g)); // prints: [ 'cat', 'dog' ]
```

To accept one of a few exact choices, put the choices in a group and add anchors:

```
const weekend = /^(Saturday|Sunday)$/;

console.log(weekend.test("Sunday")); // prints: true
console.log(weekend.test("Monday")); // prints: false
```

In plain words: the text starts (`^`), then comes either `Saturday` or `Sunday`, and then the text ends (`$`).

Escaping special characters

Some characters have special jobs in a regex: `.` `*` `+` `?` `^` `$` `( )` `[ ]` `{ }` `|` `\` `/`. To match one of them as a plain character, put a backslash in front of it. That's called **escaping** it, just like `"` inside a string in chapter 06.

Here's a pattern for prices on a menu:

```
const menu = "Tea $3.50, Cake $4.25";
console.log(menu.match(/\$\d+\.\d{2}/g)); // prints: [ '$3.50', '$4.25' ]
```

Piece	Meaning
<code>\\$</code>	a real dollar sign (a bare <code>\$</code> would mean "end of the text")
<code>\d+</code>	one or more digits: the dollars
<code>\.</code>	a real dot (a bare <code>.</code> would mean "any character")
<code>\d{2}</code>	exactly two digits: the cents

Real-world pattern: phone numbers

People type phone numbers in all sorts of ways. One regex that accepts every possible style gets huge and hard to read. Two tiny steps are much easier: throw away everything that isn't a digit, then check that exactly ten digits are left.

```
function cleanPhone(input) {
  const digits = input.replace(/\D/g, ""); // keep only the digits
  return /^\d{10}$/.test(digits) ? digits : null;
}

console.log(cleanPhone("(555) 123-4567")); // prints: 5551234567
console.log(cleanPhone("555.123.4567")); // prints: 5551234567
console.log(cleanPhone("123-4567")); // prints: null
```

Ten digits is the US style. Other countries use other lengths, so adjust the `{10}` for the numbers your app expects.

Real-world pattern: dates like 2026-09-24

```
const isoDate = /^\d{4}-\d{2}-\d{2}$/;

console.log(isoDate.test("2026-09-24")); // prints: true
console.log(isoDate.test("24/09/2026")); // prints: false
console.log(isoDate.test("2026-13-45")); // prints: true (!)
```

Look at the last one. A regex checks the **shape** of text, not its meaning. It has no idea that there's no month 13. So check the shape with a regex, then check the numbers with normal code or a `Date` ([chapter 19](#)).

Real-world pattern: a simple email check

```
const email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

console.log(email.test("sam@example.com")); // prints: true
console.log(email.test("sam@example")); // prints: false (no dot after the @)
console.log(email.test("sam smith@example.com")); // prints: false (a space)
console.log(email.test("sam@@example.com")); // prints: false (two @ signs)
```

Piece	Meaning
<code>^</code>	the text starts here
<code>[^\s@]+</code>	one or more characters that aren't a space or an <code>@</code> (the name)
<code>@</code>	exactly one <code>@</code>
<code>[^\s@]+</code>	more of the same (the domain, like <code>example</code>)
<code>\.</code>	a real dot

Piece	Meaning
<code>[^\s@]+</code>	more of the same (the ending, like <code>com</code>)
<code>\$</code>	the text ends here

Here's the honest truth about email: the official rules are so complicated that a "perfect" email regex would be enormous, and it *still* couldn't tell you whether the address exists. Real apps do a simple shape check like this one, then send a confirmation email. That's the only way to know an address really works.

Keeping regexes readable

- **Keep them small.** Two simple regexes, or one regex plus a line of normal code (like `cleanPhone`), beat one giant pattern.
- **Give them good names.** `const zipCode = /\d{5}$/;` tells the reader what it's for.
- **Explain them in a comment**, in plain words, like the tables in this chapter.
- **Test them at regex101.com.** Paste your pattern and some sample text. It highlights every match and explains each piece in plain English. Make sure the flavor is set to JavaScript (it may be listed as ECMAScript, the official name of the JavaScript standard).

Common mistakes

1. Forgetting that `match` can give you `null`

```
const note = "No numbers here";
const count = note.match(/\d/g).length;
// TypeError: Cannot read properties of null (reading 'length')
```

When nothing matches, `match` returns `null`, and `null` has no `length`. Fall back to an empty array with `??` from [chapter 07](#): `(note.match(/\d/g) ?? []).length` gives `0`.

2. Forgetting the anchors when checking input

```
const pin = /\d{4}/;
console.log(pin.test("12345678")); // prints: true
```

Without `^` and `$`, this only asks "are there four digits in a row somewhere?" To check that the *whole* input is four digits, write `/^\d{4}$/`.

3. Forgetting to escape a special character

```
const text = "Call 4550 or pay $4.50";
console.log(text.match(/\d+\.\d{2}/g)); // prints: [ '4550', '4.50' ]
```

A bare `.` means "any character", so `4550` sneaks in: the `.` matched the second `5`. Escape it to mean a real dot: `/\d+\.\d{2}/g` finds only `4.50`.

4. Using `|` without a group

```
const size = /^S|M|L$/;
console.log(size.test("Medium")); // prints: true
```

`|` splits the *whole* pattern, so this means "starts with `S`, or contains `M`, or ends with `L`". "Medium" contains an `M`, so it passes. Put the choices in a group: `/(S|M|L)$/`.

5. Reusing a regex with `g` in `test`

```
const hasCat = /cat/g;

console.log(hasCat.test("cat")); // prints: true
console.log(hasCat.test("cat")); // prints: false
console.log(hasCat.test("cat")); // prints: true
```

With the `g` flag, a regex remembers where its last match ended (in a property called `lastIndex`), and the next `test` carries on searching from there. That's why the answer flips. `test` only needs a yes or no, so leave the `g` off: `/cat/`.

6. Single backslashes in `new RegExp`

```
const digits = new RegExp("\d+");
console.log(digits); // prints: /d+/
```

Inside a normal string, a backslash starts an escape ([chapter 06](#)), and `"\d"` quietly turns into plain `"d"`. So this regex looks for the letter `d`! In a string, write the backslash twice: `new RegExp("\\d+")` gives you `/\d+/`.

Quick recap

- A regex describes what text *looks like*: `/pattern/flags`, or `new RegExp(text, flags)` when the pattern is in a variable.
- The pieces: `\d`, `\w`, `\s`, and `.` match one character from a group. `[...]` matches one character from your own list, and `[^...]` means "not these". Quantifiers (`+`, `*`, `?`, `{n}`, `{n,m}`) say how many.
- `^` and `$` pin the pattern to the start and end. Use them whenever you're checking input.

- `test` gives `true` or `false`. `match` with `g` gives every match (or `null`). `matchAll` gives every match with its groups. `replace` and `split` accept patterns too.
 - Groups `( )` capture parts of a match, named groups `(?<name>...)` make them readable, and `|` means "or".
 - Escape special characters with a backslash to match them as plain text: `\.`, `\$`, `\?`.
 - Keep regexes small, name them, and test them on regex101.com.
-

Next: try the [exercises](#), then move on to [38 Type Coercion](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Build each regex a piece at a time. Print what it finds with `match` and the `g` flag, and add the next piece only when the current one works. regex101.com is great for experimenting.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Numbers on a receipt

A grocery receipt came through as one line of text:

```
const receipt = "3 apples $2, 12 eggs $5, 1 loaf of bread $3";
```

Use `match` to pull out:

1. Every number on the receipt.
2. Only the prices, with their dollar signs.
3. Then add up the prices and print the total.

Expected output:

```
All numbers: [ '3', '2', '12', '5', '1', '3' ]
Prices: [ '$2', '$5', '$3' ]
Total: $10
```

Rule: don't type any of the numbers yourself. Let your regexes find them.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Gym membership IDs

Every member of a gym has an ID like GYM-0042 : the letters GYM , a dash, then exactly four digits. The front desk wants to check IDs before looking them up.

```
const ids = ["GYM-0042", "GYM-42", "gym-0042", "GYM-00420", "MY GYM-0042", "GYM-12A4"];
```

Write **one** regex for a valid ID and test every ID in the array. Then the manager says lowercase letters are fine too. Make a second version of your regex that accepts gym-0042 by adding one flag, and test that ID with it.

Expected output:

```
GYM-0042: valid
GYM-42: invalid
gym-0042: invalid
GYM-00420: invalid
MY GYM-0042: invalid
GYM-12A4: invalid
With the i flag, gym-0042 is valid
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Tidy phone numbers

A small business is importing its contact list, and everyone typed phone numbers differently:

```
const contacts = ["(555) 123-4567", "555.987.6543", "555 222 3333", "call me!", "555-12-345"];
```

Write a function `formatPhone(input)` that returns the number in one tidy style, 555-123-4567 , or the text "invalid" if it isn't a 10-digit number. Then print every contact.

It takes three steps:

1. Remove every character that isn't a digit.
2. Check that exactly ten digits are left. If not, return "invalid" .
3. Split the digits into groups of 3, 3, and 4, with dashes between them.

Expected output:

```
(555) 123-4567 -> 555-123-4567
555.987.6543 -> 555-987-6543
555 222 3333 -> 555-222-3333
call me! -> invalid
555-12-345 -> invalid
```

Rule: for step 3, use `replace` with a regex that has three groups. No `slice`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Appointment reminders

A calendar app exports your week as one long line:

```
const schedule =
  "Dentist on 2026-10-03 at 09:30, Gym on 2026-10-04 at 18:00, Haircut on 2026-10-10 at 12:15";
```

Write **one** regex with **named groups** for the appointment's name, year, month, day, and time. Use `matchAll` to print a reminder for each appointment in day/month/year order, then say how many you found.

Expected output:

```
03/10/2026 at 09:30: Dentist
04/10/2026 at 18:00: Gym
10/10/2026 at 12:15: Haircut
3 appointments found
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Sign-up checker

A music-streaming site checks new sign-ups before creating accounts. In [chapter 22](#), you read form values in the browser. Here, the values are already collected as objects:

```
const signups = [
  { username: "sandip_dev", email: "sandip@example.com", password: "Secret123", zip: "90210" },
  { username: "2cool", email: "cool@example", password: "password", zip: "9021" },
  { username: "Al", email: "al@mail.com", password: "Sh0rtPw", zip: "10001" },
];
```

Write a function `validateSignup(form)` that returns an **array of error messages**, or an empty array when everything is fine. Check these rules in this order, and use these exact messages:

Rule	Message when it's broken
Username: 3 to 15 letters, digits, or <code>_</code> , and it must start with a letter	Username must be 3-15 letters, digits or <code>_</code> , starting with a letter
Email: the simple shape check from the notes	Email doesn't look right
Password: at least 8 characters	Password needs at least 8 characters
Password: at least one uppercase letter	Password needs an uppercase letter
Password: at least one digit	Password needs a digit
ZIP code: exactly 5 digits	ZIP code must be exactly 5 digits

Then print a report for every sign-up.

Expected output:

```
sandip_dev: all good!
2cool:
  - Username must be 3-15 letters, digits or _, starting with a letter
  - Email doesn't look right
  - Password needs an uppercase letter
  - Password needs a digit
  - ZIP code must be exactly 5 digits
Al:
  - Username must be 3-15 letters, digits or _, starting with a letter
  - Password needs at least 8 characters
```

Rule: don't write one giant regex for the password. Use small, separate checks, as the notes suggest. (One of them doesn't need a regex at all.)

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Bonus: add a rule that the username can't contain the word `admin` in any mix of capitals (`Admin`, `ADMIN`, ...), with the message `Username can't contain "admin"`.

Before you move on

In Exercise 1, `match` gave you the prices as strings, like `"$5"`, and you had to turn them into numbers before adding them up. What if you forget? Try this in Node:

```
console.log("2" + "5" + "3");
```

You get `253` instead of `10`. JavaScript quietly joined the text instead of adding. Surprises like that are the topic of [Chapter 38: Type Coercion](#).

38 Type Coercion

What is it?

Type coercion is JavaScript changing a value from one type to another *by itself*, because an operation needs a different type. You don't ask for it. It just happens.

```
console.log("5" * 2); // prints: 10
```

* only works with numbers, so JavaScript quietly turned `"5"` into `5` first.

When *you* convert a value on purpose, with `Number()`, `String()` or `Boolean()`, that's called **explicit conversion**. Coercion is the automatic kind, so it's also called **implicit conversion**.

Why does it matter?

At the end of [chapter 37](#), this line surprised you:

```
console.log("2" + "5" + "3"); // prints: 253
```

You wanted `10`, and JavaScript gave you `253`. No error, no warning, just a wrong answer. That's what makes coercion bugs sneaky: the program keeps running, and the wrong value travels on until something far away looks odd, like a cart that says you ordered 55 notebooks.

Knowing the rules helps you:

- spot these bugs quickly when they happen,
- read older code that relies on coercion, including code that uses `==`,
- write code that never depends on coercion at all.

This chapter also keeps two old promises. [Chapter 04](#) said you'd find out why `true` turns into `1`, and [chapter 35](#) said you'd find out where that odd `"[object Object]"` comes from.

Real-world example

Type coercion is like **autocorrect** on your phone:

Autocorrect	Type coercion
Changes your words without asking	Changes types without asking

Autocorrect	Type coercion
Usually guesses what you meant	Usually does what you meant
Sometimes turns your friend "Anya" into "Anyway"	Sometimes turns <code>"5" + 5</code> into <code>"55"</code> , when you wanted 10
You stay in charge by reading before you hit Send	You stay in charge by converting values yourself and comparing with <code>===</code>

Autocorrect is handy, until the day it sends the wrong message. The fix isn't to memorize every way it can go wrong. It's to check what you're sending.

How it works

Explicit vs. implicit, side by side

```
const typed = "5"; // text, like a value from a form

// Explicit: you decide, and anyone reading the code can see it
console.log(Number(typed) + 1); // prints: 6

// Implicit: JavaScript decides, following its own rules
console.log(typed + 1); // prints: 51
console.log(typed - 1); // prints: 4
```

Same value, three different results. The explicit line says exactly what happens. The implicit lines depend on rules you have to remember, and those rules aren't even the same for `+` and `-`.

Every conversion ends up as one of three types: a string, a number, or a boolean. Let's take them one at a time.

Converting to a string

Explicit: `String(value)`. **Implicit:** `+` with a string on one side, and template literals.

Primitives turn into text the way you'd expect. They're just written out:

```
console.log(String(42));           // prints: 42
console.log(typeof String(42));    // prints: string
console.log(String(true));         // prints: true
console.log(String(null));         // prints: null
console.log(`Seats left: ${3}`);   // prints: Seats left: 3
```

Arrays and objects are more interesting:

```
console.log(String([1, 2, 3]));    // prints: 1,2,3
console.log(String({ size: "M" })); // prints: [object Object]

const user = { name: "Maya" };
console.log(`Hello, ${user}!`); // prints: Hello, [object Object]!
```

- An **array** becomes its items joined with commas, just like `join(",")` from chapter 10. An empty array becomes an empty string.
- A plain **object** becomes the text `"[object Object]"`, no matter what's inside it.

So where does `"[object Object]"` come from? When JavaScript needs an object as a string, it calls the object's `toString()` method. Every object has one, shared through `Object.prototype` ([chapter 28](#)). For a plain object, the built-in `toString()` isn't very helpful. It only says "this is an object, of the kind `Object`". Arrays have a smarter `toString()` that joins their items:

```
console.log({ size: "M" }.toString()); // prints: [object Object]
console.log([1, 2, 3].toString());    // prints: 1,2,3
```

That's exactly what happened in chapter 35, when two customer objects were used as keys in a plain object. Object keys must be strings, so both customers turned into the same key, `"[object Object]"`, and one overwrote the other.

To show what's inside an object, print it on its own (`console.log("User:", user)`), pick the property you need (`${user.name}`), or use `JSON.stringify(user)` from chapter 23.

Converting to a number

Explicit: `Number(value)`. **Implicit:** math operators like `-`, `*`, `/` and `%`, and comparisons like `<` and `>`.

You met `Number()` in chapters 03 and 05. Here's the full picture, including values you haven't tried yet:

Value	<code>Number(value)</code>	Why
<code>"42"</code>	<code>42</code>	the text is a number
<code>" 42 "</code>	<code>42</code>	spaces around the number are ignored
<code>""</code>	<code>0</code>	empty text counts as zero (a real trap with forms)
<code>"42px"</code>	<code>NaN</code>	the <i>whole</i> text must be a number
<code>true</code>	<code>1</code>	true counts as 1...

Value	Number(value)	Why
false	0	...and false as 0
null	0	"empty, on purpose" counts as zero
undefined	NaN	"no value yet" isn't a number
[]	0	the array becomes "" first, and "" becomes 0
[5]	5	the array becomes "5" first, and "5" becomes 5
{}	NaN	the object becomes "[object Object]" first, which isn't a number

Look at the last three rows. An array or object is first turned into a string, using the rules from the previous section, and then that string is turned into a number.

Try the strangest ones yourself:

```
console.log(Number(""));      // prints: 0
console.log(Number(null));    // prints: 0
console.log(Number(undefined)); // prints: NaN
console.log(Number([]));      // prints: 0
console.log(Number({}));      // prints: NaN
```

Two pairs are worth remembering: `null` becomes `0` but `undefined` becomes `NaN`, and empty text becomes `0`, not `NaN`.

You'll also see a plus sign used in front of a value, like `+"42"`. That's a short way to write `Number("42")`, and it gives `42`. It's common in other people's code, but `Number()` is much easier to read.

Converting to a boolean

Explicit: `Boolean(value)`, or the shortcut `!!value`. **Implicit:** anywhere JavaScript needs a yes-or-no answer, like `if`, `while`, the ternary `? : ,` and `!`, `&&` and `||`. (`&&` and `||` check whether a value is truthy, but hand back one of the original values, as you saw in chapter 07.)

This is the truthy and falsy idea from [chapter 07](#). Only eight values are falsy: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined` and `NaN`. Everything else is truthy, including some values that *look* false:

```

console.log(Boolean("0"));    // prints: true
console.log(Boolean("false")); // prints: true
console.log(Boolean(" "));    // prints: true
console.log(Boolean([]));     // prints: true
console.log(Boolean({}));     // prints: true
console.log(!!"hello");       // prints: true
console.log(!!0);             // prints: false

```

"0" and "false" are strings with characters in them, so they're truthy. That matters when a value comes from a form or a settings file: `if ("0")` runs its block.

`!!` is two `!` in a row. The first `!` converts the value to a boolean and flips it, and the second flips it back. It means the same as `Boolean(value)`, and you'll see it a lot in real code.

+ prefers strings

`+` has two jobs: adding numbers and joining strings. If **either** side is a string, `+` joins. Otherwise, it adds.

```

console.log("5" + 2);        // prints: 52
console.log(5 + "2");        // prints: 52
console.log(1 + 2 + "3");    // prints: 33
console.log("1" + 2 + 3);    // prints: 123

```

The last two lines look almost the same but give different answers. `+` works from left to right, one step at a time:

- `1 + 2 + "3"`: first, `1 + 2` is `3` (two numbers, so it adds). Then `3 + "3"` is `"33"`.
- `"1" + 2 + 3`: first, `"1" + 2` is `"12"` (a string, so it joins). Then `"12" + 3` is `"123"`.

Chapter 37's `"2" + "5" + "3"` works the same way. Every step has a string in it, so every step joins, and you get `"253"`.

-, *, / and % always do math

These operators only know how to do math, so they turn both sides into numbers first:

```

console.log("5" - 2);        // prints: 3
console.log("5" * "2");      // prints: 10
console.log("10" / "4");     // prints: 2.5
console.log("abc" - 1);      // prints: NaN

```

And when *neither* side of `+` is a string, `+` does math too, so booleans and `null` get turned into numbers:

```
console.log(true + true);    // prints: 2
console.log(null + 1);       // prints: 1
console.log(undefined + 1);  // prints: NaN
```

`true + true` is `2` because each `true` becomes `1`. `null + 1` is `1` because `null` becomes `0`.

So `"5" - 2` gives the number `3`, while `"5" + 2` gives the string `"52"`. One operator always does math, and the other prefers joining. That difference is exactly what makes `+` risky whenever a value might secretly be a string.

Comparing with `<` and `>`

`<`, `>`, `<=` and `>=` turn both sides into numbers, **unless both sides are strings**. Two strings are compared character by character, like words in a dictionary:

```
console.log("10" < 9);    // prints: false
console.log("10" < "9");  // prints: true
```

In the first line, `"10"` becomes `10`, and `10 < 9` is false. In the second line, both sides are strings, so JavaScript compares the first characters. `"1"` comes before `"9"`, so `"10"` counts as the smaller one. It never looks at the numbers at all. (In this character order, capital letters come before lowercase ones, so `"Zebra" < "apple"` is `true` too.)

This bites when two numbers both arrive as text, like two prices typed into a form. Convert first, then compare.

Why `true` becomes `1`

Here's the puzzle from chapter 04:

```
const age = 70;
console.log(18 <= age <= 65); // prints: true
```

JavaScript works from left to right. First, `18 <= age` is `true`. Then it has to work out `true <= 65`. A comparison needs numbers, so `true` is converted the same way `Number(true)` converts it, and that's `1`. Finally, `1 <= 65` is `true`.

But why 1? It's a habit that's much older than JavaScript: in computing, 1 has long stood for "yes" or "on", and 0 for "no" or "off". You can even see it on the power switch of a kettle or a computer, where the "I" and "O" marks come from 1 (on) and 0 (off). So whenever JavaScript needs a number, `true` becomes `1` and `false` becomes `0`. (Chapter 04's fix still stands: `age >= 18 && age <= 65`.)

`==` and `===`

`===` (**strict equality**) never converts anything. If the two types are different, the answer is `false`, full stop.

`==` (**loose equality**) converts values before it compares them. Chapter 04 promised you the rules it follows. Here they are, simplified:

1. **Same type on both sides?** Then `==` works just like `===`.
2. **`null` and `undefined`** are equal to each other, and to nothing else.
3. **A number and a string?** The string is turned into a number.
4. **A boolean on either side?** The boolean is turned into a number first: `true` becomes 1 and `false` becomes 0.
5. **An object and a primitive?** The object is turned into a primitive first, usually a string, like in "Converting to a string" above.
6. **`NaN`** is never equal to anything, not even to itself.

JavaScript keeps applying these rules until both sides have the same type. Some results:

```
console.log("1" == 1);           // prints: true
console.log("" == 0);            // prints: true
console.log("0" == false);      // prints: true
console.log(null == undefined); // prints: true
console.log(null == 0);          // prints: false
console.log("true" == true);    // prints: false
console.log([] == false);       // prints: true
console.log(NaN == NaN);        // prints: false
```

Let's walk through the strangest ones:

- `"0" == false`: rule 4 turns `false` into `0`, so now it's `"0" == 0`. Rule 3 turns `"0"` into `0`, and `0 == 0` is `true`.
- `"true" == true`: rule 4 turns `true` into `1`, so now it's `"true" == 1`. Rule 3 turns `"true"` into a number, which is `NaN`. And `NaN == 1` is `false`. So the string `"true"` isn't even loosely equal to `true`!
- `null == 0`: rule 2 says `null` only equals `null` and `undefined`, so this is `false`, even though `Number(null)` is `0`.
- `[] == false`: rule 4 turns `false` into `0`. Rule 5 turns `[]` into `""`. Rule 3 turns `""` into `0`. So it's `true`. Yet an empty array is *truthy* (chapter 10)!

That last one is why nobody wants to reason about `==`. With `===`, there's nothing to remember, which is why this course uses `===` everywhere. You'll still meet `==` in older code, and now you can read it.

Tip: one use of `==` that you may see on purpose is `value == null`. Because of rule 2, it's `true` for both `null` and `undefined`, and nothing else. It's fine to recognize, but `??` and `?.` (chapters 07 and 11) cover most of the same needs.

Object.is : the strictest check

Even `===` has two odd cases, both about special numbers:

```
console.log(NaN === NaN);      // prints: false
console.log(0 === -0);         // prints: true
console.log(Object.is(NaN, NaN)); // prints: true
console.log(Object.is(0, -0));  // prints: false
```

`Object.is(a, b)` works like `===`, with two differences: it treats `NaN` as equal to itself, and it can tell `0` and `-0` apart. (That's the negative zero from chapter 07's list of falsy values. You get it from things like `0 * -1`.)

You'll rarely need `Object.is`. To check for `NaN`, `Number.isNaN()` from chapter 05 is clearer, and `-0` almost never matters in everyday code. But now you know why it exists.

Objects are still compared by reference

No conversion rule changes what you learned in [chapter 16](#). For objects and arrays, `===` and `Object.is` ask "is this the *same* object?", not "do they look alike?":

```
console.log([] === []);      // prints: false
console.log(Object.is({}, {})); // prints: false
console.log([1, 2] == "1,2"); // prints: true
```

The last line is `==` rule 5 at work. The array is turned into the string `"1,2"` first, so `==` ends up comparing two strings. With `===`, it's `false`, because an array is never the same thing as a string.

A table of famous surprises

These show up in quizzes and in jokes about JavaScript. Every one of them follows the rules above:

Code	Result	Why
<code>"5" + 2</code>	<code>"52"</code>	one side is a string, so <code>+</code> joins
<code>"5" - 2</code>	<code>3</code>	<code>-</code> only does math, so <code>"5"</code> becomes 5
<code>true + true</code>	<code>2</code>	no strings, so <code>+</code> adds, and <code>true</code> is 1

Code	Result	Why
<code>[] + []</code>	<code>""</code>	both arrays become empty strings, then join
<code>[] + {}</code>	<code>"[object Object]"</code>	<code>""</code> joined with the object's <code>toString()</code> text
<code>[1, 2] + [3]</code>	<code>"1,23"</code>	<code>"1,2"</code> joined with <code>"3"</code>
<code>"b" + "a" + +"a" + "a"</code>	<code>"baNaNNa"</code>	<code>+"a"</code> tries to turn <code>"a"</code> into a number and gets <code>NaN</code> , which is then joined in as text
<code>null + 1</code>	<code>1</code>	<code>null</code> becomes 0
<code>undefined + 1</code>	<code>NaN</code>	<code>undefined</code> becomes <code>NaN</code>

Check the trickiest ones yourself. `JSON.stringify` from chapter 23 puts quotes around a string, which makes the empty string easy to see:

```
console.log(JSON.stringify([] + [])); // prints: ""
console.log([] + {});                // prints: [object Object]
console.log([1, 2] + [3]);           // prints: 1,23
console.log("b" + "a" + +"a" + "a"); // prints: baNaNNa
```

`[] + {}` is worth understanding fully, because it pulls everything together. `+` needs primitives, so each side is turned into one. `[]` becomes `""`, an empty array joined. `{}` becomes `"[object Object]"`, the text from its built-in `toString()`. Now one side is a string, so `+` joins: `"" + "[object Object]"` is `"[object Object]"`.

The "5" + 5 cart bug

Remember the shopping cart from [chapter 12](#)? Picture a web version of it. A customer has 5 notebooks in the cart, types 5 into a "How many more?" box, and clicks Add. Form values are always strings ([chapter 22](#)), so this is what your code gets:

```
const cartItem = { name: "Notebook", quantity: 5, priceInCents: 350 };
const typedQuantity = "5"; // from the form, so it's a string

cartItem.quantity = cartItem.quantity + typedQuantity;
console.log(cartItem.quantity); // prints: 55
console.log(typeof cartItem.quantity); // prints: string
console.log(cartItem.quantity * cartItem.priceInCents); // prints: 19250
```

The customer wanted 10 notebooks and got 55! And look at the last line: `*` quietly turns `"55"` back into a number, so the price comes out as \$192.50 without any error. The bug only shows up when someone reads the

receipt.

The fix is to convert at the edge: turn the text into a number as soon as it arrives, check it, and only then use it:

```
const cartItem = { name: "Notebook", quantity: 5, priceInCents: 350 };
const typedQuantity = "5";

const extra = Number(typedQuantity);
if (Number.isInteger(extra) && extra > 0) {
  cartItem.quantity = cartItem.quantity + extra;
}
console.log(cartItem.quantity); // prints: 10
```

Your chapter 12 cart was already protected. `addToCart` checked `Number.isInteger(quantity)`, and `Number.isInteger("5")` is `false`, so a string would have been refused instead of turning into 55.

The golden rules

1. **Use `===` and `!==`.** They never convert anything, so there's nothing to remember.
2. **Convert explicitly, as early as possible.** When a value arrives from outside your code (a form, a URL, an API, text you pulled out with a regex), turn it into the type you need straight away, with `Number()`, `String()` or `Boolean()`. Then the rest of your code only ever sees the right types.
3. **Expect input to be text.** Form fields (chapter 22), `localStorage` (chapter 23), URL query parameters (chapter 33) and regex matches (chapter 37) all give you strings.
4. **Check what a conversion gave you.** `Number("abc")` is `NaN` and `Number("")` is `0`, so check the result with `Number.isNaN()` or `Number.isInteger()`, and handle empty text on purpose.
5. **Don't write code that only works because of coercion.** If a reader needs to know these rules to understand a line, rewrite the line.

Common mistakes

1. Adding numbers that are secretly strings

```
const shirtPrice = "4.50"; // pulled out of some text
const socksPrice = "3.25";
console.log(shirtPrice + socksPrice); // prints: 4.503.25
```

Both prices are strings, so `+` joins them. Fix: convert first. `Number(shirtPrice) + Number(socksPrice)` gives `7.75`.

2. Comparing numbers that are secretly strings

```
const minimumAge = "9";
const age = "10";
console.log(age >= minimumAge); // prints: false
```

Both sides are strings, so they're compared character by character, and "1" comes before "9". A 10-year-old gets turned away from a film for ages 9 and up! Fix: `Number(age) >= Number(minimumAge)` gives `true`.

3. Printing an object inside a template literal

```
const order = { id: 1042, total: 35 };
console.log(`Order: ${order}`); // prints: Order: [object Object]
```

A template literal turns the object into a string with its built-in `toString()`. Fix: print the parts you want, like ``Order ${order.id}``, or give the object to `console.log` separately: `console.log("Order:", order)`.

4. Treating "0" or "false" as false

```
const giftWrap = "0"; // from a form: "0" means "no, thanks"
if (giftWrap) {
  console.log("Adding gift wrap");
}
// prints: Adding gift wrap
```

Every string that isn't empty is truthy, even "0" and "false". Fix: say exactly what you mean, like `if (giftWrap === "1")`, or convert first: `if (Number(giftWrap) > 0)`.

5. Using `==` and trusting the answer

```
const answer = ""; // the user left the box empty
if (answer == 0) {
  console.log("You chose option 0");
}
// prints: You chose option 0
```

`"" == 0` is `true`, because empty text becomes `0`. The user didn't choose anything, but the code thinks they chose 0. Fix: use `===` and handle the empty case on purpose: check `answer === ""` first, then compare `Number(answer) === 0`.

Quick recap

- Type coercion is JavaScript converting types by itself. Explicit conversion, with `Number()`, `String()` or `Boolean()`, is you doing it on purpose.

- To a string: arrays join their items with commas, and plain objects become "[object Object]", from their built-in `toString()`.
 - To a number: `""` and `null` become `0`, while `undefined` and text like `"42px"` become `NaN`. `true` is `1` and `false` is `0`.
 - To a boolean: only the eight falsy values give `false`. `"0"`, `"false"` and `[]` are all truthy.
 - `+` joins if either side is a string. `-`, `*`, `/` and `%` always do math. Two strings are compared character by character.
 - `==` converts before it compares, using rules that are hard to keep in your head. `===` never converts, so use `===`.
 - Convert input explicitly as soon as it arrives, and check the result.
-

Next: try the [exercises](#), then move on to [39 Project: Weather App](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Predict the output

Type these lines into `ex1.js` , but **don't run it yet**. Next to each line, write a comment with what you think it prints.

```
console.log("3" + 4);
console.log("3" - 4);
console.log("3" * "4");
console.log(3 + 4 + "5");
console.log("3" + 4 + 5);
console.log(true + 1);
console.log("7" > "10");
console.log(Number(""));
console.log(Number(" 12 "));
console.log(Boolean("0"));
```

Now run it and compare. For every line you got wrong, add a second comment saying which rule from the notes explains it.

"Check your predictions" and the hint are in the [Hints section](#) at the end.

Exercise 2 (Easy): Cinema booking form

A cinema's booking form sends the number of adults, the number of children, and the ticket price. Form values are always strings, and the code below forgot that:

```
// These arrived from a booking form, so they're all strings.
const adults = "2";
const children = "3";
const ticketPrice = "12";

const people = adults + children;
const cost = people * ticketPrice;

console.log(`People: ${people}`);
console.log(`Cost: ${cost}`);
```

Run it, and you'll see:

```
People: 23
Cost: $276
```

Fix it, so it prints:

```
People: 5
Cost: $60
```

Rule: convert each value once, near the top, and don't change the two `console.log` lines.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Loose, strict, or `Object.is`?

Write a function `compare(label, a, b)` that prints how `==`, `===` and `Object.is` each judge the same pair of values. Test it with:

```
compare('"1" and 1', "1", 1);
compare('"" and 0', "", 0);
compare('"0" and false', "0", false);
compare("null and undefined", null, undefined);
compare("null and 0", null, 0);
compare('"true" and true', "true", true);
compare("NaN and NaN", NaN, NaN);
compare("0 and -0", 0, -0);
compare('[] and ""', [], "");
```

Expected output:

```
"1" and 1 -> ==: true, ===: false, Object.is: false
"" and 0 -> ==: true, ===: false, Object.is: false
"0" and false -> ==: true, ===: false, Object.is: false
null and undefined -> ==: true, ===: false, Object.is: false
null and 0 -> ==: false, ===: false, Object.is: false
"true" and true -> ==: false, ===: false, Object.is: false
NaN and NaN -> ==: false, ===: false, Object.is: true
0 and -0 -> ==: true, ===: true, Object.is: false
[] and "" -> ==: true, ===: false, Object.is: false
```

Then, for every line where `==` says `true` but `===` says `false`, add a comment above that `compare` call naming the `==` rule (from the notes) that made it `true`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): A safe number reader

Text from a form can be anything: a number, extra spaces, words, or nothing at all. Write a function

`readNumber(text)` that returns a number when the text really is one, and `null` when it isn't:

- Ignore spaces at the start and the end.
- Empty text (or only spaces) gives `null`, **not** `0`.
- Text that isn't a number gives `null`.
- Anything else gives the number.

Test it with:

```
const inputs = ["5", " 12 ", "", "   ", "abc", "3.5", "0", "7 apples"];
for (const text of inputs) {
  console.log(`${text} -> ${readNumber(text)}`);
}
```

Expected output:

```
"5" -> 5
" 12 " -> 12
"" -> null
"  " -> null
"abc" -> null
"3.5" -> 3.5
"0" -> 0
"7 apples" -> null
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Debug the checkout

This checkout page gets every value from a web form, so everything is a string. The code runs without a single error, but it has **three coercion bugs**. Copy it into `ex5.js`:

```
// Everything here arrived as text, from a web form.
const order = [
  { name: "Notebook", price: "3.50", quantity: "2" },
  { name: "Backpack", price: "34.99", quantity: "1" },
  { name: "Pen set", price: "6.25", quantity: "3" }
];
const extraNotebooks = "2"; // the customer asked for 2 more notebooks
const giftWrap = "0";      // "0" means no gift wrap, "1" means yes

order[0].quantity = order[0].quantity + extraNotebooks;

let mostExpensive = order[0];
let total = 0;
for (const item of order) {
  console.log(`${item.name} x${item.quantity}`);
  if (item.price > mostExpensive.price) {
    mostExpensive = item;
  }
  total = total + item.price * item.quantity;
}
if (giftWrap) {
  total = total + 2; // gift wrap costs $2
}

console.log(`Most expensive: ${mostExpensive.name}`);
console.log(`Total: ${total.toFixed(2)}`);
```


Right now, it prints:

```
Notebook x22  
Backpack x1  
Pen set x3  
Most expensive: Pen set  
Total: $132.74
```

Find and fix all three bugs, so it prints:

```
Notebook x4  
Backpack x1  
Pen set x3  
Most expensive: Backpack  
Total: $67.74
```

Rule: instead of sprinkling `Number()` all over the code, convert the order's prices and quantities once, before anything else uses them.

Stuck? Hint 1, Hint 2, Hint 3 and Hint 4 are in the [Hints section](#) at the end.

Before you move on

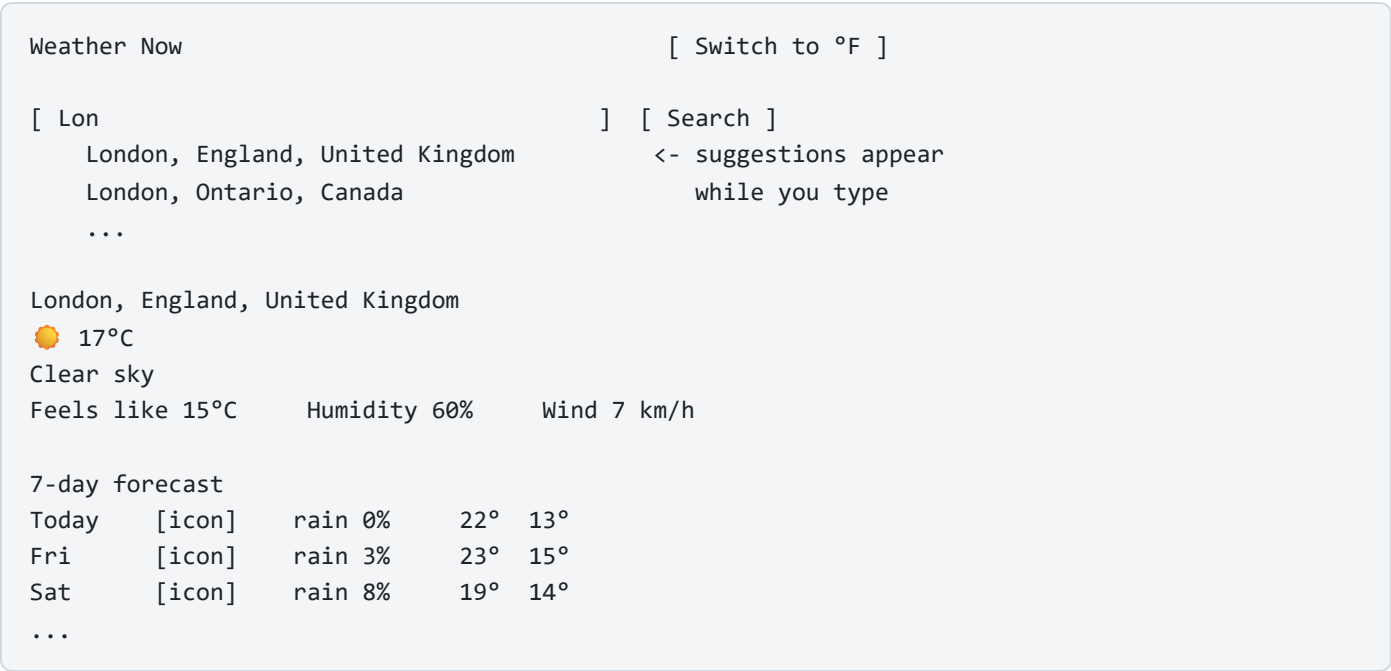
Chapter 39 is a project: a real weather app. Keep an eye on where each value comes from. The city name the user types is a string. The temperatures inside the API's JSON are already real numbers. Convert at the edges, as you did here, and none of this chapter's surprises will catch you.

On to [39 Project: Weather App](#). 🌤️

39 Project: Weather App

What you'll build

A real weather app that runs in the browser and uses **live data** from the internet. Type a city, pick it from the suggestions, and see the weather right now plus the next seven days:



By the end, your app will:

- find cities by name, with suggestions as you type
- show the current temperature, "feels like", humidity, wind, and a weather emoji
- show a 7-day forecast with day names
- show clear loading and error messages ("City not found", "No connection")
- switch between °C and °F
- remember the last city you looked at, even after a refresh

The weather data comes from **Open-Meteo**, a weather service that's free for non-commercial projects like this one. It needs no account and no API key, so you can start straight away. In return, they ask you to credit them, which is why the starter page has "Weather data by Open-Meteo.com" at the bottom.

This is the Level 3 project. It pulls together a lot of what you've learned:

Chapter	Where you'll use it
13 Array Methods	Turning API results into things on the page

Chapter	Where you'll use it
15 Destructuring	Pulling <code>current</code> and <code>daily</code> out of a response
18 Error Handling	<code>try / catch</code> around every request
19 Dates and Times	Day names for the forecast
20, 21, 22 DOM, Events, Forms	Showing data, reacting to typing, the search form
23 JSON and Local Storage	Remembering the last city
29 Modules	Splitting the app into <code>api.js</code> , <code>ui.js</code> , and <code>main.js</code>
32 Async/Await , 33 Fetch and APIs	Every request to the weather service
34 Debounce and Throttle	Suggestions as you type, without flooding the server

Your numbers will be different from the ones in this guide, of course. The weather changes!

Before you start

Set up your folder

1. Copy the `starter` (`JavaScript/39-project-weather-app/starter/`) folder and name the copy `weather-app`. Work in the copy, so the original stays clean if you ever want to start over.
2. Inside `weather-app`, create three empty files: `api.js`, `ui.js`, and `main.js`.
3. Right-click `index.html` in VS Code and choose **Open with Live Server**. The app uses ES modules, and browsers block modules loaded straight from a file, so double-clicking won't work ([chapter 29](#)).
4. Press `F12` to open the console. Keep it open the whole time.

You should see the title, the search box, and the footer. Everything else is hidden until your code shows it.

The starter gives you three files you don't need to change:

- `index.html` : the page. Its comments show the HTML your code should create for suggestions and forecast days.
- `style.css` : the looks. It works on phones too.
- `weather-codes.js` : the weather service describes the weather with a number, like `3`. This file turns each number into words and an emoji: `weatherCodes[3]` is `{ description: "Overcast", emoji: "☁️" }`.

How the files fit together

Each file has one job. Think of a restaurant:

File	Its job	In a restaurant
<code>api.js</code>	Talks to the weather service: builds URLs, calls <code>fetch</code> , returns data. Never touches the page.	The waiter, who takes your order to the kitchen and brings back the food
<code>ui.js</code>	Puts data on the page: text, lists, messages. Never calls <code>fetch</code> .	The staff who lay the food out nicely on your table
<code>main.js</code>	Listens to events, remembers what's going on, and decides what happens next.	The manager, who tells everyone what to do and when

```

main.js —imports—> api.js
      |
      |——imports—> ui.js —imports—> weather-codes.js

```

Why split it up? Each file stays small, and when something breaks, you know where to look. If the weather service ever changes, only `api.js` needs fixing.

Here's a plan for what each file exports. The names are only a suggestion: pick your own if you like, as long as you're consistent.

- `api.js`: `searchCities(name, signal)` and `getForecast(latitude, longitude, unit)`
- `ui.js`: functions like `showStatus(message, isError)`, `renderCurrent(place, forecast)`, `renderForecast(forecast)`, and `renderSuggestions(places, onSelect)`
- `main.js`: exports nothing. It's the file `index.html` loads, and it imports the other two.

Tip: build one small step at a time, and check the console after every change. A red error message is your friend: it tells you the file and the line.

The two APIs

The weather service needs a **latitude** and **longitude**: two numbers that pin down any spot on Earth (how far north or south, and how far east or west). People type city names, not numbers. So the app uses two APIs, one after the other:

1. **Geocoding** (turning a place name into coordinates): city name → a list of matching places, each with its latitude and longitude.
2. **Forecast**: latitude and longitude → the weather there.

Open both example URLs below in a new browser tab. You'll see the raw JSON the API sends back (if your browser offers a "Pretty-print" option, tick it to make the JSON easier to read).

1. Geocoding: from a city name to coordinates

```
https://geocoding-api.open-meteo.com/v1/search?name=London&count=5&language=en&format=json
```

Parameter	Example	What it means
name	London	What the user typed
count	5	How many places to send back at most
language	en	The language for place names
format	json	Send JSON back

The answer looks like this (trimmed):

```
{
  "results": [
    {
      "id": 2643743,
      "name": "London",
      "latitude": 51.50853,
      "longitude": -0.12574,
      "country_code": "GB",
      "country": "United Kingdom",
      "admin1": "England",
      "timezone": "Europe/London",
      "population": 8961989
    },
    {
      "id": 6058560,
      "name": "London",
      "latitude": 42.98339,
      "longitude": -81.23304,
      "country": "Canada",
      "admin1": "Ontario"
    }
  ],
  "generationtime_ms": 1.4225245
}
```

(`generationtime_ms` is how long the server took to answer. You can ignore it.)

The fields you need from each place:

Field	Example	Use it for
<code>name</code>	<code>"London"</code>	The city's name
<code>admin1</code>	<code>"England"</code>	The region or state. Not every place has one (Singapore doesn't).
<code>country</code>	<code>"United Kingdom"</code>	The country
<code>latitude</code> , <code>longitude</code>	<code>51.50853</code> , <code>-0.12574</code>	The forecast request

Three things that will bite you if you don't know them:

- **No match means no results at all.** Search for `xyzyy` and you get `{ "generationtime_ms": 0.15795231 } : no empty array, no error, just a missing key.` So use `data.results ?? []`.
- **Very short searches find nothing.** One letter never matches. Two letters only match a place with exactly that name. From three letters on, it finds partial matches too ("Lon" finds London).
- **Places share names.** There's a London in England, Canada, and several US states. That's why showing `name`, `admin1`, `country` in the suggestions matters.

2. Forecast: from coordinates to the weather

```
https://api.open-meteo.com/v1/forecast?
latitude=51.50853&longitude=-0.12574&current=temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto
```

It looks long, but it's only five parameters, plus an optional sixth for Fahrenheit:

Parameter	Example	What it means
<code>latitude</code> , <code>longitude</code>	<code>51.50853</code> , <code>-0.12574</code>	Where (from the geocoding result)
<code>current</code>	<code>temperature_2m,weather_code,...</code>	Which values you want for right now , separated by commas
<code>daily</code>	<code>weather_code,temperature_2m_max,...</code>	Which values you want for each day
<code>timezone</code>	<code>auto</code>	Use the place's own time zone for times and dates

Parameter	Example	What it means
<code>temperature_unit</code>	<code>fahrenheit</code>	Optional. Leave it out (or use <code>celsius</code>) for °C

You only get the values you ask for. The `_2m` in the names means "measured 2 meters above the ground", the standard height for weather readings. Wind is measured at 10 meters, hence `wind_speed_10m`.

The answer looks like this (trimmed):

```
{
  "latitude": 51.51147,
  "longitude": -0.13078308,
  "timezone": "Europe/London",
  "current_units": {
    "temperature_2m": "°C",
    "apparent_temperature": "°C",
    "relative_humidity_2m": "%",
    "weather_code": "wmo code",
    "wind_speed_10m": "km/h"
  },
  "current": {
    "time": "2026-09-24T12:15",
    "temperature_2m": 17.6,
    "apparent_temperature": 16.5,
    "relative_humidity_2m": 55,
    "weather_code": 3,
    "wind_speed_10m": 5
  },
  "daily": {
    "time": ["2026-09-24", "2026-09-25", "2026-09-26", "2026-09-27", "2026-09-28", "2026-09-29", "2026-09-30"],
    "weather_code": [3, 51, 51, 61, 61, 51, 53],
    "temperature_2m_max": [22.1, 23.2, 19.1, 22.3, 19.6, 24.5, 23.4],
    "temperature_2m_min": [12.8, 14.9, 13.2, 13.8, 14, 16, 20],
    "precipitation_probability_max": [0, 3, 8, 27, 37, 24, 37]
  }
}
```

The fields you need:

Field	Example	What it is
<code>current.temperature_2m</code>	<code>17.6</code>	The temperature now
<code>current.apparent_temperature</code>	<code>16.5</code>	"Feels like": takes wind, humidity, and sunshine into account

Field	Example	What it is
<code>current.relative_humidity_2m</code>	55	Humidity, in %
<code>current.weather_code</code>	3	The weather, as a code: look it up in <code>weatherCodes</code>
<code>current.wind_speed_10m</code>	5	Wind speed, in km/h
<code>current_units.temperature_2m</code>	"°C"	The unit. It becomes "°F" when you ask for Fahrenheit.
<code>daily.time</code>	<code>["2026-09-24", ...]</code>	The 7 dates, as <code>YYYY-MM-DD</code> text
<code>daily.weather_code</code>	<code>[3, 51, ...]</code>	Each day's weather code
<code>daily.temperature_2m_max</code> / <code>_min</code>	<code>[22.1, ...]</code> / <code>[12.8, ...]</code>	Each day's highest and lowest temperature
<code>daily.precipitation_probability_max</code>	<code>[0, 3, ...]</code>	Each day's chance of rain, in %

Two things to know:

- **daily is a set of matching lists.** Day `i` is `daily.time[i]` with `daily.temperature_2m_max[i]`, `daily.weather_code[i]`, and so on. Loop over the indexes, not over the values.
- **Names are in snake_case** (words joined with `_`). That's the API's style, not yours: read them exactly as they're spelled, like `forecast.current.weather_code`.

The latitude and longitude in the answer can be a little different from the ones you sent. The service uses the nearest point on its weather grid, and that's fine. If you send a bad parameter, you get status `400` and a body like `{ "error": true, "reason": "..."}.`

Milestone 1: Fetch the forecast for a fixed city

Goal: prove the connection works. `api.js` fetches the forecast, and `main.js` logs it for London. Nothing on the page changes yet.

1. In `api.js`, export an `async` function `getForecast(latitude, longitude)`. Build the URL from the table above, fetch it, check `response.ok`, and return the parsed JSON.
2. In `main.js`, import `getForecast`, call it with London's coordinates (`51.50853`, `-0.12574`), and log the result. Then log only the temperature.

Write import paths exactly like the file names: `"./api.js"`, with the `./` and the `.js`. Capitals matter too.

`"./Api.js"` happens to work on Windows, but it breaks on most web servers, which run Linux, where `Api.js`

and `api.js` are two different files.

What you'll see: in the console, an object with keys like `latitude`, `timezone`, `current_units`, `current`, and `daily`. Click the small arrow to expand it. Below it, a single number like `17.6`: the temperature in London right now.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Watch out: if the console shows a red error saying the script was **blocked by CORS policy**, you opened the page by double-clicking it. Close it and use **Open with Live Server** instead.

Milestone 2: Search for a city and show the weather now

Goal: type a city and press Enter (or click **Search**). The app takes the first matching place and fills in the "current weather" card.

1. In `api.js`, export `searchCities(name)`. It calls the geocoding API and returns the list of places, or an empty array when there's no match.
2. In `ui.js`, export `renderCurrent(place, forecast)`. It fills in `#place-name`, `#current-emoji`, `#current-temp`, `#current-description`, `#feels-like`, `#humidity`, and `#wind`, then shows the `#current` card. You'll need `weatherCodes` for the emoji and description: the first comment in `weather-codes.js` shows the `import` line.
3. In `main.js`, listen for `submit` on `#search-form`. Stop the page from reloading, read and trim the text, search, take the first place, fetch its forecast, and render it. Remove the test code from Milestone 1.

To show and hide things, use the `hidden` property: `card.hidden = false` shows the card, `card.hidden = true` hides it. (The starter CSS makes sure this always works.)

What you'll see: search `Paris` and the card appears, something like:

```
Paris, Île-de-France Region, France
[emoji] 18°C
Partly cloudy
Feels like 17°C      Humidity 64%      Wind 11 km/h
```

Rule: put everything that comes from the API on the page with `textContent`, never `innerHTML`. A city name is text from the internet, just like user input ([chapter 20](#)).

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 3: Loading and error states

Goal: the user always knows what's going on. Right now, a slow network shows nothing, and an unknown city or a lost connection breaks the app silently (look at the console: red errors).

1. In `ui.js`, export `showStatus(message, isError)` and `clearStatus()`. `showStatus` puts the message in `#status`, adds the `error` class only when `isError` is `true`, and shows the paragraph. `clearStatus` hides it.
2. In `main.js`, show a message before each request, like `Searching for "Paris"...` and then `Loading the weather...`. Clear it when the weather is on screen.
3. If the search finds nothing, show an error and stop.
4. Wrap your requests in `try / catch` ([chapter 18](#)). If anything fails, show a friendly error. Also `console.error` the real error, so *you* can still see what happened.

What you'll see:

- `Paris`: the messages flash by, then the weather appears.
- `Xyzyy`: a red message, `No city called "Xyzyy" found. Check the spelling and try again.`
- `No connection`: a red message, `Couldn't reach the weather service. Check your connection and try again.`

To test without a connection, you don't need to unplug anything. Open DevTools, go to the **Network** tab, and change the dropdown that says **No throttling** to **Offline**. Search again. Switch it back afterwards! The same dropdown has slow presets too: pick one to see your loading messages properly.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Milestone 4: Suggestions as you type

Goal: while the user types, show up to five matching places under the box. Clicking one shows its weather. This is where [chapter 34](#) pays off: one request per pause, and a slow old answer can never replace a newer one.

1. Type your complete `debounce(fn, delay)` from chapter 34 at the top of `main.js`. (Or give it its own `debounce.js` module with `export`, and import it. Your choice.)
2. Give `searchCities` a second parameter, `signal`, and pass it on to `fetch` as `{ signal }`. When you call `searchCities(name)` without one, `signal` is `undefined` and `fetch` works as usual.
3. In `ui.js`, export `renderSuggestions(places, onSelect)`. For each place, create the `<li>` with a `<button>` shown in the comment in `index.html`, and make each button call `onSelect(place)` when clicked. Then show the list. Also export `clearSuggestions()`, which empties and hides it.
4. In `main.js`, write `suggest(text)`: abort the previous suggestion request, skip texts shorter than 2 letters (the API can't match them), then search with a fresh `AbortController`'s signal and render the places. Call it through a debounced version (300 ms) from an `input` listener.
5. When a suggestion is chosen: clear the list, put the city's name in the box, and show its weather.

- When the form is submitted, cancel the waiting suggestion and abort any request in flight. Otherwise the list pops up again *after* the weather appears (Common mistake 5 in chapter 34).

What you'll see: type `Lon` and pause. Five suggestions appear: `London, England, United Kingdom, Loni, Uttar Pradesh, India`, and so on. Click the first one: the list closes, the box says `London`, and London's weather appears.

Now open the **Network** tab and type `London` quickly. Only one search request goes out. Type slowly, and each pause sends one. If a request is still on its way when the next one starts, you'll see the old one marked **(canceled)**.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Milestone 5: The 7-day forecast

Goal: under the current weather, list the next seven days: the day's name, an emoji, the chance of rain, and the highest and lowest temperature.

- In `ui.js`, export `renderForecast(forecast)`. Empty `#forecast-list`, then loop over the indexes of `daily.time`. For each day, build the `<li class="forecast-day">` shown in the comment in `index.html` and append it. Finally, show the `#forecast` section.
- Write a small helper, `formatDay(dateText, index)`. The first day (index `0`) is `"Today"`. For the others, turn the date text into a short day name like `"Fri"` ([chapter 19](#)).
- Call `renderForecast` right after `renderCurrent`.

What you'll see:

```
7-day forecast
Today    [icon]  rain 0%    22° 13°
Fri      [icon]  rain 3%    23° 15°
Sat      [icon]  rain 8%    19° 14°
Sun      [icon]  rain 27%   22° 14°
...
```

Watch out for a time zone trap from chapter 19. `new Date("2026-09-24")` means midnight in **UTC**. If you then format it in the user's own time zone, anyone behind UTC (across the Americas, for example) gets the day before:

```
const day = new Date("2026-09-24"); // a Thursday

console.log(day.toLocaleDateString("en-US", { weekday: "short", timeZone: "America/New_York"
  })); // prints: Wed
console.log(day.toLocaleDateString("en-US", { weekday: "short", timeZone: "UTC" })); // prints:
Thu
```

The fix is to format in UTC too, with `timeZone: "UTC"`, as in the second line. This bug is sneaky: if your own time zone is ahead of UTC, your app looks right on your computer, and only your users far to the west see the wrong days.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Milestone 6: Switch between °C and °F

Goal: the button in the header switches every temperature on the page between Celsius and Fahrenheit.

1. Give `getForecast` a third parameter, `unit`, with the default value `"celsius"`, and send it as `temperature_unit`. The API accepts `"celsius"` and `"fahrenheit"`.
2. In `main.js`, keep two pieces of **state** (what the app needs to remember while it runs): `let unit = "celsius";` and `let currentPlace = null;`. Set `currentPlace` every time you show a place.
3. When `#unit-toggle` is clicked, flip `unit`, change the button's text (`Switch to °C` or `Switch to °F`), and, if there's a current place, load its weather again.

What you'll see: click **Switch to °F** and every temperature changes (17°C becomes about 63°F). The button now says **Switch to °C**. Click it again to go back.

If you took the units from `current_units` in Milestone 2, the labels change by themselves: the API sends `"°F"` now. If you typed `"°C"` into your code, you'll see Fahrenheit numbers with a Celsius label. Time to fix that!

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Milestone 7: Remember the last city

Goal: refresh the page, and the last city's weather appears by itself, with no searching.

1. Every time a place's weather is shown successfully, save the place in `localStorage` under the key `"lastPlace"`. It's an object, so turn it into JSON first ([chapter 23](#)).
2. At the bottom of `main.js`, when the app starts, read it back. If there's a saved place, show its weather. If not, show a friendly hint with `showStatus`, like `Search for a city to see its weather.`
3. **Bonus:** remember the °C/°F choice too, and set the button's text to match when the app starts.

What you'll see: search `Tokyo`, then refresh the page. Tokyo's weather loads straight away. Open DevTools, go to **Application** → **Local storage**, and you'll find your saved place as a JSON string.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Common mistakes

1. Assuming there's always a `results` list

```
const data = await response.json(); // searched for "Xyzyz"
const place = data.results[0];
// TypeError: Cannot read properties of undefined (reading '0')
```

When nothing matches, the geocoding API leaves `results` out completely. Fix: `const places = data.results ?? []`; , then check `places.length === 0` and show "City not found".

2. Not checking `response.ok`

```
const response = await fetch(url); // a typo in a parameter: status 400
const forecast = await response.json();
console.log(forecast.current.temperature_2m);
// TypeError: Cannot read properties of undefined (reading 'temperature_2m')
```

A `400` answer is still valid JSON (`{ "error": true, "reason": "..."}`), so `response.json()` works and the real problem hides until later. Fix: check `response.ok` right after `fetch` , and `throw` an error that includes `response.status` . Then `console.log` the `reason` while you're debugging.

3. Forgetting `await`

```
const places = searchCities("Paris"); // no await: this is a Promise
const place = places[0]; // undefined
getForecast(place.latitude, place.longitude);
// TypeError: Cannot read properties of undefined (reading 'latitude')
```

An `async` function always returns a promise ([chapter 32](#)). Fix: `const places = await searchCities("Paris");` .

4. Forgetting `preventDefault()` on the form

```
form.addEventListener("submit", () => {
  // search for the city and show its weather...
});
// ...but the page reloads straight away and wipes everything
```

A form's default job is to send the page somewhere and reload it ([chapter 22](#)). The weather flashes for a moment and disappears. Fix: take the event as a parameter and call `event.preventDefault()` first.

5. Suggestions popping up after the user already chose

The user types `Tok`, presses Enter, and Tokyo's weather appears. Then the suggestion list opens anyway, because a debounced call was still waiting. Fix: whenever a search is submitted or a suggestion is chosen, call your debounced function's `cancel()`, abort the suggestion request in flight, and clear the list.

Quick recap

In this project, you:

- used **two APIs in a row**: geocoding (name → coordinates) and forecast (coordinates → weather), reading only the fields you need
- split the app into **modules** with one job each: `api.js` fetches, `ui.js` draws, `main.js` decides
- handled **loading, "not found", and network errors**, so the app never just silently breaks
- reused your own `debounce` from chapter 34 and canceled stale requests with `AbortController`
- formatted **day names** safely with `timeZone: "UTC"`, and let the API's `current_units` label your temperatures
- kept **state** (`unit`, `currentPlace`) and saved it with `localStorage`, so the app remembers you

That's a real, useful app built with nothing but plain JavaScript. Well done!

Next: try the [exercises](#), then move on to [40 The Event Loop](#).

Exercises

How to do these:

- These are stretch goals for the app you built in the [notes](#). Keep working in your `weather-app` folder, and open it with **Live Server** as before.
 - Each exercise adds one feature, so you can do them in any order. Finish one before starting the next.
 - Some of them ask you to add HTML or CSS. That's fine: it's your copy of the starter.
 - Live weather changes all the time, so most checks describe what you should see rather than exact numbers.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): A background that matches the weather

Make the app feel alive: a bright sky for sunny days, gray for clouds, a darker blue for rain, pale for snow, and a dark sky for storms.

1. In `ui.js`, write `weatherGroup(code)`. It returns one of five words, using this table:

Group	Weather codes
"clear"	0, 1
"cloudy"	2, 3, 45, 48
"rain"	51 to 67, 80 to 82
"snow"	71 to 77, 85, 86
"storm"	95 to 99

2. Whenever you show the current weather, put a class on `<body>` that matches: `weather-clear`, `weather-rain`, and so on. Remove the old one first, so there's only ever one.

3. In `style.css`, add one rule per group. Here's the first one, pick your own colors for the rest:

```
body.weather-rain {  
  background: linear-gradient(180deg, #4a6d8c, #a9c1d9) fixed;  
}
```

Test your function in the console with `console.log(weatherGroup(0), weatherGroup(3), weatherGroup(61), weatherGroup(73), weatherGroup(95));`. Expected output:

```
clear cloudy rain snow storm
```

Then search for a few cities around the world until you find a rainy one, and watch the background change.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): An offline message

On a train, the connection comes and goes. Instead of letting the next search fail, tell the user they're offline.

1. Add this paragraph to `index.html`, just above the `#status` paragraph:

```
<p id="offline" class="status error" hidden>You're offline. The weather shown may be out  
  of date.</p>
```

2. `navigator.onLine` is `true` or `false`, and `window` fires an `offline` event when the connection drops and an `online` event when it comes back. Show the message when offline, and hide it again when online.

3. Also check once when the app starts, in case it loads while offline.
4. **Bonus:** when the connection comes back, load the current place's weather again, so it's fresh.

To test it, switch your Wi-Fi off and on, or use the **Offline** option in the Network tab of DevTools (Milestone 3).

Be honest with your users: `navigator.onLine === false` means the device is definitely offline. But `true` only means it's connected to a network, not that the internet works (think of a café Wi-Fi that wants you to log in first). So keep your error handling from Milestone 3 too.

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Use my location

You're traveling and don't know the name of the town you're in. Add a button that shows the weather right where you are.

1. Add this button to `index.html`, just below the search form. The `unit-toggle` class gives it the same look as the °F button:

```
<button id="locate" class="unit-toggle" type="button">Use my location</button>
```

If it feels cramped, give `#locate` a `margin-bottom: 16px;` in `style.css`.

2. The browser's **Geolocation API** finds the device's position:
`navigator.geolocation.getCurrentPosition(onSuccess, onError)`. It uses callbacks ([chapter 30](#)), so wrap it in a promise ([chapter 31](#)) and write a `getPosition()` you can `await`.
3. On success, the position has `coords.latitude` and `coords.longitude`. Show the weather for a place object like `{ name: "Your location", latitude, longitude }`. Your place label from Milestone 2 skips the missing parts, so the card says "Your location".
4. If it fails (the user said no, or the device can't tell), show an error: `Couldn't get your location. You can still search by name.`

Good to know:

- The browser asks the user for permission first. That's how it should be: a website shouldn't know where you are without asking.
- It only works on secure pages: `https://` addresses, or your own computer. Live Server's `http://127.0.0.1` address counts as your own computer, so you're fine.

- On a laptop without GPS, the position is a guess from nearby Wi-Fi networks. It can be a few kilometers off.
- The geocoding API you're using only searches by name. It can't turn coordinates back into a city name, which is why the card says "Your location".

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Favorite cities

You check the same three cities every day: home, work, and where your family lives. Let the user save favorites and open them with one click.

1. Add a save button at the end of the `#current` card, and an empty list just below the search form:

```
<button id="save-favorite" type="button">Save to favorites</button>
```

```
<ul id="favorites" class="favorites"></ul>
```

There are no styles for these yet. Add your own to `style.css` if you like.

2. Keep the favorites as an array of place objects, saved in `localStorage` under the key `"favorites"`.
3. Clicking **Save to favorites** adds the current place, but never twice. Places from the geocoding API have an `id`, so compare those.
4. Show each favorite as a button with the place's name. Clicking it shows that city's weather. Give each one a small `×` button that removes it.
5. The favorites survive a refresh.

Check: save London, then save London again. There's still only one London in the list. Refresh the page: your favorites are still there.

(If you did Exercise 3, "Your location" has no `id`, so don't let the user save it.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): An hourly temperature chart

"Will it get warmer this afternoon?" Answer it with a bar chart of the next 24 hours, made with nothing but `<div>`s and CSS.

1. Add a new card to `index.html`, between the current weather and the 7-day forecast:

```
<section id="hourly" class="card" hidden>
  <h2>Next 24 hours</h2>
  <div id="hourly-chart" class="hourly-chart"></div>
</section>
```

2. Ask the forecast API for hourly data too, by adding two parameters: `hourly=temperature_2m` and `forecast_hours=24`. The answer then has an `hourly` object with two matching lists: `hourly.time` (24 texts like `"2026-09-24T12:00"`, starting from the current hour at that place) and `hourly.temperature_2m` (24 numbers).
3. Write `barHeights(temps)`. It turns the temperatures into bar heights, in %: the coldest hour gets `10`, the warmest gets `100`, and the others go in between, in proportion. Round each one with `Math.round`. If every temperature is the same, give every bar `50`.
4. Draw one `<div class="bar">` per hour, with its height set from `barHeights`, and a `title` like `12:00:18°` so hovering a bar shows the details.
5. Add this to `style.css`:

```
.hourly-chart {
  display: flex;
  align-items: flex-end; /* bars grow up from the bottom */
  gap: 2px;
  height: 120px;
}

.bar {
  flex: 1;
  background: var(--accent);
  border-radius: 4px 4px 0 0;
}
```

Test `barHeights` with these real temperatures from London (it works the same in the browser console or in Node):

```
const temps = [17.9, 19.4, 20.5, 21.4, 22.3, 22.5, 21.8, 20.7, 19.2, 18.3, 17.5, 16.5,
  16.2, 16.2, 16.1, 15.9, 15.7, 15.4, 15.1, 14.9, 15.5, 16.5, 17.6, 18.5];
console.log(barHeights(temps).join(" "));
```

Expected output:

```
46 63 76 87 98 100 92 79 61 50 41 29 25 25 24 22 19 16 12 10 17 29 42 53
```

Bonus: under the chart, label every third hour (12:00 , 15:00 , 18:00 , ...).

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Think about everything your app does "at the same time": it waits for `fetch`, runs debounce timers, and still reacts to every click and keystroke. Yet JavaScript runs only one piece of code at a time. So how does it juggle all of that without freezing?

[Chapter 40](#) opens the hood and shows you the event loop.

Level 4: Mastery

Goal: know how JavaScript works under the hood and write code like a professional.

40 The Event Loop

What is it?

The **event loop** is how JavaScript decides what to run next.

JavaScript can only do one thing at a time. Yet your programs juggle timers, promises, clicks, and network requests. The event loop keeps order: it runs your code, then takes the next waiting job, one at a time, following a few fixed rules.

Why does it matter?

You've been relying on the event loop since [chapter 30](#), every time you called `setTimeout`. Knowing how it works answers questions like these:

- Why does `setTimeout(fn, 0)` run *after* the code below it, even with a 0 ms delay? (Chapter 30 promised to explain.)
- Why does a promise's `.then` run before a `setTimeout` that was set up first?
- Why does one long loop freeze a whole web page, so buttons stop responding?
- Why was my timer late?

Once you know the rules, async code stops feeling like magic. You can predict the exact order of the output, and write programs that stay quick and responsive. Output-order puzzles are also a favorite in job interviews.

Real-world example

Picture a small, busy cafe with just **one cashier**.

At the cafe	In JavaScript
One cashier, serving one customer at a time	JavaScript runs one piece of code at a time
The customer at the counter right now	The call stack: the code running right now
The kitchen cooks on its own. The cashier doesn't stand there waiting	The browser or Node does the waiting (timers, network requests)
The regular line: customers whose food is ready, and new customers walking in	The task queue: finished timers, clicks, network replies
The VIP line: quick follow-ups that always go first	The microtask queue: promise callbacks
The cashier's rule: finish the current customer, serve <i>everyone</i> in the VIP line, then call <i>one</i> person from the regular line. Repeat all day.	The event loop

The rule is strict. Nobody interrupts the customer at the counter, however long they take. That's why one slow customer holds up the whole cafe, and one slow piece of code freezes a whole page.

Keep this cafe in mind. Every rule in this chapter is one of the cashier's rules.

How it works

One thing at a time

JavaScript is **single-threaded**. A **thread** is one line of work: a worker who does one step after another. JavaScript runs your code on just one of them, called the **main thread**. That's the one cashier.

So while a piece of your code is running, nothing else can run. Not a timer, not a click handler, not a promise callback. They all wait their turn.

The call stack (a quick recap)

Remember the call stack from [chapter 17](#), the stack of plates? It's JavaScript's list of "what am I in the middle of right now?" Every function call puts a plate on top, and every `return` takes one off.

```
function makeSandwich() {
  console.log("Making a sandwich");
  addFilling();
  console.log("Sandwich done");
}

function addFilling() {
  console.log("Adding cheese");
}

makeSandwich();
console.log("Next order, please");
```

Here's the stack while this file runs. `main` means the main part of your file, outside any function.

```
The file starts      [ main ]
makeSandwich() called [ main, makeSandwich ]
addFilling() called  [ main, makeSandwich, addFilling ]
addFilling returns   [ main, makeSandwich ]
makeSandwich returns [ main ]
The file ends        [ ]    <- empty! Now the event loop takes over
```

That last line is the key to this whole chapter. **Timers, promise callbacks, and clicks only get to run when the stack is empty.** JavaScript never pauses your code halfway through to run something else.

Blocking: when the cashier gets stuck

Because there's only one thread, a slow piece of code holds up everything behind it. This is called **blocking** the main thread.

Here's a reminder set for 100 ms, followed by a job that keeps JavaScript busy for about a second:

```
const start = Date.now();

setTimeout(() => {
  console.log(`Reminder! (asked for 100 ms, got ${Date.now() - start} ms)`);
}, 100);

// Keep the one cashier busy for about 1 second
while (Date.now() - start < 1000) {
  // doing nothing useful, just blocking
}

console.log("Big job finished");
```

You'll see something like this (your number will be a little different):

```
Big job finished
Reminder! (asked for 100 ms, got 1002 ms)
```

The timer was ready after 100 ms, but it couldn't run until the loop was done and the stack was empty. That's why chapter 30 said the delay is a *minimum*, not an exact time.

In the browser, blocking hurts even more. While your code runs, the page can't react to clicks or typing, and it can't redraw the screen, so it looks frozen. If it goes on long enough, the browser may offer to stop the page for you. You'll see how to avoid this later in the chapter.

Who does the waiting?

If JavaScript can only do one thing at a time, how does `setTimeout` wait without freezing everything?

JavaScript doesn't do the waiting itself. The **environment** does: the program that runs your JavaScript, which is either the browser or Node. It's the kitchen in the cafe.

When you call `setTimeout(fn, 1000)`, JavaScript hands the timer to the environment and moves straight on to the next line. The environment counts the time. When the time is up, it doesn't run `fn` right away, because the cashier might be busy. It puts `fn` in a line to wait for its turn.

The same goes for network requests (`fetch` , [chapter 33](#)), clicks and key presses in the browser, and reading files in Node. The environment waits for them. When something happens, it lines up your callback.

Two waiting lines: tasks and microtasks

There are two lines, and they are not equal.

A **queue** is a waiting line where the first to join is the first served, like the line at a bank.

- A **task** (also called a **macrotask**) is a callback waiting in the **task queue**, the regular line. Finished timers, clicks, and network replies all become tasks.
- A **microtask** is a small, urgent job waiting in the **microtask queue**, the VIP line. Promise callbacks go here.

You write...	The callback waits in...
<code>setTimeout(fn, ms)</code> or <code>setInterval(fn, ms)</code>	the task queue, once the time is up
an event listener, like <code>click</code> (browser)	the task queue, once the event happens

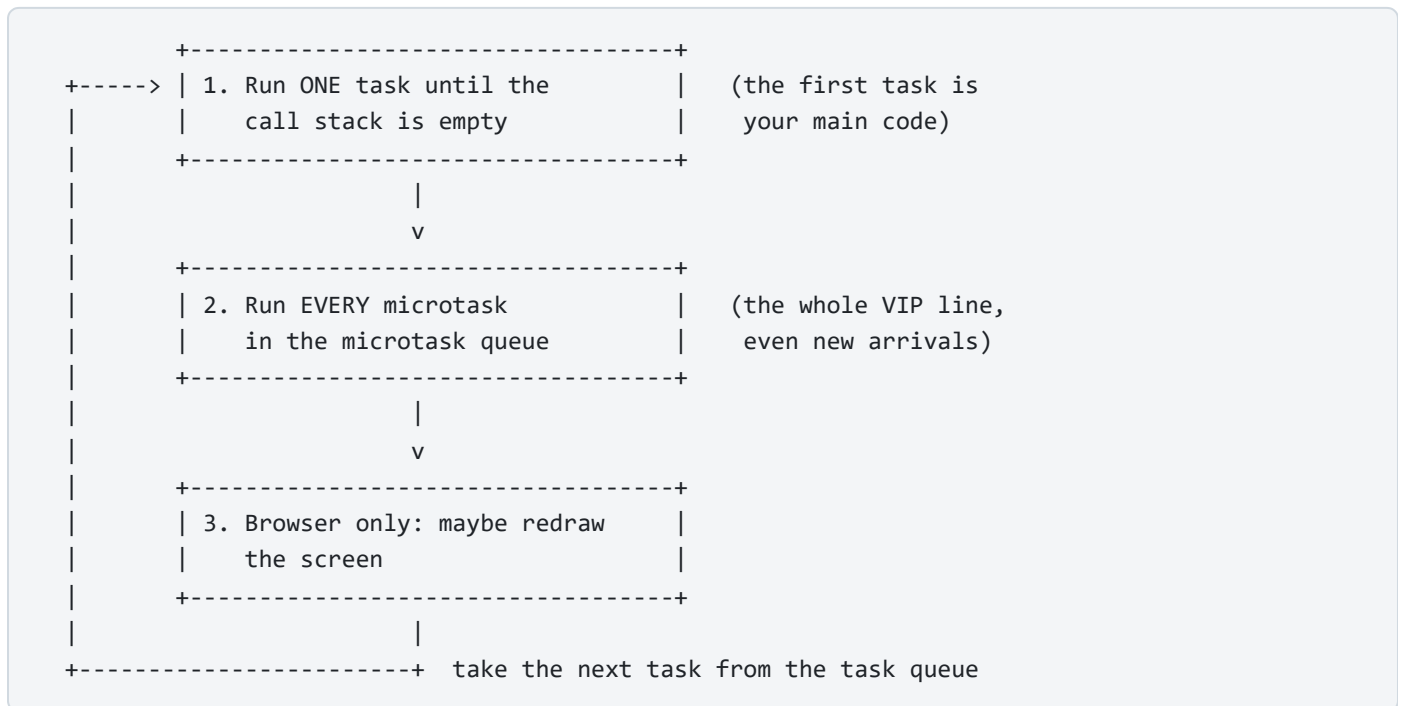
You write...	The callback waits in...
<code>promise.then(fn), .catch(fn), .finally(fn)</code>	the microtask queue, once the promise settles (is fulfilled or rejected)
the rest of an <code>async</code> function after <code>await</code>	the microtask queue, once the awaited promise settles
<code>queueMicrotask(fn)</code>	the microtask queue, straight away

`queueMicrotask(fn)` is new. It's a built-in function, in both browsers and Node, that puts `fn` straight into the VIP line, no promise needed.

The event loop: the cashier's rules

Here are the rules. The event loop repeats them for as long as the program runs:

1. **Run one task** until the call stack is empty. The very first task is your main code: the whole file, top to bottom.
2. **Run every microtask**, one after another. If a microtask adds more microtasks, run those too. Stop only when the VIP line is empty.
3. In the browser, the screen may be redrawn now.
4. **Go back to step 1** with the next task from the task queue. If there isn't one, wait until there is.



Notice the difference: microtasks are served **all at once**, tasks **one at a time**, with the VIP line emptied again after each one.

In Node, the program ends when there's nothing left to run and nothing left to wait for (no timers, no open network requests).

Your first puzzle, step by step

Try to guess the output before you read on. `Promise.resolve()` ([chapter 31](#)) makes a promise that's already fulfilled.

```
console.log("start");

setTimeout(() => {
  console.log("timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("promise");
});

console.log("end");
```

You'll see:

```
start
end
promise
timeout
```

Here's why, following the rules.

Step 1: run the first task, your main code. `start` prints. `setTimeout` hands its timer to the environment, and when the 0 ms are up, the callback joins the task queue. The promise is already fulfilled, so the `.then` callback joins the microtask queue straight away. Then `end` prints, and the file is done:

```
Call stack:      (empty)
Microtask queue: [ log "promise" ]
Task queue:      [ log "timeout" ]
Printed so far:  start, end
```

Step 2: run every microtask. `promise` prints:

```
Call stack:      (empty)
Microtask queue: (empty)
Task queue:      [ log "timeout" ]
Printed so far:  start, end, promise
```

Back to step 1: run the next task. `timeout` prints. Both lines are now empty and nothing else is waiting, so the program ends.

So `setTimeout(fn, 0)` doesn't mean "run now". It means "run as soon as possible, after the current code *and* all the microtasks are done".

Puzzle time

Now it's your turn. For each puzzle, cover the output with your hand, write down your guess, then check. Better still, type each one into a file and run it.

Puzzle 1: two ways into the VIP line

```
console.log("A");
setTimeout(() => console.log("B"), 0);
queueMicrotask(() => console.log("C"));
Promise.resolve().then(() => console.log("D"));
console.log("E");
```

You'll see:

```
A
E
C
D
B
```

`A` and `E` are plain code, so they print first. `C` and `D` are both microtasks. They wait in the VIP line in the order they joined, so `C` comes before `D`. `B` is a task, so it goes last.

Puzzle 2: the promise recipe runs right away

```

console.log("A");

const order = new Promise((resolve) => {
  console.log("B");
  resolve("pizza");
});

order.then((food) => console.log("C", food));
console.log("D");

```

You'll see:

```

A
B
D
C pizza

```

Surprised by B? The function you pass to `new Promise` runs **straight away**, as plain code. Only the `.then` callback waits in the VIP line.

Puzzle 3: `async` and `await`

```

async function loadProfile() {
  console.log("loading...");
  await Promise.resolve();
  console.log("profile ready");
}

console.log("app starts");
loadProfile();
console.log("show menu");

```

You'll see:

```

app starts
loading...
show menu
profile ready

```

An `async` function runs like a normal function **until its first `await`**. That's why `loading...` prints straight away. At the `await`, the function pauses and steps aside, and the rest of it (`profile ready`) becomes a microtask. Meanwhile, the main code carries on with `show menu`. This is what [chapter 32](#) meant by "`await` pauses only that function, not the whole program".

Puzzle 4: a promise inside a timer

```
setTimeout(() => {  
  console.log("timer 1");  
  Promise.resolve().then(() => console.log("promise inside timer 1"));  
}, 0);  
  
setTimeout(() => console.log("timer 2"), 0);
```

You'll see:

```
timer 1  
promise inside timer 1  
timer 2
```

Both timers are ready at the same time, but the loop takes only **one** task at a time. After `timer 1` finishes, the VIP line is emptied before the next task gets a turn. So the promise jumps ahead of `timer 2`.

Puzzle 5: two promise chains

```
Promise.resolve()  
  .then(() => console.log("Order 1: paid"))  
  .then(() => console.log("Order 1: shipped"));  
  
Promise.resolve()  
  .then(() => console.log("Order 2: paid"))  
  .then(() => console.log("Order 2: shipped"));
```

You'll see:

```
Order 1: paid  
Order 2: paid  
Order 1: shipped  
Order 2: shipped
```

The two chains take turns. Each `.then` waits for the step before it. So `Order 1: shipped` can only join the VIP line once `Order 1: paid` has run, and by then `Order 2: paid` is already waiting ahead of it.

Puzzle 6: a microtask that adds a microtask

```
setTimeout(() => console.log("timeout"), 0);

queueMicrotask(() => {
  console.log("micro 1");
  queueMicrotask(() => console.log("micro 2 (added by micro 1)"));
});
```

You'll see:

```
micro 1
micro 2 (added by micro 1)
timeout
```

Rule 2 says **every** microtask runs, even ones that join while the VIP line is being served. The timer has to wait until the line is completely empty.

That has a dark side. If each microtask kept adding another one forever, the VIP line would never empty. Timers, clicks, and screen updates would never get a turn, and a page would freeze for good.

The final puzzle: everything together

```
async function takeOrder() {
  console.log("Write down the order");
  await Promise.resolve();
  console.log("Send it to the kitchen");
}

console.log("Open the shop");

setTimeout(() => console.log("Deliver the pizza"), 0);

Promise.resolve().then(() => console.log("Check the oven"));

takeOrder();

queueMicrotask(() => console.log("Wipe the counter"));

console.log("Greet the next customer");
```

Take your time with this one. You'll see:

```
Open the shop
Write down the order
Greet the next customer
Check the oven
Send it to the kitchen
Wipe the counter
Deliver the pizza
```

Stage	What happens	Prints
1. The main code	The timer is handed to the environment. "Check the oven" joins the VIP line. <code>takeOrder()</code> runs until its <code>await</code> , and the rest of it joins the VIP line. "Wipe the counter" joins the VIP line.	Open the shop , Write down the order , Greet the next customer
2. Every microtask, in the order they joined	"Check the oven", then the rest of <code>takeOrder</code> , then "Wipe the counter"	Check the oven , Send it to the kitchen , Wipe the counter
3. The next task	The timer's callback	Deliver the pizza

Got it right? Great work: this one trips up plenty of experienced developers. Got it wrong? Walk through the table once more. It clicks with practice.

Redrawing the screen and `requestAnimationFrame` (browser)

In the browser, the screen can only be redrawn *between* tasks, never while your code is running, and never while microtasks are still waiting. That's why a long loop, or an endless chain of microtasks, freezes the page.

The browser doesn't redraw after every single task, though. It redraws when it's time for a new **frame**: one picture on the screen, like one page of a flip book. That's usually about 60 frames a second (more on faster screens), and it can skip frames when nothing has changed.

When you want to change something on screen smoothly, like a growing progress bar or a moving game character, use `requestAnimationFrame(fn)` . It asks the browser: "run `fn` right before you draw the next frame."

```
const bar = document.querySelector("#progress-bar");
let width = 0;

function grow() {
  width += 1;
  bar.style.width = `${width}%`;
  if (width < 100) {
    requestAnimationFrame(grow); // ask again for the next frame
  }
}

requestAnimationFrame(grow);
```

You met it briefly in [chapter 34](#) as a throttle for visual updates. It only exists in browsers (Node has no screen to draw). Most browsers pause these callbacks while the tab is hidden, which saves battery.

Watch out: The exact order between `requestAnimationFrame` callbacks and timers can differ between browsers and from frame to frame. Don't write code that depends on it.

Node extras: `process.nextTick` and `setImmediate`

Node has two extra scheduling functions that you'll meet in Node code. `process.nextTick(fn)` runs `fn` as soon as the current code finishes, even before promise callbacks: a line in front of the VIP line. `setImmediate(fn)` runs `fn` as a task, on the next turn of Node's loop.

```
console.log("sync");

setTimeout(() => console.log("setTimeout"), 0);
Promise.resolve().then(() => console.log("promise"));
process.nextTick(() => console.log("nextTick"));

console.log("sync again");
```

You'll see:

```
sync
sync again
nextTick
promise
setTimeout
```

Now try `setImmediate` against `setTimeout(..., 0)` in your main file:

```
setTimeout(() => console.log("setTimeout"), 0);
setImmediate(() => console.log("setImmediate"));
```

Run it a few times. Sometimes `setTimeout` prints first, sometimes `setImmediate` does. It depends on tiny timing details as Node starts up, so the order isn't guaranteed.

Function	Works in	Runs...
<code>process.nextTick(fn)</code>	Node only	right after the current code, before promise callbacks
<code>queueMicrotask(fn)</code> , <code>promise.then(fn)</code>	browsers and Node	right after the current code (the VIP line)
<code>setTimeout(fn, 0)</code>	browsers and Node	as a task, once the delay is up
<code>setImmediate(fn)</code>	Node only	as a task, on the next turn of Node's loop

Two honest notes:

- The `nextTick` -before-promises order above is for a normal `.js` file. In an ES module ([chapter 29](#)), promise callbacks can run first instead. Don't write code that depends on the order between the two.
- Node's task queue is really a few separate lines (for timers, for finished file and network work, and for `setImmediate`) that it serves in turns. The big rule stays the same: all microtasks first, then the next task.

In everyday code, `setTimeout` and promises (or `queueMicrotask`) are all you need, and they work everywhere. It's enough to recognize the Node extras when you see them.

Splitting heavy work into chunks

Sometimes you really do have a big job, like resizing thousands of photos. You can't make the job smaller, but you can stop it from hogging the cashier: split it into **chunks** (small pieces) and let other tasks have a turn in between.

If you resized all 3000 photos in one big loop, a click on the Cancel button would have to wait until the very end, just like the late reminder earlier. Too late to cancel anything!

So here's the job in chunks of 1000. The `setTimeout` at the bottom stands in for the user clicking Cancel. After each chunk, `setTimeout(resizeChunk, 0)` puts the next chunk at the *back* of the task queue, so anything already waiting goes first:


```
function resizePhotos(total, chunkSize) {
  let done = 0;

  function resizeChunk() {
    const end = Math.min(done + chunkSize, total);
    while (done < end) {
      done++; // pretend we resized one photo
    }
    console.log(`Resized ${done} of ${total} photos`);

    if (done < total) {
      setTimeout(resizeChunk, 0); // let others have a turn first
    }
  }

  resizeChunk();
}

setTimeout(() => console.log("Click! The user pressed Cancel"), 0);
resizePhotos(3000, 1000);
```

You'll see:

```
Resized 1000 of 3000 photos
Click! The user pressed Cancel
Resized 2000 of 3000 photos
Resized 3000 of 3000 photos
```

Now the click gets its turn after the first chunk. In a real page, the browser can also redraw the screen between chunks, so a progress bar keeps moving and buttons keep working. The whole job takes a little longer, but the page stays alive.

Two things to notice:

- `resizeChunk` is a closure ([chapter 25](#)). It remembers `done`, `total`, and `chunkSize` from one chunk to the next.
- The break has to be a **task** (`setTimeout`), not a microtask. Microtasks all run before anything else gets a turn (remember Puzzle 6), so chunks queued with `queueMicrotask` would block just like one big loop.

Web Workers: real extra threads (browser)

Chunking shares the one cashier more fairly. A **Web Worker** hires a second cashier: a real extra thread that runs a separate JavaScript file at the same time as your page.

`main.js`, the page's script:

```
const worker = new Worker("worker.js");

worker.addEventListener("message", (event) => {
  console.log("Total:", event.data);
});

worker.postMessage(10000000); // send the job to the worker
console.log("The page is still free to respond!");
```

`worker.js`, which runs on its own thread:

```
self.addEventListener("message", (event) => {
  let total = 0;
  for (let i = 0; i < event.data; i++) {
    total += i;
  }
  self.postMessage(total); // send the answer back
});
```

In the browser console, you'll see:

```
The page is still free to respond!
Total: 49999995000000
```

`new Worker("worker.js")` starts the worker file on its own thread, with its own event loop, so its long loop doesn't block the page. The two don't share variables. They talk by sending **messages**: `postMessage(data)` sends one, and the `message` event receives it in `event.data`. Inside a worker, `self` means "this worker".

Workers can't touch the page (there's no `document` inside them), so they're for heavy calculations, like processing images or crunching big lists. Chrome and Edge won't start a worker from a page you opened by double-clicking the file, so serve it with Live Server, just like modules in chapter 29. Node has its own version of workers in the `node:worker_threads` module. You'll see workers again in [chapter 48](#).

Common mistakes

1. Expecting `setTimeout(fn, 0)` to run right away

```
let seatsLeft = 10;

setTimeout(() => {
  seatsLeft = seatsLeft - 2; // book 2 seats
}, 0);

console.log(`Seats left: ${seatsLeft}`); // prints: Seats left: 10
```

The booking hasn't happened yet when the last line runs. A 0 ms timer still waits for the current code and every microtask. Fix: use the new value *inside* the callback, or turn the waiting into a promise and `await` it.

2. Thinking `await` pauses the whole program

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function saveGame() {
  await wait(500); // pretend saving takes half a second
  console.log("Game saved");
}

saveGame();
console.log("Safe to quit now!");
```

You'll see:

```
Safe to quit now!
Game saved
```

The player is told it's safe to quit before the game is saved. `await` only pauses `saveGame`. The code that *called* it carries on. Fix: wait for the promise that `saveGame()` returns, with `saveGame().then(() => console.log("Safe to quit now!"))`; or with `await` inside another `async` function.

3. Expecting the screen to update in the middle of your code (browser)

```
const message = document.querySelector("#message");

message.textContent = "Building your report...";
buildBigReport(); // pretend this takes 3 seconds
message.textContent = "Done!";
```

You'd expect "Building your report..." to show for 3 seconds. It never appears. The browser can only redraw the screen once your code has finished, and by then the text says "Done!". In the meantime the page looks frozen.

Fix: split the slow work into chunks with `setTimeout`, or move it to a Web Worker, so the browser gets a chance to redraw.

4. Wrapping slow code in a promise to make it "async"

```
const start = Date.now();

setTimeout(() => {
  console.log(`Timer ran after ${Date.now() - start} ms`);
}, 0);

const report = new Promise((resolve) => {
  while (Date.now() - start < 500) {
    // a slow calculation, still on the main thread
  }
  resolve("Report ready");
});

report.then((message) => console.log(message));
```

You'll see something like this (your number will be a little different):

```
Report ready
Timer ran after 504 ms
```

A promise doesn't move work to another thread. The function inside `new Promise` runs straight away on the main thread (Puzzle 2), so it blocks just the same. Moving the slow part into `.then` wouldn't help either: it would just block as a microtask. Fix: split the work into chunks with `setTimeout`, or move it to a Web Worker.

Quick recap

- JavaScript runs your code on one main thread, one thing at a time. A long job blocks everything else: timers, clicks, and screen updates.
- The environment (the browser or Node) does the waiting, then lines up your callbacks: timers and events in the **task queue**, promise callbacks and code after `await` in the **microtask queue**.
- The event loop runs one task until the call stack is empty, then **every** microtask, then (in the browser) maybe redraws the screen, then takes the next task.
- So the order is: plain code first, then microtasks, then tasks. `setTimeout(fn, 0)` means "after all that", not "now".
- The function inside `new Promise` runs straight away, and an `async` function runs normally until its first `await`.

- Keep the page responsive by splitting big jobs into chunks with `setTimeout` , or by moving them to a Web Worker.
 - Node adds `process.nextTick` (runs before promise callbacks) and `setImmediate` (a task). Its order against `setTimeout(fn, 0)` isn't guaranteed, so don't rely on it.
-

Next: try the [exercises](#), then move on to [41 Memory and Garbage Collection](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - For the "guess the order" parts, write your guess down *before* you run the code. Being wrong is fine: that's where the learning happens.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Opening the cafe

Every morning, the cafe's app logs the opening routine. Copy this into `ex1.js` :

```
console.log("Unlock the door");

setTimeout(() => console.log("First customer walks in"), 0);

queueMicrotask(() => console.log("Turn on the coffee machine"));

Promise.resolve().then(() => console.log("Put out the pastries"));

setTimeout(() => console.log("Lunch rush starts"), 50);

console.log("Switch on the lights");
```

1. **Before running it**, write your guess for the output at the top of the file, as comments.
2. Run it and compare. There's no expected output here on purpose: running the file is how you check your guess.

3. At the end of each `console.log` line, add a comment saying where that message waited: `// call stack`, `// microtask queue`, or `// task queue`.
4. Move the `Lunch rush starts` line to the very top of the file and run it again. Does the output change? Write one sentence explaining why.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Service with a smile

A restaurant's training app prints the steps of serving a table, but the lines in the file are in the wrong order. Copy them into `ex2.js`:

```
console.log("Serve dessert");
console.log("Take the order");
console.log("Bring the menu");
console.log("Say hello");
console.log("Show them to a table");
```

Make the program print the steps in the right order:

```
Say hello
Show them to a table
Bring the menu
Take the order
Serve dessert
```

Rule: don't move, delete, or reword any line. You may only wrap a line's `console.log(...)` inside `setTimeout`, `queueMicrotask`, or `Promise.resolve().then(...)`.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Movie night

Your cinema app books seats, but its messages come out in a confusing order. Copy this into `ex3.js`:

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function bookSeat(movie) {
  console.log(`Booking a seat for ${movie}...`);
  await wait(100);
  console.log(`Seat booked for ${movie}!`);
}

bookSeat("Dune");
setTimeout(() => console.log("Popcorn is ready"), 0);
console.log("Enjoy the movie!");
```

1. Guess the output, then run it and check.
2. Add a comment under the code explaining, in your own words, why `Enjoy the movie!` doesn't come last.
3. Now fix the bottom part. Book a seat for `"Dune"`, then one for `"Wicked"`, and only then print `Enjoy the movie!`. Keep the popcorn line exactly as it is, straight after you start the bookings.

Expected output after your fix:

```
Booking a seat for Dune...
Popcorn is ready
Seat booked for Dune!
Booking a seat for Wicked...
Seat booked for Wicked!
Enjoy the movie!
```

Rule: don't change `wait` or `bookSeat`.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 4 (Medium): Newsletter in batches

Your online shop sends a newsletter to every subscriber. On a big list, sending them all in one loop would hog the main thread, and new orders would have to wait. So you'll send the emails in batches, and give other tasks a turn between batches.

Start `ex4.js` with this list:

```
const subscribers = [
  "ana@mail.com", "ben@mail.com", "cai@mail.com", "dev@mail.com",
  "eli@mail.com", "fay@mail.com", "gus@mail.com",
];
```

Write a function `sendNewsletter(emails, batchSize)`. It "sends" one batch by printing it on one line, then waits for a new task before the next batch. When everything is sent, it prints a final message.

Test it with these two lines at the bottom of your file. The first one stands in for a customer placing an order while the newsletter goes out:

```
setTimeout(() => console.log("New order received!"), 0);
sendNewsletter(subscribers, 3);
```

Expected output:

```
Batch 1: ana@mail.com, ben@mail.com, cai@mail.com
New order received!
Batch 2: dev@mail.com, eli@mail.com, fay@mail.com
Batch 3: gus@mail.com
All 7 emails sent!
```

Rule: each batch must run in its own task. No sending everything in one loop!

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Bonus: try `batchSize` 2, and predict where `New order received!` lands before you run it.

Exercise 5 (Challenge): Build your own event loop

The best way to really understand a machine is to build a small one yourself. You'll write a pretend event loop with two arrays as the waiting lines.

Start `ex5.js` with this:

```
const microtasks = []; // the VIP line
const tasks = [];      // the regular line
```

Write three functions:

- `fakeQueueMicrotask(fn)` adds `fn` to the end of `microtasks`.
- `fakeSetTimeout(fn)` adds `fn` to the end of `tasks`. (No delay: every fake timer counts as "ready".)

- `runEventLoop()` follows the cashier's rules: run *every* microtask (including ones added along the way), then take *one* task and run it, and repeat until both lines are empty.

Then paste this scenario at the bottom of your file:

```
console.log("script start");

fakeSetTimeout(() => {
  console.log("task 1");
  fakeQueueMicrotask(() => console.log("micro inside task 1"));
});

fakeSetTimeout(() => console.log("task 2"));

fakeQueueMicrotask(() => {
  console.log("micro 1");
  fakeQueueMicrotask(() => console.log("micro 2 (added by micro 1)"));
});

console.log("script end");

runEventLoop();
```

Expected output:

```
script start
script end
micro 1
micro 2 (added by micro 1)
task 1
micro inside task 1
task 2
```

Finally, prove your loop behaves like the real one. Copy the scenario into a new file, `ex5-real.js`. Replace `fakeSetTimeout(fn)` with a real `setTimeout(fn, 0)` and `fakeQueueMicrotask` with the real `queueMicrotask`, and delete the `runEventLoop();` line. Run it. You should get exactly the same output.

Rule: in `runEventLoop`, take callbacks from the *front* of each array, so the first to join is the first served.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Try this tiny file:

```
let ticks = 0;  
setInterval(() => ticks++, 1000);
```

It never finishes (press `Ctrl + C` to stop it). The interval keeps adding a new task every second, so Node always has something left to wait for.

A timer you forget to clear keeps running, and it keeps everything it uses alive in memory, too. That's one of the classic **memory leaks**, and [chapter 41](#) is all about them. 😊

41 Memory and Garbage Collection

What is it?

Every value your program makes (a number, a string, an object, an array) needs a little space in the computer's **memory**: the fast, temporary storage where running programs keep their data, often called RAM.

Garbage collection is how JavaScript cleans up after you. Every so often, it finds the values your program can't use anymore and frees their space for new ones. It happens automatically. You never have to free memory yourself.

Why does it matter?

Most of the time, you can forget about memory. The garbage collector does its job quietly, and that's true for most of the code you'll ever write.

But it can only throw away things you've *let go of*. If your code keeps holding on to something by accident (a timer that never stops, a list that only grows), that memory can never be freed. This is called a **memory leak**.

Leaks are sneaky. Nothing breaks at first. The app just gets slower and slower: a browser tab using 2 GB after a day open, a server that has to be restarted every night. Knowing how the garbage collector decides what to keep helps you avoid leaks, and find them when they happen.

Real-world example

Think of a bunch of helium balloons at a party.

Balloons	JavaScript
A balloon	A value in memory, like an object or an array
Holding a balloon's string	A variable (or another object) that refers to the value
Balloons tied to other balloons	Objects stored inside other objects
A balloon nobody is holding, not even through another balloon	A value your program can't reach anymore
It floats away	The garbage collector frees its memory
Forgetting that a string is still tied to your wrist	A memory leak

A balloon stays as long as *someone* holds its string, or it's tied to a balloon that someone holds. Let go of the last string, and it floats away by itself. You don't have to pop it.

How it works

Where values live: the stack and the heap

JavaScript keeps your data in two main places:

	The stack	The heap
Like...	Small labeled drawers	A big warehouse
Holds	The variables of the functions running right now	Objects, arrays, and functions
Size	Small and tidy	Big: things can be any size
Cleaned up	Straight away, the moment a function returns	By the garbage collector, once nothing can reach them

The **stack** is the call stack you know from [chapter 17](#) and [chapter 40](#). Each running function gets its own drawer for its variables. When the function returns, its drawer is emptied.

The **heap** is where objects and arrays live. They can be big, and they can outlive the function that made them. A variable doesn't hold the object itself. It holds a **reference**: a slip of paper with the object's shelf number in the warehouse.

```
function makeOrder() {  
  const quantity = 2; // lives in this function's drawer  
  const order = { item: "pizza", quantity }; // the object goes to the heap  
  return order; // hand back the shelf number  
}  
  
const myOrder = makeOrder();  
console.log(myOrder); // prints: { item: 'pizza', quantity: 2 }
```

When `makeOrder` returns, its drawer is emptied: `quantity` and the local `order` variable are gone. The object itself survives in the heap, because `myOrder` still holds its shelf number.

This is a simplified picture. Real engines are cleverer (for example, long strings usually live in the heap too), and you don't need those details. The useful idea is: objects live in the heap, and variables hold references to them.

References keep things alive

Remember [chapter 16](#)? Two variables can refer to the very same object:

```
const cart = { items: ["apple"] };
const sameCart = cart; // a second string to the same balloon

sameCart.items.push("bread");
console.log(cart.items); // prints: [ 'apple', 'bread' ]
```

That's the rule the garbage collector cares about: **an object stays in memory as long as at least one reference can still reach it**. Variables, properties of other objects, items in arrays, entries in a Map, and variables used by closures all count as references.

Garbage collection: can anyone still reach it?

The **garbage collector** (GC for short) is the part of the JavaScript engine that frees memory. It runs by itself, every now and then, whenever the engine decides it's a good moment.

It works by **reachability**. It starts from the **roots**, the things that are always in use:

- global variables,
- the variables of the functions running right now (the stack),
- callbacks that are still waiting: timers, event listeners, promise callbacks.

From there, it follows every reference, like following strings from hands to balloons, and marks everything it can reach. Anything it didn't reach is garbage, and its memory is freed. This is called **mark-and-sweep**: mark what's reachable, sweep away the rest.

```
let user = { name: "Ava" }; // one string
let admin = user;           // two strings to the same object

user = null; // one string let go. admin still holds it, so it stays
admin = null; // the last string is gone: the object can be collected
```

Nothing prints here, and that's the point. You can't see the garbage collector at work, and you don't decide when it runs. Once nothing can reach the object, the GC frees it at some point later.

Objects that point at each other are no problem either:

```
function makeTeam() {
  const alex = { name: "Alex" };
  const sam = { name: "Sam" };
  alex.teammate = sam;
  sam.teammate = alex; // they hold each other's strings
}

makeTeam();
```

After `makeTeam` returns, `alex` and `sam` still refer to each other. But no root can reach either of them, so both get collected, like two balloons tied together floating away.

Watching it happen in Node

Node can tell you how much memory it's using, with `process.memoryUsage()`. Its `heapUsed` property is how many **bytes** (tiny units of memory) your objects take up right now. Dividing by 1024 twice turns bytes into **megabytes** (MB), which are easier to read.

```
function heapMB() {
  return Math.round(process.memoryUsage().heapUsed / 1024 / 1024);
}

console.log(`Start: ${heapMB()} MB`);

let orders = [];
for (let i = 0; i < 1000000; i++) {
  orders.push({ id: i, item: "pizza" });
}
console.log(`After 1 million orders: ${heapMB()} MB`);

orders = null; // let go of the only string
gc();           // ask for a clean-up right now (needs --expose-gc)
console.log(`After letting go: ${heapMB()} MB`);
```

`gc()` isn't normally available. It only exists when you start Node with a special flag, so save this as `memory.js` and run it like this:

```
node --expose-gc memory.js
```

You'll see something like this:

```
Start: 4 MB
After 1 million orders: 68 MB
After letting go: 4 MB
```

Your numbers will be different. They depend on your computer and your Node version, and they can change from run to run. What matters is the shape: memory goes up while the orders are reachable, and drops back once nothing holds them.

Without the flag, you'd get `ReferenceError: gc is not defined`. That's on purpose. The flag is for experiments like this one. Never use it in real programs: the GC knows better than you when to run.

Memory leaks: things you forgot to let go of

A memory leak is memory your program keeps even though it will never use it again. In JavaScript, it's nearly always the same story: *something is still holding the string*.

One leaky object isn't a problem. The trouble starts when the leaky code runs again and again, and the pile keeps growing: on every page visit, every opened window, every request to your server.

Here are the five usual suspects.

Leak 1: timers you never stop

An interval keeps running until you clear it, even when nobody needs it anymore. And while it's running, its callback is a root, so everything the callback uses stays alive too.

```
function startLiveScores(match) {
  const updates = [];

  setInterval(() => {
    updates.push(`${match}: update ${updates.length + 1}`);
  }, 1000);
}

startLiveScores("Arsenal vs Chelsea");
```

Even after the user leaves the match page, the interval keeps going, and `updates` grows every second, forever. In Node, it also stops the program from ever ending, as you saw at the end of [chapter 40's exercises](#).

Fix: keep the timer's id, and clear it when you're done. Returning a `stop` function makes that easy for whoever started it:

```
function startLiveScores(match) {
  const updates = [];

  const timerId = setInterval(() => {
    updates.push(`${match}: update ${updates.length + 1}`);
    console.log(updates.at(-1));
  }, 1000);

  return function stop() {
    clearInterval(timerId); // let go: the callback and `updates` can be freed
    console.log("Stopped live scores");
  };
}

const stopScores = startLiveScores("Arsenal vs Chelsea");
setTimeout(stopScores, 3500); // the user leaves the page after 3.5 seconds
```

You'll see:

```
Arsenal vs Chelsea: update 1
Arsenal vs Chelsea: update 2
Arsenal vs Chelsea: update 3
Stopped live scores
```

Then the program ends by itself, because nothing is left waiting. A good rule of thumb: **whoever starts something should be able to stop it.**

Leak 2: listeners that are never removed

An event listener stays attached until you remove it. Listeners on things that live as long as the page, like `document` and `window`, stay alive as long as the page does, along with everything their callbacks use.

It gets worse when the code that adds them runs more than once. Here's a photo viewer (a "lightbox") for a web page:


```
function openLightbox(photoName) {
  document.addEventListener("keydown", (event) => {
    if (event.key === "ArrowRight") {
      console.log(`Next photo after ${photoName}`);
    }
  });
}

openLightbox("beach.jpg");
openLightbox("forest.jpg");
openLightbox("city.jpg");
```

Now press the right arrow key once, and the console shows:

```
Next photo after beach.jpg
Next photo after forest.jpg
Next photo after city.jpg
```

Every photo you ever opened is still listening. That's a leak *and* a bug.

One fix is `removeEventListener` from [chapter 21](#), but it needs the exact same function you added, which gets fiddly with several listeners. There's a neater way: an `AbortController`, which you used to cancel `fetch` requests in [chapter 33](#). Pass its `signal` when you add each listener, and a single `abort()` removes all of them at once:

```
function openLightbox(photoName) {
  const controller = new AbortController();
  const options = { signal: controller.signal };

  document.addEventListener("keydown", (event) => {
    if (event.key === "ArrowRight") {
      console.log(`Next photo after ${photoName}`);
    }
    if (event.key === "Escape") {
      controller.abort(); // close: removes every listener that used this signal
    }
  }, options);

  window.addEventListener("resize", () => {
    console.log(`Fit ${photoName} to the new window size`);
  }, options);
}
```

After Escape, both listeners are gone, along with everything they were holding on to.

Leak 3: caches and lists that only grow

A **cache** is a place where you keep results so you don't have to work them out (or download them) again. Caches are great, but a cache that never forgets grows forever. On a server that runs for weeks, that's a leak. The same goes for arrays you keep pushing to and never trim: logs, history, "recently viewed" lists.

Fix: give it a limit. A Map ([chapter 35](#)) remembers the order its keys were added in, so its first key is always the oldest:

```
const MAX_ENTRIES = 3;
const forecastCache = new Map();

function saveForecast(city, forecast) {
  if (forecastCache.size >= MAX_ENTRIES) {
    const oldestCity = forecastCache.keys().next().value; // the first key added
    forecastCache.delete(oldestCity);
  }
  forecastCache.set(city, forecast);
}

saveForecast("London", "12°C, cloudy");
saveForecast("Paris", "15°C, sunny");
saveForecast("Rome", "21°C, sunny");
saveForecast("Oslo", "4°C, snow");

console.log([...forecastCache.keys()]); // prints: [ 'Paris', 'Rome', 'Oslo' ]
```

`keys()` gives you an iterator ([chapter 36](#)), and `.next().value` takes its first value. London was the oldest, so it made room for Oslo. A real app might allow 100 or 1000 entries, but the idea is the same.

Leak 4: closures holding big data

A closure ([chapter 25](#)) keeps alive every outside variable it uses, for as long as the closure itself is alive. That's usually exactly what you want. It turns into a leak when a small, long-lived function holds on to something big.

```
function makeGreeter(user) {
  // user is a big object: profile photo, order history, settings...
  return function greet() {
    console.log(`Hi, ${user.name}!`);
  };
}
```

`greet` only needs the name. But it uses `user`, so it keeps the *whole* user object alive, photo and all, for as long as `greet` exists.

Fix: copy out the small part you need, so the closure doesn't use the big object at all:

```
function makeGreeter(user) {  
  const name = user.name; // keep only what you need  
  return function greet() {  
    console.log(`Hi, ${name}!`);  
  };  
}
```

Now, once nothing else holds the user object, it can be collected, and `greet` still works.

Leak 5: detached DOM elements

A **detached** element is one that's been removed from the page, but that your JavaScript still refers to. It's not on screen anymore, but it can't be collected either, and neither can anything inside it.

```
const ui = {  
  popup: document.querySelector("#promo-popup"),  
};  
  
function closePopup() {  
  ui.popup.remove(); // gone from the page...  
  // ...but ui.popup still holds it, so it stays in memory  
}
```

One popup is tiny. Now imagine a chat app that removes old messages from the screen but keeps every one of those elements in an array. Fix: when you remove an element for good, drop your references to it as well (`ui.popup = null`;). For extra data about elements, use a `WeakMap`, coming up next.

Weak references: WeakMap and WeakRef

Chapter 35 introduced **WeakMap**: a Map whose keys must be objects, and which holds them *weakly*. A **weak reference** doesn't count as holding the string. When nothing else refers to a key object, the GC can collect it, and its WeakMap entry disappears along with it.

That makes a WeakMap perfect for attaching extra data to objects, like DOM elements, without keeping them alive:

```
const clickCounts = new WeakMap();

function countClick(button) {
  const count = (clickCounts.get(button) ?? 0) + 1;
  clickCounts.set(button, count);
  return count;
}
```

If a button is removed from the page later and your code lets go of it, its count goes too. A normal Map would keep every button it ever saw alive forever.

`WeakRef` is a rarer tool. `new WeakRef(object)` holds an object without keeping it alive, and `ref.deref()` gives it back, or `undefined` if it's been collected. Since you can't know when the GC will run, code built on `WeakRef` is hard to get right. You'll rarely need it, so reach for a `WeakMap` first.

Finding leaks with Chrome DevTools

When a page gets slower and slower the longer it's open, the **Memory** tab in Chrome (or Edge) DevTools can show you what's piling up. A **heap snapshot** is a photo of everything in the heap at one moment. Comparing two of them shows you what grew in between.

1. Open your page, press `F12`, and go to the **Memory** tab. (If you can't see it, click `>>` to find more tabs.)
2. Choose **Heap snapshot** and click **Take snapshot**. This is your "before" photo.
3. Do the thing you suspect is leaking, several times. For example, open and close the lightbox 10 times.
4. Take a second snapshot.
5. With the second snapshot selected, switch the view from **Summary** to **Comparison**. You now see what was added since the first photo. Click the **# Delta** or **Size Delta** column to sort the biggest growth to the top.
6. To hunt for detached elements, go back to the **Summary** view and type `Detached` into the filter box.
7. Click a suspicious item. The **Retainers** section below it shows the chain of references keeping it alive: who's still holding the string.

A snapshot only contains things that are still reachable, so if something shows up there, something really is holding on to it. DevTools changes a little between Chrome versions, so your screens may look slightly different.

Keep it in proportion

Two honest points before you go leak hunting:

- Garbage collection is automatic, and it's very good. For most code, you don't need to think about memory at all. Local variables clean themselves up when a function returns.
- Think about memory when something **lives a long time** or **happens over and over**: a page that stays open all day, a server that runs for weeks, code that runs on every click or every request. That's where

leaks grow.

Common mistakes

1. Starting a second timer without stopping the first

```
let timerId = null;
let secondsLeft = 5;

function startCountdown() {
  timerId = setInterval(() => {
    secondsLeft--;
    console.log(`Time left: ${secondsLeft}`);
    if (secondsLeft <= 0) {
      clearInterval(timerId);
    }
  }, 1000);
}

startCountdown();
startCountdown(); // the user clicked Start twice
```

You'll see this, and it never stops (press `Ctrl + C`):

```
Time left: 4
Time left: 3
Time left: 2
Time left: 1
Time left: 0
Time left: -1
Time left: -2
```

The second click started a second interval, so the countdown runs twice as fast. Worse, `timerId` now only remembers the second one. The first interval can never be cleared: it leaks and keeps counting down forever. Fix: stop the old timer before starting a new one. `clearInterval` does nothing if there's nothing to clear, so this is always safe:

```
function startCountdown() {
  clearInterval(timerId); // stop the old one first, if there is one
  timerId = setInterval(() => {
    // ...same as before
  }, 1000);
}
```

2. Setting everything to `null` "to save memory"

```
function orderTotal(prices) {  
  let total = 0;  
  for (const price of prices) {  
    total += price;  
  }  
  prices = null; // pointless  
  return total;  
}
```

`prices` and `total` live in the function's drawer, which is emptied the moment the function returns. It doesn't free the array either: the code that called `orderTotal` still holds its own reference to it. Setting a variable to `null` only helps when the variable lives a long time (a global, a property of a long-lived object, a cache) and you're really done with what it holds.

3. Using a string as a `WeakMap` key

```
const visits = new WeakMap();  
visits.set("sam", 3);  
// TypeError: Invalid value used as weak map key
```

A `WeakMap`'s whole job is to let its key *objects* be collected, so its keys must be objects. For string keys, use a normal `Map`, and give it a limit if it can keep growing.

4. Trusting a single memory reading

```
const readings = [];  
  
for (let round = 1; round <= 8; round++) {  
  const tempList = [];  
  for (let i = 0; i < 200000; i++) {  
    tempList.push({ id: i }); // garbage as soon as this round ends  
  }  
  readings.push(Math.round(process.memoryUsage().heapUsed / 1024 / 1024));  
}  
  
console.log(`${readings.join(" MB, ")} MB`);
```

You'll see something like this (your numbers will be different):

```
14 MB, 25 MB, 37 MB, 30 MB, 42 MB, 31 MB, 43 MB, 35 MB
```

Nothing leaks here: each `tempList` is garbage as soon as its round ends. Memory still jumps up and down, because the GC cleans up whenever it decides to, not the moment something becomes garbage. So one high number proves nothing. A leak is memory that keeps **climbing** as you repeat the same action, and never comes back down. Compare several readings, or two heap snapshots, before you decide.

Quick recap

- Objects and arrays live in the **heap**. Variables hold references to them, and each function's variables on the **stack** are cleaned up the moment it returns.
- The **garbage collector** frees anything that can't be reached from the roots (globals, running functions, waiting callbacks), like balloons nobody is holding. It's automatic.
- A **memory leak** is something you forgot to let go of. It can never be freed, and it grows every time the leaky code runs.
- The usual suspects: intervals never cleared, listeners never removed, caches and lists that only grow, closures holding big data, and detached DOM elements.
- The fixes: `clearInterval`, `removeEventListener` or an `AbortController` signal, a size limit, copying out only what a closure needs, and a `WeakMap` for data attached to objects.
- To find a leak, compare heap snapshots in the Memory tab. In Node, `process.memoryUsage()` shows the heap size, but the numbers bounce around, so look for a trend, not one number.

Next: try the [exercises](#), then move on to [42 Proxy and Reflect](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on), and run it with `node ex1.js`.
- Exercise 4 runs in the browser: make a folder `ex4` with `index.html` and `script.js`, open `index.html` by double-clicking it, and press `F12` → **Console** to see the output. Refresh the page after each change.
- Memory numbers are different on every computer, and can change from run to run. When an exercise prints memory, look at the *shape* (up, then down), not the exact numbers.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Final whistle

A sports app shows goals as they happen. Copy this into `ex1.js` and run it:

```
let goals = 0;

setInterval(() => {
  goals++;
  console.log(`Goal! Total goals: ${goals}`);
}, 500);
```

It never stops (press `Ctrl + C`), so the timer, and everything it uses, lives forever.

Change it so that after the third goal, the timer stops and the app prints `Final whistle!`. The program should then end all by itself.

Expected output:

```
Goal! Total goals: 1
Goal! Total goals: 2
Goal! Total goals: 3
Final whistle!
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Weigh your playlist

How much memory does a big playlist take up? Let's measure it.

1. Write a function `heapMB()` that returns `process.memoryUsage().heapUsed` in megabytes, rounded to a whole number.
2. Print the heap size at the start.
3. Make an array called `songs` (use `let`) with 300000 song objects like `{ id: 1, title: "Song 1" }`, and print the heap size again, along with how many songs there are.
4. Let go of the array, call `gc()`, and print the heap size one last time.

Run it with `node --expose-gc ex2.js`. You'll see something like this (your numbers will be different):

```
Start: 4 MB
With 300000 songs: 36 MB
After letting go: 4 MB
```

Then comment out the line where you let go of the array, and run it again. Write a comment explaining why the last number changed.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Recently viewed

Online shops show a "Recently viewed" row of products. If that list kept every product a shopper ever looked at, it would only ever grow. Yours will have a limit.

Write a function `createRecentlyViewed(limit)` that returns an object with two methods:

- `view(product)` records that the shopper looked at a product. If the product is already in the list, it moves to the front instead of appearing twice. If the list grows past `limit`, the oldest product is dropped.
- `list()` returns an array of the products, newest first.

Test it with this code:

```
const recent = createRecentlyViewed(3);
recent.view("Headphones");
recent.view("Keyboard");
recent.view("Mouse");
console.log(recent.list());

recent.view("Keyboard"); // viewed again: moves to the front
console.log(recent.list());

recent.view("Monitor"); // the list is full, so the oldest one goes
console.log(recent.list());
```

Expected output:

```
[ 'Mouse', 'Keyboard', 'Headphones' ]
[ 'Keyboard', 'Mouse', 'Headphones' ]
[ 'Monitor', 'Keyboard', 'Mouse' ]
```

Rule: after every call to `view`, the list must hold at most `limit` products.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): The chat that won't stop listening

A website has a support chat you can open and close. Customers say that after a while, every key they press shows up several times. Your job: find the leak and fix it.

Make a folder `ex4` with this `index.html` :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Support Chat</title>
  <script src="script.js" defer></script>
</head>
<body>
  <button id="open-chat">Open chat</button>
  <button id="close-chat">Close chat</button>

  <div id="chat" hidden>
    <p>Hi! How can we help? (Try typing a key.)</p>
  </div>
</body>
</html>

```

And this `script.js`:

```

const chat = document.querySelector("#chat");
const openButton = document.querySelector("#open-chat");
const closeButton = document.querySelector("#close-chat");

let chatNumber = 0;

function openChat() {
  chatNumber++;
  const thisChat = chatNumber;
  chat.hidden = false;

  document.addEventListener("keydown", (event) => {
    console.log(`Chat ${thisChat} heard: ${event.key}`);
  });
}

function closeChat() {
  chat.hidden = true;
}

openButton.addEventListener("click", openChat);
closeButton.addEventListener("click", closeChat);

```

1. Open the page and the console. Click **Open chat**, **Close chat**, **Open chat**, **Close chat**, **Open chat**, then press the `a` key. You'll see:

```
Chat 1 heard: a
Chat 2 heard: a
Chat 3 heard: a
```

2. Fix `script.js` with an `AbortController`, so that closing the chat removes its listener. After refreshing, the same clicks and key press should give only:

```
Chat 3 heard: a
```

3. Click **Close chat** and press a key. Nothing should appear.

Bonus: click **Open chat** twice in a row without closing. Make sure that doesn't add a second listener either.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Leak detective

A music player app gets slower the longer it runs. You've narrowed it down to this code. Copy it into `ex5.js`:

```

const playHistory = [];
const coverCache = new Map();

function loadCover(songId) {
  // pretend this downloads a big album cover image
  return { songId, pixels: new Array(20000).fill(0) };
}

function getCover(songId) {
  if (!coverCache.has(songId)) {
    coverCache.set(songId, loadCover(songId));
  }
  return coverCache.get(songId);
}

function drawProgressBar(cover) {
  // pretend this redraws the progress bar next to the cover
}

function playSong(songId) {
  const cover = getCover(songId);
  playHistory.push(`Played song ${songId}`);

  setInterval(() => {
    drawProgressBar(cover); // redraw the progress bar every second
  }, 1000);
}

for (let songId = 1; songId <= 500; songId++) {
  playSong(songId);
}

console.log(`History entries: ${playHistory.length}`);
console.log(`Covers in cache: ${coverCache.size}`);
console.log(`Latest: ${playHistory.at(-1)}`);

```

Run it. It prints this, and then it never ends (press `Ctrl + C`):

```

History entries: 500
Covers in cache: 500
Latest: Played song 500

```

There are three leaks. Fix all of them:

1. `playHistory` keeps every song ever played. Keep only the 10 most recent.
2. `coverCache` keeps every cover ever loaded. Keep at most 20, dropping the oldest.

3. Every song starts a new progress-bar timer, and none of them ever stops. Only the song that's playing needs one.

Then add a function `stopPlayer()` that stops the current timer and prints `Player stopped. Bye!`, and call it at the very end of the file.

Expected output:

```
History entries: 10
Covers in cache: 20
Latest: Played song 500
Player stopped. Bye!
```

And the program should end by itself, straight away.

Rule: keep the loop that plays 500 songs and the three `console.log` lines as they are.

Bonus: measure the difference. Save a copy of the leaky version too. In both files, add `gc();` followed by a line that prints the heap size in MB, just before the end, and run them with `node --expose-gc`. Your numbers will be different, but the fixed version should use many times less memory. Why do you need `gc()` first for a fair comparison? (Think back to mistake 4 in the notes.)

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

In Exercise 3, and in the caches in this chapter, *you* had to remember to check the size limit every time something was added. What if an object could run your code automatically whenever anyone reads or changes one of its properties?

That's what a **Proxy** does, and it's coming up next in [chapter 42](#). 🤖

42 Proxy and Reflect

What is it?

A **Proxy** is an object that stands in front of another object and can step in whenever someone uses it: reading a property, changing one, checking whether one exists, or deleting one.

Reflect is a built-in object full of functions that do "the normal thing" for each of those actions. You'll use it inside a Proxy, after your own checks.

Together, they're JavaScript's main tools for **metaprogramming**: code that controls how other code behaves.

Why does it matter?

A normal object just does what it's told. Write `member.age = -5`, and it stores `-5`. No questions, no warnings, no record that anything changed.

Sometimes you want an object that reacts:

- **Refuse bad values** the moment someone tries to set them (an age can't be negative).
- **Give a default** instead of `undefined` when a key is missing.
- **Log every change**, so you can find out who keeps changing the price.
- **Lock an object**, so nobody can change or delete anything.
- **Update the screen automatically** whenever the data changes. That's how Vue 3, a popular front-end framework, works under the hood.

A Proxy does all of this *without changing the code that uses the object*. That code reads and writes properties exactly as before.

Real-world example

Think of the receptionist at the front desk of an office.

At the office	With a Proxy
The office, with the people visitors want to see	The target : the original object
The receptionist that every visitor talks to first	The proxy : your code uses it instead of the target
The receptionist's instructions: "check ID", "log every delivery"	The handler : an object holding your rules

At the office	With a Proxy
One kind of request: a visit, a delivery, a question	A trap : <code>get</code> , <code>set</code> , <code>has</code> , <code>deleteProperty</code>
"Everything's fine, go on through"	<code>Reflect</code> : do the normal thing

Visitors don't need to know the receptionist is there. They ask for what they want, as usual. The receptionist can check it, change it, write it down, or say no, before passing the request on.

How it works

Your first proxy

You make a proxy with `new Proxy(target, handler)`:

- the **target** is the original object,
- the **handler** is an object whose methods are **traps**: functions that catch one kind of action.

Here's a proxy that notices every property read:

```
const book = { title: "Dune", pages: 412 };

const loggedBook = new Proxy(book, {
  get(target, property) {
    console.log(`Someone read "${property}"`);
    return target[property];
  },
});

console.log(loggedBook.title);
```

You'll see:

```
Someone read "title"
Dune
```

When you read `loggedBook.title`, JavaScript doesn't go straight to `book`. It calls your `get` trap with the target and the property name (`"title"`). Whatever the trap returns is the value you get back.

A proxy **wraps** the target. It doesn't copy it. With an empty handler, a proxy passes everything straight through, and changes land on the original object. Add this to the same file:

```
const plainProxy = new Proxy(book, {});
plainProxy.pages = 500;

console.log(book.pages);           // prints: 500
console.log(plainProxy === book); // prints: false
```

The proxy and the target are two different objects that share the same data. Your traps only run when code goes *through the proxy*.

The four traps you'll use most

There are 13 traps in total, but these four cover most real uses:

Trap	Runs when someone...	Example
<code>get(target, property, receiver)</code>	reads a property	<code>proxy.name</code>
<code>set(target, property, value, receiver)</code>	sets a property	<code>proxy.name = "Sam"</code>
<code>has(target, property)</code>	checks with <code>in</code>	<code>"name" in proxy</code>
<code>deleteProperty(target, property)</code>	deletes a property	<code>delete proxy.name</code>

You only write the traps you need. Anything without a trap goes straight through to the target, as normal.

Two details to know:

- `property` is always a string (or, now and then, a symbol from [chapter 36](#)). Even `list[0]` gives the trap the string `"0"`.
- `set` and `deleteProperty` must return `true` to say "done". Returning `false` means "refused". In strict mode, which includes modules and classes ([chapter 26](#)), a refusal throws a `TypeError`. In a plain `.js` file, it fails silently.

And what's `receiver`? It's the object the property was read or set through: usually the proxy itself. You don't have to do anything with it except pass it on to `Reflect`, which is next.

Reflect: "do the normal thing"

Most traps do a little extra work and then want the normal behavior to happen. `Reflect` has a function for every trap, taking the same arguments in the same order:

Inside this trap...	...this does the normal thing
<code>get(target, property, receiver)</code>	<code>Reflect.get(target, property, receiver)</code>

Inside this trap...	...this does the normal thing
<code>set(target, property, value, receiver)</code>	<code>Reflect.set(target, property, value, receiver)</code>
<code>has(target, property)</code>	<code>Reflect.has(target, property)</code>
<code>deleteProperty(target, property)</code>	<code>Reflect.deleteProperty(target, property)</code>

Why not just write `target[property]` ? Two reasons:

1. `Reflect.set` and `Reflect.deleteProperty` return `true` or `false`, which is exactly what those traps must return. Returning their result means you can't forget.
2. They pass `receiver` along, so getters behave properly. You met getters in classes in [chapter 27](#), and they work the same way in plain objects. Look at this user with a `fullName` getter:

```
const user = {
  firstName: "Ada",
  lastName: "Lovelace",
  get fullName() {
    return `${this.firstName} ${this.lastName}`;
  },
};

const withTarget = new Proxy(user, {
  get(target, property) {
    console.log(`read ${property}`);
    return target[property];
  },
});

const withReflect = new Proxy(user, {
  get(target, property, receiver) {
    console.log(`read ${property}`);
    return Reflect.get(target, property, receiver);
  },
});

console.log(withTarget.fullName);
console.log("---");
console.log(withReflect.fullName);
```

You'll see:

```
read fullName
Ada Lovelace
---
read fullName
read firstName
read lastName
Ada Lovelace
```

With `target[property]`, `this` inside the getter is the plain target, so its reads of `firstName` and `lastName` sneak past your trap. With `Reflect.get(..., receiver)`, `this` is the proxy, so your trap sees everything.

So here's the habit to build: **in every trap, do your extra work, then hand over to the matching `Reflect` function with the same arguments.**

Use case: checking values (validation)

A gym's app keeps member records. An age must be a positive number, however the code tries to set it:

```
function createMember(member) {
  return new Proxy(member, {
    set(target, property, value, receiver) {
      if (property === "age" && (typeof value !== "number" || value <= 0)) {
        throw new TypeError(`Age must be a positive number, got ${value}`);
      }
      return Reflect.set(target, property, value, receiver);
    },
  });
}

const member = createMember({ name: "Priya", age: 31 });
member.age = 32;
console.log(member.age); // prints: 32

try {
  member.age = -4;
} catch (error) {
  console.log(error.message); // prints: Age must be a positive number, got -4
}
```

The check lives in one place, and every part of the app that changes `member.age` gets it for free. Throwing a clear error ([chapter 18](#)) beats returning `false`: the caller finds out *what* was wrong.

Notice that `createMember` takes the object and hands back only the proxy. That keeps the target out of reach, so nobody can go around the checks. (You'll see why that matters in Common mistakes.) The starting values aren't checked, though: the trap only runs for changes made after the proxy exists.

Use case: default values

Here's a word counter for restaurant reviews. Normally you'd have to check whether each word is already in the object before adding 1. With a `get` trap, a missing word reads as `0`:

```
const wordCounts = new Proxy({}, {
  get(target, property, receiver) {
    return Reflect.get(target, property, receiver) ?? 0; // missing words count as 0
  },
});

const review = "the food was great and the staff were great";
for (const word of review.split(" ")) {
  wordCounts[word] += 1; // no need to check whether the word exists yet
}

console.log(wordCounts.great); // prints: 2
console.log(wordCounts.pizza); // prints: 0
```

`wordCounts[word] += 1` reads the old count (through your trap, so `0` for a new word), adds 1, and sets the new count. The `??` operator ([chapter 07](#)) gives the default only when the real value is `undefined` or `null`.

Use case: logging changes

When a value keeps changing and you can't work out why, let a proxy tell you:

```
function watchChanges(object, label) {
  return new Proxy(object, {
    set(target, property, value, receiver) {
      console.log(`${label}.${property}: ${target[property]} -> ${value}`);
      return Reflect.set(target, property, value, receiver);
    },
    deleteProperty(target, property) {
      console.log(`${label}.${property} was deleted`);
      return Reflect.deleteProperty(target, property);
    },
  });
}

const cart = watchChanges({ items: 0, total: 0 }, "cart");
cart.items = 2;
cart.total = 19.98;
cart.coupon = "SAVE10";
delete cart.coupon;
```

You'll see:

```
cart.items: 0 -> 2
cart.total: 0 -> 19.98
cart.coupon: undefined -> SAVE10
cart.coupon was deleted
```

Reading `target[property]` inside the trap is fine here: it reads the old value straight from the target, without going through the proxy.

Use case: read-only objects

Some objects should never change once they're set up, like an app's settings. A proxy can refuse every change, with a clear message:

```
function readOnly(object) {
  return new Proxy(object, {
    set(target, property) {
      throw new TypeError(`Can't change "${property}": this object is read-only`);
    },
    deleteProperty(target, property) {
      throw new TypeError(`Can't delete "${property}": this object is read-only`);
    },
  });
}

const config = readOnly({ apiUrl: "https://api.example.com", retries: 3 });
console.log(config.retries); // prints: 3

try {
  config.retries = 10;
} catch (error) {
  console.log(error.message); // prints: Can't change "retries": this object is read-only
}
```

`Object.freeze` ([chapter 43](#)) is a simpler way to lock an object. But in a plain `.js` file, a frozen object ignores changes silently, while this proxy explains exactly what went wrong.

Use case: a custom `in` check

The `has` trap decides what `in` answers. Here, a bakery's opening hours become something you can ask with `in`:

```
const openingHours = new Proxy({}, {
  has(target, hour) {
    return Number(hour) >= 9 && Number(hour) < 17;
  },
});

console.log(10 in openingHours); // prints: true
console.log(20 in openingHours); // prints: false
```

The object is empty. The trap works out the answer instead of looking anything up. `in` turns `10` into the string `"10"` before calling the trap, which is why it uses `Number(hour)`.

Use case: negative array indexes

In JavaScript, `list[-1]` is just `undefined`. That's why [chapter 10](#) showed you `.at(-1)` for the last item. A proxy can make negative indexes work with square brackets too:

```
function withNegativeIndexes(array) {
  return new Proxy(array, {
    get(target, property, receiver) {
      if (typeof property === "string" && Number(property) < 0) {
        property = String(target.length + Number(property)); // "-1" becomes "2"
      }
      return Reflect.get(target, property, receiver);
    },
  });
}

const queue = withNegativeIndexes(["Ana", "Ben", "Cai"]);
console.log(queue[-1]); // prints: Cai
console.log(queue[-3]); // prints: Ana
console.log(queue[0]); // prints: Ana
console.log(queue.length); // prints: 3
console.log([...queue]); // prints: [ 'Ana', 'Ben', 'Cai' ]
```

The `typeof property === "string"` check matters. When you spread or loop over the array, JavaScript reads `Symbol.iterator` through your trap too, and turning a symbol into a number throws `TypeError: Cannot convert a Symbol value to a number`. The check lets symbols pass straight through.

In real code, `.at(-1)` already does this job. The point of the example is what it shows: a proxy can change what square brackets *mean*.

Use case: a tiny reactivity demo

Reactivity means "when the data changes, the screen updates by itself". You don't have to remember to redraw after every change: you change the data, and the view follows.

Here's the idea in a few lines, with a game's scoreboard:

```
function reactive(data, onChange) {
  return new Proxy(data, {
    set(target, property, value, receiver) {
      const ok = Reflect.set(target, property, value, receiver);
      onChange(target); // tell whoever is listening that the data changed
      return ok;
    },
  });
}

function render(game) {
  console.log(`[Screen] ${game.player}: ${game.score} points, ${game.lives} lives`);
}

const game = reactive({ player: "Leo", score: 0, lives: 3 }, render);

render(game); // draw the first screen
game.score = 10; // the screen updates by itself
game.lives = 2;
```

You'll see:

```
[Screen] Leo: 0 points, 3 lives
[Screen] Leo: 10 points, 3 lives
[Screen] Leo: 10 points, 2 lives
```

The code that changes `score` and `lives` knows nothing about the screen. It sets properties, and the proxy takes care of the rest. In a real page, `render` would update the DOM ([chapter 20](#)) instead of logging.

This is the core idea behind the reactivity in Vue 3, which is built on Proxy. The real thing is much smarter: it keeps track of which parts of the page use which properties, so a change only updates the parts that need it.

The older, simpler tools: getters, setters, and `Object.defineProperty`

Long before Proxy, JavaScript already had ways to run code when a property is read or set. They're simpler, and often all you need.

Getters and setters ([chapter 27](#)) run your code for one property you name:

```

class Thermostat {
  #celsius = 20;

  get celsius() {
    return this.#celsius;
  }

  set celsius(value) {
    if (value < 5 || value > 30) {
      throw new RangeError(`Keep it between 5 and 30°C, got ${value}`);
    }
    this.#celsius = value;
  }
}

const home = new Thermostat();
home.celsius = 22;
console.log(home.celsius); // prints: 22
home.celsius = 45;
// RangeError: Keep it between 5 and 30°C, got 45

```

Object.defineProperty sets up one property with special settings, such as making it read-only:

```

const product = { name: "Desk lamp" };

Object.defineProperty(product, "id", {
  value: 1042,
  writable: false, // can't be changed
  enumerable: true, // shows up in loops and console.log
});

product.id = 7; // quietly ignored in a plain .js file
console.log(product.id); // prints: 1042
console.log(product); // prints: { name: 'Desk Lamp', id: 1042 }

```

In strict mode, that change throws instead: `TypeError: Cannot assign to read only property 'id' of object '#<Object>'`.

	Getters, setters, defineProperty	Proxy
Works on	Properties you name in advance	Every property, even ones that don't exist yet
Can react to	Reading and setting	Reading, setting, <code>in</code> , <code>delete</code> , and more
Speed	Fast	A little slower

	Getters, setters, <code>defineProperty</code>	Proxy
Easy to follow?	Yes	Takes more explaining

A good rule: if you know the property names in advance, use a getter and setter (or a class method) first. Use a Proxy when you need to handle *any* property, including ones you can't list ahead of time, like the words in the word counter.

Cautions

Proxies are powerful, and that's exactly why you should use them sparingly:

- **Extra complexity.** Code that looks like a plain property read might be running your trap. Someone reading `member.age = 32` can't tell there's a check behind it. It's even hidden from `console.log`: in Node, logging a proxy prints the target's properties and doesn't mention the proxy at all. Clear names like `createMember` and `readOnly` help.
- **Some speed cost.** Every action on the proxy goes through your trap. That's fine for settings or form data, but keep proxies out of code that runs millions of times in a loop.
- **Some built-in objects don't work through a proxy.** A Map, a Set, a Date, or a class with private `#fields` keeps its data in hidden slots that only the real object can reach. For example, calling `get` on a proxied Map throws `TypeError: Method Map.prototype.get called on incompatible receiver #<Map>`. Stick to plain objects and arrays.

The golden rule: **reach for a Proxy only when simpler tools, like a function or a class with getters and setters, can't do the job.**

Common mistakes

1. Forgetting to return `true` from `set`


```

"use strict";

const settings = new Proxy({}, {
  set(target, property, value) {
    console.log(`Saving ${property}`);
    target[property] = value;
    // forgot: return true;
  },
});

settings.theme = "dark";
// prints: Saving theme
// TypeError: 'set' on proxy: trap returned falsish for property 'theme'

```

A function without `return` returns `undefined`, which counts as "refused". In strict mode, like here or in any module or class, that throws. In a plain `.js` file it's sneakier: no error at all, so the bug only shows up once the code moves into a module. Fix: end the trap with `return Reflect.set(target, property, value, receiver);` and the right answer is returned for you.

2. Changing the target directly, around the proxy

```

const scores = { alice: 10 };

const safeScores = new Proxy(scores, {
  set(target, property, value, receiver) {
    if (value < 0) {
      throw new RangeError("Scores can't be negative");
    }
    return Reflect.set(target, property, value, receiver);
  },
});

scores.alice = -999; // straight to the target: no check at all!
console.log(safeScores.alice); // prints: -999

```

The receptionist can only check visitors who come in through the front door. Anyone who uses `scores` directly walks in the back. Fix: don't keep a separate variable for the target. Create the proxy inside a function, like `createMember`, or pass the object straight in: `new Proxy({ alice: 10 }, handler)`.

3. Reading through the proxy inside its own `get` trap

```
const user = new Proxy({ name: "Sam" }, {
  get(target, property, receiver) {
    return receiver[property]; // oops: reads through the proxy again
  },
});

console.log(user.name);
// RangeError: Maximum call stack size exceeded
```

`receiver` is the proxy itself, so `receiver[property]` calls the `get` trap again, which calls it again, and so on until the call stack runs out ([chapter 17](#)). Fix: read from the target with `Reflect.get(target, property, receiver)`. Pass `receiver` to `Reflect`, but never read properties *from* it inside a trap.

4. Comparing property names with numbers

```
const seats = new Proxy(["A1", "A2", "A3"], {
  get(target, property, receiver) {
    if (property === 0) {
      return `Front row: ${target[0]}`;
    }
    return Reflect.get(target, property, receiver);
  },
});

console.log(seats[0]); // prints: A1
```

The trap never kicked in. Property names reach the trap as strings, even for arrays, and `"0" === 0` is `false` ([chapter 38](#)). Fix: compare with the string `"0"`, and the line prints `Front row: A1`. Or convert with `Number(property)`, after checking that it's a string, as in the negative index example.

Quick recap

- A **Proxy** wraps a **target** object. Its **handler's traps** step in when the object is used: `get` (reading), `set` (writing), `has` (`in`), and `deleteProperty` (`delete`).
- Anything without a trap passes straight through. The proxy shares the target's data: it doesn't copy it.
- In a trap, do your extra work, then call the matching **Reflect** function with the same arguments. `set` and `deleteProperty` must return `true` for success, and `Reflect` does that for you.
- Common uses: validation, default values, logging changes, read-only objects, and reactivity (the idea behind Vue 3).
- Getters, setters, and `Object.defineProperty` are simpler and faster when you know the property names in advance.

- Proxies add complexity and a small speed cost, and don't work with every built-in object. Use one only when simpler tools can't do the job.

Next: try the [exercises](#), then move on to [43 Functional Programming](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
- Run each one with `node ex1.js` .
- In every trap, finish by handing over to the matching `Reflect` function, unless the exercise says otherwise.
- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): Recipe card spy

A cooking app has a recipe card, and you want to see exactly how the app uses it. Start `ex1.js` with this:

```
const recipe = { name: "Pancakes", servings: 4, minutes: 20 };
```

Make a proxy called `trackedRecipe` with two traps:

- `get` prints `Reading "servings"` (with the real property name) before returning the value.
- `set` prints `Changing "servings" from 4 to 8` (with the real name and values) before saving the new value.

Test it with these lines:

```
console.log(trackedRecipe.name);
trackedRecipe.servings = 8;
console.log(`Serves ${trackedRecipe.servings}`);
```

Expected output:

```
Reading "name"
Pancakes
Changing "servings" from 4 to 8
Reading "servings"
Serves 8
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Hotel welcome screen

A hotel's check-in kiosk greets each guest in their own language. It only has a few languages, so anyone else should get the English message. Start `ex2.js` with this:

```
const welcome = {
  en: "Welcome!",
  fr: "Bienvenue!",
  de: "Willkommen!",
  it: "Benvenuto!",
};

const guests = [
  { name: "Chloe", language: "fr" },
  { name: "Hiro", language: "ja" },
  { name: "Lena", language: "de" },
  { name: "Marco", language: "it" },
];
```

1. Make a proxy called `welcomeMessages` that returns the English message for any language that's missing.
2. Loop over `guests` and print a greeting for each one.
3. Finally, print whether `"ja"` is in the proxy.

Expected output:

```
Bienvenue! Enjoy your stay, Chloe.
Welcome! Enjoy your stay, Hiro.
Willkommen! Enjoy your stay, Lena.
Benvenuto! Enjoy your stay, Marco.
Has a Japanese message: false
```

Rule: the loop must not check whether a language exists. It just reads `welcomeMessages[guest.language]`, and the proxy takes care of the rest.

Then add a comment explaining why the last line says `false` , even though reading `welcomeMessages.js` gives you a message.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Guarding the shop's products

In an online shop, several parts of the app can change a product: the admin page, the stock checker, the sale planner. You want one guard that catches bad values, whichever part of the app sends them.

Write a function `createProduct(product)` that returns a proxy with a `set` trap. It checks these three properties, and lets every other property through as normal:

Property	Must be	Otherwise, throw a <code>TypeError</code> with this message
<code>price</code>	a number above 0	<code>price must be a number above 0, got -5 (with the real value)</code>
<code>stock</code>	a whole number, 0 or more	<code>stock must be a whole number, 0 or more, got 2.5 (with the real value)</code>
<code>name</code>	a string that isn't empty, even after trimming	<code>name can't be empty</code>

Test it with this code:

```

const lamp = createProduct({ name: "Desk lamp", price: 24.99, stock: 10 });

lamp.price = 19.99;
lamp.stock = 8;
lamp.color = "black"; // other properties work as normal

const badChanges = [
  ["price", -5],
  ["price", "cheap"],
  ["stock", 2.5],
  ["name", ""],
];

for (const [property, value] of badChanges) {
  try {
    lamp[property] = value;
  } catch (error) {
    console.log(`${error.name}: ${error.message}`);
  }
}

console.log(lamp);

```

Expected output:

```

TypeError: price must be a number above 0, got -5
TypeError: price must be a number above 0, got cheap
TypeError: stock must be a whole number, 0 or more, got 2.5
TypeError: name can't be empty
{ name: 'Desk lamp', price: 19.99, stock: 8, color: 'black' }

```

The last line shows that none of the bad values got through. (Logging a proxy in Node prints the target's properties.)

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Undo button

A drawing app lets you change the brush settings, and it needs an **Undo** button. Every change should be remembered, so that undo can put things back, one step at a time.

Write a function `createUndoable(object)` that returns an array with two things: a proxy for the object, and an `undo` function.

- Every time a property is set through the proxy, remember the property and its *old* value.
- `undo()` puts back the most recent change. If there's nothing left to undo, it prints `Nothing to undo`.

Test it with this code:

```
const [brush, undo] = createUndoable({ color: "black", size: 2 });

brush.color = "red";
brush.size = 5;
console.log(brush.color, brush.size);

undo();
console.log(brush.color, brush.size);

undo();
console.log(brush.color, brush.size);

undo();
```

Expected output:

```
red 5
red 2
black 2
Nothing to undo
```

Rule: undoing must not count as a new change. After two changes, exactly two undos should work.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Smart home dashboard

A smart home app shows the room's temperature and humidity. Different parts of the app want to know when the readings change: the display on the wall, the fan, maybe more later. You'll build a small **store**: a proxy that holds the data and tells everyone who has signed up whenever something changes.

Write a function `createStore(initialState)` that returns an object with two properties:

- `state`: a proxy for the data. When a property is set to a **different** value, it saves it, then calls every listener with `(property, oldValue, newValue)`. Setting the same value again tells nobody.
- `subscribe(listener)`: signs a listener function up for changes. It returns an `unsubscribe` function that signs that listener off again.

Test it with this code:

```
const { state, subscribe } = createStore({ temperature: 21, humidity: 40 });

const stopDisplay = subscribe((property, oldValue, newValue) => {
  console.log(`[Display] ${property}: ${oldValue} -> ${newValue}`);
});

subscribe((property, oldValue, newValue) => {
  if (property === "temperature" && newValue > 25) {
    console.log("[Fan] Too warm, turning on");
  }
});

state.temperature = 23;
state.temperature = 23; // the same value: nobody is told
state.temperature = 27;
stopDisplay();          // the display switches off and unsubscribes
state.humidity = 55;
state.temperature = 28;
console.log(state);
```

Expected output:

```
[Display] temperature: 21 -> 23
[Display] temperature: 23 -> 27
[Fan] Too warm, turning on
[Fan] Too warm, turning on
{ temperature: 28, humidity: 55 }
```

Rule: listeners are called in the order they subscribed. And once a listener unsubscribes, the store must not keep it anywhere. (Remember [chapter 41](#): a list of callbacks that only grows is a leak.)

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

In this chapter, objects changed all the time, and proxies helped you watch, check, and react to every change.

[Chapter 43](#) looks at the opposite idea: code where functions don't change anything at all, and every change makes a *new* value instead. It sounds strange, but it makes programs much easier to test and to trust. 😊

43 Functional Programming

What is it?

Functional programming (FP for short) is a style of writing code. You build your program out of small, predictable functions. Data goes into a function, a new result comes out, and nothing else gets changed along the way.

It isn't a new language or a library. It's a way of using the JavaScript you already know: functions, `map` and `filter`, closures, and spread.

Why does it matter?

Think about the worst bugs you've hit so far. Many were probably "who changed my data?" bugs: an array that got sorted behind your back, or an object that two parts of your program were quietly sharing (remember [chapter 16?](#)).

The FP style prevents a lot of those, and it brings other wins too:

- **Predictable:** a pure function always gives the same answer for the same input. No surprises.
- **Easy to test:** no setup and no hidden state. Call it, check the result. ([Chapter 46](#) will love this.)
- **Easy to reuse:** small functions snap together into bigger ones, like LEGO bricks.
- **Easy to change:** you can add, swap, or remove one step without breaking the others.

You'll see these ideas all over modern JavaScript. React, for example, expects your components to be pure, and it wants you to update data by making copies instead of changing it.

Real-world example

A functional program works like a **sandwich shop's assembly line**.

On the assembly line	In functional programming
Each worker has one job: slice, fill, toast, or wrap	Each function does one small thing
A worker only uses what they're handed, and passes the result on	A function only uses its inputs, and returns a result
Nobody reaches over and changes another worker's sandwich	No side effects: a function doesn't change things outside itself

On the assembly line	In functional programming
Want toasted sandwiches? Add a toasting station. Don't rebuild the line	Add a function to the pipeline. Don't rewrite the others
Every sandwich goes through the same steps, the same way	Same input, same output, every time

Keep this picture in mind. Most of this chapter is about building good workers (pure functions) and lining them up (composition).

How it works

Pure functions

A **pure function** follows two rules:

1. **Same input, same output.** Call it with the same arguments, and you always get the same answer.
2. **No side effects.** It doesn't change anything outside itself. It just returns a value.

A pure function is like a calculator. 4×3 is 12 today, tomorrow, and next year. And pressing `=` doesn't change anything else in the room.

```
function calculateTotal(price, quantity) {
  return price * quantity;
}

console.log(calculateTotal(4, 3)); // prints: 12
console.log(calculateTotal(4, 3)); // prints: 12 (always)
```

Here's an impure one. It changes a variable that lives outside it:

```
let cartTotal = 0;

function addToCart(price) {
  cartTotal += price; // changes a variable outside the function
  return cartTotal;
}

console.log(addToCart(10)); // prints: 10
console.log(addToCart(10)); // prints: 20 (same input, different answer!)
```

To know what `addToCart(10)` will return, you need to know everything that happened before it was called. That's what makes impure code hard to follow.

A function is also impure if it *reads* something that can change, like the clock:

```
function isShopOpen() {  
  const hour = new Date().getHours();  
  return hour >= 9 && hour < 17;  
}
```

Its answer depends on *when* you call it. The fix is to pass the changing thing in as a parameter:

```
function isShopOpen(hour) {  
  return hour >= 9 && hour < 17;  
}  
  
console.log(isShopOpen(10)); // prints: true  
console.log(isShopOpen(20)); // prints: false
```

Now it's pure, and you can check any hour you like without waiting until 8 p.m. The code that calls it can still pass in the real time: `isShopOpen(new Date().getHours())`.

Side effects, and where to put them

A **side effect** is anything a function does besides working out its return value. For example:

- changing a variable outside the function
- changing (mutating) an array or object that was passed in
- printing with `console.log`
- saving to `localStorage`, sending a `fetch` request, changing the page, or starting a timer

But wait: a program with *no* side effects would be useless. It couldn't print, save, or show anything! So FP doesn't ban side effects. It **keeps them at the edges**. Do the thinking in pure functions, and do the side effects in a few clear places, usually at the very end:

```
// Pure: it only calculates
function getOrderSummary(items) {
  const total = items.reduce((sum, item) => sum + item.price, 0);
  return `${items.length} items, ${total.toFixed(2)}`;
}

// The edge: where the result meets the outside world
const cart = [
  { name: "Tea", price: 3.5 },
  { name: "Cake", price: 4.25 },
];
console.log(getOrderSummary(cart)); // prints: 2 items, $7.75
```

`getOrderSummary` can be tested, reused, and trusted. The one side effect, printing, happens outside it.

The sneakiest side effect is changing an argument:

```
function addItem(cart, item) {
  cart.push(item); // changes the caller's array!
  return cart;
}
```

Whoever called `addItem` now has a changed cart, whether they wanted one or not. The pure version builds a new array and leaves the original alone:

```
function addItem(cart, item) {
  return [...cart, item];
}

const myCart = ["tea"];
const newCart = addItem(myCart, "cake");

console.log(myCart); // prints: [ 'tea' ]
console.log(newCart); // prints: [ 'tea', 'cake' ]
```

Immutability: change a copy, not the original

Immutable means "can't be changed". Strings already work this way ([chapter 06](#)): every string method gives you a new string. FP treats arrays and objects the same way. Instead of changing the data, you make a new copy with the change in it.

Think of editing a photo on your phone. A good app keeps the original safe and saves your edit as a new picture. If you don't like the edit, the original is still there.

For objects, spread the old one and override what changed ([chapter 15](#)):

```
const user = { name: "Mia", city: "Kathmandu", plan: "free" };
const movedUser = { ...user, city: "Pokhara" };

console.log(user);           // prints: { name: 'Mia', city: 'Kathmandu', plan: 'free' }
console.log(movedUser);      // prints: { name: 'Mia', city: 'Pokhara', plan: 'free' }
```

For arrays, there's a non-mutating way to do every common job. You met most of these in [chapter 13](#) and [chapter 16](#):

Job	Changes the original	Makes a new array instead
Add an item	<code>list.push(item)</code>	<code>[...list, item]</code>
Remove an item	<code>list.splice(index, 1)</code>	<code>list.filter(...)</code>
Change every item	a loop that assigns	<code>list.map(...)</code>
Sort	<code>list.sort()</code>	<code>list.toSorted()</code>
Reverse	<code>list.reverse()</code>	<code>list.toReversed()</code>
Replace one item	<code>list[index] = value</code>	<code>list.with(index, value)</code>

```
const tasks = ["buy milk", "walk dog", "pay rent"];

console.log(tasks.with(1, "walk cat")); // prints: [ 'buy milk', 'walk cat', 'pay rent' ]
console.log(tasks.toSorted());          // prints: [ 'buy milk', 'pay rent', 'walk dog' ]
console.log(tasks);                    // prints: [ 'buy milk', 'walk dog', 'pay rent' ]
```

The original list never changes, however many versions you make from it.

`Object.freeze` : a lock for your objects

Making copies is a habit. `Object.freeze` is a lock: it stops an object from being changed at all.

```
const settings = Object.freeze({ theme: "dark", fontSize: 16 });

settings.fontSize = 30; // quietly ignored

console.log(settings.fontSize); // prints: 16
```

In a normal `.js` file, the change is ignored without a word. In strict mode ([chapter 26](#)), which includes modules and classes, you get a proper error instead: `TypeError: Cannot assign to read only property 'fontSize' of object '#<Object>'`. The error is better, because silent failures are hard to spot.

Watch out: `Object.freeze` is **shallow**. It locks the top level only. Anything nested inside can still be changed:

```
const game = Object.freeze({ level: 3, inventory: ["sword"] });

game.level = 99;           // ignored
game.inventory.push("shield"); // works!

console.log(game); // prints: { level: 3, inventory: [ 'sword', 'shield' ] }
```

The object is frozen, but the `inventory` array inside it isn't. It's the same "only the top level" story as shallow copies in chapter 16.

Functions as values: higher-order functions

In JavaScript, functions are **first-class**: they're values, just like numbers and strings ([chapter 09](#)). You can store them in variables, pass them into other functions, and return them from functions.

A **higher-order function** is a function that takes a function as an argument, returns a function, or both. You've been using them for a while:

Higher-order function	What it does with functions	Chapter
<code>map</code> , <code>filter</code> , <code>reduce</code>	Takes a callback and calls it for each item	13
<code>setTimeout</code> , <code>addEventListener</code>	Takes a callback and calls it later	30 , 21
<code>createCounter()</code>	Returns a new function	25
<code>once(fn)</code> , <code>debounce(fn, delay)</code>	Takes a function <i>and</i> returns a new one	25 , 34

Higher-order functions are the power tools of FP. Everything in the rest of this chapter, from `pipe` to `memoize`, is one.

Declarative vs. imperative

There are two ways to tell someone how to get to your house. You can give turn-by-turn directions: "Go straight for 200 meters, turn left at the bakery, take the second right..." Or you can give them the address, and let their map app work out the route.

Code is the same. **Imperative** code spells out *how*, step by step. **Declarative** code describes *what* you want. Here are both, adding up the paid orders in a shop:

```
const orders = [
  { id: 1, total: 25, status: "paid" },
  { id: 2, total: 80, status: "refunded" },
  { id: 3, total: 40, status: "paid" },
];
```

Imperative (turn-by-turn directions):

```
let paidTotal = 0;

for (const order of orders) {
  if (order.status === "paid") {
    paidTotal += order.total;
  }
}

console.log(paidTotal); // prints: 65
```

Declarative (the address):

```
const paidTotal = orders
  .filter((order) => order.status === "paid")
  .reduce((sum, order) => sum + order.total, 0);

console.log(paidTotal); // prints: 65
```

The declarative version reads almost like the task itself: "keep the paid orders, then add up their totals". Nothing is reassigned along the way, and each step is a small function you could reuse.

Neither one is "wrong". A `for` loop is still the right tool when you need to stop early, or when a chain of methods would be harder to read.

Composition: building an assembly line with `pipe`

Composition means building a bigger function out of smaller ones, where each one's output becomes the next one's input. It's the assembly line from the start of this chapter.

A blog needs a **slug** for every post: the tidy, lowercase part of a web address, like `example.com/blog/summer-sale-50-off`. Here's a helper called `pipe` that lines up any number of steps:

```
function pipe(...steps) {
  return (value) => steps.reduce((result, step) => step(result), value);
}
```

And here's the slug maker, built from four small steps:

```
function removeSymbols(text) {
  return text.replace(/[^\a-z0-9 ]/g, ""); // keep letters, digits and spaces
}

const makeSlug = pipe(
  (text) => text.trim(),
  (text) => text.toLowerCase(),
  removeSymbols,
  (text) => text.replaceAll(" ", "-"),
);

console.log(makeSlug(" Summer Sale: 50% Off! ")); // prints: summer-sale-50-off
```

Here's the title moving down the line, one station at a time:

Station	Value after it
(start)	" Summer Sale: 50% Off! "
trim	"Summer Sale: 50% Off!"
lowercase	"summer sale: 50% off!"
removeSymbols	"summer sale 50 off"
spaces to dashes	"summer-sale-50-off"

How does `pipe` do it? It's `reduce` from chapter 13. The starting value goes into the first step. Whatever comes out becomes `result`, which goes into the next step, and so on. The last result is the answer.

Why not just chain string methods? Chaining only works with methods the value already has. `removeSymbols` is your own function (with a regular expression from [chapter 37](#)), so you can't chain it, but it slots into a pipe like any other step. And `makeSlug` is now a single, named, reusable function. You can call it anywhere, or pass it to `map` to make slugs for a whole list of titles.

You'll also meet `compose`. It does the same job, but runs the steps from right to left, the way math writes $f(g(x))$. So `compose(c, b, a)` gives the same function as `pipe(a, b, c)`:


```
function compose(...steps) {
  return (value) => steps.reduceRight((result, step) => step(result), value);
}
```

`reduceRight` is `reduce` starting from the end. This course uses `pipe`, because it reads in the order things actually happen.

Partial application: pre-set some arguments

Lots of coffee machines have a "favorites" button. You set up your usual order once (large, oat milk, extra shot), and after that, one press does the rest.

Partial application is the same idea for functions: you fill in some of the arguments now, and get back a function that only needs the rest. Here's a price formatter for a shop that sells in several currencies:

```
function formatPrice(currency, amount) {
  return new Intl.NumberFormat("en-US", { style: "currency", currency }).format(amount);
}

console.log(formatPrice("USD", 1234.5)); // prints: $1,234.50
console.log(formatPrice("GBP", 1234.5)); // prints: £1,234.50
```

The US website always passes `"USD"`. So pre-set it, once:

```
function formatDollars(amount) {
  return formatPrice("USD", amount);
}

console.log(formatDollars(19.99)); // prints: $19.99
```

That's partial application by hand: a small function that fills in the first argument and passes the rest along. `bind` from [chapter 26](#) can do it in one line. Its first argument sets `this`, which we don't need here, so we pass `null`. The arguments after that get pre-filled:

```
const formatEuros = formatPrice.bind(null, "EUR");

console.log(formatEuros(1234.5)); // prints: €1,234.50
```

Currying: one argument at a time

Currying means writing a function so it takes its arguments one at a time. Each call takes one argument and returns a new function that waits for the next one:

```
function applyDiscount(percent) {
  return (price) => price - (price * percent) / 100;
}

const blackFriday = applyDiscount(30); // pre-set: 30% off
const staffDiscount = applyDiscount(15); // pre-set: 15% off

console.log(blackFriday(200)); // prints: 140
console.log(staffDiscount(200)); // prints: 170
console.log(applyDiscount(10)(50)); // prints: 45 (pre-set and use in one go)
```

`applyDiscount(30)` doesn't work anything out yet. It returns a function that remembers `percent` in its backpack (a closure, [chapter 25](#)) and waits for a price.

In FP code, you'll often see curried functions written as a chain of arrows. It means exactly the same thing:

```
const applyDiscount = (percent) => (price) => price - (price * percent) / 100;
```

(Pick one style per function. You can't have both in one file, because they share a name.)

Why bother? Because a pre-set function is exactly the shape that `map` and `filter` want: one argument in, one result out.

```
const prices = [100, 50, 20];
console.log(prices.map(blackFriday)); // prints: [ 70, 35, 14 ]

function isCheaperThan(limit) {
  return (product) => product.price < limit;
}

const products = [
  { name: "Mug", price: 12 },
  { name: "Lamp", price: 45 },
  { name: "Notebook", price: 8 },
];

console.log(products.filter(isCheaperThan(20)));
// prints: [ { name: 'Mug', price: 12 }, { name: 'Notebook', price: 8 } ]
```

`products.filter(isCheaperThan(20))` reads almost like English. And `isCheaperThan(50)` or `isCheaperThan(100)` are one small change away.

Currying or partial application? They're close cousins. *Currying* is a way of writing a function (one argument at a time). *Partial application* is pre-filling some arguments, however you do it: with a curried function, a small

wrapper, or `bind`. Libraries like Ramda and lodash even have a `curry` helper that turns a normal function into a curried one. You don't need them to use the idea.

Memoization: remembering results

Your teacher asks you what 37×43 is. You work it out on paper (1591) and write the answer on a sticky note. The next time someone asks, you don't work it out again. You just read the note.

Memoization gives a function those sticky notes. The first time it's asked a question, it does the real work and saves the answer in a **cache** (a store of saved answers). When it's asked the same question again, it answers straight from the cache.

Here's a `memoize` helper. It takes any one-argument function and returns a new version with a cache:

```
function memoize(fn) {
  const cache = new Map();

  return (input) => {
    if (cache.has(input)) {
      return cache.get(input); // asked before: read the sticky note
    }
    const result = fn(input);
    cache.set(input, result); // new question: write a sticky note
    return result;
  };
}
```

The cache is a `Map` ([chapter 35](#)), and it lives in the returned function's backpack, so it survives between calls. Let's try it on a delivery app that works out how many days a parcel needs:

```
function getDeliveryDays(distanceKm) {
  console.log(`Working out delivery for ${distanceKm} km...`);
  return Math.ceil(distanceKm / 300) + 1;
}

const getDeliveryDaysFast = memoize(getDeliveryDays);

console.log(getDeliveryDaysFast(1200));
console.log(getDeliveryDaysFast(1200));
console.log(getDeliveryDaysFast(450));
```

You'll see:

```
Working out delivery for 1200 km...
5
5
Working out delivery for 450 km...
3
```

The second question about 1200 km was answered from the cache, so the real function didn't run at all. (The `console.log` inside is only there so you can see when the real work happens. Strictly speaking, it's a side effect, and notice that the cached call skipped it!)

Here, the real work is tiny. Memoization pays off when a function is slow and keeps getting the same questions, like the recursive example in [chapter 48](#), where it turns seconds into less than a millisecond.

Memoization comes with two rules:

1. **Only memoize pure functions.** If the answer can change (because the function reads the clock, a random number, or a price list that gets updated), the cache will happily hand out old answers. Common mistake 3 below shows it happening.
2. **Don't let the cache grow forever.** Every new question adds a sticky note. In an app that runs for days, that's a memory leak ([chapter 41](#)). Limit its size, or clear it now and then.

This `memoize` only works for functions with one argument, because the cache looks only at `input`. For several arguments, you need a key made from all of them, like ``${weightKg}-${zone}``. You'll build that version in the exercises.

FP in everyday JavaScript: it's not all-or-nothing

You don't have to write "100% functional" code to get the benefits. Most professional JavaScript mixes styles. These are the habits worth keeping:

- **Write calculations as pure functions.** Pass in everything they need, and return a result.
- **Push side effects to the edges.** Fetch, print, save, and change the page in a few clear places.
- **Don't change what you were handed.** Return new arrays and objects instead (`spread`, `map`, `filter`, `toSorted`, `with`).
- **Build big things from small, well-named functions.** Line them up with a `filter / map` chain or a `pipe`.
- **Use a loop when it's clearer.** FP is a toolbox, not a set of strict rules.

One warning: don't go overboard. Five layers of curried arrows, a 15-step pipe, or a clever one-liner can be much harder to read than a plain loop. If a teammate (or you, six months from now) can't follow it, simplify it. That's KISS again, and [chapter 45](#) has lots more on writing code people enjoy reading.

Common mistakes

1. Changing the originals inside `map`

```
const products = [
  { name: "Lamp", price: 40 },
  { name: "Rug", price: 90 },
];

const halfPrice = products.map((product) => {
  product.price = product.price / 2; // changes the original object!
  return product;
});

console.log(halfPrice[0].price); // prints: 20
console.log(products[0].price); // prints: 20 (the original changed too!)
```

`map` makes a new array, but the objects inside it are the same objects as before. Changing `product.price` changes them for everyone. Fix: return a new object instead: `products.map((product) => ({ ...product, price: product.price / 2 })).` (The extra parentheses around `{ ... }` tell JavaScript you're returning an object, not starting a function body.)

2. Giving a curried function all its arguments at once

```
function applyDiscount(percent) {
  return (price) => price - (price * percent) / 100;
}

console.log(applyDiscount(30, 200)); // prints: [Function (anonymous)]
```

`applyDiscount` only takes one argument. The `200` is ignored, and you get back the waiting function instead of a price. Fix: one pair of parentheses per argument: `applyDiscount(30)(200)` gives `140`.

3. Memoizing a function that isn't pure

```

let pricePerKg = 2;

function getShippingCost(weightKg) {
  return weightKg * pricePerKg; // reads a variable that can change
}

const getShippingCostFast = memoize(getShippingCost); // memoize from earlier in this chapter

console.log(getShippingCostFast(10)); // prints: 20
pricePerKg = 3; // the courier raises its prices
console.log(getShippingCostFast(10)); // prints: 20 (should be 30!)

```

The cache only looks at the weight. It has no idea that the price per kilo changed, so it keeps handing out the old answer. Fix: make the function pure by passing everything it depends on as arguments, `getShippingCost(weightKg, pricePerKg)`, and build the cache key from both.

4. A pipeline step that forgets to return

```

// pipe from earlier in this chapter
const makeTag = pipe(
  (text) => text.trim(),
  (text) => { text.toLowerCase(); }, // oops: curly braces, no return
  (text) => `#${text.replaceAll(" ", "")}`,
);

console.log(makeTag(" Summer Vibes "));
// TypeError: Cannot read properties of undefined (reading 'replaceAll')

```

The second step returns `undefined`, so the third step receives `undefined` and crashes. Notice that the error shows up one step *after* the real bug. When a pipe breaks, check the step before the one that crashed. Fix: drop the curly braces, `(text) => text.toLowerCase()`, and you get `#summervibes`.

Quick recap

- **Functional programming** builds programs from small, predictable functions that take input and return new output.
- A **pure function** gives the same output for the same input and has no side effects. Keep side effects (printing, saving, fetching, changing the page) at the edges of your program.
- **Immutability**: don't change data, make changed copies (`spread`, `map`, `filter`, `toSorted`, `toReversed`, `with`). `Object.freeze` locks an object, but only the top level.
- **Higher-order functions** take or return functions. `pipe` composes small functions into an assembly line.
- **Partial application** pre-fills some arguments. **Currying** takes arguments one at a time, which fits `map` and `filter` nicely.

- **Memoization** caches answers. Only memoize pure functions, and keep the cache from growing forever.
 - FP isn't all-or-nothing. Use the habits that make your code clearer, and keep loops where they read better.
-

Next: try the [exercises](#), then move on to [44 Design Patterns](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` from a terminal opened in this folder.
 - You may copy `pipe` and `memoize` from the notes when an exercise needs them, but make sure you understand every line you copy.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Pure or impure?

You're reviewing code for a banking app. Copy these functions into `ex1.js` :

```

function addInterest(balance, ratePercent) {
  return balance + (balance * ratePercent) / 100;
}

let totalDeposits = 0;
function deposit(amount) {
  totalDeposits += amount;
  return totalDeposits;
}

function formatBalance(balance) {
  return `$$${balance.toFixed(2)}`;
}

function logTransaction(amount) {
  console.log(`Transaction: $$${amount}`);
}

function isOverdrawn(account) {
  return account.balance < 0;
}

function chargeFee(account) {
  account.balance -= 5;
  return account;
}

function createStatementId() {
  return `ST-${Date.now()}`;
}

```

Part 1: above each function, write a comment saying whether it's **pure** or **impure**, and why (one short sentence).

Part 2: rewrite `chargeFee` so it's pure: it must return a new account and leave the original alone.

Part 3: rewrite `createStatementId` so it's pure. (Hint: what could you pass in instead?)

Then test your new versions:

```

const account = { owner: "Mia", balance: 100 };
const charged = chargeFee(account);

console.log(account);
console.log(charged);
console.log(createStatementId(1767225600000));

```

Expected output:


```
{ owner: 'Mia', balance: 100 }  
{ owner: 'Mia', balance: 95 }  
ST-1767225600000
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): A frozen playlist

Your music app keeps the playlist frozen, so nothing can change it by accident:

```
const playlist = Object.freeze(["Levitating", "Anti-Hero", "Flowers", "Blinding Lights"]);
```

Make five new versions of it, each in its own variable, and print them. Then print the original:

1. sorted A to Z
2. in reverse order
3. with "Flowers" (at index 2) replaced by "Espresso"
4. with "Houdini" added at the end
5. without "Anti-Hero"

Expected output:

```
[ 'Anti-Hero', 'Blinding Lights', 'Flowers', 'Levitating' ]  
[ 'Blinding Lights', 'Flowers', 'Anti-Hero', 'Levitating' ]  
[ 'Levitating', 'Anti-Hero', 'Espresso', 'Blinding Lights' ]  
[ 'Levitating', 'Anti-Hero', 'Flowers', 'Blinding Lights', 'Houdini' ]  
[ 'Levitating', 'Flowers', 'Blinding Lights' ]  
[ 'Levitating', 'Anti-Hero', 'Flowers', 'Blinding Lights' ]
```

The frozen playlist is your safety net. If you use a method that changes the array, like `sort()` or `push()`, you'll get a `TypeError`, even in a normal `.js` file. (A plain assignment like `playlist[0] = "x"` is different: it's quietly ignored, like in the notes.)

Stuck? The hint is in the [Hints section](#) at the end.

Bonus: on purpose, call `playlist.sort()` and read the error message. What is it telling you?

Exercise 3 (Medium): A username pipeline

A game's sign-up page turns whatever people type into a clean username. Use `pipe` to build a function `makeUsername` from small steps, in this order:

1. Remove spaces from both ends.
2. Make it lowercase.
3. Turn every remaining space into an underscore `_`.
4. Remove every character that isn't a letter, a digit, or an underscore.
5. Cut it to at most 15 characters.
6. Put an `@` in front.

Test it:

```
console.log(makeUsername(" Sandip Shakya "));  
console.log(makeUsername("Captain Awesome!!!"));  
console.log(makeUsername("The Real Slim Shady 2026"));  
console.log(makeUsername(" GAMER-GIRL_99 "));
```

Expected output:

```
@sandip_shakya  
@captain_awesome  
@the_real_slim_s  
@gamergirl_99
```

Rule: step 5 must use a curried helper, `limitLength(maxLength)`, so the step inside the pipe reads `limitLength(15)`.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Memoize with several arguments

A courier app works out shipping prices. In real life, this calculation is slow (it checks lots of routes), so you want to cache it. Here's the function, with a counter that shows how often the real work happens:

```
let calculations = 0;

function calculateShipping(weightKg, service) {
  calculations++; // only here so we can count the real work
  const ratePerKg = service === "express" ? 4 : 2;
  return weightKg * ratePerKg + 5;
}
```

The `memoize` from the notes only looks at one argument. Write a new version of `memoize` that works with **any number of arguments**. Then test it:

```
const getShipping = memoize(calculateShipping);

console.log(getShipping(3, "standard"));
console.log(getShipping(3, "express"));
console.log(getShipping(3, "standard"));
console.log(getShipping(10, "standard"));
console.log(`Real calculations: ${calculations}`);
```

Expected output:

```
11
17
11
25
Real calculations: 3
```

The second `getShipping(3, "standard")` comes from the cache, so there are only 3 real calculations. And `getShipping(3, "express")` must *not* reuse the standard price, even though the weight is the same.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: give `memoize` a second parameter, `maxSize`. When the cache is full and a new answer arrives, delete the oldest entry first. A `Map` remembers the order things were added, and `cache.keys().next().value` gives you the oldest key ([chapter 36](#)). With `memoize(calculateShipping, 2)`, calling `(1, "standard")`, `(2, "standard")`, `(3, "standard")`, `(2, "standard")`, and then `(1, "standard")` should make 4 real calculations.

Exercise 5 (Challenge): Refactor a report that changes its own data

A coffee shop prints a report of its coffee sales, with a 10% member discount. Here's the current code. Copy it into `ex5.js` and run it:

```

const sales = [
  { product: "Coffee beans", category: "coffee", price: 12, quantity: 3 },
  { product: "Green tea", category: "tea", price: 6, quantity: 5 },
  { product: "Espresso cups", category: "kitchen", price: 18, quantity: 1 },
  { product: "Cold brew kit", category: "coffee", price: 30, quantity: 2 },
  { product: "Chai mix", category: "tea", price: 8, quantity: 4 },
];

function printCoffeeReport(sales) {
  sales.sort((a, b) => b.price * b.quantity - a.price * a.quantity);
  let total = 0;
  console.log("Coffee sales (with 10% member discount):");
  for (let i = 0; i < sales.length; i++) {
    if (sales[i].category === "coffee") {
      sales[i].price = sales[i].price * 0.9;
      const lineTotal = sales[i].price * sales[i].quantity;
      total += lineTotal;
      console.log(` - ${sales[i].product}: ${lineTotal.toFixed(2)}`);
    }
  }
  console.log(`Total: ${total.toFixed(2)}`);
}

printCoffeeReport(sales);
printCoffeeReport(sales); // the same report again... right?

```

The first report is right. The second one isn't: it shows \$48.60, \$29.16, and a total of \$77.76. Every time the report runs, the prices shrink by another 10%, and the order of the `sales` array changes too. The report is changing its own data!

Rewrite it in a functional style:

- Write a **pure** function `buildCoffeeReport(sales)` that **returns** the whole report as one string. It must not print anything, and it must not change `sales` or anything in it.
- Build it from small functions. Use at least two **curried** helpers: `isInCategory(category)` for `filter`, and `withDiscount(percent)` for `map`.
- Keep the side effect (printing) at the edge, outside your pure functions.

Replace the last two lines with these, and run it:

```

const snapshot = JSON.stringify(sales);

console.log(buildCoffeeReport(sales));
console.log(buildCoffeeReport(sales));
console.log(`Original data unchanged: ${JSON.stringify(sales) === snapshot}`);

```

Expected output:

```
Coffee sales (with 10% member discount):  
- Cold brew kit: $54.00  
- Coffee beans: $32.40  
Total: $86.40  
Coffee sales (with 10% member discount):  
- Cold brew kit: $54.00  
- Coffee beans: $32.40  
Total: $86.40  
Original data unchanged: true
```

(`JSON.stringify` turns the whole array into one string, so comparing the before and after strings is a quick way to check that nothing inside changed. See [chapter 23](#).)

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

Look back at `memoize`. It took a function and gave it a new ability, a cache, without changing the original function at all. It's like gift wrapping: the gift inside stays the same, but now it has something extra around it.

That trick is so useful that it has a name. It's one of the proven solutions you'll meet in [chapter 44: Design Patterns](#).

44 Design Patterns

What is it?

A **design pattern** is a named, proven solution to a problem that keeps coming up in programming.

It isn't code you copy and paste. It's an idea, a shape for your code, that you adapt to your own program. Developers have met the same problems again and again, and these are the answers that kept working.

Why does it matter?

Three reasons:

- **You don't reinvent the wheel.** When you hit a familiar problem, like "five parts of my app need to know when an order is placed", there's already a good answer waiting.
- **You get a shared vocabulary.** Saying "let's use an observer here" tells another developer a whole paragraph's worth of design in three words.
- **You'll recognize them everywhere.** Libraries, frameworks, and Node itself are full of patterns. Once you know the names, other people's code gets much easier to read.

Here's the good news: you've already used several of them without knowing their names. `addEventListener` from [chapter 21](#) is a pattern. So is your `debounce` from [chapter 34](#). This chapter gives those ideas names, and shows you a few new ones.

Real-world example

Think of a cookbook.

A recipe doesn't describe *your* dinner party. It describes a way of cooking that works, and you adjust it: more salt, no onions, double the amount. Chefs also use recipe names as shorthand. One chef says "make a roux" (a flour-and-butter base for sauces), and the other knows exactly what to do, without a long explanation.

Design patterns are the recipes of programming. Here are the ones in this chapter, with an everyday version of each:

Pattern	Everyday version	The problem it solves
Module	A vending machine: buttons on the outside, machinery hidden inside	Hide the details, show only a small set of controls

Pattern	Everyday version	The problem it solves
Factory	A pizza counter: you say "margherita", the kitchen knows how to make it	Create objects without the caller knowing every detail
Singleton	The one thermostat in a house that everybody shares	There must be exactly one of something
Observer	The subscribe button and bell on a video channel	Tell many listeners when something happens
Strategy	Picking standard or express delivery at checkout	Swap one way of doing a job for another
Decorator	Gift wrapping: the same gift, with an extra layer around it	Add behavior without changing the original
Adapter	A travel plug adapter	Make two things fit that have different shapes

Many of these names come from a famous 1994 book, *Design Patterns*, written by four authors nicknamed the "Gang of Four". Their examples used languages like C++. In JavaScript, functions and closures make many patterns much shorter, as you're about to see.

How it works

Each pattern below follows the same order: the problem first, then the pattern that solves it.

Module: hide the details

The problem: if your data sits out in the open, any line of code anywhere can change it, by accident or on purpose.

The pattern: keep the data private, and hand out only a small set of functions to work with it. That small set is called the **public interface**: the buttons on the vending machine.

Before ES modules existed, people built this with a closure ([chapter 25](#)). You'll still see it in older code:

```

const scoreboard = (function () {
  const scores = []; // private: only the functions below can see it

  function add(player, points) {
    scores.push({ player, points });
  }

  function best() {
    return scores.toSorted((a, b) => b.points - a.points)[0];
  }

  return { add, best }; // the public part
})();

scoreboard.add("Mia", 420);
scoreboard.add("Raj", 610);
console.log(scoreboard.best()); // prints: { player: 'Raj', points: 610 }
console.log(scoreboard.scores); // prints: undefined

```

The strange `(function () { ... })()` shape is an **IIFE** (Immediately Invoked Function Expression): a function that's created and called straight away. It runs once, and `scoreboard` keeps the object it returns. `scores` lives on inside the closure, where nothing outside can reach it.

Today, you get the same thing for free with ES modules ([chapter 29](#)). A module file is its own private room: anything you don't `export` stays inside.

```

// scoreboard.js
const scores = []; // not exported, so it stays private

export function addScore(player, points) {
  scores.push({ player, points });
}

export function bestScore() {
  return scores.toSorted((a, b) => b.points - a.points)[0];
}

```

```

// main.js
import { addScore, bestScore } from "./scoreboard.js";

addScore("Mia", 420);
addScore("Raj", 610);
console.log(bestScore()); // prints: { player: 'Raj', points: 610 }

```


(Remember: for `import` and `export` in Node, the folder needs a `package.json` containing `{ "type": "module" }`.)

If `main.js` tries `import { scores } from "../scoreboard.js";`, Node refuses with a `SyntaxError`: the module "does not provide an export named 'scores'". For new code, use ES modules. Knowing the IIFE version helps you read older code.

Factory: let a function build the objects

The problem: creating an object sometimes needs a lot of know-how. In a game, every place that creates a character would need to remember that a mage has 80 health and a warrior has 150. If those numbers change, you have to find every one of those places.

The pattern: a **factory** is a function whose job is to create objects. You tell it *what* you want, and it handles *how*. That's the pizza counter: you say "margherita", and the kitchen knows the recipe.

```
const characterTypes = {
  warrior: { health: 150, attack: 12 },
  mage: { health: 80, attack: 20 },
  archer: { health: 100, attack: 15 },
};

function createCharacter(type, name) {
  const stats = characterTypes[type];
  if (!stats) {
    throw new Error(`Unknown character type: ${type}`);
  }
  return { name, type, ...stats, level: 1 };
}

const aria = createCharacter("mage", "Aria");
console.log(aria);
// prints: { name: 'Aria', type: 'mage', health: 80, attack: 20, level: 1 }
```

Now the stats live in one place. Change a number there, and every new mage gets it.

A factory can also decide *which kind* of object to make. Here, some customers want text messages and some want emails:

```
function createNotifier(user) {
  if (user.prefers === "sms") {
    return { send: (text) => console.log(`SMS to ${user.phone}: ${text}`) };
  }
  return { send: (text) => console.log(`Email to ${user.email}: ${text}`) };
}

const customers = [
  { name: "Lena", prefers: "email", email: "lena@example.com" },
  { name: "Tom", prefers: "sms", phone: "555-0142" },
];

for (const customer of customers) {
  const notifier = createNotifier(customer);
  notifier.send("Your parcel is on its way!");
}
```

You'll see:

```
Email to lena@example.com: Your parcel is on its way!
SMS to 555-0142: Your parcel is on its way!
```

Look at the loop. It never asks "is this an SMS or an email?" It just calls `send`. Only the factory knows the difference, so adding push notifications next month means changing the factory, not the loop. (A factory can also return instances of different classes from [chapter 27](#). The caller still doesn't need to know.)

Singleton: exactly one of something

The problem: some things should exist exactly once in an app, like the settings or a logger. If two parts of the app each made their own settings object, one could say "dark mode" while the other says "light mode".

The pattern: a **singleton** is an object that has only one instance, shared by everyone, like the single thermostat in a house.

In JavaScript, the easiest singleton is an object exported from a module. Remember from chapter 29 that a module runs only once, and every file that imports it gets the same copy:

```
// settings.js
export const settings = {
  theme: "light",
  language: "en",
};
```

```
// header.js
import { settings } from "../settings.js";

export function turnOnDarkMode() {
  settings.theme = "dark";
}
```

```
// main.js
import { settings } from "../settings.js";
import { turnOnDarkMode } from "../header.js";

turnOnDarkMode();
console.log(settings.theme); // prints: dark
```

`header.js` and `main.js` both imported `settings`, and they got the very same object, so a change made in one file shows up in the other. (In languages like Java, you'll see singletons built with a special `getInstance()` method. In JavaScript, a module does the job with no extra code.)

Watch out: a singleton is shared state that the whole app can change, which makes it a global variable in disguise. When something goes wrong, *any* file could be the culprit. Tests get harder too, because one test's changes leak into the next ([chapter 46](#)). Keep singletons for things that truly are one-of-a-kind, and prefer passing values in as parameters. For settings that shouldn't change, `Object.freeze` from [chapter 43](#) helps.

Observer: tell everyone who's interested

The problem: when a customer places an order in an online shop, lots of things should happen: send a thank-you email, pack the items, add loyalty points. The obvious way is for `placeOrder` to call `sendEmail(order)`, `packItems(order)`, `addPoints(order)`, and so on. But then `placeOrder` has to know about every other feature in the shop, and each new idea means editing it again.

The pattern: with the **observer** pattern, the order code only announces "an order was placed!", and anyone who cares signs up to hear about it. It's the subscribe button and bell on a video channel: the creator uploads once, every subscriber gets notified, and the creator doesn't need to know who they are.

You've used this already: `button.addEventListener("click", handleClick)` in chapter 21. The button announces clicks, and your function is an **observer** (also called a **listener**): a function waiting to be told that something happened.

Let's build a tiny version ourselves. It's called an **emitter**, because it emits (sends out) events:

```

class Emitter {
  #listeners = new Map(); // event name -> array of listener functions

  on(eventName, listener) {
    if (!this.#listeners.has(eventName)) {
      this.#listeners.set(eventName, []);
    }
    this.#listeners.get(eventName).push(listener);
  }

  off(eventName, listener) {
    const list = this.#listeners.get(eventName) ?? [];
    this.#listeners.set(eventName, list.filter((fn) => fn !== listener));
  }

  emit(eventName, data) {
    const list = this.#listeners.get(eventName) ?? [];
    for (const listener of list) {
      listener(data);
    }
  }
}

```

Three methods, three jobs: `on` subscribes ("call this function when that event happens"), `off` unsubscribes, and `emit` announces an event by calling every subscribed function with the data. The `Map` from [chapter 35](#) keeps one list per event name, and the private `#listeners` field ([chapter 27](#)) keeps those lists safe from outside code.

Now the shop can announce orders without knowing who's listening:

```

const shop = new Emitter();

shop.on("orderPlaced", (order) => console.log(`Email: thanks for order #${order.id}`));
shop.on("orderPlaced", (order) => console.log(`Stock: pack ${order.items.join(" and ")}`));

function addPoints(order) {
  console.log(`Points: +${Math.floor(order.total)}`);
}
shop.on("orderPlaced", addPoints);

shop.emit("orderPlaced", { id: 101, items: ["mug", "tea"], total: 23.5 });
shop.off("orderPlaced", addPoints); // the points promotion is over
shop.emit("orderPlaced", { id: 102, items: ["kettle"], total: 40 });

```

You'll see:

```
Email: thanks for order #101
Stock: pack mug and tea
Points: +23
Email: thanks for order #102
Stock: pack kettle
```

The second order gets no points, because `addPoints` unsubscribed. Just like `removeEventListener`, `off` needs the *same function*, which is why `addPoints` has a name. And adding a new feature is now one more `on` call. The code that emits `"orderPlaced"` never changes.

You'll also hear the name **pub-sub** (publish-subscribe). It's the same idea with a middleman, like a notice board: publishers pin up messages under a topic, subscribers watch the topics they care about, and neither side knows the other. One `Emitter` shared by the whole app can be that notice board. Many developers use the two names loosely.

Tip: Node has a fuller, built-in version of this class called `EventEmitter`, with the same `on`, `off`, and `emit` methods. You'll use it in [chapter 49](#).

Strategy: swap how a job gets done

The problem: a shop offers several shipping methods, and each one is priced differently. The first version usually looks like this:

```
function shippingCost(method, weightKg) {
  if (method === "standard") {
    return 5;
  } else if (method === "express") {
    return 10 + weightKg * 2;
  } else if (method === "overnight") {
    return 25 + weightKg * 3;
  } else if (method === "pickup") {
    return 0;
  } else {
    throw new Error(`Unknown shipping method: ${method}`);
  }
}
```

It works, but it grows with every new method, and all the pricing rules are tangled together in one function.

The pattern: a **strategy** is one way of doing a job, packed into its own function. Keep all the strategies in an object, and pick one by name:

```

const shippingMethods = {
  standard: () => 5,
  express: (weightKg) => 10 + weightKg * 2,
  overnight: (weightKg) => 25 + weightKg * 3,
  pickup: () => 0,
};

function shippingCost(method, weightKg) {
  const calculate = shippingMethods[method];
  if (!calculate) {
    throw new Error(`Unknown shipping method: ${method}`);
  }
  return calculate(weightKg);
}

console.log(shippingCost("express", 3)); // prints: 16
console.log(shippingCost("pickup", 3)); // prints: 0

```

Each rule is now small and sits on its own line. A new method, like drone delivery, is one new line in the object, and `shippingCost` doesn't change at all. The options are data now, too: `Object.keys(shippingMethods)` gives you `[ 'standard', 'express', 'overnight', 'pickup' ]`, ready to show in a menu.

You've used strategies before. `toSorted` in [chapter 13](#) doesn't know how you want things ordered. You hand it a compare function, and that function is the strategy:

```

const products = [
  { name: "Lamp", price: 35, rating: 4.1 },
  { name: "Desk", price: 120, rating: 4.7 },
  { name: "Chair", price: 60, rating: 3.9 },
];

const cheapestFirst = (a, b) => a.price - b.price;
const bestRatedFirst = (a, b) => b.rating - a.rating;

console.log(products.toSorted(cheapestFirst).map((p) => p.name)); // prints: [ 'Lamp', 'Chair', 'Desk' ]
console.log(products.toSorted(bestRatedFirst).map((p) => p.name)); // prints: [ 'Desk', 'Lamp', 'Chair' ]

```

Same `toSorted`, different strategy, different result.

Decorator: wrap a function to add behavior

The problem: you want some extra behavior around a function, like logging every call, without changing the function itself. Maybe it's someone else's code, or you want the same extra behavior on ten different functions.

The pattern: a **decorator** is a function that takes a function and returns a new function. The new one does something extra and calls the original inside. It's gift wrapping: the gift inside doesn't change, but now it has a layer around it.

```
function withLogging(fn) {
  return function (...args) {
    console.log(`Calling ${fn.name}(${args.join(", ")} )`);
    const result = fn(...args);
    console.log(`${fn.name} returned ${result}`);
    return result;
  };
}

function addTax(price, rate) {
  return price + price * rate;
}

const addTaxLogged = withLogging(addTax);
const total = addTaxLogged(40, 0.25);
console.log(total);
```

You'll see:

```
Calling addTax(40, 0.25)
addTax returned 50
50
```

A few things to notice:

- Every function has a `name` property that holds its name, so `fn.name` is `"addTax"`.
- `...args` collects whatever arguments come in, and `fn(...args)` passes them all on ([chapter 15](#)).
- The wrapper **returns the original's result**, so the caller gets the same answer as before, plus the extra behavior. And `addTax` itself didn't change at all.

Here's a surprise: you've already built several decorators.

You built it in...	Decorator	The extra behavior it adds
Chapter 25	<code>once(fn)</code>	Runs the function only the first time
Chapter 34	<code>debounce(fn, delay)</code>	Waits until the calls calm down
Chapter 43	<code>memoize(fn)</code>	Remembers answers it has already worked out

Because a decorator returns a normal function, you can wrap a gift twice. `once(withLogging(chargeCard))` gives you a payment function that logs its first call and ignores every call after that.

Tip: if you decorate an object's method, call the original with `fn.apply(this, args)` instead of `fn(...args)`, so `this` still points at the object. It's the same trick your `debounce` used in chapter 34.

You may also see `@something` written above a class in TypeScript or Angular code. That's a newer decorator *syntax*. Plain JavaScript in Node 24 doesn't support it yet (you'd get a `SyntaxError`), but the idea is the same: wrap something to add behavior.

Adapter: make mismatched shapes fit

The problem: your code expects data in one shape, but the outside world hands it over in another. Weather services are a good example. Each one names its fields differently, and none of them match what your display code wants.

The pattern: an **adapter** is a small piece of code that sits in between and converts one shape into the other. It's a travel plug adapter: your laptop charger stays the same, the wall socket stays the same, and the adapter makes them fit.

```
// Your display code expects: { city, tempC, windKph }
function showWeather(weather) {
  console.log(`${weather.city}: ${weather.tempC}°C, wind ${weather.windKph} km/h`);
}

// Open-Meteo's reply, trimmed (you met it in chapter 39)
const openMeteoReply = {
  current: { temperature_2m: 15.4, wind_speed_10m: 7.6 },
};

// A made-up second provider: the same facts, a different shape
const otherReply = {
  place: "Paris",
  conditions: { tempC: 18.1, wind: { kph: 11.2 } },
};
```

One adapter per provider turns its reply into your shape:


```
function adaptOpenMeteo(reply, city) {
  return {
    city,
    tempC: reply.current.temperature_2m,
    windKph: reply.current.wind_speed_10m,
  };
}

function adaptOther(reply) {
  return {
    city: reply.place,
    tempC: reply.conditions.tempC,
    windKph: reply.conditions.wind.kph,
  };
}

showWeather(adaptOpenMeteo(openMeteoReply, "London"));
showWeather(adaptOther(otherReply));
```

You'll see:

```
London: 15.4°C, wind 7.6 km/h
Paris: 18.1°C, wind 11.2 km/h
```

`showWeather` never learns that two providers exist. The outside world's strange field names, like `temperature_2m`, stay inside the adapters. So if a provider renames a field next year, you fix one small function instead of hunting through your whole app. An adapter can also convert units (Fahrenheit to Celsius, say) or rename methods on an object, not just fields.

Don't force patterns in

Patterns are tools, not goals. A 30-line script doesn't need a factory, a strategy, and a singleton. Every pattern adds a layer, and every layer is something the next reader has to understand.

Two rules of thumb help here:

- **KISS** ("Keep It Simple"): the simplest code that works is usually the best code.
- **YAGNI** ("You Aren't Gonna Need It"): don't build flexibility for needs you don't have yet.

[Chapter 45](#) goes deeper into both. The healthy habit is to write the plain version first, and reach for a pattern when you *notice* the problem it solves:

When you notice...	Consider...
A long <code>if / else</code> chain picking between ways to do the same job	Strategy
Several parts of the app need to react when something happens	Observer
You keep writing the same "before and after" code around different functions	Decorator
Outside data or code has a shape that doesn't fit yours	Adapter
Creating an object takes lots of setup, or you must pick which kind to make	Factory
Data that should be private is sitting out in the open	Module
There must be exactly one of something	Singleton (carefully)

Common mistakes

1. Using a pattern where a plain function would do

```
const calculatorFactory = {
  create() {
    return { strategies: { add: (a, b) => a + b } };
  },
};

const calculator = calculatorFactory.create();
console.log(calculator.strategies.add(2, 3)); // prints: 5
```

A factory and a strategy object... to add two numbers. It works, but it wraps one line of real work in three layers of extra code. Fix: `function add(a, b) { return a + b; }`. Wait until the problem a pattern solves really shows up, like a second and third way of doing the job.

2. Forgetting to unsubscribe

```
const game = new Emitter();

function startGame() {
  game.on("score", (points) => console.log(`Score: ${points}`));
}

startGame();
startGame(); // the player pressed "Play again"
game.emit("score", 10);
```

You'll see:

```
Score: 10  
Score: 10
```

Every restart added *another* listener, and the old ones never left. The score now prints twice, and listeners that are never removed also keep memory busy (a leak, [chapter 41](#)). Fix: subscribe once, outside `startGame`, or give the listener a name and call `off` when the game ends.

3. A decorator that forgets to return

```
function withLogging(fn) {  
  return function (...args) {  
    console.log(`Calling ${fn.name}`);  
    fn(...args); // the result is thrown away!  
  };  
}  
  
function double(n) {  
  return n * 2;  
}  
  
const loggedDouble = withLogging(double);  
console.log(loggedDouble(21));
```

You'll see:

```
Calling double  
undefined
```

The wrapper calls `double` but drops its answer, so the wrapped version always returns `undefined`. Fix: `return fn(...args);`. A decorator should add behavior, never take any away.

4. Treating a singleton as a scratchpad

```

const settings = { currency: "USD" }; // one shared object for the whole app

function showPriceInEuros(price) {
  settings.currency = "EUR"; // changed for one screen...
  console.log(`${price} ${settings.currency}`);
}

function showCartTotal(total) {
  console.log(`Total: ${total} ${settings.currency}`);
}

showPriceInEuros(20); // prints: 20 EUR
showCartTotal(99); // prints: Total: 99 EUR   <- surprise!

```

One screen changed the shared settings for its own needs, and now the cart shows the wrong currency. Bugs like this are hard to track down, because the line that caused it is far from the line that shows it. Fix: pass what a function needs as a parameter, like `showPrice(price, currency)`, and change shared settings only in one clearly named place.

5. Picking a strategy without a guard

```

const discounts = {
  student: (price) => price * 0.8,
  senior: (price) => price * 0.7,
};

const customerType = "Student"; // typed into a form, with a capital S
const discount = discounts[customerType];
console.log(discount(50));
// TypeError: discount is not a function

```

There's no `Student` key, so `discount` is `undefined`, and calling `undefined` crashes with a message that doesn't say what really went wrong. Fix: clean up the input (`customerType.toLowerCase()`) and check that the strategy exists before you call it. Throw a clear error, like the `shippingCost` example does, or fall back to a default such as `none: (price) => price`.

Quick recap

- A design pattern is a named, proven solution to a common problem: an idea you adapt, not code you paste.
- **Module** hides the details behind a small public interface. **Factory** builds objects so callers don't need to know how.

- **Singleton** means exactly one shared instance. In JavaScript, export an object from a module, and use it sparingly, because it's shared state.
 - **Observer** lets many listeners react to one event (`on` , `off` , `emit`) without the sender knowing who they are.
 - **Strategy** swaps one way of doing a job for another, using an object of functions instead of a long `if` chain.
 - **Decorator** wraps a function to add behavior (`once` , `debounce` , and `memoize` are all decorators). **Adapter** converts a mismatched shape into the one your code expects.
 - Write the simple version first. Reach for a pattern when you notice the problem it solves (KISS, YAGNI).
-

Next: try the [exercises](#), then move on to [45 Clean Code](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js` , `ex2.js` , and so on).
 - Run each one with `node ex1.js` .
 - Exercise 5 uses several files and `import / export` , so it gets its own folder with a `package.json` (details in the exercise).
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Ride fares

A ride-hailing app works out fares with a growing `if` chain. Copy this into `ex1.js` and run it:

```
function calculateFare(rideType, km) {
  if (rideType === "economy") {
    return 2.5 + km * 1.2;
  } else if (rideType === "comfort") {
    return 4 + km * 1.8;
  } else if (rideType === "x1") {
    return 5 + km * 2.1;
  } else {
    throw new Error(`Unknown ride type: ${rideType}`);
  }
}

const rides = [
  { rideType: "economy", km: 10 },
  { rideType: "comfort", km: 10 },
  { rideType: "x1", km: 4 },
  { rideType: "helicopter", km: 3 },
];

for (const ride of rides) {
  try {
    const fare = calculateFare(ride.rideType, ride.km);
    console.log(`${ride.rideType}, ${ride.km} km: ${fare.toFixed(2)}`);
  } catch (error) {
    console.log(error.message);
  }
}
```

1. Rewrite `calculateFare` with the **strategy** pattern: put each fare rule in an object called `fareStrategies`, and make `calculateFare` pick the right one. The output must stay exactly the same.
2. The company launches a `green` ride: \$3, plus \$1.50 per km. Add it with **one new line** in `fareStrategies`. Then add `{ rideType: "green", km: 6 }` to the end of `rides`.

Expected output after both steps:

```
economy, 10 km: $14.50
comfort, 10 km: $22.00
x1, 4 km: $13.40
Unknown ride type: helicopter
green, 6 km: $12.00
```

Rule: no `else if` chain and no `switch`. One `if` to catch unknown ride types is fine.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Free trial

A language app lets free users try its tools a few times before asking them to upgrade. Instead of changing every tool, you'll write one **decorator** that works for all of them.

Write `limitCalls(fn, maxCalls)`. It returns a new function that:

- calls `fn` with the same arguments and returns its result, for the first `maxCalls` calls,
- after that, throws an `Error` with the message `Free trial over: <function name> is limited to <maxCalls> uses`.

Test it with this code:

```
function translateToSpanish(word) {
  const dictionary = { hello: "hola", cat: "gato", thanks: "gracias", friend: "amigo" };
  return dictionary[word] ?? "(not in dictionary)";
}

function celsiusToFahrenheit(celsius) {
  return (celsius * 9) / 5 + 32;
}

const trialTranslate = limitCalls(translateToSpanish, 3);
const trialConvert = limitCalls(celsiusToFahrenheit, 2);

for (const word of ["hello", "cat", "thanks", "friend"]) {
  try {
    console.log(`${word}: ${trialTranslate(word)}`);
  } catch (error) {
    console.log(error.message);
  }
}

for (const celsius of [0, 100, 37]) {
  try {
    console.log(`${celsius}°C = ${trialConvert(celsius)}°F`);
  } catch (error) {
    console.log(error.message);
  }
}

console.log(translateToSpanish("friend")); // the original still works
```

Expected output:

```
hello: hola
cat: gato
thanks: gracias
Free trial over: translateToSpanish is limited to 3 uses
0°C = 32°F
100°C = 212°F
Free trial over: celsiusToFahrenheit is limited to 2 uses
amigo
```

Rule: don't change `translateToSpanish` or `celsiusToFahrenheit`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Parcel tracker

A delivery company sends updates as a parcel moves. Customers can switch alerts off, and some messages should only ever be sent once.

Type out the `Emitter` class from the notes, then give it two new features:

1. `on` **returns an unsubscribe function**. Calling it removes that listener.
2. A new method, `once(eventName, listener)`: the listener runs only the first time the event happens, then removes itself.

Test it with this code:

```
const tracker = new Emitter();

const stopSms = tracker.on("status", (parcel) => {
  console.log(`SMS: parcel ${parcel.id} is ${parcel.status}`);
});
tracker.on("status", (parcel) => {
  console.log(`Website: ${parcel.id} -> ${parcel.status}`);
});
tracker.once("status", (parcel) => {
  console.log(`Tip: track ${parcel.id} in our app!`);
});

tracker.emit("status", { id: "PX-12", status: "packed" });
tracker.emit("status", { id: "PX-12", status: "shipped" });
stopSms(); // the customer turned off text messages
tracker.emit("status", { id: "PX-12", status: "delivered" });
```

Expected output:


```
SMS: parcel PX-12 is packed
Website: PX-12 -> packed
Tip: track PX-12 in our app!
SMS: parcel PX-12 is shipped
Website: PX-12 -> shipped
Website: PX-12 -> delivered
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Library search

A library app searches two book services at once. Each one describes books in its own shape, but your display function wants a single shape: `{ title, author, year }`.

```
// Shaped like a real Open Library search reply (trimmed)
const openLibraryReply = {
  numFound: 3,
  docs: [
    { title: "The Hobbit", author_name: ["J.R.R. Tolkien"], first_publish_year: 1937 },
    { title: "Good Omens", author_name: ["Terry Pratchett", "Neil Gaiman"], first_publish_year: 1990 },
    { title: "Beowulf", first_publish_year: 1000 },
  ],
};

// A made-up second-hand bookshop API: similar data, different shape
const bookshopReply = {
  results: [
    { name: "Dune", writers: [{ fullName: "Frank Herbert" }], published: "1965-08-01" },
    { name: "Mystery Diary", writers: [], published: "2019" },
  ],
};

function printBooks(books) {
  for (const book of books) {
    console.log(`${book.title} by ${book.author} (${book.year})`);
  }
}
```

Write two **adapters**, `fromOpenLibrary(reply)` and `fromBookshop(reply)`. Each one returns an array of `{ title, author, year }` objects:

- `author` joins several authors with " & ". If there are no authors, it's "Unknown author".
- `year` must be a **number**. The bookshop sometimes sends a full date, sometimes only a year.

Then add this at the bottom of your file:

```
const allBooks = [...fromOpenLibrary(openLibraryReply), ...fromBookshop(bookshopReply)];
printBooks(allBooks.toSorted((a, b) => a.year - b.year));
console.log("All years are numbers:", allBooks.every((book) => typeof book.year === "number"));
```

Expected output:

```
Beowulf by Unknown author (1000)
The Hobbit by J.R.R. Tolkien (1937)
Dune by Frank Herbert (1965)
Good Omens by Terry Pratchett & Neil Gaiman (1990)
Mystery Diary by Unknown author (2019)
All years are numbers: true
```

Rule: don't change `printBooks`. All the shape-fixing happens in the adapters.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Pizza shop

Build the order system for a small pizza shop, split into modules ([chapter 29](#)). It uses five patterns from this chapter at once.

Make a folder called `ex5` with a `package.json` containing `{ "type": "module" }`, and these files:

File	Pattern	Its job
<code>emitter.js</code>	Observer	Exports your <code>Emitter</code> class from the notes
<code>events.js</code>	Singleton	Exports <code>orderEvents</code> : the one emitter the whole app shares
<code>menu.js</code>	Module + Factory	Exports <code>createPizza(name, size)</code> . The prices stay private (not exported)
<code>delivery.js</code>	Strategy	Exports <code>deliveryCost(method, km)</code>
<code>orders.js</code>		Exports <code>placeOrder(order)</code>
<code>kitchen.js</code>		Exports <code>startKitchenScreen()</code> , which subscribes to orders
<code>main.js</code>		Given below

The shop's rules:

- **Pizzas:** margherita \$8, pepperoni \$10, veggie \$9. Sizes add to the price: small +\$0, medium +\$2, large +\$4. `createPizza` returns `{ name, size, price }`, and throws `Unknown pizza: <name>` or `Unknown size: <size>`.
- **Delivery:** pickup is free, bike is \$3, and car is \$2 plus \$1 per km. Anything else throws `Unknown delivery method: <method>`.
- `placeOrder({ customer, items, delivery, km })` builds the pizzas, adds up the total (pizzas plus delivery), and then emits `"orderPlaced"` with `{ customer, pizzas, deliveryFee, total }`.
- `startKitchenScreen()` listens for `"orderPlaced"` and prints the pizzas, like `Kitchen: large margherita, small veggie`.

Here's `main.js`:

```
import { orderEvents } from "./events.js";
import { placeOrder } from "./orders.js";
import { startKitchenScreen } from "./kitchen.js";

startKitchenScreen();

orderEvents.on("orderPlaced", (order) => {
  console.log(`Receipt for ${order.customer}: ${order.total}`);
});

let ordersToday = 0;
orderEvents.on("orderPlaced", () => {
  ordersToday++;
});

const orders = [
  { customer: "Ana", items: [["margherita", "large"], ["veggie", "small"]], delivery: "bike", km: 2 },
  { customer: "Ben", items: [["pepperoni", "medium"]], delivery: "pickup", km: 0 },
  { customer: "Cleo", items: [["hawaiian", "large"]], delivery: "car", km: 5 },
  { customer: "Dev", items: [["veggie", "medium"]], delivery: "drone", km: 1 },
  { customer: "Eli", items: [["pepperoni", "large"], ["margherita", "small"]], delivery: "car", km: 4 },
];

for (const order of orders) {
  try {
    placeOrder(order);
  } catch (error) {
    console.log(`Could not place ${order.customer}'s order: ${error.message}`);
  }
}

console.log(`Orders today: ${ordersToday}`);
```

Run it from inside the `ex5` folder with `node main.js`. Expected output:

```
Kitchen: large margherita, small veggie
Receipt for Ana: $24
Kitchen: medium pepperoni
Receipt for Ben: $12
Could not place Cleo's order: Unknown pizza: hawaiian
Could not place Dev's order: Unknown delivery method: drone
Kitchen: large pepperoni, small margherita
Receipt for Eli: $28
Orders today: 3
```

Rules:

- Only `events.js` creates an `Emitter`. Every other file imports `orderEvents` from there.
- `placeOrder` never calls the kitchen or receipt code directly. It only emits.
- A failed order must not reach the kitchen, and must not count towards `Orders today`.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Look back at your Exercise 1 code. The fare rules are clearer now, but what do `2.5` and `1.2` actually mean? Someone reading your code next year would have to guess.

[Chapter 45: Clean Code](#) is all about writing code that's easy to read, starting with names that explain themselves.



45 Clean Code

What is it?

Clean code is code that's easy to read, easy to understand, and easy to change, both for other people and for you in six months' time.

It isn't about clever tricks or fewer lines. Clean code is *clear* code: someone reading it can tell what it does, and why, without having to decode it.

Why does it matter?

Code is read far more often than it's written. You write a function once, but you (and your teammates) will read it again every time you fix a bug near it, add a feature, or wonder why something happened.

Messy code costs you every single time it's read:

- **Bugs hide in it.** When you can't easily see what code does, you can't easily see what it does *wrong*.
- **Changes get scary.** If you don't understand a function, you're afraid to touch it, so problems pile up.
- **Your future self is a stranger.** In six months, you'll have forgotten why you wrote `x * 0.85`. Write for that person.

The good news: clean code is mostly a set of small habits. None of them are hard, and every idea in this chapter comes as a before/after pair, so you can see the difference for yourself.

Real-world example

Think of a kitchen.

You can cook a great meal in a messy kitchen, as long as you're the one who made the mess. The trouble starts when someone else cooks there, or when you come back after a month away. Is the white powder in that unlabeled jar salt or sugar?

Messy kitchen	Tidy kitchen	In code
Three identical jars with no labels	Every jar has a label	Meaningful names
One giant drawer for everything	A drawer for each kind of thing	Small functions that each do one job
A recipe that says "add 2 of the white stuff"	"Add 2 teaspoons of sugar"	Named constants, not mystery numbers

Messy kitchen	Tidy kitchen	In code
The same sauce recipe scribbled in five places	One recipe card that everyone uses	Don't repeat yourself
A note saying "stir"	A note saying "stir all the time, or it burns"	Comments that explain <i>why</i>

A tidy kitchen doesn't make the food taste better. It makes cooking faster, safer, and possible for more than one person. Clean code does the same for programs.

How it works

In most of the before/after pairs below, the "after" does exactly what the "before" does. Only the way it reads changes. (Two of them also fix a bug, and they'll tell you when.)

Meaningful names

Names are the labels on your jars. A good name tells the reader what something *is* or what it *does*, so they don't have to work it out.

Before:

```
function calc(d, t) {
  const x = d / t;
  return x > 100;
}

console.log(calc(300, 2)); // prints: true
```

After:

```
function isSpeeding(distanceKm, hours) {
  const speedKmh = distanceKm / hours;
  return speedKmh > 100;
}

console.log(isSpeeding(300, 2)); // prints: true
```

Same logic, but now you can read it out loud and it makes sense. Some simple rules:

Kind of thing	Name it with	Examples
A value	A noun	<code>total</code> , <code>userName</code> , <code>speedKmh</code>
A list	A plural noun	<code>products</code> , <code>scores</code> , <code>openOrders</code>
A function	A verb	<code>calculateTotal</code> , <code>sendReminder</code> , <code>formatPrice</code>
A true/false value, or a function that answers yes or no	<code>is</code> , <code>has</code> , or <code>can</code>	<code>isOpen</code> , <code>hasDiscount</code> , <code>canVote</code> , <code>isSpeeding</code>

A few more habits that help:

- **A function that only works out a value** can be named after that value instead of a verb, like `average(numbers)` or `shippingCost(order)`. What matters is that the name says what you get back.
- **Put units in the name** when there are any: `delayMs`, `weightKg`, `priceInCents`.
- **Avoid vague names** like `data`, `info`, `temp`, or `doStuff`. They tell the reader nothing.
- **Avoid shortening words**: `customer` beats `cstmr`. Short, well-known names like `i` in a small loop, or `a` and `b` in a sort compare function, are fine.
- **Be consistent**. If you call it `fetchUser` in one place, don't call the same idea `getCustomer` somewhere else.

There's still a mystery number in `isSpeeding`, though. What is `100`? That's next.

No magic numbers

A **magic number** is a number in your code with no explanation. It works like magic: nobody knows why it's there, and nobody dares change it.

Before:

```
function finalPrice(price) {
  if (price > 50) {
    return price * 1.2;
  }
  return price * 1.2 + 4.99;
}
```

What's `1.2`? What's `50`? Is the `4.99` the same `4.99` that appears in the checkout code, or a different one?

After:

```
const TAX_RATE = 0.2;
const FREE_SHIPPING_OVER = 50;
const SHIPPING_FEE = 4.99;

function finalPrice(price) {
  const priceWithTax = price * (1 + TAX_RATE);
  if (price > FREE_SHIPPING_OVER) {
    return priceWithTax;
  }
  return priceWithTax + SHIPPING_FEE;
}

console.log(finalPrice(40).toFixed(2)); // prints: 52.99
```

Now the numbers have names, and each lives in exactly one place. When the tax rate changes, you change one line.

Notice the `UPPER_SNAKE_CASE` names: capital letters with underscores between the words. That's the usual way to write a **constant**: a fixed setting that's known before the program runs and never changes. Ordinary `const` variables inside functions, like `priceWithTax`, stay in camelCase.

Not every number is magic. `0`, `1`, `* 2` for doubling, or `/ 100` for a percentage are usually clear on their own. Ask yourself: would a new reader know what this number means?

Small functions that do one thing

A function should do one job, and its name should say what that job is. If you need the word "and" to describe it, like "checks the cart *and* works out the total *and* prints a receipt", it's doing too much.

Before:


```
function checkout(cart) {
  // check the cart
  if (cart.length === 0) {
    console.log("Your cart is empty.");
    return;
  }
  // work out the total
  let total = 0;
  for (const item of cart) {
    total += item.price * item.quantity;
  }
  // print the receipt
  for (const item of cart) {
    console.log(`${item.name} x${item.quantity}: ${item.price * item.quantity.toFixed(2)}`);
  }
  console.log(`Total: ${total.toFixed(2)}`);
}
```

Look at those comments: `// check the cart`, `// work out the total`, `// print the receipt`. When you need a comment to label a *section* of a function, that section usually wants to be its own function, with the comment as its name.

After:

```
function lineTotal(item) {
  return item.price * item.quantity;
}

function cartTotal(cart) {
  return cart.reduce((sum, item) => sum + lineTotal(item), 0);
}

function printReceipt(cart) {
  for (const item of cart) {
    console.log(`${item.name} x${item.quantity}: ${lineTotal(item).toFixed(2)}`);
  }
  console.log(`Total: ${cartTotal(cart).toFixed(2)}`);
}

function checkout(cart) {
  if (cart.length === 0) {
    console.log("Your cart is empty.");
    return;
  }
  printReceipt(cart);
}
```

Try both versions with the same cart:

```
checkout([
  { name: "Notebook", price: 3.5, quantity: 2 },
  { name: "Pen", price: 1.25, quantity: 4 },
]);
```

Both print exactly the same:

```
Notebook x2: $7.00
Pen x4: $5.00
Total: $12.00
```

The "after" version is longer, and that's fine. Clean doesn't mean short. What you gained:

- `checkout` now reads like a table of contents: check the cart, then print the receipt.
- `item.price * item.quantity` lives in one place (`lineTotal`) instead of two.
- Each small function can be reused on its own, and tested on its own ([chapter 46](#)).

Guard clauses instead of deep nesting

You met guard clauses in [chapter 18](#): checks at the top of a function that leave early. They're one of the quickest ways to make code easier to read.

Before:

```
function checkLoan(member, book) {
  if (member) {
    if (member.isActive) {
      if (book.copiesLeft > 0) {
        return `${member.name} can borrow ${book.title}`;
      } else {
        return "Sorry, no copies left";
      }
    } else {
      return "Your membership has expired";
    }
  } else {
    return "Please sign in first";
  }
}
```

To find out what happens when a member isn't signed in, your eyes have to travel all the way to the bottom and match up the braces. Deeply nested code like this drifts to the right, and it gets harder to follow with every level.

After:

```
function checkLoan(member, book) {
  if (!member) {
    return "Please sign in first";
  }
  if (!member.isActive) {
    return "Your membership has expired";
  }
  if (book.copiesLeft <= 0) {
    return "Sorry, no copies left";
  }

  return `${member.name} can borrow ${book.title}`;
}
```

Each problem gets one flat check, right next to its message. The normal case, the **happy path**, sits at the bottom with nothing wrapped around it.

DRY, KISS, and YAGNI

These three short rules of thumb come up all the time. You met KISS and YAGNI briefly in [chapter 44](#).

DRY ("Don't Repeat Yourself"): each piece of knowledge should live in one place. When the same logic is copied around, a fix in one copy gets forgotten in the others.

Before:

```
const shirtPrice = 20 - 20 * 0.15;
const shoesPrice = 60 - 60 * 0.15;
const hatPrice = 12 - 12 * 0.15;
```

After:

```
const SALE_DISCOUNT = 0.15;

function salePrice(price) {
  return price - price * SALE_DISCOUNT;
}

const shirtPrice = salePrice(20); // 17
const shoesPrice = salePrice(60); // 51
const hatPrice = salePrice(12); // 10.2
```

When the sale changes to 20% off, you change one number instead of hunting for every `0.15`.

But be careful: DRY is about repeated *knowledge*, not code that happens to look alike. Two bits of code can look the same today and need to change for different reasons tomorrow. A common rule of thumb is to wait until you've written the same thing three times before you pull it into a shared function.

KISS ("Keep It Simple"): pick the plain, obvious solution over the clever one.

Before:

```
function isWeekend(day) {  
  return [0, 6].indexOf(day) !== -1 ? true : false;  
}
```

After:

```
function isWeekend(day) {  
  return day === 0 || day === 6; // 0 is Sunday, 6 is Saturday  
}
```

A comparison already gives you `true` or `false`, so `? true : false` adds nothing. And a reader understands `day === 0 || day === 6` at a glance.

YAGNI ("You Aren't Gonna Need It"): don't build features or options "just in case". Unused options still have to be read, tested, and kept working.

Before:

```
function greet(name, greeting = "Hello", punctuation = "!", shout = false, times = 1) {  
  // ...20 lines handling options that nobody uses  
}
```

After:

```
function greet(name) {  
  return `Hello, ${name}!`;  
}
```

When a real need for another option shows up, add it then. You'll know much more about what's needed by that point.

Comments explain why, not what

The code already says *what* it does. A comment that repeats it is just noise, and it can go out of date. Save comments for what the code *can't* say: why it's done this way.

Before:

```
// multiply the price by the quantity
const cost = price * quantity;

// add 1 to the page index
const pageNumber = pageIndex + 1;
```

After:

```
const cost = price * quantity;

// The API counts pages from 0, but people count from 1.
const pageNumber = pageIndex + 1;
```

Good reasons to write a comment:

- **Why** something looks odd: "The bank rejects amounts with more than 2 decimal places, so we round here."
- **A warning**: "Don't reorder these steps: the stock must be checked before charging the card."
- **A note for later**: `// TODO: support gift cards`.

Tip: a comment just above a function that starts with `/**` is a **doc comment**. VS Code shows it when you hover over the function's name anywhere in your code, which makes it a handy place to explain what a function expects and returns.

Consistent formatting

Formatting is the layout of your code: spaces, indentation, line breaks, quotes, semicolons. JavaScript doesn't care about most of it, but people do.

Before:

```
function sumPrices(items){
let sum=0
  for(const item of items){ sum+=item.price }
  return sum}
```

After:

```
function sumPrices(items) {
  let sum = 0;
  for (const item of items) {
    sum += item.price;
  }
  return sum;
}
```

Both work. Only one is pleasant to read. The exact style matters less than using the *same* style everywhere. The best way is to let a tool do it for you: **Prettier** can reformat your whole file every time you save. You'll set it up in [chapter 50](#).

An options object instead of a long parameter list

What do `true`, `false`, and `30` mean here?

Before:

```
createAccount("Sam", "sam@example.com", true, false, 30);
```

You'd have to open the function to find out, and it's easy to mix up the order. The options object from [chapter 15](#) fixes both problems:

After:

```
function createAccount({ name, email, newsletter = false, isAdmin = false, trialDays = 14 }) {
  return { name, email, newsletter, isAdmin, trialDays };
}

const account = createAccount({
  name: "Sam",
  email: "sam@example.com",
  newsletter: true,
  trialDays: 30,
});

console.log(account.trialDays, account.isAdmin); // prints: 30 false
```

Now every value is labeled at the place where it's used, the order doesn't matter, and you can leave out anything that has a sensible default. A good rule of thumb: once a function needs more than three parameters, or any true/false flags, reach for an options object.

Avoid shared global state

Global state is data that lives outside your functions, where any function can read it and change it. It causes bugs that are hard to find, because the result of a function depends on what *other* code did earlier.

Before:

```
let discount = 0;

function applyVoucher(code) {
  if (code === "SAVE10") {
    discount = 10;
  }
}

function checkoutTotal(subtotal) {
  return subtotal - discount;
}

applyVoucher("SAVE10");
console.log(checkoutTotal(50)); // prints: 40
console.log(checkoutTotal(80)); // prints: 70  <- the next customer got the voucher too!
```

After:

```
function voucherDiscount(code) {
  return code === "SAVE10" ? 10 : 0;
}

function checkoutTotal(subtotal, voucherCode) {
  return subtotal - voucherDiscount(voucherCode);
}

console.log(checkoutTotal(50, "SAVE10")); // prints: 40
console.log(checkoutTotal(80)); // prints: 80
```

This one is more than a tidy-up: it fixes a real bug. Each function now gets everything it needs through its parameters and hands back a result, like the pure functions in [chapter 43](#). Nothing leaks from one customer to the next.

Fail loudly

When something is wrong, say so straight away, with a clear message. Code that quietly carries on with bad data is much harder to debug later.

Before:

```
function lineTotal(price, quantityText) {  
  const quantity = Number(quantityText);  
  return price * quantity;  
}  
  
console.log(lineTotal(4.5, "2")); // prints: 9  
console.log(lineTotal(4.5, "two")); // prints: NaN
```

That `NaN` won't crash anything. It will quietly flow into the cart total, and a customer will see "Total: \$NaN" days later, far away from the line that caused it.

After:

```
function lineTotal(price, quantityText) {  
  const quantity = Number(quantityText);  
  if (!Number.isInteger(quantity) || quantity < 1) {  
    throw new Error(`Invalid quantity: "${quantityText}"`);  
  }  
  return price * quantity;  
}  
  
console.log(lineTotal(4.5, "2")); // prints: 9  
console.log(lineTotal(4.5, "two"));  
// Error: Invalid quantity: "two"
```

This is the second bug fix: the problem now shows up at the exact spot where it happens, with a message that says what's wrong. A loud crash today beats a quiet wrong answer next month. (Chapter 18 covers when to catch errors like this one.)

Refactor in small steps

Refactoring means changing how code is written *without* changing what it does. Every "before → after" in this chapter is a refactoring (apart from the two bug fixes).

The safe way to refactor is in small steps:

1. **Run the code and save what it prints**, so you know what "working" looks like. (Tests do this job even better. They're next, in chapter 46.)
2. **Make one small change**: rename one variable, or pull one piece into a function.
3. **Run it again and compare**. Same output? Great, keep going. Different? Undo that one step and look closer.
4. **Repeat**.

Small steps mean that when something breaks, you know exactly which change broke it.

Tip: VS Code can do some refactorings for you. Click a name and press `F2` (**Rename Symbol**) to rename it everywhere it's used, in one go. Select a few lines and press `Ctrl+Shift+R` to see the **Refactor** menu, which can pull the selected lines out into a new function for you.

Also, don't mix refactoring with fixing bugs or adding features. Do one kind of change at a time. If you find a bug while you're tidying up, write it down, finish the tidy-up, and then fix the bug as a separate step.

Common mistakes

1. Changing the behavior while "cleaning up"

```
// Before
function canRentCar(age) {
  if (age >= 21) {
    return true;
  }
  return false;
}

console.log(canRentCar(21)); // prints: true
```

```
// After "cleaning up"
const MIN_RENTAL_AGE = 21;

function canRentCar(age) {
  return age > MIN_RENTAL_AGE;
}

console.log(canRentCar(21)); // prints: false
```

The new version is shorter and has a named constant, but `>=` quietly became `>`. Every 21-year-old is now turned away. This is why you run the code before and after every small step, and compare. The fix: `return age >= MIN_RENTAL_AGE;`

2. A name that lies

```
function getHighScore(scores) {
  scores.sort((a, b) => b - a);
  return scores[0];
}

const scores = [40, 95, 70];
console.log(getHighScore(scores)); // prints: 95
console.log(scores); // prints: [ 95, 70, 40 ] <- the original order is gone
```

A name starting with `get` promises to *look*, not to *change*. But `sort` changes the array it's called on ([chapter 16](#)), so the caller's list got rearranged behind their back. Either make the function match its name (`Math.max(...scores)`, or `toSorted`), or give it an honest name.

3. A comment that lies

```
// Members get 10% off
const memberPrice = price * 0.85;
```

Is the discount 10% or 15%? Someone changed the code and forgot the comment. Now nobody knows which one is right. Comments don't get checked when the code runs, so they go out of date without anyone noticing. Here, a named constant says it better, and it can't go out of date:

```
const MEMBER_DISCOUNT = 0.15;
const memberPrice = price * (1 - MEMBER_DISCOUNT);
```

4. Over-DRY: forcing different things into one function

```
function format(value, type) {
  if (type === "price") {
    return `$$${value.toFixed(2)}`;
  }
  if (type === "percent") {
    return `${value * 100}%`;
  }
  if (type === "date") {
    return value.toISOString().slice(0, 10);
  }
}

console.log(format(5, "price")); // prints: $5.00
console.log(format(5, "Price")); // prints: undefined
```

These three jobs share nothing except the word "format". Squashing them together made one function that's harder to read and easy to call wrongly. Three small functions, `formatPrice`, `formatPercent`, and `formatDate`,

would each be clearer, and a typo in a function name gives you a helpful `ReferenceError` instead of a silent `undefined`.

Quick recap

- Code is read far more often than it's written, so write it for the reader (including future you).
 - Use names that explain themselves: nouns for values, verbs for functions, `is` / `has` / `can` for true/false values.
 - Replace magic numbers with named constants in `UPPER_SNAKE_CASE`, and keep functions small, each doing one job.
 - Use guard clauses instead of deep nesting, and an options object instead of a long list of parameters.
 - DRY, KISS, and YAGNI: don't repeat knowledge, keep it simple, and don't build things you don't need yet.
 - Comments explain *why*. Let a tool like Prettier handle formatting. Avoid shared global state, and fail loudly.
 - Refactor in small steps, and check that the output stays the same after every step.
-

Next: try the [exercises](#), then move on to [46 Testing](#).

Exercises

How to do these:

- Every exercise gives you some messy code that works. Copy it into a new file in this folder (`ex1.js`, `ex2.js`, and so on) and run it with `node ex1.js` **before** you change anything.
 - Then clean it up in small steps. After every step, run it again. The output must stay **exactly** the same as the expected output shown. That's how you know you refactored, instead of breaking something.
 - Keep a copy of the messy version (like `ex1-before.js`) so you can compare. In VS Code, right-click one file, choose **Select for Compare**, then right-click the other and choose **Compare with Selected**.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Name game

A step-counter app works fine, but nobody can tell what anything means:

```

function f(a, b) {
  return a * b;
}

function chk(n) {
  if (n >= 10000) {
    return true;
  }
  return false;
}

const d = [8200, 12050, 10000, 4300, 15800];
let t = 0;
let c = 0;

for (const s of d) {
  t += s;
  if (chk(s)) {
    c++;
  }
}

console.log(`Total steps: ${t}`);
console.log(`Days at goal: ${c}`);
console.log(`Distance walked: ${f(t, 0.0008).toFixed(1)} km`);

```

Give every function, parameter, and variable a name that explains itself. Use the naming table from the notes: verbs for functions, `is / has / can` for true/false, plural nouns for lists.

Expected output (before and after):

```

Total steps: 50350
Days at goal: 3
Distance walked: 40.3 km

```

Rule: change names only. Leave the numbers and the logic alone (the next exercise deals with the numbers).

Bonus: `chk` can become a one-line function without changing what it returns. How?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Gym fees

A gym's fee calculator is full of numbers that only the person who wrote it understands:

```
function monthlyFee(age, months, hasTowelService) {
  let fee = 35;
  if (age < 18 || age >= 65) {
    fee = fee * 0.6;
  }
  if (months >= 12) {
    fee = fee - 5;
  }
  if (hasTowelService) {
    fee = fee + 4;
  }
  return fee;
}

console.log(`Adult, 1 month: ${monthlyFee(30, 1, false)}`);
console.log(`Teen, 12 months, towel: ${monthlyFee(16, 12, true)}`);
console.log(`Senior, 6 months, towel: ${monthlyFee(70, 6, true)}`);
console.log(`Adult, 24 months: ${monthlyFee(45, 24, false)}`);
```

Replace every magic number inside `monthlyFee` with a named constant, declared at the top of the file.

Expected output (before and after):

```
Adult, 1 month: $35
Teen, 12 months, towel: $20
Senior, 6 months, towel: $25
Adult, 24 months: $30
```

Rule: no bare numbers left inside `monthlyFee`. The constants use `UPPER_SNAKE_CASE`.

When you're done, imagine the gym puts the price up to \$38. How many lines would you change now?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Cinema door

The ticket scanner at a cinema door runs this check. It works, but it's a staircase of `if` s:

```

function checkTicket(ticket, movie) {
  if (ticket) {
    if (ticket.movieId === movie.id) {
      if (!ticket.used) {
        if (ticket.age >= movie.minAge) {
          return `Enjoy ${movie.title}! Screen ${movie.screen}`;
        } else {
          return `Sorry, ${movie.title} is for ages ${movie.minAge}+`;
        }
      } else {
        return "This ticket has already been used";
      }
    } else {
      return "This ticket is for a different movie";
    }
  } else {
    return "No ticket scanned";
  }
}

const movie = { id: 7, title: "Space Pirates", minAge: 12, screen: 3 };

const tickets = [
  { movieId: 7, used: false, age: 15 },
  { movieId: 7, used: false, age: 9 },
  { movieId: 7, used: true, age: 30 },
  { movieId: 4, used: false, age: 30 },
  { movieId: 4, used: true, age: 8 },
  null,
];

for (const ticket of tickets) {
  console.log(checkTicket(ticket, movie));
}

```

Rewrite `checkTicket` with guard clauses.

Expected output (before and after):

```

Enjoy Space Pirates! Screen 3
Sorry, Space Pirates is for ages 12+
This ticket has already been used
This ticket is for a different movie
This ticket is for a different movie
No ticket scanned

```

Rule: no `else` at all, and no `if` inside another `if`. The "Enjoy" line (the happy path) is the last line of the function.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Report card

A teacher's report function does everything in one long block, and it adds up marks in three different places:

```

function report(students) {
  console.log("=== Report card ===");
  for (const s of students) {
    let sum = 0;
    for (const m of s.marks) {
      sum += m;
    }
    const avg = sum / s.marks.length;
    let grade;
    if (avg >= 90) {
      grade = "A";
    } else if (avg >= 75) {
      grade = "B";
    } else if (avg >= 60) {
      grade = "C";
    } else {
      grade = "F";
    }
    console.log(`${s.name.padEnd(8)}${avg.toFixed(1).padStart(5)} ${grade}`);
  }
  let all = 0;
  let count = 0;
  for (const s of students) {
    for (const m of s.marks) {
      all += m;
      count++;
    }
  }
  console.log(`Class average: ${(all / count).toFixed(1)}`);
  let best = students[0];
  for (const s of students) {
    let sum1 = 0;
    for (const m of s.marks) {
      sum1 += m;
    }
    let sum2 = 0;
    for (const m of best.marks) {
      sum2 += m;
    }
    if (sum1 / s.marks.length > sum2 / best.marks.length) {
      best = s;
    }
  }
  console.log(`Top student: ${best.name}`);
}

const students = [
  { name: "Amara", marks: [88, 92, 95] },
  { name: "Ben", marks: [72, 65, 80, 71] },

```



```

    { name: "Chloe", marks: [55, 62, 58] },
    { name: "Dmitri", marks: [91, 89, 94, 90] },
];

report(students);

```

Split it into small functions that each do one job, so that `report` itself reads like a table of contents.

Expected output (before and after):

```

=== Report card ===
Amara    91.7  A
Ben      72.0  C
Chloe    58.3  F
Dmitri   91.0  A
Class average: 78.7
Top student: Amara

```

Rules:

- Write one `average(numbers)` function, and use it everywhere an average is needed. No other function adds up marks.
- `report` is at most 8 lines long, and it has no loops of its own apart from one `for...of` that prints the rows.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): The cafe till

This is the till program at a small cafe. It has almost every problem from the notes: mystery names, magic numbers, deep nesting, a long list of true/false parameters, global variables, and even a comment that lies.

```

let orders = [];
let total = 0;

// Add an order
function add(n, s, m, x, t) {
  let p = 0;
  if (s === "small") {
    p = 3;
  } else {
    if (s === "medium") {
      p = 3.5;
    } else {
      if (s === "large") {
        p = 4;
      }
    }
  }
  if (m) {
    p = p + 0.5;
  }
  if (x) {
    p = p + 0.75;
  }
  // 10% off for takeaway
  if (t) {
    p = p * 0.8;
  }
  orders.push({ n: n, p: p });
  total = total + p;
}

function print() {
  for (let i = 0; i < orders.length; i++) {
    console.log(orders[i].n + ": $" + orders[i].p.toFixed(2));
  }
  console.log("Total: $" + total.toFixed(2));
  if (total > 10) {
    console.log("Loyalty stamp earned!");
  }
}

add("Latte", "medium", true, false, false);
add("Mocha", "large", false, true, true);
add("Flat white", "small", true, true, false);
add("Latte", "huge", false, false, false);
print();

```

(In `add`: `m` means oat milk, `x` means an extra shot, and `t` means takeaway.)

Clean it up so that:

- every name explains itself, and there are no magic numbers,
- the drink's options are passed as an **options object**, like `{ size: "large", extraShot: true, takeaway: true }`, with sensible defaults,
- there are **no global variables** that functions change. Functions take what they need as parameters and return results. The total is worked out from the list of orders, not kept in a separate running total,
- there's no `if` inside another `if`, and no `if` chain for the sizes,
- string joining with `+` becomes template literals.

Expected output (before and after):

```
Latte: $4.00
Mocha: $3.80
Flat white: $4.25
Latte: $0.00
Total: $12.05
Loyalty stamp earned!
```

Yes, that includes the free `huge` latte. An unknown size silently costs \$0, which is a bug, but a refactoring must not change behavior. Write the bug down, and fix it *after* the clean-up, as its own step:

Bonus: now make unknown sizes fail loudly. Throw an error from the function that prices a drink, catch it where the orders are created, and skip that order. The new expected output:

```
Could not add Latte: Unknown size "huge"
Latte: $4.00
Mocha: $3.80
Flat white: $4.25
Total: $12.05
Loyalty stamp earned!
```

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

After every step in these exercises, you ran the code and compared the output by eye. That works for five lines of output. What about a real app with hundreds of functions?

In [chapter 46](#), you'll write small programs that check your code for you, in a fraction of a second, every time you change something. 

46 Testing

What is it?

A **test** is a small piece of code that checks whether another piece of code does what it should. You run it, and it answers with a simple "pass" or "fail".

Automated testing means writing these checks once and letting the computer run them for you, as often as you like. Node comes with everything you need, so there's nothing to install.

Why does it matter?

Until now, you've tested your code by running it and reading the output. In [chapter 45](#), you compared outputs by eye after every refactoring step. That works for five lines of output. It doesn't work for an app with 200 functions.

Automated tests help in four big ways:

- **They catch bugs before your users do.** A test fails in your terminal, not in a customer's shopping cart.
- **They make changes safe.** Refactoring and adding features stop being scary, because the tests tell you straight away if you broke something.
- **They stop old bugs from coming back.** When something that used to work breaks again, that's called a **regression**. A test written for a fixed bug guards against it forever.
- **They're fast.** Hundreds of tests run in about a second, every time you ask.

Real-world example

Think of a smoke detector.

You install it once. After that, it checks the air all day and all night, and it only makes a noise when something is wrong. You don't have to remember to sniff every room before bed.

Tests are smoke detectors for your code. And like a pilot's pre-flight checklist, they go through the same list every single time, even the boring items that "never" go wrong. That's exactly where the surprises hide.

	Checking by hand	Automated tests
Who does the checking	You, reading the output	The computer
How long it takes	Minutes, and it grows with your app	A second or two
Checks every case, every time?	Rarely: people forget and get bored	Always

	Checking by hand	Automated tests
When it happens	When you remember to	Every time you run <code>node --test</code>

Checking by hand still has its place, like trying out a new page to see if it *feels* right. But the "does 2 + 2 still give 4?" work belongs to the computer.

How it works

Your first test

Tests usually check one small piece of code at a time, like a single function. Those are called **unit tests** (a "unit" is one small piece). They're what this chapter is about.

Make a new folder for this. Node's test tools use `import`, so the folder needs a `package.json` containing `{ "type": "module" }`, just like in [chapter 29](#).

Here's the code we want to test. Save it as `prices.js`:

```
export function addTax(price, rate) {
  return price + price * rate;
}
```

And here's a test for it, in a file called `prices.test.js`:

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { addTax } from "./prices.js";

test("adds 20% tax to a price", () => {
  const result = addTax(50, 0.2);
  assert.equal(result, 60);
});

test("leaves the price alone when the rate is 0", () => {
  assert.equal(addTax(50, 0), 50);
});
```

Now open a terminal in that folder and run:

```
node --test
```

You'll see something like this (your times will be different):

```
✓ adds 20% tax to a price (0.5526ms)
✓ leaves the price alone when the rate is 0 (0.1052ms)
i tests 2
i suites 0
i pass 2
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 165.2655
```

Two green ticks: both tests pass. Let's look at the pieces:

- `node:test` and `node:assert/strict` are built into Node. The `node:` at the front means "Node's own module", so there's nothing to install.
- `test(name, fn)` creates one test. The name says in plain words what *should* happen. When a test fails, that name is the first thing you'll read, so make it clear.
- `assert.equal(actual, expected)` checks that the two values are equal. `actual` is what your code gave you, and `expected` is what it should be. If they don't match, `assert.equal` throws an error, and the test fails.
- The `/strict` part means the comparisons work like `===`. So `"60"` does not equal `60`. Always use the strict version.

To run just one file, name it: `node --test prices.test.js`. And `node --test --watch` keeps running and re-runs your tests every time you save a file. Press `Ctrl + C` to stop it.

How `node --test` finds your tests

When you run `node --test` without naming a file, Node searches the folder (and the folders inside it) for files with test-like names. In Node 24, these are picked up:

File name	Example
Ends in <code>.test.js</code>	<code>prices.test.js</code>
Ends in <code>-test.js</code> or <code>_test.js</code>	<code>prices-test.js</code> , <code>prices_test.js</code>
Starts with <code>test-</code>	<code>test-prices.js</code>
Is called <code>test.js</code>	<code>test.js</code>
Any <code>.js</code> file inside a folder called <code>test</code>	<code>test/prices.js</code>

The same patterns work for `.mjs` and `.cjs` files. Folders called `node_modules` are skipped.

A file called `prices.spec.js` or `pricesTest.js` is *not* found. That catches a lot of people out (see Common mistakes). The simplest habit: put `something.test.js` right next to `something.js`.

When a test fails

Tests earn their keep when they fail. Here's a discount function with a bug:

```
// discount.js
export function applyDiscount(price, percent) {
  return price - percent / 100; // bug: forgot to multiply by the price
}
```

```
// discount.test.js
import { test } from "node:test";
import assert from "node:assert/strict";
import { applyDiscount } from "../discount.js";

test("takes 25% off a $80 jacket", () => {
  assert.equal(applyDiscount(80, 25), 60);
});

test("0% off leaves the price alone", () => {
  assert.equal(applyDiscount(80, 0), 80);
});
```

Run `node --test`, and you'll see this (trimmed a little):

```

X takes 25% off a $80 jacket (2.1571ms)
✓ 0% off leaves the price alone (0.3569ms)
i tests 2
i suites 0
i pass 1
i fail 1
...

X failing tests:

test at discount.test.js:5:1
X takes 25% off a $80 jacket (2.1571ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

    79.75 !== 60

      at TestContext.<anonymous> (file:///C:/.../discount.test.js:6:10)
      ...

```

How to read it:

1. **The summary** shows a red cross next to the failing test's name, and `fail 1`.
2. **The failing tests section** repeats each failure with details. `test at discount.test.js:5:1` is where the test starts.
3. `79.75 !== 60` is the heart of it: the actual value, then the expected one. The function took off 0.25 dollars instead of 25%.
4. **The at line** points at the exact `assert` that failed: line 6 of `discount.test.js`. You read these lines just like the stack traces in [chapter 18](#).

Notice that the second test *passed*. With 0% off, the bug happens to give the right answer. That's why one test is rarely enough.

(Behind the scenes, a failed run also ends with an **exit code** of 1: a number every program hands back when it finishes, where 0 means "all good". Tools that run your tests automatically use it to spot failures.)

Fix the bug with `return price - (price * percent) / 100;`, run the tests again, and both go green.

Arrange, act, assert

Most good tests have the same three steps, in the same order:

1. **Arrange**: set up everything the test needs.
2. **Act**: do the one thing you're testing.
3. **Assert**: check the result.

Here's a library that charges 25 cents for every day a book is late:

```
// library.js
const FEE_PER_DAY_CENTS = 25;
const MS_PER_DAY = 24 * 60 * 60 * 1000;

export function lateFeeCents(dueDate, returnedDate) {
  const daysLate = Math.ceil((returnedDate - dueDate) / MS_PER_DAY);
  return Math.max(0, daysLate) * FEE_PER_DAY_CENTS;
}
```

```
// library.test.js
import { test } from "node:test";
import assert from "node:assert/strict";
import { lateFeeCents } from "../library.js";

test("charges 25 cents for each day late", () => {
  // Arrange
  const dueDate = new Date("2026-03-01");
  const returnedDate = new Date("2026-03-05");

  // Act
  const fee = lateFeeCents(dueDate, returnedDate);

  // Assert
  assert.equal(fee, 100);
});
```

You don't have to write the three comments every time. But keeping the steps in this order makes every test easy to scan. (The fee is in cents, so the test can compare whole numbers and skip the decimal surprises from [chapter 05](#).)

Grouping tests with `describe` and `it`

As tests pile up, you can group the ones about the same thing with `describe`. Inside it, `it` works exactly like `test`, and it reads nicely as a sentence: "withdraw: it takes the amount off the balance".

Here's the `withdraw` function from chapter 18, exported from `bank.js`:

```
// bank.js
export function withdraw(balance, amount) {
  if (typeof amount !== "number" || Number.isNaN(amount)) {
    throw new TypeError("Amount must be a number");
  }
  if (amount <= 0) {
    throw new RangeError("Amount must be more than 0");
  }
  if (amount > balance) {
    throw new Error("Not enough money");
  }
  return balance - amount;
}
```

```
// bank.test.js
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { withdraw } from "../bank.js";

describe("withdraw", () => {
  it("takes the amount off the balance", () => {
    assert.equal(withdraw(100, 30), 70);
  });

  it("allows taking out everything", () => {
    assert.equal(withdraw(100, 100), 0);
  });
});
```

The output shows the group with its tests indented underneath:

```
▶ withdraw
  ✓ takes the amount off the balance (0.6489ms)
  ✓ allows taking out everything (0.0987ms)
✓ withdraw (3.325ms)
```

A group of tests is called a **suite**. That's what `i suites 1` counts in the summary.

The assertions you'll use most

An **assertion** is one check inside a test: "I assert that this is true." Here are the ones you'll reach for most:

Assertion	Passes when...	Use it for
<code>assert.equal(actual, expected)</code>	the two values are the same, like <code>===</code>	numbers, strings, true/false
<code>assert.deepEqual(actual, expected)</code>	the two have the same contents	arrays and objects
<code>assert.notEqual(actual, expected)</code>	the two are different	"this must have changed"
<code>assert.ok(value)</code>	the value is truthy	quick yes/no checks
<code>assert.throws(fn, expected)</code>	calling <code>fn</code> throws the expected error	code that should refuse bad input
<code>await assert.rejects(promise, expected)</code>	the promise rejects with the expected error	async code that should fail

Why two kinds of "equal"? Remember from [chapter 16](#) that `===` compares arrays and objects by reference. Two separate arrays with the same items are *not* `===`. `deepEqual` looks inside and compares the contents.

Edge cases

An **edge case** is an unusual input at the very edge of what your code should handle. It's where bugs love to hide. For each function, ask yourself:

- What about **empty** things? An empty array, an empty string, an empty cart.
- What about **zero**, **negative** numbers, and very **big** numbers?
- What about values **exactly on a boundary**? If shipping is free "from \$50", test \$49.99, \$50, and \$50.01.
- What about **missing or wrong** input? `undefined`, `null`, or `"fifty"` where you expected `50`.

`withdraw(100, 100)` above is an edge case: taking out *exactly* everything. It's easy to write `amount >= balance` by mistake and refuse it.

You don't need a test for every number in the world. Pick one normal case, then the edges.

Testing errors

Refusing bad input is part of a function's job, so test it too. `assert.throws` takes a **function** to call, and passes only if calling it throws the error you describe:

```
// bank-errors.test.js
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import { withdraw } from "./bank.js";

describe("withdraw refuses bad requests", () => {
  it("refuses to go below zero", () => {
    assert.throws(() => withdraw(100, 500), {
      name: "Error",
      message: "Not enough money",
    });
  });

  it("refuses negative amounts", () => {
    assert.throws(() => withdraw(100, -5), RangeError);
  });

  it("refuses amounts that aren't numbers", () => {
    assert.throws(() => withdraw(100, "fifty"), /must be a number/);
  });
});
```

The second argument describes the error you expect. There are three handy ways to write it:

You pass...	It checks that...
An object, like <code>{ name: "Error", message: "Not enough money" }</code>	the error has exactly those properties
An error type, like <code>RangeError</code>	the error is that kind of error
A regular expression, like <code>/must be a number/</code>	the message matches the pattern (chapter 37)

Notice the `() =>` in front of each call. You're handing `assert.throws` a function *to* call, so it can catch the error itself. Forgetting the arrow is a classic mistake (see Common mistakes).

Testing async code

What about code that waits, like the async functions from [chapter 32](#)? Make the test function `async`, and `await` inside it. The test runner waits for it to finish.

Here's a pretend library catalog that takes a moment to answer:

```
// catalog.js
const books = {
  "978-0261103344": { title: "The Hobbit", author: "J.R.R. Tolkien" },
  "978-0441172719": { title: "Dune", author: "Frank Herbert" },
};

export async function findBook(isbn) {
  // Pretend we're asking a server, which takes a moment
  await new Promise((resolve) => setTimeout(resolve, 50));
  const book = books[isbn];
  if (!book) {
    throw new Error(`No book with ISBN ${isbn}`);
  }
  return book;
}
```

```
// catalog.test.js
import { test } from "node:test";
import assert from "node:assert/strict";
import { findBook } from "../catalog.js";

test("finds a book by its ISBN", async () => {
  const book = await findBook("978-0261103344");
  assert.equal(book.title, "The Hobbit");
});

test("rejects an unknown ISBN", async () => {
  await assert.rejects(findBook("000"), {
    message: "No book with ISBN 000",
  });
});
```

`assert.rejects` is the async version of `assert.throws`. It returns a promise, so it needs its own `await`, or the test finishes before the check happens.

Fakes with `mock.fn()`

Some code does things you don't want in a test: sending an email, charging a card, talking to a server. You can't send a real receipt every time you run your tests.

The trick is to pass that job into your function as a parameter. Then, in the test, hand it a **mock**: a fake function that does nothing except remember how it was called.

```
// checkout.js
export function checkout(cart, sendReceipt) {
  const total = cart.reduce((sum, item) => sum + item.price, 0);
  if (total > 0) {
    sendReceipt(`Thanks! You paid ${total}`);
  }
  return total;
}
```

```
// checkout.test.js
import { test, mock } from "node:test";
import assert from "node:assert/strict";
import { checkout } from "../checkout.js";

test("sends one receipt with the total", () => {
  const sendReceipt = mock.fn(); // a fake that remembers every call

  checkout([
    { price: 12 },
    { price: 8 }
  ], sendReceipt);

  assert.equal(sendReceipt.mock.callCount(), 1);
  assert.deepEqual(sendReceipt.mock.calls[0].arguments, ["Thanks! You paid $20"]);
});

test("sends no receipt for an empty cart", () => {
  const sendReceipt = mock.fn();
  checkout([], sendReceipt);
  assert.equal(sendReceipt.mock.callCount(), 0);
});
```

- `mock.fn()` makes the fake. `sendReceipt.mock.callCount()` tells you how many times it was called.
- `sendReceipt.mock.calls[0].arguments` is an array of the arguments from the first call.
- If the fake needs to give something back, pass it a function: `mock.fn(() => 0.2)` returns `0.2` every time it's called.

In the real app, you'd call `checkout(cart, sendEmail)` with a function that really sends emails. Passing a job into a function like this also makes code more flexible, just like the strategies in [chapter 44](#).

Red, green, refactor

So far, you've written the code first and the tests after. **Test-driven development** (TDD) flips that around. You work in small loops of three steps:

1. **Red:** write a test for something your code can't do yet. Run it and watch it fail.
2. **Green:** write the *simplest* code that makes it pass.

3. **Refactor:** tidy up the code (chapter 45), while the tests make sure nothing breaks.

Let's try one loop for a music player that shows track lengths like `1:05`. First the test, in `track.test.js`:

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { formatTrackTime } from "./track.js";

test("formats 65 seconds as 1:05", () => {
  assert.equal(formatTrackTime(65), "1:05");
});
```

Red. Create `track.js` with an empty function, `export function formatTrackTime(seconds) {}`, and run `node -test`. It fails, as it should. For strings, the failure shows a small "diff": a line starting with `+` for what your code gave (`+ undefined`), and a line starting with `-` for what the test expected (`- '1:05'`).

Green. Now the simplest code that passes:

```
export function formatTrackTime(seconds) {
  const minutes = Math.floor(seconds / 60);
  const rest = seconds % 60;
  return `${minutes}:${String(rest).padStart(2, "0")}`;
}
```

Run it again: a green tick. Next, add tests for the edges: `600` should be `"10:00"`, `9` should be `"0:09"`, and a track that's `125.6` seconds long should round to `"2:06"`. Run them, and the last one goes red:

```
✗ rounds to the nearest second (1.6563ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:
    + actual - expected

    + '2:5.5999999999999994'
    - '2:06'
      ^
```

The `^` points at the first character that's different. And there's the decimal surprise from chapter 05: `125.6 % 60` isn't exactly `5.6`. Round first, and it goes green. Then **refactor**, with clearer names and no magic number:

```
const SECONDS_PER_MINUTE = 60;

export function formatTrackTime(seconds) {
  const totalSeconds = Math.round(seconds);
  const minutes = Math.floor(totalSeconds / SECONDS_PER_MINUTE);
  const remainingSeconds = totalSeconds % SECONDS_PER_MINUTE;
  return `${minutes}:${String(remainingSeconds).padStart(2, "0")}`;
}
```

All four tests still pass, so you know the tidy-up didn't break anything. That's the loop: red, green, refactor, and repeat.

Why write the test first? Because it makes you decide exactly what the code should do before you write it. And you see the test fail, so you know it can really catch a problem. You don't have to use TDD for everything, but it's a great way to build tricky logic.

Coverage: which lines did your tests run?

Coverage tells you which lines of your code were run by your tests, and which weren't. Node can measure it with a flag:

```
node --test --experimental-test-coverage
```

Say `bank.test.js` only tested a normal withdrawal and the "not enough money" error. You'd see this at the end of the output:

```
i start of coverage report
i -----
i file      | line % | branch % | funcs % | uncovered lines
i -----
i bank.js   | 66.67  | 60.00    | 100.00  | 3-4 6-7
i -----
i all files | 66.67  | 60.00    | 100.00  |
i -----
i end of coverage report
```

- **line %:** how many lines ran at least once.
- **branch %:** how many paths through your `if` s were taken. An `if` has two paths: the condition was true, or it was false.
- **funcs %:** how many functions were called.
- **uncovered lines:** the lines no test ever ran. Lines 3–4 and 6–7 of `bank.js` are the "not a number" and "not above 0" errors, so those rules have no tests yet.

The test files themselves are left out of the report. "Experimental" means the feature is still being worked on, so the output may look a little different in future Node versions.

Watch out: 100% coverage doesn't mean bug-free. It only means every line *ran*, not that every result was *checked*. Use coverage to find the parts you forgot to test, not as a score to chase.

What to test (and what not to)

Do test:

- Your own logic: calculations, rules, and decisions (prices, discounts, grades, "can this member borrow a book?").
- Edge cases and bad input.
- Every bug you fix. Write a test that fails because of the bug, then fix it. The bug can never quietly come back.

Don't bother testing:

- Other people's code. JavaScript's `Math.round` and well-known libraries already have their own tests.
- Code with no logic at all, like a function that only returns a fixed value.
- *How* a function does its job on the inside. Test what goes in and what comes out, so you're free to refactor the inside.

And keep each test **independent**: it sets up its own data and doesn't rely on another test running first. That's another reason to avoid shared state, like the singletons in chapter 44.

Other test tools

Node's built-in runner is a great place to start. Many projects use **Vitest** or **Jest** instead: popular test tools you install with npm ([chapter 50](#)). They use the same `describe` and `it` idea, with a slightly different way of writing checks:

```
expect(addTax(50, 0.2)).toBe(60); // Vitest and Jest style
```

There are also tools like Playwright and Cypress that test whole web pages by clicking buttons like a real user. Whatever tool you use later, the ideas from this chapter carry straight over.

Common mistakes

1. A test file that never runs

```
cart.js
cartTests.js    <- your tests are in here
package.json
```

You run `node --test` and get:

```
i tests 0
i suites 0
i pass 0
i fail 0
...
```

No errors and no failures, so it looks fine. But zero tests ran, because `cartTests.js` doesn't match any of the name patterns. Rename it to `cart.test.js`. Always glance at the `tests` count to check that your tests really ran.

2. Using `equal` for arrays and objects

```
test("builds a user", () => {
  assert.equal({ name: "Mia" }, { name: "Mia" });
});
// AssertionError [ERR_ASSERTION]: Values have same structure but are not reference-equal:
```

The two objects look the same, but they're two different objects, and `equal` compares like `===`. Use `assert.deepEqual` to compare what's *inside* arrays and objects.

3. Calling the function inside `assert.throws`

```
test("refuses to go below zero", () => {
  assert.throws(withdraw(100, 500), { message: "Not enough money" });
});
// Error: Not enough money
```

The test fails with the very error it was meant to expect. `withdraw(100, 500)` runs *before* `assert.throws` even starts, so the error flies straight out of the test. Pass a function instead, so `assert.throws` can call it and catch the error: `assert.throws(() => withdraw(100, 500), ...)`.

4. Forgetting `await` in an async test

```
test("rejects an unknown ISBN", () => {
  assert.rejects(findBook("978-0261103344"), { message: "No book with ISBN 000" });
});
```

This ISBN exists, so `findBook` doesn't reject, and the test *should* fail. Put it in a file called `noawait.test.js` (with the usual imports at the top) and run it. Instead of a clear failure, you get a confusing mix: a green tick for the test, then a failure for the whole file:

```
✓ rejects an unknown ISBN (0.9089ms)
i Error: Test "rejects an unknown ISBN" at noawait.test.js:5:1 generated asynchronous activity
after the test ended. ...
X noawait.test.js (306.1462ms)
```

Without `await`, the test finished before the check did. The fix: make the test `async` and write `await` `assert.rejects(...)`. The same goes for any async function you call in a test: `await` it.

5. Tests that depend on each other

```
const cart = []; // shared by every test in this file

test("adds the first item", () => {
  cart.push("apple");
  assert.equal(cart.length, 1);
});

test("adds a second item", () => {
  cart.push("pear");
  assert.equal(cart.length, 2);
});
```

Both pass when you run the whole file. But run only the second one, with `node --test --test-name-pattern="second"`, and it fails with `1 !== 2`. It was secretly relying on the first test's apple. Each test should build its own data: move `const cart = []`; inside each test.

Quick recap

- A test is code that checks your code. Automated tests catch bugs early, make refactoring safe, and stop fixed bugs from coming back.
- Node has a test runner built in: `import { test } from "node:test"` and `import assert from "node:assert/strict"`, in a folder whose `package.json` has `"type": "module"`.
- Run `node --test`. It finds files like `name.test.js` and anything in a `test` folder. Read failures from the top: which test, what was expected, and which line.
- Structure each test as arrange, act, assert. Test the normal case and the edge cases.
- Use `equal` for simple values, `deepEqual` for arrays and objects, `assert.throws(() => ...)` for errors, and `await assert.rejects(...)` in `async` tests.

- `mock.fn()` makes a fake that records its calls. `--experimental-test-coverage` shows which lines your tests never ran.
 - TDD is a loop: red (a failing test), green (the simplest code that passes), then refactor.
-

Next: try the [exercises](#), then move on to [47 Debugging](#).

Exercises

How to do these:

- Give each exercise its own folder inside this chapter's folder (`ex1` , `ex2` , and so on). `node --test` searches the whole folder it runs in, so separate folders keep each exercise's tests apart.
 - In each exercise folder, add a `package.json` containing `{ "type": "module" }` (you need it for `import` and `export` , as in [chapter 29](#)). Then put the code file and its `.test.js` file next to it.
 - Open a terminal in the exercise folder and run `node --test` .
 - The times in brackets, like `(0.6ms)` , will be different on your computer. Everything else should match.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Weather station

A weather station's app has two small helpers. Save them as `weather.js` :

```
export function celsiusToFahrenheit(celsius) {  
  return (celsius * 9) / 5 + 32;  
}  
  
export function isFreezing(celsius) {  
  return celsius <= 0;  
}
```

Write `weather.test.js` with four tests, using exactly these names:

1. `converts 100°C to 212°F`
2. `converts -40°C to -40°F` (the one temperature where both scales agree!)
3. `says 0°C is freezing`
4. `says 5°C is not freezing`

Expected output:

```
✓ converts 100°C to 212°F (1.2721ms)
✓ converts -40°C to -40°F (0.3768ms)
✓ says 0°C is freezing (0.192ms)
✓ says 5°C is not freezing (0.1673ms)
i tests 4
i suites 0
i pass 4
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 297.3724
```

Now **break the code on purpose**, to check that your tests can catch a bug: in `isFreezing`, change `<=` to `<` and run the tests again. Exactly one test should fail:

```
✗ says 0°C is freezing (1.4286ms)
...
AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

false !== true
```

Change it back when you're done. A test you've never seen fail is a test you can't fully trust.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): The shipping bug

An online shop promises these shipping prices on its website:

Order total	Shipping
Under \$20	\$4.99
From \$20 up to \$49.99	\$2.99
\$50 or more	Free

Here's the code that's supposed to follow those rules. Save it as `shipping.js`, and **don't fix anything yet**, even if you spot a problem:

```
export function shippingCost(orderTotal) {
  if (orderTotal > 50) {
    return 0;
  }
  if (orderTotal > 20) {
    return 2.99;
  }
  return 4.99;
}
```

Write `shipping.test.js` with six tests, based on the table, not on the code:

1. charges \$4.99 for a \$10 order
2. charges \$4.99 for a \$19.99 order
3. charges \$2.99 for a \$20 order
4. charges \$2.99 for a \$49.99 order
5. ships a \$50 order for free
6. ships an \$80 order for free

Run them. You should see two failures:

```
✓ charges $4.99 for a $10 order (1.6334ms)
✓ charges $4.99 for a $19.99 order (0.2781ms)
✗ charges $2.99 for a $20 order (1.581ms)
✓ charges $2.99 for a $49.99 order (0.3555ms)
✗ ships a $50 order for free (0.4045ms)
✓ ships an $80 order for free (0.2168ms)
i tests 6
i suites 0
i pass 4
i fail 2
```

Read the "failing tests" part of the output to see what each wrong answer was. Then fix `shippingCost` so all six pass (`i pass 6, i fail 0`).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Username rules

A game's sign-up form cleans up usernames and refuses bad ones. Save this as `username.js`:

```
export function cleanUsername(input) {
  if (typeof input !== "string") {
    throw new TypeError("Username must be text");
  }
  const name = input.trim().toLowerCase();
  if (name.length < 3 || name.length > 15) {
    throw new RangeError("Username must be 3 to 15 characters");
  }
  if (name.includes(" ")) {
    throw new Error("Username can't contain spaces");
  }
  return name;
}
```

In `username.test.js`, write a `describe("cleanUsername", ...)` group with these eight tests. Choose your own test values, except where one is given:

1. trims and lowercases a valid name (use `" SkyWalker "`)
2. accepts exactly 3 characters
3. accepts exactly 15 characters
4. refuses anything that isn't text (use the number `42`)
5. refuses names that are too short (use `"a1"`)
6. refuses names that are too long
7. refuses spaces in the middle
8. checks the length after trimming (use `" ab "`: it's 6 characters long, but only 2 after trimming)

Expected output:

```
► cleanUsername
  ✓ trims and lowercases a valid name (1.9361ms)
  ✓ accepts exactly 3 characters (0.1945ms)
  ✓ accepts exactly 15 characters (0.1289ms)
  ✓ refuses anything that isn't text (0.465ms)
  ✓ refuses names that are too short (0.5325ms)
  ✓ refuses names that are too long (0.1789ms)
  ✓ refuses spaces in the middle (0.1492ms)
  ✓ checks the length after trimming (0.1456ms)
✓ cleanUsername (4.941ms)
i tests 8
i suites 1
i pass 8
i fail 0
```

Rule: use each of the three ways to describe an expected error at least once: an object with `name` and/or `message`, an error type like `RangeError`, and a regular expression.

When they all pass, weaken one rule on purpose: change `name.length < 3` to `name.length < 2`. Which of your tests fail now? You should see two of them, each with a message starting `Missing expected exception` (the rest depends on how you described the error). Then change it back.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Coffee order with a fake card machine

A coffee shop's app charges customers through a payment service. You don't want to charge a real card every time you run your tests, so you'll hand the code a fake one.

Save this as `orders.js`:

```
export async function placeOrder(items, payments) {
  if (items.length === 0) {
    throw new Error("Your order is empty");
  }
  const total = items.reduce((sum, item) => sum + item.price, 0);
  try {
    const receipt = await payments.charge(total);
    return `Paid ${total.toFixed(2)} (receipt ${receipt.id})`;
  } catch (error) {
    throw new Error(`Payment failed: ${error.message}`, { cause: error });
  }
}
```

`payments` is an object with a `charge(amount)` method. In the real app, it talks to the bank and resolves with a receipt like `{ id: "R-1001" }`. In your tests, `charge` will be a mock.

Start `orders.test.js` with this test data:

```
const coffee = { name: "Flat white", price: 3.5 };
const muffin = { name: "Blueberry muffin", price: 4 };
```

Then write a `describe("placeOrder", ...)` group with three tests:

1. charges the card once, with the total: ordering the coffee and the muffin returns `"Paid $7.50 (receipt R-1001)"`, and `charge` was called exactly once, with `7.5`.
2. explains why a payment failed: when `charge` rejects with an error saying `Card declined`, `placeOrder` rejects with the message `Payment failed: Card declined`.
3. refuses an empty order without charging: an empty order rejects with `Your order is empty`, and `charge` was never called.

Expected output:

```
▶ placeOrder
  ✓ charges the card once, with the total (2.1386ms)
  ✓ explains why a payment failed (0.8516ms)
  ✓ refuses an empty order without charging (0.3044ms)
✓ placeOrder (5.5439ms)
i tests 3
i suites 1
i pass 3
i fail 0
```

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Test-drive a playlist

Build a `Playlist` class ([chapter 27](#)) for a music app, **test first**. This time there's no code to start from: you write each test, watch it fail, then write just enough code to make it pass.

Songs are plain objects like `{ title: "Heroes", seconds: 371 }`. Start `playlist.test.js` with these three songs:

```
const dancingQueen = { title: "Dancing Queen", seconds: 231 };
const hereComesTheSun = { title: "Here Comes the Sun", seconds: 185 };
const heroes = { title: "Heroes", seconds: 371 };
```

Here's what a `Playlist` must do. Each line is one test, inside a `describe("Playlist", ...)` group. Use exactly these test names:

Test name	What it checks
starts empty	<code>new Playlist("Road Trip").songCount</code> is <code>0</code>
adds songs	after adding two songs, <code>songCount</code> is <code>2</code>
refuses the same song twice	adding a song with the same title again throws <code>Song already in playlist: Dancing Queen</code>
adds up the total length	with Dancing Queen and Here Comes the Sun, <code>totalLength</code> is <code>"6:56"</code>
plays songs in order and wraps around	with Dancing Queen and Heroes, calling <code>next()</code> three times gives <code>"Dancing Queen", "Heroes", "Dancing Queen"</code>

Test name	What it checks
removes a song by its title	after <code>remove("Dancing Queen")</code> , <code>songCount</code> goes down by one
refuses to remove a song that isn't there	<code>remove("Heroes")</code> on an empty playlist throws <code>Song not found: Heroes</code>
has nothing to play when it's empty	<code>next()</code> on an empty playlist returns <code>null</code>

Work through the table one row at a time: **red** (write the test and see it fail), **green** (make it pass with the simplest code), **refactor** (tidy up, and run again). Keep the songs private with a `#songs` field, and make `songCount` and `totalLength` getters.

When all eight pass, run the tests with coverage:

```
node --test --experimental-test-coverage
```

Expected output (the coverage table is at the end):

```

▶ Playlist
  ✓ starts empty (1.1618ms)
  ✓ adds songs (0.3063ms)
  ✓ refuses the same song twice (1.1869ms)
  ✓ adds up the total length (0.2586ms)
  ✓ plays songs in order and wraps around (0.8402ms)
  ✓ removes a song by its title (0.2772ms)
  ✓ refuses to remove a song that isn't there (0.2335ms)
  ✓ has nothing to play when it's empty (0.1582ms)
✓ Playlist (6.0955ms)
i tests 8
i suites 1
i pass 8
i fail 0
...
i file      | line % | branch % | funcs % | uncovered lines
i -----
i playlist.js | 100.00 |   100.00 |   100.00 |

```

If your coverage isn't 100%, look at the uncovered lines. Either they need a test, or they're code you don't need yet (YAGNI, [chapter 45](#)).

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Before you move on

Tests tell you *that* something is broken, and roughly where. But when a test fails and you stare at the code and still can't see why, what do you do next?

[Chapter 47](#) is all about that: finding bugs like a detective, with the tools that pause your code and let you look inside it. 🔍

47 Debugging

What is it?

Debugging is finding out *why* your program doesn't do what you expected, and then fixing it.

A **bug** is any mistake that makes a program behave wrongly: a crash, a wrong answer, a button that does nothing.

Why does it matter?

Everyone writes bugs. Beginners do, and so do developers with twenty years of experience. What changes with experience isn't how many bugs you write. It's how quickly you find them.

Without a method, debugging means staring at the code, changing random things, and hoping. That can eat a whole evening. With a method and the right tools, most bugs take minutes. That's what this chapter gives you.

A fun fact about the word: in 1947, a team working on the Harvard Mark II computer (it included Grace Hopper, one of the pioneers of programming) found a real moth stuck in one of the machine's parts. They taped it into their logbook with the note "First actual case of bug being found." Engineers were already calling faults "bugs" back then, which is why they found it funny. The logbook page, moth and all, is now at the Smithsonian's National Museum of American History.

Real-world example

Debugging is detective work. A good detective doesn't guess who did it. They follow a method, and so can you:

A detective...	When you debug, you...
Reads the report of what happened	Read the error message and the stack trace
Visits the scene and works out exactly what happened	Reproduce the bug: make it happen again, on purpose
Narrows down the list of suspects	Isolate the bug: find the exact function, line, and value that goes wrong
Checks every alibi instead of trusting stories	Check your assumptions with <code>console.log</code> and breakpoints
Catches the real culprit, not just a bystander	Fix the cause, not just the symptom
Makes sure the case is really closed	Verify the fix, and add a test so it can't come back

How it works

A method for every bug

Here are five steps that work for almost any bug:

1. **Read the error.** If there's an error message, it tells you *what* went wrong and the stack trace tells you *where* ([chapter 18](#)). If there's no error, just a wrong result, write down exactly what you expected and exactly what you got.
2. **Reproduce it.** Find the input or the steps that make the bug happen every time. If you can't make it happen on purpose, you can't be sure you've fixed it.
3. **Isolate it.** Narrow it down: which function? Which line? Which value is wrong? The tools in this chapter are all for this step.
4. **Fix the cause, not the symptom.** If a total shows `NaN`, hiding the `NaN` on the screen isn't a fix. Find out where the `NaN` came from.
5. **Verify.** Run the same steps again and check the bug is gone. Then check that the things around it still work. Best of all, write a test for it ([chapter 46](#)), so this bug can never sneak back.

The method in action

A furniture shop shows the average star rating of each product:

```
const products = [
  { name: "Lamp", ratings: [4, 5, 3] },
  { name: "Desk", ratings: [] },
  { name: "Chair", ratings: [5, 4] },
];

function averageRating(product) {
  const total = product.ratings.reduce((sum, rating) => sum + rating, 0);
  return total / product.ratings.length;
}

for (const product of products) {
  console.log(`${product.name}: ${averageRating(product).toFixed(1)} stars`);
}
```

You'll see:

```
Lamp: 4.0 stars
Desk: NaN stars
Chair: 4.5 stars
```

Let's follow the five steps.

1. **Read.** There's no error message, but "NaN stars" is clearly wrong. We expected something sensible for the desk.
2. **Reproduce.** It happens every time, and only for the desk. What's different about the desk? It has no ratings yet.
3. **Isolate.** Log the values inside `averageRating`, just before the division. For the desk, you'll see:

```
{ name: 'Desk', total: 0, count: 0 }
```

So the function works out `0 / 0`, which is `NaN` ([chapter 05](#)).

4. **Fix the cause.** A product with no ratings doesn't *have* an average. That's the real problem, so handle it where the average is worked out:

```
function averageRating(product) {  
  if (product.ratings.length === 0) {  
    return null; // no ratings yet: there is no average  
  }  
  const total = product.ratings.reduce((sum, rating) => sum + rating, 0);  
  return total / product.ratings.length;  
}  
  
for (const product of products) {  
  const average = averageRating(product);  
  const label = average === null ? "no ratings yet" : `${average.toFixed(1)} stars`;  
  console.log(`${product.name}: ${label}`);  
}
```

Hiding the text "NaN" at the printing step would only have fixed the symptom: every other part of the shop that uses `averageRating` would still get `NaN`.

5. **Verify.** Run it again:

```
Lamp: 4.0 stars  
Desk: no ratings yet  
Chair: 4.5 stars
```

The desk is fixed, and the lamp and chair still show the same numbers as before.

That's the whole method. The rest of this chapter is about tools that make step 3, isolating the bug, much faster.

Log smarter: labels, tables, and deep objects

`console.log` is the simplest debugging tool there is, and professionals use it every day. A few habits make it much more useful.

Label everything. Three bare numbers in a row tell you nothing:

```
const price = 5;
const quantity = 3;
const discount = 0.1;

console.log(price, quantity, discount); // prints: 5 3 0.1
console.log({ price, quantity, discount }); // prints: { price: 5, quantity: 3, discount: 0.1 }
```

Wrapping the variables in `{ }` builds a quick object with shorthand properties ([chapter 11](#)), so every value is printed with its name. It's the fastest labeled log there is.

`console.table` shows an array of objects as a table:

```
const cart = [
  { item: "Coffee", price: 4, qty: 2 },
  { item: "Bagel", price: 3, qty: 1 },
  { item: "Juice", price: 5, qty: 3 },
];

console.table(cart);
```

You'll see:

(index)	item	price	qty
0	'Coffee'	4	2
1	'Bagel'	3	1
2	'Juice'	5	3

A wrong price or a missing value jumps out much faster in a table than in a wall of curly braces.

`console.dir` **for deeply nested objects.** Node's `console.log` only shows objects a couple of levels deep. After that, it prints `[Object]` instead of the details:

```
const order = {
  id: 1042,
  customer: { name: "Ana", address: { city: "Lisbon", street: { name: "Rua Augusta", number: 12 } } },
};

console.log(order);
```

You'll see:

```
{
  id: 1042,
  customer: { name: 'Ana', address: { city: 'Lisbon', street: [Object] } }
}
```

That `[Object]` doesn't mean the data is missing. Node just stopped printing. To see everything, use `console.dir` with `{ depth: null }` ("no depth limit"):

```
console.dir(order, { depth: null });
```

You'll see:

```
{
  id: 1042,
  customer: {
    name: 'Ana',
    address: { city: 'Lisbon', street: { name: 'Rua Augusta', number: 12 } }
  }
}
```

In the browser, you can click to open nested objects instead, so `[Object]` isn't a problem there. But `console.dir` is still handy in the browser: `console.log(element)` shows a page element as HTML, while `console.dir(element)` shows it as an object, with all its properties.

More console tools

`console.group` indents everything until the matching `console.groupEnd()`, so related messages stay together:


```
console.group("Order 1042");
console.log("Checking stock...");
console.group("Payment");
console.log("Card accepted");
console.groupEnd();
console.log("Order confirmed");
console.groupEnd();
console.log("Next order");
```

You'll see:

```
Order 1042
  Checking stock...
  Payment
    Card accepted
  Order confirmed
Next order
```

`console.count` counts how many times a line has run. It's perfect for questions like "why does this run three times when I clicked once?"

```
function addToCart(item) {
  console.count("addToCart called");
}

addToCart("tea");
addToCart("cake");
addToCart("tea");
```

You'll see:

```
addToCart called: 1
addToCart called: 2
addToCart called: 3
```

`console.time` and `console.timeEnd` measure how long the code between them takes. Give both the same label:

```
console.time("build report");
let lines = 0;
for (let i = 0; i < 1000000; i++) {
  lines += 1;
}
console.timeEnd("build report"); // prints something like: build report: 3.516ms
```

Your number will be different every time. [Chapter 48](#) uses this to make code faster.

`console.trace` prints the stack trace from where it's called, *without* crashing. Use it to answer "who called this function?":

```
function applyDiscount(price) {
  console.trace("applyDiscount was called");
  return price * 0.9;
}

function checkout(cartTotal) {
  return applyDiscount(cartTotal);
}

checkout(50);
```

You'll see (with your own file paths):

```
Trace: applyDiscount was called
  at applyDiscount (C:\...\47-debugging\checkout.js:2:11)
  at checkout (C:\...\47-debugging\checkout.js:7:10)
  at Object.<anonymous> (C:\...\47-debugging\checkout.js:10:1)
  ...
```

`console.assert` checks something that should be true, and only prints when it *isn't*:

```
const stock = 3;
const ordered = 5;
console.assert(ordered <= stock, `Ordered ${ordered}, but only ${stock} in stock`);
console.log("The program keeps going");
```

You'll see:

```
Assertion failed: Ordered 5, but only 3 in stock
The program keeps going
```

Unlike the `assert` in tests ([chapter 46](#)), `console.assert` doesn't stop anything. It's a quiet alarm you can leave in while you hunt.

`console.warn` and `console.error` work like `console.log`, but mark the message as a warning or an error. In the terminal they look the same as `log`. In the browser console, warnings are yellow and errors are red, and you can filter the console to show only those.

Tool	Use it to...
<code>console.log({ a, b })</code>	print values together with their names
<code>console.table(list)</code>	see an array of objects as a table
<code>console.dir(obj, { depth: null })</code>	see every level of a nested object (in Node)
<code>console.group("label") ... console.groupEnd()</code>	keep related messages together
<code>console.count("label")</code>	count how many times a line runs
<code>console.time("label") ... console.timeEnd("label")</code>	measure how long something takes
<code>console.trace("message")</code>	see how the code got here, without crashing
<code>console.assert(condition, "message")</code>	complain only when something that should be true isn't
<code>console.warn()</code> , <code>console.error()</code>	mark warnings and problems

Watch out: in Chrome and Edge, a logged object isn't a frozen snapshot. The console reads it again when you click to open it. If your code changed the object in between, you'll see the *new* values, not the ones from when you logged it. When you're hunting a mutation bug ([chapter 16](#)), log a copy instead:

```
console.log(structuredClone(cart)) .
```

Reading stack traces in depth

[Chapter 18](#) showed you how to read a stack trace: the message says *what* went wrong, the first `at` line says *where*, and the other `at` lines say *how the code got there*. Here are four more things that help in real programs.

1. Start from your own code. A real trace also contains Node's own code (lines starting with `node:internal`), and in bigger projects, code written by other people that you've installed (paths containing `node_modules`; you'll install your first packages in [chapter 50](#)). The bug is almost always in *your* code, so scan down for the first line that points to one of your files, and start there.

2. Named functions make clearer traces. Here's a crash inside an arrow function that has no name:

```
const orders = [
  { id: 1, items: ["tea", "cake"] },
  { id: 2 },
];

const itemCounts = orders.map((order) => order.items.length);
console.log(itemCounts);
```

The trace starts like this:

```
TypeError: Cannot read properties of undefined (reading 'length')
    at C:\...\47-debugging\orders.js:6:54
    at Array.map (<anonymous>)
    at Object.<anonymous> (C:\...\47-debugging\orders.js:6:27)
```

The first `at` line has only a location, because the callback has no name. `Array.map (<anonymous>)` is the built-in `map`, calling your callback. Now give the callback a name:

```
function countItems(order) {
  return order.items.length;
}

const orders = [
  { id: 1, items: ["tea", "cake"] },
  { id: 2 },
];

const itemCounts = orders.map(countItems);
console.log(itemCounts);
```

```
TypeError: Cannot read properties of undefined (reading 'length')
    at countItems (C:\...\47-debugging\orders.js:2:22)
    at Array.map (<anonymous>)
    at Object.<anonymous> (C:\...\47-debugging\orders.js:10:27)
```

Now the trace tells you the function's name, which is much easier to follow in a big file. Short arrow callbacks are fine, but it's one more reason to give bigger jobs a named function.

3. `async` functions keep their callers. When an error happens after an `await`, Node still shows which `async` function was waiting for it:

```
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function fetchUser(id) {
  await wait(100); // pretend to ask a server
  throw new Error(`User ${id} not found`);
}

async function loadDashboard() {
  const user = await fetchUser(42);
  console.log(`Welcome, ${user.name}`);
}

loadDashboard();
```

```
Error: User 42 not found
    at fetchUser (C:\...\47-debugging\dashboard.js:7:9)
    at async loadDashboard (C:\...\47-debugging\dashboard.js:11:16)
```

The word `async` in `at async loadDashboard` means "this function was waiting on an `await` when the error came back" ([chapter 32](#)).

4. The crash line isn't always the bug line. This is the most important one:

```
function createUser(name) {
  return { nmae: name, points: 0 };
}

function greet(user) {
  return `Hello, ${user.name.toUpperCase()}!`;
}

const user = createUser("ana");
console.log(greet(user));
```

```
TypeError: Cannot read properties of undefined (reading 'toUpperCase')
    at greet (C:\...\47-debugging\greet.js:6:30)
    at Object.<anonymous> (C:\...\47-debugging\greet.js:10:13)
```

The trace points at `greet`, but there's nothing wrong with `greet`. The mistake is the typo `nmae` in `createUser`, and `createUser` isn't even in the trace: it had already finished when the crash happened.

A stack trace shows where the program *noticed* the problem, which isn't always where the problem was *made*. So ask two questions:

1. **Which value is wrong?** The message says it tried to read `toUpperCase` of `undefined`, so `user.name` is `undefined`.
2. **Where did that value come from?** Follow it backwards. Log the whole `user`:

```
console.log({ user }); // prints: { user: { nmae: 'ana', points: 0 } }
```

There's the typo. Fix it where the bad value was made, in `createUser`, not where it crashed.

Timers make this even more obvious. By the time a `setTimeout` callback runs, the function that set the timer has long finished ([chapter 40](#) explains why), so the trace only shows the callback:

```
function remindMe(task) {
  setTimeout(() => {
    console.log(`Reminder: ${task.title.toUpperCase()}`);
  }, 100);
}

function planDay() {
  remindMe({ title: "stretch" });
  remindMe({ name: "drink water" });
}

planDay();
```

You'll see:

```
Reminder: STRETCH
C:\...\47-debugging\reminders.js:3
  console.log(`Reminder: ${task.title.toUpperCase()}`);
                                ^
TypeError: Cannot read properties of undefined (reading 'toUpperCase')
    at Timeout._onTimeout (C:\...\47-debugging\reminders.js:3:41)
    at listOnTimeout (node:internal/timers:605:17)
    at process.processTimers (node:internal/timers:541:7)
```

`planDay` and `remindMe` are nowhere in the trace. The real bug, `name` instead of `title` on line 9, is something you have to find by following the bad value back to where it came from.

Breakpoints: pause and look around

At the end of chapter 46, a failing test could tell you *that* something was broken, but not *why*. This is where you find out why.

`console.log` shows you the values you *thought* to print. A **debugger** is a tool that pauses your program while it runs, so you can look at *every* value at that moment. A **breakpoint** is the spot where you tell it to pause. It's like pausing a football replay and moving forward frame by frame to see exactly who touched the ball.

You can set a breakpoint in two ways:

- In the tool itself: click next to a line number in the browser's DevTools or in VS Code (both shown below). This is the usual way.
- In your code: write the `debugger` statement.

```
let total = 0;
for (const price of [4, 6]) {
  debugger; // pauses here, but only when a debugger is attached
  total += price;
}
console.log(total); // prints: 10
```

With no debugger attached (like a plain `node run`), `debugger;` does nothing at all, and the program prints `10`. With DevTools open, or in VS Code's debugger, it pauses on that line.

Once your program is paused, you control it with four buttons:

Button	What it does	Chrome and Edge	VS Code
Resume (Continue)	Run on until the next breakpoint	F8	F5
Step over	Run this line, then pause on the next one	F10	F10
Step into	If this line calls a function, go inside it and pause on its first line	F11	F11
Step out	Finish the current function, and pause back where it was called	Shift + F11	Shift + F11

Think of following a recipe that says "make the sauce (see page 12)". **Step over** makes the sauce without reading page 12, and moves on to the next step. **Step into** turns to page 12 and follows the sauce recipe line by line. **Step out** finishes the sauce recipe and takes you back to the main recipe.

Most of the time you'll step over. Step into when you suspect the function being called. If you're not sure which button is which, hover over it to see its name and its shortcut in your version.

Debugging in the browser: the Sources panel

Let's debug a real page. You met this bug in [chapter 22](#), which makes it perfect for learning the tools: you already know the answer, so you can watch how the debugger reveals it. Make a folder with these two files:

index.html :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Tip calculator</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <label for="bill">Bill ($)</label>
    <input id="bill" type="number" value="40">
    <label for="tip">Tip ($)</label>
    <input id="tip" type="number" value="5">
    <button id="calculate">Calculate total</button>
    <p id="result"></p>
  </body>
</html>
```

script.js :

```
const button = document.querySelector("#calculate");

button.addEventListener("click", () => {
  const bill = document.querySelector("#bill").value;
  const tip = document.querySelector("#tip").value;
  const total = bill + tip;
  document.querySelector("#result").textContent = `Total: ${total}`;
});
```

Open `index.html` and click the button. It says `Total: $405` instead of `Total: $45`. Now let's catch it in the act:

1. Press `F12` and open the **Sources** tab. (If you can't see it, click `>>` to find more tabs.)
2. In the panel on the left, find `script.js` and click it to open it. (In bigger projects, `Ctrl + P` finds a file by name.)
3. Click the line number next to `const total = bill + tip;`. A blue marker appears: that's your breakpoint.

4. Click **Calculate total** on the page. The page freezes and shows **Paused in debugger**, and the line is highlighted. The code has stopped *just before* running that line.
5. Hover your mouse over `bill` in the code. It shows `"40"`, in quotes. The **Scope** section on the right shows the same: `bill: "40"` and `tip: "5"`. Those quotes are the clue: both values are strings.
6. Press **Step over** (`F10`). The line runs, and Scope now shows `total: "405"`. Two strings were glued together.
7. Press **Resume** (`F8`). Fix the code with `Number()` as in chapter 22, save, and refresh the page.

While you're paused, the right-hand side has more to show you:

- **Scope** lists every variable the paused line can see, grouped by where they live: its own variables at the top, the outer ones further down. It's the scope chain from [chapter 14](#), laid out for you.
- **Watch** lets you type any expression, like `typeof bill`, and updates it every time you step.
- **Call Stack** is the stack of plates from [chapter 17](#) at this exact moment. Click a line in it to see what the calling function was doing.
- The **Console** works while you're paused, and it can see the paused line's variables. Type `typeof bill` and press `Enter` to get `'string'`.

Conditional breakpoints save a lot of clicking. If a loop runs 500 times and only one item goes wrong, right-click the line number, choose **Add conditional breakpoint...**, and type a condition like `item.price < 0`. The code pauses only when the condition is true.

Logpoints come from the same menu. **Add logpoint...** prints a message every time the line runs, without pausing and without editing your file.

To remove a breakpoint, click its marker again. Menus move around a little between browser versions, but all of these features have been in Chrome and Edge for years.

Three more DevTools panels

Console. Errors show up in red, with the file name and line number on the right. Click that link to jump straight to the line in the Sources panel. The dropdown at the top lets you choose which levels to show, so you can hide everything except errors and warnings.

Network. Every request the page makes shows up here: the HTML, the CSS, images, and your `fetch` calls ([chapter 33](#)). It only records while DevTools is open, so open it first, then reload the page or click your button.

- Click **Fetch/XHR** to show only the data requests.
- Check the **Status** column. `200` means OK. Failures like `404` (not found) or `500` (server error) show up in red.
- Click a request to see its **Headers** (the full URL and the method), and what the server sent back under **Response** or **Preview**.

Remember that `fetch` doesn't treat a `404` as an error. The Network tab shows you what really happened. And while DevTools is open, ticking **Disable cache** makes the browser load fresh copies of your files, which helps when a change "doesn't show up".

Elements. This is the live page, the DOM from [chapter 20](#). Use it to check your assumptions: is the element really there? Does it have the class you think it has, right now? The **Styles** pane shows which CSS rules apply to it, and crossed-out rules are the ones being overridden. Two more tricks:

- Click an element, then type `$0` in the Console. `$0` is the element you selected, so `$0.textContent` shows its text.
- Right-click an element and choose **Break on**, then **attribute modifications**. Your script pauses at the exact line that changes it, which is great for "who keeps removing this class?"

Debugging Node in VS Code

VS Code has a debugger built in, and it works the same way as DevTools. The easiest way to use it is the **JavaScript Debug Terminal**:

1. Open the terminal panel. Click the small arrow next to the `+` button and choose **JavaScript Debug Terminal**. (Or press `Ctrl + Shift + P` to open the Command Palette and run **Debug: JavaScript Debug Terminal**.)
2. Set a breakpoint: click just to the left of a line number in your file. A red dot appears.
3. In the debug terminal, run your file as usual: `node app.js`.
4. VS Code pauses on the breakpoint and highlights the line. The **Run and Debug** view on the left shows **Variables**, **Watch**, **Call Stack**, and **Breakpoints**, and a small toolbar appears with Continue, Step Over, Step Into, Step Out, Restart, and Stop.

The debugger attaches to whatever you run with `node` in that terminal, including `node --test` from chapter 46. So when a test fails, put a breakpoint in the function it tests, run the tests in the debug terminal, and look inside.

VS Code also has conditional breakpoints and logpoints: right-click the red dot (or the space next to a line number) to find them.

Two other ways in:

- **Run and Debug:** with your file open, press `F5` (or click **Run and Debug** in the left sidebar), and pick **Node.js** if VS Code asks. It runs the current file with the debugger attached.
- **Chrome for Node:** run `node --inspect-brk app.js` in a normal terminal, open `chrome://inspect` in Chrome, and click **inspect** under your script. `--inspect-brk` means "turn on the debugger, and pause on the first line".

VS Code renames things now and then. If a menu item here doesn't match yours, search for it in the Command Palette.

When you're stuck

Explain it to a rubber duck. Go through your code line by line, and explain out loud what each line does. The listener can be a friend, Claude, or a rubber duck on your desk. The trick is called **rubber duck debugging**, after a story in the book *The Pragmatic Programmer*. Saying "...and then this line adds the tip to the bill" out loud is very often the moment you hear the mistake.

Search by halves. When you have no idea where the problem is, check the middle. In a program made of steps, like this sketch, put a log halfway through:

```
const orders = loadOrders();
const paidOrders = removeUnpaid(orders);
const withTotals = addTotals(paidOrders);
console.log({ count: withTotals.length, first: withTotals[0] }); // already wrong here?
const sorted = sortByDate(withTotals);
const report = buildReport(sorted);
```

If the value is already wrong at that point, the bug is in the first half. If not, it's in the second half. Then check the middle of *that* half, and so on. Like the guessing game where each guess cuts the possible numbers in half, even a 1,000-line program gets down to one line in about ten checks. This is called **binary search**.

Check your assumptions. Most bugs hide behind something you're *sure* about. "This is definitely a number." Is it? Log `typeof price`. "This function definitely runs." Does it? Put a `console.count` in it. "The list definitely has three items." Log its `length`. Each check takes seconds, and the thing you're most sure about is often the thing that's wrong. Old favorites: form values are strings (chapter 22), `sort` changes the array (chapter 13), and a function with no `return` gives back `undefined` (chapter 09).

Change one thing at a time. If you change five things and it starts working, you don't know which change fixed it, or whether the other four broke something else. Change one thing, run it, look. Undo the changes that didn't help.

Take a break. It sounds too simple, but it works. After a walk, or a night's sleep, a bug that beat you for an hour often takes two minutes.

Find the change that broke it. If it worked yesterday and doesn't today, the bug is in something that changed since then. If you use Git (it'll get its own folder later), `git bisect` does the "search by halves" for you through your saved versions, and finds the exact change that broke it.

Common bug families

Most bugs belong to a handful of families. Once you know the symptom, you know where to look:

Family	What you notice	What to check
Off-by-one	A missing first or last item, an extra <code>undefined</code> , a loop that runs once too often	<code><</code> vs. <code><=</code> , indexes start at 0, the last index is <code>length - 1</code> , the end of <code>slice</code> isn't included (chapter 10)
Reading a property of <code>undefined</code>	<code>TypeError: Cannot read properties of undefined (reading '...')</code>	Which value is <code>undefined</code> , and why: a misspelled property, a <code>find</code> that found nothing, a missing <code>return</code> , an index that's out of range
Async timing	<code>Promise { <pending> }, [object Promise]</code> , things happening in the wrong order	A missing <code>await</code> , an <code>await</code> inside <code>forEach</code> , code after a <code>setTimeout</code> running first (chapter 32 , chapter 40)
Accidental mutation	Data that changes "by itself", an undo button that doesn't undo	A function changing what it was given: <code>sort</code> , <code>push</code> , <code>splice</code> , or setting a property on a shared object (chapter 16)
Typos	<code>ReferenceError: ... is not defined</code> , or a quiet <code>undefined</code>	Spelling and capital letters, in variable names <i>and</i> in property names

The exercises give you bugs from each family to hunt down.

Asking for help well

Sometimes you need a second pair of eyes: a colleague, a forum, or Claude. A good question gets a good answer fast. Include:

1. **What you expected, and what happened instead.** Copy the exact error message or output. Don't retype it from memory, because one changed word can send your helper the wrong way.
2. **A minimal reproduction:** the smallest piece of code that still shows the bug. Delete everything that isn't needed, then run it again to check the bug is still there.
3. **What you've already tried**, so nobody suggests it again.
4. **Your versions**, when they might matter: the output of `node --version`, or which browser you use.

A minimal reproduction is often the whole answer. Say your 300-line shop program crashes whenever the cart is empty. Cut it down step by step, and you might end up with this:

```
const prices = [];
const total = prices.reduce((sum, price) => sum + price);
// TypeError: Reduce of empty array with no initial value
```

Two lines. By the time you've cut the code down, you've very often found the bug yourself (here, it's the missing starting value from [chapter 13](#)). If you haven't, your helper can see the problem in seconds instead of reading 300 lines.

Hard to answer	Easy to answer
"My shop is broken, can you help?"	"My cart total crashes when the cart is empty."
"I get some error about reduce."	"I get <code>TypeError: Reduce of empty array with no initial value</code> from this code: ..."
300 lines pasted in	The two-line reproduction
Nothing about what you tried	"I logged <code>prices.length</code> just before, and it's <code>0</code> ."

Common mistakes

1. Not reading the error message

```
C:\...\47-debugging\cart.js:6
console.log(`Total: ${totl}`);
                        ^
ReferenceError: totl is not defined
```

It's tempting to panic and start changing things. But read it: the message names the problem (`totl` doesn't exist), and the line and the `^` show exactly where. It's a typo for `total`. Error messages are often the whole answer, so always read them first, slowly, from the top.

2. Fixing the symptom instead of the cause

```
function averageRating(product) {
  const total = product.ratings.reduce((sum, rating) => sum + rating, 0);
  return total / product.ratings.length || 0; // "fixed" it!
}
// the desk now prints: Desk: 0.0 stars
```

The `NaN` is gone, but now the shop tells customers the desk has *zero stars*, which is a new bug and a worse one. `|| 0` hid the symptom without asking where it came from. Fix the cause: decide what "no ratings" should mean, like in "The method in action" above.

3. Running old code

```
> node receipt.js
Total: $NaN
```

You've fixed the bug, but it still prints exactly the same thing? Before you doubt your fix, check the boring things. Did you save the file? (VS Code shows a dot on the tab of an unsaved file, and **File** → **Auto Save** saves for you.) Is the terminal in the right folder, running the right file? In the browser, did you refresh? If a change still doesn't show up, do a hard reload with `Ctrl + F5`.

4. Ignoring a log that never printed

```
const scores = [72, 88, 95];

for (let i = 0; i > scores.length; i++) {
  console.log("checking score", scores[i]);
}
console.log("done");
// prints: done
```

You added a log to watch the loop, and it printed nothing. That isn't a failed experiment: no output is a clue. It means that code never ran. Here the condition `i > scores.length` is `false` from the very start, so the loop body never runs. Fix: `i < scores.length`.

5. Leaving debugging code behind

```
function checkout(cart) {
  debugger;
  console.log("HERE!!!", cart);
  return cart.length;
}
```

A forgotten `debugger` pauses the app for anyone who has DevTools open, and forgotten logs clutter the console. Once the bug is fixed, remove them. Search your files for `debugger` and `console.log` before you call it done. Linters like ESLint ([chapter 50](#)) can warn you about leftovers.

Quick recap

- Everyone writes bugs. Follow the method: read the error, reproduce, isolate, fix the cause (not the symptom), and verify, ideally with a test.
- Label your logs (`console.log({ total })`), and use the rest of the console too: `table`, `dir`, `group`, `count`, `time`, `trace`, and `assert`.
- In a stack trace, start from the first line in *your* code, and remember that the crash line isn't always the bug line: follow the bad value back to where it came from.

- Breakpoints pause your code so you can see every variable and step over, into, and out. Use the Sources panel in the browser, and the JavaScript Debug Terminal in VS Code.
 - The Network tab shows what really happened to your requests, and the Elements panel shows the live page.
 - Stuck? Explain it to a rubber duck, search by halves, check your assumptions, and change one thing at a time.
 - When you ask for help, include the exact error and a minimal reproduction.
-

Next: try the [exercises](#), then move on to [48 Performance](#).

Exercises

How to do these:

- Each exercise gives you a buggy program. Copy it into a file in this folder (`ex1.js` , `ex2.js` , and so on) and run it with `node ex1.js` .
 - Don't just stare at the code. Use the method from the notes: **read the error, reproduce, isolate, fix, verify**. Try `console.log` , `console.table` , or a breakpoint in VS Code's JavaScript Debug Terminal.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your fix.
-

Exercise 1 (Easy): The average that isn't a number

A teacher wants the average test score, but the program prints `NaN` .

```
// Average test score
const scores = [90, 75, 81];
let total = 0;
for (let i = 0; i <= scores.length; i++) {
  total += scores[i];
}
console.log("Average:", total / scores.length);
```

It prints:

```
Average: NaN
```

When it's fixed, you should see:

Average: 82

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Read the stack trace

A shop looks up a customer's email, but the program crashes.

```
// Look up a customer's email
const customers = [
  { name: "Sam", email: "sam@example.com" },
  { name: "Priya", email: "priya@example.com" },
];

function findEmail(name) {
  const customer = customers.find((c) => c.name === name);
  return customer.email;
}

console.log("Email:", findEmail("sam"));
```

Node prints (paths shortened):

```
C:\...\ex2.js:9
  return customer.email;
                ^
TypeError: Cannot read properties of undefined (reading 'email')
```

First, answer in your own words: **which variable was undefined, and on which line?** Then find out *why*, and fix it so the search works no matter how the name is capitalized:

```
Email: sam@example.com
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): The price that changed by itself

A restaurant adds a \$1 service fee for delivery orders. But now the restaurant's own menu is wrong too.


```
// Add a $1 service fee for the delivery menu
const menu = [
  { name: "Soup", price: 5 },
  { name: "Pasta", price: 12 },
];

function withServiceFee(items) {
  const result = items;
  for (const item of result) {
    item.price = item.price + 1;
  }
  return result;
}

const deliveryMenu = withServiceFee(menu);
console.log("Restaurant soup:", menu[0].price);
console.log("Delivery soup:", deliveryMenu[0].price);
```

It prints:

```
Restaurant soup: 6
Delivery soup: 6
```

When it's fixed, you should see:

```
Restaurant soup: 5
Delivery soup: 6
```

Rule: `withServiceFee` must not change the `menu` it's given.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Use the debugger

This program should print a score and its double, but something is off.

```
// Load a player's score, then show it doubled
function wait(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

async function loadScore() {
  await wait(100); // pretend this is a slow server
  return 42;
}

async function main() {
  const score = loadScore();
  console.log("Score:", score);
  console.log("Double:", score * 2);
}

main();
```

It prints:

```
Score: Promise { <pending> }
Double: NaN
```

Rule: find the bug with a **breakpoint**, not `console.log`. Open a JavaScript Debug Terminal in VS Code, click in the margin next to the `console.log("Score: "...` line to set a breakpoint, run `node ex4.js`, and look at `score` in the Variables panel.

When it's fixed, you should see:

```
Score: 42
Double: 84
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Four bugs at the gym

A gym wants a small report: the member with the most visits, everyone who earned a reward (10 or more visits), and a check that the original list wasn't changed. This program has **4 bugs**.

```
// Gym report: the top member, and who gets a reward (10 or more visits)
const members = [
  { name: "Asha", visits: 12 },
  { name: "Ben", visits: 3 },
  { name: "Chen", visits: 25 },
  { name: "Dana", visits: 10 },
];

function sortByVisits(list) {
  return list.sort((a, b) => a.visits - b.visits);
}

function rewardNames(list) {
  const names = [];
  for (let i = 1; i < list.length; i++) {
    if (list[i].visits > 10) {
      names.push(list[i].name);
    }
  }
  return names;
}

const ranked = sortByVisits(members);
console.log("Top member:", ranked[0].name);
console.log("Rewards:", rewardNames(members).join(", "));
console.log("First in the original list:", members[0].name);
```

It prints:

```
Top member: Ben
Rewards: Asha, Chen
First in the original list: Ben
```

When all 4 bugs are fixed, you should see:

```
Top member: Chen
Rewards: Asha, Chen, Dana
First in the original list: Asha
```

Fix one bug at a time and run the program after each fix. Notice how fixing one bug can reveal another one that was hiding.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

You've now fixed bugs from four families: off-by-one, reading a property of `undefined`, accidental mutation, and async timing. Next, in [chapter 48](#), the code works fine, but it's *slow*. You'll learn how to find out why.

48 Performance

What is it?

Performance is how quickly your program does its work, and how smooth it feels to the people using it.

Making code faster is called **optimization**. This chapter shows you how to do it the smart way: find the slow part first, then fix just that part.

Why does it matter?

People notice slow software. A page that takes ages to load, a button that freezes for a second, a list that stutters when you scroll: users get annoyed and leave.

Your computer is probably faster than your users' devices, too. Code that feels instant on your laptop can crawl on a cheap phone.

But here's the surprising part: most of your code is already fast enough. Usually, one small part of a program causes most of the slowness. Speeding up everything else is wasted effort, and it often makes the code harder to read.

Changing code to make it faster before you know it's slow is called **premature optimization**. The computer scientist Donald Knuth famously called it "the root of all evil". So the golden rule of this chapter is: **measure first, then optimize**.

Real-world example

A supermarket keeps getting complaints: the checkout is too slow. The manager can guess what's wrong, or measure it.

At the supermarket	In your code
Customers complain that checkout is slow	Users say your app feels slow
The manager guesses, and hires faster cashiers. Nothing changes.	You rewrite code you <i>think</i> is slow. Nothing changes.
The manager times each step with a stopwatch: scanning takes 20 seconds, but the card reader takes 2 minutes	You time your code with <code>console.time</code> or <code>performance.now()</code>
The card reader is the bottleneck : the slowest step, which holds everything else up	One slow loop, function, or request holds up the whole app

At the supermarket	In your code
They replace the card reader, then time the checkout again	You fix the slow part, then measure again to check it helped

The word "bottleneck" comes from the neck of a bottle. However big the bottle is, the water can only pour out as fast as the narrow neck allows. Making the rest of the bottle bigger doesn't help.

How it works

Step 1: Measure first

Remember `console.time` and `console.timeEnd` from [chapter 47](#)? They're the quickest way to time a piece of code. Give both the same label:

```
console.time("adding up");

let total = 0;
for (let i = 1; i <= 10_000_000; i++) {
  total += i;
}

console.timeEnd("adding up");
// prints something like: adding up: 12.467ms
```

`10_000_000` is the same number as `10000000`. JavaScript ignores the underscores. They're only there to make big numbers easier to read, like the commas in 10,000,000.

When you want the time as a number (to compare two runs, or to show it on a page), use `performance.now()`. It gives you the current time in milliseconds, with decimals, counted from when your page or program started. Read it before and after, and subtract:

```
const start = performance.now();

let total = 0;
for (let i = 1; i <= 10_000_000; i++) {
  total += i;
}

const ms = performance.now() - start;
console.log(`Adding up took ${ms.toFixed(1)} ms`);
// prints something like: Adding up took 8.6 ms
```

`performance.now()` works the same in the browser and in Node. Why not `Date.now()` from [chapter 19](#)? It only counts whole milliseconds, and it follows the computer's clock, which can jump when the clock gets adjusted. `performance.now()` is more precise and only ever moves forward.

Watch out: every timing in this chapter is "roughly". Your numbers will be different from mine, and they'll change a little every time you run the code. What matters is the *comparison*: is A much faster than B?

Measuring fairly

A **benchmark** is a test that times some code, so you can compare it with something else. Benchmarks are easy to get wrong. Watch what happens when you time the same function five times in a row:

```
function addUp(count) {
  let total = 0;
  for (let i = 1; i <= count; i++) {
    total += i;
  }
  return total;
}

for (let run = 1; run <= 5; run++) {
  const start = performance.now();
  addUp(1_000_000);
  console.log(`Run ${run}: ${(performance.now() - start).toFixed(2)} ms`);
}
```

You'll see something like this (your numbers will be different):

```
Run 1: 7.52 ms
Run 2: 2.75 ms
Run 3: 2.32 ms
Run 4: 0.78 ms
Run 5: 0.79 ms
```

The first run is usually the slowest. The JavaScript engine is still getting to know your code: once it notices a function running a lot, it optimizes it. This is called **warm-up**. So for a fair benchmark:

- **Run it several times**, and don't trust the first run on its own.
- **Use realistic amounts of data.** With 10 items, everything is fast. Test with as much data as your real users will have.
- **Time only the part you care about.** Keep `console.log` out of the timed part, because printing is slow.
- **Compare on the same machine**, and remember your users' devices may be slower than yours.

The browser's tools: Performance tab and Lighthouse

For web pages, Chrome and Edge have two tools built into DevTools (`F12`). The tabs and buttons move around a little between browser versions, but the ideas stay the same.

The Performance tab records everything the browser does. Click the record button (a circle), do the slow thing on your page (click the button, scroll the list), then click **Stop**. You get a timeline. Wide blocks are where the time went, and long tasks are marked in red. Click a block to see which of your functions it was.

Lighthouse checks a whole page for you. In the **Lighthouse** tab, click **Analyze page load**. After a short wait, you get a score out of 100 and a list of suggestions, like "properly size images" or "reduce unused JavaScript".

Tip: Run Lighthouse in a private window (Incognito in Chrome, InPrivate in Edge). Browser extensions run code on every page, and they can spoil your results.

Big O: how the work grows

Timing tells you how fast your code is *today*, with *today's* data. But data grows. Your app has 100 users this month and 100,000 next year. Will it still be fast?

Big O is a way of describing how the amount of work grows as the data grows. It doesn't give you seconds. It gives you the *shape* of the growth. The letter **n** stands for the size of the data: the number of items in an array, songs in a playlist, users in a list.

Here are the four shapes you'll meet most often.

O(1), "constant time": the work stays the same, however much data there is. It's like opening locker number 3 at the gym. You walk straight to it, whether the gym has 50 lockers or 5,000.

```
const lockers = ["towel", "shoes", "water bottle", "gym bag"];
console.log(lockers[2]); // prints: water bottle
```

Reading an array item by its index, `push`, `pop`, `set.has()`, and `map.get()` are all O(1).

O(n), "linear time": the work grows in step with the data. It's like looking for lost keys in a cinema by checking every seat. Twice as many seats, twice the work.

A loop over the array is O(n). So are `includes`, `indexOf`, `find`, `filter`, `map`, and `reduce`, because they all loop over the array behind the scenes.

O(n²), "quadratic time": for every item, you look at every other item. It's like a party where everyone shakes hands with everyone else. 10 people make 45 handshakes. 100 people make 4,950. Ten times the people, about a hundred times the handshakes.


```
function hasDuplicateName(guests) {
  for (let i = 0; i < guests.length; i++) {
    for (let j = i + 1; j < guests.length; j++) { // a loop inside a loop
      if (guests[i] === guests[j]) {
        return true;
      }
    }
  }
  return false;
}

console.log(hasDuplicateName(["Mia", "Leo", "Ava", "Leo"])); // prints: true
```

A loop inside a loop over the same data is the classic sign of $O(n^2)$.

$O(\log n)$, "logarithmic time": every step throws away half of what's left. Think of the guessing game "I'm thinking of a number from 1 to 100", where the only answers are "too high" and "too low". The smart move is to always guess the middle:

Your guess	Answer	Numbers still possible
50	Too high	1 to 49
25	Too low	26 to 49
37	Correct!	

You never need more than 7 guesses for 1 to 100, and never more than 20 for 1 to 1,000,000. Looking up a name in a paper phone book works the same way: open it in the middle, pick the right half, repeat. The catch is that the data must be **sorted**. This way of searching is called **binary search**, and you'll write one in the exercises.

Here's how many steps each shape takes as the data grows:

n (data size)	$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$
10	1	about 3	10	100
1,000	1	about 10	1,000	1,000,000
1,000,000	1	about 20	1,000,000	1,000,000,000,000

Look at the bottom-right corner: a million million steps. That's the difference between an app that works and one that freezes.

Big O skips small details. The party makes about *half* of n^2 handshakes, but it's still called $O(n^2)$, because what matters is the shape: double the guests, and you get about four times the handshakes.

Watch out for hidden loops. `includes`, `indexOf`, `find`, and `filter` look like one small step, but each one is a loop. Put one inside another loop and you've written a loop inside a loop: $O(n^2)$.

Pick the right tool for lookups: Set and Map

Picking the right way to store your data is often the biggest speed-up of all. Here's a real example: a concert with 100,000 valid ticket codes. At the gate, 10,000 fans scan their tickets, and each code gets checked.

You could check with `array.includes`, or with a `Set` from [chapter 35](#). Let's measure both:

```
// 100,000 valid ticket codes: "T0", "T1", ... "T99999"
const validTickets = [];
for (let i = 0; i < 100_000; i++) {
  validTickets.push(`T${i}`);
}
const validTicketSet = new Set(validTickets);

// 10,000 fans scan their tickets at the gate
const scanned = [];
for (let i = 0; i < 10_000; i++) {
  scanned.push(`T${i * 10}`);
}

function countValid(isValid) {
  let valid = 0;
  for (const code of scanned) {
    if (isValid(code)) valid++;
  }
  return valid;
}

console.time("array.includes");
countValid((code) => validTickets.includes(code));
console.timeEnd("array.includes");

console.time("set.has");
countValid((code) => validTicketSet.has(code));
console.timeEnd("set.has");
```

You'll see something like this (your numbers will be different):

```
array.includes: 1.157s
set.has: 2.429ms
```

When a timing goes over 1,000 ms, Node switches to seconds, so `1.157s` means more than a second. The Set was hundreds of times faster.

Why? `includes` walks along the array, comparing one code after another: $O(n)$ for every single fan. A Set is organized so it can jump straight to the right spot, a bit like the numbered lockers: $O(1)$. Maps work the same way for looking things up by a key.

You want to...	With an array	With a Set or Map
Check "is this in here?"	<code>array.includes(x) : $O(n)$</code>	<code>set.has(x) : $O(1)$</code>
Get the item with a certain id	<code>array.find(...) : $O(n)$</code>	<code>map.get(id) : $O(1)$</code>

Two honest notes:

- **For a handful of items, it doesn't matter.** Checking a T-shirt size against `["S", "M", "L", "XL"]` is instant either way, so use whatever reads best.
- **Building a Set takes time too**, because it has to look at every item once. It pays off when you check many times, like 10,000 fans at the gate.

Don't do the same work twice

Move work out of loops. A teacher wants the scores that are above the class average:

```
const scores = [72, 88, 95, 60, 81];

// Slow: works out the average again for every single score
const slow = scores.filter(
  (score) => score > scores.reduce((sum, s) => sum + s, 0) / scores.length
);

// Fast: works out the average once, before filtering
const average = scores.reduce((sum, s) => sum + s, 0) / scores.length;
const fast = scores.filter((score) => score > average);

console.log(slow); // prints: [ 88, 95, 81 ]
console.log(fast); // prints: [ 88, 95, 81 ]
```

Both give the right answer. But in the slow version, the average never changes, yet `reduce` works it out again for every score. That's a hidden loop inside a loop: $O(n^2)$. With 20,000 scores, the slow version took about 1.5 seconds on my machine, and the fast one about 1 ms.

Build a lookup once. A shop wants to print each order with the customer's name:

```

const customers = [
  { id: 1, name: "Asha" },
  { id: 2, name: "Ben" },
  { id: 3, name: "Chen" },
];
const orders = [
  { id: 101, customerId: 2 },
  { id: 102, customerId: 3 },
  { id: 103, customerId: 2 },
];

// Slow: find searches the customer list again for every order
for (const order of orders) {
  const customer = customers.find((c) => c.id === order.customerId);
  console.log(`Order ${order.id}: ${customer.name}`);
}

```

`find` is a hidden loop, so this is $O(\text{orders} \times \text{customers})$. Instead, build a Map from id to customer once, then look each customer up in $O(1)$:

```

// Fast: build a Map once...
const customersById = new Map();
for (const customer of customers) {
  customersById.set(customer.id, customer);
}

// ...then every lookup is O(1)
for (const order of orders) {
  const customer = customersById.get(order.customerId);
  console.log(`Order ${order.id}: ${customer.name}`);
}

```

Both versions print the same three lines, starting with `Order 101: Ben`. But with 20,000 customers and 50,000 orders, the `find` version took roughly a second on my machine, and the Map version about 8 ms.

Keeping a result you've worked out, so you don't have to work it out again, is called **caching**. The Map above is a cache. The next idea takes caching one step further.

Memoization: remember your results

In [chapter 43](#), you met **memoization**: a function keeps a cache of its answers, so when it's asked the same question again, it answers straight from the cache.

Here's where it really shines. A fitness app wants to know how many different ways you can climb a staircase if you take either 1 or 2 steps at a time. Your last move onto step 4 came either from step 3 (a small step) or from

step 2 (a big step). So the ways to reach step 4 are the ways to reach step 3 plus the ways to reach step 2. That's a job for recursion ([chapter 17](#)):

```
function countWays(steps) {
  if (steps <= 1) {
    return 1; // 0 or 1 steps left: only one way
  }
  return countWays(steps - 1) + countWays(steps - 2);
}

console.log(countWays(4)); // prints: 5
console.log(countWays(35)); // prints: 14930352
```

It works, but it does a huge amount of work. When I counted, `countWays(35)` made the function run 29,860,703 times, which took about 110 ms. And it gets worse fast: `countWays(45)` took over 15 seconds on my machine.

Why so many? The same questions get asked over and over: `countWays(33)` gets worked out twice, `countWays(32)` three times, `countWays(31)` five times, and it gets worse all the way down.

Now let's remember each answer in a Map:

```
const waysCache = new Map();

function countWaysFast(steps) {
  if (steps <= 1) {
    return 1;
  }
  if (waysCache.has(steps)) {
    return waysCache.get(steps); // asked before: reuse the answer
  }
  const ways = countWaysFast(steps - 1) + countWaysFast(steps - 2);
  waysCache.set(steps, ways);
  return ways;
}

console.log(countWaysFast(35)); // prints: 14930352
```

Same answer, but the function ran just 69 times, and took well under a millisecond. Even `countWaysFast(45)` answers in under a millisecond. And ask for `countWaysFast(35)` again, and it answers in one step, straight from the cache.

Memoization has two rules: only memoize **pure** functions (same input, same output, every time), and don't let the cache grow forever. Common mistakes 4 and 5 below show what goes wrong otherwise.

Do it less often: debounce, throttle, and `requestAnimationFrame`

Some events fire far too often: `input` on every keystroke, `scroll` dozens of times a second. You already have the tools for this from [chapter 34](#):

- **Debounce** waits until things calm down, then runs once. With `debounce(searchProducts, 300)`, typing "lamp" quickly runs one search, not four. Use it for search boxes, autosave, and form checks.
- **Throttle** runs at most once every so many milliseconds. Use it for scroll, resize, and mouse movement.

For changes you can see on the page, the browser has a helper of its own. `requestAnimationFrame(callback)` asks the browser to run your function just before it draws the next **frame**, which is one fresh picture of the page on the screen. Most screens draw about 60 frames a second (some draw more). Updating the page more often than that is wasted work, because nobody can see it.

```
let updateScheduled = false;

window.addEventListener("scroll", () => {
  if (updateScheduled) {
    return; // an update is already booked for the next frame
  }
  updateScheduled = true;
  requestAnimationFrame(() => {
    updateProgressBar(); // your own function that sets the bar's width
    updateScheduled = false;
  });
});
```

However many scroll events fire, the progress bar updates at most once per frame.

Faster pages: go easy on the DOM

When you change the page, the browser has work to do before it can show the new version:

1. **Style:** work out which CSS rules apply to each element.
2. **Layout:** work out the size and position of every element. (You'll also hear this called **reflow**.)
3. **Paint:** draw the pixels.

Browsers are clever about this. They collect your changes and do this work once, just before the next frame. Your job is to not get in their way. Three habits help.

1. Batch your changes. Build new elements off the page first, then add them all in one go. A

DocumentFragment is an invisible container made for exactly this. You fill it up, then `append` it to the page. The fragment itself disappears, and everything inside it lands in the list.

```

const list = document.querySelector("#products");
const fragment = document.createDocumentFragment();

for (const product of products) {
  const li = document.createElement("li");
  li.textContent = `${product.name}: ${product.price}`;
  fragment.append(li); // off the page: nothing to redraw yet
}

list.append(fragment); // one change to the live page

```

To be honest, modern browsers already group a lot of changes for you, so the speed-up is sometimes small. But it's a cheap habit, and it guarantees the live page changes once, not a thousand times.

2. Change a class, not lots of styles. In [chapter 20](#), you learned the tidy way to change looks: describe them in CSS, and switch classes on and off. It also means fewer changes for the browser to deal with:

```

// Four separate style changes
card.style.border = "2px solid gold";
card.style.background = "#fffbe6";
card.style.transform = "scale(1.05)";
card.style.boxShadow = "0 4px 12px rgba(0, 0, 0, 0.2)";

// One class change: the styles live in your CSS file
card.classList.add("featured");

```

The browser gets one change instead of four, and your styles stay in the CSS file, where they're easy to find and reuse.

3. Read first, then write. Some properties make the browser work out the layout *right now*, so it can give you an up-to-date answer. `offsetWidth`, `offsetHeight`, `getBoundingClientRect()`, and `scrollTop` are the common ones. Reading them is fine. The trouble starts when you mix reading and writing:

```

const boxes = [...document.querySelectorAll(".box")];

// Slow: read, write, read, write...
for (const box of boxes) {
  const width = box.offsetWidth; // read: needs an up-to-date layout
  box.style.height = `${width / 2}px`; // write: now the layout is out of date
}

```

Every write makes the layout out of date, so the next read forces the browser to work it all out again. With 500 boxes, the browser may have to do the layout 500 times instead of once. This is called **layout thrashing**.

It's like a tailor who measures you, sews one seam, measures you again, sews another seam, and so on. A good tailor takes all the measurements first, then does all the sewing:

```
const widths = boxes.map((box) => box.offsetWidth); // all the reads first

boxes.forEach((box, i) => {
  box.style.height = `${widths[i] / 2}px`; // then all the writes
});
```

Now the browser only has to work out the layout once.

Lazy loading: don't load it until it's needed

Lazy loading means loading something only when it's needed, instead of everything at the start. It's like a restaurant that makes your dessert when you're ready for it, not the moment you walk in.

Images. Add `loading="lazy"` to an image, and the browser waits to download it until it's close to scrolling into view. A visitor who never scrolls down never downloads it.

```

```

Don't lazy-load the images at the very top of the page, though. Visitors see those straight away.

Code. Remember dynamic `import()` from [chapter 29](#)? It loads a module only when your code asks for it:

```
reportButton.addEventListener("click", async () => {
  const { drawSalesChart } = await import("./charts.js"); // downloaded on the first click
  drawSalesChart(sales);
});
```

Visitors who never open the report never download the chart code. And because modules are cached, the second click doesn't download it again.

Infinite scroll. A feed that loads more posts as you reach the bottom needs to know when the bottom comes into view. You *could* check the scroll position in a `scroll` listener, but an **IntersectionObserver** is better: it watches an element for you, and tells you when it comes into view (when it "intersects" the visible part of the page). Put an empty marker element at the end of your list and watch it:


```
const bottomMarker = document.querySelector("#bottom-marker");

const observer = new IntersectionObserver((entries) => {
  if (entries[0].isIntersecting) {
    loadMorePosts(); // your own function that fetches and shows more posts
  }
});

observer.observe(bottomMarker);
```

The browser does the watching, and your code only runs when the marker actually appears.

Web Workers: heavy work on another thread

As you saw in [chapter 40](#), JavaScript runs your code on one thread, like the cafe with just one cashier. A long calculation blocks everything else: clicks, typing, scrolling, and animations all have to wait.

A **Web Worker** runs a script on a separate thread. It's like the cafe hiring a second worker for the back room: the cashier keeps serving customers while the helper crunches the numbers. The two sides talk by sending messages:

```
// main.js
const worker = new Worker("worker.js");

worker.addEventListener("message", (event) => {
  console.log("Total sales:", event.data);
});

worker.postMessage(salesFigures); // send the work to the worker
```

```
// worker.js
self.addEventListener("message", (event) => {
  const total = event.data.reduce((sum, amount) => sum + amount, 0);
  self.postMessage(total); // send the answer back
});
```

`postMessage` sends a copy of the data, and `event.data` holds whatever arrived. Inside a worker, `self` means the worker itself.

Workers can't touch the page (there's no `document` inside a worker), so they're for calculations, not for updating the DOM. Like modules, they need a local server such as Live Server. Node has its own version, called worker threads (`node:worker_threads`). And adding up a few thousand numbers doesn't need a worker: reach for one when you've *measured* a long task that freezes the page.

The network: fewer, smaller, cached requests

For most web pages, the slowest part isn't your JavaScript at all. It's waiting for data to arrive over the network. Every request is a trip to the server and back, and on a weak phone signal, one trip can take a second or more.

- **Fewer requests.** Don't ask for the same thing twice. When you need several things, ask for them all at once with `Promise.all` ([chapter 32](#)) instead of one after another.
- **Smaller requests.** Ask only for what you need. Many APIs let you ask for one page of results at a time, like `?_limit=10` on JSONPlaceholder. Shrink your images, which are usually the biggest files on a page. Build tools also shrink your code by removing spaces and shortening names, which is called **minifying** ([chapter 50](#)).
- **Cached requests.** Keep answers you already have. Browsers have their own **HTTP cache**, a store of files they've already downloaded, and the server decides how long each file may be kept there. In your own code, a Map works:

```
const cache = new Map();

async function getJSON(url) {
  if (cache.has(url)) {
    return cache.get(url); // already downloaded: no request needed
  }
  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}`);
  }
  const data = await response.json();
  cache.set(url, data);
  return data;
}

const url = "https://jsonplaceholder.typicode.com/users/1";
const user = await getJSON(url); // goes over the network
const sameUser = await getJSON(url); // comes straight from the cache
console.log(user.name); // prints: Leanne Graham
```

This uses top-level `await`, so run it as a module: put a `package.json` containing `{ "type": "module" }` next to it ([chapter 29](#)). When I timed the two calls, the first took over 100 ms and the second about 0.01 ms, because it never touched the network. Just remember that cached data can go out of date. Cache things that change slowly, like a list of countries, not things that change every second, like a live score.

Putting it all together

When something feels slow, work through these questions in order:

1. **Is it really slow?** Measure it, and find the bottleneck.

2. **Can it do less work?** Pick a better tool for the job, like a Set or Map for lookups, or binary search instead of checking every item.
3. **Is it doing the same work twice?** Move work out of loops, build lookups once, memoize.
4. **Can it run less often?** Debounce, throttle, or `requestAnimationFrame`.
5. **Can it wait?** Lazy-load it.
6. **Can it happen somewhere else?** A Web Worker, or the server.

Then **measure again** to check that your change really helped. If it didn't, undo it and keep the simpler code.

Common mistakes

1. Optimizing without measuring

```
// Before: clear and easy to read
const total = cart.reduce((sum, item) => sum + item.price, 0);

// After some "optimizing": harder to read, and no faster that anyone could notice
let sum = 0;
for (let i = 0, len = cart.length; i < len; ++i) sum += cart[i].price;
```

For a cart of 20 items, both versions take a tiny fraction of a millisecond. The code got harder to read, and nobody gained anything. Measure first, and keep the readable version until the numbers tell you otherwise.

2. A hidden loop inside a loop

```
function removeDuplicates(emails) {
  const unique = [];
  for (const email of emails) {
    if (!unique.includes(email)) { // includes is a loop too
      unique.push(email);
    }
  }
  return unique;
}
```

It gives the right answer, but `includes` searches the whole `unique` array for every email: $O(n^2)$. With 10,000 different emails, it took almost 2 seconds on my machine. A Set ([chapter 35](#)) does the same job in about a millisecond: `const unique = [...new Set(emails)];`

3. Adding to `innerHTML` in a loop

```
for (const song of playlist) {
  songlist.innerHTML += `<li>${song.title}</li>`;
}
```

Every `+=` turns the whole list back into text, adds one item, then throws away every element and rebuilds them all from that text. With 1,000 songs, the list gets rebuilt 1,000 times. It's also a security risk if a title came from a user ([chapter 51](#)). Create each item with `createElement` and `textContent`, collect them in a `DocumentFragment`, and append it once.

4. Memoizing a function that isn't pure

```
let discountPercent = 10;
const priceCache = new Map();

function discountedPrice(price) {
  if (priceCache.has(price)) {
    return priceCache.get(price);
  }
  const result = price - (price * discountPercent) / 100;
  priceCache.set(price, result);
  return result;
}

console.log(discountedPrice(50)); // prints: 45
discountPercent = 50; // the big sale starts!
console.log(discountedPrice(50)); // prints: 45 (wrong: it should be 25)
```

The cache only looks at `price`, but the answer also depends on `discountPercent`, which lives outside the function. So the cache hands out an old answer. Only memoize pure functions. Here, you could pass the discount in as a second parameter and use both values in the cache key, like ``${price}-${percent}``.

5. A cache that grows forever

```
const searchCache = new Map();

async function searchProducts(query) {
  if (!searchCache.has(query)) {
    searchCache.set(query, await fetchResults(query));
  }
  return searchCache.get(query);
}
```

Every search anyone types stays in memory for as long as the page is open. For a page that stays open all day, that's a memory leak ([chapter 41](#)). Give the cache a size limit. A `Map` remembers the order you added things, so

its first key is always the oldest one:

```
const MAX_CACHE_SIZE = 100;

function addToCache(query, results) {
  if (searchCache.size >= MAX_CACHE_SIZE) {
    const oldestQuery = searchCache.keys().next().value;
    searchCache.delete(oldestQuery); // make room by removing the oldest
  }
  searchCache.set(query, results);
}
```

Quick recap

- Measure first, with `console.time`, `performance.now()`, and the browser's Performance tab. Fix the bottleneck, then measure again.
- Big O describes how work grows as data grows: $O(1)$ and $O(\log n)$ stay fast, $O(n)$ grows steadily, and $O(n^2)$ gets out of hand quickly. Watch out for hidden loops like `includes` and `find` inside another loop.
- For lookups, a Set or Map beats searching an array, as soon as there's a lot of data.
- Don't repeat work: move it out of loops, build lookups once, and memoize pure functions (with a size limit on the cache).
- Run code less often (debounce, throttle, `requestAnimationFrame`), later (lazy loading), or somewhere else (Web Workers).
- In the DOM, batch your changes, prefer classes, and read the layout before you write to it.
- On the network, make fewer, smaller, cached requests.

Next: try the [exercises](#), then move on to [49 Node.js Basics](#).

Exercises

How to do these:

- Exercises 1 to 4 run in Node. Make a new file for each one in this folder (`ex1.js`, `ex2.js`, and so on), and run it with `node ex1.js`.
- Exercise 5 runs in the browser. Make a folder called `ex5` with an `index.html` and a `script.js` inside. Open `index.html` by double-clicking it, press `F12` and open the **Console** to see errors, and refresh the page after each change.
- Timings will be different on your machine, and a little different on every run. That's normal. Every other line of output should match exactly.

- Try on your own first. Only open a hint if you've been stuck for a while.
- When you're done, ask Claude to check your code.

Exercise 1 (Easy): A reusable stopwatch

You're tired of writing `performance.now()` twice every time you want to time something. Wrap it in a helper you can reuse.

1. Write a function `timeIt(label, fn)`. It runs `fn`, prints the label and how long it took (with one decimal place), and **returns whatever `fn` returned**.
2. Write `sumUpTo(n)`, which adds up all the whole numbers from 1 to `n` with a loop.
3. Call `sumUpTo(1_000_000)` once *without* timing it, to warm up the engine (remember "Measuring fairly" in the notes).
4. Then use `timeIt` to time `sumUpTo` for 1,000,000, then 10,000,000, then 100,000,000, and print each sum.

Expected output (your times will be different):

```
Sum to 1,000,000: 0.7 ms
500000500000
Sum to 10,000,000: 8.1 ms
50000005000000
Sum to 100,000,000: 81.7 ms
5000000050000000
```

Each size is 10 times bigger than the one before. Is each time *roughly* 10 times bigger too? Write a comment at the bottom of your file saying which Big O `sumUpTo` is, and why.

Then delete your warm-up line and run the file again a few times. What happens to the first timing?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Double the data

You don't always need a stopwatch to see how code grows. You can count the steps instead. Copy these four functions into `ex2.js`. Each one does its job and adds 1 to `steps` for every step of work.

```

let steps = 0;

// 1. Open Locker number 7
function openLocker(lockers) {
  steps++;
  return lockers[7];
}

// 2. Check every seat in the cinema for lost keys
function findLostKeys(seats) {
  for (const seat of seats) {
    steps++;
    if (seat === "keys") {
      return true;
    }
  }
  return false;
}

// 3. Every guest shakes hands with every other guest, once
function shakeAllHands(guests) {
  for (let i = 0; i < guests.length; i++) {
    for (let j = i + 1; j < guests.length; j++) {
      steps++;
    }
  }
}

// 4. A knockout tournament: after each round, half the players are left
function countRounds(players) {
  let playersLeft = players.length;
  while (playersLeft > 1) {
    steps++;
    playersLeft = Math.ceil(playersLeft / 2);
  }
}

```

Your job:

1. Write `countSteps(fn, list)`. It sets `steps` back to 0, calls `fn(list)`, and returns how many steps it took.
2. Make two lists of numbers: one with 1,000 items, and one with 2,000 items. (None of them is "keys", so `findLostKeys` has to check every seat.)
3. Print a table of the steps each function takes for both lists. Use `padEnd(14)` for the names and `padStart(12)` for the numbers, so the columns line up.

Expected output:

Function	1000 items	2000 items
openLocker	1	1
findLostKeys	1000	2000
shakeAllHands	499500	1999000
countRounds	10	11

Rule: don't change the four functions. Let them count their own steps.

Finally, look at what happens to each row when the data doubles, and write a comment matching each function to $O(1)$, $O(\log n)$, $O(n)$, or $O(n^2)$.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Find the book, fast

A library has a million books. Their ids are stored in a sorted array, smallest first. Start with this code:

```
// Book ids: 3, 6, 9, ... up to 3,000,000 (sorted, smallest first)
const bookIds = [];
for (let id = 3; id <= 3_000_000; id += 3) {
  bookIds.push(id);
}
```

Write two search functions. Both return an object like `{ index: 833332, steps: 20 }`, where `index` is `-1` if the book isn't there.

1. `linearSearch(ids, target)` checks the ids one by one, from the start. Count one step for each id it checks.
2. `binarySearch(ids, target)` plays the guessing game from the notes. Keep track of the lowest and highest index the book could still be at. Each time round, look at the item in the middle, using `Math.floor((low + high) / 2)`, and count one step. If it's the book, you're done. If not, throw away the half where the book can't be. Keep going until there's nothing left to check.

Then search for book 2499999, book 3, and book 1000000 (which isn't in the library, because 1,000,000 isn't a multiple of 3).

Expected output:


```
Book 2499999
  linear search: index 833332, steps: 833333
  binary search: index 833332, steps: 20
Book 3
  linear search: index 0, steps: 1
  binary search: index 0, steps: 19
Book 1000000
  linear search: index -1, steps: 1000000
  binary search: index -1, steps: 20
```

Notice that linear search wins for book 3, because it's right at the start. But it's a lucky case: binary search never needs more than 20 steps, wherever the book is.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Speed up the sales report

An online shop runs this sales report every night, and it's getting slower as the shop grows. Copy it into `ex4.js` and run it:

```

// --- Test data (don't change this part) ---
const products = [];
for (let i = 1; i <= 10_000; i++) {
  products.push({ id: i, name: `Product ${i}`, price: (i % 50) + 1 });
}

const orders = [];
for (let i = 1; i <= 50_000; i++) {
  orders.push({ id: i, productId: ((i * 7) % 8_000) + 1, quantity: (i % 3) + 1 });
}

const discontinuedIds = [];
for (let id = 1; id <= 10_000; id += 4) {
  discontinuedIds.push(id);
}

// --- The slow report ---
console.time("report");

let revenue = 0;
let discontinuedSold = 0;
for (const order of orders) {
  const product = products.find((p) => p.id === order.productId);
  revenue += product.price * order.quantity;
  if (discontinuedIds.includes(order.productId)) {
    discontinuedSold += order.quantity;
  }
}

const orderedIds = [];
for (const order of orders) {
  if (!orderedIds.includes(order.productId)) {
    orderedIds.push(order.productId);
  }
}
const neverOrdered = products.filter((p) => !orderedIds.includes(p.id));

console.log(`Revenue: ${revenue}`);
console.log(`Discontinued items sold: ${discontinuedSold}`);
console.log(`Products never ordered: ${neverOrdered.length}`);
console.timeEnd("report");

```

You'll see something like this (your time will be different):

```
Revenue: $2549990
Discontinued items sold: 25001
Products never ordered: 2000
report: 864.513ms
```

Rewrite the report part so it gets the same answers much faster. Your version should be at least 10 times faster.

Rules:

- The first three lines of output must stay exactly the same.
- Don't change the test data.
- Keep everything you build (Maps, Sets) *inside* the timed part, so the comparison is fair.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A fast product search

An online shop has 2,000 products and a search box. The first version searched and redrew the whole list on every single keystroke, and it felt sluggish on cheap phones. Build a faster one.

Make `ex5/index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Product Search</title>
  <script src="script.js" defer></script>
  <style>
    body { font-family: sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
    input { width: 100%; padding: 0.5rem; font-size: 1rem; box-sizing: border-box; }
    #status { color: #555; }
  </style>
</head>
<body>
  <h1>Product search</h1>
  <input id="search" type="search" placeholder="Search 2000 products..." autocomplete="off">
  <p id="status"></p>
  <ul id="results"></ul>
</body>
</html>
```

Start `ex5/script.js` with this product data:

```
const colors = ["red", "blue", "green", "black", "white"];
const items = ["lamp", "chair", "desk", "mug", "rug", "shelf", "clock", "vase"];

const products = [];
for (let i = 1; i <= 2000; i++) {
  const color = colors[i % colors.length];
  const item = items[i % items.length];
  products.push({ id: i, name: `${color} ${item} ${i}`, price: (i % 90) + 10 });
}
```

Then build these pieces:

1. `searchProducts(query)` returns the products whose name includes `query`. Memoize it with a Map, and make it log `Searching for "desk"...` (with the real query) only when it actually has to search. Limit the cache to 50 queries, like Common mistake 5 in the notes.
2. `renderProducts(list)` shows the products in `#results`, one `<li>` each, like `blue chair 1: $11`. Only show the first 50: nobody scrolls through 2,000 items, and every element costs time to build. Build the new items off the page with a DocumentFragment, then swap them in for the old ones in one go.
3. `showResults(text)` cleans up the text (trims it and makes it lower case), searches, renders, and updates `#status`. Time the search and the render together with `performance.now()`, and show the time with one decimal place.
4. Debounce the search box by 300 ms, using your `debounce(fn, delay)` from [chapter 34](#).
5. When the page opens, show all the products (an empty search matches everything).

Check your page with these searches (your times will be different):

You type	The status line says	The first item is
(nothing: the page just opened)	Showing 50 of 2000 products (1.3 ms)	blue chair 1: \$11
desk	Showing 50 of 250 products (0.4 ms)	green desk 2: \$12
DESK, with spaces before and after it	Showing 50 of 250 products (0.2 ms)	green desk 2: \$12
mug 99	Showing 2 of 2 products (0.3 ms)	white mug 99: \$19
xyz	No products match "xyz"	(the list is empty)

Now check the cache in the console. You'll see `Searching for ""...` once when the page opens. Then type `desk` quickly, clear the box, and type `desk` quickly again: `Searching for "desk"...` should appear only once. And the spaced-out `DESK` shouldn't search at all, because it cleans up to `desk`, which is already in the cache.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

You've run lots of exercises in this course with `node`. But so far, you've mostly used Node to run code that could just as well run in a browser.

Node can do much more: read and write files, take commands from the terminal, and even be the server that a web page fetches its data from. That's [chapter 49](#). 😊

49 Node.js Basics

What is it?

Node.js (or just Node) is a program that runs JavaScript outside the browser: on your own computer, or on a server.

You've used it since [chapter 01](#) to run your files. In this chapter, you'll use the things that only Node can do: read and write files, take commands from the terminal, and run your own web server.

Why does it matter?

Browsers keep JavaScript in a **sandbox**: a closed-off, safe area. A website can't read the files on your computer or start programs, and that's a good thing.

Node has no sandbox. Your code can read and write files, start a server, and talk to databases. That's why Node is everywhere:

- **Servers and APIs.** The back end of a website, the part that runs on the server, can be written in JavaScript. An API like the ones you fetched from in [chapter 33](#) can be built with Node.
- **Command-line tools.** npm, the tool you'll meet in [chapter 50](#), is written in JavaScript and runs on Node. So are many other developer tools.
- **Desktop apps.** VS Code runs on JavaScript, using a tool called Electron, which combines Chrome's engine with Node.

And there's no new language to learn. Everything you know about JavaScript works in Node too.

Real-world example

Think of the same engine in two different cars.

A car company can put the exact same engine into a family car or a delivery van. The engine works the same way, but the controls around it are different: the family car has seats and a radio, the van has a cargo door and a big loading space.

JavaScript engines work like this. Chrome and Edge use an engine called **V8** to run JavaScript. Node uses V8 too, so your JavaScript runs the same way. What changes is what's around the engine:

	Chrome	Node.js
The engine	V8	V8 (the same one)

	Chrome	Node.js
What it's built into	A web browser	A program in your terminal
What it's connected to	A web page and the person looking at it	Your computer's files, terminal, and network
Extra tools it gives you	<code>document</code> , <code>window</code> , <code>alert</code> , <code>localStorage</code>	<code>process</code> , <code>files (fs)</code> , <code>paths (path)</code> , <code>servers (http)</code>
Built for	Showing pages to people	Running programs and servers

How it works

Same language, different surroundings

Everything from the language itself works in Node: variables, functions, arrays, objects, classes, promises, `async / await` , `timers` , `console` , and even `fetch` .

What's missing is everything that belongs to a web page. There's no page in Node, so there's no `document` , no `window` , no `alert` , and no `localStorage` :

```
console.log(typeof window);    // prints: undefined
console.log(typeof globalThis); // prints: object

console.log(document.title);
// ReferenceError: document is not defined
```

In the browser, the **global object** (the object that holds everything available everywhere) is called `window` . In Node, it's called `global` . `globalThis` is the name that works in both, so use it if your code needs to run in either place.

What Node adds instead is a set of tools for talking to your computer. You'll meet them one by one in this chapter.

Reminder: all the examples in this chapter are ES modules, so they use `import` . Put a `package.json` containing `{ "type": "module" }` in your chapter folder, like you learned in [chapter 29](#). One is enough: Node uses the nearest `package.json` it can find, looking upward from your file's folder.

Built-in modules and the `node:` prefix

Node comes with a toolbox of **built-in modules**: modules that are part of Node itself, so there's nothing to install. You import them like any other module, with `node:` in front of the name:

```
import { readFile } from "node:fs/promises";
import path from "node:path";
import http from "node:http";
```

The `node:` prefix makes it clear that this is one of Node's own modules, not a package somebody installed. You'll still see `import ... from "fs"` without the prefix in older code. It works too, but `node:` is the modern habit.

Here are the ones you'll use in this chapter:

Module	What it's for
<code>node:fs/promises</code>	Reading and writing files and folders ("fs" stands for file system)
<code>node:path</code>	Building file paths that work on every computer
<code>node:readline/promises</code>	Asking questions in the terminal
<code>node:http</code>	Running a web server
<code>node:events</code>	<code>EventEmitter</code> , Node's built-in version of the observer pattern

`process` : information about your program

`process` is a global object, so you don't need to import it. It holds information about the program that's running right now.

`process.argv` holds the **command-line arguments**: the extra words you type after the file name when you run it. Save this as `greet.js` :

```
console.log(process.argv);
```

Then run `node greet.js Sam 30` . You'll see something like this (your paths will be different):


```
[
  'C:\\Program Files\\nodejs\\node.exe',
  'C:\\Users\\you\\JavaScript\\49-nodejs-basics\\greet.js',
  'Sam',
  '30'
]
```

The first two items are always the Node program itself and your file. (The doubled backslashes are just how Node shows a single `\` inside a string, like the escape characters from [chapter 06](#).) Your own words start at index 2, so skip the first two with `slice(2)` and destructure the rest:

```
const [name, age] = process.argv.slice(2);
console.log(`Hi ${name}, you are ${age}.`);
```

Now `node greet.js Sam 30` prints `Hi Sam, you are 30`.

Watch out: arguments are always strings, just like form values ([chapter 22](#)). If you run `node add.js 2 3` and add them with `+`, you get `"23"`. Convert them with `Number()` first.

`process.env` holds the **environment variables**: named settings that live outside your program, in the terminal or on the computer. They're used for things that change from one computer to another, like which port a server uses, or a secret key that must never be written in your code ([chapter 51](#)).

```
const port = process.env.PORT ?? 3000;
console.log(`Using port ${port}`);
```

Save it as `env.js` and run `node env.js`, and you'll see `Using port 3000`, the default. To set a variable in PowerShell (the VS Code terminal on Windows), type `$env:PORT = "4000"`, then run the file again: `Using port 4000`. It stays set until you close that terminal. (On macOS and Linux, you'd write `PORT=4000 node env.js`.) Like arguments, environment variables are always strings.

`process.exit(code)` stops your program immediately. The **exit code** tells whoever started the program how it went: `0` means success, and any other number means something went wrong.

```
const [name] = process.argv.slice(2);

if (name === undefined) {
  console.error("Please give me a name, like: node hello.js Sam");
  process.exit(1);
}

console.log(`Hello, ${name}!`);
```

Run `node hello.js` without a name, and it prints the message and stops with code 1. Nothing after `process.exit` runs. Other programs and tools check this code to know whether your program worked. In PowerShell, you can see it yourself by typing `$LASTEXITCODE` after the program finishes.

`process.cwd()` returns the **current working directory**: the folder your terminal is standing in. As you'll see in a moment, that isn't always the folder your file is in.

Reading and writing files

In the browser, you saved data in `localStorage`, the wall of labeled lockers from [chapter 23](#). In Node, you get the whole file system: real files and folders that are still there after your program stops.

The functions in `node:fs/promises` return promises, so you `await` them ([chapter 32](#)). Top-level `await` works because these are modules.

```
import { writeFile, appendFile, readFile } from "node:fs/promises";

await writeFile("shopping.txt", "milk\n"); // creates the file (or replaces it)
await appendFile("shopping.txt", "bread\n"); // adds to the end
await appendFile("shopping.txt", "eggs\n");

const text = await readFile("shopping.txt", "utf8");
console.log(text.trim().split("\n")); // prints: [ 'milk', 'bread', 'eggs' ]
```

Run it, and a `shopping.txt` file appears in your folder. Open it in VS Code: it holds three lines.

- `writeFile(file, text)` creates the file, or **replaces everything in it** if it already exists.
- `appendFile(file, text)` adds to the end of the file, and creates it if it's missing.
- `readFile(file, "utf8")` reads the whole file as a string. The `"utf8"` is the **encoding**: the rule for turning the bytes stored on the disk back into text. Leave it out, and you get raw bytes instead of text (see [Common mistakes](#)).

Folders work the same way:

```
import { mkdir, writeFile, readdir } from "node:fs/promises";

await mkdir("reports", { recursive: true }); // no error if it already exists
await writeFile("reports/january.txt", "Sales: 120");
await writeFile("reports/february.txt", "Sales: 95");

const files = await readdir("reports");
console.log(files); // prints: [ 'february.txt', 'january.txt' ]
```

`readdir` gives you an array of the names in a folder. (The order comes from the operating system. On Windows it's alphabetical, but if the order matters to you, sort the array.) Without `{ recursive: true }`, `mkdir` throws an error when the folder already exists. With it, there's no error, and it also creates any missing folders along the way, like `photos/2026/june` in one go.

When something goes wrong, these functions throw errors with a `code` property that tells you what happened. The one you'll see most is `ENOENT`, short for "Error: NO ENTry", which means the file or folder doesn't exist:

```
import { readFile } from "node:fs/promises";

try {
  const settings = await readFile("settings.json", "utf8");
  console.log(settings);
} catch (error) {
  console.log(error.code); // prints: ENOENT
}
```

Paths: finding the right file

There's a trap in the code above. A **relative path** like `"shopping.txt"` doesn't mean "next to my file". It means "in the folder my terminal is standing in", which is `process.cwd()`.

Say your file lives in `JavaScript/49-nodejs-basics` and reads `"welcome.txt"` from that same folder. From inside that folder, it works. But run it from the `JavaScript` folder with `node 49-nodejs-basics/read-welcome.js`, and Node looks for `JavaScript/welcome.txt` instead:

```
Error: ENOENT: no such file or directory, open 'C:\...\JavaScript\welcome.txt'
```

The fix is to build the full path from the folder your file is in. In an ES module, that's `import.meta.dirname`, and the built-in `node:path` module joins the pieces together for you:

```
import { readFile } from "node:fs/promises";
import path from "node:path";

const welcomeFile = path.join(import.meta.dirname, "welcome.txt");
const text = await readFile(welcomeFile, "utf8");
console.log(text);
```

Now it works from any folder. `import.meta.dirname` is the full path of the folder the current file is in, like `C:\Users\you\JavaScript\49-nodejs-basics`.

Why `path.join` and not `+`? Windows separates folders with `\`, while macOS and Linux use `/`. `path.join` uses the right one for the computer it's running on, and tidies up doubled or missing slashes. Two more helpers from

`node:path` are handy:

```
import path from "node:path";

const receipt = path.join("receipts", "2026", "june.pdf");
console.log(receipt);           // prints: receipts\2026\june.pdf (on Windows)
console.log(path.basename(receipt)); // prints: june.pdf
console.log(path.extname(receipt)); // prints: .pdf
```

`basename` gives you the last part of a path (usually the file name), and `extname` gives you the **extension**, the ending that tells you what kind of file it is.

Tip: `import.meta.dirname` is fairly new (Node 20.11 and later). In older ES module tutorials, you'll see a longer trick with `fileURLToPath`, and in CommonJS files you'll see `__dirname`. They all do the same job.

Asking questions in the terminal

Arguments are great when you know everything up front. Sometimes you'd rather ask, like a form in the terminal. That's what `node:readline/promises` is for:

```
import readline from "node:readline/promises";

const rl = readline.createInterface({
  input: process.stdin, // read what the user types
  output: process.stdout, // show the questions in the terminal
});

const bill = Number(await rl.question("How much was the bill? "));
const percent = Number(await rl.question("What percent tip? "));
rl.close(); // done asking: let the program finish

const tip = bill * (percent / 100);
console.log(`Tip: ${tip.toFixed(2)}, total: ${ (bill + tip).toFixed(2) }`);
```

`process.stdin` is what you type on the keyboard, and `process.stdout` is the terminal output. `rl.question()` shows the question, waits for you to type an answer and press `Enter`, then gives you the answer as a string. Here's a run, where `42.80` and `20` are typed in:

```
How much was the bill? 42.80
What percent tip? 20
Tip: $8.56, total: $51.36
```

Don't forget `rl.close()`. Until you close `rl`, it keeps listening for more typing, so your program never finishes.

Build it: a notes app for the terminal

Time to put files, paths, and arguments together. You'll build a tiny notes app that you use like this:

```
node notes.js add Buy milk
node notes.js list
node notes.js remove 1
```

The notes are saved in a `notes.json` file next to `notes.js`, so they're still there next time. Start with loading and saving, using JSON from [chapter 23](#):

```
import { readFile, writeFile } from "node:fs/promises";
import path from "node:path";

const NOTES_FILE = path.join(import.meta.dirname, "notes.json");

async function loadNotes() {
  try {
    const text = await readFile(NOTES_FILE, "utf8");
    return JSON.parse(text);
  } catch (error) {
    if (error.code === "ENOENT") {
      return []; // no file yet: start with an empty list
    }
    throw error; // anything else is a real problem
  }
}

async function saveNotes(notes) {
  await writeFile(NOTES_FILE, JSON.stringify(notes, null, 2));
}
```

The first time you run the app, there's no `notes.json` yet. That's not really an error, so `loadNotes` starts with an empty list. Any other problem (like broken JSON) is re-thrown, just like in [chapter 18](#).

Next come the three commands:

```

async function addNote(text) {
  const notes = await loadNotes();
  notes.push(text);
  await saveNotes(notes);
  console.log(`Added note ${notes.length}: ${text}`);
}

async function listNotes() {
  const notes = await loadNotes();
  if (notes.length === 0) {
    console.log("No notes yet.");
    return;
  }
  notes.forEach((note, index) => {
    console.log(`${index + 1}. ${note}`);
  });
}

async function removeNote(numberText) {
  const notes = await loadNotes();
  const number = Number(numberText);
  if (!Number.isInteger(number) || number < 1 || number > notes.length) {
    console.error(`There's no note number ${numberText}.`);
    process.exit(1);
  }
  const [removed] = notes.splice(number - 1, 1);
  await saveNotes(notes);
  console.log(`Removed: ${removed}`);
}

```

People count notes from 1, but arrays count from 0, so `removeNote` subtracts 1 before calling `splice` ([chapter 10](#)). A bad number, like `abc` or `7` when there are only 2 notes, stops the program with exit code 1.

Finally, read the command and hand it to the right function:

```

const [command, ...words] = process.argv.slice(2);

if (command === "add") {
  await addNote(words.join(" "));
} else if (command === "list") {
  await listNotes();
} else if (command === "remove") {
  await removeNote(words[0]);
} else {
  console.log("Usage: node notes.js add <text> | list | remove <number>");
}

```

The rest parameter ([chapter 15](#)) collects every word after the command, so `add Buy milk` and `add "Buy milk"` both save `Buy milk`. Here's a session. The lines starting with `>` are what you type:

```
> node notes.js add Buy milk
Added note 1: Buy milk
> node notes.js add "Call the dentist"
Added note 2: Call the dentist
> node notes.js list
1. Buy milk
2. Call the dentist
> node notes.js remove 1
Removed: Buy milk
> node notes.js list
1. Call the dentist
```

Open `notes.json` in VS Code, and you'll see your notes saved as a JSON array. A program that you control by typing commands in the terminal like this is called a **CLI** (command-line interface). Many developer tools, like `node` itself and `npm`, are CLIs.

Build it: a tiny web API

In [chapter 33](#), you were the customer. You sent your order with `fetch`, and the API was the waiter who brought back your food. Now you'll build the kitchen: a **server**, a program that waits for requests and answers them.

Save this as `server.js`:

```
import http from "node:http";

const books = [
  { id: 1, title: "The Hobbit", available: true },
  { id: 2, title: "Dune", available: false },
  { id: 3, title: "Matilda", available: true },
];

function sendJSON(response, status, data) {
  response.writeHead(status, { "Content-Type": "application/json" });
  response.end(JSON.stringify(data));
}

const server = http.createServer((request, response) => {
  console.log(`${request.method} ${request.url}`);

  if (request.method === "GET" && request.url === "/books") {
    sendJSON(response, 200, books);
  } else {
    sendJSON(response, 404, { error: "Not found" });
  }
});

server.listen(3000, () => {
  console.log("Library API running at http://localhost:3000/books");
});
```

Here's what each part does:

- `http.createServer(...)` makes the server. The function you give it runs once for **every request** that arrives.
- `request` is what the client asked for. `request.method` is the HTTP method, like `"GET"`, and `request.url` is the part of the address after the port, like `"/books"`.
- `response` is your answer. `writeHead` sets the status code and the headers (`Content-Type` tells the client "this is JSON"), and `end` sends the body and finishes the answer.
- `server.listen(3000, ...)` starts listening on port 3000, the "door number" from [chapter 29](#). `localhost` means "this computer".

Run `node server.js`. It prints its message and then... keeps running. That's what servers do: they wait for requests. You'll stop it later with `Ctrl + C`, just like the infinite loops in [chapter 08](#).

While it runs, open `http://localhost:3000/books` in your browser, and you'll see your books as JSON. Your server's terminal prints `GET /books`. You may also see `GET /favicon.ico`: browsers ask every site for the little icon on the tab. Your server answers 404, which is fine.

Now fetch from your own API with JavaScript. The server is busy in its terminal, so open a **second** terminal (the + button in VS Code's terminal panel) and run this `client.js`:

```
const response = await fetch("http://localhost:3000/books");
console.log(response.status); // prints: 200
console.log(await response.json());

const missing = await fetch("http://localhost:3000/movies");
console.log(missing.status, await missing.json()); // prints: 404 { error: 'Not found' }
```

You'll see:

```
200
[
  { id: 1, title: 'The Hobbit', available: true },
  { id: 2, title: 'Dune', available: false },
  { id: 3, title: 'Matilda', available: true }
]
404 { error: 'Not found' }
```

And the server's terminal shows each request as it arrives:

```
Library API running at http://localhost:3000/books
GET /books
GET /movies
```

That's a real API, and your own `fetch` code talks to it exactly the way it talked to JSONPlaceholder. When you're done, click the server's terminal and press `Ctrl + C` to stop it.

Tip: `request.url` includes any query string, so `/books?available=true` isn't equal to `"/books"`. To read the path and the query separately, use the `URL` class from chapter 33: `new URL(request.url, "http://localhost")` gives you `.pathname` and `.searchParams`.

EventEmitter : Node's built-in observer

In [chapter 44](#), you built a tiny emitter with `on`, `off`, and `emit`: the subscribe button and bell. Node has a fuller version built in, called `EventEmitter`, with the same method names:

```
import { EventEmitter } from "node:events";

const orders = new EventEmitter();

// Two different parts of the app listen for the same event
orders.on("placed", (order) => {
  console.log(`Kitchen: start making a ${order.dish}`);
});
orders.on("placed", (order) => {
  console.log(`Email: thanks for your order, ${order.customer}!`);
});

orders.emit("placed", { dish: "pizza", customer: "Ana" });
orders.emit("placed", { dish: "salad", customer: "Raj" });
```

You'll see:

```
Kitchen: start making a pizza
Email: thanks for your order, Ana!
Kitchen: start making a salad
Email: thanks for your order, Raj!
```

`off` removes a listener (it needs the same function, just like `removeEventListener`), and `once` adds a listener that removes itself after the first time, like the `once` from chapter 44's exercises. One difference from your own `Emitter`: Node's `on` doesn't return an unsubscribe function. It returns the emitter itself, so to unsubscribe, give your listener a name and pass it to `off`.

`EventEmitter` matters because Node uses it everywhere. Your HTTP server is an event emitter, and so is `process`. For example, a server emits an `error` event when it can't start. You can listen for it and give a friendlier message than a crash:

```
server.on("error", (error) => {
  if (error.code === "EADDRINUSE") {
    console.error("Port 3000 is busy. Is the server already running in another terminal?");
    process.exit(1);
  }
  throw error;
});
```

CommonJS vs ES modules: a quick recap

You'll meet a lot of older Node code that uses **CommonJS**, Node's original module system, which you met in [chapter 29](#). Here's how to read it:

	ES modules (this course)	CommonJS (older code)
Import something	<code>import path from "node:path";</code>	<code>const path = require("node:path");</code>
Export something	<code>export function addTax() {}</code>	<code>module.exports = { addTax };</code>
This file's folder	<code>import.meta.dirname</code>	<code>__dirname</code>
Top-level <code>await</code>	Yes	No
How Node knows	<code>"type": "module"</code> in <code>package.json</code> , or a <code>.mjs</code> file	<code>"type": "commonjs"</code> , a <code>.cjs</code> file, or (usually) no <code>"type"</code> at all

Stick with ES modules in your own code. They're the standard, and they work the same in the browser and in Node.

Where to go from here

The `node:http` module is great for learning how servers work, but real servers need a lot more: many routes, reading data sent with `POST`, and security checks. That's what **frameworks** are for. A framework is a ready-made set of tools that handles the repetitive parts, so you only write what's special about your app. The popular ones for Node are **Express**, **Fastify**, and **Hono**. You install them with npm, which is the next chapter.

Common mistakes

1. Using browser-only tools in Node

```
localStorage.setItem("theme", "dark");
// ReferenceError: localStorage is not defined
```

Node has no web page, so there's no `document`, `window`, `alert`, or `localStorage`. To save data in Node, write it to a file with `node:fs/promises`, like the notes app does.

2. Forgetting `"utf8"` when reading a file

```
const text = await readFile("shopping.txt");
console.log(text); // prints: <Buffer 6d 69 6c 6b 0a 62 72 65 61 64 0a 65 67 67 73 0a>
```

Without an encoding, Node gives you a **Buffer**: the raw bytes of the file, shown as numbers. Add `"utf8"` as the second argument, and you get text: `readFile("shopping.txt", "utf8")`.

3. Forgetting `await`

```
const text = readFile("shopping.txt", "utf8"); // forgot await
console.log(text); // prints: Promise { <pending> }
```

The functions in `node:fs/promises` return promises. Without `await`, you get the promise itself, not what's in the file ([chapter 32](#)).

4. Using `require` or `__dirname` in an ES module

```
const fs = require("node:fs");
// ReferenceError: require is not defined in ES module scope, you can use import instead
```

Code copied from older tutorials often uses CommonJS. In an ES module, use `import` instead of `require`, and `import.meta.dirname` instead of `__dirname` (which fails with `ReferenceError: __dirname is not defined in ES module scope`).

5. Starting the server twice

```
Error: listen EADDRINUSE: address already in use :::3000
```

Only one program can listen on a port at a time. This almost always means your server is still running in another terminal. Find that terminal, press `Ctrl + C`, and start it again.

6. A request that never gets an answer

```
const server = http.createServer((request, response) => {
  if (request.url === "/books") {
    sendJSON(response, 200, books);
  }
  // no else: any other request never gets an answer!
});
```

There's no error message, which makes this one confusing. A request for `/movies` just hangs: the browser's loading spinner keeps spinning, and `fetch` waits and waits. Every request needs exactly one answer, so make sure every path through your code ends with `response.end()`, like the 404 `else` in `server.js`.

Quick recap

- Node runs JavaScript outside the browser, on the same V8 engine as Chrome. There's no `document` or `window`. Instead, you get `process` and built-in modules like `node:fs/promises`, `node:path`, and `node:http`.
 - `process.argv.slice(2)` gives you the command-line arguments and `process.env` the environment variables. Both are always strings. `process.exit(1)` stops the program and reports a problem.
 - `readFile`, `writeFile`, `appendFile`, `mkdir`, and `readdir` all return promises, so `await` them, and pass `"utf8"` to read text.
 - Build file paths with `path.join(import.meta.dirname, ...)`, so your program finds its files whichever folder the terminal is in.
 - `node:readline/promises` asks questions in the terminal (don't forget `r1.close()`), and `node:http` lets you build your own API that `fetch` can talk to.
 - `EventEmitter` (`on`, `off`, `once`, `emit`) is Node's built-in observer, and much of Node is built on it.
 - Write ES modules, and recognize CommonJS (`require`, `module.exports`, `__dirname`) when you see it in older code.
-

Next: try the [exercises](#), then move on to [50 Tooling](#).

Exercises

How to do these:

- Make a new file for each exercise in this folder (`ex1.js`, `ex2.js`, and so on), and run it with `node ex1.js`, adding any extra words the exercise asks for.
 - These exercises use `import`, so make sure this folder has a `package.json` containing `{ "type": "module" }` ([chapter 29](#)).
 - Some exercises create files and folders next to your code. That's expected. Delete them whenever you like.
 - In the expected outputs, lines starting with `>` are what you type. To check a program's exit code in PowerShell, type `$LASTEXITCODE` after it finishes.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Distance converter

A running club wants a quick way to convert race distances in the terminal. Write `ex1.js`, which takes a distance and a unit, and converts between kilometers and miles. One mile is 1.609344 km. Show the answer with two

decimal places.

Expected output:

```
> node ex1.js 5 km
5 km is 3.11 miles
> node ex1.js 10 miles
10 miles is 16.09 km
> node ex1.js 21.1 km
21.1 km is 13.11 miles
```

When something is wrong, print a helpful message with `console.error`, and stop with exit code 1:

```
> node ex1.js
Usage: node ex1.js <distance> <km|miles>
> node ex1.js five km
"five" is not a number.
> node ex1.js 5 feet
Unknown unit "feet". Use km or miles.
```

Rule: use guard clauses ([chapter 18](#)), and keep 1.609344 in a named constant instead of a magic number ([chapter 45](#)).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Word counter

A writing app shows a few stats about a text file. Create a file called `poem.txt` in this folder, with this text:

```
The rain fell on the town all day.
The children stayed inside and played.
By night the rain had gone away,
and the moon came out to watch the bay.
```

Then write `ex2.js`, which reads the file named on the command line and prints its stats.

Expected output:

```
> node ex2.js poem.txt
File: poem.txt
Lines: 4
Words: 30
Longest word: children
Most common word: "the" (6 times)
```

Rules:

- Look for the file in the same folder as `ex2.js`, so the program works from any folder. Test it: from the `JavaScript` folder, `node 49-nodejs-basics/ex2.js poem.txt` must print the same stats.
- Ignore capital letters and punctuation when comparing words: `The` and `the` are the same word, and `day.` counts as `day`.
- If there's no file name, print `Usage: node ex2.js <file name>`. If the file doesn't exist, print `Can't find a file called "missing.txt"`. (with the real name). Both stop with exit code 1.

Stuck? [Hint 1](#), [Hint 2](#) and [Hint 3](#) are in the [Hints section](#) at the end.

Exercise 3 (Medium): Monthly sales report

A small bookshop's till saves each month's sales in its own text file, one sale per line: the title, a comma, then the price. First, copy this into `setup3.js` and run it once. It creates a `sales` folder with some example files:

```
import { mkdir, writeFile } from "node:fs/promises";
import path from "node:path";

const salesFolder = path.join(import.meta.dirname, "sales");
await mkdir(salesFolder, { recursive: true });

await writeFile(path.join(salesFolder, "2026-01.txt"), "The
Hobbit,9.99\nDune,12.50\nMatilda,6.99\n");
await writeFile(path.join(salesFolder, "2026-02.txt"), "Dune,12.50\nWonder,8.49\nThe
Hobbit,9.99\nHoles,7.25\n");
await writeFile(path.join(salesFolder, "2026-03.txt"), "Coraline,8.75\nMatilda,6.99\n");
await writeFile(path.join(salesFolder, "readme.md"), "One file per month. One sale per line:
title,price\n");

console.log("Sales files are ready.");
```

Now write `ex3.js`. It reads **every** **.txt** **file** in the `sales` folder (and skips anything else, like `readme.md`), works out the number of sales and the money taken each month, and prints a report. Then it saves the same report in a new file called `summary.txt`, next to `ex3.js`.

Expected output:

```
2026-01: 3 sales, $29.48
2026-02: 4 sales, $38.23
2026-03: 2 sales, $15.74
Total: 9 sales, $83.45
Saved summary.txt
```

Open `summary.txt` afterwards. It should hold the first four lines.

Rule: don't type the file names into `ex3.js`. Find them with `readdir`, so next month's file gets picked up without changing any code.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Cinema ticket booth

The Starlight Cinema wants a ticket booth that runs in the terminal. It asks how many tickets you'd like, then the age of each guest, and prints the bill. Children under 13 pay \$7.00, guests aged 65 and over pay \$8.50, and everyone else pays \$12.00.

If an answer isn't a whole number in the allowed range (1 to 10 tickets, ages 0 to 120), the booth says so and asks the same question again.

Here's a run. The answers after each `?` are typed in:

```
Welcome to the Starlight Cinema!
How many tickets? two
Please type a whole number from 1 to 10.
How many tickets? 3
Age of guest 1? 34
Age of guest 2? 8
Age of guest 3? abc
Please type a whole number from 0 to 120.
Age of guest 3? 70
Guest 1: adult $12.00
Guest 2: child $7.00
Guest 3: senior $8.50
Total: $27.50
Enjoy the film!
```

Also check what happens when you press `Enter` without typing anything. It should ask again, too.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A recipes API

Build a small API for a recipe website. The recipes live in a JSON file, and your server answers requests for all of them, some of them, or just one.

Create `recipes.json` in this folder:

```
[
  { "id": 1, "name": "Pancakes", "minutes": 20, "vegetarian": true },
  { "id": 2, "name": "Chicken curry", "minutes": 45, "vegetarian": false },
  { "id": 3, "name": "Tomato soup", "minutes": 30, "vegetarian": true },
  { "id": 4, "name": "Fish tacos", "minutes": 25, "vegetarian": false }
]
```

Write `ex5-server.js`. When it starts, it reads the recipes from `recipes.json` (found with `import.meta.dirname`). Then it listens on port 3000, logs each request, and answers these `GET` requests with JSON:

Request	Answer
<code>/recipes</code>	<code>200</code> and all the recipes
<code>/recipes?vegetarian=true</code>	<code>200</code> and only the vegetarian recipes
<code>/recipes?maxMinutes=25</code>	<code>200</code> and only the recipes that take 25 minutes or less (any number works)
Both filters together	<code>200</code> and the recipes that pass both
<code>/recipes/3</code>	<code>200</code> and just the recipe with id 3
<code>/recipes/99</code>	<code>404</code> and <code>{ "error": "Recipe 99 not found" }</code>
Anything else	<code>404</code> and <code>{ "error": "Not found" }</code>

To test it, copy this into `ex5-client.js`:

```

const BASE_URL = "http://localhost:3000";

const paths = [
  "/recipes",
  "/recipes?vegetarian=true",
  "/recipes?maxMinutes=25",
  "/recipes?vegetarian=true&maxMinutes=25",
  "/recipes/3",
  "/recipes/99",
  "/menu",
];

for (const path of paths) {
  const response = await fetch(BASE_URL + path);
  const data = await response.json();

  let summary;
  if (!response.ok) {
    summary = data.error;
  } else if (Array.isArray(data)) {
    summary = data.map((recipe) => recipe.name).join(", ");
  } else {
    summary = `${data.name} (${data.minutes} minutes)`;
  }
  console.log(`GET ${path} -> ${response.status}: ${summary}`);
}

```

Start your server with `node ex5-server.js`, then run `node ex5-client.js` in a second terminal.

Expected output from the client:

```

GET /recipes -> 200: Pancakes, Chicken curry, Tomato soup, Fish tacos
GET /recipes?vegetarian=true -> 200: Pancakes, Tomato soup
GET /recipes?maxMinutes=25 -> 200: Pancakes, Fish tacos
GET /recipes?vegetarian=true&maxMinutes=25 -> 200: Pancakes
GET /recipes/3 -> 200: Tomato soup (30 minutes)
GET /recipes/99 -> 404: Recipe 99 not found
GET /menu -> 404: Not found

```

And your server's terminal shows:

```
Recipes API running at http://localhost:3000/recipes
GET /recipes
GET /recipes?vegetarian=true
GET /recipes?maxMinutes=25
GET /recipes?vegetarian=true&maxMinutes=25
GET /recipes/3
GET /recipes/99
GET /menu
```

Try the addresses in your browser too, while the server is running. Remember to stop it with `Ctrl + C` when you're done.

Stuck? [Hint 1](#), [Hint 2](#), [Hint 3](#) and [Hint 4](#) are in the [Hints section](#) at the end.

Before you move on

So far, you've only used what comes built into Node. But there are millions of free packages out there, from web frameworks like Express to tools that check and format your code for you.

In [chapter 50](#), you'll learn how to install and use them with npm, and meet the tools professional developers use every day. 😊

50 Tooling

What is it?

Tooling means the helper programs developers use *around* their code: to install other people's code, run tasks, check for mistakes, format code neatly, and bundle everything up for the web.

The most important one is **npm**, which comes with Node.js. You already have it.

Why does it matter?

So far, every line of code in your projects was written by you. Real projects don't work like that. Why write your own date formatter when a tested, popular one already exists?

Tools also do boring jobs for you:

- **Installing and updating** other people's code (npm).
- **Restarting** your program every time you save (`node --watch`).
- **Formatting** your code the same way every time (Prettier).
- **Catching bugs** before you run anything (ESLint).
- **Bundling and shrinking** your files so websites load faster (Vite).

Every professional JavaScript project uses these tools, so knowing them makes other people's projects much less scary to open.

Real-world example

Think of a carpenter's workshop:

In the workshop	In JavaScript
A hardware store full of ready-made parts	The npm registry : millions of free packages
The shopping list for this job	<code>package.json</code>
The receipt, listing the exact parts bought	<code>package-lock.json</code>
The storeroom where the parts are kept	The <code>node_modules</code> folder
Power tools that save hours of hand work	Prettier, ESLint, Vite

You don't build your own hammer. You pick good tools, and spend your time on the actual furniture.

How it works

npm and packages

A **package** is a folder of reusable code that someone published for others to use. The **npm registry** (at npmjs.com) is the huge online library where packages live. **npm** is the program that downloads them for you.

Check that you have it (it came with Node.js in [chapter 01](#)):

```
npm --version
```

You'll see a version number, like `11.12.1`. The exact number doesn't matter.

Starting a project: `package.json`

Every npm project has a `package.json` file: its ID card and shopping list. Make a new folder, open a terminal in it, and run:

```
npm init -y
```

The `-y` means "yes to all the questions". npm writes a `package.json` like this:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Watch out: recent versions of npm write `"type": "commonjs"`. This course uses `import` and `export`, so change it to `"type": "module"`, exactly like in [chapter 29](#). Otherwise Node will refuse your `import` lines.

Installing a package

Say you want nicer date formatting than chapter 19's `Date`. The popular **dayjs** package does that. Install it:

```
npm install dayjs
```

Three things happen:

1. npm downloads dayjs into a new `node_modules` folder.
2. It adds dayjs to `package.json`, under `dependencies`:

```
"dependencies": {  
  "dayjs": "^1.11.23"  
}
```

3. It creates `package-lock.json` (more on that below).

Now you can import it by its name, with no `./` in front:

```
import dayjs from "dayjs";  
  
const deadline = dayjs("2026-10-01");  
console.log("Deadline:", deadline.format("D MMMM YYYY"));  
console.log("A week later:", deadline.add(7, "day").format("dddd, D MMM"));
```

You'll see:

```
Deadline: 1 October 2026  
A week later: Thursday, 8 Oct
```

`./math.js` means "my own file". `"dayjs"` (no dot) means "a package from `node_modules`".

The same command installs anything. For example, `npm install express` installs Express, the popular framework for building web servers that [chapter 49](#) mentioned.

dependencies vs devDependencies

Some packages are needed while your app *runs*. Others are only needed while you *build* it, like a formatter:

```
npm install --save-dev prettier
```

That puts it in a separate list:

```
"devDependencies": {  
  "prettier": "^3.9.9"  
}
```

List	What goes there	Example
dependencies	Code your app needs to work	dayjs, express
devDependencies	Tools you only use while developing	prettier, eslint, vite

Think of it as ingredients (dependencies) versus kitchen tools (devDependencies). The customer eats the ingredients, never the whisk.

node_modules : never share it

`node_modules` can hold thousands of files, because packages depend on other packages. Never copy it around or put it in git. Anyone can re-create it from `package.json` with one command:

```
npm install
```

With no package name, `npm install` installs everything listed in `package.json`. If you use git, add this line to a file called `.gitignore`, so git skips the folder:

```
node_modules
```

package-lock.json : the exact receipt

`package.json` says roughly which versions are okay. `package-lock.json` records the **exact** version of every package that was installed, including the packages your packages use.

Always keep it (and commit it, if you use git). It makes sure everyone on your team, and your server, installs exactly the same code. You never edit it by hand: npm does that.

Version numbers: what `^1.11.23` means

Packages use **semantic versioning**: three numbers, `MAJOR.MINOR.PATCH`.

Part	Changes when...	Safe to update?
1.11.23 (major)	Something breaking changed. Your code might need fixing.	Careful

Part	Changes when...	Safe to update?
1. 11 .23 (minor)	New features were added, nothing broken.	Yes
1.11. 23 (patch)	Only bug fixes.	Yes

The `^` in front means "this version, or any newer one with the same major number". So `^1.11.23` allows `1.12.0`, but never `2.0.0`.

npm scripts: named shortcuts

The `scripts` section of `package.json` holds commands you run often:

```
"scripts": {
  "start": "node index.js",
  "dev": "node --watch index.js",
  "test": "node --test",
  "format": "prettier --write ."
}
```

Run them with `npm run` and the name:

```
npm run dev
npm run format
```

`start` and `test` are special: you can leave out `run` and type `npm start` or `npm test`. Scripts are a great habit. Anyone who opens your project can see how to run it, without asking you.

(`node --test` is the test runner from [chapter 46](#).)

npx : run a tool without a script

`npx` runs a tool from your project's `node_modules` (or downloads it for a one-time run):

```
npx prettier --write index.js
```

Tools already built into Node

Node itself has two handy tools, so you don't need extra packages for them:

- `node --watch index.js` reruns your file every time you save it. Great while you're working. Stop it with `Ctrl + C`.

- `node --env-file=.env index.js` reads settings from a `.env` file into `process.env` ([chapter 49](#)). Keep secret keys there, never in your code, and add `.env` to `.gitignore` ([chapter 51](#)).

For example, with a `.env` file containing `GREETING=Hello from .env`:

```
console.log(process.env.GREETING);
```

`node --env-file=.env env.js` prints `Hello from .env`. Plain `node env.js` prints `undefined`, because Node only reads the file when you ask it to.

Prettier: automatic formatting

Prettier is a **formatter**: it rewrites your code's spacing, quotes, and line breaks into one consistent style. You stop arguing about style, and just press save.

Before:

```
const x={a:1,b:[1,2,3]}
console.log( x )
```

After `npx prettier --write messy.js`:

```
const x = { a: 1, b: [1, 2, 3] };
console.log(x);
```

In VS Code, install the **Prettier** extension and turn on "Format On Save" in the settings, and it happens every time you save.

ESLint: a spell-checker for bugs

ESLint is a **linter**: a tool that reads your code without running it and warns you about likely bugs, like a variable you never use, or one you use before it exists. It's like the red squiggly lines under spelling mistakes, but for code.

To set it up in a project, run:

```
npm init @eslint/config@latest
```

It asks a few questions (are you using modules? browser or Node?), installs what it needs, and creates a config file. Then check your code with:

```
npx eslint .
```

Be honest about versions: ESLint's setup questions and config format have changed between versions, so the prompts you see may differ a little. The VS Code **ESLint** extension shows the warnings right in your editor.

Vite: a dev server and bundler for websites

Remember Live Server from [chapter 29](#)? **Vite** (French for "quick", said "veet") is the professional version. Create a new project with:

```
npm create vite@latest
```

It asks for a project name and a framework (choose **Vanilla** and **JavaScript** for plain JavaScript). Then:

```
cd my-project  
npm install  
npm run dev
```

`npm run dev` starts a local server and prints an address like `http://localhost:5173`. Every time you save, the page updates instantly.

When the site is ready to publish, `npm run build` creates a `dist` folder with your files **bundled** (many files joined into a few) and **minified** (spaces, comments, and long names removed, so files download faster). You upload `dist`, not your source files.

(The exact prompts can change between Vite versions, but the idea stays the same.)

What's next: TypeScript

Many projects also use **TypeScript**: JavaScript plus labels for what type each value should be. A tool checks those labels and catches mistakes like passing a string where a number belongs. It'll get its own folder after this course.

Common mistakes

1. Forgetting to change "type"

```
import dayjs from "dayjs";  
// SyntaxError: Cannot use import statement outside a module
```

`npm init -y` wrote `"type": "commonjs"`. Change it to `"type": "module"` in `package.json`.

2. Putting `./` in front of a package name

```
import dayjs from "./dayjs"; // looks for YOUR file called dayjs
```

Packages are imported by their bare name: `import dayjs from "dayjs";`.

3. Sharing or committing `node_modules`

It's huge, and it can be re-created any time with `npm install`. Share `package.json` and `package-lock.json` instead.

4. Running `npm install` in the wrong folder

If there's no `package.json` in the folder, `npm` may create files in the wrong place. Check that your terminal is in your project folder first (just like `node` in chapter 01).

5. Installing a package for a tiny job

Every package is someone else's code in your project, and a possible security risk ([chapter 51](#)). If ten lines of your own code can do it, write the ten lines.

Quick recap

- `npm` installs packages from the `npm` registry. `package.json` lists them, `package-lock.json` pins exact versions, and `node_modules` stores them.
- `npm init -y` starts a project. Change `"type"` to `"module"` to use `import`.
- Use `dependencies` for code your app needs, and `devDependencies` (`--save-dev`) for tools.
- `^1.2.3` accepts newer minor and patch versions, never a new major one.
- `npm` scripts (`npm run dev`, `npm test`) are named shortcuts. `npx` runs a tool once.
- Node has `--watch` and `--env-file` built in. Prettier formats, ESLint catches bugs, and Vite serves and bundles websites.

Next: try the [exercises](#), then move on to [51 Security Basics](#).

Exercises

How to do these:

- Make a folder called `practice` inside this chapter folder, and a new folder inside it for each exercise (`practice/ex1`, `practice/ex2`, ...).
 - Open the terminal **in that exercise's folder** before running any `npm` command.
 - These exercises download packages, so you need to be online.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your `package.json` and code.
-

Exercise 1 (Easy): Your first package.json

Create a project for a "birthday countdown" app.

1. Run `npm init -y` in `practice/ex1`.
2. Open `package.json` and change the `"type"` so `import` works.
3. Change the `"description"` to `"Counts the days until my birthday"`.
4. Answer in your own words: what is the `"version"` of your project, and which part of it would you change if you fixed a small bug?

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Install and use dayjs

In the same project, install `dayjs` and write `index.js` so it prints the number of days until a fixed date.

Use this date so your output matches:

```
const today = dayjs("2026-09-24");
const birthday = dayjs("2026-12-25");
```

Expected output:

```
Days until my birthday: 92
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Scripts for everything

Add these npm scripts to your `package.json`, and check that each one works:

Command	What it should do
<code>npm start</code>	Run <code>index.js</code> once
<code>npm run dev</code>	Run <code>index.js</code> again every time you save it
<code>npm run format</code>	Format all your files with Prettier

Prettier is a tool you only need while developing. Install it in the right list.

Test `npm run dev` by changing the birthday date and saving: the new number should appear by itself. Stop it with `Ctrl + C`.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Keep a secret out of your code

A weather app needs a (pretend) API key. Right now it's written straight in the code:

```
const apiKey = "pretend-key-123";
console.log("Using key that starts with:", apiKey.slice(0, 7));
```

Move the key into a `.env` file, and change the code so it reads the key from `process.env`. Add an npm `start` script that loads the `.env` file.

Expected output of `npm start`:

```
Using key that starts with: pretend
```

Then create a `.gitignore` file that would stop both `.env` and `node_modules` from being shared.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): Rebuild the birthday app with Vite

Turn the birthday countdown into a small web page, using Vite as the dev server.

Must-haves:

- Create the project with `npm create vite@latest` (choose Vanilla and JavaScript).
- Install `dayjs` in the new project.
- Show "N days until my birthday" on the page, using the DOM ([chapter 20](#)) and **today's real date** this time.
- Run it with `npm run dev`, then create the production version with `npm run build`.

After the build, open the `dist` folder and look inside the `.js` file. Answer: what does minified code look like, and why is it smaller?

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Before you move on

You just installed code written by strangers into your project. How do you know it's safe? [Chapter 51](#) covers that, along with the most common security holes in JavaScript apps.

51 Security Basics

What is it?

Security means protecting your app, and the people who use it, from anyone trying to misuse it: stealing data, taking over accounts, or making your page do things you never wrote.

This chapter is about **defense**. You'll learn the most common holes in JavaScript apps, and how to close each one.

Why does it matter?

Your users trust you with their names, their messages, their passwords, and sometimes their money. And attackers don't have to be geniuses. They use automated tools that scan thousands of websites for the same handful of well-known mistakes.

Small mistakes can have big effects. In 2005, a MySpace user found he could sneak his own JavaScript into his profile page. Anyone who viewed the profile automatically "added him as a friend" and got a copy of the code in their own profile. In less than a day, it had spread to over a million profiles. It was a prank, but the same hole could just as easily have been used to steal accounts.

The good news: most of these mistakes have simple fixes, and you already know several of them from earlier chapters.

Watch out: use what you learn here to protect your *own* projects. Testing other people's websites for holes without their permission is illegal in most countries.

Real-world example

Security is like keeping your home safe. There's no single magic lock. You close each door and window, one at a time:

Keeping your home safe	Keeping your app safe
Check who's at the door before you let them in	Check everything users send you
A note pushed under your door isn't an order you must follow	Show user text as text. Never run it as code
Don't hide a spare key under the doormat	Don't put secret keys in front-end code

Keeping your home safe	Keeping your app safe
Don't let strangers copy your house keys	Keep login tokens where other scripts can't read them
Only let builders you trust work on your house	Only install packages you trust
Have private conversations behind closed doors	Always use HTTPS

A burglar only needs one open window. The rest of this chapter walks through the windows, one by one.

How it works

Rule 1: never trust user input

User input is anything that comes from outside your own code: form fields, the page URL, data from an API, even `localStorage` (users can edit it in DevTools). Treat all of it as possibly harmful until you've checked it.

Remember the form validation from [chapter 22](#)? Those checks are great for the *user*: quick, friendly messages. But they don't protect anything. Anyone can open DevTools, delete a `required` attribute, change a hidden field, or skip your page completely and send a request straight to your server with `fetch`.

So here's the rule: **the browser checks for convenience, the server checks for safety**. The server must check everything again. And it should never trust a value it can work out by itself, like a price.

Here's a classic mistake. An online shop's page sends the price along with the order, and a sneaky user changes it to 1 cent before sending. A safe server ignores that price and uses its own:


```

const prices = { mug: 12, poster: 8 }; // the server's own price list
const MAX_QUANTITY = 10;

function calculateTotal(order) {
  if (!Object.hasOwn(prices, order.productId)) {
    throw new Error("Unknown product");
  }
  if (!Number.isInteger(order.quantity) || order.quantity < 1 || order.quantity > MAX_QUANTITY)
    {
      throw new Error(`Quantity must be a whole number from 1 to ${MAX_QUANTITY}`);
    }
  // Ignore any price the browser sent. Use the server's own price.
  return prices[order.productId] * order.quantity;
}

console.log(calculateTotal({ productId: "mug", quantity: 2, price: 0.01 })); // prints: 24

try {
  calculateTotal({ productId: "mug", quantity: -5 });
} catch (error) {
  console.log("Rejected:", error.message); // prints: Rejected: Quantity must be a whole number
    from 1 to 10
}

```

Two details worth copying:

- **It lists what's allowed** (known products, 1 to 10 items) and rejects everything else. That's called an **allowlist**. It's much safer than a blacklist that tries to name every bad value, because attackers are good at finding the one you forgot.
- `Object.hasOwn` checks that the product really is in the price list. A plain `prices[order.productId]` lookup can be fooled by sneaky IDs like `"constructor"`, which every object has behind the scenes ([chapter 28](#)). You'll see why that matters later in this chapter.

A real server does these checks in code like this, running on Node ([chapter 49](#)).

XSS: when user text turns into code

XSS (cross-site scripting) is one of the most common holes on the web. It happens when text from a user ends up being run as code on your page. [Chapter 20](#) warned you about it. Here's the full story.

Picture a recipe site where visitors leave comments, and the page shows them with `innerHTML`:

```
const commentList = document.getElementById("comments");

// A visitor typed this into the comment box:
const comment = ``;

// Vulnerable: the browser reads the comment as HTML
commentList.innerHTML += `<li>${comment}</li>`;
```

The browser turns that comment into a real `<img>` element. It tries to load an image called `x`, fails, and then runs the code inside `onerror`.

Here, that code only shows a harmless alert. But it runs in the browser of **every visitor** who opens the page, with the same powers as your own JavaScript. A real attacker's code could:

- read everything on the page, and everything in `localStorage`,
- send requests to your server as the logged-in visitor,
- swap your page for a fake login form that sends passwords to the attacker.

You might think: "I'll block `<script>` tags." That's not enough. Browsers don't even run `<script>` tags added with `innerHTML`, which is exactly why attackers use tricks like `onerror` instead. There are dozens of tags and attributes that can run code, and people keep finding new ones.

So don't try to spot the bad stuff. Make sure user text can **never** become HTML in the first place.

`innerHTML` isn't the only way in. `outerHTML`, `insertAdjacentHTML()` and `document.write()` also read strings as HTML, so the same rules apply to them.

XSS fix 1: use `textContent`

This is the best fix, and the simplest. `textContent` treats everything as plain text, so the comment appears exactly as typed, angle brackets and all, and nothing runs:

```
const commentList = document.getElementById("comments");
const comment = ``;

const item = document.createElement("li");
item.textContent = comment; // shown as text, never run
commentList.append(item);
```

The page now shows `` as plain text. Anyone reading the comments can see what the attacker tried, and nothing happens.

Build new elements with `createElement` and fill them with `textContent`, as you learned in chapter 20. That covers almost every case.

XSS fix 2: escape user text before it goes into HTML

Sometimes you're building a string of HTML anyway, like a big template with lots of parts. Then you **escape** every piece of user text first. Escaping means swapping each character that has a special meaning in HTML for a safe code, called an **HTML entity**. The browser shows the entity as the normal character, but never treats it as HTML:

Character	Escaped as
&	&
<	<
>	>
"	"
'	'

After escaping, `<img ...>` becomes `<img ...>`. The browser shows `<img ...>` on the screen, but as text, not as a tag.

You'll write your own `escapeHTML` function in the exercises. In bigger projects, frameworks like React and Vue escape text for you automatically, which is one of the reasons people like them.

Tip: here's a slogan worth remembering: **validate what comes in, escape what goes out.**

XSS fix 3: when you need user HTML, use a sanitizer

Some apps want users to add a little formatting, like **bold** text in a comment. Then you can't escape everything. Instead, you pass the HTML through a **sanitizer**: a library that removes everything dangerous and keeps only safe tags. The best-known one is **DOMPurify**:

```
const userHtml = `Great <b>recipe</b>! `;
const cleanHtml = DOMPurify.sanitize(userHtml);
console.log(cleanHtml); // prints: Great <b>recipe</b>! 
```

The bold stays, and the `onerror` is gone. To use it, install it with `npm install dompurify` and write `import DOMPurify from "dompurify";` in a project built with a bundler like Vite ([chapter 50](#)).

Don't write your own sanitizer. HTML has far too many tricks, and the DOMPurify team has spent years finding them.

Watch out for links too

A link can run code if its address starts with `javascript:`. So if users can add a website to their profile, someone will try `javascript:alert('hacked')`, and anyone who clicks the link runs that code.

The fix is another allowlist. Read the address with `URL` ([chapter 33](#)) and allow only `http:` and `https:`:

```
function isSafeLink(address) {
  try {
    const url = new URL(address);
    return url.protocol === "https:" || url.protocol === "http:";
  } catch (error) {
    return false; // not a valid address at all
  }
}

console.log(isSafeLink("https://sam.example")); // prints: true
console.log(isSafeLink("javascript:alert('hacked')")); // prints: false
console.log(isSafeLink("  JavaScript:alert('hacked')")); // prints: false
```

Why not check the text yourself, with something like `address.startsWith("javascript:")`? The last example would get past it, thanks to the spaces and the capital letter, and browsers would still run it. `URL` reads the address the same way the browser does, so it isn't fooled.

A safety net: Content Security Policy

A **Content Security Policy** (CSP) is a set of rules you give the browser about which scripts are allowed to run on your page. A common strict rule allows only script files from your own site:

```
<meta http-equiv="Content-Security-Policy" content="script-src 'self'">
```

With this rule in the page's `<head>`, the browser refuses to run code written inside the HTML itself, like an `onerror="..."` attribute or a `<script>` block, even if an attacker sneaks one onto the page. Your own files, like `<script src="script.js" defer>`, still work. (It also blocks inline `onclick="..."` attributes, one more reason [chapter 21](#) told you to avoid them.) Real sites usually send this rule from the server, as an HTTP header.

Think of CSP as a seatbelt. You still drive carefully, with `textContent` and escaping, but if something slips through, CSP can stop it from doing harm.

Never run user input as code: `eval` and `new Function`

`eval()` takes a string and runs it as JavaScript. `new Function()` does the same thing, building a function out of strings:

```
console.log(eval("2 + 3")); // prints: 5

const add = new Function("a", "b", "return a + b");
console.log(add(2, 3)); // prints: 5
```

That looks like an easy way to build a calculator: take whatever the user typed and `eval` it. But then the user isn't typing a sum. They're writing *your program*. In a browser, they can run any code on your page (XSS again). On a server, it's even worse: they could read or delete your files.

So the rule is: **never pass user input to `eval` or `new Function`**. You'll almost never need them at all.

The safe way is, once again, an allowlist. Decide exactly what's allowed, and handle each case yourself:

```
const operations = {
  "+": (a, b) => a + b,
  "-": (a, b) => a - b,
  "*": (a, b) => a * b,
  "/": (a, b) => a / b,
};

function calculate(a, operator, b) {
  if (!Object.hasOwn(operations, operator)) {
    throw new Error(`Unknown operator: ${operator}`);
  }
  return operations[operator](Number(a), Number(b));
}

console.log(calculate("12", "*", "3")); // prints: 36

try {
  calculate("2", "alert('hacked')", "3");
} catch (error) {
  console.log(error.message); // prints: Unknown operator: alert('hacked')
}
```

It's the strategy pattern from [chapter 44](#): a lookup table of allowed actions. Whatever the user types, the only code that can ever run is the four small functions you wrote. And to turn text into data, use `JSON.parse` ([chapter 23](#)), never `eval`.

One more hidden `eval`: browsers let you pass a string of code to `setTimeout` and `setInterval`, like `setTimeout("save()", 1000)`, and they run that string the same way. Always pass a function instead ([chapter 30](#)). Node doesn't allow the string version at all, and throws a `TypeError`.

Everything in front-end code is public

Every file your page loads, like `script.js`, is downloaded to the visitor's computer, and they can read all of it. Press `F12`, open the **Sources** tab, and there it is. Minified code ([chapter 50](#)) is harder to read, but it's not secret.

So front-end code must never contain **secrets**: anything that gives access or costs money if someone else gets hold of it, like a paid API's secret key, a database password, or an admin password.

```
// Never do this in front-end code: anyone can open DevTools and copy it
const paymentApiKey = "secret-key-12345";
```

Bots scan public websites and code on GitHub for leaked keys all day long. A key that slips into a public project can be found and misused within minutes.

The fix is to keep secrets on a server. Your page asks *your* server, and your server adds the secret and talks to the paid API:

```
Browser ---> Your server (keeps the secret key) ---> Paid API
```

On the server, keep the secret out of the code too. Put it in a `.env` file, and add `.env` to your `.gitignore` so it never ends up in git:

```
WEATHER_API_KEY=your-real-key-goes-here
```

Then start the server with `node --env-file=.env server.js` ([chapter 50](#)) and read the key with `process.env.WEATHER_API_KEY` ([chapter 49](#)).

Some keys are *meant* to be public, like a key for showing a map that only works on your own website. The service's documentation will tell you which kind you have. When in doubt, treat a key as secret.

Login tokens: keep them away from JavaScript

When you log in to a website, the server gives your browser a **token**: a long random string that proves who you are, like a wristband at a festival. As far as the server knows, whoever holds the token *is* you.

It's tempting to save it with `localStorage.setItem("token", token)` ([chapter 23](#)). But `localStorage` can be read by **any** script running on your page. That includes an attacker's XSS code, and third-party scripts (for analytics, ads or chat) that might get hacked themselves. One line, `localStorage.getItem("token")`, and the account is theirs.

The safer home for a token is an **httpOnly cookie**. A **cookie** is a small piece of data a browser keeps for a website, and sends back automatically with every request to it. The server creates it, with a few safety settings:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax
```

- `HttpOnly` means JavaScript can't read the cookie at all. It doesn't even show up in `document.cookie`, so XSS code can't steal it.
- `Secure` means it's only ever sent over HTTPS.
- `SameSite=Lax` means it isn't sent along when another website secretly sends a request to yours (see CSRF, next).

This is all set up on the server. Your front-end code doesn't need to touch the token at all. `localStorage` is still great for things that aren't secret: a theme, a draft, the items in a cart.

CSRF, in brief

CSRF (cross-site request forgery) is a trick where another website makes your browser send a request to a site you're logged in to.

Say you're logged in to your bank in one tab, and you open an evil page in another. That page has a hidden form that sends "transfer 500 dollars" to your bank. Your browser helpfully attaches your bank cookies, so the bank thinks you sent it.

The defenses are mostly the server's job:

- **SameSite cookies** (above), so the browser doesn't attach them to requests started by other sites.
- **A CSRF token:** a secret random value the server puts into its own forms and checks on every request. The evil site can't know it.
- **Never change data with a GET request.** Use `POST`, `PUT` or `DELETE` for that ([chapter 33](#)).

Passwords are never stored as plain text

Real websites never store your password as you typed it. They store a **hash**: the result of a one-way scramble that can't be turned back into the password. When you log in, the server scrambles what you typed the same way and compares the two results. If the database is ever stolen, the attackers get scrambles, not passwords.

That's server work, done with slow, purpose-built tools like `bcrypt` or `Argon2` (never a homemade scheme). On the front end, your job is simpler: send passwords only over HTTPS, and never save them in `localStorage` or print them with `console.log`.

Dependencies: trust, but check

Every `npm install` brings strangers' code into your project, plus all the packages that *those* packages need. Even a small project can end up with hundreds. This chain of other people's code is called your **supply chain**.

Most of it is fine, but attackers know it's a way in:

- **Hijacked packages.** In 2018, a popular package called `event-stream` was handed over to a new maintainer, who quietly added code that tried to steal from a Bitcoin wallet app. It went unnoticed for about two months.
- **Look-alike names.** In 2017, a fake package called `crossenv` (the real one is `cross-env`) stole secret settings from every project that installed it by mistake.

Good habits:

- **Install only what you need.** Before adding a package, check it's really the one you want: the exact name, lots of weekly downloads, recent updates, and a real project page. And ask yourself: could 20 lines of your own code do the job?
- **Commit your `package-lock.json`** ([chapter 50](#)), so everyone installs exactly the same versions.
- **Run `npm audit` now and then.** It checks your packages against a public list of known security holes.

Here's what `npm audit` printed for a practice project that used an old version of `lodash`, a popular helper library (trimmed):

```
# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
fix available via `npm audit fix`
node_modules/lodash

1 high severity vulnerability
```

Running `npm audit fix` updated `lodash` to a fixed version, and after that `npm audit` said `found 0 vulnerabilities`. Your output will look different, because new problems are reported all the time.

Be careful with `npm audit fix --force`. It may jump to a new major version with breaking changes (remember semantic versioning from chapter 50), so check what it changed and run your tests.

Prototype pollution, in brief

Remember `__proto__` from [chapter 28](#)? It's behind a sneaky attack called **prototype pollution**. It can happen when your code merges data from a user into one of your objects, and that data contains a key named `__proto__`.

Here's a simple "deep merge" helper that copies settings from one object into another. Lots of projects write something like it:


```

function merge(target, source) {
  for (const key in source) {
    const value = source[key];
    if (typeof value === "object" && value !== null) {
      if (typeof target[key] !== "object" || target[key] === null) {
        target[key] = {};
      }
      merge(target[key], value);
    } else {
      target[key] = value;
    }
  }
  return target;
}

const settings = { theme: "light" };
const fromUser = JSON.parse('{"theme": "dark", "__proto__": {"isAdmin": true}}');
merge(settings, fromUser);

const brandNewObject = {};
console.log(brandNewObject.isAdmin); // prints: true

```

A brand-new, empty object claims to be an admin! When the loop reached the key `"__proto__"`, `target["__proto__"]` wasn't a normal property. It was `Object.prototype`, the shared parent of every object. So `isAdmin: true` was written onto that parent, and now **every** object in the program inherits it. If any code checks `user.isAdmin`, everyone is an admin.

The fix: skip the dangerous keys, and only loop over the object's own keys:

```

const FORBIDDEN_KEYS = ["__proto__", "constructor", "prototype"];

function safeMerge(target, source) {
  for (const key of Object.keys(source)) {
    if (FORBIDDEN_KEYS.includes(key)) {
      continue; // never copy a key that could reach a prototype
    }
    const value = source[key];
    if (typeof value === "object" && value !== null) {
      if (typeof target[key] !== "object" || target[key] === null) {
        target[key] = {};
      }
      safeMerge(target[key], value);
    } else {
      target[key] = value;
    }
  }
  return target;
}

const settings = { theme: "light" };
safeMerge(settings, JSON.parse('{"theme": "dark", "__proto__": {"isAdmin": true}}'));
console.log(settings.theme, {}.isAdmin); // prints: dark undefined

```

Other ways to stay safe: when the keys come from users, store the data in a `Map` ([chapter 35](#)) instead of a plain object. Or copy only the fields you expect, like `{ theme: fromUser.theme }`. And keep your libraries updated. The "Prototype Pollution in lodash" line in the `npm audit` report above is exactly this bug, in a library millions of projects use.

HTTPS

Plain **HTTP** sends everything as readable text. Anyone on the same cafe Wi-Fi, or anywhere along the way, could read it or even change it. **HTTPS** encrypts the connection between the browser and the server. It's the padlock next to the address in your browser.

- Serve your site over HTTPS, and only call APIs with `https://` addresses. Hosts like GitHub Pages and Netlify give you HTTPS for free ([chapter 52](#)).
- Browsers mark plain HTTP pages as "Not secure". Some features, like finding the user's location with `navigator.geolocation` and the clipboard API, only work on HTTPS pages (or on `localhost` while you develop).
- If an HTTPS page tries to load a script over plain `http://`, the browser blocks it.

Common mistakes

1. Putting user text into `innerHTML`

```
const username = new URLSearchParams(location.search).get("name");
greeting.innerHTML = `Welcome back, ${username}!`;
```

Anyone can send a friend a link with HTML hidden in `?name=...`. Fix: `greeting.textContent = ...`. The same goes for anything that came from a form, an API or `localStorage`.

2. Checking only in the browser

```
<input type="number" name="quantity" min="1" max="10">
```

`max="10"` stops honest users from typing 11. It doesn't stop anyone who opens DevTools or sends a request with `fetch`. The server must check again.

3. Blocking what you fear instead of allowing what you expect

```
const address = "  JavaScript:alert('hacked')";
console.log(!address.startsWith("javascript:")); // prints: true
```

This "is it safe?" check says yes, fooled by two spaces and a capital letter. Use an allowlist instead: read the address with `URL` and accept only `https:` and `http:`.

4. Committing a secret to git

A `.env` file that isn't in `.gitignore`, or a key typed into the code "just for a quick test". Once a secret has been pushed, deleting it later isn't enough: it stays in the git history, and bots may already have copied it. Cancel the key straight away (services call this **revoking** it) and create a new one.

5. Logging secrets

```
console.log("Logging in with", email, password);
```

Logs end up in files, terminals and error-tracking tools that other people can read. Log *that* something happened, like `"Login attempt"`, never the password or token itself.

Quick recap

- Never trust user input. The browser checks for convenience, and the server checks for safety.
- Allow what you expect (an allowlist), instead of trying to block every bad thing.

- Never put user text into `innerHTML`: use `textContent`, escape it, or clean it with `DOMPurify`. Only allow `http:` and `https:` links, and add a CSP as a safety net.
 - Never run user input as code, whether with `eval`, `new Function` or a string in `setTimeout`.
 - Front-end code is public. Keep secrets on a server, in a `.env` file that's listed in `.gitignore`.
 - Keep login tokens in `httpOnly` cookies, not `localStorage`, and use HTTPS everywhere.
 - Install only packages you trust, commit your lockfile, run `npm audit`, and skip `__proto__` keys when merging user data.
-

Next: try the [exercises](#), then move on to [52 Final Project](#).

Exercises

How to do these:

- Exercises 1, 2, 4 and 5 are Node exercises: make one file per exercise in this folder (`ex1.js`, `ex2.js`, and so on) and run it with `node ex2.js`.
 - Exercises 2 and 5 use `import / export`, so add a `package.json` containing `{ "type": "module" }` to this folder first ([chapter 29](#)).
 - Exercise 3 is a browser exercise: make a folder `ex3` with `index.html` and `script.js`, open `index.html` by double-clicking it, and press `F12` for the Console.
 - Only ever try these attacks on your own pages.
 - Try on your own first. Only open a hint if you've been stuck for a while.
 - When you're done, ask Claude to check your code.
-

Exercise 1 (Easy): Spot the vulnerability

A friend asks you to review their app before it goes live. Each of these six snippets has one security problem. In `ex1.js`, write a comment for each one: **what's wrong**, **what an attacker could do**, and **how to fix it**.

A. Showing product reviews that came from an API (visitors wrote them):

```
for (const review of reviews) {  
  reviewList.innerHTML += `- ${review.author}: ${review.text}</li>`;  
}

```

B. A weather widget in the page's `script.js`:

```
const weatherApiSecret = "secret-key-98765";
fetch(`https://api.weather.example/today?city=Oslo&key=${weatherApiSecret}`);
```

C. After a successful login:

```
localStorage.setItem("authToken", loginResult.token);
```

D. A calculator page:

```
const answer = eval(calculationInput.value);
resultBox.textContent = answer;
```

E. On the server, saving a user's settings with the simple `merge` helper from the notes:

```
const userSettings = merge({}, JSON.parse(requestBody));
```

F. On the server, working out an order's total:

```
const total = order.price * order.quantity; // order was sent by the browser
```

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 2 (Easy): Build `escapeHTML`

Write a function `escapeHTML(value)` that swaps the five special characters from the notes (`&`, `<`, `>`, `"` and `'`) for their HTML entities. Put it in its own file, `escape.js`, and export it. You'll reuse it in Exercise 5.

Then create `ex2.js` with these tests:

```
import { escapeHTML } from "../escape.js";

console.log(escapeHTML("Tom & Jerry"));
console.log(escapeHTML("<b>bold</b>"));
console.log(escapeHTML(``));
console.log(escapeHTML("5 < 10 && 10 > 5"));
console.log(escapeHTML("Hello, world!"));
console.log(escapeHTML(42));
```

Expected output:

```
Tom & Jerry
<b>

5 < 10 & 10 > 5
Hello, world!
42
```

Rule: replace every special character, not only the first one of each kind. And it must work for numbers too (the last test).

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 3 (Medium): Fix the comment box

A recipe website lets visitors leave comments, but its code has an XSS hole. Use this HTML for `ex3/index.html`:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Recipe Comments</title>
    <script src="script.js" defer></script>
  </head>
  <body>
    <h1>Grandma's Apple Pie</h1>
    <form id="comment-form">
      <label for="name">Name</label>
      <input id="name" required>
      <label for="comment">Comment</label>
      <textarea id="comment" required></textarea>
      <button type="submit">Post comment</button>
    </form>
    <ul id="comments"></ul>
  </body>
</html>
```

And this vulnerable code for `ex3/script.js`:

```
const form = document.getElementById("comment-form");
const nameInput = document.getElementById("name");
const commentInput = document.getElementById("comment");
const commentList = document.getElementById("comments");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  commentList.innerHTML += `<li><strong>${nameInput.value}</strong>: ${commentInput.value}
    </li>`;
  form.reset();
});
```

1. **See the hole.** Post a normal comment first. Then post one with the name `Sam` and this comment (it's harmless, it only shows an alert): ``. An alert pops up: your page just ran code that a visitor typed.
2. **Fix `script.js`** so every comment is shown as plain text, and the name is still bold.

After the fix, posting that same comment shows this on the page, and no alert appears:

- Sam: ``

Rule: no `innerHTML` (or `outerHTML` or `insertAdjacentHTML`) anywhere in your fixed code.

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Bonus: try the attack in the *name* field too. Is your fix safe there as well?

Exercise 4 (Medium): Find the leaked secrets

A teammate is about to make their weather project public on GitHub, and asks you to review it first. Here are the files that matter.

`public/app.js`, which runs in the browser:

```
const WEATHER_KEY = "wk-live-4f8a2c91";
const ADMIN_PASSWORD = "sunflower42";

async function loadWeather(city) {
  const response = await fetch(`https://api.weather.example/v1?
    city=${city}&key=${WEATHER_KEY}`);
  return response.json();
}

function isAdmin(typedPassword) {
  return typedPassword === ADMIN_PASSWORD;
}
```

server.js, which runs on the server:

```
const databasePassword = "practice-only-123";
console.log("Connecting with password:", databasePassword);
```

And .gitignore:

```
node_modules
```

Part 1: review. In a comment at the top of ex4.js, list every problem you can find (there are at least four) and how you'd fix each one.

Part 2: fix the server. Write the server code again in ex4.js, this time without the password in it:

1. Create a .env file in this folder containing DATABASE_PASSWORD=practice-only-123 (a made-up practice password).
2. Create a .gitignore file next to it that lists .env, the way you'd fix your teammate's.
3. Read the password with process.env. Don't print it. Print whether it loaded, and a masked preview: the first 3 characters, then one * for each remaining character.

Running node --env-file=.env ex4.js should print:

```
Database password loaded: yes
Password preview: pra*****
```

And running plain node ex4.js, without --env-file, shouldn't crash (Node only reads .env when you ask it to):

```
Database password loaded: no
Password preview: (none)
```


Rule: the password must not appear anywhere in `ex4.js`, and the full password must never be printed.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Exercise 5 (Challenge): A safe profile card

On a hobby website, users fill in a profile: a username, a short bio, and a website. Your job is to turn each profile into a small piece of HTML, safely. Start `ex5.js` with this:

```
import { escapeHTML } from './escape.js';

const profiles = [
  { username: "sam_codes", bio: "I love <b>JavaScript</b> & hiking.", website:
    "https://sam.example" },
  { username: "mallory", bio: ``, website:
    "javascript:alert('hacked')"},
  { username: "<script>", bio: "Hi!", website: "https://ok.example" },
  { username: "long_story", bio: "I have baked bread every weekend since I was twelve, & I still
    love it.", website: "not a url" },
];

// Write renderProfile(profile) here

for (const profile of profiles) {
  try {
    console.log(renderProfile(profile));
  } catch (error) {
    console.log("Rejected:", error.message);
  }
}
```

`renderProfile(profile)` returns one line of HTML and follows these rules:

- **Username:** 3 to 20 characters, using only letters, digits and `_`. Anything else throws an error with the message `Invalid username`.
- **Bio:** if it's longer than 60 characters, keep the first 60 and add `...`.
- **Website:** only `https:` and `http:` addresses become a link, like `<a href="https://sam.example">https://sam.example</a>`. Anything else becomes `<p>No website</p>`.
- **Everything the user typed** (username, bio and website) goes through `escapeHTML` before it goes into the HTML.
- The result looks like this: `<div class="profile"><h2>username</h2><p>bio</p>` + the website part + `</div>`.

Expected output:

```
<div class="profile"><h2>sam_codes</h2><p>I love <b>JavaScript</b> & hiking.</p>
<a href="https://sam.example">https://sam.example</a></div>
<div class="profile"><h2>mallory</h2><p></p><p>No website</p></div>
Rejected: Invalid username
<div class="profile"><h2>long_story</h2><p>I have baked bread every weekend since I was twelve,
& I sti...</p><p>No website</p></div>
```

Rules: check the username with a regular expression ([chapter 37](#)), and the website with `URL`. Don't change the starter data or the loop.

Stuck? Hint 1, Hint 2 and Hint 3 are in the [Hints section](#) at the end.

Before you move on

You can now build apps *and* keep them safe. In [chapter 52](#), you'll plan and build a project of your own, from the first idea to sharing it online. Give it the same security check you practiced here before you show it to the world.

52 Final Project

You've reached the last chapter of the course. 🎉

This time, there's no step-by-step recipe to follow. You'll pick your own project and build it from start to finish, the way professional developers do. That's a big jump from following milestones, so this chapter shows you the process, one step at a time.

What you'll build

Something of your own! The [exercises](#) have a menu of eight project ideas, from a small Pomodoro timer to a Kanban board. You can also bring your own idea.

To show you the process, this chapter follows one small example project all the way through: **Plant Pal**, a page that shows which of your houseplants need water today. It isn't on the menu, so there's nothing to copy. Watch *how* it gets built, then do the same with your project.

Here are the steps:

1. Pick one small idea.
2. Split the must-haves from the nice-to-haves.
3. Sketch the screens and the data.
4. Plan your files.
5. Build the smallest working version, in small steps.
6. Test the logic.
7. Ask for a code review.
8. Polish.
9. Share it.

Professionals follow roughly this path whether the project takes a weekend or a year. The steps are what make a project finishable.

Step 1: Pick one small idea

A good project starts with a problem you can describe in one sentence:

Plant Pal: see which of my plants need water today.

When you pick your idea:

- **Pick something you'd actually use**, or that a friend would. You'll care more about finishing it, and you'll notice what's missing.
- **Keep it small.** A small app that's finished beats a big app that isn't. Cutting a project down to size is a real skill that professionals use every week.
- **Watch out for "and".** If your one-sentence description needs "and" three times, it's probably three projects.

Too big	Just right
A social network for gardeners	A page that shows which plants need water
A budgeting app that connects to your bank	An expense tracker for one person
A 3D racing game	A typing speed test

Step 2: Must-haves and nice-to-haves

Before you write any code, make two lists:

- **Must-haves** are the features that make the app useful at all. Without any one of them, the app doesn't do its job.
- **Nice-to-haves** would be great, *later*.

Together, the must-haves are your **MVP**, short for **minimum viable product**: the smallest version of the app that's actually useful. You build that first, and nothing else.

Must-have (the MVP)	Nice-to-have (later, maybe)
See a list of my plants	Delete a plant
See when each one needs water next	Sort them, thirstiest first
Press "Watered" to reset a plant's timer	A photo of each plant
Add a new plant	A reminder notification
My plants are still there after a refresh	Share the list with my roommate

Look at the last nice-to-have. Sharing needs a server and user accounts, which is a whole project of its own. That's exactly why it sits on the right. Remember YAGNI from [chapter 45](#): "You Aren't Gonna Need It". Nice-to-haves wait until the must-haves work.

Step 3: Sketch the screens and the data

Grab a pen and paper. Boxes and words are enough: what does the person see, and what can they click?

```
Plant Pal
-----
Fern      Water today!    [Watered]
Basil     in 1 day       [Watered]
Cactus    in 12 days     [Watered]
-----
Name [_____] Every [ 3 ] days [Add plant]
```

Then sketch the **data**: what does the app need to remember? For Plant Pal, it's an array of objects, one per plant ([chapter 11](#)):

```
const plants = [
  { id: 1, name: "Fern", everyDays: 3, lastWatered: "2026-09-21" },
  { id: 2, name: "Basil", everyDays: 2, lastWatered: "2026-09-23" },
  { id: 3, name: "Cactus", everyDays: 14, lastWatered: "2026-09-22" },
];
```

(If today is September 24th, 2026, that's exactly the sketch above.)

Three good habits are hiding in that data:

- **Store the facts, and work out the rest.** The data remembers *when* each plant was last watered, not the words "Water today!". That label changes every day, so the app works it out fresh each time it draws the page.
- **Store dates as ISO strings**, like `"2026-09-21"`. They survive the trip through JSON and `localStorage` ([chapter 23](#)), and nobody has to guess what `03/04` means ([chapter 19](#)).
- **Give every item an id**. Names can change or repeat, but an id never does. A "Watered" button can carry its plant's id in a `data-id` attribute, so one listener knows which plant was clicked ([chapter 21](#)).

Sketching takes ten minutes, and it saves hours. Most "how do I build this?" questions turn out to be "what should my data look like?" questions.

Step 4: Plan your files

Now decide which file does which job:

```
plant-pal/
├─ index.html      the page
├─ style.css       the looks
├─ main.js         starts the app, listens to events, decides what happens
├─ plants.js       the logic: plain functions, no page code at all
├─ storage.js      save() and load(), from chapter 23
├─ ui.js           draws the plants on the page
├─ plants.test.js  tests for plants.js
└─ package.json    { "type": "module" }, so Node can run the tests
```

It's the same idea as the weather app in [chapter 39](#): each file has one job, and when something breaks, you know where to look.

The most important line in that plan is `plants.js`. It never touches the page. It takes data in and hands answers back, like the pure functions in [chapter 43](#). That has a big payoff: Node can run its tests. Node has no `document`, so any file that touches the page can't be tested there, but a file of plain functions can.

Because the app uses modules, open it with **Live Server**, not by double-clicking ([chapter 29](#)).

Step 5: Build the smallest working version first

An **MVP** (minimum viable product) is the smallest version of your app that works from start to finish. For Plant Pal: show a fixed list of plants with their "water in N days" labels. No form, no saving, no styling yet.

Then add one must-have at a time, and run the app after each one:

1. Show a hard-coded list of plants.
2. Add a plant with the form.
3. The "Watered" button updates a plant.
4. Save and load with `localStorage`.
5. Only then: styling and nice-to-haves.

Think of building a house: foundation first, then walls, then paint. If you paint first, you'll repaint everything later.

Tip: if a step feels too big, it is. Split it until each piece takes under an hour.

Step 6: Test the logic

Because `plants.js` has no page code, you can test it with Node's built-in test runner ([chapter 46](#)). Here's one function and its tests:

```
// plants.js
const DAY_MS = 24 * 60 * 60 * 1000;

// How many days until this plant needs water? 0 means "today" (or overdue).
export function daysUntilWater(plant, today) {
  const last = new Date(plant.lastWatered);
  const next = new Date(last.getTime() + plant.everyDays * DAY_MS);
  const days = Math.round((next - new Date(today)) / DAY_MS);
  return Math.max(days, 0);
}
```

```
// plants.test.js
import { test } from "node:test";
import assert from "node:assert/strict";
import { daysUntilWater } from "../plants.js";

test("a fern watered 3 days ago needs water today", () => {
  const fern = { name: "Fern", everyDays: 3, lastWatered: "2026-09-21" };
  assert.equal(daysUntilWater(fern, "2026-09-24"), 0);
});

test("an overdue plant still says 0, never a negative number", () => {
  const basil = { name: "Basil", everyDays: 2, lastWatered: "2026-09-01" };
  assert.equal(daysUntilWater(basil, "2026-09-24"), 0);
});
```

Run `node --test` in the project folder. Notice that `today` is a parameter instead of reading the real date inside the function. That makes the function pure, so the tests give the same answer every day.

Step 7: Save your progress often

Professionals use **git** to save snapshots of their code, so they can go back if something breaks. It'll get its own folder in this course. Until then, at least copy your project folder to a backup after each working step.

Step 8: Ask for a code review

When a feature works, ask Claude to review it. A good request looks like this:

"Here's my `plants.js` and `main.js` for Plant Pal. It works, but please review it for bugs, naming, and anything I could make simpler."

Use the feedback from [chapter 45](#) and [chapter 51](#) as your own checklist before you ask.

Step 9: Polish and share

Before you share, check:

- It works on a phone-sized window (F12, then the device toolbar icon).
- Errors show friendly messages, not a blank page ([chapter 18](#)).
- User text goes in with `textContent`, never `innerHTML` ([chapter 51](#)).
- Import paths match file names exactly, including capital letters. Windows forgives `./Plants.js` for `plants.js`, but web servers usually don't.

To put it online for free, **GitHub Pages** and **Netlify** can host a folder of HTML, CSS, and JavaScript. Both have short "getting started" guides, and both give you HTTPS automatically.

Your skills checklist

Try to show off as many of these as fit your idea (don't force them in):

Skill	Chapter
Arrays of objects and array methods	10 , 11 , 13
DOM, events, and event delegation	20 , 21
Forms with validation	22
Saving with <code>localStorage</code>	23
Modules	29
<code>fetch</code> with loading and error states	33
Debounce or throttle	34
Pure functions and tests	43 , 46
Clean code and security basics	45 , 51

Common mistakes

- 1. Starting too big.** "A social network with chat" never gets finished. "A habit tracker that saves to `localStorage`" does. Finish small, then grow it.
- 2. Styling before it works.** Hours of CSS on a page whose buttons don't work yet. Build the MVP first.

3. Mixing logic and page code. If your calculations live inside click handlers, you can't test them. Keep them in plain functions in their own file.

4. Writing a lot before running anything. Run the app after every small change. A bug from the last five minutes is easy to find. A bug from the last five hours isn't.

Quick recap

- Pick one small idea, and split features into must-haves and nice-to-haves.
 - Sketch the screens and the data before writing code.
 - Give each file one job, and keep the logic in pure, testable functions.
 - Build an MVP first, then add one feature at a time, testing as you go.
 - Get a review, polish, and share it online.
-

Congratulations! 🎉

You've gone from `console.log("Hello, world!")` to building, testing, and shipping real apps. That's a huge journey. Be proud of it.

Where to go next:

- **TypeScript:** JavaScript with types, which catches many bugs before you even run the code.
- **React:** the most popular library for building bigger user interfaces.
- **Node.js frameworks** like Express: build real servers and APIs.
- **Data structures and algorithms:** the deeper computer science behind fast code.

Each of these will get its own folder here. Until then, pick a project idea from the [exercises](#) and build it.

Back to the roadmap

Exercises

How to do these:

- Pick **one** project from this menu. (Or your own idea, if you follow the same steps from the notes.)
 - Make a new folder for it, like `plant-pal/`, anywhere you like.
 - Follow the 9 steps in [notes.md](#): must-haves first, then nice-to-haves.
 - Share your code with Claude for a review after each working feature.
-

Exercise 1 (Easy): Expense Tracker

Track what you spend in a week, so you can see where your money goes.

Must-haves:

- Add an expense: description, amount, category (food, transport, fun...).
- Show the list and the total.
- Delete an expense.
- Save everything with `localStorage`.

Nice-to-haves: a total per category, filter by category, a simple bar chart made with plain CSS widths.

Practices: forms (22), array methods (13), `localStorage` (23), numbers and money (05).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 2 (Easy): Pomodoro Timer

A study timer: 25 minutes of focus, then a 5-minute break.

Must-haves:

- Show the time left as `mm:ss`.
- Start, pause, and reset buttons.
- Switch between "Focus" and "Break" automatically.

Nice-to-haves: count finished sessions (saved in `localStorage`), let the user change the lengths, play a sound when time is up.

Practices: timers (30), DOM and events (20, 21), `padStart` (06).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 3 (Medium): Book Search

Search for books and keep a "want to read" list.

Must-haves:

- Search box with debounced suggestions as you type.
- Show title, author, and first publish year for each result.
- Loading and error messages.

- Add a book to your reading list (saved in `localStorage`).

Uses the free Open Library API, no key needed: `https://openlibrary.org/search.json?q=the+hobbit&limit=5`

Nice-to-haves: cover images (`https://covers.openlibrary.org/b/id/<cover_i>-M.jpg`), mark books as read, cancel stale searches with `AbortController` .

Practices: fetch (33), debounce (34), modules (29), localStorage (23).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Exercise 4 (Medium): Habit Tracker

Tick off daily habits (drink water, read, exercise) and see your streaks.

Must-haves:

- Add and remove habits.
- A checkbox for each habit for today.
- Show each habit's current streak (days in a row).
- Save with `localStorage` .

Nice-to-haves: a 7-day grid for each habit, a "best streak" record.

Practices: dates (19), pure functions and tests (43, 46), events (21).

Stuck? The hint is in the [Hints section](#) at the end.

Exercise 5 (Challenge): Kanban Board

A mini Trello: cards in three columns, "To do", "Doing", and "Done".

Must-haves:

- Add a card to "To do".
- Move a card to the next or previous column with buttons.
- Delete a card.
- Save the whole board with `localStorage` .

Nice-to-haves: drag and drop, edit a card's text, a search box that filters cards (debounced).

Practices: event delegation (21), classes or pure functions (27, 43), clean code (45), tests (46).

Stuck? Hint 1 and Hint 2 are in the [Hints section](#) at the end.

Hints

Try each exercise on your own first. Come here only when you're stuck, and read one hint at a time.

01 Getting Started

Exercise 1 (Easy): Say hello

[Back to the exercise](#)

Hint

Use one `console.log()` per line. Remember that text needs quotes.

Exercise 2 (Easy): Calculator

[Back to the exercise](#)

Hint

In JavaScript, `*` means multiply and `/` means divide. Remember the comma trick: you can print a label and a calculation in the same `console.log()`.

Exercise 3 (Medium): Cafe receipt

[Back to the exercise](#)

Hint

For the total, you can write the whole calculation on one line. JavaScript does `*` before `+`, just like the math you learned in school.

Exercise 4 (Medium): Bug hunt

[Back to the exercise](#)

Hint 1

Read each error message carefully. It tells you the line number and what's wrong.

Hint 2

The errors might not show up in line order. JavaScript first checks the whole file for "grammar" mistakes (typos, missing brackets), and only then starts running it. That's normal.

Hint 3

Look closely at the text `Let's`. It contains an apostrophe, which is the same symbol as a single quote. Which kind of quotes should wrap it?

Exercise 5 (Challenge): Your life in numbers

[Back to the exercise](#)

Hint

Each fact builds on the one before it. Hours alive is just days alive $\times$ 24. It's fine to write a long calculation on one line, like `25 * 365 * 24`.

02 Variables

Exercise 1 (Easy): Your profile card

[Back to the exercise](#)

Hint

Remember the comma trick from chapter 01: you can put a label and a value in the same `console.log()`, separated by a comma. A variable name works as the value.

Exercise 2 (Easy): Basketball scoreboard

[Back to the exercise](#)

Hint 1

The score changes, so which keyword should you use to create it: `let` or `const`?

Hint 2

Look at "Updating from the old value" in the notes. The new score is the old score plus the points for that basket.

Exercise 3 (Medium): Bug hunt at the library

[Back to the exercise](#)

Hint 1

Same method as the bug hunt in chapter 01: run the file, read the error and its line number, fix that one thing, and run it again.

Hint 2

Two of the names break the naming rules (check the table in the notes). When you rename a variable, rename it everywhere it's used, not only where it's created.

Hint 3

One of the error messages is confusing: "Missing initializer in const declaration". JavaScript reads the dashes in `second-book-days` as minus signs, so it thinks you made a `const` called `second` and forgot its value. What does that tell you about dashes in names?

Hint 4

The last bug only shows up after the others are fixed. Ask yourself: which value changes after it's created?

Exercise 4 (Medium): The mixed-up drinks

[Back to the exercise](#)

Hint 1

Try writing `table1 = table2;` and then print both tables. Where did the orange juice go?

Hint 2

Picture a real waiter holding a drink in each hand. To swap them, they'd put one drink down on a tray for a moment. Your program needs a "tray" too.

Exercise 5 (Challenge): Your life in numbers, upgraded

[Back to the exercise](#)

Hint 1

Start with the fixed values at the top of the file, each in its own `const` with a clear name, such as `daysPerYear`.

Hint 2

Work down the list in order. Days alive uses your age. Hours alive uses days alive. Which variable should minutes alive use?

Hint 3

Heartbeats and breaths both come from the same fact. Which one?

03 Data Types

Exercise 1 (Easy): The library record

[Back to the exercise](#)

Hint 1

"Yes" is a job for a boolean. For "empty, on purpose" and "not decided yet", reread the section on `undefined` and `null`.

Hint 2

One `console.log()` can hold a label, a variable, the text `"->"` and the variable's type, all separated by commas.

Exercise 2 (Easy): Conference name badge

[Back to the exercise](#)

Hint 1

`+` never adds spaces for you. Look at the expected output and find every space. Each one has to come from a string you write.

Hint 2

The last line contains an apostrophe in `I'm`. Remember the `Let's` bug in chapter 01's bug hunt: which kind of quotes can safely go around it?

Exercise 3 (Medium): The broken basket

[Back to the exercise](#)

Hint 1

Check each of the three values with `typeof`. Which ones are strings, and which is a number? How does that explain `20125`?

Hint 2

Turn the text into numbers before adding them. The `Number()` section in the notes shows how.

Exercise 4 (Medium): Hotel check-in card

[Back to the exercise](#)

Hint 1

"None" and "not assigned yet" are two different kinds of nothing. Which one do *you* put in the box, and which one does JavaScript give you?

Hint 2

To get `Room: undefined`, create the room variable without a value (you did this with `let winner;` in chapter 02). Later, put `214` into it.

Exercise 5 (Challenge): Cinema booking summary

[Back to the exercise](#)

Hint 1

Convert the two ticket counts into numbers first, each in its own variable. Everything else builds on those two.

Hint 2

Seeing `Total: 248 dollars` or `Tickets: 21`? You've hit the trap from "The `"5" + 3` surprise" in the notes. Work out the totals in their own variables before joining them into text.

Hint 3

The `Tickets:` line has many pieces: text, a number, text, another number, and so on. Build it one piece at a time, and run your file after each piece to check the spaces and brackets.

04 Operators

Exercise 1 (Easy): Pizza party

[Back to the exercise](#)

Hint 1

Which operator tells you what's left over after sharing things out equally?

Hint 2

For "Slices each", use the same trick as "Putting it together: minutes into hours" in the notes.

Hint 3

"Even split" is `true` only when nothing is left over. Which operator asks "are these two values equal?"

Exercise 2 (Easy): Cafe loyalty card

[Back to the exercise](#)

Hint 1

You need two `let` variables: one for the points and one for the visits.

Hint 2

Doubling means multiplying by 2. Which shortcut multiplies a variable by something?

Exercise 3 (Medium): Movie marathon

[Back to the exercise](#)

Hint 1

Take it one step at a time, with a variable for each step: the films, the breaks, then the total.

Hint 2

Turning 493 minutes into hours and minutes is the same trick as the film example in the notes.

Hint 3

For the end time, count in minutes since midnight. 10:00 is `10 * 60` minutes after midnight. Add the total, then turn the result back into hours and minutes.

Hint 4

To print `18:13` with no spaces around the colon, join the hours, `":"` and the minutes with `+` (remember chapter 03?).

Exercise 4 (Medium): Theme park gatekeeper

[Back to the exercise](#)

Hint 1

Turn each part of a rule into a comparison first, then join the parts. "At least 140 cm tall" is `height >= 140`.

Hint 2

The haunted house rule mixes AND with OR. Put brackets around the OR part, like the free delivery example in the notes.

Hint 3

"Not with an adult" is `!withAdult`.

Exercise 5 (Challenge): Bookshop checkout

[Back to the exercise](#)

Hint 1

Convert the quantities into numbers first (chapter 03), each into its own variable. Everything else builds on them.

Hint 2

Free delivery needs just one of two things to be true. Which logical operator means "at least one of these"?

Hint 3

"Even number of books" is the even/odd check from the notes: `%`, then `== 0`.

05 Numbers and Math

Exercise 1 (Easy): Party planner

[Back to the exercise](#)

Hint 1

"Tables needed" and "Full tables" use the same division. One rounds up and one rounds down. If you round down for "Tables needed", where does the 47th guest sit?

Hint 2

For the empty seats: how many seats do all your tables have together? Take the guests away from that. Spare cupcakes work the same way.

Exercise 2 (Easy): The corner shop till

[Back to the exercise](#)

Hint 1

8% of an amount is the amount times `0.08`. The tax can come out with a fraction of a cent. Which `Math` tool rounds to the nearest whole number?

Hint 2

To show 1848 cents as `$18.48`: divide by 100, use `toFixed(2)`, and join a `"$"` in front with `+`. (Or try an `Intl.NumberFormat` formatter from the notes.)

Exercise 3 (Medium): Parcel shipping calculator

[Back to the exercise](#)

Hint 1

Which one keeps the `.75` in `"2.75kg"`: `parseInt` or `parseFloat`? For the sizes, whole numbers are fine.

Hint 2

The last line asks "is it a number?", but `Number.isNaN` answers the opposite question: "is it `NaN`?". Which operator from chapter 04 flips `true` and `false`?

Exercise 4 (Medium): Board game night

[Back to the exercise](#)

Hint 1

The dice recipe is in the notes. Do you need to change it to pick one of 4 players?

Hint 2

A double means both dice show the same number. Which comparison operator checks that?

Exercise 5 (Challenge): Split the bill

[Back to the exercise](#)

Hint 1

Start by turning the three texts into numbers. Which tool suits "87.40", and which suits "15" and "3" ?

Hint 2

For the bill in cents, multiply by 100 and round the result. The multiplication can leave one of those tiny decimal errors, and rounding cleans it up.

Hint 3

Which rounding tool always goes up? And to check "Splits evenly", ask whether sharing the total cents between the friends leaves anything over (chapter 04).

06 Strings

Exercise 1 (Easy): Name badge

[Back to the exercise](#)

Hint 1

One string method repeats text as many times as you like.

Hint 2

The first letter of a string is at index 0. For the letter count, the space between the names doesn't count, so add up the two lengths.

Exercise 2 (Easy): Clean up a sign-up form

[Back to the exercise](#)

Hint 1

You can do the whole clean-up in one line by chaining two methods. Save the result in a new variable.

Hint 2

`includes` and `endsWith` give you `true` or `false`, and you can put them straight into a `${ }` slot. For Gmail, what does every Gmail address end with?

Exercise 3 (Medium): Library usernames

[Back to the exercise](#)

Hint 1

Clean the name first. Then find *where* the space is. There's a method that tells you the index of some text.

Hint 2

Once you know the space's index, `slice` can cut out everything before it and everything after it. Remember that `slice(start, end)` stops just before `end`.

Hint 3

`padStart` only works on strings, and `memberNumber` is a number. You saw how to fix that in the notes.

Exercise 4 (Medium): Restaurant receipt

[Back to the exercise](#)

Hint 1

Build each item line from two pieces. The left piece (like `2 x Margherita pizza`) is padded at the end to 20 characters. The right piece (like `$23.00`) is padded at the start to 8 characters. $20 + 8 = 28$.

Hint 2

`toFixed(2)` turns a number into a string with two decimals. That's exactly what you need before padding. You can pad a template literal directly: ``${something}`.padStart(8)`.

Exercise 5 (Challenge): Parcel tracking codes

[Back to the exercise](#)

Hint 1

Every code has the same shape, so the pieces are always at the same positions. Write the clean code down with the index under each character, like the date example in the notes. Then `slice` out each piece.

Hint 2

To print the parts, use the comma trick from [chapter 01](#): `console.log("Parts:", ...)`. If you put the parts inside a `${ }` slot instead, they'll print as plain text without the brackets.

Hint 3

`Number()` from [chapter 03](#) turns `"00451"` into `451`. For "Looks valid", you need `===` and `&&` from [chapter 04](#). For "Next parcel", keep the first 12 characters of the code as they are and build a new ending.

07 Conditionals

Exercise 1 (Easy): Gym lockers

[Back to the exercise](#)

Hint 1

Remember `%` from [chapter 04](#)? It gives you the remainder of a division. An even number divided by 2 leaves a remainder of 0.

Hint 2

Check for the lockers that don't exist *first*. That's the problem case. Once you know the number is between 1 and 100, you only have to decide left or right.

Exercise 2 (Easy): Exam results

[Back to the exercise](#)

Hint 1

Remember that the first true condition wins. Which grade should you check for first?

Hint 2

Save the grade in a variable created with `let` before your `if`, like the cinema ticket example in the notes. Then print it once at the end.

Exercise 3 (Medium): Game settings

[Back to the exercise](#)

Hint 1

For each setting, ask the question from the notes: "Is `0` or an empty string a real, valid answer here?"

Hint 2

For the (muted) part, a ternary inside the template literal works well. When the volume isn't `0`, it can give you an empty string `""`.

Exercise 4 (Medium): Gym opening hours

[Back to the exercise](#)

Hint 1

Stack the cases that share hours on top of each other, like the `w` key and the up arrow in the notes.

Hint 2

If "Funday" prints the wrong thing, which part of the `switch` runs when no case matches? And if one day prints *several* lines, check your `break` s.

Exercise 5 (Challenge): Login check

[Back to the exercise](#)

Hint 1

Clean up the typed username first, using two string methods from [chapter 06](#). Save it in a new variable and use that one in all your checks.

Hint 2

One `if` with three `else if` s, in the order above. The "wrong" check is true if the username doesn't match **or** the password doesn't match.

Hint 3

For the name in the welcome message, you need a default value for when `nickname` is `null`. You learned two operators for that.

08 Loops

Exercise 1 (Easy): Times table

[Back to the exercise](#)

Hint

You know exactly how many lines you need (10), so a `for` loop fits. The counter can be the number you multiply by.

Exercise 2 (Easy): Vowel counter

[Back to the exercise](#)

Hint 1

Look at the Mississippi example in the notes. You need a counter created before the loop, and a `for...of` loop to look at each character.

Hint 2

Checking `letter === "a" || letter === "e" || ...` for all ten letters is a lot of typing. Try it the other way round: is the letter included in the string `"aeiou"`? And what could you do to the letter first so that `"A"` counts too?

Exercise 3 (Medium): FizzBuzz

[Back to the exercise](#)

Hint 1

A number is a multiple of 3 when dividing it by 3 leaves no remainder. Which operator from [chapter 04](#) gives you the remainder?

Hint 2

If 15 prints `Fizz` instead of `FizzBuzz`, remember the "wrong order" mistake from [chapter 07](#). The first true condition wins, so which check should come first?

Exercise 4 (Medium): Cinema seat map

[Back to the exercise](#)

Hint 1

This is a nested loop. The outer loop goes through the row letters (a `for...of` over `rows` works nicely). The inner loop counts the seats from 1 to `seatsPerRow`.

Hint 2

Build each row as a string, like the multiplication table in the notes: start with the row letter and a space, add one `[X]` or `[ ]` per seat, and print the string after the inner loop.

Hint 3

Inside the inner loop, make the seat's name from the row letter and the seat number (like `"B2"`) with a template literal. Then ask whether `bookedSeats` includes it.

Exercise 5 (Challenge): Phone PIN lock

[Back to the exercise](#)

Hint 1

Attempt 1 is at indexes 0 to 3, attempt 2 at 4 to 7, and attempt 3 at 8 to 11. Can you work out the start index from the attempt number? Then `slice` from [chapter 06](#) can cut out the 4 digits.

Hint 2

When the PIN is right, print the message and `break` out of the loop.

Hint 3

After the loop, how do you know whether it ended because of `break` or because the attempts ran out? Create a boolean like `let isUnlocked = false;` before the loop, and change it to `true` when the phone unlocks. After the loop, an `if` can check it.

09 Functions

Exercise 1 (Easy): Temperature converter

[Back to the exercise](#)

Hint 1

In JavaScript, $\times$ is `*`. Use parentheses for the `(F - 32)` part, so the subtraction happens first.

Hint 2

To round to 1 decimal place and keep a number, multiply by 10, round, then divide by 10: `Math.round(value * 10) / 10`. Do the rounding inside the function, just before you return.

Exercise 2 (Easy): Workout tracker helpers

[Back to the exercise](#)

Hint 1

For the hours, how many whole times does 60 go into the minutes? `Math.floor` from [chapter 05](#) can help. For the leftover minutes, think `%`.

Hint 2

An arrow function can return a template literal, a calculation, or a comparison directly. A comparison like `minutes >= 60` is already `true` or `false`, so you don't need an `if`.

Exercise 3 (Medium): Bug hunt

[Back to the exercise](#)

Hint 1

The first run prints a lonely `50` at the top and then crashes with `TypeError: Cannot read properties of undefined (reading 'toFixed')`. The error points at `formatPrice`, but `formatPrice` is fine! It was handed `undefined`. Follow that value back: where did it come from?

Hint 2

Two of the bugs are about the difference between showing an answer and handing it back. Look again at "console.log vs return" and at the arrow function mistake in the notes.

Hint 3

If you get `3 months: $NaN`, count how many arguments `getTotalCost` expects, and how many it's getting.

Exercise 4 (Medium): The coffee machine

[Back to the exercise](#)

Hint 1

Default parameters go right in the parentheses: `function orderCoffee(drink = "espresso", size = "medium")`.

Hint 2

A `switch` on the drink ([chapter 07](#)) is a nice way to set the base price. Its `default` case is the perfect spot for the early `return` for drinks that aren't on the menu.

Hint 3

Once you have the base price in a `let` variable, adjust it for the size. Then build the message with a template literal, using `toFixed(2)` for the price.

Exercise 5 (Challenge): Password strength checker

[Back to the exercise](#)

Hint 1

`countDigits` is a loop over the characters with a counter, like the Mississippi example in [chapter 08](#). The string `"0123456789"` and `includes` can tell you whether a character is a digit.

Hint 2

How can you tell if a character is a capital letter? A capital letter changes when you make it lowercase, and other characters don't. So compare the character with its `toLowerCase()` version. And as soon as you find one capital, you can `return true` straight away, even from inside the loop.

Hint 3

In `getPasswordStrength`, use guard clauses: check for "weak" first and return early. Then check the "strong" rules. Whatever is left is "medium". For `maskPassword`, remember `slice` and `repeat` from [chapter 06](#).

10 Arrays

Exercise 1 (Easy): Weekly shopping list

[Back to the exercise](#)

Hint 1

The first item is at index `0`. For the last item, `.at()` with a negative number counts from the end.

Hint 2

One method adds to the end and another removes from the start. Check the table in the notes if you can't remember which is which.

Exercise 2 (Easy): Road trip playlist

[Back to the exercise](#)

Hint 1

`splice` gives back an *array* of the removed items, even when it removes only one. How do you get the first item out of an array?

Hint 2

For step 3, you need two methods chained together: one to copy the first three songs, and one to turn them into a string.

Hint 3

For step 5, `indexOf` gives you an index, which starts at 0. What do you add to turn it into a track number?

Exercise 3 (Medium): Test score report

[Back to the exercise](#)

Hint 1

You can do all of this in a single `for...of` loop. Before the loop, set up a variable for each thing you're tracking: a total, the highest, the lowest, a pass counter, and an empty array for retakes.

Hint 2

The average is the total divided by the number of scores. `toFixed(1)` from chapter 05 gives you 1 decimal place.

Hint 3

If you put an array inside a template literal, you get `45,38` without the brackets. To see Node's `[ 45, 38 ]`, pass the array to `console.log` after a comma: `console.log("Label:", myArray)`.

Exercise 4 (Medium): Hashtag maker

[Back to the exercise](#)

Hint 1

Plan it in steps: clean up the phrase, break it into an array of words, fix each word, then glue the words back together.

Hint 2

Try `console.log("learn JAVASCRIPT".split(" "))`. Where do those empty strings `' '` come from? You can skip them inside your loop with `continue` from chapter 08.

Hint 3

To fix one word: take its first letter and uppercase it, then add the rest of the word in lowercase. `slice(1)` gives you "everything after the first letter".

Exercise 5 (Challenge): Cinema seat booking

[Back to the exercise](#)

Hint 1

For `printSeats`, you need each row *and* its position (to find the matching letter in `rowNames`). Which kind of loop gives you the index? And which array method turns `["O", "O", "X"]` into `0 0 X`?

Hint 2

For `countFreeSeats`, use a loop inside a loop: one for the rows, and one for the seats in each row.

Hint 3

In `bookSeat`, turn the customer's words into indexes first. `rowNames.indexOf(rowName)` gives the row index (or `-1` if there's no such row). The seat index is one less than the seat number. Check that both are in range *before* you read `seats[rowIndex][seatIndex]`.

11 Objects

Exercise 1 (Easy): Contact card

[Back to the exercise](#)

Hint 1

Adding a property and changing one look exactly the same: `object.key = value`.

Hint 2

To count the properties, get an array of the keys first. Every array has a `length`.

Exercise 2 (Easy): Cinema tickets

[Back to the exercise](#)

Hint 1

Inside your loop, the ticket type is in a variable. Which notation lets you use a variable as the key?

Hint 2

To check whether a ticket type exists, use one of the checks from "Does this property exist?" in the notes. `Object.hasOwnProperty` is a good choice.

Exercise 3 (Medium): Library catalog

[Back to the exercise](#)

Hint 1

To print double quotes inside a template literal, type them as they are: backticks don't mind double quotes. The ternary operator from chapter 07 is handy for choosing between `available` and `checked out`.

Hint 2

For step 4, `findBook("Emma")` gives you `null`, and `null.author` would crash. Which operator stops safely when the thing on its left is missing? And which one gives a default value when the result is `undefined`?

Exercise 4 (Medium): Lunch poll

[Back to the exercise](#)

Hint 1

Start with an empty object: `const counts = {};`. Then loop over the votes. The food is in a variable, so use bracket notation.

Hint 2

The first time you see a food, `counts[food]` is `undefined`, because that key doesn't exist yet. In that case, set it to `1`. Otherwise, add 1 to what's already there.

Hint 3

Finding the winner is the "finding the biggest" pattern from chapter 10. This time, remember two things as you go: the best food so far and its number of votes.

Exercise 5 (Challenge): Playlist with methods

[Back to the exercise](#)

Hint 1

For `formatTime`: `Math.floor` gives you whole minutes, and `%` from chapter 04 gives you the seconds left over. `padStart(2, "0")` from chapter 06 turns `"7"` into `"07"`, but it's a string method, so turn the number into a string first.

Hint 2

Inside a method, `this.tracks` is the playlist's `tracks` array. Remember to write the methods with the method shorthand, `addTrack(title, artist, seconds) { ... }`, and not as arrow functions.

Hint 3

A method can call another method of the same object through `this`. In `print`, you can use `this.getTotalSeconds()` and pass the result to `formatTime`.

12 Project: Shopping Cart

Milestone 1: The product catalog

[Back to the lesson](#)

Hint 1

From cents to dollars: divide by 100. Then `toFixed(2)` from chapter 05 gives you exactly two decimal places, and it hands you back a string.

Hint 2

`formatMoney` should `return` the text, not print it. That way, every other function can use it inside its own messages.

Hint 3

In `printMenu`, loop over `products` with `for...of`. Each menu line uses the product's `id` and `name`, plus `formatMoney` for the price.

Milestone 2: Add to the cart

[Back to the lesson](#)

Hint 1

Start with a helper function, `findProduct(productId)`, that returns the product with that ID, or `null` if there isn't one. It's very close to the `findProduct` example in [chapter 11](#).

Hint 2

`Number.isInteger(quantity)` from chapter 05 is `false` for `1.5`, and also for anything that isn't a number at all. You still need a second check for numbers below 1.

Hint 3

After each error message, stop the function straight away with `return`. Those checks are the guard clauses from chapter 09.

Hint 4

For rule 3, write a second helper, `findCartItem(productId)`, that searches the cart instead of the catalog. If it finds an item, change that item's `quantity`. If it gives you `null`, push a new object into the cart. Shorthand properties from chapter 11 make that object very short to write.

Milestone 3: The subtotal

[Back to the lesson](#)

Hint 1

This is the "adding everything up" pattern from chapter 10. The cart only stores each product's ID, so inside the loop, use `findProduct` to get the price.

Hint 2

In `printCart`, check for an empty cart first and `return` early. Remember that an empty array is truthy, so check its `length`.

Hint 3

`printCart` can call `getSubtotal()` for its last line. There's no need to add everything up twice.

Milestone 4: Discount codes

[Back to the lesson](#)

Hint 1

`getDiscount` only calculates. It returns a number and prints nothing. An `if...else if` chain or a `switch` from chapter 07 both work well.

Hint 2

10% of an amount is `amount * 0.1`. Wrap it in `Math.round` so the result is a whole number of cents.

Hint 3

To clean up the code, chain `trim()` and `toUpperCase()` from chapter 06. To check whether it's valid, you could keep the valid codes in an array and use `includes` from chapter 10.

Milestone 5: Tax and shipping

[Back to the lesson](#)

Hint 1

`getShipping` asks one question: is the amount at least the free-shipping amount? If yes, shipping costs `0`. If not, it costs the shipping fee.

Hint 2

In `getTotals`, work out the values in order, each one using the ones before it: the subtotal, then the discount (using the stored `discountCode`), then the amount after the discount, and so on.

Hint 3

Once you have six variables with the right names, shorthand properties from chapter 11 let you return them all in one short line.

Milestone 6: The receipt

[Back to the lesson](#)

Hint 1

`padEnd(20)` adds spaces after the text until it's 20 characters long, so it lines up on the left. `padStart(10)` adds spaces in front, so it lines up on the right. `"=".repeat(34)` makes the long lines. All three are from chapter 06.

Hint 2

`padStart` is a string method, but the quantity is a number. Turn it into a string first with `String()`.

Hint 3

All the totals rows have the same shape: a label on the left and an amount on the right. A small helper like `printLine(label, value)` saves you from writing the padding again and again.

Hint 4

Start `printReceipt` by calling `getTotals()` once. Then read everything you need from the object it gives back.

Milestone 7: Remove items and change quantities

[Back to the lesson](#)

Hint 1

`splice` from chapter 10 can remove the item, but it needs to know the item's index. Loop over the cart with a `for` loop and an index, and look for the item whose `productId` matches.

Hint 2

Once you have the index, `cart.splice(index, 1)` removes one item at that spot.

Hint 3

`updateQuantity` can reuse what you've already built: `findCartItem` to find the item, and `removeFromCart` for a quantity of 0.

Exercise 1 (Easy): Prices with commas

[Back to the exercise](#)

Hint 1

In chapter 05, you made a formatter with `new Intl.NumberFormat(...)`. You gave it a locale like `"en-US"`, plus options with `style: "currency"` and a `currency`. Then you called its `format` method.

Hint 2

The formatter works with dollars, not cents. What do you need to do to the cents first?

Exercise 2 (Easy): Loyalty points

[Back to the exercise](#)

Hint 1

To get whole dollars, divide the cents by 100 and round *down*. Which `Math` method always rounds down?

Hint 2

Check your function with the first test: \$54.89 is 54 whole dollars. The total is \$50.00 or more, so the points are doubled: 108.

Exercise 3 (Medium): Stock limits

[Back to the exercise](#)

Hint 1

`findCartItem` tells you whether the product is already in the cart. If it is, its `quantity` is how many are already there. If it isn't, there are 0 already there.

Hint 2

The third test line tries to add 1 backpack when there are already 2 in the cart. That would make 3, which is more than the stock of 2.

Exercise 4 (Medium): Buy 2, get 1 free

[Back to the exercise](#)

Hint 1

For every full group of 3 packs, one pack is free. `Math.floor(quantity / 3)` counts the full groups.

Hint 2

The other products don't have a `buyTwoGetOneFree` property at all. Reading it gives `undefined`, which is falsy, so a plain `if` can tell the two kinds of product apart.

Hint 3

Did you write `price times quantity` in several places (like `getSubtotal`, `printCart` and `printReceipt`)? Then each one has to change. That's why it pays to give each calculation its own function: next time, there's only one place to change.

Exercise 5 (Challenge): A receipt you can keep

[Back to the exercise](#)

Hint 1

Start with an empty array of lines. Everywhere `printReceipt` printed a line, `push` that line into the array instead.

Hint 2

At the end, one array method from chapter 10 glues all the lines into a single string, with `"\n"` between them.

Hint 3

If you have a `printLine(label, value)` helper, turn it into `formatLine(label, value)`, which returns the padded line instead of printing it.

13 Array Methods

Exercise 1 (Easy): Price tags

[Back to the exercise](#)

Hint 1

`toFixed(2)` from chapter 05 turns a number into a string with two decimals. In a template literal, ``${...}` gives you a plain ``` followed by the inserted value.

Hint 2

The `forEach` callback can take a second parameter: the index. Indexes start at 0, but item numbers start at 1.

Exercise 2 (Easy): Exam results

[Back to the exercise](#)

Hint 1

Match each question to a method: "only some items" is `filter`, "the first one" is `find`, "anyone?" is `some`, and "everyone?" is `every`.

Hint 2

For the last line, think about which method gives you an array of the failed scores. How do you count the items in an array?

Exercise 3 (Medium): Order history

[Back to the exercise](#)

Hint 1

The item count works just like the total. Only the property you add up is different.

Hint 2

The average is the total divided by the number of orders. Use `toFixed(2)` to show two decimals.

Hint 3

For the biggest order, the accumulator isn't a number: it's "the biggest order so far". Start it with the first order, `orders[0]`. In each step, return whichever of the two orders (the biggest so far, or the current one) has the larger total. A ternary from chapter 07 fits well here.

Exercise 4 (Medium): Game leaderboard

[Back to the exercise](#)

Hint 1

Which sorting method leaves the original array alone? For "highest first", think about whether you need `a - b` or `b - a`, and remember you're comparing a property, not the whole object.

Hint 2

`slice` from chapter 10 can give you the first three items of the sorted copy. `padEnd` from chapter 06 adds spaces to the end of a string.

Hint 3

For the alphabetical list, first turn the players into an array of names, then sort that. `join` from chapter 10 glues the names together.

Exercise 5 (Challenge): Bookshop stock report

[Back to the exercise](#)

Hint 1

`inStockTitles` is a chain of three steps: keep some books, turn them into titles, sort the titles. Sorting text? `localeCompare` is your friend.

Hint 2

`stockValue` is a `reduce` with a starting value of `0`. Inside the callback, multiply before you add.

Hint 3

For the genre counts, group the books by `book.genre`. Then `Object.keys` from chapter 11 gives you the genre names as an array, which you can loop over with `forEach`. Each group is an array, so it has a `length`.

14 Scope and Hoisting

Exercise 1 (Easy): Scope detective

[Back to the exercise](#)

Hint 1

Remember the one-way windows. `model` lives inside the function's room and `slogan` lives inside the `if` block's cupboard. The lines at the bottom are out in the garden.

Hint 2

For `model`: if you move its declaration to the top level, can `showBike` still see it? For `slogan`: declare it before the `if` with `let`, and give it its value inside.

Exercise 2 (Easy): Leaky loops

[Back to the exercise](#)

Hint 1

`var` only respects functions, not blocks. There's no function here, so all of those `var` variables live at the top level of the file.

Hint 2

A `const` inside a loop body is fine, even though the loop runs many times. Each time round, the body is a fresh block, so it gets a brand-new variable.

Exercise 3 (Medium): Too early at the gym

[Back to the exercise](#)

Hint 1

Look at how each of the four things is created: a function declaration, a `const` holding a number, a `const` holding an arrow function, and a `var` holding a function expression. Check the table at the end of the hoisting section in the notes.

Hint 2

If you move the two `const` lines up but forget the `var`, you'll get a different error: `TypeError: applyDiscount is not a function`. Why does `var` give a *TypeError* instead of a *ReferenceError*?

Exercise 4 (Medium): Library late fees

[Back to the exercise](#)

Hint 1

Strict mode reports one problem per run, so fix one error at a time. Read each message: `ReferenceError: total is not defined` tells you exactly which name has no declaration.

Hint 2

For each variable, ask two questions. Which function uses it? That's where it belongs. Does its value change after it's first set? That decides `let` or `const`.

Hint 3

One of the errors isn't a missing `let` at all. Read that variable's name letter by letter.

Exercise 5 (Challenge): Clean up the cinema

[Back to the exercise](#)

Hint 1

Follow `movie` from one call to the next. When no title matches, the loop never assigns it. So what is `movie` still holding from the call before?

Hint 2

`find` returns `undefined` when nothing matches. Check for that first, and use an early `return` (from chapter 09) before you touch `movie.seatsLeft`.

Hint 3

For the total: create it once at the top level with `let`, then add each booking's returned cost to it. Something like `totalSpent = totalSpent + book(...)` works.

15 Destructuring, Spread and Rest

Exercise 1 (Easy): Race results

[Back to the exercise](#)

Hint 1

For "everyone else", use rest: three dots in front of the last variable name. To print the array the way Node shows it, pass it to `console.log` as a separate value after a comma, not inside a template literal.

Hint 2

To skip an item, leave its spot empty and write only the comma. Skipping three items means three commas before the name you want.

Exercise 2 (Easy): Profile card

[Back to the exercise](#)

Hint 1

Renaming uses a colon: `property: newName`. A default uses `=: name = defaultValue`.

Hint 2

To reach into `address`, write `address:` followed by another `{ }` pattern. Remember that this doesn't create an `address` variable.

Exercise 3 (Medium): Music app settings

[Back to the exercise](#)

Hint 1

With object spread, the object that comes later wins. Which one should come last in step 1?

Hint 2

In step 2, spread `settings` first, then write the properties you want to change after it.

Hint 3

`Math.max` and `Math.min` want separate numbers. How do you turn an array into separate values?

Exercise 4 (Medium): Restaurant bookings

[Back to the exercise](#)

Hint 1

Destructure the options object right in the parameter list, with a default for each property that has one.

Hint 2

`bookTable()` is called with nothing at all. Look at Common mistakes #2 in the notes to stop it from crashing. Then check whether `name` is missing before building the confirmation.

Hint 3

For `addToOrder`, a rest parameter collects the dishes into an array. Its `length` tells you how many there are, and a ternary from chapter 07 can pick "dish" or "dishes".

Exercise 5 (Challenge): Online orders

[Back to the exercise](#)

Hint 1

In `parseOrder`, the first two variables take the id and the name, and a rest variable collects everything after them. Then return an object using the shorthand properties from chapter 11: `{ id, customer, items }`.

Hint 2

A `reduce` callback's second parameter is the current item, and that item is an object. You can destructure it right there: `(sum, { price, qty }) => ...`.

Hint 3

In `addDelivery`, spread the order into a new object, then add `delivery` after it. For the last line, `some` from chapter 13 and the `in` operator from chapter 11 work well together.

16 Values vs. References

Exercise 1 (Easy): Flatmates' shopping

[Back to the exercise](#)

Hint 1

The budgets are numbers and the lists are arrays. Which of those is copied by value, and which is shared by reference?

Hint 2

Find the line where the second list is created. Instead of copying the reference, make a new array with the same items. Chapter 15 gave you a short way to do that.

Exercise 2 (Easy): Same order again?

[Back to the exercise](#)

Hint 1

Two lists with different lengths can never match, so check that first.

Hint 2

`every` from chapter 13 gives its callback the index as a second parameter. Use it to look at the item in the same position in the other list.

Hint 3

Test `shortOrder` vs `lastOrder` carefully. If you skip the length check, `every` only looks at the items of the first list, so a short list can wrongly "match" a longer one.

Exercise 3 (Medium): A sale that breaks the shop

[Back to the exercise](#)

Hint 1

There are two separate problems. `applySale` changes each product object, and `cheapestFirst` changes the order of the array. Fix one at a time, and run the file after each fix.

Hint 2

For `applySale`, a new array isn't enough: the objects inside must be new too. The notes show how to copy objects inside `map`. Remember the parentheses around an object that an arrow function returns.

Hint 3

For `cheapestFirst`, check the "Makes a new copy instead" column of the table in the notes.

Exercise 4 (Medium): Save slots

[Back to the exercise](#)

Hint 1

`join(", ")` from chapter 10 turns the inventory into the text you need.

Hint 2

For step 5, draw it. Which values in `game` are primitives, and which are references to other objects? What does a shallow copy do with each kind?

Exercise 5 (Challenge): Undo button

[Back to the exercise](#)

Hint 1

`addTask` needs a new array with one extra item. Spread from chapter 15 does that in one line, and shorthand properties from chapter 11 make the new task short to write.

Hint 2

`removeTask` is a `filter`. `completeTask` is a `map` where most tasks pass through unchanged, and the matching one is replaced by a *copy* with `done: true`.

Hint 3

Inside `completeTask`, writing `task.done = true` changes the task object that the older snapshots also point at. That's exactly the bug the "Snapshot 1" line catches. Build a copy of the task with object spread instead, and set `done` in the copy.

17 Recursion

Exercise 1 (Easy): Can pyramid

[Back to the exercise](#)

Hint 1

What's the smallest pyramid you can answer without thinking? That's your base case.

Hint 2

A pyramid with 4 rows is the bottom row of 4 cans, plus a smaller pyramid with 3 rows sitting on top of it.

Exercise 2 (Easy): Backwards words

[Back to the exercise](#)

Hint 1

Which words are already their own reverse, without doing anything? Think about very short ones. That's your base case.

Hint 2

`word[0]` is the first letter, and `word.slice(1)` is everything after it (chapter 06). The reverse of "cat" is the reverse of "at", followed by "c".

Exercise 3 (Medium): Comment thread

[Back to the exercise](#)

Hint 1

This works just like `printTree` in the notes, except that your function receives a whole *list* of comments. Loop over the list: print each comment, then hand its `replies` to `printThread`, one level deeper.

Hint 2

Where's the base case? Look at the comments with `replies: []`. What does a `for...of` loop do with an empty array?

Hint 3

For `countComments`, each comment counts as 1, plus however many comments are in its replies.

Exercise 4 (Medium): Boxes inside boxes

[Back to the exercise](#)

Hint 1

All three functions have the same shape as `flatten` in the notes: loop over the items, and treat a box (an array) differently from a gift (a number).

Hint 2

In `totalValue`, a gift adds its own value, and a box adds the total value of everything inside it. What does an empty box add up to?

Hint 3

For `deepestLevel`, keep track of the deepest level you've found *inside* this box, starting at 0. For every item that's a box, ask `deepestLevel` about it, and keep the bigger number with `Math.max`. This box's level is 1 more than the deepest one inside it.

Exercise 5 (Challenge): Shop menu and breadcrumbs

[Back to the exercise](#)

Hint 1

`countProducts` works like `countFiles` in the notes. A category without `children` is the base case: its answer is its own `products` number.

Hint 2

`printMenu` is `printTree` from the notes, plus a call to `countProducts` for each line.

Hint 3

`findPath` is the tricky one. Base case: if this category's name is the target, the path is an array holding just this name. Otherwise, try each child. If a child comes back with a path, put this category's name in front of that path (spread from chapter 15 helps) and return it. If no child finds it, return `null`.

Hint 4

Test `findPath` on its own before the breadcrumbs: `console.log(findPath(menu, "Garden"))` should print `[ 'All products', 'Home', 'Garden' ]`. Then `join(" > ")` from chapter 10 turns a path into breadcrumbs.

18 Error Handling

Exercise 1 (Easy): Name that error

[Back to the exercise](#)

Hint 1

If you put all four pieces in *one* `try`, only the first error gets caught, because `try` stops at the first problem. That's why each piece needs its own.

Hint 2

The error object has `name` and `message` properties. A template literal can put them together with `" : "` in between.

Exercise 2 (Easy): Cinema booking

[Back to the exercise](#)

Hint

The cash machine example in the notes has the same shape: a few guard clauses, then the "happy path" `return` at the bottom. The loop with `try / catch` inside it is there too.

Exercise 3 (Medium): Self-checkout scanner

[Back to the exercise](#)

Hint 1

Looking up a barcode that isn't in the catalog gives `undefined` (remember bracket notation from [chapter 11](#)). That's your signal to throw.

Hint 2

`finally` runs even when the `try` block uses `return`, and even when an error is on its way out of the function.

Exercise 4 (Medium): Bug hunt

[Back to the exercise](#)

Hint 1

The first error you'll see is about `message`. Where was it created, and where is it being used? The "Common mistakes" section of the notes has this exact problem.

Hint 2

Once the first bug is fixed, Ben and Chen seem to pass without any message at all. Where did their errors go?

Hint 3

If Chen's line shows `undefined - undefined`, look at what `checkAge` throws for a non-number. Does a plain string have a `name` or a `message`?

Exercise 5 (Challenge): Grade importer

[Back to the exercise](#)

Hint 1

`split(",")` gives you an array of parts ([chapter 06](#)). Check its `length` before you destructure it ([chapter 15](#)). Don't forget to `trim()` each part, because of the space after the comma.

Hint 2

`Number("A+")` gives `NaN`, and `Number.isNaN` spots it ([chapter 05](#)). For the line numbers, `forEach` gives you each item's index as the second argument of the callback ([chapter 13](#)).

Hint 3

The `try / catch` goes *inside* the loop in `importGrades`, around the call to `parseLine`, so each line gets its own chance. `reduce` can add up the grades, and `toSorted` with a compare function can find the top student.

19 Dates and Times

Exercise 1 (Easy): Wedding invitation

[Back to the exercise](#)

Hint 1

Remember that months count from 0 when you create the date, and add 1 back when you show it.

Hint 2

`getMinutes()` gives `5`, not `05`. Turn it into a string and use `padStart` from [chapter 06](#). For the last line, use `toLocaleDateString` with the `"en-GB"` locale and an options object.

Exercise 2 (Easy): Library late fee

[Back to the exercise](#)

Hint 1

`setDate(date.getDate() + 21)` moves a date 21 days forward, and it handles the jump into December for you.

Hint 2

Subtract the due date from the return date to get milliseconds, then turn that into days. Don't forget `Math.round`.

Exercise 3 (Medium): What's coming up?

[Back to the exercise](#)

Hint 1

Sort with `toSorted` and a compare function that subtracts one date from another. Look back at the guest bookings example in the notes.

Hint 2

If you ever see something like `in 24.958333333333332 days`, you forgot `Math.round`. That's exactly what happens in Sydney, where the clocks change in early October.

Exercise 4 (Medium): Meeting across time zones

[Back to the exercise](#)

Hint 1

The football final example in the notes shows a date in several time zones with the `timeZone` option. Destructuring in the `for...of` loop keeps it tidy.

Hint 2

To check the work hours, you need the hour as a number in *that person's* time zone. `getHours()` won't do, because it uses *your* time zone. Instead, format the date with only `{ timeZone, hour: "numeric" }` and the `"en-GB"` locale (24-hour clock). You'll get a string like `"21"`, which `Number()` can turn into `21`.

Exercise 5 (Challenge): Gym memberships

[Back to the exercise](#)

Hint 1

`"2026-08-30".split("-")` gives `[ '2026', '08', '30' ]`. Turn those strings into numbers with `map((part) => Number(part))` ([chapter 13](#)), destructure them into `year`, `month`, and `day` ([chapter 15](#)), and build the date from parts. Remember the month!

Hint 2

`new Date(2026, 1, 30)` doesn't fail. It rolls over to March 2nd. That rollover is your clue: after building the date, check whether its year, month, and day are still the ones you asked for. If anything changed, the date was impossible, so throw.

Hint 3

Put the `try / catch` inside the loop over `members`, like in the grade importer from [chapter 18](#). Keep the good results in one array (with each member's name, end date, and days left) and the problem messages in another. Sort the good ones by end date at the end.

20 DOM Basics

Exercise 1 (Easy): Cafe menu board

[Back to the exercise](#)

Hint 1

The heading, the special, and the count paragraph all have an `id`. The drinks share a class, so `querySelectorAll` can find them all, and a `NodeList` has a `length`.

Hint 2

`forEach` gives your callback the index as a second argument, just like with arrays. Indexes start at 0, but the list should start at 1.

Exercise 2 (Easy): Weather card

[Back to the exercise](#)

Hint 1

The class names in the CSS are exactly the same words as the conditions. So you can hand `weather.condition` straight to one of the `classList` methods.

Hint 2

The alert paragraph is hidden by its `hidden` class. An empty string is falsy ([chapter 07](#)), so `if (weather.alert)` tells you whether there's an alert to show.

Exercise 3 (Medium): Road trip playlist

[Back to the exercise](#)

Hint 1

For `formatTime`, `Math.floor(totalSeconds / 60)` gives the minutes and `totalSeconds % 60` gives the seconds left over ([chapter 04](#)). `padStart` turns `5` into `"05"`.

Hint 2

`reduce` can add up all the seconds ([chapter 13](#)), and your `formatTime` works for the total too.

Exercise 4 (Medium): Book of the week

[Back to the exercise](#)

Hint 1

Images and links have `src` and `href` properties, and template literals are perfect for building the addresses. `replaceAll` from [chapter 06](#) can swap the spaces for underscores.

Hint 2

To add a `data-isbn` attribute, you can use `setAttribute("data-isbn", ...)`, or set `dataset.isbn`. Right-click the card and choose **Inspect** to check that it's there.

Exercise 5 (Challenge): Library loans dashboard

[Back to the exercise](#)

Hint 1

Look back at `renderMenu` in the notes for the shape of a render function. For the dates, an `Intl.DateTimeFormat` with the `"en-US"` locale and the options `weekday`, `month`, and `day` gives `Sun, Sep 20`.

Hint 2

Work out the days left the same way as in chapter 19: subtract the dates, divide by the milliseconds in a day, and round. A negative number means overdue.

Hint 3

A tiny helper like `daysWord(count)` that returns `"1 day"` or `"4 days"` keeps all three labels simple. Count the overdue books with a variable that goes up inside your loop, or with `filter` before the loop.

21 Events

Exercise 1 (Easy): Cup counter

[Back to the exercise](#)

Hint

Keep the count in a `let` variable at the top of your script. The Like button at the start of the notes has the same shape: change the variable, then update the text.

Exercise 2 (Easy): Bio character counter

[Back to the exercise](#)

Hint 1

Which event fires on every keystroke? Inside the handler, `event.target.value` is the text, and strings have a `length` ([chapter 06](#)).

Hint 2

Use an `if / else`: one branch adds the class and shows the longer message, the other removes the class. Removing a class that isn't there does no harm.

Exercise 3 (Medium): Road trip game

[Back to the exercise](#)

Hint 1

Listen for `keydown` on `document`, and check `event.key`. The road is three pieces joined together: some `_`s, the car, then more `_`s. `repeat` from [chapter 06](#) makes the `_`s.

Hint 2

Only change `position` when the move is allowed, then call `render()`. Call `event.preventDefault()` for the two arrow keys only, so that other keys keep working normally.

Exercise 4 (Medium): Shopping list

[Back to the exercise](#)

Hint 1

The shopping list example in the notes is a great starting point. For the count, `list.querySelectorAll("li").length` tells you how many items are left.

Hint 2

Inside the list's listener, first check `event.target.closest(".remove")`. If that finds a button, remove its item and stop there (`return`). Otherwise, use `closest("li")` to find the item to cross off.

Exercise 5 (Challenge): Cinema seat picker

[Back to the exercise](#)

Hint 1

Write a `render()` function that clears `#seats`, rebuilds every seat button from the data (like `renderMenu` in [chapter 20](#)), and updates the summary. Call it once at the start, and again after every change. Because the listener sits on `#seats` itself, it keeps working even though the buttons are rebuilt.

Hint 2

In the listener, `event.target.closest(".seat")` finds the seat that was clicked (or `null` if the click landed in a gap). Its `dataset.seat` tells you which one. `includes` ([chapter 10](#)) can check for a seat, and `filter` ([chapter 13](#)) can take one out of the array. Disabled buttons don't fire click events at all, so taken seats look after themselves.

Hint 3

`toSorted()` puts `["B2", "A1"]` in seat order without changing `selectedSeats`. For the price, remember `toFixed` from [chapter 05](#).

22 Forms

Exercise 1 (Easy): Library card

[Back to the exercise](#)

Hint 1

Listen for `submit` on the form, not `click` on the button. What's the very first thing your handler should do?

Hint 2

`trim()` first, then compare the result with `""`. To empty the box, you can set its `.value`, or reset the whole form.

Exercise 2 (Easy): Split the bill

[Back to the exercise](#)

Hint 1

Every value is a string, even the tip from the dropdown. Convert all three with `Number()` before doing any math.

Hint 2

The tip is `bill * tipPercent / 100`. Call `toFixed(2)` from [chapter 05](#) only at the very end, when you build the text, because it gives you back a string.

Exercise 3 (Medium): Pizza order

[Back to the exercise](#)

Hint 1

`formData.get("size")` gives you the chosen size. The three toppings all share the name `topping`. Which `FormData` method gives you *all* the ticked ones?

Hint 2

Keep the size prices in an object, like `{ small: 8, medium: 10, large: 12 }`. Then bracket notation from [chapter 11](#) looks up the price for whichever size was chosen.

Hint 3

The toppings come back as an array. Its `length` tells you how many to charge for, and `join(", ")` from [chapter 10](#) turns them into text. What should the text say when the array is empty?

Exercise 4 (Medium): Restaurant feedback

[Back to the exercise](#)

Hint 1

For the counter, listen for the `input` event on the textarea. The number left is 120 minus the length of what's in the box.

Hint 2

Does `form.reset()` fire an `input` event? Look back at the notes. If it doesn't, what do you have to update yourself after resetting?

Exercise 5 (Challenge): Loan calculator

[Back to the exercise](#)

Hint 1

Build it in three steps, and test each one before moving on: first `calculateLoan` with `console.log`, then the submit handler (remove `disabled` from the button while you test it), and the live checks last.

Hint 2

The messages fit nicely in an object whose keys are the field names: `{ amount: "Enter an amount...", years: ..., rate: ... }`. Loop over `Object.entries()` of it. For each name, you can find the field with ``#${name}`` and its message paragraph with ``#${name}-error``.

Hint 3

A field "isn't empty but breaks its rules" when its `value` isn't `""` and its `checkValidity()` is `false`. And remember: the values from `FormData` are strings. Convert them before calling `calculateLoan`.

23 JSON and Local Storage

Exercise 1 (Easy): Parcel labels

[Back to the exercise](#)

Hint

Pretty-printing puts each item of an array on its own line. That's normal. For the sentence, the parsed `items` is a real array, so `length` and `join(", ")` from [chapter 10](#) work on it.

Exercise 2 (Easy): Remember my name

[Back to the exercise](#)

Hint 1

The code that checks for a saved name goes at the top level of `script.js`, not inside a handler. It runs once, every time the page loads. `getItem` gives you `null` when nothing is saved.

Hint 2

Remember `event.preventDefault()` from [chapter 22](#), or the page reloads before you see your message. To forget the name, use `removeItem`.

Exercise 3 (Medium): Glitchy thermostats

[Back to the exercise](#)

Hint 1

Put the `try / catch` inside `parseReading`. In the `catch` block, print the message and `return null`. Then the loop only has to check whether it got `null` back.

Hint 2

Collect the good readings in an array as you go. For the average, add up their `celsius` values (a loop or `reduce` from [chapter 13](#)) and divide by how many there are.

Exercise 4 (Medium): Reader settings

[Back to the exercise](#)

Hint 1

The `change` event bubbles up to the form ([chapter 21](#)), so one listener on the form hears both fields. Read the dropdown with `.value` and the tick box with `.checked` ([chapter 22](#)).

Hint 2

For the text size, remove all three size classes first, then add the chosen one. `classList.remove()` accepts several names at once: `classList.remove("small", "medium", "large")`.

Hint 3

Steps 3 and 4 both "put settings into the form, then apply them". A small function for that saves you from writing it twice.

Exercise 5 (Challenge): Recently viewed

[Back to the exercise](#)

Hint 1

Keep the list in an array, and write one `render()` function that rebuilds the `<ol>` from that array, like rendering a list in chapter 20. Call it once when the page loads, and again after every change.

Hint 2

To move a product to the top, first take it out with `filter` ([chapter 13](#)), then put it at the front with `unshift`, then keep the first three with `slice` ([chapter 10](#)).

Hint 3

A click can land on the `<ul>` itself, in the gaps between the buttons. `event.target.closest("button")` gives you the button that was clicked, or `null` if the click wasn't on one.

24 Project: To-Do List App

Milestone 1: Tasks in an array, drawn on the page

[Back to the lesson](#)

Hint 1

Build each part with `createElement`, give it its class with `classList.add`, and put the parts into the `<li>` with `append`, like in chapter 20. You can copy the `×` symbol from this page.

Hint 2

For the tick box, set `checkbox.type = "checkbox"` and `checkbox.checked = task.completed`. For the id, `li.dataset.id = task.id`. And `setAttribute` sets any attribute, including `aria-label`.

Hint 3

Split the work: a function `createTaskElement(task)` that builds and returns one `<li>`, and `render()`, which empties the list and appends one element per task. Small functions are easier to test and fix.

Milestone 2: Add a task

[Back to the lesson](#)

Hint

Listen for `submit` on the form, not `click` on the button, and Enter works for free. `push` adds the new task to the end of the array. To empty the box, set its `value` to `""`, or call `reset()` on the form.

Milestone 3: Mark a task as done

[Back to the lesson](#)

Hint 1

`tasks.find((task) => task.id === id)` gives you the task object itself ([chapter 13](#)). Changing its `completed` changes the task inside the array, because they're the same object ([chapter 16](#)).

Hint 2

Don't add or remove the `completed` class on the `<li>` yourself. Change the data and call `render()`. The new `<li>` gets the right class automatically. That's the big idea at work.

Milestone 4: Delete a task

[Back to the lesson](#)

Hint

It's fine to have two listeners on `#todo-list`: one for `change` (ticking) and one for `click` (deleting). Clicking a tick box fires a `click` too, but your `todo-delete` check makes that listener ignore it.

Milestone 5: How many tasks are left?

[Back to the lesson](#)

Hint

`tasks.filter((task) => !task.completed)` gives you the tasks that still need doing. Its `length` is the number you want.

Milestone 6: Filters

[Back to the lesson](#)

Hint 1

If a click lands in the gap between the buttons, `event.target` is the box around them, and its `dataset.filter` is `undefined`. Ignore those clicks.

Hint 2

`getVisibleTasks()` has three cases. Two of them are a `filter`, each with a different test. The third returns `tasks` as it is.

Hint 3

`document.querySelectorAll(".filter-button")` gives you the three buttons, ready to loop over with `for...of` ([chapter 20](#)).

Milestone 7: Remember the tasks

[Back to the lesson](#)

Hint

Look for every place that changes `tasks`: the `push`, the line that sets `completed`, and the `filter`. Each one needs a save afterwards. If ticks or deletions come back after a refresh, one of those places is missing its save.

Milestone 8: Clear completed

[Back to the lesson](#)

Hint

`tasks.some((task) => task.completed)` answers the question "is at least one task done?" ([chapter 13](#)).

Exercise 1 (Easy): Dark mode

[Back to the exercise](#)

Hint 1

`document.body` is the `<body>` element. `classList.toggle("dark")` switches the class, and `classList.contains("dark")` tells you whether dark mode is on right now ([chapter 20](#)).

Hint 2

A boolean is all you need to save. Your `load` helper, with `false` as the default, gives new visitors the light theme.

Exercise 2 (Easy): Keyboard shortcuts

[Back to the exercise](#)

Hint 1

Listen for `keydown` on `document`, and check `event.key`: it's `"/"` or `"Escape"` ([chapter 21](#)).

Hint 2

`todoInput.focus()` puts the cursor in the box, and `todoInput.blur()` takes it out again. Call `event.preventDefault()` for the `/` shortcut, so the browser doesn't type the slash for you.

Hint 3

When the user is typing in the box, `event.target` is the box itself. That's how you can tell "a `/` for the shortcut" from "a `/` inside a task".

Exercise 3 (Medium): Search box

[Back to the exercise](#)

Hint 1

Listen for the `input` event on the search box ([chapter 21](#)). Store the search text already trimmed and lowercased, so you only do that once.

Hint 2

In `getVisibleTasks()`, filter in two steps: first by `currentFilter`, then by the search text. `filter` calls can be chained ([chapter 13](#)), and an empty search text matches every task, because every string `includes("")`.

Exercise 4 (Medium): Due dates

[Back to the exercise](#)

Hint 1

`"2026-10-05".split("-").map(Number)` gives you `[2026, 10, 5]`. Destructure it ([chapter 15](#)), and remember that months start at 0 when you build the date.

Hint 2

An empty date box has the value `""`. And an old task's `due` is `undefined`. A simple `if (task.due)` skips both, because `""`, `null` and `undefined` are all falsy ([chapter 07](#)).

Hint 3

"Before today" means before midnight at the start of today. Make a date for right now, then set its hours, minutes, seconds and milliseconds to 0 with `setHours(0, 0, 0, 0)` ([chapter 19](#)). Dates can be compared with `<`.

Exercise 5 (Challenge): Edit a task by double-clicking

[Back to the exercise](#)

Hint 1

Use delegation once more: a `dblclick` listener on `#todo-list` that checks for the `todo-text` class. (`dblclick` is the double-click event.)

Hint 2

Editing is just more state. Keep the id of the task being edited in a variable, like `let editingId = null;`. When it's set, `createTaskElement` builds an `<input class="todo-edit">` instead of the `<span>` for that one task. Change the state, render, and the text box appears. That's the big idea again.

Hint 3

The text box is brand new after every render, so it's fine to add its listeners right where you create it: `keydown` for Enter and Escape, and `blur` for "clicked somewhere else". (`blur` fires when an element loses focus, the opposite of `focus()`.) Call `focus()` on the box after it's on the page.

Hint 4

Watch out for saving twice. Pressing Enter saves and renders, and the render removes the text box while it still has focus. In some browsers, that fires `blur` as well. Make your "finish editing" function do nothing when `editingId` is already `null`.

25 Closures

Exercise 1 (Easy): Ticket machine

[Back to the exercise](#)

Hint

Start from `createCounter` in the notes. What needs to change so the returned value is a string with the prefix in front?

Exercise 2 (Easy): Price tag makers

[Back to the exercise](#)

Hint 1

Look at `makeDiscount` in the notes. Your factory has the same shape: take a setting, return an arrow function that uses it.

Hint 2

`toFixed(2)` from [chapter 05](#) gives you exactly two decimal places, as a string.

Exercise 3 (Medium): Free trial

[Back to the exercise](#)

Hint 1

This is a bigger version of the `once` helper from the notes. Instead of a true/false "has it run?" flag, what should the backpack hold?

Hint 2

To pass along any number of arguments, collect them with `...args` and hand them on with `fn(...args)`.

Exercise 4 (Medium): The food truck bug

[Back to the exercise](#)

Hint 1

After the loop finishes, what is the value of `i`? What is `dishes[3]`?

Hint 2

For the second fix: each call to `makeOrderButton` gets its own `dish` parameter, which means its own backpack. Pass it `dishes[i]` while the loop is still on that round.

Exercise 5 (Challenge): A private playlist

[Back to the exercise](#)

Hint 1

The songs array is the private data. It lives in `createPlaylist`, not on the returned object, just like `balance` in the bank account example.

Hint 2

For the total, `reduce` from [chapter 13](#) can add up the seconds. Then `Math.floor(total / 60)` gives the minutes and `total % 60` gives the seconds left over. `padStart` turns `4` into `04`.

Hint 3

Look at common mistake 4 in the notes. How can `getSongs()` hand out a copy instead of the real array? (For full safety, think about [chapter 16](#): the song objects inside are shared too.)

26 The `this` Keyword

Exercise 1 (Easy): Smart thermostats

[Back to the exercise](#)

Hint

Look at the `ironTemple` and `fitHub` example in the notes. The shorthand property `{ warmer }` puts the `warmer` function into the object.

Exercise 2 (Easy): Phone bills

[Back to the exercise](#)

Hint

Remember: `call` takes the arguments separated by commas, `apply` takes them as an `array`, and `bind` gives you back a function that you call later.

Exercise 3 (Medium): The broken scoreboard button

[Back to the exercise](#)

Hint 1

Inside `pressButtonThreeTimes`, look at how `onPress` is called. Is there anything before the dot?

Hint 2

For the arrow fix: instead of handing over the method itself, hand over a small function that calls the method *with* the dot.

Exercise 4 (Medium): Cinema seat picker

[Back to the exercise](#)

Hint 1

To count the selected seats, you can select all elements that have *both* classes with `document.querySelectorAll(".seat.selected")` and check its `length`.

Hint 2

For step 4, the event object has a property that is always the element the listener is attached to. Look near the end of the notes.

Exercise 5 (Challenge): Pizza order builder

[Back to the exercise](#)

Hint 1

Default parameters from [chapter 09](#) handle the missing quantity. `padEnd` and `padStart` from [chapter 06](#) build each line.

Hint 2

The subtotal is a job for `reduce` from [chapter 13](#). The discount line shows a negative amount: `-5.75`.

Hint 3

If you get `TypeError: this.formatLine is not a function`, check which kind of function your `forEach` callback is. Which kind shares the method's `this`?

27 Classes

Exercise 1 (Easy): Movie listings

[Back to the exercise](#)

Hint

`Math.floor(minutes / 60)` gives the whole hours, and `%` from [chapter 04](#) gives the minutes left over. Inside a method, the minutes are `this.minutes`.

Exercise 2 (Easy): Flooring shop

[Back to the exercise](#)

Hint 1

A getter can use another getter: inside `flooringCost`, you can read `this.area`.

Hint 2

For the setter, `split("x")` from [chapter 06](#) turns `"5x6"` into `[ '5', '6' ]`. Those are strings, so convert them with `Number()`.

Exercise 3 (Medium): Cinema tickets

[Back to the exercise](#)

Hint 1

Inside the constructor, a static property is reached through the class name: `Ticket.sold`, not `this.sold`.

Hint 2

Why doesn't `first.used = false` work? Because `used` only has a getter. The real value lives in `#used`, which only the class can change.

Exercise 4 (Medium): Vehicle rental

[Back to the exercise](#)

Hint 1

In `Van`, `super.rentFor(days)` gives you the normal price. What do you add to it?

Hint 2

`needsLicence` is a one-liner with `instanceof` and `!` (not).

Exercise 5 (Challenge): Community library

[Back to the exercise](#)

Hint 1

Start with the error classes, and test them on their own first. Each constructor builds the message and passes it to `super(...)`.

Hint 2

`find` from [chapter 13](#) returns `undefined` when nothing matches. That's your signal to throw a `BookNotFoundError`. A private helper method like `#findBook(title)` can do this for both `borrow` and `giveBack`.

Hint 3

For `listAvailable`, chain `filter` and `map`: first keep the available books, then turn each book into its title.

28 Prototypes

Exercise 1 (Easy): Default settings

[Back to the exercise](#)

Hint

Reading a property walks up the chain. Setting a property creates an own property on the object you set it on. Look at "Reading walks the chain, writing doesn't" in the notes.

Exercise 2 (Easy): Plant care, the old way

[Back to the exercise](#)

Hint 1

Methods on a prototype are written as `Plant.prototype.describe = function () { ... };`. Use `function`, not an arrow function. (Common mistake 3 in the notes shows why.)

Hint 2

For step 3, remember that the link to the prototype is live, not a copy.

Exercise 3 (Medium): Modernize a recipe app

[Back to the exercise](#)

Hint

Compare the two `Book` versions in the notes ("Constructor functions" and "Classes are syntax sugar"). The function body becomes the `constructor`, and each `Recipe.prototype.something = function` becomes a method in the class body.

Exercise 4 (Medium): Today's specials

[Back to the exercise](#)

Hint 1

For task 1, look at the own-vs-inherited table in the notes. Which tools look only at own properties?

Hint 2

For task 2, the buggy loop is exactly what you need, used on purpose this time. Notice that it visits `burger` only once, even though both menus have one.

Exercise 5 (Challenge): Property detective

[Back to the exercise](#)

Hint 1

Keep a variable for "the object I'm looking at now", starting with `obj`, and a step counter starting at `0`. A `while` loop from [chapter 08](#) can keep going until that variable is `null`, the end of every chain.

Hint 2

At each step, `Object.hasOwn` tells you whether the property lives right here. If not, move one step up with `Object.getPrototypeOf`.

Hint 3

To get a prototype's name, remember the `constructor` property from the notes:

`Dog.prototype.constructor.name` is `"Dog"`. And watch out for `1 step` vs `2 steps`: the ternary operator from [chapter 07](#) can pick the right word.

29 Modules

Exercise 1 (Easy): Travel converter

[Back to the exercise](#)

Hint 1

To round to 1 decimal place and still get a number, multiply by 10, round with `Math.round`, then divide by 10 ([chapter 05](#)). A small private helper function in `converter.js` saves you from repeating that.

Hint 2

If Node complains with `Cannot use import statement outside a module`, or prints a `MODULE_TYPELESS_PACKAGE_JSON` warning, check your `package.json`.

Exercise 2 (Easy): Gym welcome screen

[Back to the exercise](#)

Hint 1

A default import has no curly braces, and you choose its name. Named imports use curly braces, and `as` renames them.

Hint 2

For `dates.js`, `toISOString()` from [chapter 19](#) gives you `2026-10-01T09:00:00.000Z`. You only need the first 10 characters.

Exercise 3 (Medium): Quote of the day page

[Back to the exercise](#)

Hint 1

For `getQuote`, the remainder operator `%` is perfect for "start again from the beginning": `4 % 4` is `0`.

Hint 2

`ui.js` can find its three elements with `document.querySelector` once, at the top of the file, and reuse them every time `showQuote` runs.

Hint 3

In `main.js`, keep a `let` variable for the current index. The click listener changes it and shows the quote again.

Exercise 4 (Medium): One cart, many files

[Back to the exercise](#)

Hint 1

Look at "A module runs only once, so it can share data" in the notes. Your `store.js` works like the one there, with an extra function or two.

Hint 2

For the copy, spread from [chapter 15](#) works. For the total, think `reduce` ([chapter 13](#)) with a starting value of `0`, so an empty cart gives `0`.

Exercise 5 (Challenge): Tidy up the gradebook

[Back to the exercise](#)

Hint 1

Before you move any code, make a list: for each function or class, which other pieces does it use? For example, `Student` uses `average`, and `formatRow` uses `letterGrade`. Each "uses" becomes an `import`.

Hint 2

Converting `grading.js` takes two small changes: put `export` in front of the function, and delete the `module.exports` line.

Hint 3

Paths are relative to the file doing the importing. From `models/Student.js`, the stats file is `"../utils/stats.js"`. From `utils/format.js`, the grading file is right next door: `"./grading.js"`.

30 Timers and Callbacks

Exercise 1 (Easy): Tea timer

[Back to the exercise](#)

Hint

`setTimeout` takes the function to run first, and the delay in milliseconds second. How many milliseconds are in 3 seconds?

Exercise 2 (Easy): Countdown to the sale

[Back to the exercise](#)

Hint 1

Keep the number of seconds in a `let` variable outside the interval, and count it down by one on every tick.

Hint 2

`setInterval` gives you back an id. Store it, and pass it to `clearInterval` at the moment the countdown is over.

Exercise 3 (Medium): Undo send

[Back to the exercise](#)

Hint 1

Whatever `setTimeout` gives back is the timer id. Your function can `return` it, and you can store it in a variable when you call `sendWithUndo`.

Hint 2

The undo also has to happen later, not right away. Which timer function runs something once, after a delay?

Exercise 4 (Medium): Library loans

[Back to the exercise](#)

Hint 1

Do all the checking *inside* the `setTimeout` callback, after the wait. That's when the librarian has actually looked at the shelf.

Hint 2

`find` gives you `undefined` when nothing matches. Check for that first, then check `available`. Don't forget to `return` after calling back with an error.

Hint 3

"Only after the one before has finished" means the next `borrowBook` call goes *inside* the callback of the previous one. Your callback can call `showResult(error, book)` first, and then start the next loan.

Exercise 5 (Challenge): Holiday slideshow

[Back to the exercise](#)

Hint 1

Keep the current position in `let current = 0;`. Write a `showSlide()` function that updates both paragraphs from `current` ([chapter 20](#)), and call it once at the start.

Hint 2

To wrap around to the first slide, the remainder operator from [chapter 04](#) is your friend: `(current + 1) % slides.length`.

Hint 3

Store the interval id in a `let` variable outside your functions. When you pause, clear the interval and set that variable back to `null`. Before starting a new interval, check whether one is already running.

31 Promises

Exercise 1 (Easy): Your parcel is on its way

[Back to the exercise](#)

Hint

Inside `new Promise((resolve, reject) => { ... })`, start a `setTimeout`. When the timer goes off, call `resolve` with the text.

Exercise 2 (Easy): Gym session

[Back to the exercise](#)

Hint 1

Start the chain with `wait(1000)`. In each `.then`, print the step that just finished, and start the next wait.

Hint 2

If a step finishes too early, check that you `return` the `wait(...)` promise. Without `return`, the next step doesn't wait.

Exercise 3 (Medium): Cash machine

[Back to the exercise](#)

Hint 1

It's the `orderFood` pattern from the notes: a `setTimeout` inside `new Promise`, with a check for each problem and a `return` after each `reject`.

Hint 2

Which method runs "either way"? Put it at the end of the chain inside `tryWithdrawal`, and put `return` in front of the whole chain.

Exercise 4 (Medium): Holiday booking

[Back to the exercise](#)

Hint 1

To measure the time, save `Date.now()` in a variable just before `Promise.all`. Inside `.then`, subtract it from a new `Date.now()`, divide by 1000, and round.

Hint 2

Each `allSettled` result has a `status`. When it's `"fulfilled"`, the text is in `result.value`. When it's `"rejected"`, the error is in `result.reason`.

Exercise 5 (Challenge): Hungry and in a hurry

[Back to the exercise](#)

Hint 1

Look at the "Which one do I need?" table in the notes. You need one tool for "the first one to *succeed*" and another for "the first one to *finish*".

Hint 2

`withTimeout` is a race between the real promise and a timer promise that only ever rejects, like the `timeout(ms)` helper in the notes.

Hint 3

`restaurants.map((restaurant) => requestOrder(restaurant))` gives you an array of promises, which is exactly what `Promise.any` takes.

32 Async/Await

Exercise 1 (Easy): From chain to `await`

[Back to the exercise](#)

Hint

Follow the three-step recipe from the notes. Each `return wait(...)` inside a `.then` becomes an `await wait(...);` line of its own.

Exercise 2 (Easy): Bug hunt at the pizza shop

[Back to the exercise](#)

Hint 1

The first error stops the whole file before anything runs. Read the message: which function uses `await` without being allowed to?

Hint 2

Seeing `Promise { <pending> }`? Something is being printed before it has finished. What's missing in front of `getStatus(...)`?

Hint 3

If `All orders checked` comes first, the loop isn't waiting. Which kind of loop waits for `await` on every turn?

Exercise 3 (Medium): Cinema box office

[Back to the exercise](#)

Hint 1

`finally` runs even when `try` or `catch` has a `return` in it. So "Next in line, please." still prints before the function hands back `true` or `false`.

Hint 2

In `buyTickets`, use a `for...of` loop and `await buyTicket(seat)`, so each customer waits their turn. The value you get back tells you whether to add one to your counter.

Exercise 4 (Medium): Morning dashboard

[Back to the exercise](#)

Hint 1

Weather and calendar need the user, so the user has to come first, on its own. The news doesn't need anything, but it's fine to start it together with the other two.

Hint 2

`await Promise.all([...])` with the three calls inside, and array destructuring on the left to give each result a name.

Exercise 5 (Challenge): Online shop order queue

[Back to the exercise](#)

Hint 1

In an `async` function, `throw` rejects the promise it returns. So the caller can catch that error with `try / catch` around its `await`.

Hint 2

Put the `try / catch` *inside* the `for...of` loop, around one order. Then a failure only affects that order, and the loop carries on with the next one.

Hint 3

Keep an array of the orders that shipped. After the loop, its `length` is the number shipped, and `reduce` can add up their prices.

33 Fetch and APIs

Exercise 1 (Easy): Contact card

[Back to the exercise](#)

Hint 1

You need two `await` s: one for `fetch` , and one for `response.json()` .

Hint 2

The city and the company name are inside nested objects. `console.log` the whole user first to see where they live.

Exercise 2 (Easy): Check before you trust

[Back to the exercise](#)

Hint 1

Each to-do has a `title` and a `completed` property (`true` or `false`). A ternary from [chapter 07](#) can turn `completed` into "done" or "not done yet".

Hint 2

`getTodo` throws, and `showTodo` catches. Put a `try / catch` in `showTodo` , around the `await getTodo(id)` .

Exercise 3 (Medium): Post a book review

[Back to the exercise](#)

Hint 1

The second argument to `fetch` is an options object with three properties: `method` , `headers` and `body` .

Hint 2

The body has to be text, so turn the object into JSON with `JSON.stringify` first ([chapter 23](#)).

Hint 3

The mirrored headers are in `data.headers` of the JSON answer. The response's own header comes from `response.headers.get("Content-Type")` .

Exercise 4 (Medium): Weather for any city

[Back to the exercise](#)

Hint 1

Copy the `getJSON` helper from the notes. It saves you from writing the `response.ok` check twice.

Hint 2

When nothing matches, the geocoding answer has no `results` property at all. Check for that before you use `results[0]`.

Hint 3

Put the `try / catch` inside the loop, around one city, so a failure only affects that city ([chapter 32](#) did the same with orders).

Exercise 5 (Challenge): Customer lookup page

[Back to the exercise](#)

Hint 1

Start from the "Loading and error states" example in the notes. Listen for `submit` on the form instead of `click` on the button.

Hint 2

`const [user, todos] = await Promise.all([...])` with two requests inside. For the count, `filter` the to-dos that are `completed` ([chapter 13](#)) and take the `length`.

Hint 3

To tell a 404 apart from other errors, the error has to carry the status. One neat way is a small `HttpError` class that extends `Error` and has a `status` property ([chapter 27](#)), thrown by your `getJSON` helper. A timeout is an error whose `name` is `"TimeoutError"`.

34 Debounce and Throttle

Exercise 1 (Easy): Library search box

[Back to the exercise](#)

Hint 1

Create the debounced function **once**, near the top of your file, and call that same function inside the `input` listener. If you create it inside the listener, you'll see one search per keystroke (Common mistake 1 in the notes).

Hint 2

To find the matches, `filter` the books ([chapter 13](#)) and compare lowercase versions of both the title and the query with `includes`.

Exercise 2 (Easy): Autosave a draft

[Back to the exercise](#)

Hint 1

You only need one `input` listener. Inside it, set the status to `Typing...` (that part isn't debounced), then call your debounced save function.

Hint 2

`localStorage.getItem("draft")` gives you `null` when nothing has been saved yet. Check for that before you restore anything.

Exercise 3 (Medium): Reading progress bar

[Back to the exercise](#)

Hint 1

The width needs a unit: set `bar.style.width` to a string like ``${percent}%``.

Hint 2

For step 4, the bar stops short because the last `scroll` event arrived "too soon" and was ignored. Make a second, debounced version of `updateProgressBar` (150 ms is fine) and call **both** versions from your listener.

Exercise 4 (Medium): Protect the "Pay" button

[Back to the exercise](#)

Hint 1

For Part 1, the options object goes in as the third argument: `debounce(chargeCard, 1000, { leading: true })`.

Hint 2

A disabled button doesn't fire `click` events at all, so after the first click, nothing gets logged. You met the `disabled` property in [chapter 22](#). So why keep `once` as well? Because other code (a keyboard shortcut, a second button) might call `chargeCard` too. `once` is the safety net.

Exercise 5 (Challenge): Autosave with "Save now" and "Discard"

[Back to the exercise](#)

Hint 1

`flush()` needs to know three things the debounced function saw last time: the arguments, the `this`, and whether a call is waiting at all. Keep them in variables next to `timerId`, in the closure.

Hint 2

Write a small inner function that does the real run: it marks "nothing is waiting" and calls `fn` with the saved `this` and arguments. Then both the countdown and `flush()` can use it.

Hint 3

When you call `notebook.autosave.flush()`, the `this` inside `flush` is **not** `notebook`. That's why you must use the `this` you saved during the last normal call to `notebook.autosave(...)`.

35 Map and Set

Exercise 1 (Easy): Newsletter clean-up

[Back to the exercise](#)

Hint 1

`new Set(signups)` does all the duplicate removal in one step. A Set counts its values with `size`, not `length`.

Hint 2

A Set has no indexes, so you can't use `[i]`. Loop over it with `for...of` and keep your own counter with `let`, or spread it into an array first and use `forEach` with its `index`.

Exercise 2 (Easy): Parcel lockers

[Back to the exercise](#)

Hint 1

You need `set`, `get`, `delete`, `has`, and `size`. Use real numbers as keys: `lockers.set(3, "Ravi")`, not `"3"`.

Hint 2

`for (const [locker, name] of lockers)` gives you both parts of each pair at once.

Exercise 3 (Medium): Top songs of the week

[Back to the exercise](#)

Hint 1

The counting loop from the notes works here too: `get` the count so far (with `?? 0` for the first play), then `set` it one higher.

Hint 2

`[...counts]` gives you an array of `[song, count]` pairs. Sort it by the count, biggest first, then keep the first three with `slice`.

Hint 3

For "played only once", `filter` the pairs where the count is `1`, then `map` each pair to just its song. Destructuring works in the callback's parameters: `([song, count]) => ...`.

Exercise 4 (Medium): Movie night

[Back to the exercise](#)

Hint 1

Every line prints a list the same way, so write a small function first, like `listOf(movies)`, that turns a Set into a sorted, comma-separated string. Spread the Set into an array, then `sort` and `join`.

Hint 2

Watch which Set comes first in `difference`: `ana.difference(ben)` and `ben.difference(ana)` answer different questions. For "ready", ask whether the quiz movies are a subset of what that person has seen.

Exercise 5 (Challenge): Library loans

[Back to the exercise](#)

Hint 1

The first time a member borrows, they don't have a Set yet. `this.#loans.get(member) ?? new Set()` gives you their Set, or a fresh empty one. Don't forget to `set` it back into the Map.

Hint 2

Check the rules in `borrow` *before* you change anything. That way, a broken rule never leaves a half-finished loan behind.

Hint 3

In `giveBack`, the member might not be in the Map at all, so `get` could give you `undefined`. Check for that before you call `has` on it. For the report, loop over the Map with `for...of` and add up each Set's `size`.

36 Iterators and Generators

Exercise 1 (Easy): Next customer, please

[Back to the exercise](#)

Hint 1

The iterator comes from the method with the symbol name: `queue[Symbol.iterator]()`. Don't forget the `()` at the end. You're calling that method.

Hint 2

Store each result in a `let` variable. The loop condition is "not done yet", and the last thing inside the loop is asking for the next result.

Exercise 2 (Easy): Rocket countdown

[Back to the exercise](#)

Hint 1

A regular `for` loop that counts down works well inside a generator: `yield` each number. After the loop, one more `yield` gives the final message.

Hint 2

A generator only reports `done: true` once its function has actually *finished*. After `yield "Liftoff!"`, the function is paused at that line, not finished yet.

Exercise 3 (Medium): An iterable bookshelf

[Back to the exercise](#)

Hint 1

A generator method in a class has a star in front of its name: `*[Symbol.iterator]() { ... }`, and `*byAuthor(author) { ... }`. Inside, `this.#books` works like in any other method.

Hint 2

For `[Symbol.iterator]`, `yield*` hands out every book in the array in one line. For `byAuthor`, loop over the books and only `yield` the title when the author matches.

Hint 3

For the question: think about *when* JavaScript calls `[Symbol.iterator]()`, and what it gets back each time.

Exercise 4 (Medium): Bus timetable

[Back to the exercise](#)

Hint 1

`formatTime` needs the hours (`Math.floor(minutes / 60)`) and the leftover minutes (`minutes % 60`), each padded to two digits with `padStart` from [chapter 06](#).

Hint 2

`find` is lazy too: it stops the moment it finds a match. For the other questions, the chain usually ends with `.map(formatTime).toArray()`, and `join(", ")` makes the list.

Hint 3

Call `departures(...)` again for each question. Helpers like `take` and `find` finish off the generator they read from, so a used one can't answer the next question.

Exercise 5 (Challenge): Order history, page by page

[Back to the exercise](#)

Hint 1

Inside `allOrders`, a `while (true)` loop with a `page` counter works well. `await` the page. If it's empty, `return` to finish the generator. Otherwise, `yield*` the orders and move on to the next page.

Hint 2

Part 1 is the counting pattern from chapter 35: `get` the total so far (or `0`), add the order's `total`, and `set` it back. For the dollar sign in a template literal, ``${total}`` gives you `$115`.

Hint 3

In Part 2, `break` is all you need to stop. When you leave a `for await...of` loop early, the generator is finished off, and it never asks for the next page.

37 Regular Expressions

Exercise 1 (Easy): Numbers on a receipt

[Back to the exercise](#)

Hint 1

For whole numbers, you want "one or more digits in a row". Don't forget the `g` flag, or you'll only get the first one.

Hint 2

A price is a dollar sign followed by digits. `$` is a special character in a regex, so it needs a backslash in front to mean a real dollar sign.

Hint 3

The prices come back as strings like `"$5"`. Remove the `$`, turn each one into a number with `Number`, and add them up, for example with `map` and `reduce` from [chapter 13](#).

Exercise 2 (Easy): Gym membership IDs

[Back to the exercise](#)

Hint 1

Start with the parts in order: the letters `GYM`, a dash, and four digits. A normal letter or a dash matches itself.

Hint 2

If `GYM-00420` or `MY GYM-0042` come out as valid, your regex is only checking that a valid ID appears *somewhere*. Which two pieces make the whole text match?

Exercise 3 (Medium): Tidy phone numbers

[Back to the exercise](#)

Hint 1

Steps 1 and 2 are exactly what `cleanPhone` did in the notes. You can start from there.

Hint 2

For step 3, write a pattern with three groups: three digits, three digits, four digits. In the replacement text, `$1`, `$2`, and `$3` stand for the groups.

Exercise 4 (Medium): Appointment reminders

[Back to the exercise](#)

Hint 1

Build it up. First, a pattern for just the dates. Print `schedule.match(yourPattern)` with the `g` flag and check that you see all three dates. Then add `at` and the time, then the name and `on` in front.

Hint 2

A time like `09:30` is two digits, a colon, and two digits. The name is one or more word characters. Once each piece works, wrap it in a named group like `(?<time>...)`.

Hint 3

`matchAll` needs the `g` flag. Inside the loop, destructure `match.groups` to get all five parts at once. Keep a count with `let`, or spread the matches into an array and use its `length`.

Exercise 5 (Challenge): Sign-up checker

[Back to the exercise](#)

Hint 1

Destructuring the parameter ([chapter 15](#)) keeps the function tidy: `function validateSignup({ username, email, password, zip })`. Start with an empty `errors` array and `push` a message for every rule that fails.

Hint 2

For the username, split the rule in two: *one* letter at the start, then 2 to 14 more word characters (that's 3 to 15 in total). Don't forget the anchors.

Hint 3

"Contains at least one uppercase letter" is a search, not a whole-text check, so that regex needs no anchors. The length rule is plain `password.length`.

38 Type Coercion

Exercise 1 (Easy): Predict the output

[Back to the exercise](#)

Check your predictions

```
34
-1
12
75
345
2
true
0
12
true
```

Hint

For each line, ask two questions. Which operator is it (`+` behaves differently from `-`, `*` and the comparisons)? And what types are on each side?

Exercise 2 (Easy): Cinema booking form

[Back to the exercise](#)

Hint 1

Why is `people` wrong, but `cost` still gives a number? Look at which operator each line uses.

Hint 2

Wrap each form value in the conversion function from this chapter, right where it's created. Then every line below it works with real numbers.

Exercise 3 (Medium): Loose, strict, or `Object.is`?

[Back to the exercise](#)

Hint 1

The label is only there for printing. The function compares `a` and `b`, and puts the three results into one template literal.

Hint 2

This is one of the few times you'll write `==` on purpose, just to see what it does. Keep using `===` in your real code.

Exercise 4 (Medium): A safe number reader

[Back to the exercise](#)

Hint 1

`Number("")` is `0`, which is why empty text needs its own check. Do that check after trimming, so text with only spaces counts as empty too.

Hint 2

After converting, how do you find out whether you got `NaN`? Remember that `NaN === NaN` is `false`, so you need the special check from chapter 05.

Hint 3

Watch the `"0"` line. If you check the number with something like `if (!number)`, you'll wrongly turn `0` into `null`, because `0` is falsy.

Exercise 5 (Challenge): Debug the checkout

[Back to the exercise](#)

Hint 1

Look at each wrong line of output, and ask which operator produced it. `x22` comes from `+`. The wrong "most expensive" comes from `>`. And the extra \$2 comes from an `if`.

Hint 2

A small `for...of` loop right after the `order` array can replace each item's `price` and `quantity` with a real number. Don't forget that `extraNotebooks` is a string too.

Hint 3

For the gift wrap, remember that `"0"` is truthy. Compare the value with exactly what "yes" looks like.

Hint 4

Interesting detail: the line `total = total + item.price * item.quantity` was never a bug. Why not? Which operator runs first, and what does it do with strings?

39 Project: Weather App

Milestone 1: Fetch the forecast for a fixed city

[Back to the lesson](#)

Hint 1

Build the URL with `URL` and `searchParams` from [chapter 33](#), one `set` per parameter. The comma-separated lists go in as one string each: `url.searchParams.set("daily", "weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max")`.

Hint 2

`fetch` doesn't reject on a `400` or `500` answer. If `response.ok` is `false`, throw an `Error` yourself and put `response.status` in the message. That way the caller finds out ([chapter 33](#)).

Hint 3

`main.js` is a module, so you can use `await` at the top level, outside any function ([chapter 29](#)).

Milestone 2: Search for a city and show the weather now

[Back to the lesson](#)

Hint 1

Round the temperatures with `Math.round` ([chapter 5](#)) and take the unit from the response instead of typing `"°C"` yourself: `` ${Math.round(current.temperature_2m)}${units.temperature_2m}``, where `units` is `forecast.current_units`. Destructuring with renaming ([chapter 15](#)) makes this tidy: `const { current, current_units: units } = forecast;`

Hint 2

To build the place label, put the parts in an array and join them: `[place.name, place.admin1, place.country]`. Some places have no `admin1`, so remove the empty parts first. `.filter(Boolean)` does exactly that: `Boolean` works as the callback, and it keeps only truthy values ([chapter 7](#)).

Hint 3

Look up the weather with `weatherCodes[current.weather_code]`. If the service ever sends a code that isn't in the list, you'd get `undefined`, so add a fallback with `??`, like `{ description: "Unknown weather", emoji: "?" }`.

Milestone 3: Loading and error states

[Back to the lesson](#)

Hint 1

When there's no connection at all, `fetch` rejects with a `TypeError` (in Chrome and Edge, the message is `Failed to fetch`). When the server answers with an error status, your own `throw` from Milestone 1 kicks in. One `catch` handles both.

Hint 2

`classList.toggle("error", isError)` adds the class when the second argument is `true` and removes it when it's `false`. That's handy when the same paragraph is used for loading messages and errors.

Milestone 4: Suggestions as you type

[Back to the lesson](#)

Hint 1

Keep one `let controller;` at the top level of `main.js`, outside any function. Each new suggestion request does `controller?.abort()` first, then `controller = new AbortController()`. It's the same pattern as "Debounced search without stale results" in chapter 34.

Hint 2

An aborted request lands in your `catch` with `error.name === "AbortError"`. That's not a real problem, so just `return`. For other errors, clear the suggestions quietly: they're a nice extra, not worth a scary red message.

Hint 3

Create the button's click listener inside the loop, with an arrow function: `() => onSelect(place)`. Thanks to closures ([chapter 25](#)), each button remembers its own `place`.

Milestone 5: The 7-day forecast

[Back to the lesson](#)

Hint 1

A small helper keeps the building tidy, for example `makeSpan(className, text)`, which creates a `<span>`, sets its `className` and `textContent`, and returns it. `append` can take several elements at once ([chapter 20](#)).

Hint 2

The temperatures in the list don't need the full unit. `22°` is clear enough, and it works for both °C and °F in the next milestone.

Milestone 6: Switch between °C and °F

[Back to the lesson](#)

Hint 1

Flipping between two values is a job for the ternary operator ([chapter 7](#)): if `unit` is `"celsius"` now, it becomes `"fahrenheit"`, and the other way round.

Hint 2

Want wind in miles per hour for Fahrenheit fans? Add the parameter `wind_speed_unit=mph`. One surprise: the API labels that unit `"mp/h"`, so you may want to show your own label.

Milestone 7: Remember the last city

[Back to the lesson](#)

Hint 1

Use the "load with a default value" pattern from chapter 23: a small function that reads a key, returns the fallback when there's nothing saved, and wraps `JSON.parse` in `try / catch` in case the saved text is broken.

Hint 2

Save the whole place object from the geocoding API. It already has everything you need later: the name, `admin1`, `country`, `latitude`, and `longitude`.

Exercise 1 (Easy): A background that matches the weather

[Back to the exercise](#)

Hint 1

Check the ranges in order, from small codes to big ones, and `return` as soon as one matches. Once you know the code isn't `0` or `1`, `code <= 48` is enough for "cloudy".

Hint 2

`classList.remove` accepts several class names at once: `document.body.classList.remove("weather-clear", "weather-cloudy", ...)`.

Exercise 2 (Easy): An offline message

[Back to the exercise](#)

Hint

Write one small function that sets the paragraph's `hidden` property from `navigator.onLine`. Then call it from both event listeners and once at startup.

Exercise 3 (Medium): Use my location

[Back to the exercise](#)

Hint 1

`getCurrentPosition` wants two functions: one to call with the position, and one to call with an error. Inside `new Promise((resolve, reject) => { ... })`, you already have two functions like that.

Hint 2

The error object has a `code: 1` means the user said no. You can log `error.message` while testing. To test the "no" path, click the icon at the left of the address bar and block location for the page.

Exercise 4 (Medium): Favorite cities

[Back to the exercise](#)

Hint 1

Write two small functions that return a **new** array instead of changing the old one ([chapter 16](#)):

`addFavorite(favorites, place)` and `removeFavorite(favorites, id)`. `some` ([chapter 13](#)) tells you whether a place is already in the list, and `filter` removes one.

Hint 2

After every change: save to `localStorage`, then re-render the whole list. Re-rendering everything is simpler than updating one item, and for a handful of cities it's plenty fast.

Exercise 5 (Challenge): An hourly temperature chart

[Back to the exercise](#)

Hint 1

Find the coldest and warmest hour with `Math.min` and `Math.max`, spreading the array into them ([chapter 15](#)). The distance between them is the "range".

Hint 2

For one temperature, `(temp - min) / (max - min)` is a number from `0` (coldest) to `1` (warmest). Stretch that to the 10 to 100 range, and `map` it over the whole array ([chapter 13](#)).

Hint 3

The time texts all look like `"2026-09-24T12:00"`, so the hour is always at the same spot: `slice(11, 16)` gives `"12:00"` ([chapter 6](#)). No `Date` needed, and no time zone trouble.

40 The Event Loop

Exercise 1 (Easy): Opening the cafe

[Back to the exercise](#)

Hint 1

Sort the lines into three groups first: plain code, microtasks, and tasks. Plain code always goes first, then every microtask, then the tasks.

Hint 2

Inside each group, think about the order they joined their line. For the two timers, the delay matters too: a 50 ms timer can't be ready before a 0 ms one.

Exercise 2 (Easy): Service with a smile

[Back to the exercise](#)

Hint 1

Two of the lines are already in the right order and should come first. Which ones? They can stay as plain code.

Hint 2

"Bring the menu" sits *below* "Take the order" in the file but has to print *before* it. Which of the two waiting lines gets served first?

Hint 3

Two timers with the same delay run in the order they were created. "Serve dessert" is created first but must print last. What else can you change about a timer?

Exercise 3 (Medium): Movie night

[Back to the exercise](#)

Hint 1

`bookSeat` is an `async` function, so it returns a promise. What can you do with a promise to run something only after it has finished?

Hint 2

One clean way is a new `async` function (for example `movieNight`) that uses `await` for each booking, then prints the last message. Then call it, followed by the popcorn line.

Hint 3

Why does the popcorn still print second? It's a 0 ms timer, and the first booking needs a 100 ms timer. Which one is ready first?

Exercise 4 (Medium): Newsletter in batches

[Back to the exercise](#)

Hint 1

Look back at `resizePhotos` in the notes. You need an inner function that sends *one* batch, and variables outside it (a closure) that remember where you are in the list.

Hint 2

`slice(start, start + batchSize)` gives you one batch, and `join(", ")` turns it into one line of text. Remember that `slice` is fine with an end index past the end of the array.

Hint 3

At the end of a batch: if there are emails left, schedule the next batch with `setTimeout(..., 0)`. Otherwise, print the final message. Why does the order message sneak in after batch 1, and not before it?

Exercise 5 (Challenge): Build your own event loop

[Back to the exercise](#)

Hint 1

Functions are values ([chapter 09](#)), so an array can hold them. `shift()` ([chapter 10](#)) removes the first item and gives it back to you. Then you can call it with `()`.

Hint 2

"Run every microtask, even ones added along the way" fits a `while` loop that keeps going as long as `microtasks.length > 0`, taking one callback off the front each time. New arrivals join the back, so the loop reaches them too.

Hint 3

The outer loop keeps going while *either* array still has something in it. Inside it: empty the VIP line first, then run just one task, if there is one.

41 Memory and Garbage Collection

Exercise 1 (Easy): Final whistle

[Back to the exercise](#)

Hint 1

`setInterval` returns an id. Keep it in a variable so you can stop the timer later.

Hint 2

You can call `clearInterval` from *inside* the interval's own callback, as soon as `goals` reaches 3.

Exercise 2 (Easy): Weigh your playlist

[Back to the exercise](#)

Hint 1

To turn bytes into megabytes, divide by 1024, then by 1024 again. `Math.round` ([chapter 05](#)) tidies it into a whole number.

Hint 2

"Letting go" means making sure nothing refers to the array anymore. The `songs` variable is the only string holding it. What can you set it to?

Exercise 3 (Medium): Recently viewed

[Back to the exercise](#)

Hint 1

Keep the products in a variable inside `createRecentlyViewed`, so the two methods share it through a closure ([chapter 25](#)). A Set ([chapter 35](#)) is a good fit: it has no duplicates, and it remembers the order things were added.

Hint 2

To move a product to the newest spot, delete it and add it again. Deleting something that isn't there is harmless.

Hint 3

The first value in a Set is the oldest. `values().next().value` gives it to you, just like `keys()` on the Map in the notes. For `list()`, spread the Set into an array, then flip it without changing anything: `toReversed()` ([chapter 10](#)).

Exercise 4 (Medium): The chat that won't stop listening

[Back to the exercise](#)

Hint 1

Every `openChat` adds a new listener to `document`, and nothing ever removes it. Each old listener also keeps its own `thisChat` alive through its closure.

Hint 2

`openChat` creates the controller, but `closeChat` needs to call `abort()` on it. Where can you keep the controller so both functions can see it?

Hint 3

Pass `{ signal: controller.signal }` as the third argument of `addEventListener`. For the bonus: if a controller already exists, the chat is already open.

Exercise 5 (Challenge): Leak detective

[Back to the exercise](#)

Hint 1

For the history, `shift()` removes the oldest (first) entry of an array. For the cache, look back at `saveForecast` in the notes.

Hint 2

Keep the current timer's id in a variable *outside* `playSong`, starting as `null`. What should `playSong` do with the old id before it starts a new timer?

Hint 3

Look at what each timer's callback uses: `cover`. As long as a timer is alive, its `cover` can't be collected, even after it's been removed from the cache. That's why fixing the cache alone isn't enough.

42 Proxy and Reflect

Exercise 1 (Easy): Recipe card spy

[Back to the exercise](#)

Hint 1

The traps' parameters are `get(target, property, receiver)` and `set(target, property, value, receiver)`. Both go in one handler object.

Hint 2

In the `set` trap, you need the *old* value for the message. If you read it through the proxy, your `get` trap will print an extra "Reading" line. Where can you read it from without going through the proxy?

Exercise 2 (Easy): Hotel welcome screen

[Back to the exercise](#)

Hint 1

This is the "default values" use case from the notes. Get the normal value with `Reflect.get(...)`, and use `??` to fall back on something else.

Hint 2

The fallback is the English message, which lives on the target. For the last question: which trap does `in` use, and did you write one?

Exercise 3 (Medium): Guarding the shop's products

[Back to the exercise](#)

Hint 1

Look back at `createMember` in the notes. Use one guard clause ([chapter 18](#)) per property: `if (property === "price" && ...)`, throw. Only the good values reach the `Reflect.set` at the bottom.

Hint 2

`typeof` catches `"cheap"`. For whole numbers, `Number.isInteger` ([chapter 05](#)) is your friend. For the name, `trim()` ([chapter 06](#)) turns `" "` into `""`.

Exercise 4 (Medium): Undo button

[Back to the exercise](#)

Hint 1

Keep a `history` array inside `createUndoable`. Both the `set` trap and `undo` can see it, thanks to closures ([chapter 25](#)). The most recent change is at the end, and `pop()` takes it off.

Hint 2

Push a small object into the history for each change, like `{ property, oldValue }`. Read the old value in the trap *before* you call `Reflect.set`.

Hint 3

If `undo` puts the old value back *through the proxy*, the `set` trap records it as a brand-new change. Then the next `undo` just flips that same change back again, and you never reach the older ones. Which object can `undo` write to without going through the trap? (Common mistake 2 in the notes was about doing this by accident. Here, you do it on purpose.)

Exercise 5 (Challenge): Smart home dashboard

[Back to the exercise](#)

Hint 1

Keep an array of listener functions inside `createStore`. The `set` trap and `subscribe` can both reach it through a closure ([chapter 25](#)). Calling each listener is a loop over that array.

Hint 2

In the trap: read the old value, and compare it with the new one. If they're the same, return `true` straight away (that's "done", with nothing to do). Otherwise, save with `Reflect.set`, call the listeners, and return what `Reflect.set` gave you.

Hint 3

`unsubscribe` is a closure too: it remembers which `listener` it belongs to. `filter` ([chapter 13](#)) can build a new array without that one function, because `!==` compares functions by reference ([chapter 16](#)). Use `let` for the array if you replace it.

43 Functional Programming

Exercise 1 (Easy): Pure or impure?

[Back to the exercise](#)

Hint 1

For each function, ask the two questions from the notes. Does it always give the same answer for the same input? Does it change (or print, or read) anything outside itself?

Hint 2

Spread makes a new object with the same properties, and you can override one of them: `{ ...account, balance: ... }`. For Part 3, remember how `isShopOpen` became pure in the notes.

Exercise 2 (Easy): A frozen playlist

[Back to the exercise](#)

Hint

The table in the "Immutability" section of the notes has a non-mutating way to do each of these jobs.

Exercise 3 (Medium): A username pipeline

[Back to the exercise](#)

Hint 1

Look at how `makeSlug` is built in the notes. Most of your steps are one-line arrow functions, like `(text) => text.trim()`.

Hint 2

For step 4, a regular expression with a "not in this set" group works well ([chapter 37](#)). Compare it with `removeSymbols` in the notes: this time the set also includes `_`.

Hint 3

`limitLength` works like `applyDiscount` in the notes: it takes the limit, and returns a function that takes the text. `slice(0, maxLength)` does the cutting.

Exercise 4 (Medium): Memoize with several arguments

[Back to the exercise](#)

Hint 1

Collect all the arguments with a rest parameter, `(...args) =>`, and pass them all on with `fn(...args)` ([chapter 15](#)).

Hint 2

The cache needs one key that stands for *all* the arguments together. Joining them into a string, like `"3|express"`, works for simple values like numbers and strings.

Exercise 5 (Challenge): Refactor a report that changes its own data

[Back to the exercise](#)

Hint 1

Find the two side effects in the old code first. Which line changes the array's order, and which line changes the objects inside it? Your version needs a non-mutating replacement for each.

Hint 2

`withDiscount(10)` should return a function that takes one sale and returns a *new* sale object with a lower price. Look at the fix for Common mistake 1 in the notes.

Hint 3

One way to shape `buildCoffeeReport`: `filter` the coffee sales, `map` them with the discount, sort a copy with `toSorted` (biggest line total first), and `reduce` for the total. Build an array of lines, then turn it into one string with `join("\n")`.

44 Design Patterns

Exercise 1 (Easy): Ride fares

[Back to the exercise](#)

Hint 1

The `shippingMethods` example in the notes has exactly this shape. Each strategy is a small arrow function that takes `km` and returns the fare.

Hint 2

Looking up a key that isn't in the object, like `fareStrategies["helicopter"]`, gives `undefined`. That's your signal to throw.

Exercise 2 (Easy): Free trial

[Back to the exercise](#)

Hint 1

Each wrapped function needs its own call counter that survives between calls. That's a closure, like `createCounter` in [chapter 25](#).

Hint 2

Look at `withLogging` in the notes: `...args` to collect the arguments, `fn.name` for the name, and don't forget to return the original's result.

Exercise 3 (Medium): Parcel tracker

[Back to the exercise](#)

Hint 1

The unsubscribe function only needs to call `off` with the same two things `on` received. Return an arrow function, so `this` still means the emitter inside it ([chapter 26](#)).

Hint 2

`once` is a small decorator. Wrap the listener in a new function that first removes *itself*, then calls the original listener. Subscribe the wrapper with `on`.

Hint 3

If the tip shows up on every update, check which function your wrapper passes to `off`. The emitter's list holds the *wrapper*, not the original listener, so removing the original does nothing.

Exercise 4 (Medium): Library search

[Back to the exercise](#)

Hint 1

`map` ([chapter 13](#)) turns each item into a new object. When an arrow function returns an object directly, wrap it in parentheses: `(doc) => ({ ... })`.

Hint 2

The missing author looks different in each reply. Beowulf has no `author_name` property at all, while Mystery Diary has an empty `writers` array. `??` ([chapter 07](#)) can turn a missing array into an empty one, and then one small helper can handle both cases.

Hint 3

For the year, the first four characters of `"1965-08-01"` are the year, and so are the first four of `"2019"`. `slice` ([chapter 06](#)) and `Number` ([chapter 05](#)) do the rest.

Exercise 5 (Challenge): Pizza shop

[Back to the exercise](#)

Hint 1

Start with the files that don't import anything: `emitter.js`, `menu.js`, and `delivery.js`. Test each one with a quick `console.log` before you move on.

Hint 2

`items` is an array of `[name, size]` pairs. Array destructuring in a `map` callback ([chapter 15](#)) turns each pair into a pizza neatly, and `reduce` adds up the prices.

Hint 3

Dev's pizza is fine, but his delivery method isn't. If his order reaches the kitchen, think about the *order* of the steps inside `placeOrder`: do everything that can fail first, and emit last.

45 Clean Code

Exercise 1 (Easy): Name game

[Back to the exercise](#)

Hint 1

Work out what each thing *is* before you rename it. `d` holds one number per day: what are those numbers? `c` goes up by one for each day that... what?

Hint 2

Try VS Code's **Rename Symbol**: click a name and press `F2`. It renames every place that name is used, so you can't miss one.

Exercise 2 (Easy): Gym fees

[Back to the exercise](#)

Hint 1

There are seven numbers in `monthlyFee`. For each one, ask: "What would I call this if I had to explain it to a new member of staff?" `35` might be `BASE_FEE`.

Hint 2

`18` and `65` are both age limits, but they mean different things. Give them different names that say which is which.

Exercise 3 (Medium): Cinema door

[Back to the exercise](#)

Hint 1

Start from the outside of the staircase and work in. The outermost `if` checks for a ticket, so your first guard clause is "if there's *no* ticket, leave". Flip each condition around: `===` becomes `!==`, `>=` becomes `<`.

Hint 2

If the fifth line of your output says "already been used", your checks are in a different order from the original. That ticket is both used *and* for the wrong movie, so the order decides which message wins. Keep the original order.

Exercise 4 (Medium): Report card

[Back to the exercise](#)

Hint 1

Look for the jobs hiding inside `report` : work out an average, turn an average into a letter grade, format one row, find the class average, find the top student. Each one can become a function with a verb or a clear noun as its name.

Hint 2

`reduce` ([chapter 13](#)) makes `average` short. For the letter grade, guard clauses with early `return`s replace the `let grade` and the `else if` chain.

Hint 3

Careful with the class average: it's the average of *all 14 marks*, not the average of the four students' averages. Those give different answers here (78.7 vs 78.3). `flatMap` can put every student's marks into one array for `average`.

Exercise 5 (Challenge): The cafe till

[Back to the exercise](#)

Hint 1

Look closely at the takeaway comment and the code under it. `* 0.8` doesn't take 10% off. When a comment and the code disagree, the code is what actually runs, so keep the behavior and fix the words. A constant with an honest name can replace the comment completely.

Hint 2

The size prices can live in one object, like the strategies in [chapter 44](#): `{ small: 3, medium: 3.5, large: 4 }`. Looking up a missing size gives `undefined`, and `??` can turn that into the same `0` the old code used.

Hint 3

One way to split the jobs: a function that works out one drink's price from the options, one that creates an order object, one that adds up a list of orders, and one that prints them. Build the list of orders as an array at the bottom of the file, and pass it to the printing function.

46 Testing

Exercise 1 (Easy): Weather station

[Back to the exercise](#)

Hint 1

Start from the `prices.test.js` example in the notes: one `import` line for `test`, one for `assert`, and one for your own functions. You can import both functions in one line: `import { celsiusToFahrenheit, isFreezing } from "../weather.js";`

Hint 2

`isFreezing` returns `true` or `false`, so compare its result with `assert.equal(..., true)` or `assert.equal(..., false)`.

Exercise 2 (Easy): The shipping bug

[Back to the exercise](#)

Hint 1

Both failures are exactly on a boundary: \$20 and \$50. The table says "\$50 or more". Which comparison operator means "or more"?

Hint 2

Notice that the normal cases (\$10, \$80) passed even with the bug. Edge cases found it. That's why the notes say to test exactly on the boundaries.

Exercise 3 (Medium): Username rules

[Back to the exercise](#)

Hint 1

For tests 2 and 3, make up names that are *exactly* 3 and 15 characters long, and check what `cleanUsername` returns. `"abcdefghijklmno"` is 15 characters, if you don't want to count.

Hint 2

Every error test needs the arrow: `assert.throws(() => cleanUsername(42), TypeError);` Without `() =>`, the error escapes before `assert.throws` can catch it.

Exercise 4 (Medium): Coffee order with a fake card machine

[Back to the exercise](#)

Hint 1

`placeOrder` calls `payments.charge(...)`, so pass it an object that has a `charge` property: `placeOrder([coffee, muffin], { charge })`, where `charge` is your mock. To make the mock behave like the real service, give it an `async` function: `mock.fn(async () => ({ id: "R-1001" })).`

Hint 2

For test 2, the mock's function can `throw` inside an `async` function, which makes the promise reject. All three tests are `async`, and every `assert.rejects` needs an `await` in front of it.

Hint 3

`placeOrder` is `async`, so even the "empty order" error comes out as a rejected promise, not a thrown error. That's why test 3 uses `assert.rejects`, not `assert.throws`.

Exercise 5 (Challenge): Test-drive a playlist

[Back to the exercise](#)

Hint 1

For the very first red step, `playlist.js` can be just `export class Playlist {}`. The test fails because `songCount` is `undefined`, which is exactly the red you want. Then add the smallest getter that makes it green.

Hint 2

Each test should create its own `new Playlist(...)`, so no test depends on another (Common mistake 5 in the notes). For the total length, the `formatTrackTime` function from the notes does the formatting you need.

Hint 3

For `next()`, keep a private position counter that goes up by one each time. The remainder operator `%` from [chapter 04](#) turns "position 2 in a list of 2 songs" back into position 0, which is the wrap-around.

47 Debugging

Exercise 1 (Easy): The average that isn't a number

[Back to the exercise](#)

Hint 1

Add `console.log(i, scores[i], total);` inside the loop and watch what happens on the last turn.

Hint 2

This is one of the "common bug families" from the notes. What's the last valid index of an array with 3 items?

Exercise 2 (Easy): Read the stack trace

[Back to the exercise](#)

Hint 1

The error says "reading 'email'", so look at what's just before `.email`. Log that variable on the line before.

Hint 2

`"sam" === "Sam"` is `false`. Chapter 06 has a string method that makes both sides the same case.

Exercise 3 (Medium): The price that changed by itself

[Back to the exercise](#)

Hint 1

Add `console.log(result === items);` inside the function. What does it tell you?

Hint 2

This is the "accidental mutation" bug family. Chapter 16 shows how to make copies, and why a copy of the array alone isn't enough when it holds objects.

Exercise 4 (Medium): Use the debugger

[Back to the exercise](#)

Hint 1

When the code pauses, what kind of value is `score`? Is it a number?

Hint 2

This is one of the mistakes from chapter 32. An `async` function always returns a promise. What gets the value out of a promise?

Exercise 5 (Challenge): Four bugs at the gym

[Back to the exercise](#)

Hint 1

Use `console.table(members)` right after `sortByVisits` runs. Is the original list still in its original order?

Hint 2

Two bugs are in `sortByVisits`: the direction of the sort, and the fact that `sort` changes the array in place (chapter 13 has a method that doesn't).

Hint 3

The other two are in `rewardNames`. Check where the loop starts, and read the comment at the top again: "10 or more".

48 Performance

Exercise 1 (Easy): A reusable stopwatch

[Back to the exercise](#)

Hint 1

Inside `timeIt`, read `performance.now()`, call `fn()` and keep its result in a variable, then read `performance.now()` again. Print, then `return` the result you kept.

Hint 2

`timeIt` needs a function it can call itself, not the answer. If you write `timeIt("...", sumUpTo(1_000_000))`, the sum is worked out *before* `timeIt` even starts. Wrap the call in a small arrow function instead, so `timeIt` gets something it can call later.

Exercise 2 (Easy): Double the data

[Back to the exercise](#)

Hint 1

`Array.from` from [chapter 13](#) can build an array of any length, like `Array.from({ length: 1000 }, ...)`. Its second argument is a function that works out each item. That function gets the index as its *second* parameter.

Hint 2

Functions are values ([chapter 09](#)), so you can put all four in an array and loop over them. Every function also has a `name` property: `openLocker.name` is `"openLocker"`. That's handy for the first column.

Exercise 3 (Medium): Find the book, fast

[Back to the exercise](#)

Hint 1

Start with `low` at the first index and `high` at the last index. Keep looping `while (low <= high)`. When `low` goes past `high`, there's nowhere left to look, so the book isn't there.

Hint 2

If the middle id is *smaller* than the one you want, the book must be to the right of the middle. So the new `low` is one place after the middle. If it's *bigger*, move `high` to one place before the middle. Moving to exactly the middle (instead of one past it) can make your loop run forever.

Exercise 4 (Medium): Speed up the sales report

[Back to the exercise](#)

Hint 1

Hunt for the hidden loops first. There are four places where a loop is hiding inside another loop. Look for `find`, `includes`, and `filter`.

Hint 2

Which tool answers "give me the product with this id" in one step? Which one answers "is this id in here?" in one step? Build them once, before the main loop.

Hint 3

The list of ordered ids is only used to answer "was this product ever ordered?". A Set can collect the ids as you go through the orders, and it ignores duplicates for you.

Exercise 5 (Challenge): A fast product search

[Back to the exercise](#)

Hint 1

Use the cleaned-up query as the cache key. In `searchProducts`, check the cache first, and return straight away if the answer is there. Only search, log, and save to the cache when it isn't.

Hint 2

`slice(0, 50)` gives you the first 50 matches ([chapter 10](#)). `resultsList.replaceChildren(fragment)` removes all the old items and adds everything in the fragment, in one step.

Hint 3

The input listener passes the box's current text to the *debounced* version of `showResults`. Call the plain `showResults("")` once at the bottom of your script for the first render.

49 Node.js Basics

Exercise 1 (Easy): Distance converter

[Back to the exercise](#)

Hint 1

`process.argv.slice(2)` gives you just your own words. Destructure them into two variables. If the user typed nothing, both are `undefined`.

Hint 2

`Number("five")` gives `NaN`, and `Number.isNaN` spots it ([chapter 05](#)). After `process.exit(1)`, nothing else in your file runs, so each guard clause can end with it.

Exercise 2 (Easy): Word counter

[Back to the exercise](#)

Hint 1

Split the text into lines at `"\n"`, `trim()` each line, and skip the empty ones. The trim matters on Windows: files saved there often end each line with `"\r\n"`, so without it, every line would end with an invisible `\r`.

Hint 2

To clean a word, make it lower case, then remove everything that isn't a letter using `replace` with a regular expression like `/[^a-z]/g` ([chapter 37](#)).

Hint 3

Counting words is a job for a Map, like the word-frequency example in [chapter 35](#). For the missing file, catch the error from `readFile` and check its `code`.

Exercise 3 (Medium): Monthly sales report

[Back to the exercise](#)

Hint 1

`path.extname(name)` tells you whether a name ends in `.txt`, so you can `filter` the list from `readdir`. Sort the names too, since `readdir` doesn't promise any order. Because the files are named like `2026-01`, alphabetical order is also date order, one more reason to write dates the ISO way ([chapter 19](#)).

Hint 2

`path.basename(name, ".txt")` gives you the name without its extension, like `2026-01`. For each line, `split(",")` gives you the title and the price, and the price is still a string.

Hint 3

Collect the report lines in an array. Then you can print them with `join("\n")`, and write the very same text to `summary.txt`.

Exercise 4 (Medium): Cinema ticket booth

[Back to the exercise](#)

Hint 1

Write one helper, `askNumber(question, min, max)`, that keeps asking until it gets a good answer, then returns the number. A `while` loop with a `return` inside it works well ([chapter 08](#)). Use it for both questions.

Hint 2

An empty answer is sneaky: `Number("")` is `0`, not `NaN` ([chapter 38](#)), so an empty age would sneak through as a baby's ticket. Check for an empty answer before you convert it. `Number.isInteger` rules out answers like `2.5`.

Hint 3

Ask all the questions first and call `rl.close()`, then print the bill. A small function that turns an age into `{ type: "child", price: 7 }` keeps the loop short.

Exercise 5 (Challenge): A recipes API

[Back to the exercise](#)

Hint 1

Read and parse the file once, at the top of the server file, before `http.createServer`. Top-level `await` makes that easy. Then every request uses the same `recipes` array.

Hint 2

`new URL(request.url, "http://localhost:3000")` splits the address for you ([chapter 33](#)). `url.pathname` is `"/recipes/3"`, without the query, and `url.searchParams.get("vegetarian")` is the string `"true"` (or `null` when it's missing).

Hint 3

`"/recipes/3".split("/")` gives `[ '', 'recipes', '3' ]`. Filter out the empty string, and you can check the parts one by one: the first must be `recipes`, and a second part, if there is one, is the id. It's still a string, so convert it before comparing it with the ids in your data.

Hint 4

For the filters, start with the whole list of recipes, then narrow it down one filter at a time with `filter` ([chapter 13](#)). A request that fails a check can get its 404 answer and `return` straight away, like a guard clause.

50 Tooling

Exercise 1 (Easy): Your first package.json

[Back to the exercise](#)

Hint

The notes' "Watch out" box says which value `"type"` needs. For the last question, look at the semantic versioning table: MAJOR.MINOR.PATCH.

Exercise 2 (Easy): Install and use dayjs

[Back to the exercise](#)

Hint 1

Install with `npm install dayjs`, then `import dayjs from "dayjs";` at the top of `index.js`.

Hint 2

dayjs objects have a `.diff()` method: `later.diff(earlier, "day")` gives the number of days between them.

Exercise 3 (Medium): Scripts for everything

[Back to the exercise](#)

Hint 1

Node has a built-in flag that reruns a file on save. It's in the notes, under "Tools already built into Node".

Hint 2

`npm install --save-dev prettier` puts it under `devDependencies`. The format command is `prettier --write .` (the dot means "this whole folder").

Exercise 4 (Medium): Keep a secret out of your code

[Back to the exercise](#)

Hint 1

In `.env`, write one line: `API_KEY=pretend-key-123` (no quotes, no spaces around `=`).

Hint 2

Node only reads `.env` when you pass `--env-file=.env` before the file name. Put that in your `start` script.

Exercise 5 (Challenge): Rebuild the birthday app with Vite

[Back to the exercise](#)

Hint 1

Vite's starter has a `main.js` and an `index.html` with a `<div id="app">`. You can replace everything in `main.js` with your own code.

Hint 2

`dayjs()` with no argument means "right now".

51 Security Basics

Exercise 1 (Easy): Spot the vulnerability

[Back to the exercise](#)

Hint 1

Each snippet matches one section of the notes. Match them up first, then reread that section.

Hint 2

For each one, ask: "If a stranger controlled this data (or could read this file), what could they do?"

Exercise 2 (Easy): Build `escapeHTML`

[Back to the exercise](#)

Hint 1

Which string method from chapter 06 replaces every match, not just the first? You can chain five of them, one per character.

Hint 2

Seeing `&lt;` in your output? The `&` inside `<` got escaped a second time. One character has to be replaced first, before all the others. Which one?

Hint 3

`42` is a number, and numbers don't have string methods. Turn the value into a string first (chapter 05).

Exercise 3 (Medium): Fix the comment box

[Back to the exercise](#)

Hint 1

Build the `<li>` and the `<strong>` with `createElement` ([chapter 20](#)), and fill them with `textContent`.

Hint 2

`append()` accepts plain strings as well as elements, and it always adds strings as text, never as HTML.

Exercise 4 (Medium): Find the leaked secrets

[Back to the exercise](#)

Hint 1

For Part 1, look for two kinds of problems: secrets in a file that every visitor downloads, and secrets that end up in git or in logs. Also think about `isAdmin`: can a password check that runs in the browser ever be safe?

Hint 2

When the variable isn't set, `process.env.DATABASE_PASSWORD` is `undefined` ([chapter 49](#)).

Hint 3

`slice` and `repeat` from chapter 06 can build the preview. The number of stars depends on the password's `length`.

Exercise 5 (Challenge): A safe profile card

[Back to the exercise](#)

Hint 1

For the username, use a character set with a `{3,20}` quantifier, between `^` and `$`. Try it without `^` and `$`: `"<script>"` passes! Why?

Hint 2

For the website, reuse the `isSafeLink` idea from the notes: `new URL()` inside `try / catch`, then check the `protocol`.

Hint 3

Cut the bio *before* you escape it. The 60-character limit is about what the user typed. After escaping, one `&` has become five characters (`&`), which throws off the count, and a cut could even land in the middle of an entity.

52 Final Project

Exercise 1 (Easy): Expense Tracker

[Back to the exercise](#)

Hint

Store amounts in cents (whole numbers) like the shopping cart in chapter 12, and only turn them into dollars for display.

Exercise 2 (Easy): Pomodoro Timer

[Back to the exercise](#)

Hint

Don't trust `setInterval` to tick exactly once per second. Save the end time with `Date.now()` and work out the time left on each tick (chapter 30 explains why).

Exercise 3 (Medium): Book Search

[Back to the exercise](#)

Hint 1

Log one response with `console.log` first, and look at its shape before writing any page code. The books are in `data.docs`.

Hint 2

Your chapter 39 weather app already has the search-with-suggestions pattern. Reuse its structure.

Exercise 4 (Medium): Habit Tracker

[Back to the exercise](#)

Hint

Store the dates each habit was done as ISO strings, like `"2026-09-24"`. Then write `getStreak(doneDates, today)` as a pure function and test it with Node before connecting it to the page.

Exercise 5 (Challenge): Kanban Board

[Back to the exercise](#)

Hint 1

One array of cards is enough: give each card a `column` property, like `{ id: 1, text: "Buy paint", column: "todo" }`. Moving a card only changes that property.

Hint 2

Use one click listener on the whole board, and read `event.target.closest("[data-id]")` to find the card (chapter 21).